
林月的 χ^2 PC 算法模板

χ^2 PC Algorithm Template

v1.0.0 2026.8.1

林月

目录

林月 · v1.0.0 · 2026.8.1 · 页码与纸面物理页一致

基础算法	5
README	5
基础算法与STL速查	6
01A - 头文件 (完整版)	8
01B - 头文件 (极简版)	9
02 - 常用函数重载	9
03 - 常用常量定义	11
04 - fastio	11
04A - fastio (Linux, mmap极致)	12
05 - i128函数重载	16
06 - 手写随机哈希	17
07 - 随机数	18
08 - 对拍	18
09 - 最长上升子序列 (LIS)	18
10 - 单调栈	19
11 - 坐标压缩	20
12 - 二分查找 (边界)	21
13 - 前缀和与差分	21
14 - 逆序对 (归并排序)	22
15 - 随机数据生成 (树与图)	23
16 - 单调队列	24
linyue	24
树上问题：静态树、分治与离线	26
README	26
树上问题知识与结论	27
01 - 树的直径	27
01A - 树论大封装 (直径+中心+重心)	28
02 - 点分治 (重心树骨架)	30
02A - 点分治 (距离询问)	31
02B - 点分树 (动态点集最近距离)	32
03A - 最近公共祖先 (Tree, 树链剖分)	34
03B - 最近公共祖先 (Tree, 倍增)	35
03C - 最近公共祖先 (Tree, 倍增, 有权图)	36
03D - 最近公共祖先 (Tree, Euler, RMQ)	38
03E - 最近公共祖先 (Tarjan, 离线)	39
03F - 长链剖分 (Tree, O(1) 跳祖先)	40
04 - 树上启发式合并 (通用框架)	41
04A - 树上启发式合并 (众数颜色和)	42
05 - 树上路径交	43
06 - 树链剖分 (路径与子树操作)	44
07 - 树上差分	45
08 - 换根 DP (全树 DP)	46
09 - 虚树	48
树上问题：重构树、动态树与编码	49
10 - Kruskal 重构树	49
11 - Link-Cut Tree (动态树)	50
12 - 树同构 (AHU)	52
13 - Prüfer序列	53
数据结构：并查集、区间结构与 RMQ	55
README	55
数据结构知识与结论	57
01A - 并查集 (DSU)	58
01B - 可撤销并查集 (DSU)	59
01C - 可删除并查集 (DSU)	60
01D - 权值并查集 (DSU, with Int)	61
02A - 线段树 (SegTree, with Int)	62
02B - 最值线段树 (SegTree, with Int)	63
02C - 可持久化线段树 (SegTree)	65
02D - 可合并信息线段树 (MergeSegTree)	67
02E - 计数求和主席树 (CountSumSegTree)	68
02F - 动态开点线段树	69
02G - Segment Tree Beats	71
02H - Li Chao Tree	74
02I - 仿射懒标记线段树 (自动取模)	75
02J - 可合并信息懒标记线段树 (LazyMergeSegTree)	76
03 - 树状数组 (BIT, with Int)	78
03A - 区间加区间和树状数组	79
03B - 二维树状数组	79

03C - 二维树状数组 (区间加、矩形和)	80
04A - ST表 (ST, with Int)	81
04B - 可结合区间查询 (Disjoint Sparse Table)	82
04C - 线性RMQ (状压)	82
05 - Bitset	83
数据结构：顺序统计、Trie、堆、平衡树与哈希	88
06 - 小波矩阵 (WaveletMatrix)	88
06A - 区间第k大 (WaveletMatrix)	90
07 - 线性基	92
08A - 字典树	94
08B - 二进制字典树 (XOR Trie)	95
08C - PBDS前缀字典树 (GNU环境)	96
09A - 可并堆 (LeftistHeap, 可移植)	97
09B - PBDS可并堆 (GNU环境)	98
10 - 可合并信息FHQTreap (MergeFHQTreap)	99
10A - FHQTreap (有序多重集)	100
10B - 隐式 FHQTreap (序列)	102
10BA - 隐式 FHQTreap (自定义聚合与懒标记)	104
10C - PBDS有序多重集 (GNU环境)	107
10D - vector有序多重集 (小数据环境)	108
10E - PBDS有序集合与映射 (GNU环境)	108
10EA - PBDS平衡树 (GNU环境, 基础)	109
11A - KDTree (二维静态查询)	110
11B - KDTree (二维动态插入与偏序计数)	111
12 - 珂朵莉树 (ODT, 随机区间赋值环境)	112
13A - PBDS开放寻址哈希表 (gp_hash_table, GNU环境)	113
13B - PBDS拉链哈希表 (cc_hash_table, GNU环境)	114
14 - 分块 (区间加、区间和)	115
图论：存图、路径、生成树与连通性	117
README	117
图论知识与结论	119
00A - 图遍历 (BFS、DFS、二分图判定)	120
00B - 链式前向星 (大图环境)	121
01A - 单源最短路 (Dijkstra) (无负权)	122
01B - 单源最短路 (Bellman-ford, 负权图)	122
01C - 单源最短路 (SPFA, 负权图)	123
01D - 多源最短路 (APSP)	124
01E - 0-1 BFS	125
01F - 差分约束	125
01G - Johnson全源最短路 (稀疏负权图)	126
01H - 单源最短路 (Dijkstra, 稠密图)	127
01I - DAG路径 (最短、最长)	127
01J - 最短和次短路 (Dijkstra, 计数)	128
02A - 最小生成树 (Kruskal)	129
02B - 最小生成树 (Prim)	130
02C - 有向最小生成树 (Chu-Liu-Edmonds)	131
02D - 最小生成树 (Prim, 稠密图)	132
03A - 拓扑排序 (Kahn)	133
03B - 强连通分量 (Tarjan)	133
03C - 2-SAT	134
04A - 边双连通分量 (桥)	135
04B - 点双连通分量 (割点、圆方树)	136
05A - 欧拉路 (有向、无向)	137
图论：匹配、全局最小割与环	140
06A - 二分图最大匹配 (Hopcroft-Karp)	140
06B - 一般图最大匹配 (带花树)	141
06C - 二分图最优匹配 (Hungarian, KM)	142
06D - 一般图最大权匹配 (带权带花树)	144
06E - 二分图最大匹配 (Kuhn, 小图)	147
07 - 全局最小割 (Stoer-Wagner)	148
08 - 最大团 (位集分支限界)	148
09 - 平面嵌入面与对偶图	149
10 - 最小环 (Floyd)	150
10A - 最小环 (长度与计数)	151
11 - 简单环计数 (状态压缩)	152
12 - 环检测与恢复 (有向、无向)	153
13 - 支配树 (Lengauer-Tarjan)	153
14 - Steiner树 (终端点较少)	154

网络流	156
README	156
网络流建模与结论	156
01A - 最大流 (Dinic)	157
01B - 最大流 (HLPP)	159
01C - 最大流 (HLPP, 极致优化)	161
02 - 最小割	164
02A - 最大权闭合图 (最小割建模)	164
03 - 最小割树	165
04 - 最小费用最大流 (MinCostFlow)	166
05 - 上下界网络流	168
数学：数论、组合、线性代数与几何	170
README	170
数学知识与结论	174
01 - 素数 (线性筛)	183
01A - 质因数分解 (最小质因子)	184
01B - 欧拉函数 (单点试除)	184
01C - 欧拉函数 (整表筛)	184
01D - 莫比乌斯函数 (Mertens、杜教筛)	185
01DA - 莫比乌斯反演 (约数和变换)	185
01DB - 加权莫比乌斯函数 (d^2)	186
01DC - 狄利克雷卷积与逆	187
01EA - 检查是否是素数的幂 (试除)	187
01EB - 检查是否是素数的幂 (最小质因子)	188
01F - Miller-Rabin 素性检验	188
01FA - Pollard-Rho 质因数分解	189
01G - 素性检验 (试除)	190
01H - 约数函数 (枚举、个数、和)	190
01I - 质数计数 (Lehmer)	191
01J - 质数和 (Min_25筛)	192
02 - 快速幂 (ksm)	193
02A - 自动取模 (MInt)	193
02AA - Barrett约简 (u32)	195
02B - 裴蜀定理	196
02BA - 扩展欧几里得 (exgcd)	196
02BAA - 中国剩余定理	196
02BAAB - 扩展 Lucas (组合数模合数)	197
02C - BSGS	198
02D - 逆元线性预处理	199
02E - 模平方根 (Tonelli-Shanks)	199
02EA - 模平方根 (Cipolla)	200
02F - 原根	200
02G - 整除分块求和 (floor_sum)	201
02GA - 加权整除和 (万能欧几里得)	202
02H - 大指数模幂 (十进制)	202
02I - Pohlig-Hellman (离散对数, 光滑群阶)	203
02J - 值域GCD预处理	204
03 - 组合数学 (Comb, with Z)	205
03A - Lucas 定理	206
03B - 康托展开 (暴力, 小 n)	206
03C - 康托展开 (树状数组, 大 n)	207
03D - 卡特兰数	208
03E - 斯特林数 (第一、二类)	208
03EA - 斯特林数 (NTT快速求法)	209
03F - 错排数与贝尔数	210
03FA - 贝尔数 (NTT快速求法)	210
03G - 拉格朗日插值 (任意横坐标)	210
03H - 拉格朗日插值 (连续横坐标)	211
03I - 牛顿插值 (差分级数)	211
03J - 整数分拆 (五边形数)	212
03JA - 整数分拆 (NTT快速求法)	212
03K - Berlekamp-Massey (最短线性递推)	212
03KA - 线性递推第 n 项 (Kitamasa)	213
03L - Burnside 引理 (群作用计数)	213
03M - 快速阶乘 (NTT)	213
04A - 容斥 (bitmask, 小 m)	214
04B - 容斥 (DFS 剪枝)	215
05 - 矩阵 (Matrix, with Int)	215
05A - Min-Plus矩阵 (MinPlusMatrix, with Int)	217
05B - 高斯消元 (一般线性方程组)	218
05C - 对称 LDLT 分解 (线性方程组)	218
05D - 高斯消元 (模质数)	219
05E - 行列式 (模质数)	219
05F - 线性规划 (单纯形)	220
05G - 高斯消元 (GF(2) 位集)	221
05H - 矩阵树定理 (Kirchhoff)	222

06 - 分数 (Frac, with Int)	222
06A - 复数 (Complex, with Int)	223
06B - 三次单位根复数 (Complex3)	224
07 - 平面几何 (Point&Line, with Int)	225
07A - 凸包与旋转卡壳 (依赖07)	229
07AA - 闵可夫斯基和 (依赖07A)	230
07B - 圆与最小覆盖圆 (依赖07)	231
07C - 三维几何	232
07D - 平面最近点对 (依赖07)	233

数学：多项式、博弈、数值计算与位运算	234
08 - 快速数论变换 (NTT)	234
08A - 多项式 (Poly, int) (NTT)	235
08B - 多项式 (Poly, Z) (NTT)	237
08C - 轻量多项式 (Poly, Z) (NTT, ln, exp)	239
08D - 快速傅里叶变换 (FFT, with Complex)	241
08E - 快速傅里叶变换 (ModFFT, with Complex)	242
08F - 多模快速数论变换 (CRTNTT, with NTT)	243
08G - FWT (OR、AND、XOR 卷积)	244
08H - SOS DP (子集、超集 Zeta 变换)	245
08HA - 子集卷积 (Subset Convolution)	245
08I - 快速数论变换 (NTT, 极致优化)	246
08IA - 任意模卷积 (CRTNTT, 极致优化)	249
08J - 二维卷积 (2D NTT, 依赖08)	250
08K - 有理函数第n项 (Bostan-Mori, 依赖08B)	250
08L - 连续点值平移 (依赖08B)	251
08M - 稀疏多项式除法	251
08N - 多项式开方与幂 (依赖08B)	252
08NA - 多项式大指数幂 (依赖08B)	253
08O - Power Projection (基础)	253
08OA - Power Projection (极致优化, 依赖08I)	254
08P - 任意因子长度DFT (依赖08)	256
09 - 博弈论 (Nim、SG)	257
09A - Wythoff Nim	259
10A - 圆周率计算 (Machin, with FixedPoint)	259
10B - 圆周率计算 (Chudnovsky, with FixedPoint)	259
11A - 牛顿迭代 (方程、平方根、开 k 次方)	260
11B - 自适应辛普森积分	261
11C - 三分搜索 (实数、整数)	261
11D - 数值求根 (二分、割线)	262
12A - 位运算与子集枚举	262
12B - 异或线性基 (XorBasis64)	263
12C - 二进制 GCD 与 Gray 编码	264
13A - Fibonacci (快速倍增)	264
13B - 约瑟夫环	264
14 - 伯努利数与幂和	265
14A - 伯努利数 (NTT快速求法)	265

字符串	267
README	267
字符串知识与结论	268
01A - KMP	268
01AA - KMP自动机	269
01B - Z函数 (扩展KMP)	270
02A - Manacher	270
02B - 回文自动机 (PalindromicTree)	272
03 - AC自动机	273
04A - 后缀自动机	275
04B - 后缀数组 (SuffixArray)	277
04C - 后缀数组 (SA-IS, 线性)	277
05 - 字符串哈希 (StringHash, with T)	279
06 - 最小表示法 (Booth)	280
07 - Lyndon 分解 (Duval)	280
08A - 子序列自动机 (位置表)	281
08B - 子序列自动机 (稠密转移)	281

高精度	283
README	283
01A - 整数 (BigInt, 常规)	283
01B - 整数 (BigInt, 极致优化)	288
02A - 定点数 (FixedPoint, 常规)	295
02B - 定点数 (FastFixedPoint, 极致优化)	299
03A - 浮点数 (Decimal, 常规)	301
03B - 浮点数 (FastDecimal, 极致优化)	306
04A - 2至62进制转换 (常规)	306
04B - 2至62进制转换 (极致优化)	307

杂类	309
-----------	------------

README	309
杂类知识与结论	310
错题本	312
09 - Python短代码	316
01A - 普通莫队	318
01B - 带修改莫队	319
01C - 回滚莫队	319
01D - Hilbert序莫队	320
02 - 笛卡尔树	321
03 - CDQ分治（三维偏序）	321
04 - 整体二分（带修改区间第k小）	322
05 - WQS二分（带权二分）	324
06 - 舞蹈链（DLX，精确覆盖）	324
07 - 模拟退火	325
08 - 公历日期换算	326
10 - 德州扑克五张牌比较	326
11A - 物品装箱（First Fit线段树）	327
11B - 物品装箱（Best Fit多重集）	327
12 - 奇阶幻方	328
13 - 阿达马矩阵	328
14A - 选数枚举（DFS）	328
14B - 选数枚举（Gosper位运算）	329
15 - 麻将胡牌判定（标准、七对子、国士）	329

动态规划：背包与状态设计 331

README	331
动态规划知识与结论	331
01A - 01背包	332
01B - 完全背包	333
01C - 多重背包（二进制分组）	333
01D - 多重背包（单调队列）	333
01E - 分组背包	334
01F - 二维费用背包	334
01G - 01背包最优方案数（自动取模）	334
01H - 混合背包	335
01I - 有依赖的背包（树形）	335
01J - 01背包具体方案	336
02 - 数位DP（DFA）	336
03 - 状压DP（TSP）	337
04 - 轮廓线DP（多米诺骨牌）	337

动态规划：优化与最长公共子序列 339

05A - 分治DP优化	339
05B - Knuth优化	339
05C - 斜率优化（CHT，单调斜率）	339
06A - 最长公共子序列（常规）	340
06B - 最长公共子序列（Bitset）	341
07 - Slope Trick	341

基础算法

19 个代码模板 · 林月 · v1.0.0 · 2026.8.1

版块说明与知识结论

README.md

基础算法

代码片段默认依赖 `bits/stdc++.h` 和 `std` 命名空间；除专门说明外，数组接口均为 0-indexed。

需求	模板	说明
比赛主程序与常用宏	01A - 头文件（完整版）；01B - 头文件（极简版）	完整版内联常用宏与函数，本地调试时包含 <code>linyue.hpp</code> ；极简版仅保留必要环境。
整数平方根、整除、编码、字符工具	02 - 常用函数重载	<code>floor/ceil</code> 是数学整除
常量、方向数组	03 - 常用常量定义	按题目修改 <code>MOD</code> 、 <code>INF</code>
高速整数读写	04 - <code>fastio</code>	可移植分块输入输出， <code>Cin</code> 在 EOF 或格式/范围错误时返回 <code>false</code>
极限输入规模（Linux）	04A - <code>fastio</code> （Linux， <code>mmap</code> 极致）	<code>mmap</code> + 查表批量读写；仅 Linux/POSIX，交互题不要使用
i128 输入输出	05 - <code>i128</code> 函数重载	含安全解析函数
抗 Hack 哈希	06 - 手写随机哈希	用于 <code>unordered_map/set</code>
随机整数	07 - 随机数	使用 <code>mt19937_64</code> 与均匀分布
Windows 本地对拍	08 - 对拍	依赖 <code>data.exe</code> 、 <code>std.exe</code> 、 <code>solve.exe</code>
LIS/LNDS 长度与一组下标方案	09 - 最长上升子序列（LIS）	<code>LISIndices</code> / <code>LNDSIndices</code> 返回 0-indexed 下标
单调栈的最近元素	10 - 单调栈	严格/非严格比较均已封装
滑动窗口最值	16 - 单调队列	默认最大值， <code>greater<T></code> 分支维护最小值
离散化	11 - 坐标压缩	<code>index</code> 、 <code>lowerBound</code> 、 <code>upperBound</code>
单调谓词边界二分	12 - 二分查找（边界）	<code>first</code> 为假→真， <code>last</code> 为真→假
一维/二维前缀和、区间加差分	13 - 前缀和与差分	查询/更新区间均为闭区间
逆序对计数	14 - 逆序对（归并排序）	严格逆序对，输入不修改
对拍数据生成	15 - 随机数据生成（树与图）	排列、树、简单连通无向图、随机字符串
本地容器调试输出	<code>linyue.hpp</code>	完整版在非 <code>ONLINE_JUDGE</code> 环境自动包含；提交环境把 <code>_</code> 定义为空宏
编译参数、Sanitizer、计时与定位	基础算法与 STL 速查 第 5 节	按“警告 → 不变量 → 对拍 → Sanitizer → 性能基准”排查

使用提示

- 本地可写 `auto t = tic(); ...; toc(t, "name");` 计时；`_(x, y)` 输出到 `cerr`，不会污染标准输出。
- 多数模板假设计算结果能装进目标类型；路径和、区间和、逆序对通常直接使用默认 `int`。
- `first` 适用于假→真、`last` 适用于真→假的单调谓词；边界不存在时不要解引用返回的 `nullopt`。
- 单调栈处理子数组贡献时，左右边界必须一侧严格、另一侧非严格，避免重复值重复计数。

容器复杂度、库算法、输入输出与宏性能边界见 [基础算法与 STL 速查](#)。

基础算法与STL速查.md

基础算法与 STL 速查

1. 公共环境

提交前至少用 `-std=c++17 -O2 -Wall -Wextra` 本地编译一次；依赖 PBDS、i128、bits/stdc++.h 的代码属于 GCC/libstdc++ 环境特化。

`#define int long long` 对算术代码通常方便，但会让图边下标、自动机状态、DP 决策和二维表从 4 字节变成 8 字节。库中仍默认使用 `int` 保持风格统一；规模达到数百万或存在 $O(n \log n) / O(nA)$ 状态并实际超限时，再把纯下标和状态局部改为 `i32`。位并行代码保留 `u64`。

2. 容器复杂度

需求	首选	复杂度与提醒
尾部动态数组	<code>vector</code>	尾插均摊 $O(1)$ ；扩容会使迭代器、指针失效。
双端队列/单调队列	<code>deque</code>	两端 $O(1)$ ，内存不连续。
先进先出/后进先出	<code>queue / stack</code>	适合作为受限接口。
有序集合/映射	<code>set / map</code>	$O(\log n)$ ；需要排名用 FHQ/PBDS。
哈希集合/映射	<code>unordered_*</code>	平均 $O(1)$ 、最坏 $O(n)$ ；对抗数据使用自定义随机哈希。
优先队列	<code>priority_queue</code>	默认大根堆；小根堆写 <code>greater<T></code> 。
固定长度元组	<code>array</code>	可比较、连续内存，常比小 <code>vector</code> 更轻。

`map[key]` 在键不存在时会插入默认值；只查询应使用 `find/contains`。删除遍历中的元素写 `it = c.erase(it)`。`reserve` 只改容量，`resize` 才改元素个数。

GNU PBDS

PBDS 仅可用于 GCC/libstdc++，完整可抄模板见数据结构目录：10C/10E 为有序多重集及唯一键 `tree`，13A/13B 为 `gp_hash_table/cc_hash_table`，09B 为带节点句柄的可并堆，08C 为可做 `prefix_range` 的 Patricia trie。特别注意：`tree::find_by_order(k)` 使用 0-indexed；PBDS 哈希的默认策略不能照搬 `unordered_map::reserve`；有序多重集不能用违反严格弱序的 `less_equal` 假装实现。

3. 高频库算法

- `lower_bound` 找第一个 $\geq x$ ，`upper_bound` 找第一个 $> x$ ；容器必须已按同一比较器排序。
- `nth_element(a.begin(), a.begin()+k, a.end())` 把第 k 小放到位，平均线性，不保证两侧有序。
- `partial_sort` 需要有序的前若干项；只需一个次序统计量不要全排序。
- `unique` 只去除相邻重复，通常先 `sort`，再 `erase(unique(...), end())`。
- `next_permutation` 要从有序初态开始才能遍历全部排列。
- `accumulate` 的初值决定运算类型：本库写 `int{}`，泛型模板写 `T{}`，不要让裸 `0` 把累加器固定成原生 32 位整数。
- 浮点排序比较器必须满足严格弱序；不要把 `abs(a-b)<eps` 直接当 `operator<`。

4. 输入、输出与本地验证

- 混用 `getline` 与 `cin >>` 前先消费行末；未知个数的一行可 `getline` 后交给 `stringstream`。
- 浮点输出用 `fixed << setprecision(k)`；`setprecision` 在非 `fixed` 下表示有效数字数。
- `endl` 原本会刷新缓冲，本库宏把它换成换行字符串；交互题需要显式 `flush`。
- 随机造数必须使用均匀分布，不能用 `rng()%range` 验证概率性质；普通对拍小范围取模足够。
- 对拍至少保存随机种子、完整输入和两份输出；发现反例后先缩小数据，再改模板。

5. 考场调试、编译参数与卡常定位

5.1 四套编译方式

以下命令以 GCC/MinGW 的 C++17 为准；可执行文件名按系统改为 a 或 a.exe。

目的	建议参数	注意
日常警告	<code>g++ a.cpp -std=c++17 -O2 -g -Wall -Wextra -Wshadow=compatible-local -Winit-self -Wuninitialized</code>	先清末初始化、遮蔽和返回值问题。
容器边界	<code>g++ a.cpp -std=c++17 -O0 -g3 -D_GLIBCXX_DEBUG -D_GLIBCXX_ASSERTIONS</code>	会改变 STL 容器 ABI 与常数，只用于整份程序的本地调试。
未定义行为	<code>g++ a.cpp -std=c++17 -O1 -g3 -fsanitize=address,undefined -fno-omit-frame-pointer -fno-sanitize-recover=all</code>	不要与 <code>-static</code> 混用；评测机通常不提供 Sanitizer。
提交/基准	<code>g++ a.cpp -std=c++17 -O2 -pipe -DNDEBUG</code>	只在所有断言均已验证后加 <code>-DNDEBUG</code> 。

- ASan 重点检查越界、释放后使用和栈生命周期；UBSan 重点检查有符号溢出、非法移位、除零与错误对齐。
- 部分精简 MinGW 未附带 libasan/libubsan，链接时报 `cannot find -lasan/-lubsan` 时改用 WSL/Linux 或完整 LLVM/MinGW 工具链，不要误判为代码错误。
- `-D_GLIBCXX_DEBUG` 最适合定位失效迭代器和容器越界，但会大幅变慢；不要拿该版本做性能判断。
- Linux 递归爆栈先估算单层栈帧，再用 `ulimit -s unlimited` 验证；MinGW 可临时加 `-Wl,--stack=268435456`。最终仍优先改成显式栈。
- `-march=native`、`#pragma GCC target(...)` 依赖评测 CPU；未确认指令集时不要提交。`-Ofast` 会放宽浮点语义，可能破坏 NaN、无穷与精度相关算法。
- `#pragma GCC optimize("Ofast", "unroll-loops")` 不是通用加速开关。只有基准证明瓶颈在纯整数热点且题目允许时才启用，并保留不带 `pragma` 的正确版本。

5.2 定位流程

1. 固定随机种子并保存首个失败输入；先把规模、值域、边数和操作数缩到仍会失败的最小附近。
2. 在状态变化后检查不变量，不要只在最终答案处打印。二分插入检查点，先确定第一轮出现分歧的位置。
3. 小数据与朴素算法逐项比较；没有朴素解时使用交换输入、整体平移、加零项、重复求解等变形关系验证。
4. WA 优先核对边界、清空、重复值和溢出；RE 跑 ASan/UBSan；只在结果正确后再做计时与卡常。
5. 对拍输出必须写入 `cerr` 或独立日志；提交前搜索 `_()`、`toc()`、`freopen`、随机种子和临时断言。

常用算法不变量：

- 网络流：每条边满足 $0 \leq f \leq c$ ，除源汇外流量守恒，最终 `maxflow == mincut`。
- NTT/FFT：先验证 `intt(ntt(a)) == a`，再对拍朴素卷积；另检查 `sum(c) == sum(a)sum(b)`（同模意义）。任意模极限数据优先三模 NTT，不用浮点误差“碰运气”。
- 高精度：与 Python `int` 对拍正负号和前导零；除法同时检查 `a == q*b+r`、`|r| < |b|` 且余数符号约定一致。
- 数据结构：每次修改后用小数组长重算全部查询；可持久化结构额外检查旧版本未变化。

5.3 本地计时与基准

需要引入 `linyue.hpp`：

```
1 auto t = tic();
2 // 被测代码
3 toc(t, "ntt"); // 输出到 cerr; ONLINE_JUDGE 下提交前删除
```

- 使用 `steady_clock`，不要用可能受系统时间调整影响的时钟。先预热一次，再测 3-5 次并看中位数。
- 算法基准与读写基准分开；计时区间内禁止调试输出、随机设备初始化和重复申请无关大内存。
- 比较卡常模板时使用同一编译器、参数、输入、CPU 与进程位数。一次快几毫秒不足以证明常数更低。
- 性能异常先看复杂度与内存访问：连续数组、窄下标、减少 %、减少整轮变换通常比手动 `inline` 或循环展开更可靠。

5.4 GDB 最小命令集

g++ -O0 -g3 编译后执行 gdb ./a, 常用命令为 run < data.in、bt、frame n、print x、display x、break file.cpp:line、watch x、continue。优化后的变量可能被消除；先在 -O0 下定位正确性，再用 -O2 -g 复现只在优化时出现的问题。

6. 基础恒等式与边界

- 前缀和查询闭区间 $[l, r]$ 为 $s[r+1]-s[l]$ ；二维前缀和用四项容斥。
- 差分做闭区间加： $d[l]+=x, d[r+1]-=x$ ，必须给 $r+1$ 留哨兵。
- 单调栈统计重复值贡献时，一侧用严格比较、另一侧用非严格比较。
- 滑动窗口最值让队首到队尾按答案优先级单调；新位置入队前弹掉永远不可能成为答案的尾部，窗口左端越界后再弹队首。默认 `MonoQueue<T>` 维护最大值，最小值传 `greater<T>`。
- 二分只依赖单调谓词；先写出“不存在答案”时返回什么，再确定开闭边界。
- 整数中点用 $l+(r-l)/2$ ；带负数的数学 `floor/ceil` 除法不能直接套 C++ 向零截断。

01A - 头文件 (完整版) .cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  #ifndef ONLINE_JUDGE
5      #include "linyue.hpp"
6  #else
7      #define _(...)
8  #endif
9
10 namespace LYE {
11     #define int long long
12     #define pii array<int, 2>
13     #define endl "\n"
14     #define all(x) x.begin(), x.end()
15     #define rall(x) x.rbegin(), x.rend()
16     #define vi vector<int>
17     #define vvi vector<vector<int>>
18     #define pb push_back
19     #define pob pop_back
20     #define pf push_front
21     #define pof pop_front
22     template <class T> istream &operator>>(istream &is, vector<T> &v) {
23         for (auto &x : v) is >> x;
24         return is;
25     }
26     template <typename T> T min(const vector<T> &v) {
27         assert(!v.empty());
28         return *std::min_element(v.begin(), v.end());
29     }
30     template <typename T> T max(const vector<T> &v) {
31         assert(!v.empty());
32         return *std::max_element(v.begin(), v.end());
33     }
34     template <typename T> T acc(const vector<T> &v) {
35         return std::accumulate(v.begin(), v.end(), T{});
36     }
37     template <class T> void reverse(vector<T> &v) {
38         std::reverse(v.begin(), v.end());
39     }
40     template <class T> void sort(vector<T> &v) {
41         std::sort(v.begin(), v.end());
42     }
43     template <class T> void rsort(vector<T> &v) {
44         std::sort(v.rbegin(), v.rend());
45     }
46     template <class T> void cmax(T &x, T y) {
47         x = std::max(x, y);
48     }
49     template <class T> void cmin(T &x, T y) {
50         x = std::min(x, y);
51     }
52     template <class T> T sign(const T &a) {
53         return a == 0 ? 0 : (a < 0 ? -1 : 1);
54     }
55     template <class T> T floor(const T &a, const T &b) {
56         assert(b != 0);
57         assert(!(a == std::numeric_limits<T>::min() && b == -1));
58         T q = a / b, r = a % b;

```



```

59     if (r != 0 && ((r > 0) != (b > 0))) --q;
60     return q;
61 }
62 template <class T> T ceil(const T &a, const T &b) {
63     assert(b != 0);
64     assert(!(a == std::numeric_limits<T>::min() && b == -1));
65     T q = a / b, r = a % b;
66     if (r != 0 && ((r > 0) == (b > 0))) ++q;
67     return q;
68 }
69 int mypow(int n, int k, int p) {
70     using i128 = __int128_t;
71     assert(k >= 0 && p > 0);
72     n %= p;
73     if (n < 0) n += p;
74     int r = 1 % p;
75     while (k > 0) {
76         if (k & 1) r = i128(r) * n % p;
77         n = i128(n) * n % p;
78         k >>= 1;
79     }
80     return r;
81 }
82 constexpr int N = 1e6 + 7;
83 template <typename T = int> constexpr T INF = std::numeric_limits<T>::max() / 2;
84 }; // namespace LYE
85 using namespace LYE;
86
87 void solve() {}
88
89 signed main() {
90     ios::sync_with_stdio(0);
91     cin.tie(0);
92     cout << fixed << setprecision(20);
93     int t = 1;
94     cin >> t;
95     for (int i = 0; i < t; ++i) {
96         solve();
97     }
98     return 0;
99 }

```

01B - 头文件（极简版）.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  #define int long long
5  #define pii array<int, 2>
6  #define endl "\n"
7
8  void solve() {}
9
10 signed main() {
11     ios::sync_with_stdio(0), cin.tie(0);
12     cout << fixed << setprecision(20);
13     int t = 1;
14     cin >> t;
15     for (int i = 0; i < t; ++i) {
16         solve();
17     }
18     return 0;
19 }

```

02 - 常用函数重载.cpp

```

1  using i128 = __int128_t;
2
3  // 常用整型辅助函数与无上下文的小工具。
4  // 根号
5
6  int mysqrt(int n) {
7     assert(n >= 0);
8     int ans = sqrtl(n);
9     while (i128(ans + 1) * (ans + 1) <= n) ans++;
10    while (i128(ans) * ans > n) ans--;
11    return ans;
12 }

```

```

13 // 最小公倍数（约定返回非负值）。
14 int mylcm(int a, int b) {
15     if (a == 0 || b == 0) return 0;
16     int ans = a / gcd(a, b) * b;
17     return ans < 0 ? -ans : ans;
18 }
19 // log2向下取整
20 template <class T> int log2f(T n) {
21     assert(n > 0);
22     int ans = 0;
23     while (n > 1) n >>= 1, ++ans;
24     return ans;
25 }
26 int log2f(int n) {
27     assert(n > 0);
28     return 63 - __builtin_clzll(n);
29 }
30 // log2向上取整
31 template <class T> int log2c(T n) {
32     assert(n > 0);
33     return log2f(n) + ((n & (n - 1)) != 0);
34 }
35 int log2c(int n) {
36     assert(n > 0);
37     return log2f(n) + ((n & (n - 1)) != 0);
38 }
39 template <class T> T sign(const T &a) {
40     return a == 0 ? 0 : (a < 0 ? -1 : 1);
41 }
42 template <class T> T floor(const T &a, const T &b) {
43     assert(b != 0);
44     assert(!(a == std::numeric_limits<T>::min() && b == -1));
45     T q = a / b, r = a % b;
46     if (r != 0 && ((r > 0) != (b > 0))) --q;
47     return q;
48 }
49 template <class T> T ceil(const T &a, const T &b) {
50     assert(b != 0);
51     assert(!(a == std::numeric_limits<T>::min() && b == -1));
52     T q = a / b, r = a % b;
53     if (r != 0 && ((r > 0) == (b > 0))) ++q;
54     return q;
55 }
56 // a <= x && x <= b
57 template <typename T> bool In(T x, T a, T b) {
58     return std::min(a, b) <= x && x <= std::max(a, b);
59 }
60
61 // 矩阵边界（0-indexed）。
62 inline bool inGrid(int x, int y, int n, int m) {
63     return 0 <= x && x < n && 0 <= y && y < m;
64 }
65
66 inline bool inSq(int x, int y, int n) {
67     return inGrid(x, y, n, n);
68 }
69
70 // 把若干位按 base 进制编码：调用者保证不会溢出。
71 template <class T, class... A> T encode(T bas, A... a) {
72     T res = 0;
73     ((res = res * bas + static_cast<T>(a)), ...);
74     return res;
75 }
76
77 // ASCII 转大写/小写。
78 inline char tomaj(char c) {
79     if ('a' <= c && c <= 'z') return c - 32;
80     return c;
81 }
82
83 inline char tolow(char c) {
84     if ('A' <= c && c <= 'Z') return c + 32;
85     return c;
86 }
87 // 若干字符等于c
88 template <typename... A> bool sameto(char c, A... a) {
89     return ((a == c) && ...);
90 }
91 // 若干字符存在等于c
92 template <typename... A> bool exi(char c, A... a) {
93     return ((a == c) || ...);
94 }

```

03 - 常用常量定义.cpp

```

1 // 最大值
2 using ld = long double;
3
4 constexpr int inf = 1'000'000'007;
5 constexpr int INF = 2'000'000'000'000'000'000'009LL;
6 // 若需要更大的上界，替换上一行：
7 // constexpr int INF = 4'000'000'000'000'000'000'000LL;
8 // 模数
9 constexpr int MOD = 1'000'000'007;
10 // constexpr int MOD = 998'244'353;
11 // 很小的正数常量
12 constexpr ld eps = 1e-10L;
13 // horse-左上起逆时针
14 int dx[8] = {-1, -2, -2, -1, 1, 2, 2, 1};
15 int dy[8] = {2, 1, -1, -2, -2, -1, 1, 2};
16 // 4-左起逆时针
17 int dx4[4] = {-1, 0, 1, 0}, dy4[4] = {0, -1, 0, 1};
18 // 8-左上起逆时针
19 int dx8[8] = {-1, -1, -1, 0, 1, 1, 1, 0};
20 int dy8[8] = {1, 0, -1, -1, -1, 0, 1, 1};

```

04 - fastio.cpp

```

1 namespace FastIO {
2     constexpr int BSZ = 1 << 21;
3
4     char ib[BSZ];
5     int p1 = 0, p2 = 0;
6
7     inline int getc() {
8         if (p1 >= p2) {
9             p2 = fread(ib, 1, BSZ, stdin);
10            p1 = 0;
11            if (p2 == 0) return EOF;
12        }
13        return static_cast<unsigned char>(ib[p1++]);
14    }
15
16    template <typename T> bool Cin(T &a) {
17        static_assert(std::is_integral_v<T> && !std::is_same_v<std::remove_cv_t<T>, bool>);
18
19        int c = getc();
20        while (c != EOF && (c < '0' || c > '9') && c != '-' && c != '+') c = getc();
21        if (c == EOF) return false;
22
23        bool y = false;
24        if (c == '-') {
25            if constexpr (!std::is_signed_v<T>) return false;
26            y = true;
27            c = getc();
28        } else if (c == '+') {
29            c = getc();
30        }
31        if (c < '0' || c > '9') return false;
32
33        using U = std::make_unsigned_t<T>;
34        const U lim = [&] {
35            if constexpr (std::is_signed_v<T>) {
36                return y ? static_cast<U>(std::numeric_limits<T>::max()) + U{1} : static_cast<U>(std::numeric_limits<T>::max());
37            } else {
38                return std::numeric_limits<T>::max();
39            }
40        }();
41
42        U val = 0;
43        while (c >= '0' && c <= '9') {
44            const U dig = c - '0';
45            if (val > (lim - dig) / 10) return false;
46            val = val * 10 + dig;
47            c = getc();
48        }
49
50        if constexpr (std::is_signed_v<T>) {
51            if (y) {
52                a = (val == lim) ? std::numeric_limits<T>::min() : -static_cast<T>(val);
53            } else {

```

```

54         a = static_cast<T>(val);
55     }
56 } else {
57     a = val;
58 }
59 return true;
60 }
61
62 template <typename T, typename U, typename... A> bool Cin(T &a, U &b, A &...c) {
63     return Cin(a) && Cin(b, c...);
64 }
65
66 char ob[BSZ];
67 int p3 = 0;
68
69 inline void flush() {
70     if (p3 == 0) return;
71     fwrite(ob, 1, p3, stdout);
72     p3 = 0;
73 }
74
75 inline void putc(char c) {
76     if (p3 == BSZ) flush();
77     ob[p3++] = c;
78 }
79
80 template <typename T> void Cout(T x, char end = '\n') {
81     static_assert(std::is_integral_v<T> && !std::is_same_v<std::remove_cv_t<T>, bool>);
82     using U = std::make_unsigned_t<T>;
83
84     U val;
85     if constexpr (std::is_signed_v<T>) {
86         if (x < 0) {
87             putc('-');
88             val = U{0} - static_cast<U>(x);
89         } else {
90             val = static_cast<U>(x);
91         }
92     } else {
93         val = x;
94     }
95
96     char s[std::numeric_limits<U>::digits10 + 3];
97     int n = 0;
98     do {
99         s[n++] = static_cast<char>('0' + val % 10);
100        val /= 10;
101    } while (val > 0);
102    while (n > 0) putc(s[--n]);
103    putc(end);
104 }
105
106 struct Flush {
107     ~Flush() {
108         flush();
109     }
110 };
111 inline Flush io;
112 } // namespace FastIO
113
114 using namespace FastIO;

```

04A - fastio (Linux, mmap极致) .cpp

```

1  /*
2  用途: Linux mmap + 分块表的极致整数输入输出。
3  限制: 仅 Linux/POSIX; Windows 或需要可移植性的题目使用 04 - fastio。
4  性能: 普通文件输入走 mmap, 管道输入回退 read; 输出为连续缓冲, 常数显著小于 iostream; 不支持交互题。
5  */
6  #ifdef __linux__
7      #include <fcntl.h>
8      #include <cerrno>
9      #include <sys/mman.h>
10     #include <sys/stat.h>
11     #include <unistd.h>
12 #endif
13
14 #ifdef __linux__
15 namespace fastio {
16     using i32 = std::int32_t;

```

```

17 using u32 = std::uint32_t;
18 using i64 = std::int64_t;
19 using u64 = std::uint64_t;
20 using u8 = std::uint8_t;
21
22 struct Tables {
23     alignas(64) std::int8_t r2[1 << 16];
24     alignas(64) u32 p4[10000];
25
26     Tables() {
27         std::memset(r2, -1, sizeof(r2));
28
29         for (int hi = '0'; hi <= '9'; ++hi)
30             for (int lo = '0'; lo <= '9'; ++lo) r2[(hi << 8) | lo] = (lo & 15) * 10 + (hi & 15);
31
32         for (u32 x = 0; x < 10000; ++x) {
33             p4[x] = u32('0' + x / 1000) | (u32('0' + x / 100 % 10) << 8) | (u32('0' + x / 10 % 10) << 16) |
34                 (u32('0' + x % 10) << 24);
35         }
36     };
37
38     static Tables tab;
39
40     struct Input {
41         std::vector<char> buf;
42         const char *p = nullptr;
43         const char *e = nullptr;
44         alignas(64) char tail[64]{};
45         bool mm = false;
46
47         Input() {
48             struct stat st{};
49             if (fstat(STDIN_FILENO, &st) == 0 && S_ISREG(st.st_mode) && st.st_size > 0) {
50                 void *q = mmap(nullptr, st.st_size, PROT_READ, MAP_PRIVATE, STDIN_FILENO, 0);
51                 if (q != MAP_FAILED) {
52                     madvise(q, st.st_size, MADV_SEQUENTIAL);
53                     p = reinterpret_cast<const char *>(q);
54                     e = p + st.st_size;
55                     mm = true;
56                     return;
57                 }
58             }
59
60             // 评测机常把 stdin 接成管道; 此时 mmap 不可用, 回退到一次性 read。
61             buf.reserve(1u << 20);
62             char t[1u << 16];
63             for (;;) {
64                 ssize_t n = read(STDIN_FILENO, t, sizeof(t));
65                 if (n > 0) {
66                     buf.insert(buf.end(), t, t + n);
67                     continue;
68                 }
69                 if (n < 0 && errno == EINTR) continue;
70                 break;
71             }
72             buf.push_back('\n');
73             buf.push_back('\0');
74             p = buf.data();
75             e = p + buf.size();
76         }
77
78         void guard(const char *q) {
79             if (!mm || static_cast<std::size_t>(e - q) >= 32) return;
80             std::size_t n = e - q;
81             std::memcpy(tail, q, n);
82             std::memset(tail + n, 0, sizeof(tail) - n);
83             q = tail;
84             mm = false;
85         }
86     };
87
88     static Input in;
89     static const char *cur = in.p;
90
91 #define READ2(N, LABEL)
92 \
93 \
94 \
95 \
96 \
97 \
98 \
99 \
100 \
101 \
102 \
103 \
104 \
105 \
106 \
107 \
108 \
109 \
110 \
111 \
112 \
113 \
114 \
115 \
116 \
117 \
118 \
119 \
120 \
121 \
122 \
123 \
124 \
125 \
126 \
127 \
128 \
129 \
130 \
131 \
132 \
133 \
134 \
135 \
136 \
137 \
138 \
139 \
140 \
141 \
142 \
143 \
144 \
145 \
146 \
147 \
148 \
149 \
150 \
151 \
152 \
153 \
154 \
155 \
156 \
157 \
158 \
159 \
160 \
161 \
162 \
163 \
164 \
165 \
166 \
167 \
168 \
169 \
170 \
171 \
172 \
173 \
174 \
175 \
176 \
177 \
178 \
179 \
180 \
181 \
182 \
183 \
184 \
185 \
186 \
187 \
188 \
189 \
190 \
191 \
192 \
193 \
194 \
195 \
196 \
197 \
198 \
199 \
200 \
201 \
202 \
203 \
204 \
205 \
206 \
207 \
208 \
209 \
210 \
211 \
212 \
213 \
214 \
215 \
216 \
217 \
218 \
219 \
220 \
221 \
222 \
223 \
224 \
225 \
226 \
227 \
228 \
229 \
230 \
231 \
232 \
233 \
234 \
235 \
236 \
237 \
238 \
239 \
240 \
241 \
242 \
243 \
244 \
245 \
246 \
247 \
248 \
249 \
250 \
251 \
252 \
253 \
254 \
255 \
256 \
257 \
258 \
259 \
260 \
261 \
262 \
263 \
264 \
265 \
266 \
267 \
268 \
269 \
270 \
271 \
272 \
273 \
274 \
275 \
276 \
277 \
278 \
279 \
280 \
281 \
282 \
283 \
284 \
285 \
286 \
287 \
288 \
289 \
290 \
291 \
292 \
293 \
294 \
295 \
296 \
297 \
298 \
299 \
300 \
301 \
302 \
303 \
304 \
305 \
306 \
307 \
308 \
309 \
310 \
311 \
312 \
313 \
314 \
315 \
316 \
317 \
318 \
319 \
320 \
321 \
322 \
323 \
324 \
325 \
326 \
327 \
328 \
329 \
330 \
331 \
332 \
333 \
334 \
335 \
336 \
337 \
338 \
339 \
340 \
341 \
342 \
343 \
344 \
345 \
346 \
347 \
348 \
349 \
350 \
351 \
352 \
353 \
354 \
355 \
356 \
357 \
358 \
359 \
360 \
361 \
362 \
363 \
364 \
365 \
366 \
367 \
368 \
369 \
370 \
371 \
372 \
373 \
374 \
375 \
376 \
377 \
378 \
379 \
380 \
381 \
382 \
383 \
384 \
385 \
386 \
387 \
388 \
389 \
390 \
391 \
392 \
393 \
394 \
395 \
396 \
397 \
398 \
399 \
400 \
401 \
402 \
403 \
404 \
405 \
406 \
407 \
408 \
409 \
410 \
411 \
412 \
413 \
414 \
415 \
416 \
417 \
418 \
419 \
420 \
421 \
422 \
423 \
424 \
425 \
426 \
427 \
428 \
429 \
430 \
431 \
432 \
433 \
434 \
435 \
436 \
437 \
438 \
439 \
440 \
441 \
442 \
443 \
444 \
445 \
446 \
447 \
448 \
449 \
450 \
451 \
452 \
453 \
454 \
455 \
456 \
457 \
458 \
459 \
460 \
461 \
462 \
463 \
464 \
465 \
466 \
467 \
468 \
469 \
470 \
471 \
472 \
473 \
474 \
475 \
476 \
477 \
478 \
479 \
480 \
481 \
482 \
483 \
484 \
485 \
486 \
487 \
488 \
489 \
490 \
491 \
492 \
493 \
494 \
495 \
496 \
497 \
498 \
499 \
500 \
501 \
502 \
503 \
504 \
505 \
506 \
507 \
508 \
509 \
510 \
511 \
512 \
513 \
514 \
515 \
516 \
517 \
518 \
519 \
520 \
521 \
522 \
523 \
524 \
525 \
526 \
527 \
528 \
529 \
530 \
531 \
532 \
533 \
534 \
535 \
536 \
537 \
538 \
539 \
540 \
541 \
542 \
543 \
544 \
545 \
546 \
547 \
548 \
549 \
550 \
551 \
552 \
553 \
554 \
555 \
556 \
557 \
558 \
559 \
560 \
561 \
562 \
563 \
564 \
565 \
566 \
567 \
568 \
569 \
570 \
571 \
572 \
573 \
574 \
575 \
576 \
577 \
578 \
579 \
580 \
581 \
582 \
583 \
584 \
585 \
586 \
587 \
588 \
589 \
590 \
591 \
592 \
593 \
594 \
595 \
596 \
597 \
598 \
599 \
600 \
601 \
602 \
603 \
604 \
605 \
606 \
607 \
608 \
609 \
610 \
611 \
612 \
613 \
614 \
615 \
616 \
617 \
618 \
619 \
620 \
621 \
622 \
623 \
624 \
625 \
626 \
627 \
628 \
629 \
630 \
631 \
632 \
633 \
634 \
635 \
636 \
637 \
638 \
639 \
640 \
641 \
642 \
643 \
644 \
645 \
646 \
647 \
648 \
649 \
650 \
651 \
652 \
653 \
654 \
655 \
656 \
657 \
658 \
659 \
660 \
661 \
662 \
663 \
664 \
665 \
666 \
667 \
668 \
669 \
670 \
671 \
672 \
673 \
674 \
675 \
676 \
677 \
678 \
679 \
680 \
681 \
682 \
683 \
684 \
685 \
686 \
687 \
688 \
689 \
690 \
691 \
692 \
693 \
694 \
695 \
696 \
697 \
698 \
699 \
700 \
701 \
702 \
703 \
704 \
705 \
706 \
707 \
708 \
709 \
710 \
711 \
712 \
713 \
714 \
715 \
716 \
717 \
718 \
719 \
720 \
721 \
722 \
723 \
724 \
725 \
726 \
727 \
728 \
729 \
730 \
731 \
732 \
733 \
734 \
735 \
736 \
737 \
738 \
739 \
740 \
741 \
742 \
743 \
744 \
745 \
746 \
747 \
748 \
749 \
750 \
751 \
752 \
753 \
754 \
755 \
756 \
757 \
758 \
759 \
760 \
761 \
762 \
763 \
764 \
765 \
766 \
767 \
768 \
769 \
770 \
771 \
772 \
773 \
774 \
775 \
776 \
777 \
778 \
779 \
780 \
781 \
782 \
783 \
784 \
785 \
786 \
787 \
788 \
789 \
790 \
791 \
792 \
793 \
794 \
795 \
796 \
797 \
798 \
799 \
800 \
801 \
802 \
803 \
804 \
805 \
806 \
807 \
808 \
809 \
810 \
811 \
812 \
813 \
814 \
815 \
816 \
817 \
818 \
819 \
820 \
821 \
822 \
823 \
824 \
825 \
826 \
827 \
828 \
829 \
830 \
831 \
832 \
833 \
834 \
835 \
836 \
837 \
838 \
839 \
840 \
841 \
842 \
843 \
844 \
845 \
846 \
847 \
848 \
849 \
850 \
851 \
852 \
853 \
854 \
855 \
856 \
857 \
858 \
859 \
860 \
861 \
862 \
863 \
864 \
865 \
866 \
867 \
868 \
869 \
870 \
871 \
872 \
873 \
874 \
875 \
876 \
877 \
878 \
879 \
880 \
881 \
882 \
883 \
884 \
885 \
886 \
887 \
888 \
889 \
890 \
891 \
892 \
893 \
894 \
895 \
896 \
897 \
898 \
899 \
900 \
901 \
902 \
903 \
904 \
905 \
906 \
907 \
908 \
909 \
910 \
911 \
912 \
913 \
914 \
915 \
916 \
917 \
918 \
919 \
920 \
921 \
922 \
923 \
924 \
925 \
926 \
927 \
928 \
929 \
930 \
931 \
932 \
933 \
934 \
935 \
936 \
937 \
938 \
939 \
940 \
941 \
942 \
943 \
944 \
945 \
946 \
947 \
948 \
949 \
950 \
951 \
952 \
953 \
954 \
955 \
956 \
957 \
958 \
959 \
960 \
961 \
962 \
963 \
964 \
965 \
966 \
967 \
968 \
969 \
970 \
971 \
972 \
973 \
974 \
975 \
976 \
977 \
978 \
979 \
980 \
981 \
982 \
983 \
984 \
985 \
986 \
987 \
988 \
989 \
990 \
991 \
992 \
993 \
994 \
995 \
996 \
997 \
998 \
999 \

```

```

94     if (__builtin_expect(d < 0, 0)) goto LABEL;
95     \
96     (N) = (N) * 100u + static_cast<unsigned>(d);
97     \
98     cur += 2;
99     \
100     } while (0)
101 #define IS_DIGIT(C) (static_cast<unsigned>(static_cast<unsigned char>(C) - '0') < 10u)
102 #define LAST_AND_NEXT(N)
103 \
104 do {
105 \
106     if (IS_DIGIT(*cur)) (N) = (N) * 10u + static_cast<unsigned>(*cur++ & 15);
107 \
108     while (__builtin_expect(static_cast<unsigned>(static_cast<unsigned char>(*++cur)) - 1u < 32u, 0)) {
109 \
110     }
111 \
112     return (N);
113 \
114 } while (0)
115
116 static inline u32 ru32_body() {
117     u32 n = 0;
118
119     READ2(n, ru32_done);
120     READ2(n, ru32_done);
121     READ2(n, ru32_done);
122     READ2(n, ru32_done);
123     READ2(n, ru32_done);
124
125     ru32_done:
126     LAST_AND_NEXT(n);
127 }
128
129 static inline u64 ru64_body() {
130     u64 n = 0;
131     READ2(n, ru64_done);
132     READ2(n, ru64_done);
133     READ2(n, ru64_done);
134     READ2(n, ru64_done);
135     READ2(n, ru64_done);
136     READ2(n, ru64_done);
137     READ2(n, ru64_done);
138     READ2(n, ru64_done);
139     READ2(n, ru64_done);
140
141     ru64_done:
142     LAST_AND_NEXT(n);
143 }
144
145 #undef READ2
146 #undef IS_DIGIT
147 #undef LAST_AND_NEXT
148
149 static inline u32 ru32() {
150     in.guard(cur);
151     if (__builtin_expect(*cur == '+', 0)) ++cur;
152     return ru32_body();
153 }
154
155 static inline u64 ru64() {
156     in.guard(cur);
157     if (__builtin_expect(*cur == '+', 0)) ++cur;
158     return ru64_body();
159 }
160
161 static inline i32 r32() {
162     in.guard(cur);
163     const bool neg = (*cur == '-');
164     cur += neg;
165     const u32 x = ru32_body();
166
167     return static_cast<i32>(neg ? 0u - x : x);
168 }
169
170 static inline i64 r64() {
171     in.guard(cur);
172     const bool neg = (*cur == '-');

```

```

167     cur += neg;
168     const u64 x = ru64_body();
169
170     return static_cast<i64>(neg ? 0ull - x : x);
171 }
172
173 static constexpr std::size_t OUT_BYTES = 1u << 22;
174 alignas(64) static u8 obuf[OUT_BYTES + 32];
175 static u8 *now = obuf;
176
177 static inline void flush() {
178     if (now != obuf) {
179         std::fwrite(obuf, 1, now - obuf, stdout);
180         now = obuf;
181     }
182 }
183
184 static inline void reserve(std::size_t n) {
185     if (__builtin_expect(now + n > obuf + OUT_BYTES, 0)) flush();
186 }
187
188 template <class U, int K> static inline void pu_unchecked(U x) {
189     u8 *const end = now + K;
190     u8 *p = end;
191
192     while (x > 9999) {
193         p -= 4;
194         *reinterpret_cast<u32 *>(p) = tab.p4[static_cast<u32>(x % 10000)];
195         x /= 10000;
196     }
197
198     const u32 w = tab.p4[static_cast<u32>(x)];
199
200     if (x < 10) {
201         *--p = static_cast<u8>('0' + x);
202     } else if (x < 100) {
203         p -= 2;
204         *reinterpret_cast<std::uint16_t *>(p) = static_cast<std::uint16_t>(w >> 16);
205     } else if (x < 1000) {
206         p -= 4;
207         *reinterpret_cast<u32 *>(p) = w;
208         ++p;
209     } else {
210         p -= 4;
211         *reinterpret_cast<u32 *>(p) = w;
212     }
213
214     std::memmove(now, p, end - p);
215     now += end - p;
216 }
217
218 static inline void p32(u32 x) {
219     reserve(10);
220     pu_unchecked<u32, 10>(x);
221 }
222
223 static inline void p64(u64 x) {
224     reserve(20);
225     pu_unchecked<u64, 20>(x);
226 }
227
228 static inline void p32(i32 x) {
229     reserve(11);
230     if (x < 0) {
231         *now++ = '-';
232         pu_unchecked<u32, 10>(0u - static_cast<u32>(x));
233     } else {
234         pu_unchecked<u32, 10>(static_cast<u32>(x));
235     }
236 }
237
238 static inline void p64(i64 x) {
239     reserve(21);
240     if (x < 0) {
241         *now++ = '-';
242         pu_unchecked<u64, 20>(0ull - static_cast<u64>(x));
243     } else {
244         pu_unchecked<u64, 20>(static_cast<u64>(x));
245     }
246 }
247
248 static inline void pc(char c) {

```

```

249     reserve(1);
250     *now++ = static_cast<u8>(c);
251 }
252
253 template <class T> static inline void write(T x) {
254     if constexpr (sizeof(T) <= 4) {
255         if constexpr (std::is_signed_v<T>)
256             p32(static_cast<i32>(x));
257         else
258             p32(static_cast<u32>(x));
259     } else {
260         if constexpr (std::is_signed_v<T>)
261             p64(static_cast<i64>(x));
262         else
263             p64(static_cast<u64>(x));
264     }
265 }
266
267 struct Flush {
268     ~Flush() { flush(); }
269 };
270 static Flush out;
271
272 } // namespace fastio
273 #endif

```

05 - i128函数重载.cpp

```

1  using i128 = __int128_t;
2  using u128 = __uint128_t;
3
4  inline bool read128(const std::string &s, i128 &n) {
5      if (s.empty()) return false;
6
7      int pos = 0;
8      bool y = false;
9      if (s[pos] == '-' || s[pos] == '+') {
10         y = (s[pos] == '-');
11         ++pos;
12     }
13     if (pos == s.size()) return false;
14
15     const u128 lim = y ? (u128{1} << 127) : ((u128{1} << 127) - 1);
16     u128 val = 0;
17     for (; pos < s.size(); ++pos) {
18         const char c = s[pos];
19         if (c < '0' || c > '9') return false;
20         const u128 dig = static_cast<u128>(c - '0');
21         if (val > (lim - dig) / 10) return false;
22         val = val * 10 + dig;
23     }
24
25     if (y) {
26         n = (val == lim) ? -static_cast<i128>(lim - 1) - 1 : -static_cast<i128>(val);
27     } else {
28         n = static_cast<i128>(val);
29     }
30     return true;
31 }
32
33 inline std::istream &operator>>(std::istream &is, i128 &n) {
34     std::string s;
35     if (!(is >> s)) return is;
36     if (!read128(s, n)) is.setstate(std::ios::failbit);
37     return is;
38 }
39
40 inline std::ostream &operator<<(std::ostream &os, i128 n) {
41     if (n == 0) return os << '0';
42
43     u128 val;
44     if (n < 0) {
45         os << '-';
46         val = u128{0} - static_cast<u128>(n);
47     } else {
48         val = static_cast<u128>(n);
49     }
50
51     std::string s;
52     while (val > 0) {

```



```

53     s += static_cast<char>('0' + val % 10);
54     val /= 10;
55 }
56 std::reverse(s.begin(), s.end());
57 return os << s;
58 }
59
60 inline i128 toi128(const std::string &s) {
61     i128 n = 0;
62     assert(read128(s, n));
63     return n;
64 }

```

06 - 手写随机哈希.cpp

```

1  using u64 = uint64_t;
2
3  // 抗 hack 哈希
4
5  struct Hash {
6
7      static u64 hash(u64 x) {
8          x += 0x9e3779b97f4a7c15;
9          x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
10         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
11         return x ^ (x >> 31);
12     }
13
14     static const u64 sd;
15
16     static u64 hash(const string &x) {
17         return std::hash<string>{}(x);
18     }
19
20     template <class T> size_t operator()(T x) const {
21         return hash(u64(x) + sd);
22     }
23     size_t operator()(const string &s) const {
24         u64 h = 0;
25         for (char c : s) h = h * 131 + c;
26         return hash(h + sd);
27     }
28     template <class T1, class T2> size_t operator()(const pair<T1, T2> &p) const {
29         return operator()(p.first) ^ (operator()(p.second) >> 1);
30     }
31     template <class T> size_t operator()(const vector<T> &v) const {
32         u64 h = 0;
33         for (auto &x : v) {
34             h = h * 131 + operator()(x);
35         }
36         return hash(h);
37     }
38 };
39 const u64 Hash::sd = chrono::steady_clock::now().time_since_epoch().count();
40
41 // struct , 需要在结构体中重载等于号
42 struct Node {
43     string x, y;
44     int z;
45     friend bool operator==(const Node &a, const Node &b) {
46         return a.x == b.x && a.y == b.y && a.z == b.z;
47     }
48 };
49 // 结构体版本; 也可以把这个 operator() 作为重载加入上面的 Hash。
50 struct NodeHash {
51     size_t operator()(const Node &p) const {
52         Hash h;
53         return h(p.x) ^ (h(p.y) << 1) ^ (h(p.z) << 2);
54     }
55 };

```

07 - 随机数.cpp

```

1 namespace Random {
2     mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
3
4     template <class T> T randint(T a, T b) {
5         static_assert(std::is_integral_v<T> && !std::is_same_v<std::remove_cv_t<T>, bool>);
6         assert(a <= b);
7         return uniform_int_distribution<T>(a, b)(rng);
8     }
9 } // namespace Random
10
11 using namespace Random;

```

08 - 对拍.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 #define int long long
5 #define pii array<int, 2>
6 #define endl "\n"
7
8 signed main(signed ac, char* av[]) {
9     int n = 10000;
10    if (ac >= 2) n = atoi(av[1]);
11    for (int _ = 1; _ <= n; ++_) {
12        if (system("data.exe > data.in") != 0) {
13            cerr << "data.exe failed on test " << _ << "\n";
14            return 1;
15        }
16        if (system("std.exe < data.in > std.out") != 0) {
17            cerr << "std.exe failed on test " << _ << "\n";
18            return 1;
19        }
20        if (system("solve.exe < data.in > solve.out") != 0) {
21            cerr << "solve.exe failed on test " << _ << "\n";
22            return 1;
23        }
24
25        int dif = system("fc std.out solve.out > diff.log");
26        if (dif == 1) {
27            cout << "WA on test " << _ << "\n";
28            break;
29        }
30        if (dif != 0) {
31            cerr << "fc failed on test " << _ << "\n";
32            return 1;
33        }
34        cout << "AC " << _ << "\n";
35    }
36    return 0;
37 }

```

09 - 最长上升子序列 (LIS) .cpp

```

1 // 严格递增子序列长度
2 template <class T> int LIS(const vector<T> &a) {
3     vector<T> d;
4     for (auto x : a) {
5         auto it = lower_bound(d.begin(), d.end(), x);
6         if (it == d.end()) d.push_back(x);
7         else *it = x;
8     }
9     return d.size();
10 }
11
12 // 非降子序列长度
13 template <class T> int LNDS(const vector<T> &a) {
14     vector<T> d;
15     for (auto x : a) {
16         auto it = upper_bound(d.begin(), d.end(), x);
17         if (it == d.end()) d.push_back(x);
18         else *it = x;
19     }
20     return d.size();
21 }

```

```

22
23 // 返回一组严格递增 LIS 的原数组下标 (0-indexed)。
24 template <class T> vector<int> lis(const vector<T> &a) {
25     int n = a.size();
26     vector<T> d;
27     vector<int> id, pre(n, -1);
28     d.reserve(n);
29     id.reserve(n);
30
31     for (int i = 0; i < n; ++i) {
32         int p = lower_bound(d.begin(), d.end(), a[i]) - d.begin();
33         if (p > 0) pre[i] = id[p - 1];
34         if (p == d.size()) {
35             d.push_back(a[i]);
36             id.push_back(i);
37         } else {
38             d[p] = a[i];
39             id[p] = i;
40         }
41     }
42
43     vector<int> ans;
44     if (id.empty()) return ans;
45     for (int cur = id.back(); cur != -1; cur = pre[cur]) {
46         ans.push_back(cur);
47     }
48     reverse(ans.begin(), ans.end());
49     return ans;
50 }
51
52 // 返回一组非降 LNDS 的原数组下标 (0-indexed)。
53 template <class T> vector<int> lnds(const vector<T> &a) {
54     int n = a.size();
55     vector<T> d;
56     vector<int> id, pre(n, -1);
57     d.reserve(n);
58     id.reserve(n);
59
60     for (int i = 0; i < n; ++i) {
61         int p = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
62         if (p > 0) pre[i] = id[p - 1];
63         if (p == d.size()) {
64             d.push_back(a[i]);
65             id.push_back(i);
66         } else {
67             d[p] = a[i];
68             id[p] = i;
69         }
70     }
71
72     vector<int> ans;
73     if (id.empty()) return ans;
74     for (int cur = id.back(); cur != -1; cur = pre[cur]) {
75         ans.push_back(cur);
76     }
77     reverse(ans.begin(), ans.end());
78     return ans;
79 }

```

10 - 单调栈.cpp

```

1  /*
2  用途: 求每个位置左侧 (prev) 或右侧 (next) 最近的满足大小关系的位置。
3
4  约定:
5  - 输入 vector 为 0-indexed, 返回值也是下标; 不存在时返回 -1。
6  - less / greater 为严格比较, less_equal / greater_equal 为非严格比较。
7  - 求所有子数组最小值/最大值贡献时, 左右两侧应一边用严格比较、另一边用非严格比较,
8    以保证重复元素只计算一次。
9  */
10
11 template <class T> struct MonoStack {
12     private:
13         template <class F> static vector<int> prev(const vector<T> &a, F f) {
14             int n = a.size();
15             vector<int> ans(n), st;
16             st.reserve(n);
17             for (int i = 0; i < n; ++i) {
18                 while (!st.empty() && !f(a[st.back()], a[i])) st.pop_back();
19                 ans[i] = st.empty() ? -1 : st.back();

```

```

20     st.push_back(i);
21 }
22     return ans;
23 }
24
25 template <class F> static vector<int> next(const vector<T> &a, F f) {
26     int n = a.size();
27     vector<int> ans(n), st;
28     st.reserve(n);
29     for (int i = n - 1; i >= 0; --i) {
30         while (!st.empty() && !f(a[st.back()], a[i])) st.pop_back();
31         ans[i] = st.empty() ? -1 : st.back();
32         st.push_back(i);
33     }
34     return ans;
35 }
36
37 public:
38     static vector<int> preL(const vector<T> &a) {
39         return prev(a, [](const T &x, const T &y) { return x < y; });
40     }
41
42     static vector<int> preLE(const vector<T> &a) {
43         return prev(a, [](const T &x, const T &y) { return x <= y; });
44     }
45
46     static vector<int> preG(const vector<T> &a) {
47         return prev(a, [](const T &x, const T &y) { return x > y; });
48     }
49
50     static vector<int> preGE(const vector<T> &a) {
51         return prev(a, [](const T &x, const T &y) { return x >= y; });
52     }
53
54     static vector<int> nxtL(const vector<T> &a) {
55         return next(a, [](const T &x, const T &y) { return x < y; });
56     }
57
58     static vector<int> nxtLE(const vector<T> &a) {
59         return next(a, [](const T &x, const T &y) { return x <= y; });
60     }
61
62     static vector<int> nxtG(const vector<T> &a) {
63         return next(a, [](const T &x, const T &y) { return x > y; });
64     }
65
66     static vector<int> nxtGE(const vector<T> &a) {
67         return next(a, [](const T &x, const T &y) { return x >= y; });
68     }
69 };

```

11 - 坐标压缩.cpp

```

1  /*
2  用途：离散化任意可比较值；下标从 0 开始。
3  接口：先 add / 构造，再 build: askId(x) 要求 x 已出现，ask / askR 可查询任意值。
4  复杂度：build  $O(k \log k)$ ，其余查询  $O(\log k)$ 。
5  */
6  template <class T> struct Compress {
7      vector<T> a;
8
9      Compress() = default;
10
11     explicit Compress(vector<T> a) : a(move(a)) {
12         build();
13     }
14
15     void add(const T& x) {
16         a.push_back(x);
17     }
18
19     template <class It> void add(It l, It r) {
20         a.insert(a.end(), l, r);
21     }
22
23     void build() {
24         sort(a.begin(), a.end());
25         a.erase(unique(a.begin(), a.end()), a.end());
26     }
27

```

```

28     int askSz() const {
29         return a.size();
30     }
31
32     int ask(const T& x) const {
33         return lower_bound(a.begin(), a.end(), x) - a.begin();
34     }
35
36     int askR(const T& x) const {
37         return upper_bound(a.begin(), a.end(), x) - a.begin();
38     }
39
40     int askId(const T& x) const {
41         int p = ask(x);
42         assert(p < askSz() && a[p] == x);
43         return p;
44     }
45
46     const T& operator[](int p) const {
47         assert(0 <= p && p < askSz());
48         return a[p];
49     }
50 };

```

12 - 二分查找（边界）.cpp

```

1  /*
2  用途：单调谓词的边界二分。
3  约定：区间均为闭区间 [l, r]。first 要求 f 从 false 单调变为 true；
4  last 要求 f 从 true 单调变为 false。两者均不要求端点满足谓词，找不到时返回 nullopt。
5  复杂度：O(log(right - left + 1))。
6  */
7  template <class I, class F> optional<I> first(I l, I r, F f) {
8      static_assert(std::is_integral_v<I> && std::is_signed_v<I>);
9      optional<I> a;
10     while (l <= r) {
11         I m = l + (r - l) / 2;
12         if (f(m)) {
13             a = m;
14             r = m - 1;
15         } else {
16             l = m + 1;
17         }
18     }
19     return a;
20 }
21
22 template <class I, class F> optional<I> last(I l, I r, F f) {
23     static_assert(std::is_integral_v<I> && std::is_signed_v<I>);
24     optional<I> a;
25     while (l <= r) {
26         I m = l + (r - l) / 2;
27         if (f(m)) {
28             a = m;
29             l = m + 1;
30         } else {
31             r = m - 1;
32         }
33     }
34     return a;
35 }

```

13 - 前缀和与差分.cpp

```

1  /*
2  用途：静态一维/二维区间和与一维区间加差分。
3  约定：输入数组下标均为 0-indexed, ask / update 的区间均为闭区间 [l,r]。
4  复杂度：前缀和预处理 O(n) 或 O(nm)、查询 O(1)、差分每次更新 O(1)、build O(n)。
5  */
6  template <class T> struct PrefixSum {
7      vector<T> pre;
8
9      PrefixSum() = default;
10
11     explicit PrefixSum(const vector<T>& a) {
12         build(a);
13     }
14 }

```

```

15 void build(const vector<T>& a) {
16     pre.assign(a.size() + 1, T{});
17     for (int i = 0; i < a.size(); ++i) {
18         pre[i + 1] = pre[i] + a[i];
19     }
20 }
21
22 T ask(int l, int r) const {
23     assert(0 <= l && l <= r && r + 1 < pre.size());
24     return pre[r + 1] - pre[l];
25 }
26 };
27
28 template <class T> struct Prefix2D {
29     vector<vector<T>> pre;
30
31     Prefix2D() = default;
32
33     explicit Prefix2D(const vector<vector<T>>& a) {
34         build(a);
35     }
36
37     void build(const vector<vector<T>>& a) {
38         int n = a.size();
39         int m = n == 0 ? 0 : a[0].size();
40         for (const auto& v : a) assert(v.size() == m);
41
42         pre.assign(n + 1, vector<T>(m + 1, T{}));
43         for (int i = 0; i < n; ++i) {
44             for (int j = 0; j < m; ++j) {
45                 pre[i + 1][j + 1] = a[i][j] + pre[i][j + 1] + pre[i + 1][j] - pre[i][j];
46             }
47         }
48     }
49
50     T ask(int x1, int y1, int x2, int y2) const {
51         assert(0 <= x1 && x1 <= x2 && 0 <= y1 && y1 <= y2 &&
52             x2 + 1 < pre.size() && y2 + 1 < pre[0].size());
53         return pre[x2 + 1][y2 + 1] - pre[x1][y2 + 1] - pre[x2 + 1][y1] + pre[x1][y1];
54     }
55 };
56
57 template <class T> struct Diff {
58     vector<T> d;
59
60     Diff() = default;
61
62     explicit Diff(const vector<T>& a) {
63         build(a);
64     }
65
66     void build(const vector<T>& a) {
67         d.assign(a.size() + 1, T{});
68         for (int i = 0; i < a.size(); ++i) {
69             d[i] += a[i];
70             d[i + 1] -= a[i];
71         }
72     }
73
74     void update(int l, int r, const T& x) {
75         assert(0 <= l && l <= r && r + 1 < d.size());
76         d[l] += x;
77         d[r + 1] -= x;
78     }
79
80     vector<T> ask() const {
81         assert(!d.empty());
82         vector<T> res(d.size() - 1);
83         T cur{};
84         for (int i = 0; i < res.size(); ++i) {
85             cur += d[i];
86             res[i] = cur;
87         }
88         return res;
89     }
90 };

```

14 - 逆序对 (归并排序) .cpp

```
1 /*
```

```

2 用途：统计严格逆序对数，即  $i < j$  且  $values[i] > values[j]$  的对数。
3 约定：Compare 默认 less<T>；传入的比较器应满足严格弱序。
4 复杂度： $O(n \log n)$  时间、 $O(n)$  额外空间；输入数组不被修改。
5 */
6
7 template <class T, class Cmp = less<T>> int invCnt(vector<T> a, Cmp cmp = Cmp{}) {
8     vector<T> buf(a.size());
9
10     auto mg = [&](auto&& go, int l, int r) -> int {
11         if (r - l <= 1) return 0;
12         int mid = l + (r - l) / 2;
13         int ans = go(go, l, mid) + go(go, mid, r);
14
15         int i = l, j = mid, p = l;
16         while (i < mid || j < r) {
17             if (j == r || (i < mid && !cmp(a[j], a[i]))) {
18                 buf[p++] = a[i++];
19             } else {
20                 ans += mid - i;
21                 buf[p++] = a[j++];
22             }
23         }
24         for (int id = l; id < r; ++id) a[id] = buf[id];
25         return ans;
26     };
27
28     return mg(mg, 0, a.size());
29 }

```

15 - 随机数据生成（树与图）.cpp

```

1  /*
2  用途：对拍时生成随机数、排列、树、简单连通无向图和字符串。
3  约定：点编号 1..n；图无自环、无重边。稠密图改为枚举补边，避免拒绝采样卡死。
4  性能：默认使用展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
5  */
6  struct Gen {
7      using E = array<int, 2>;
8      mt19937_64 g;
9
10     explicit Gen(int s = chrono::steady_clock::now().time_since_epoch().count()) : g(s) {}
11
12     template <class T> T num(T l, T r) {
13         assert(l <= r);
14         return uniform_int_distribution<T>(l, r)(g);
15     }
16
17     template <class T> void mix(vector<T>& a) {
18         shuffle(a.begin(), a.end(), g);
19     }
20
21     vector<int> perm(int n, int s = 1) {
22         assert(n >= 0);
23         vector<int> a(n);
24         iota(a.begin(), a.end(), s);
25         mix(a);
26         return a;
27     }
28
29     vector<E> tree(int n) {
30         assert(n >= 1);
31         auto p = perm(n);
32         vector<E> e;
33         for (int i = 1; i < n; ++i) e.push_back({p[num<int>(0, i - 1)], p[i]});
34         mix(e);
35         return e;
36     }
37
38     vector<E> graph(int n, int m, bool con = true) {
39         assert(n >= 1);
40         int lim = n * (n - 1) / 2;
41         assert(0 <= m && m <= lim && (!con || m >= n - 1));
42         vector<E> e = con ? tree(n) : vector<E>{};
43         unordered_set<int> st;
44         auto key = [n](int u, int v) {
45             if (u > v) swap(u, v);
46             return u * (n + 1) + v;
47         };
48         for (auto [u, v] : e) st.insert(key(u, v));
49         int nd = m - e.size(), lf = lim - st.size();

```

```

50     if (nd * 2 > lf) {
51         vector<E> a;
52         for (int u = 1; u <= n; ++u) for (int v = u + 1; v <= n; ++v)
53             if (!st.count(key(u, v))) a.push_back({u, v});
54         mix(a);
55         e.insert(e.end(), a.begin(), a.begin() + nd);
56     } else {
57         while (nd) {
58             int u = num<int>(1, n), v = num<int>(1, n);
59             if (u == v) continue;
60             if (u > v) swap(u, v);
61             if (st.insert(key(u, v)).second) e.push_back({u, v}), --nd;
62         }
63     }
64     mix(e);
65     return e;
66 }
67
68 string str(int n, string_view a = "abcdefghijklmnopqrstuvwxyz") {
69     assert(n >= 0 && !a.empty());
70     string s(n, ' ');
71     for (char& c : s) c = a[num<int>(0, a.size() - 1)];
72     return s;
73 }
74 };

```

16 - 单调队列.cpp

```

1  /*
2  用途：滑动窗口最大/最小值。
3  用法：MonoQueue<int> q(k) 默认维护最大值；MonoQueue<int, greater<int>> 维护最小值。
4  接口：push(x) 加入下一个位置；ask() 在窗口已满时返回窗口答案，否则返回 nullopt。
5  复杂度：每个元素至多进出队一次，总 O(n)，空间 O(k)。
6  */
7
8  template <class T, class C = less<T>> struct MonoQueue {
9      int k, p;
10     C cmp;
11     deque<pair<T, int>> q;
12
13     explicit MonoQueue(int k) : k(k), p(-1) {
14         assert(k > 0);
15     }
16
17     void push(const T& x) {
18         ++p;
19         while (!q.empty() && cmp(q.back().first, x)) q.pop_back();
20         q.push_back({x, p});
21         while (q.front().second <= p - k) q.pop_front();
22     }
23
24     optional<T> ask() const {
25         if (p + 1 < k) return nullopt;
26         return q.front().first;
27     }
28 };

```

linyue.hpp

```

1  template <class T1, class T2> ostream &operator<<(ostream &os, const pair<T1, T2> &p) {
2      return os << '{' << p.first << ", " << p.second << '>';
3  }
4
5  template <class T, class = decltype(begin(declval<T>()))>, class = enable_if_t<!is_same_v<T, string>>> ostream
&operator<<(ostream &os, const T &c) {
6      os << '[';
7      for (auto it = c.begin(); it != c.end(); ++it) os << (it == c.begin() ? "" : ", ") << *it;
8      return os << ']';
9  }
10
11 template <class... A> void _(A&&... a) {
12     cerr << "---- ";

```



```
13     ((cerr << forward<A>(a) << ' '), ...);
14     cerr << "\n";
15 }
16
17 inline auto tic() {
18     return chrono::steady_clock::now();
19 }
20
21 template <class T> void toc(T t, const char* s = "") {
22     double x = chrono::duration<double, milli>(chrono::steady_clock::now() - t).count();
23     cerr << "[time] " << s << (s[0] ? ": " : "") << fixed << setprecision(3) << x << " ms\n";
24 }
```

树上问题：静态树、分治与离线

18 个代码模板 · 林月 · v1.0.0 · 2026.8.1

版块说明与知识结论

README.md

树上问题模板

默认约定为 $1..n$ 编号的连通无向树。除专门说明外，均为静态树模板。

需求	模板	主要复杂度
直径、顶点树心、树重心	无权直径用 01；完整树论封装用 01A	$O(n)$
仅 LCA、无权距离，且查询很多	03D (Euler 序 + RMQ)	预处理/空间 $O(n \log n)$ ，查询 $O(1)$
LCA、距离、跳祖先	03A (重链) 或 03B (倍增)；有权/路径最大边权用 03C	预处理 $O(n \log n)$ ，查询 $O(\log n)$
所有 LCA 询问离线已知	03E (Tarjan)	$O((n+q) \alpha(n))$
大量第 k 个祖先	03F (长链剖分)	预处理 $O(n \log n)$ ，跳祖先 $O(1)$
路径/子树区间操作	06 - 树链剖分 (路径与子树操作)	每条路径 $O(\log n)$ 段
多条路径统一加到点或边	07 - 树上差分	预处理/汇总 $O(n)$ ，每次更新 $O(\log n)$
每个点作为根的全树 DP	08 - 换根 DP (全树 DP)	$O(n)$ 次合并
只保留关键点及其 LCA	09 - 虚树	$O(k \log k)$
子树离线统计	04 - 树上启发式合并 (通用框架)；04A 是颜色众数和的成品示例	$O(n \log n)$
距离类分治	02 - 点分治 (重心树骨架)；02A 是“是否存在指定距离”的成品代码块	$O(n \log n)$
动态点集最近距离	02B - 点分树 (动态点集最近距离)	单次 $O(\log^2 n)$
两条简单路径的交集长度	05 - 树上路径交	$O(\log n)$
离线连通阈值、最小化路径最大边	10 - Kruskal 重构树	建树 $O(m \log m)$ ，查询 $O(\log n)$
动态森林的连边、断边与路径聚合	11 - Link-Cut Tree (动态树)	单次操作均摊 $O(\log n)$
无标号树同构	12 - 树同构 (AHU)	$O(n \log n)$
有标号树编码、度数计数	13 - Prüfer 序列	$O(n \log n)$

使用提示

- 边权距离、路径和直接使用默认 `int`；03C 维护边权，而 03A/03B 的 `calc` 是边数。
- 03D 用 DFS 欧拉序上深度最小值 RMQ 求 LCA；只需静态 LCA/边数距离且查询量很大时选它。需要跳祖先或路径最大边权时仍选 03B/03C。
- 03E 只能在所有询问登记后调用 `work`；在线询问不能使用它。03F 的 `askJump` 为 $O(1)$ ，但其 LCA 仍为 $O(\log n)$ ，纯 LCA 高频时优先 03D。
- 用 HLD 做边权操作时，将边权存到较深端点的 `dfn`；`forEachEdgePath` 会排除 LCA。
- 树上差分的结果需要在所有更新后自底向上汇总，模板中的 `collect...` 已处理这一步。
- 虚树输入点可以重复，模板会自动去重并补齐必要 LCA；虚树边仍对应原树节点编号。
- 点分树默认维护激活点集合；用 `add/del/ask` 即可处理经典“动态红点最近距离”问题。

- 递归版模板在链状树且规模很大时应改为显式栈，或按比赛栈限制调整。
- Prüfer 序列中点 v 出现 $\deg(v)-1$ 次；固定度数 d_i 的有标号树数量为 $(n-2)! / \text{product}((d_i-1)!)$ (度数均正且和为 $2n-2$)。

覆盖边界

这里保留竞赛中常用的静态树、离线重构树和动态树基础。LCA 已分别覆盖在线、离线、纯 LCA 高频与跳祖先高频；更专用的动态树分块、Top Tree 等按题目需要另写。

直径/重心、路径公式、虚树与 Prüfer 计数等性质见 [树上问题知识与结论](#)。

树上问题知识与结论.md

树上问题知识与结论

1. 基本性质

- 连通无向图是树当且仅当边数为 $n-1$ 且无环；树上任意两点间简单路径唯一。
- 所有点度数和为 $2(n-1)$ ，叶子数可由度数统计；删去一条边恰分成两个连通块。
- 以根定向后，子树对应 DFS 序连续区间；树上路径可被 HLD 拆成 $O(\log n)$ 个区间。

2. 直径、中心与重心

- 非负权树从任一点走到最远点，再从该点走到最远点，可得到一条直径；负边权时该结论需重新检查定义。
- 所有直径共享树中心：无权树中心是一个点或一条边的两端；树的半径为直径长度向上取半。
- 重心删除后每个连通块大小不超过 $n/2$ ，至多两个且若有两个必相邻。点分治每层经过重心，深度 $O(\log n)$ 。

3. LCA 与路径公式

- $\text{dist}(u,v) = \text{dep}[u] + \text{dep}[v] - 2 * \text{dep}[\text{lca}(u,v)]$ ；有权时把深度换成根距离。
- Euler Tour LCA：DFS 过程中记录每次到达的点，长度为 $2n-1$ ； $\text{lca}(u,v)$ 是 $\text{first}[u]$ 、 $\text{first}[v]$ 之间深度最小的欧拉序节点。用 ST 表做 RMQ 可 $O(n \log n)$ 预处理、 $O(1)$ 查询，对应 03D；代价是 $O(n \log n)$ 空间。
- Tarjan LCA 在 DFS 回溯子树后把子树代表元并到当前点；当另一端点已访问时，该并查集代表元就是 LCA。它是所有询问提前给定时的 $O((n+q)\alpha(n))$ 分支，对应 03E，不能用于在线查询。
- 长链剖分按最长向下链划分；跳过最高二进制位后，剩余步数可在链头向上/向下数组中 $O(1)$ 回答。它优化的是第 k 个祖先，对应 03F；纯 LCA 高频仍优先 Euler + RMQ。
- 路径点数为边数 +1。边权映射到较深端点后，查询边路径必须排除 LCA 对应位置。
- 三点 x,y,z 的中位点是三个两两 LCA 中深度最大的一个；多条路径交可转化为端点与 LCA 的有限候选判断。
- 树上点差分路径加： $d[u]++, d[v]++, d[\text{lca}]-, d[\text{parent}(\text{lca})]-$ ；边差分： $d[u]++, d[v]++, d[\text{lca}]-=2$ 。

4. 离线与重构结构

- DSU on tree 保留重儿子信息、清空轻儿子，适合“以节点为根的子树统计”；修改/撤销必须与 DFS 进入退出完全对称。
- 虚树只保留关键点及相邻关键点的 LCA，点数不超过约 $2k$ ；先按 DFS 序去重再建栈。
- Kruskal 重构树中叶子是原点，内部点权为合并边权；两点路径最大边权等于其重构树 LCA 的权。
- 点分树记录点到各层重心的距离。支持删除时不能只存一个最小值，通常为每个重心维护可删堆或多重集。

5. 动态树与编码

- Link-Cut Tree 的实边/虚边语义与原树方向无关；每次访问子节点前必须下传翻转标记。
- Prüfer 序列长度为 $n-2$ ，点 v 出现 $\deg(v)-1$ 次；Cayley 公式给出有标号树数量 n^{n-2} 。
- 固定正度数 d_i 且和为 $2n-2$ 时，树数为 $(n-2)! / \text{product}((d_i-1)!)$ 。

01 - 树的直径.cpp

```
1  /*
2  用途：无权树直径模板。两次 DFS/遍历即可求直径长度。
3  */
```

```

4  接口:
5  - Tree(n): 建立 1-indexed 的 n 点树。
6  - add(u, v): 加入无向边。
7  - askEnds(root = 1): 返回一组直径端点。
8  - askLen(root = 1): 返回直径长度 (边数)。
9  */
10 struct Tree {
11     int n;
12     vector<vector<int>> ver;
13
14     explicit Tree(int n) : n(n), ver(n + 1) {}
15
16     void add(int u, int v) {
17         ver[u].push_back(v);
18         ver[v].push_back(u);
19     }
20
21     pair<int, vector<int>> far(int src) const {
22         vector<int> dep(n + 1, -1), st = {src};
23         dep[src] = 0;
24         for (int i = 0; i < st.size(); ++i) {
25             int u = st[i];
26             for (int v : ver[u]) {
27                 if (dep[v] != -1) continue;
28                 dep[v] = dep[u] + 1;
29                 st.push_back(v);
30             }
31         }
32
33         int far = src;
34         for (int u = 1; u <= n; ++u) {
35             if (dep[u] > dep[far]) far = u;
36         }
37         return {far, dep};
38     }
39
40     pair<int, int> askEnds(int rt = 1) const {
41         int u = far(rt).first;
42         int v = far(u).first;
43         return {u, v};
44     }
45
46     int askLen(int rt = 1) const {
47         int u = far(rt).first;
48         auto [v, dep] = far(u);
49         return dep[v];
50     }
51 };

```

01A - 树论大封装 (直径+中心+重心) .cpp

```

1  /*
2  用途: 带非负边权树的基础树论封装。
3
4  支持:
5  - 直径端点与直径长度;
6  - 顶点树心 (使到最远顶点距离最小的点) 与半径;
7  - 树重心 (删去该点后最大连通块最小);
8  - 指定根下的 parent / dep / maxdep。
9
10 约定:
11  - 点编号为 1..n, 树连通; 边权必须非负。
12  - center 是“顶点树心”。若题目允许把中心放在边的内部, 需要在直径上另行二分位置。
13
14 接口:
15  - add(u, v, w = 1): 加入无向边。
16  - work(root = 1): 一次性完成全部预处理。
17  - askEnds(): 返回一组直径端点, 同时更新 diam。
18  */
19
20 struct Tree {
21     int n;
22     vector<vector<pair<int, int>>> e;
23
24     // dep 为无权深度; maxdep 为该根树内最大的无权深度。
25     vector<int> dep, fa, mxd;
26     // d1 / d2: 向下第一、第二长距离; up: 经过父边到树外的最长距离。
27     vector<int> d1, d2, up;
28     vector<int> s1, s2;
29     vector<bool> vis;

```

```

30     int rem, cog;
31     int rad, dia;
32     int cen;
33
34     explicit Tree(int n) : n(n), e(n + 1), dep(n + 1), fa(n + 1), mxd(n + 1), d1(n + 1), d2(n + 1), up(n + 1), s1(n
+ 1), s2(n + 1), vis(n + 1) {}
35
36     void add(int u, int v, int w = 1) {
37         e[u].push_back({v, w});
38         e[v].push_back({u, w});
39     }
40
41     void dfs(int u, int _fa) {
42         fa[u] = _fa;
43         mxd[u] = dep[u];
44         for (auto [v, w] : e[u]) {
45             if (v == _fa) continue;
46             dep[v] = dep[u] + 1;
47             dfs(v, u);
48             mxd[u] = max(mxd[u], mxd[v]);
49         }
50     }
51
52     void dfs1(int u, int fa) {
53         for (auto [v, w] : e[u]) {
54             if (v == fa) continue;
55             dfs1(v, u);
56             int cur = d1[v] + w;
57             if (cur > d1[u]) {
58                 d2[u] = d1[u], s2[u] = s1[u];
59                 d1[u] = cur, s1[u] = v;
60             } else if (cur > d2[u]) {
61                 d2[u] = cur, s2[u] = v;
62             }
63         }
64     }
65
66     void dfs2(int u, int fa) {
67         for (auto [v, w] : e[u]) {
68             if (v == fa) continue;
69             up[v] = (s1[u] == v ? max(up[u], d2[u]) : max(up[u], d1[u])) + w;
70             dfs2(v, u);
71         }
72     }
73
74     void calCen() {
75         cen = 1;
76         for (int u = 1; u <= n; ++u) {
77             if (max(d1[u], up[u]) < max(d1[cen], up[cen])) cen = u;
78         }
79         rad = max(d1[cen], up[cen]);
80
81         dia = 0;
82         for (int u = 1; u <= n; ++u) {
83             dia = max(dia, d1[u] + max(d2[u], up[u]));
84         }
85     }
86
87     int dfsCog(int u, int fa) {
88         int siz = 1, mx = 0;
89         for (auto [v, w] : e[u]) {
90             if (v == fa) continue;
91             int sub = dfsCog(v, u);
92             siz += sub;
93             mx = max(mx, sub);
94         }
95         mx = max(mx, n - siz);
96         if (mx < rem) rem = mx, cog = u;
97         return siz;
98     }
99
100    void calCog() {
101        rem = numeric_limits<int>::max();
102        cog = 1;
103        dfsCog(1, 0);
104    }
105
106    pair<int, vector<int>> far(int src) const {
107        vector<int> fa(n + 1, -1), st = {src};
108        vector<int> dis(n + 1);
109        fa[src] = 0;
110        for (int i = 0; i < st.size(); ++i) {

```

```

111         int u = st[i];
112         for (auto [v, w] : e[u]) {
113             if (v == fa[u]) continue;
114             fa[v] = u;
115             dis[v] = dis[u] + w;
116             st.push_back(v);
117         }
118     }
119
120     int far = src;
121     for (int u = 1; u <= n; ++u) {
122         if (dis[u] > dis[far]) far = u;
123     }
124     return {far, dis};
125 }
126
127 pair<int, int> askEnds() {
128     auto [u, ban] = far(1);
129     auto [v, dis] = far(u);
130     dia = dis[v];
131     return {u, v};
132 }
133
134 void work(int rt = 1) {
135     dep.assign(n + 1, 0);
136     fa.assign(n + 1, 0);
137     mxd.assign(n + 1, 0);
138     d1.assign(n + 1, 0);
139     d2.assign(n + 1, 0);
140     up.assign(n + 1, 0);
141     s1.assign(n + 1, 0);
142     s2.assign(n + 1, 0);
143
144     dfs(rt, 0);
145     dfs1(rt, 0);
146     dfs2(rt, 0);
147     calCen();
148     calCog();
149     askEnds();
150 }
151 };

```

02 - 点分治（重心树骨架）.cpp

```

1  /*
2  用途：点分治（重心树）通用骨架。
3
4  适用：树上距离统计、动态点集距离查询、分治时按“经过重心”的路径统计等。
5  build 的回调会在每个重心被标记 removed 后调用：callback(centroid, centroidParent, level)。
6  回调可读取 e、removed、centroidParent、centroidDepth；随后每个未删除连通块会继续分治。
7
8  接口：
9  - add(u, v, w = 1)：加入无向边；w 仅供回调使用，分治本身不依赖边权。
10 - build(callback, root = 1)：建立重心树。
11
12 复杂度：建树  $O(n \log n)$ ，递归深度  $O(\log n)$ ；原树 DFS 仍可能有  $O(n)$  调用深度。
13  */
14
15 struct CenDec {
16
17     struct Edge {
18         int to;
19         int w;
20     };
21
22     int n;
23     vector<vector<Edge>> e;
24     vector<int> siz, cfa, dep;
25     vector<char> ban;
26
27     explicit CenDec(int n) : n(n), e(n + 1), siz(n + 1),
28                             cfa(n + 1), dep(n + 1),
29                             ban(n + 1) {}
30
31     void add(int u, int v, int w = 1) {
32         e[u].push_back({v, w});
33         e[v].push_back({u, w});
34     }
35
36     int dfsSz(int u, int fa) {

```

```

37     siz[u] = 1;
38     for (auto [v, w] : e[u]) {
39         if (v == fa || ban[v]) continue;
40         siz[u] += dfsSz(v, u);
41     }
42     return siz[u];
43 }
44
45 int cen(int u, int fa, int tot) {
46     for (auto [v, w] : e[u]) {
47         if (v == fa || ban[v]) continue;
48         if (siz[v] > tot / 2) return cen(v, u, tot);
49     }
50     return u;
51 }
52
53 template <class F>
54 void divide(int st, int fa, int d, F &f) {
55     int tot = dfsSz(st, 0);
56     int c = cen(st, 0, tot);
57     cfa[c] = fa;
58     dep[c] = d;
59     ban[c] = true;
60     f(c, fa, d);
61
62     for (auto [v, w] : e[c]) {
63         if (!ban[v]) divide(v, c, d + 1, f);
64     }
65 }
66
67 template <class F>
68 void build(F f, int rt = 1) {
69     fill(siz.begin(), siz.end(), 0);
70     fill(cfa.begin(), cfa.end(), 0);
71     fill(dep.begin(), dep.end(), 0);
72     fill(ban.begin(), ban.end(), 0);
73     divide(rt, 0, 0, f);
74 }
75 };

```

02A - 点分治 (距离询问) .cpp

```

1  /*
2  用途: 点分治模板, 常用于统计/判断树上点对距离, 当前写法用于回答是否存在距离为 q[i] 的点对。
3  依赖:
4  - 外部已有 n, m。
5  - ver[u] 存边 {v, w}。
6  - q[1..m] 存询问距离, ans[1..m] 会被置为 1 表示存在。
7  接口/流程:
8  - dfsSz(x, fa): 计算当前未删除连通块大小。
9  - cen(x, fa, total): 在大小为 total 的连通块内找重心 root。
10 - calc(x): 处理所有经过重心 x 的路径贡献。
11 - dfz(root, 0): 从重心 root 开始分治。
12 */
13 int rt = 0, mt = 0;
14 vector<int> vis(n + 1), siz(n + 1);
15
16 auto fsz = [&](auto go, int x, int fa) -> int {
17     siz[x] = 1;
18     for (auto [y, w] : ver[x]) {
19         if (y == fa || vis[y]) continue;
20         siz[x] += go(go, y, x);
21     }
22     return siz[x];
23 };
24
25 auto cen = [&](auto go, int x, int fa, int tot) -> void {
26     siz[x] = 1;
27     int val = 0;
28     for (auto [y, w] : ver[x]) {
29         if (y == fa || vis[y]) continue;
30         go(go, y, x, tot);
31         siz[x] += siz[y];
32         val = max(val, siz[y]);
33     }
34     val = max(val, tot - siz[x]);
35     if (val < mt) {
36         mt = val;
37         rt = x;
38     }

```

```

39 };
40
41 auto cal = [&](int x) -> void {
42     set<int> pre = {0};
43     vector<int> dis(n + 1);
44     for (auto [y, w] : ver[x]) {
45         if (vis[y]) continue;
46         vector<int> son;
47         auto dfs = [&](auto go, int u, int fa) -> void {
48             son.push_back(dis[u]);
49             for (auto [y, w] : ver[u]) {
50                 if (y == fa || vis[y]) continue;
51                 dis[y] = dis[u] + w;
52                 go(go, y, u);
53             }
54         };
55         dis[y] = w;
56         dfs(dfs, y, x);
57
58         for (auto it : son) {
59             for (int i = 1; i <= m; i++) {
60                 if (q[i] < it || !pre.count(q[i] - it)) continue;
61                 ans[i] = 1;
62             }
63         }
64         pre.insert(son.begin(), son.end());
65     }
66 };
67
68 auto dfz = [&](auto go, int x, int fa) -> void {
69     vis[x] = 1;
70     cal(x);
71     for (auto [y, w] : ver[x]) {
72         if (y == fa || vis[y]) continue;
73         int tot = fsz(fsz, y, x);
74         rt = 0;
75         mt = tot;
76         cen(cen, y, x, tot);
77         go(go, rt, x);
78     }
79 };
80
81 int tot = fsz(fsz, 1, 0);
82 rt = 0;
83 mt = tot;
84 cen(cen, 1, 0, tot);
85 dfz(dfz, rt, 0);

```

02B - 点分树（动态点集最近距离）.cpp

```

1  /*
2  用途：点分树（重心树）维护动态点集到任意点的最近距离。
3
4  约定：点编号为 1..n，树连通，边权非负。
5  接口：
6  - add(u, v, w = 1)、build(root = 1)。
7  - add(u) / del(u) / modify(u, on)。重复操作会被忽略并返回 false。
8  - askMin(u)：返回到最近激活点的距离；不存在激活点时返回 nullopt。
9  - ask(u)：askMin 的 int 版本；不存在激活点时返回 -1。
10
11 复杂度：建树  $O(n \log n)$ ，每次激活、撤销、查询均为  $O(\log^2 n)$ 。
12 path[u] 公开保存 u 到每层重心的 {centroid, distance}，可在此基础上扩展计数、距离和等信息。
13 */
14
15 struct CenTree {
16
17     struct Edge {
18         int to;
19         int w;
20     };
21
22     struct PathInfo {
23         int cen;
24         int dis;
25     };
26
27     int n;
28     vector<vector<Edge>> e;
29     vector<int> siz, cfa;
30     vector<char> ban, on;

```



```

31 vector<vector<PathInfo>> pth;
32 vector<multiset<int>> q;
33
34 explicit CenTree(int n)
35 : n(n), e(n + 1), siz(n + 1), cfa(n + 1), ban(n + 1), on(n + 1),
36   pth(n + 1), q(n + 1) {}
37
38 void add(int u, int v, int w = 1) {
39     e[u].push_back({v, w});
40     e[v].push_back({u, w});
41 }
42
43 int dfsSz(int u, int fa) {
44     siz[u] = 1;
45     for (auto [v, w] : e[u]) {
46         if (v == fa || ban[v]) continue;
47         siz[u] += dfsSz(v, u);
48     }
49     return siz[u];
50 }
51
52 int cen(int u, int fa, int tot) {
53     for (auto [v, w] : e[u]) {
54         if (v == fa || ban[v]) continue;
55         if (siz[v] > tot / 2) return cen(v, u, tot);
56     }
57     return u;
58 }
59
60 void dfsDis(int u, int fa, int dis, int cen) {
61     pth[u].push_back({cen, dis});
62     for (auto [v, w] : e[u]) {
63         if (v == fa || ban[v]) continue;
64         dfsDis(v, u, dis + w, cen);
65     }
66 }
67
68 void build(int st, int fa) {
69     int tot = dfsSz(st, 0);
70     int cen = this->cen(st, 0, tot);
71     cfa[cen] = fa;
72     ban[cen] = true;
73     dfsDis(cen, 0, 0, cen);
74
75     for (auto [v, w] : e[cen]) {
76         if (!ban[v]) build(v, cen);
77     }
78 }
79
80 void build(int rt = 1) {
81     fill(siz.begin(), siz.end(), 0);
82     fill(cfa.begin(), cfa.end(), 0);
83     fill(ban.begin(), ban.end(), false);
84     fill(on.begin(), on.end(), false);
85     for (int u = 1; u <= n; ++u) {
86         pth[u].clear();
87         q[u].clear();
88     }
89     build(rt, 0);
90 }
91
92 bool modify(int u, bool _on) {
93     if ((bool)on[u] == _on) return false;
94     on[u] = _on;
95     for (auto [cen, dis] : pth[u]) {
96         if (_on) {
97             q[cen].insert(dis);
98         } else {
99             auto it = q[cen].find(dis);
100             assert(it != q[cen].end());
101             q[cen].erase(it);
102         }
103     }
104     return true;
105 }
106
107 bool add(int u) {
108     return modify(u, true);
109 }
110
111 bool del(int u) {
112     return modify(u, false);

```

```

113     }
114
115     void clear() {
116         fill(on.begin(), on.end(), false);
117         for (int u = 1; u <= n; ++u) q[u].clear();
118     }
119
120     optional<int> askMin(int u) const {
121         optional<int> ans;
122         for (auto [cen, dis] : pth[u]) {
123             if (q[cen].empty()) continue;
124             int cur = dis + *q[cen].begin();
125             if (!ans || cur < *ans) ans = cur;
126         }
127         return ans;
128     }
129
130     int ask(int u) const {
131         auto ans = askMin(u);
132         return ans ? *ans : -1;
133     }
134 };

```

03A - 最近公共祖先 (Tree, 树链剖分) .cpp

```

1  /*
2  用途：树链剖分版 LCA，适合只需要最近公共祖先、无权距离或 DFS 序的场景。
3
4  接口：
5  - Tree(n): 建立 1-indexed 的 n 点树。
6  - add(u, v): 加入无向边。
7  - work(root = 1): 预处理 siz / dep / parent / son / top / dfn / out。
8  - askLCA(u, v): 返回最近公共祖先。
9  - askDis(u, v): 返回 u 到 v 的边数距离。
10 - askAnc(u, v): 判断 u 是否为 v 的祖先 (含 u == v)。
11
12 dfn 为重链优先 DFS 序，但子树仍是连续区间 [dfn[u], out[u]]，可直接用于虚树。
13 */
14 struct Tree {
15     int n, idx;
16     vector<vector<int>> ver;
17     vector<int> siz, dep, top, son, fa, dfn, out, rev;
18
19     explicit Tree(int n) : n(n), idx(0), ver(n + 1), siz(n + 1), dep(n + 1),
20                           top(n + 1), son(n + 1), fa(n + 1), dfn(n + 1),
21                           out(n + 1), rev(n + 1) {}
22
23     void add(int u, int v) {
24         ver[u].push_back(v);
25         ver[v].push_back(u);
26     }
27
28     void dfs1(int u) {
29         siz[u] = 1;
30         for (int v : ver[u]) {
31             if (v == fa[u]) continue;
32             fa[v] = u;
33             dep[v] = dep[u] + 1;
34             dfs1(v);
35             siz[u] += siz[v];
36             if (siz[v] > siz[son[u]]) son[u] = v;
37         }
38     }
39
40     void dfs2(int u, int t) {
41         top[u] = t;
42         dfn[u] = ++idx;
43         rev[idx] = u;
44         if (son[u]) dfs2(son[u], t);
45         for (int v : ver[u]) {
46             if (v == fa[u] || v == son[u]) continue;
47             dfs2(v, v);
48         }
49         out[u] = idx;
50     }
51
52     int askLCA(int u, int v) const {
53         while (top[u] != top[v]) {
54             if (dep[top[u]] > dep[top[v]]) {
55                 u = fa[top[u]];

```

```

56         } else {
57             v = fa[top[v]];
58         }
59     }
60     return dep[u] < dep[v] ? u : v;
61 }
62
63 int askDis(int u, int v) const {
64     return dep[u] + dep[v] - 2 * dep[askLCA(u, v)];
65 }
66
67 bool askAnc(int u, int v) const {
68     return dfn[u] <= dfn[v] && dfn[v] <= out[u];
69 }
70
71 void work(int rt = 1) {
72     idx = 0;
73     fill(siz.begin(), siz.end(), 0);
74     fill(dep.begin(), dep.end(), 0);
75     fill(top.begin(), top.end(), 0);
76     fill(son.begin(), son.end(), 0);
77     fill(fa.begin(), fa.end(), 0);
78     fill(dfn.begin(), dfn.end(), 0);
79     fill(out.begin(), out.end(), 0);
80     fill(rev.begin(), rev.end(), 0);
81     dfs1(rt);
82     dfs2(rt, rt);
83 }
84 };

```

03B - 最近公共祖先 (Tree, 倍增) .cpp

```

1  /*
2  用途: 倍增版 LCA, 适合静态无权树上的最近公共祖先、距离和跳祖先查询。
3
4  接口:
5  - work(root = 1): 预处理 dep / val / tin / tout。
6  - askLCA(u, v): 最近公共祖先。
7  - askDis(u, v): 边数距离。
8  - askJmp(u, k): u 的第 k 个祖先: 越过根时返回 0。
9  - askKth(u, v, k): 路径 u -> v 上第 k 条边到达的点: 越界返回 0。
10
11 tin / tout 可直接配合“虚树”模板。点编号为 1..n。
12 性能: 倍增祖先表默认使用 int: 极限内存不足时可只将祖先编号改为 i32。
13  */
14 struct Tree {
15     int n, lg, tim;
16     vector<vector<int>> ver, val;
17     vector<int> dep, tin, out;
18
19     explicit Tree(int n) : n(n), tim(0), ver(n + 1), dep(n + 1),
20         tin(n + 1), out(n + 1) {
21         lg = 1;
22         while ((1LL << lg) <= max<int>(1, n)) ++lg;
23         val.assign(n + 1, vector<int>(lg));
24     }
25
26     void add(int u, int v) {
27         ver[u].push_back(v);
28         ver[v].push_back(u);
29     }
30
31     void dfs(int u, int fa) {
32         val[u][0] = fa;
33         for (int k = 1; k < lg; ++k) {
34             val[u][k] = val[val[u][k - 1]][k - 1];
35         }
36         tin[u] = ++tim;
37         for (int v : ver[u]) {
38             if (v == fa) continue;
39             dep[v] = dep[u] + 1;
40             dfs(v, u);
41         }
42         out[u] = tim;
43     }
44
45     int askJmp(int u, int k) const {
46         if (k < 0 || k > dep[u]) return 0;
47         for (int bit = 0; bit < lg; ++bit) {
48             if ((k >> bit) & 1) u = val[u][bit];

```

```

49     }
50     return u;
51 }
52
53 int askLCA(int u, int v) const {
54     if (dep[u] < dep[v]) swap(u, v);
55     u = askJmp(u, dep[u] - dep[v]);
56     if (u == v) return u;
57     for (int k = lg - 1; k >= 0; --k) {
58         if (val[u][k] == val[v][k]) continue;
59         u = val[u][k];
60         v = val[v][k];
61     }
62     return val[u][0];
63 }
64
65 int askDis(int u, int v) const {
66     return dep[u] + dep[v] - 2 * dep[askLCA(u, v)];
67 }
68
69 int askKth(int u, int v, int k) const {
70     int p = askLCA(u, v);
71     int l = dep[u] - dep[p], r = dep[v] - dep[p];
72     if (k < 0 || k > l + r) return 0;
73     return k <= l ? askJmp(u, k) : askJmp(v, l + r - k);
74 }
75
76 bool askAnc(int u, int v) const {
77     return tin[u] <= tin[v] && out[v] <= out[u];
78 }
79
80 void work(int rt = 1) {
81     tim = 0;
82     fill(dep.begin(), dep.end(), 0);
83     fill(tin.begin(), tin.end(), 0);
84     fill(out.begin(), out.end(), 0);
85     for (auto &row : val) fill(row.begin(), row.end(), 0);
86     dfs(rt, 0);
87 }
88 };

```

03C - 最近公共祖先 (Tree, 倍增, 有权图) .cpp

```

1  /*
2  用途: 倍增版有权树 LCA, 支持路径距离、路径最大边权和跳祖先查询。
3
4  接口:
5  - add(u, v, w): 加入权值为 w 的无向边 (w 可为负)。
6  - work(root = 1): 预处理 dep / dis / val / mx / tin / out。
7  - askLCA(u, v)、askDis(u, v)、askMax(u, v)、askJmp(u, k)、askKth(u, v, k)。
8
9  askDis 返回路径权值和; askMax 返回路径最大边权。对空路径 (u == v), askMax 返回 0。
10 性能: 倍增祖先表默认使用 int; 极限内存不足时可只将祖先编号改为 i32, 边权与距离保持 int。
11  */
12
13 struct Tree {
14     static constexpr int NI = numeric_limits<int>::lowest() / 4;
15
16     int n, lg, tim;
17     vector<vector<int>> val;
18     vector<vector<int>> mx;
19     vector<vector<pair<int, int>>> ver;
20     vector<int> dep, tin, out;
21     vector<int> dis;
22
23     explicit Tree(int n) : n(n), tim(0), ver(n + 1), dep(n + 1), tin(n + 1),
24         out(n + 1), dis(n + 1) {
25         lg = 1;
26         while ((1LL << lg) <= max<int>(1, n)) ++lg;
27         val.assign(n + 1, vector<int>(lg));
28         mx.assign(n + 1, vector<int>(lg, NI));
29     }
30
31     void add(int u, int v, int w) {
32         ver[u].push_back({v, w});
33         ver[v].push_back({u, w});
34     }
35
36     void dfs(int u, int fa) {
37         val[u][0] = fa;

```

```

38     for (int k = 1; k < lg; ++k) {
39         val[u][k] = val[val[u][k - 1]][k - 1];
40         mx[u][k] = max(mx[u][k - 1], mx[val[u][k - 1]][k - 1]);
41     }
42     tin[u] = ++tim;
43     for (auto [v, w] : ver[u]) {
44         if (v == fa) continue;
45         dep[v] = dep[u] + 1;
46         dis[v] = dis[u] + w;
47         mx[v][0] = w;
48         dfs(v, u);
49     }
50     out[u] = tim;
51 }
52
53 int askJmp(int u, int k) const {
54     if (k < 0 || k > dep[u]) return 0;
55     for (int bit = 0; bit < lg; ++bit) {
56         if ((k >> bit) & 1) u = val[u][bit];
57     }
58     return u;
59 }
60
61 int askLCA(int u, int v) const {
62     if (dep[u] < dep[v]) swap(u, v);
63     u = askJmp(u, dep[u] - dep[v]);
64     if (u == v) return u;
65     for (int k = lg - 1; k >= 0; --k) {
66         if (val[u][k] == val[v][k]) continue;
67         u = val[u][k];
68         v = val[v][k];
69     }
70     return val[u][0];
71 }
72
73 int askDis(int u, int v) const {
74     int p = askLCA(u, v);
75     return dis[u] + dis[v] - 2 * dis[p];
76 }
77
78 int askMax(int u, int v) const {
79     if (u == v) return 0;
80     int ans = NI;
81     auto up = [&](int x, int t) {
82         for (int k = lg - 1; k >= 0; --k) {
83             if (dep[x] - (1LL << k) >= dep[t]) {
84                 ans = max(ans, mx[x][k]);
85                 x = val[x][k];
86             }
87         }
88     };
89
90     int p = askLCA(u, v);
91     up(u, p);
92     up(v, p);
93     return ans;
94 }
95
96 int askKth(int u, int v, int k) const {
97     int p = askLCA(u, v);
98     int l = dep[u] - dep[p], r = dep[v] - dep[p];
99     if (k < 0 || k > l + r) return 0;
100    return k <= l ? askJmp(u, k) : askJmp(v, l + r - k);
101 }
102
103 bool askAnc(int u, int v) const {
104     return tin[u] <= tin[v] && out[v] <= out[u];
105 }
106
107 void work(int rt = 1) {
108     tim = 0;
109     fill(dep.begin(), dep.end(), 0);
110     fill(tin.begin(), tin.end(), 0);
111     fill(out.begin(), out.end(), 0);
112     fill(dis.begin(), dis.end(), 0);
113     for (auto &row : val) fill(row.begin(), row.end(), 0);
114     for (auto &row : mx) fill(row.begin(), row.end(), NI);
115     dfs(rt, 0);
116 }
117 };

```

03D - 最近公共祖先 (Tree, Euler, RMQ) .cpp

```

1  /*
2  用途：欧拉序 + ST 表版 LCA。
3  适合静态树上大量 LCA 查询；单次 LCA 为  $O(1)$ 。
4
5  接口：
6  - Tree(n): 建立 1-indexed 的 n 点树。
7  - add(u, v): 加入无向边。
8  - work(root = 1): 预处理 dep / parent / dfn / out, 以及  $O(1)$  LCA 查询表。
9  - askLCA(u, v): 返回最近公共祖先,  $O(1)$ 。
10 - askDis(u, v): 返回 u 到 v 的边数距离。
11 - askAnc(u, v): 判断 u 是否为 v 的祖先 (含  $u == v$ )。
12
13 dfn 为普通 DFS 先序, 子树连续为 [dfn[u], out[u]], 可用于虚树。
14 预处理  $O(n \log n)$ , 空间  $O(n \log n)$ 。
15 注意: 默认 int 为 long long 时 ST 表内存翻倍; 内存卡紧时可将下标表改为 i32。
16 */
17 struct Tree {
18     int n, idx;
19     vector<vector<int>> ver;
20
21     vector<int> dep, fa, dfn, out, rev;
22     vector<int> first, euler, lg;
23     vector<vector<int>> st;
24
25     explicit Tree(int n) : n(n), idx(0), ver(n + 1), dep(n + 1), fa(n + 1), dfn(n + 1), out(n + 1), rev(n + 1),
26 first(n + 1, -1) {}
27
28     void add(int u, int v) {
29         ver[u].push_back(v);
30         ver[v].push_back(u);
31     }
32
33     int better(int x, int y) const {
34         return dep[euler[x]] < dep[euler[y]] ? x : y;
35     }
36
37     void dfs(int u, int f) {
38         fa[u] = f;
39         dfn[u] = ++idx;
40         rev[idx] = u;
41
42         first[u] = euler.size();
43         euler.push_back(u);
44
45         for (int v : ver[u]) {
46             if (v == f) continue;
47             dep[v] = dep[u] + 1;
48             dfs(v, u);
49             euler.push_back(u);
50         }
51
52         out[u] = idx;
53     }
54
55     void work(int rt = 1) {
56         idx = 0;
57         fill(dep.begin(), dep.end(), 0);
58         fill(fa.begin(), fa.end(), 0);
59         fill(dfn.begin(), dfn.end(), 0);
60         fill(out.begin(), out.end(), 0);
61         fill(rev.begin(), rev.end(), 0);
62         fill(first.begin(), first.end(), -1);
63
64         euler.clear();
65         euler.reserve(2 * n - 1);
66         dfs(rt, 0);
67
68         int m = euler.size();
69         lg.assign(m + 1, 0);
70         for (int i = 2; i <= m; ++i) lg[i] = lg[i >> 1] + 1;
71
72         int K = lg[m] + 1;
73         st.assign(K, vector<int>(m));
74         iota(st[0].begin(), st[0].end(), 0);
75
76         for (int k = 1; k < K; ++k) {
77             for (int i = 0; i + (1 << k) <= m; ++i) {
78                 st[k][i] = better(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
79             }
80         }
81     }
82 }

```

```

79     }
80 }
81
82 int askLCA(int u, int v) const {
83     int l = first[u], r = first[v];
84     if (l > r) swap(l, r);
85
86     int k = lg[r - l + 1];
87     int x = st[k][l];
88     int y = st[k][r - (1 << k) + 1];
89     return euler[better(x, y)];
90 }
91
92 int askDis(int u, int v) const {
93     return dep[u] + dep[v] - 2 * dep[askLCA(u, v)];
94 }
95
96 bool askAnc(int u, int v) const {
97     return dfn[u] <= dfn[v] && dfn[v] <= out[u];
98 }
99 };

```

03E - 最近公共祖先 (Tarjan, 离线) .cpp

```

1  /*
2  用途: Tarjan 离线 LCA。适合所有询问已知、只需一次性回答的静态树。
3
4  接口:
5  - Tree(n): 建立 1-indexed 的 n 点树。
6  - add(u, v): 加入无向边。
7  - ask(u, v): 登记一个询问, 返回其编号。
8  - work(root = 1): 返回按登记顺序排列的 LCA。
9
10 复杂度:  $O((n + q) \alpha(n))$  时间,  $O(n + q)$  空间。
11 不支持在线新增询问后立即作答; 递归版在极深链上需留意栈限制。
12  */
13
14 struct Tree {
15     int n;
16     vector<vector<int>> ver;
17     vector<vector<pii>> qry;
18     vector<int> fa, ans;
19     vector<char> vis;
20
21     explicit Tree(int n) : n(n), ver(n + 1), qry(n + 1), fa(n + 1), vis(n + 1) {}
22
23     void add(int u, int v) {
24         ver[u].push_back(v);
25         ver[v].push_back(u);
26     }
27
28     int ask(int u, int v) {
29         int id = ans.size();
30         ans.push_back(0);
31         qry[u].push_back({v, id});
32         if (u != v) qry[v].push_back({u, id});
33         return id;
34     }
35
36     int find(int u) {
37         return fa[u] == u ? u : fa[u] = find(fa[u]);
38     }
39
40     void dfs(int u, int f) {
41         fa[u] = u;
42         for (int v : ver[u]) {
43             if (v == f) continue;
44             dfs(v, u);
45             fa[find(v)] = u;
46         }
47         vis[u] = true;
48         for (auto [v, id] : qry[u]) {

```

```

49         if (vis[v]) ans[id] = find(v);
50     }
51 }
52
53 vector<int> work(int rt = 1) {
54     iota(fa.begin(), fa.end(), 0);
55     fill(vis.begin(), vis.end(), false);
56     fill(ans.begin(), ans.end(), 0);
57     dfs(rt, 0);
58     return ans;
59 }
60 };

```

03F - 长链剖分 (Tree, $O(1)$ 跳祖先) .cpp

```

1  /*
2  用途：长链剖分。适合静态树上大量第  $k$  个祖先查询，askJmp 为  $O(1)$ 。
3
4  接口：
5  - Tree(n)、add(u,v)、work(root = 1)。
6  - askJmp(u,k):  $u$  的第  $k$  个祖先；越过根返回 0。
7  - askLCA(u,v)、askDis(u,v): 额外保留  $O(\log n)$  LCA 与距离。
8
9  复杂度：预处理  $O(n \log n)$ , askJmp  $O(1)$ , askLCA  $O(\log n)$ 。
10 与 03D 的  $O(1)$  LCA 不同：本模板优化的是跳祖先；只查 LCA 时优先 03D。
11 */
12
13 struct Tree {
14     int n, lg;
15     vector<vector<int>> ver, val, up, dn;
16     vector<int> dep, fa, son, len, top;
17
18     explicit Tree(int n) : n(n), ver(n + 1), dep(n + 1), fa(n + 1), son(n + 1),
19                             len(n + 1), top(n + 1) {
20         lg = 1;
21         while ((1LL << lg) <= max<int>(1, n)) ++lg;
22         val.assign(n + 1, vector<int>(lg));
23     }
24
25     void add(int u, int v) {
26         ver[u].push_back(v);
27         ver[v].push_back(u);
28     }
29
30     void dfs1(int u, int f) {
31         fa[u] = f;
32         val[u][0] = f;
33         for (int k = 1; k < lg; ++k) val[u][k] = val[val[u][k - 1]][k - 1];
34         len[u] = 1;
35         for (int v : ver[u]) {
36             if (v == f) continue;
37             dep[v] = dep[u] + 1;
38             dfs1(v, u);
39             if (len[v] > len[son[u]]) son[u] = v;
40         }
41         len[u] = len[son[u]] + 1;
42     }
43
44     void dfs2(int u, int t) {
45         top[u] = t;
46         if (u == t) {
47             up[u].assign(len[u], 0);
48             dn[u].assign(len[u], 0);
49             up[u][0] = u;
50             for (int i = 1; i < len[u]; ++i) up[u][i] = fa[up[u][i - 1]];
51         }
52         dn[t][dep[u] - dep[t]] = u;
53         if (son[u]) dfs2(son[u], t);
54         for (int v : ver[u]) {
55             if (v == fa[u] || v == son[u]) continue;
56             dfs2(v, v);
57         }
58     }
59
60     int askJmp(int u, int k) const {
61         if (k < 0 || k > dep[u]) return 0;
62         if (!k) return u;
63         int b = 63 - __builtin_clzll(k);
64         u = val[u][b];
65         k -= 1LL << b;

```



```

66     if (!k) return u;
67     int t = top[u], d = dep[u] - dep[t];
68     return k <= d ? dn[t][d - k] : up[t][k - d];
69 }
70
71 int askLCA(int u, int v) const {
72     if (dep[u] < dep[v]) swap(u, v);
73     u = askJmp(u, dep[u] - dep[v]);
74     if (u == v) return u;
75     for (int k = lg - 1; k >= 0; --k) {
76         if (val[u][k] == val[v][k]) continue;
77         u = val[u][k];
78         v = val[v][k];
79     }
80     return val[u][0];
81 }
82
83 int askDis(int u, int v) const {
84     return dep[u] + dep[v] - 2 * dep[askLCA(u, v)];
85 }
86
87 void work(int rt = 1) {
88     fill(dep.begin(), dep.end(), 0);
89     fill(fa.begin(), fa.end(), 0);
90     fill(son.begin(), son.end(), 0);
91     fill(len.begin(), len.end(), 0);
92     fill(top.begin(), top.end(), 0);
93     for (auto& x : val) fill(x.begin(), x.end(), 0);
94     up.assign(n + 1, {});
95     dn.assign(n + 1, {});
96     dfs1(rt, 0);
97     dfs2(rt, rt);
98 }
99 };

```

04 - 树上启发式合并（通用框架）.cpp

```

1  /*
2  用途: DSU on Tree / Sack 通用框架。
3
4  适用: 对每个子树统计颜色、值域、出现次数等可“加入/删除一个点”的信息。
5
6  接口:
7  - add(u, v): 加入无向边。
8  - work(addVertex, answer, root = 1):
9    * addVertex(vertex, delta): delta 为 +1 或 -1, 维护当前数据结构;
10   * answer(u): 此时当前数据结构恰为 u 的子树, 记录答案。
11
12  内部维护 tin / tout / euler / siz / heavy, 子树区间为 [tin[u], tout[u]]。
13  复杂度: 每个点至多被加入/删除  $O(\log n)$  次, 总计  $O(n \log n)$  次回调。
14  */
15 struct DSUOnTree {
16     int n, tim;
17     vector<vector<int>> e;
18     vector<int> fa, siz, son, tin, out, ord;
19
20     explicit DSUOnTree(int n) : n(n), tim(0), e(n + 1), fa(n + 1), siz(n + 1),
21                               son(n + 1), tin(n + 1), out(n + 1), ord(n + 1) {}
22
23     void add(int u, int v) {
24         e[u].push_back(v);
25         e[v].push_back(u);
26     }
27
28     void dfsSz(int u, int _fa) {
29         fa[u] = _fa;
30         siz[u] = 1;
31         tin[u] = ++tim;
32         ord[tim] = u;
33         for (int v : e[u]) {
34             if (v == _fa) continue;
35             dfsSz(v, u);
36             siz[u] += siz[v];
37             if (siz[v] > siz[son[u]]) son[u] = v;
38         }
39         out[u] = tim;
40     }
41
42     template <class Add>
43     void addSub(int u, int d, Add &add) const {

```

```

44     for (int i = tin[u]; i <= out[u]; ++i) add(ord[i], d);
45 }
46
47 template <class A, class F>
48 void solve(int u, bool kp, A &add, F &ans) {
49     for (int v : e[u]) {
50         if (v == fa[u] || v == son[u]) continue;
51         solve(v, false, add, ans);
52     }
53     if (son[u]) solve(son[u], true, add, ans);
54
55     for (int v : e[u]) {
56         if (v == fa[u] || v == son[u]) continue;
57         addSub(v, +1, add);
58     }
59     add(u, +1);
60     ans(u);
61
62     if (!kp) addSub(u, -1, add);
63 }
64
65 template <class A, class F>
66 void work(A add, F ans, int rt = 1) {
67     tim = 0;
68     fill(fa.begin(), fa.end(), 0);
69     fill(siz.begin(), siz.end(), 0);
70     fill(son.begin(), son.end(), 0);
71     fill(tin.begin(), tin.end(), 0);
72     fill(out.begin(), out.end(), 0);
73     dfsSz(rt, 0);
74     solve(rt, true, add, ans);
75 }
76 };

```

04A - 树上启发式合并 (众数颜色和) .cpp

```

1  /*
2  用途: 树上启发式合并 (DSU on Tree), 当前模板用于统计每个子树中出现次数最多的颜色编号之和。
3  约定:
4  - 构造时传入 1-indexed 颜色数组 `c`; 若颜色值可能大于 n, 需要先离散化或扩大 cnt。
5  接口:
6  - DSUOnTree(n, c): 建立 1-indexed 的 n 点树辅助结构并设置颜色; 也可随后调用 `setCol`。
7  - add(u, v): 加入无向边。
8  - dfs1(root, 0): 预处理子树大小和重儿子。
9  - dfs2(root, 0, 0): 计算 ans[u]。
10 - work(root = 1): 一次性完成 dfs1 和 dfs2。
11 - ans[u]: 子树 u 中最高出现次数的颜色编号之和。
12 */
13
14 struct DSUOnTree {
15     vector<vector<int>> e;
16     vector<int> siz, son, cnt, col;
17     vector<int> ans;
18     int sum, mx;
19     int hs;
20     DSUOnTree(int n, const vector<int>& c = {}) {
21         e.resize(n + 1);
22         siz.resize(n + 1);
23         son.resize(n + 1);
24         ans.resize(n + 1);
25         cnt.resize(n + 1);
26         col.assign(n + 1, 0);
27         if (!c.empty()) {
28             assert(c.size() == n + 1);
29             col = c;
30             for (int i = 1; i <= n; ++i) assert(0 <= col[i] && col[i] <= n);
31         }
32         hs = 0;
33         sum = 0;
34         mx = 0;
35     }
36     void setCol(const vector<int>& c) {
37         assert(c.size() == col.size());
38         col = c;
39         for (int i = 1; i < col.size(); ++i) assert(0 <= col[i] && col[i] < cnt.size());
40     }
41     void add(int u, int v) {
42         e[u].push_back(v);
43         e[v].push_back(u);
44     }

```

```

45 void dfs1(int u, int fa) {
46     siz[u] = 1;
47     for (auto v : e[u]) {
48         if (v == fa) continue;
49         dfs1(v, u);
50         siz[u] += siz[v];
51         if (siz[v] > siz[son[u]]) son[u] = v;
52     }
53 }
54 void calc(int u, int fa, int val) {
55     cnt[col[u]] += val;
56     if (cnt[col[u]] > mx) {
57         mx = cnt[col[u]];
58         sum = col[u];
59     } else if (cnt[col[u]] == mx) {
60         sum += col[u];
61     }
62     for (auto v : e[u]) {
63         if (v == fa || v == hs) continue;
64         calc(v, u, val);
65     }
66 }
67 void dfs2(int u, int fa, int opt) {
68     for (auto v : e[u]) {
69         if (v == fa || v == son[u]) continue;
70         dfs2(v, u, 0);
71     }
72     if (son[u]) {
73         dfs2(son[u], u, 1);
74         hs = son[u];
75     }
76     calc(u, fa, 1);
77     hs = 0;
78     ans[u] = sum;
79     if (!opt) {
80         calc(u, fa, -1);
81         sum = 0;
82         mx = 0;
83     }
84 }
85 void work(int rt = 1) {
86     dfs1(rt, 0);
87     dfs2(rt, 0, 0);
88 }
89 };

```

05 - 树上路径交.cpp

```

1  /*
2  用途：求两条树上简单路径的公共点数。
3
4  依赖：
5  - askLCA(u, v): 返回最近公共祖先。
6  - askDis(u, v): 返回 u 到 v 的边数距离。
7
8  接口：ask(a, b, c, d) 返回路径 a-b 与 c-d 的公共点个数，
9  不相交时返回 0。时间复杂度  $O(\log n)$ ，常数很小。
10 */
11 bool askOn(int a, int b, int x) {
12     return askDis(a, x) + askDis(x, b) == askDis(a, b);
13 }
14
15 int ask(int a, int b, int c, int d) {
16     vector<int> pnt = {a, b, c, d};
17     for (int i = 0; i < 4; ++i) {
18         for (int j = i + 1; j < 4; ++j) {
19             pnt.push_back(askLCA(pnt[i], pnt[j]));
20         }
21     }
22     sort(pnt.begin(), pnt.end());
23     pnt.erase(unique(pnt.begin(), pnt.end()), pnt.end());
24
25     vector<int> g;
26     for (int x : pnt) {

```

```

27         if (askOn(a, b, x) && askOn(c, d, x)) g.push_back(x);
28     }
29     if (g.empty()) return 0;
30
31     int ans = 1;
32     for (int x : g) {
33         for (int y : g) {
34             ans = max(ans, askDis(x, y) + 1);
35         }
36     }
37     return ans;
38 }

```

06 - 树链剖分（路径与子树操作）.cpp

```

1  /*
2  用途：树链剖分，把路径/子树转为 DFS 序区间。
3  约定：节点、DFS 序和重儿子数组使用 int；区间回调仍可接收默认 int。
4  性能：默认使用展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
5  */
6  struct HLD {
7
8      int n, tm = 0;
9      vector<vector<int>> g;
10     vector<int> fa, dep, sz, son, top, in, out, rid;
11
12     explicit HLD(int n)
13         : n(n), g(n + 1), fa(n + 1), dep(n + 1), sz(n + 1), son(n + 1), top(n + 1), in(n + 1),
14           out(n + 1), rid(n + 1) {
15         assert(n >= 0);
16     }
17
18     void add(int u, int v) {
19         g[u].push_back(v);
20         g[v].push_back(u);
21     }
22
23     void dfs1(int u, int f) {
24         fa[u] = f;
25         sz[u] = 1;
26         for (int v : g[u]) {
27             if (v == f) continue;
28             dep[v] = dep[u] + 1;
29             dfs1(v, u);
30             sz[u] += sz[v];
31             if (sz[v] > sz[son[u]]) son[u] = v;
32         }
33     }
34
35     void dfs2(int u, int h) {
36         top[u] = h;
37         in[u] = ++tm;
38         rid[tm] = u;
39         if (son[u]) dfs2(son[u], h);
40         for (int v : g[u]) {
41             if (v == fa[u] || v == son[u]) continue;
42             dfs2(v, v);
43         }
44         out[u] = tm;
45     }
46
47     void work(int rt = 1) {
48         assert(1 <= rt && rt <= n);
49         tm = 0;
50         fill(fa.begin(), fa.end(), 0);
51         fill(dep.begin(), dep.end(), 0);
52         fill(sz.begin(), sz.end(), 0);
53         fill(son.begin(), son.end(), 0);
54         fill(top.begin(), top.end(), 0);
55         fill(in.begin(), in.end(), 0);
56         fill(out.begin(), out.end(), 0);
57         fill(rid.begin(), rid.end(), 0);
58         dfs1(rt, 0);
59         dfs2(rt, rt);
60     }
61
62     int askLCA(int u, int v) const {
63         while (top[u] != top[v]) {
64             if (dep[top[u]] > dep[top[v]]) u = fa[top[u]];
65             else v = fa[top[v]];

```

```

66     }
67     return dep[u] < dep[v] ? u : v;
68 }
69
70 int askDis(int u, int v) const {
71     int p = askLCA(u, v);
72     return dep[u] + dep[v] - 2 * dep[p];
73 }
74
75 pair<int, int> askSub(int u) const {
76     return {in[u], out[u]};
77 }
78
79 template <class F> void pathV(int u, int v, F&& f) const {
80     while (top[u] != top[v]) {
81         if (dep[top[u]] < dep[top[v]]) swap(u, v);
82         f(in[top[u]], in[u]);
83         u = fa[top[u]];
84     }
85     if (dep[u] > dep[v]) swap(u, v);
86     f(in[u], in[v]);
87 }
88
89 template <class F> void pathE(int u, int v, F&& f) const {
90     while (top[u] != top[v]) {
91         if (dep[top[u]] < dep[top[v]]) swap(u, v);
92         f(in[top[u]], in[u]);
93         u = fa[top[u]];
94     }
95     if (dep[u] > dep[v]) swap(u, v);
96     if (u != v) f(in[u] + 1, in[v]);
97 }
98 };

```

07 - 树上差分.cpp

```

1  /*
2  用途：多次树上路径加法后的离线汇总，支持“加到路径点”与“加到路径边”。
3
4  接口：
5  - add(u, v)：加入无向边；work(root = 1) 预处理 LCA 和自顶向下 order。
6  - updateV(u, v, x)：给路径上所有点加 x。
7  - updateE(u, v, x)：给路径上所有边加 x。
8  - askV()：返回每个点的最终值。
9  - askE()：返回每个点到父亲边的最终值，root 位置恒为 0。
10
11 T 需要支持默认构造、+= 和 -=，通常直接使用 TreeDiff<int>。
12 所有路径更新为 O(log n)，两种 collect 均为 O(n)，且不会清空已记录的更新。
13 性能：倍增祖先表默认使用 int；极限内存不足时可只将祖先编号改为 i32。
14 */
15
16 template <class T = int>
17 struct TreeDiff {
18     int n, lg;
19     vector<vector<int>> e, up;
20     vector<int> fa, dep, ord;
21     vector<T> tv, te;
22     int rt0 = 1;
23
24     explicit TreeDiff(int n)
25         : n(n), e(n + 1), fa(n + 1), dep(n + 1), tv(n + 1), te(n + 1) {
26         lg = 1;
27         while ((1LL << lg) <= max<int>(1, n)) ++lg;
28         up.assign(lg, vector<int>(n + 1));
29     }
30
31     void add(int u, int v) {
32         e[u].push_back(v);
33         e[v].push_back(u);
34     }
35
36     void dfs(int u, int _fa) {
37         fa[u] = _fa;
38         up[0][u] = _fa;
39         for (int bit = 1; bit < lg; ++bit) up[bit][u] = up[bit - 1][up[bit - 1][u]];
40         ord.push_back(u);
41         for (int v : e[u]) {
42             if (v == _fa) continue;
43             dep[v] = dep[u] + 1;
44             dfs(v, u);

```

```

45     }
46 }
47
48 void work(int rt = 1) {
49     rt0 = rt;
50     fill(fa.begin(), fa.end(), 0);
51     fill(dep.begin(), dep.end(), 0);
52     for (auto &row : up) fill(row.begin(), row.end(), 0);
53     ord.clear();
54     dfs(rt0, 0);
55 }
56
57 int askJmp(int u, int k) const {
58     if (k < 0 || k > dep[u]) return 0;
59     for (int bit = 0; bit < lg; ++bit) {
60         if ((k >> bit) & 1) u = up[bit][u];
61     }
62     return u;
63 }
64
65 int askLCA(int u, int v) const {
66     if (dep[u] < dep[v]) swap(u, v);
67     u = askJmp(u, dep[u] - dep[v]);
68     if (u == v) return u;
69     for (int bit = lg - 1; bit >= 0; --bit) {
70         if (up[bit][u] == up[bit][v]) continue;
71         u = up[bit][u];
72         v = up[bit][v];
73     }
74     return fa[u];
75 }
76
77 void clear() {
78     fill(tv.begin(), tv.end(), T{});
79     fill(te.begin(), te.end(), T{});
80 }
81
82 void updateV(int u, int v, const T &d) {
83     int p = askLCA(u, v);
84     tv[u] += d;
85     tv[v] += d;
86     tv[p] -= d;
87     if (fa[p]) tv[fa[p]] -= d;
88 }
89
90 void updateE(int u, int v, const T &d) {
91     int p = askLCA(u, v);
92     te[u] += d;
93     te[v] += d;
94     te[p] -= d;
95     te[p] -= d;
96 }
97
98 vector<T> collect(vector<T> val) const {
99     for (int i = ord.size(); i--;) {
100         int u = ord[i];
101         if (fa[u]) val[fa[u]] += val[u];
102     }
103     return val;
104 }
105
106 vector<T> askV() const {
107     return collect(tv);
108 }
109
110 vector<T> askE() const {
111     vector<T> res = collect(te);
112     res[rt0] = T{};
113     return res;
114 }
115 };

```

08 - 换根 DP (全树 DP) .cpp

```

1  /*
2  用途: 通用换根 DP (Rerooting), 在一次  $O(n)$  遍历中计算每个点作为根时的全树答案。
3
4  DP 约定:
5  - base[u] 是节点 u 自身的初始贡献;
6  - merge(a, b) 合并同一节点收到的两份贡献, 必须满足结合律, identity 为其单位元;

```

```

7 - transfer(value, from, to, edgeId) 将以 from 为根的一侧贡献跨边传给 to。
8
9 例: 全树距离和可令 T = pair<int, int> = {点数, 距离和},
10 base[u] = {1, 0}, merge 为逐项相加, transfer({cnt, sum}) = {cnt, sum + cnt * w[edgeId]}。
11
12 接口:
13 - Rerooting(n, identity, merge, transfer), add(u, v) 返回边编号。
14 - 设置 base[u] 后调用 work(root = 1), 答案写入 answer[u]。
15 */
16 template <class T>
17 struct Rerooting {
18     struct Edge {
19         int to, id;
20     };
21     using Merge = function<T(const T &, const T &)>;
22     using Transfer = function<T(const T &, int, int, int)>;
23
24     int n, m = 0;
25     T id;
26     Merge mg;
27     Transfer trn;
28     vector<vector<Edge>> e;
29     vector<int> fa, pe, ord;
30     vector<T> bas, dn, up, ans;
31
32     Rerooting(int n, T id, Merge mg, Transfer trn)
33         : n(n), id(id), mg(move(mg)), trn(move(trn)), e(n + 1),
34           fa(n + 1), pe(n + 1), bas(n + 1, id), dn(n + 1, id),
35           up(n + 1, id), ans(n + 1, id) {}
36
37     int add(int u, int v) {
38         int id = m++;
39         e[u].push_back({v, id});
40         e[v].push_back({u, id});
41         return id;
42     }
43
44     void work(int rt = 1) {
45         fill(fa.begin(), fa.end(), 0);
46         fill(pe.begin(), pe.end(), -1);
47         ord.clear();
48         ord.push_back(rt);
49         for (int i = 0; i < ord.size(); ++i) {
50             int u = ord[i];
51             for (auto [v, id] : e[u]) {
52                 if (v == fa[u]) continue;
53                 fa[v] = u;
54                 pe[v] = id;
55                 ord.push_back(v);
56             }
57         }
58
59         fill(dn.begin(), dn.end(), id);
60         for (int i = ord.size(); i--;) {
61             int u = ord[i];
62             dn[u] = bas[u];
63             for (auto [v, id] : e[u]) {
64                 if (fa[v] != u) continue;
65                 dn[u] = mg(dn[u], trn(dn[v], v, u, id));
66             }
67         }
68
69         fill(up.begin(), up.end(), id);
70         for (int u : ord) {
71             int deg = e[u].size();
72             vector<T> get(deg), pre(deg + 1, id), suf(deg + 1, id);
73             for (int i = 0; i < deg; ++i) {
74                 auto [v, id] = e[u][i];
75                 get[i] = (v == fa[u]) ? up[u] : trn(dn[v], v, u, id);
76                 pre[i + 1] = mg(pre[i], get[i]);
77

```

```

78         for (int i = deg - 1; i >= 0; --i) suf[i] = mg(get[i], suf[i + 1]);
79         ans[u] = mg(bas[u], pre[deg]);
80
81         for (int i = 0; i < deg; ++i) {
82             auto [v, id] = e[u][i];
83             if (fa[v] != u) continue;
84             T oth = mg(bas[u], mg(pre[i], suf[i + 1]));
85             up[v] = trn(oth, u, v, id);
86         }
87     }
88 }
89 };

```

09 - 虚树.cpp

```

1  /*
2  用途：从若干关键点及其必要 LCA 构造虚树，压缩无关节点。
3
4  接口：
5  - build(points, lca, dfn):
6    * points 可重复；
7    * lca(u, v) 可传入原树的 askLCA；
8    * dfn(u) 返回原树的先序 DFS 编号（祖先必须在子孙之前）。
9  - vertex: 虚树节点（原树编号，按 dfn 排序）。
10 - edge: 有向边 parent -> child（均为原树节点编号）。
11 - root: 虚树根；空输入时为 0。
12
13 使用 03A 或 03B 的 askLCA + dfn/tin 即可。复杂度  $O(k \log k)$ （含 LCA 查询）。
14 */
15 struct VTree {
16     int rt = 0;
17     vector<int> u;
18     vector<pair<int, int>> e;
19
20     template <class Lca, class Dfn>
21     void build(vector<int> p, Lca lca, Dfn dfn) {
22         u.clear();
23         e.clear();
24         rt = 0;
25         if (p.empty()) return;
26
27         auto cmp = [&](int u, int v) { return dfn(u) < dfn(v); };
28         sort(p.begin(), p.end(), cmp);
29         p.erase(unique(p.begin(), p.end()), p.end());
30
31         int n0 = p.size();
32         for (int i = 1; i < n0; ++i) p.push_back(lca(p[i - 1], p[i]));
33         sort(p.begin(), p.end(), cmp);
34         p.erase(unique(p.begin(), p.end()), p.end());
35         u = p;
36
37         vector<int> st;
38         for (int u : u) {
39             while (!st.empty() && lca(st.back(), u) != st.back()) st.pop_back();
40             if (!st.empty()) e.push_back({st.back(), u});
41             st.push_back(u);
42         }
43         rt = u.front();
44     }
45
46     vector<vector<int>> buildG(int n) const {
47         vector<vector<int>> g(n + 1);
48         for (auto [u, v] : e) g[u].push_back(v);
49         return g;
50     }
51 };

```


树上问题：重构树、动态树与编码

4 个代码模板 · 林月 · v1.0.0 · 2026.8.1

10 - Kruskal 重构树.cpp

```

1  /*
2  用途: Kruskal 重构树 (升序), 把“边权不超过 w 时的连通性”转化为树上的 LCA。
3
4  build 后:
5  - askMin(u, v) 返回 u、v 连通所需的最小阈值, 即原图 u-v 路径中“最大边权最小”的值;
6  - askLCA(u, v) 对应两点首次被 Kruskal 合并时的重构树节点;
7  - sz[askLCA(u, v)] 是该时刻所在连通块大小;
8  - askDisc(u, v) 判断原图是否连通。
9
10 输入可为非连通图, 点编号为 1..n。边按权值升序处理; 相同权值的合并顺序不影响 askMin,
11 但若题目需要“同权边同时合并”的连通块语义, 需要将同权边额外缩点后再建树。
12 性能: 倍增祖先表默认使用 int; 极限内存不足时可只将祖先编号改为 i32, 权值保持 int。
13  */
14
15 struct KRT {
16
17     struct Edge {
18         int u, v;
19         int w;
20     };
21
22     int n, tot = 0, lg = 1;
23     vector<int> dsu, bel, sz, fa, dep;
24     vector<vector<int>> son, up;
25     vector<int> val;
26     vector<int> rts;
27
28     explicit KRT(int n) : n(n) {
29         reset();
30     }
31
32     void reset() {
33         int cap = 2 * n + 5;
34         tot = n;
35         dsu.resize(cap);
36         bel.assign(n + 1, 0);
37         sz.assign(cap, 0);
38         fa.assign(cap, 0);
39         dep.assign(cap, 0);
40         son.assign(cap, {});
41         val.assign(cap, 0);
42         rts.clear();
43         for (int u = 1; u <= n; ++u) dsu[u] = u, sz[u] = 1;
44     }
45
46     int find(int x) {
47         return dsu[x] == x ? x : dsu[x] = find(dsu[x]);
48     }
49
50     void dfs(int u, int _fa) {
51         fa[u] = _fa;
52         up[0][u] = _fa;
53         for (int bit = 1; bit < lg; ++bit) up[bit][u] = up[bit - 1][up[bit - 1][u]];
54         for (int v : son[u]) {
55             dep[v] = dep[u] + 1;
56             dfs(v, u);
57         }
58     }
59
60     void build(vector<Edge> es) {
61         reset();
62         sort(es.begin(), es.end(), [](const Edge &a, const Edge &b) { return a.w < b.w; });
63         for (const auto &[u, v, w] : es) {
64             int x = find(u), y = find(v);
65             if (x == y) continue;
66             int now = ++tot;
67             val[now] = w;
68             son[now] = {x, y};
69             sz[now] = sz[x] + sz[y];
70             dsu[x] = dsu[y] = now;
71             dsu[now] = now;

```

```

72     }
73
74     for (int u = 1; u <= n; ++u) bel[u] = find(u);
75     rts.assign(bel.begin() + 1, bel.end());
76     sort(rts.begin(), rts.end());
77     rts.erase(unique(rts.begin(), rts.end()), rts.end());
78
79     lg = 1;
80     while ((1LL << lg) <= max<int>(1, tot)) ++lg;
81     up.assign(lg, vector<int>(tot + 1));
82     fill(fa.begin(), fa.end(), 0);
83     fill(dep.begin(), dep.end(), 0);
84     for (int r : rts) dfs(r, 0);
85 }
86
87 bool askDisc(int u, int v) const {
88     return bel[u] != bel[v];
89 }
90
91 int askLCA(int u, int v) const {
92     if (askDisc(u, v)) return 0;
93     if (dep[u] < dep[v]) swap(u, v);
94     int dif = dep[u] - dep[v];
95     for (int bit = 0; bit < lg; ++bit) {
96         if ((dif >> bit) & 1) u = up[bit][u];
97     }
98     if (u == v) return u;
99     for (int bit = lg - 1; bit >= 0; --bit) {
100         if (up[bit][u] == up[bit][v]) continue;
101         u = up[bit][u];
102         v = up[bit][v];
103     }
104     return up[0][u];
105 }
106
107 optional<int> askMin(int u, int v) const {
108     if (askDisc(u, v)) return nullopt;
109     if (u == v) return 0;
110     return val[askLCA(u, v)];
111 }
112 };

```

11 - Link-Cut Tree (动态树) .cpp

```

1  /*
2  用途: Link-Cut Tree, 维护动态森林上的路径和与路径最大值。
3
4  接口 (点编号 1..n):
5  - modify(u, v): 修改点权。
6  - ask(u, v): 判断是否连通。
7  - link(u, v): 连一条边; 若会成环则返回 false。
8  - cut(u, v): 删除边 u-v; 若该边不存在则返回 false。
9  - askSum(u, v) / askMax(u, v): 返回路径点权和/最大值, 调用前保证两点连通。
10
11 T 应为支持加法、比较和 numeric_limits 的数值类型, 默认 int。该版本维护点权,
12 边权可通过“为每条边新建一个点并把权值放在该点”转换。
13 */
14
15 template <class T = int>
16 struct LCT {
17     int n;
18     vector<array<int, 2>> son;
19     vector<int> fa;
20     vector<char> tag;
21     vector<T> val, sum, mx;
22
23     explicit LCT(int n, T v = T{})
24         : n(n), son(n + 1), fa(n + 1), tag(n + 1), val(n + 1, v),
25           sum(n + 1, v), mx(n + 1, v) {
26         sum[0] = T{};
27         mx[0] = numeric_limits<T>::lowest();
28     }
29
30     bool isRoot(int x) const {
31         int p = fa[x];
32         return p == 0 || (son[p][0] != x && son[p][1] != x);
33     }
34
35     void pull(int x) {
36         sum[x] = sum[son[x][0]] + val[x] + sum[son[x][1]];

```

```

37     mx[x] = max(val[x], max(mx[son[x][0]], mx[son[x][1]]));
38 }
39
40 void rev(int x) {
41     if (!x) return;
42     swap(son[x][0], son[x][1]);
43     tag[x] ^= 1;
44 }
45
46 void push(int x) {
47     if (!x || !tag[x]) return;
48     rev(son[x][0]);
49     rev(son[x][1]);
50     tag[x] = 0;
51 }
52
53 void pushAll(int x) {
54     static vector<int> pth;
55     pth.clear();
56     pth.push_back(x);
57     for (int u = x; !isRoot(u); u = fa[u]) pth.push_back(fa[u]);
58     for (int i = pth.size(); i--;) push(pth[i]);
59 }
60
61 void rotate(int x) {
62     int p = fa[x], g = fa[p];
63     int sd = (son[p][1] == x);
64     int mid = son[x][sd ^ 1];
65     if (!isRoot(p)) son[g][son[g][1] == p] = x;
66     fa[x] = g;
67     son[x][sd ^ 1] = p;
68     fa[p] = x;
69     son[p][sd] = mid;
70     if (mid) fa[mid] = p;
71     pull(p);
72     pull(x);
73 }
74
75 void splay(int x) {
76     pushAll(x);
77     while (!isRoot(x)) {
78         int p = fa[x], g = fa[p];
79         if (!isRoot(p)) {
80             if ((son[p][0] == x) == (son[g][0] == p)) rotate(p);
81             else rotate(x);
82         }
83         rotate(x);
84     }
85     pull(x);
86 }
87
88 void access(int x) {
89     int lst = 0;
90     for (int u = x; u; u = fa[u]) {
91         splay(u);
92         son[u][1] = lst;
93         pull(u);
94         lst = u;
95     }
96     splay(x);
97 }
98
99 void mkRoot(int x) {
100     access(x);
101     rev(x);
102 }
103
104 int root(int x) {
105     access(x);
106     while (son[x][0]) {
107         push(x);
108         x = son[x][0];
109     }
110     splay(x);
111     return x;
112 }
113
114 bool ask(int u, int v) {
115     return u == v || root(u) == root(v);
116 }
117
118 bool link(int u, int v) {

```

```

119     mkRoot(u);
120     if (root(v) == u) return false;
121     fa[u] = v;
122     return true;
123 }
124
125 bool cut(int u, int v) {
126     mkRoot(u);
127     access(v);
128     if (son[v][0] != u || son[u][1] != 0) return false;
129     son[v][0] = 0;
130     fa[u] = 0;
131     pull(v);
132     return true;
133 }
134
135 void split(int u, int v) {
136     mkRoot(u);
137     access(v);
138 }
139
140 void modify(int u, T v) {
141     access(u);
142     val[u] = v;
143     pull(u);
144 }
145
146 T askSum(int u, int v) {
147     split(u, v);
148     return sum[v];
149 }
150
151 T askMax(int u, int v) {
152     split(u, v);
153     return mx[v];
154 }
155 };

```

12 - 树同构 (AHU) .cpp

```

1  /*
2  用途：无根树同构判定（AHU canonical form, 无随机哈希碰撞）。
3
4  输入邻接表必须为 1-indexed: g.size() == n + 1, g[1..n] 构成一棵树。
5  ask(g,h) 返回两棵无标号树是否同构，复杂度  $O(n \log n)$ （子树类型排序与 map）。
6  若题目要求保留点权/颜色，可把它们加入 rootId 的 key 中。
7  */
8  struct TreeIso {
9      static vector<int> centers(const vector<vector<int>> &g) {
10         int n = g.size() - 1;
11         if (n == 1) return {1};
12
13         vector<int> deg(n + 1);
14         vector<char> ban(n + 1);
15         queue<int> q;
16         for (int u = 1; u <= n; ++u) {
17             deg[u] = g[u].size();
18             if (deg[u] <= 1) q.push(u);
19         }
20
21         int rem = n;
22         while (rem > 2) {
23             int cnt = q.size();
24             rem -= cnt;
25             while (cnt-- > 0) {
26                 int u = q.front();
27                 q.pop();
28                 ban[u] = true;
29                 for (int v : g[u]) {
30                     if (!ban[v] && --deg[v] == 1) q.push(v);
31                 }
32             }
33         }
34
35         vector<int> res;
36         for (int u = 1; u <= n; ++u) {
37             if (!ban[u]) res.push_back(u);
38         }
39         return res;
40     }

```

```

41 static int rootId(const vector<vector<int>> &g, int u, int fa, map<vector<int>, int> &mp) {
42     vector<int> key;
43     for (int v : g[u]) {
44         if (v != fa) key.push_back(rootId(g, v, u, mp));
45     }
46     sort(all(key));
47     auto it = mp.find(key);
48     if (it == mp.end()) {
49         int id = mp.size() + 1;
50         it = mp.emplace(move(key), id).first;
51     }
52     return it->second;
53 }
54
55 static bool ask(const vector<vector<int>> &g, const vector<vector<int>> &h) {
56     if (g.size() != h.size()) return false;
57     map<vector<int>, int> mp;
58     set<int> id;
59     for (int rt : centers(g)) id.insert(rootId(g, rt, 0, mp));
60     for (int rt : centers(h)) {
61         if (id.count(rootId(h, rt, 0, mp))) return true;
62     }
63     return false;
64 }
65 }
66 };

```

13 - Prüfer序列.cpp

```

1  /*
2  用途：有标号树与 Prüfer 序列互转；点编号 1..n。
3  性质：序列长度 n-2，点 v 的度数等于其出现次数 + 1，因此有标号树共 n^(n-2) 棵。
4  复杂度：优先队列实现 O(n log n)。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  struct PruferCode {
8
9      static vector<int> encode(const vector<vector<int>> &g) {
10         int n = g.size() - 1;
11         if (n <= 2) return {};
12         vector<int> d(n + 1);
13         priority_queue<int, vector<int>, greater<int>> q;
14         for (int i = 1; i <= n; ++i) {
15             d[i] = g[i].size();
16             if (d[i] == 1) q.push(i);
17         }
18         vector<int> p;
19         p.reserve(n - 2);
20         for (int k = 0; k < n - 2; ++k) {
21             int u = q.top();
22             q.pop();
23             int v = 0;
24             for (int x : g[u]) if (d[x]) {
25                 v = x;
26                 break;
27             }
28             assert(v);
29             p.push_back(v);
30             d[u] = 0;
31             if (--d[v] == 1) q.push(v);
32         }
33         return p;
34     }
35
36     static vector<array<int, 2>> decode(int n, const vector<int> &p) {
37         assert(n >= 1 && p.size() == max<int>(0, n - 2));
38         if (n == 1) return {};
39         vector<int> d(n + 1, 1);
40         for (int x : p) assert(1 <= x && x <= n), ++d[x];
41         priority_queue<int, vector<int>, greater<int>> q;
42         for (int i = 1; i <= n; ++i) if (d[i] == 1) q.push(i);
43         vector<array<int, 2>> e;
44         e.reserve(n - 1);
45         for (int v : p) {

```

```
46         int u = q.top();
47         q.pop();
48         e.push_back({u, v});
49         --d[u];
50         if (--d[v] == 1) q.push(v);
51     }
52     int u = q.top(); q.pop();
53     int v = q.top();
54     e.push_back({u, v});
55     return e;
56 }
57 };
```

数据结构：并查集、区间结构与 RMQ

22 个代码模板 · 林月 · v1.0.0 · 2026.8.1

版块说明与知识结论

README.md

数据结构模板

需求	模板	复杂度
静态连通性、连通块信息	01A - 并查集	均摊 $O(\alpha(n))$
离线动态连通性	01B - 可撤销并查集	$O(\log n)$ / 次合并或查找
合并集合并删除元素	01C - 可删除并查集	均摊 $O(\alpha(n))$
相对距离/模意义关系	01D - 权值并查集	均摊 $O(\alpha(n))$
区间加、区间和	02A - 线段树	$O(\log n)$
区间加、区间最值	02B - 最值线段树	$O(\log n)$
历史版本计数/第 k 小	02C - 可持久化线段树、02E - 计数求和主席树	$O(\log V)$
自定义可合并信息	02D - 可合并信息线段树	$O(\log n)$
大值域、按需开点	02F - 动态开点线段树	$O(\log n)$
区间 chmin/chmax/add	02G - Segment Tree Beats	均摊 $O(\log n)$
直线最小值 / 动态 CHT	02H - Li Chao Tree	$O(\log X)$, 线段插入为 $O(\log^2 X)$
模意义区间乘、加、赋值与区间和	02I - 仿射懒标记线段树	$O(\log n)$, 计数类型自动取模
区间作用 + 自定义可合并信息	02J - 可合并信息懒标记线段树	$O(\log n)$
单点加前缀和	03 - 树状数组	$O(\log n)$
区间加区间和	03A - 区间加区间和树状数组	$O(\log n)$
稠密二维点加、矩形和	03B - 二维树状数组	$O(\log n \log m)$
稠密二维矩形加、矩形和	03C - 二维树状数组 (区间加、矩形和)	$O(\log n \log m)$
静态最值及位置	04A - ST表	预处理 $O(n \log n)$, 查询 $O(1)$
任意结合静态区间查询	04B - 可结合区间查询 (Disjoint Sparse Table)	同上
大规模静态最值 RMQ	04C - 线性RMQ (状压)	预处理 $O(n)$, 查询 $O(1)$
中等规模区间加、区间和、单点赋值	14 - 分块	每次 $O(\sqrt{n})$, 短实现分支
运行时位集	05 - Bitset	位运算 $O(n / 64)$
区间第 k 小、值域计数	06 - 小波矩阵; 只需第 k 小可用 06A	$O(\log \sigma)$
子集异或、可表示性	07 - 线性基	$O(B^2)$, $B \leq 64$
字符串集合	08A - 字典树	$O(\text{len}(s))$
异或最值/小于阈值计数	08B - 二进制字典树	$O(B)$
GNU 环境前缀范围枚举	08C - PBDS前缀字典树 (GNU环境)	$O(\text{len}(s) + \text{输出量})$
多个可合并优先队列	09A - 可并堆 (LeftistHeap, 可移植)	$O(\log n)$
GNU 环境可修改/删除节点的可并堆	09B - PBDS可并堆 (GNU环境)	push/join $O(1)$, pop 均摊 $O(\log n)$
自定义聚合的键值多重集	10 - 可合并信息 FHQTreap	期望 $O(\log n)$

需求	模板	复杂度
常规有序多重集	10A - FHQTreap (有序多重集)	期望 $O(\log n)$
可修改序列	10B - 隐式 FHQTreap (序列)	期望 $O(\log n)$
非交换聚合、翻转与兼容懒标记的可编辑序列	10BA - 隐式 FHQTreap (自定义聚合与懒标记)	期望 $O(\log n)$
GNU 环境有序多重集	10C - PBDS有序多重集 (GNU环境)	$O(\log n)$
小数据/少修改有序多重集	10D - vector有序多重集 (小数据环境)	查询 $O(\log n)$, 修改 $O(n)$
GNU 环境唯一键有序集合/映射	10E - PBDS有序集合与映射 (GNU环境)	$O(\log n)$, 支持排名、切分、合并
GNU 环境基础唯一键平衡树	10EA - PBDS平衡树 (GNU环境, 基础)	$O(\log n)$, 提供 add/del/rank/kth/前驱/后继
二维静态最近点、矩形计数	11A - KDTree (二维静态查询)	平均查询约 $O(\sqrt{n})$
二维动态插入、偏序计数	11B - KDTree (二维动态插入与偏序计数)	重构均摊, 查询最坏 $O(n)$
随机区间赋值下的分段序列	12 - 珂朵莉树 (ODT)	依赖区间数, 无通用最坏保证
GNU 开放寻址哈希集合/映射	13A - PBDS开放寻址哈希表 (gp_hash_table)	平均 $O(1)$
GNU 拉链哈希集合/映射	13B - PBDS拉链哈希表 (cc_hash_table)	平均 $O(1)$

选型提示

- 02A、02B、02F 都定义了 Tag、Info、SegTree，它们是独立片段，不能直接同时粘贴；按题意保留一个即可。模意义下同时乘/加/赋值时直接使用 02I。
- 需要同时维护自定义区间信息和懒标记时使用 02J；其中 `Tag::operator+=` 表示先作用当前标记，再作用右侧标记。
- 只有需要同时维护区间截断 (`a[i] = min/max(a[i], x)`) 时才使用 Beats；普通加法优先用更短的 02A/02B。
- Li Chao Tree 维护的是**最小值**。求最大值时可整体取相反数；直线计算结果应落在默认 `int` 范围内。
- 04A - ST表 仅适合幂等运算 (如 `min/max/gcd`)；区间和、矩阵乘法等结合但不幂等的操作使用 04B。
- 03B 只支持二维单点加；要修改整个矩形并查询矩形和时使用四棵树状数组的 03C。二者是操作语义不同的分支，不能省略成一个模板。
- 04A 与 04C 是同一静态 RMQ 的不同实现：普通规模优先短且稳的 ST 表；超大数组、反复查询时选状压线性 RMQ。
- 线段树能覆盖 14 的语义，但根号分块在中等规模、操作容易按块维护或需要现场短改时更直接；两者属于复杂度与实现成本不同的分支。
- 07 - 线性基的 `kthquery` 按“非空子集异或的不同结果”从小到大、1-indexed 返回；不存在时返回 -1。
- 动态树的 Link-Cut Tree 在 [树上问题](#)；树上路径/子树映射的 HLD 也放在该目录。
- 08C 只在需要直接取得“具有某前缀的全部键”时使用；普通字符串计数与转移仍优先使用紧凑的 08A。
- 09A/09B 是同一可并堆问题的可移植/GNU 分支；只有需要节点句柄的 `modify/erase` 时才优先 09B。
- 10A 是可移植的平衡树；10C 处理重复键；10D 以线性修改换极短实现；10E 提供 PBDS 唯一键集合/映射和 `split/join`；10EA 是只需要基础平衡树操作时的最短 PBDS 写法。环境与语义不同，均保留。
- 10BA 面向矩阵积、函数复合等非交换聚合；要求点值作用保持聚合且与区间翻转交换。只有普通区间加和时仍优先更短的 10B。
- 13A/13B 是同一哈希字典的开放寻址/拉链实现。通常 13A 常数更小；存大对象、担心聚簇时再实测 13B，二者都没有最坏复杂度保证。
- 11A 适合点集静态后批量查询；11B 支持在线插入与二维前缀计数。KD Tree 会受点分布影响，不能把平均复杂度当最坏保证。
- 12 只在随机区间赋值等题目保证下使用；`powsum<M>` 可直接传入自动取模的 `MInt`。

覆盖边界

本目录优先维护比赛中可直接复用的通用结构。PBDS 的基础 tree (10EA)、有序集合/映射 (10E)、多重集 (10C)、trie、priority_queue、gp_hash_table、cc_hash_table 已分别收录；低频且线性查找的 list_update 仅在知识稿说明，不单独占用代码模板。ODT 因旧版参考中存在而作为**带前提的环境模板**收录，并在文件头标明不保证复杂度；扫描线线段树、可持久化平衡树仍需按题目操作扩展。二维 BIT 已收录稠密版，坐标极大时需离线压缩。

操作选型、不变量、离线边界与环境分支见 [数据结构知识与结论](#)。

数据结构知识与结论.md

数据结构知识与结论

1. 先按操作选结构

操作	常用结构
单点改、前缀可逆聚合	树状数组
二维矩形加、矩形和	四棵二维树状数组 (03C)
区间改、区间聚合	懒标记线段树
中等规模区间加、区间和、单点赋值	根号分块
静态幂等 RMQ	ST 表 / 线性 RMQ
静态结合运算但不幂等	Disjoint Sparse Table
区间第 k 小	主席树 / 小波矩阵
动态有序多重集	FHQ Treap / GNU PBDS
唯一键排名、第 k 小、按键切分/合并	PBDS tree
只需唯一键平衡树的基础排名、前驱后继	10EA - PBDS 平衡树 (GNU 环境, 基础)
大量唯一键哈希字典	PBDS gp_hash_table / cc_hash_table
保留句柄修改/删除、合并优先队列	PBDS pairing heap
枚举同一字符串前缀范围	PBDS Patricia trie
动态序列翻转、切分	隐式 FHQ Treap
动态直线最值	Li Chao Tree
二维静态最近点	KD Tree

2. 不变量与常见错误

- 并查集只有在不需要撤销时做路径压缩；可撤销版本只按大小合并，并保存每次真正修改的字段。
- Fenwick 的 kth 要求点权非负，否则前缀和不单调。
- 懒标记的组合顺序表示“新操作作用在旧操作之后”还是相反，必须与 apply/push 一致。
- 主席树查询两个版本之差；值域先离散，返回的是离散下标，不要忘记映回原值。
- Segment Tree Beats 的均摊证明依赖只修改当前最大/最小的严格次值区间；任意截断不能直接复用。
- Li Chao Tree 比较的是同一横坐标处的函数值；乘法可能溢出时用 i128。

3. 静态、动态与离线

- 坐标压缩只保序，不保相邻差；涉及区间长度时要保存原坐标差或插入边界。
- 二维 BIT 的稠密空间是 $O(nm)$ 。03B 是点加、矩形和；03C 用四棵二维 BIT 支持矩形加、矩形和。坐标很大时，离线收集每个外层 Fenwick 节点会访问的 y 再分别压缩。
- KD Tree 的复杂度依赖数据分布与剪枝；高维会出现“维数灾难”，最坏仍可退化为线性扫描。11A 是静态最近点/矩形计数，11B 是动态插入/二维偏序计数，属于不同环境分支。
- ODT 只在随机赋值使区间数较少时表现好，没有通用复杂度保证。12 为兼容旧参考而保留，但它是**有条件模板**，不能替代具有复杂度保证的线段树。
- 10D 的排序 vector 与平衡树接口相同，但修改是线性的；它只适合小 n、修改少或希望现场极短实现的环境。
- 莫队、CDQ、整体二分属于离线技巧，分别要求可增删、偏序事件顺序或答案单调；见杂类目录。

4. GNU PBDS 操作速查

PBDS 是 GCC/libstdc++ 扩展，不属于 ISO C++。本库在包含 PBDS 头文件时临时撤销 `#define int long long`，避免宏改写库内部声明；模板参数中的 `int` 仍按公共宏成为 64 位。只有海量节点实际超时或超空间时，才局部声明并使用 `i32` 压

缩纯下标字段。

结构	本库模板	核心操作	关键限制
唯一键有序集合/映射	10E	order_of_key(x)、find_by_order(k)、split、join	第 k 小为 0-indexed; 比较器必须是严格弱序
重复键有序多重集	10C	insert、eraseOne、count、rank、kth	用 (值, 唯一编号), 不要使用 less_equal
开放寻址哈希	13A	[], insert、find、erase	修改可能使全部迭代器失效; 默认不能 reserve/resize
拉链哈希	13B	[], insert、find、erase	节点与指针开销较大
可并堆	09B	push 返回句柄, modify、erase、join	句柄只能交还原堆; join 会清空右侧堆
前缀 trie	08C	prefix_range(s)	键唯一; 内存通常高于手写 trie
基础有序平衡树	10EA	add/del/askRank/askKth/askPre/askNxt	唯一键; GNU/libstdc++ 专用

- 10EA 将 PBDS 的 0-indexed find_by_order 封装为 1-indexed askKth, 并提供最短的唯一键 add/del/askRank/askPre/askNxt; 只需要基础平衡树时优先抄它。
- tree::split(x, b) 在默认 less 下让原树保留 $\leq x$, 把 $> x$ 移入 b; join 要求两棵树的整个键域严格分离, 否则前置条件不成立。
- PBDS 哈希表与 unordered_map 一样只有平均 $O(1)$, 仍可能退化; 模板自带运行时盐, 但不能把它当作确定性最坏复杂度保证。
- gp_hash_table 使用开放寻址, 值对象过大时搬移成本和表空间都可能较高; cc_hash_table 使用拉链, 通常多一次指针访问和节点分配。赛时默认先试 13A。
- list_update 也是 PBDS 关联容器, 但查找为线性, 只适合极小集合或高度偏斜的访问分布; 竞赛中通常不如排序 vector 或哈希表, 故不单列模板。

5. 环境分支

FHQ Treap、Leftist Heap、手写 Trie 是标准 C++17 可移植版本; PBDS 只在 GCC/libstdc++ 可用。相同问题的可移植/GNU 实现因编译环境与接口不同而同时保留。节点、状态和边下标默认使用 int; 极限规模才局部改为 i32, 聚合值始终按题意保留 int 或更宽类型。

01A - 并查集 (DSU) .cpp

```

1  /*
2  用途: 普通并查集, 维护连通块大小、数量和可选的连通块信息。
3
4  约定: 点编号 1..n。T 需要支持默认构造和 +=, 合并时 v[根] += v[子根]。
5  接口: find、merge、ask、askSz、operator[]、askCnt。
6  时间复杂度: 均摊  $O(\alpha(n))$ 。
7  */
8  template <class T = int>
9  struct DSU {
10     vector<int> fa, sz;
11     vector<T> v;
12     int cc = 0;
13
14     DSU() = default;
15     explicit DSU(int n, const T &x = T()) {
16         init(n, x);
17     }
18
19     void init(int n, const T &x = T()) {
20         fa.resize(n + 1);
21         iota(all(fa), 0);
22         sz.assign(n + 1, 1);
23         v.assign(n + 1, x);
24         cc = n;
25     }
26
27     int find(int x) {
28         return fa[x] == x ? x : fa[x] = find(fa[x]);
29     }
30
31     bool merge(int x, int y) {

```

```

32     x = find(x);
33     y = find(y);
34     if (x == y) return false;
35     if (sz[x] < sz[y]) swap(x, y);
36     fa[y] = x;
37     sz[x] += sz[y];
38     v[x] += v[y];
39     --cc;
40     return true;
41 }
42
43 bool ask(int x, int y) {
44     return find(x) == find(y);
45 }
46
47 int askSz(int x) {
48     return sz[find(x)];
49 }
50
51 T &operator[](int x) {
52     return v[find(x)];
53 }
54
55 int askCnt() const {
56     return cc;
57 }
58 };

```

01B - 可撤销并查集 (DSU) .cpp

```

1  /*
2  用途：可撤销并查集。适合线段树分治、离线动态连通性等场景。
3
4  注意：find 不做路径压缩，否则无法回滚；按大小合并保证单次 find 为  $O(\log n)$ 。
5  接口：time、merge、undo、ask、askSz、askCnt；undo(t) 回到时刻 t。
6  */
7  struct DSU {
8      struct Ch {
9          int x, y, sz;
10     };
11
12     vector<int> fa, sz;
13     vector<Ch> his;
14     int cc = 0;
15
16     DSU() = default;
17     explicit DSU(int n) {
18         init(n);
19     }
20
21     void init(int n) {
22         fa.resize(n + 1);
23         iota(all(fa), 0);
24         sz.assign(n + 1, 1);
25         his.clear();
26         cc = n;
27     }
28
29     int find(int x) const {
30         while (fa[x] != x) x = fa[x];
31         return x;
32     }
33
34     bool merge(int x, int y) {
35         x = find(x);
36         y = find(y);
37         if (x == y) return false;
38         if (sz[x] < sz[y]) swap(x, y);
39         his.push_back({x, y, sz[x]});
40         fa[y] = x;
41         sz[x] += sz[y];
42         --cc;
43         return true;
44     }
45
46     int time() const {
47         return his.size();
48     }
49
50     void undo(int t) {

```

```

51     assert(0 <= t && t <= his.size());
52     while (his.size() > t) {
53         auto [x, y, s] = his.back();
54         his.pop_back();
55         fa[y] = y;
56         sz[x] = s;
57         ++cc;
58     }
59 }
60
61 bool ask(int x, int y) const {
62     return find(x) == find(y);
63 }
64
65 int askSz(int x) const {
66     return sz[find(x)];
67 }
68
69 int askCnt() const {
70     return cc;
71 }
72 };

```

01C - 可删除并查集 (DSU) .cpp

```

1  /*
2  用途：可删除并查集（“替身节点”技巧）。
3
4  erase(x) 把原编号 x 从当前集合移出，并让 x 重新成为一个大小为 1 的单点集合；
5  此前的内部节点仍保留在旧集合中，但不再代表任何当前编号。适合“合并集合 + 删除元素”。
6
7  接口：merge(x,y)、erase(x)、ask(x,y)、askSz(x)。点编号为 1..n。
8  */
9  struct DSU {
10     vector<int> fa, sz, id;
11     int tot = 0;
12
13     DSU() = default;
14     explicit DSU(int n) {
15         init(n);
16     }
17
18     void init(int n) {
19         tot = n;
20         fa.resize(n + 1);
21         iota(all(fa), 0);
22         sz.assign(n + 1, 1);
23         id.resize(n + 1);
24         for (int i = 1; i <= n; ++i) id[i] = i;
25     }
26
27     int find(int x) {
28         return fa[x] == x ? x : fa[x] = find(fa[x]);
29     }
30
31     int node() {
32         ++tot;
33         fa.push_back(tot);
34         sz.push_back(1);
35         return tot;
36     }
37
38     bool merge(int x, int y) {
39         int a = find(id[x]);
40         int b = find(id[y]);
41         if (a == b) return false;
42         if (sz[a] < sz[b]) swap(a, b);
43         fa[b] = a;
44         sz[a] += sz[b];
45         return true;
46     }
47
48     bool link(int a, int b) {
49         a = find(a);
50         b = find(b);
51         if (a == b) return false;
52         if (sz[a] < sz[b]) swap(a, b);
53         fa[b] = a;
54         sz[a] += sz[b];
55         return true;

```

```

56     }
57
58     void erase(int x) {
59         --sz[find(id[x])];
60         id[x] = node();
61     }
62
63     bool ask(int x, int y) {
64         return find(id[x]) == find(id[y]);
65     }
66
67     int askSz(int x) {
68         return sz[find(id[x])];
69     }
70 };

```

01D - 权值并查集 (DSU, with Int) .cpp

```

1  /*
2  用途：带势能（权值）并查集，维护形如  $\text{potential}[x] - \text{potential}[y] = \text{diff}$  的约束。
3
4   $w[x]$  表示  $\text{potential}[x] - \text{potential}[\text{fa}[x]]$ ；ask(x) 返回  $x$  相对所在根的势能。
5  merge(x,y,d)：加入  $\text{potential}[x] - \text{potential}[y] = d$ ，
6  若两点已连通则返回该约束是否与已有约束一致。
7
8  T 通常为 int 或 MInt，需支持 +、-、== 和默认构造。
9  */
10 template <class T>
11 struct DSU {
12     vector<int> fa, sz;
13     vector<T> w;
14
15     DSU() = default;
16     explicit DSU(int n) {
17         init(n);
18     }
19
20     void init(int n) {
21         fa.resize(n + 1);
22         iota(all(fa), 0);
23         sz.assign(n + 1, 1);
24         w.assign(n + 1, T{});
25     }
26
27     int find(int x) {
28         if (fa[x] == x) return x;
29         int p = fa[x];
30         int r = find(p);
31         w[x] = w[x] + w[p];
32         return fa[x] = r;
33     }
34
35     T ask(int x) {
36         find(x);
37         return w[x];
38     }
39
40     bool merge(int x, int y, const T &d) {
41         int fx = find(x), fy = find(y);
42         if (fx == fy) return w[x] - w[y] == d;
43
44         if (sz[fx] < sz[fy]) {
45             fa[fx] = fy;
46             w[fx] = d - w[x] + w[y];
47             sz[fy] += sz[fx];
48         } else {
49             fa[fy] = fx;
50             w[fy] = w[x] - w[y] - d;
51             sz[fx] += sz[fy];
52         }
53         return true;
54     }
55
56     bool ask(int x, int y) {
57         return find(x) == find(y);
58     }
59 };

```

02A - 线段树 (SegTree, with Int) .cpp

```

1  /*
2  用途：区间加、区间和、单点赋值的懒标记线段树。
3
4  约定：初始数组和所有区间均为 1-indexed 闭区间。
5  接口：build(a)、update(l, r, Tag<T>{delta})、modify(pos, value)、ask(l, r).val。
6  T 需要支持 +、*、-= 等常规数值运算，通常使用 int。
7  */
8  template <class T>
9  struct Tag {
10     T v = T{};
11
12     Tag &operator+=(const Tag &o) {
13         v += o.v;
14         return *this;
15     }
16
17     bool has() const {
18         return v != T{};
19     }
20 };
21
22 template <class T>
23 struct Info {
24     T val = T{};
25     int l = 1, r = 0; // 空信息满足 l > r。
26
27     bool askEmp() const {
28         return l > r;
29     }
30
31     bool hasSon() const {
32         return l < r;
33     }
34
35     Info operator+(const Info &o) const {
36         if (askEmp()) return o;
37         if (o.askEmp()) return *this;
38         return {val + o.val, l, o.r};
39     }
40
41     void operator+=(const Tag<T> &tag) {
42         if (!askEmp()) val += tag.v * (r - l + 1);
43     }
44 };
45
46 template <class T>
47 class SegTree {
48 private:
49     int n;
50     vector<Info<T>> tr;
51     vector<Tag<T>> tag;
52
53     static int ls(int x) {
54         return x << 1;
55     }
56     static int rs(int x) {
57         return x << 1 | 1;
58     }
59
60     void apply(int x, const Tag<T> &v) {
61         tr[x] += v;
62         tag[x] += v;
63     }
64
65     void pull(int x) {
66         tr[x] = tr[ls(x)] + tr[rs(x)];
67     }
68
69     void push(int x) {
70         if (!tr[x].hasSon() || !tag[x].has()) return;
71         apply(ls(x), tag[x]);
72         apply(rs(x), tag[x]);
73         tag[x] = Tag<T>{};
74     }
75
76     template <class A>
77     void build(int x, int l, int r, const A &a) {
78         tr[x].l = l;
79         tr[x].r = r;

```

```

80     tag[x] = Tag<T>{};
81     if (l == r) {
82         tr[x].val = a[l];
83         return;
84     }
85     int mid = (l + r) >> 1;
86     build(ls(x), l, mid, a);
87     build(rs(x), mid + 1, r, a);
88     pull(x);
89 }
90
91 void update(int x, int l, int r, int ql, int qr, const Tag<T> &v) {
92     if (qr < l || r < ql) return;
93     if (ql <= l && r <= qr) {
94         apply(x, v);
95         return;
96     }
97     push(x);
98     int mid = (l + r) >> 1;
99     update(ls(x), l, mid, ql, qr, v);
100    update(rs(x), mid + 1, r, ql, qr, v);
101    pull(x);
102 }
103
104 void modify(int x, int l, int r, int pos, const T &v) {
105     if (l == r) {
106         tr[x].val = v;
107         tag[x] = Tag<T>{};
108         return;
109     }
110     push(x);
111     int mid = (l + r) >> 1;
112     if (pos <= mid) modify(ls(x), l, mid, pos, v);
113     else modify(rs(x), mid + 1, r, pos, v);
114     pull(x);
115 }
116
117 Info<T> ask(int x, int l, int r, int ql, int qr) {
118     if (qr < l || r < ql) return {};
119     if (ql <= l && r <= qr) return tr[x];
120     push(x);
121     int mid = (l + r) >> 1;
122     return ask(ls(x), l, mid, ql, qr) + ask(rs(x), mid + 1, r, ql, qr);
123 }
124
125 public:
126     explicit SegTree(int n) : n(n), tr(4 * n + 5), tag(4 * n + 5) {
127         assert(n > 0);
128     }
129
130     template <class A>
131     void build(const A &a) {
132         build(1, 1, n, a);
133     }
134
135     void update(int l, int r, const Tag<T> &v) {
136         if (l > r) return;
137         assert(1 <= l && r <= n);
138         update(1, 1, n, l, r, v);
139     }
140
141     void modify(int pos, const T &v) {
142         assert(1 <= pos && pos <= n);
143         modify(1, 1, n, pos, v);
144     }
145
146     Info<T> ask(int l, int r) {
147         if (l > r) return {};
148         assert(1 <= l && r <= n);
149         return ask(1, 1, n, l, r);
150     }
151 };

```

02B - 最值线段树 (SegTree, with Int) .cpp

```

1  /*
2  用途：区间加、区间最小值/最大值、单点赋值的懒标记线段树。
3
4  约定：初始数组和所有区间均为 1-indexed 闭区间。
5  接口：build(a)、update(l, r, Tag<T>{delta})、modify(pos, value)、ask(l, r)。

```

```

6  查询结果的 mx / mn 分别为区间最大值 / 最小值。
7  */
8  template <class T>
9  struct Tag {
10     T v = T{};
11
12     Tag &operator+=(const Tag &o) {
13         v += o.v;
14         return *this;
15     }
16
17     bool has() const {
18         return v != T{};
19     }
20 };
21
22 template <class T>
23 struct Info {
24     T mx = numeric_limits<T>::lowest();
25     T mn = numeric_limits<T>::max();
26     int l = 1, r = 0;
27
28     bool askEmp() const {
29         return l > r;
30     }
31
32     bool hasSon() const {
33         return l < r;
34     }
35
36     Info operator+(const Info &o) const {
37         if (askEmp()) return o;
38         if (o.askEmp()) return *this;
39         return {max(mx, o.mx), min(mn, o.mn), l, o.r};
40     }
41
42     void operator+=(const Tag<T> &tag) {
43         if (!askEmp()) mx += tag.v, mn += tag.v;
44     }
45 };
46
47 template <class T>
48 class SegTree {
49 private:
50     int n;
51     vector<Info<T>> val;
52     vector<Tag<T>> tag;
53
54     static int ls(int x) {
55         return x << 1;
56     }
57     static int rs(int x) {
58         return x << 1 | 1;
59     }
60
61     void apply(int x, const Tag<T> &val) {
62         val[x] += val;
63         tag[x] += val;
64     }
65
66     void pull(int x) {
67         val[x] = val[ls(x)] + val[rs(x)];
68     }
69
70     void push(int x) {
71         if (!val[x].hasSon() || !tag[x].has()) return;
72         apply(ls(x), tag[x]);
73         apply(rs(x), tag[x]);
74         tag[x] = Tag<T>{};
75     }
76
77     template <class A>
78     void build(int x, int l, int r, const A &a) {
79         val[x].l = l;
80         val[x].r = r;
81         tag[x] = Tag<T>{};
82         if (l == r) {
83             val[x].mx = val[x].mn = a[l];
84             return;
85         }
86         int mid = (l + r) >> 1;
87         build(ls(x), l, mid, a);

```



```

88     build(rs(x), mid + 1, r, a);
89     pull(x);
90 }
91
92 void update(int x, int l, int r, int ql, int qr, const Tag<T> &val) {
93     if (qr < l || r < ql) return;
94     if (ql <= l && r <= qr) {
95         apply(x, val);
96         return;
97     }
98     push(x);
99     int mid = (l + r) >> 1;
100    update(ls(x), l, mid, ql, qr, val);
101    update(rs(x), mid + 1, r, ql, qr, val);
102    pull(x);
103 }
104
105 void modify(int x, int l, int r, int pos, const T &val) {
106     if (l == r) {
107         val[x].mx = val[x].mn = val;
108         tag[x] = Tag<T>{};
109         return;
110     }
111     push(x);
112     int mid = (l + r) >> 1;
113     if (pos <= mid) modify(ls(x), l, mid, pos, val);
114     else modify(rs(x), mid + 1, r, pos, val);
115     pull(x);
116 }
117
118 Info<T> ask(int x, int l, int r, int ql, int qr) {
119     if (qr < l || r < ql) return {};
120     if (ql <= l && r <= qr) return val[x];
121     push(x);
122     int mid = (l + r) >> 1;
123     return ask(ls(x), l, mid, ql, qr) + ask(rs(x), mid + 1, r, ql, qr);
124 }
125
126 public:
127     explicit SegTree(int n) : n(n), val(4 * n + 5), tag(4 * n + 5) {
128         assert(n > 0);
129     }
130
131     template <class A>
132     void build(const A &a) {
133         build(1, 1, n, a);
134     }
135
136     void update(int l, int r, const Tag<T> &val) {
137         if (l > r) return;
138         assert(1 <= l && r <= n);
139         update(1, 1, n, l, r, val);
140     }
141
142     void modify(int pos, const T &val) {
143         assert(1 <= pos && pos <= n);
144         modify(1, 1, n, pos, val);
145     }
146
147     Info<T> ask(int l, int r) {
148         if (l > r) return {};
149         assert(1 <= l && r <= n);
150         return ask(1, 1, n, l, r);
151     }
152 };

```

02C - 可持久化线段树 (SegTree) .cpp

```

1  /*
2  用途：可持久化“计数 + 权值和”线段树（主席树）。
3
4  值域为半开区间 [L, R)，版本 0 为空树。
5  - modify(ver, p, v): 从旧版本克隆一个新版本，位置 p 的计数 +1、权值和 +v。
6  - ask(ver, l, r): 查询该版本在 [l, r) 的计数和权值和。
7  - askKth(x, y, k): 版本差 y-x 中第 k 小的位置 (k 从 1 开始)，非法返回 -1。
8  */
9
10 struct SegTree {
11
12     struct Info {

```

```

13     int cnt = 0;
14     int sum = 0;
15
16     Info operator+(const Info &o) const {
17         return {cnt + o.cnt, sum + o.sum};
18     }
19 };
20
21 struct Node {
22     int ch[2]{};
23     Info val;
24 };
25
26 int lo = 0, hi = 0;
27 vector<Node> tr;
28 vector<int> rt;
29
30 SegTree() = default;
31 SegTree(int l, int r) {
32     init(l, r);
33 }
34
35 void init(int l, int r) {
36     assert(l < r);
37     lo = l;
38     hi = r;
39     tr.assign(1, Node{});
40     rt.assign(1, 0);
41 }
42
43 int clone(int p) {
44     tr.push_back(tr[p]);
45     return tr.size() - 1;
46 }
47
48 int modify0(int u, int l, int r, int p, int v) {
49     int x = clone(u);
50     ++tr[x].val.cnt;
51     tr[x].val.sum += v;
52     if (r - l == 1) return x;
53     int mid = l + (r - l) / 2;
54     int o = p >= mid;
55     if (o) tr[x].ch[1] = modify0(tr[u].ch[1], mid, r, p, v);
56     else tr[x].ch[0] = modify0(tr[u].ch[0], l, mid, p, v);
57     return x;
58 }
59
60 Info ask0(int u, int l, int r, int ql, int qr) const {
61     if (!u || qr <= l || r <= ql) return {};
62     if (ql <= l && r <= qr) return tr[u].val;
63     int mid = l + (r - l) / 2;
64     return ask0(tr[u].ch[0], l, mid, ql, qr) + ask0(tr[u].ch[1], mid, r, ql, qr);
65 }
66
67 int askKth0(int x, int y, int l, int r, int k) const {
68     int tot = tr[y].val.cnt - tr[x].val.cnt;
69     if (k <= 0 || k > tot) return -1;
70     while (r - l > 1) {
71         int cnt = tr[tr[y].ch[0]].val.cnt - tr[tr[x].ch[0]].val.cnt;
72         int mid = l + (r - l) / 2;
73         if (k <= cnt) {
74             x = tr[x].ch[0];
75             y = tr[y].ch[0];
76             r = mid;
77         } else {
78             k -= cnt;
79             x = tr[x].ch[1];
80             y = tr[y].ch[1];
81             l = mid;
82         }
83     }
84     return l;
85 }
86
87 int modify(int ver, int p, int v) {
88     assert(0 <= ver && ver < rt.size());
89     assert(lo <= p && p < hi);
90     rt.push_back(modify0(rt[ver], lo, hi, p, v));
91     return rt.size() - 1;
92 }
93
94 Info ask(int ver, int l, int r) const {

```

```

95     assert(0 <= ver && ver < rt.size());
96     l = max(l, lo);
97     r = min(r, hi);
98     return l >= r ? Info{} : ask0(rt[ver], lo, hi, l, r);
99 }
100
101 int askKth(int x, int y, int k) const {
102     assert(0 <= x && x < rt.size());
103     assert(0 <= y && y < rt.size());
104     return askKth0(rt[x], rt[y], lo, hi, k);
105 }
106
107 int askCnt() const {
108     return rt.size();
109 }
110 };

```

02D - 可合并信息线段树 (MergeSegTree) .cpp

```

1  /*
2  用途: 不带懒标记的通用线段树。
3
4  Info 需要默认构造, 并定义 Info operator+(const Info&); 数组与区间均为 1-indexed 闭区间。
5  支持 build、单点 modify、区间 ask 和整段 ask, 均为 O(log n)。
6  */
7  template <class T> class SegTree {
8  private:
9      vector<T> val;
10     int n = 0;
11
12     int ls(int x) {
13         return x << 1;
14     }
15     int rs(int x) {
16         return x << 1 | 1;
17     }
18
19     template <class A> void build(int x, int l, int r, const A &a) {
20         if (l == r) {
21             val[x] = a[l];
22             return;
23         }
24         int mid = (l + r) >> 1;
25         build(ls(x), l, mid, a);
26         build(rs(x), mid + 1, r, a);
27         val[x] = val[ls(x)] + val[rs(x)];
28     }
29
30     void modify(int x, int l, int r, int p, const T &v) {
31         if (l == r) {
32             val[x] = v;
33             return;
34         }
35         int mid = (l + r) >> 1;
36         if (p <= mid) modify(ls(x), l, mid, p, v);
37         else modify(rs(x), mid + 1, r, p, v);
38         val[x] = val[ls(x)] + val[rs(x)];
39     }
40
41     T ask(int x, int l, int r, int ql, int qr) {
42         if (ql <= l && r <= qr) return val[x];
43         int mid = (l + r) >> 1;
44         if (qr <= mid) return ask(ls(x), l, mid, ql, qr);
45         if (ql > mid) return ask(rs(x), mid + 1, r, ql, qr);
46         return ask(ls(x), l, mid, ql, qr) + ask(rs(x), mid + 1, r, ql, qr);
47     }
48
49 public:
50     SegTree(int _n = 0) {
51         if (_n) init(_n);
52     }
53
54     void init(int _n) {
55         n = _n;
56         val.assign(4 * n + 5, T());
57     }
58
59     template <class A> void build(const A &a) {
60         build(1, 1, n, a);
61     }

```

```

62
63 void modify(int p, const T &v) {
64     modify(1, 1, n, p, v);
65 }
66
67 T ask(int l, int r) {
68     return ask(1, 1, n, l, r);
69 }
70
71 T ask() {
72     return val[1];
73 }
74 };

```

02E - 计数求和主席树 (CountSumSegTree) .cpp

```

1  /*
2  用途: 前缀版本主席树, 维护值域 [L, R] 上的出现次数和附加权值和。
3
4  add(x,v) 追加一个前缀版本; ask(l,r,ql,qr) 查询第 l..r 个插入元素
5  (l/r 均为 1-indexed 闭区间) 在值域 [ql, qr] 的计数和权值和。
6  */
7
8  struct SegTree {
9      struct Info {
10         int cnt = 0;
11         int sum = 0;
12         Info operator+(const Info &o) const {
13             return {cnt + o.cnt, sum + o.sum};
14         }
15     };
16
17     int lo, hi;
18     vector<array<int, 2>> ch;
19     vector<int> cnt, rt;
20     vector<int> sum;
21
22     SegTree() {}
23     SegTree(int l, int r) {
24         init(l, r);
25     }
26
27     void init(int l, int r) {
28         lo = l;
29         hi = r;
30         ch.assign(1, {0, 0});
31         cnt.assign(1, 0);
32         sum.assign(1, 0);
33         rt.assign(1, 0);
34     }
35
36     int clone(int p) {
37         ch.push_back(ch[p]);
38         cnt.push_back(cnt[p]);
39         sum.push_back(sum[p]);
40         return cnt.size() - 1;
41     }
42
43     int modify(int p, int l, int r, int x, int v) {
44         int q = clone(p);
45         ++cnt[q];
46         sum[q] += v;
47         if (l < r) {
48             int mid = (l + r) >> 1;
49             if (x <= mid) {
50                 ch[q][0] = modify(ch[p][0], l, mid, x, v);
51             } else {
52                 ch[q][1] = modify(ch[p][1], mid + 1, r, x, v);
53             }
54         }
55         return q;
56     }
57
58     int modify(int pre, int x, int v) {
59         return modify(pre, lo, hi, x, v);
60     }
61
62     int add(int x) {
63         rt.push_back(modify(rt.back(), x, x));
64         return rt.back();

```

```

65     }
66
67     int add(int x, int v) {
68         rt.push_back(modify(rt.back(), x, v));
69         return rt.back();
70     }
71
72     Info ask0(int u, int v, int l, int r, int ql, int qr) const {
73         if (ql > qr || qr < l || r < ql) {
74             return {};
75         }
76         if (ql <= l && r <= qr) {
77             return {cnt[u] - cnt[v], sum[u] - sum[v]};
78         }
79         int mid = (l + r) >> 1;
80         return ask0(ch[u][0], ch[v][0], l, mid, ql, qr) +
81             ask0(ch[u][1], ch[v][1], mid + 1, r, ql, qr);
82     }
83
84     Info ask0(int u, int v, int ql, int qr) const {
85         ql = max(ql, lo);
86         qr = min(qr, hi);
87         if (ql > qr) {
88             return {};
89         }
90         return ask0(u, v, lo, hi, ql, qr);
91     }
92
93     Info ask(int l, int r, int ql, int qr) const {
94         if (l > r) {
95             return {};
96         }
97         return ask0(rt[r], rt[l - 1], ql, qr);
98     }
99
100     int askCnt() const {
101         return rt.size() - 1;
102     }
103 };

```

02F - 动态开点线段树.cpp

```

1  /*
2  用途：按需开点的区间加、区间和、单点赋值线段树。
3
4  接口与 02A 一致，区间和数组均为 1-indexed；适用于 n 很大、实际访问稀疏的场景。
5  注意查询非整段时也可能创建子节点，以保证懒标记语义正确。
6  */
7  template <class T> struct Tag {
8      T v = 0;
9
10     void operator+=(const Tag<T> &o) {
11         v += o.v;
12     }
13
14     bool has() const {
15         return v != 0;
16     }
17 };
18
19 template <class T> struct Info {
20     T val = 0;
21     int l = 1, r = 0;
22
23     Info operator+(const Info<T> &o) const {
24         if (l > r) return o;
25         if (o.l > o.r) return *this;
26         Info res;
27         res.l = l;
28         res.r = o.r;
29         res.val = val + o.val;
30         return res;
31     }
32
33     void operator+=(const Tag<T> &o) {
34         if (l <= r) val += o.v * (r - l + 1);
35     }
36
37     bool hasSon() const {
38         return l < r;

```

```

39     }
40 };
41
42 template <class T> class SegTree {
43 private:
44     struct Node {
45         Info<T> in;
46         Tag<T> tag;
47         int ls = 0, rs = 0;
48     };
49
50     vector<Node> tr;
51     int n, rt;
52
53     int node(int l, int r) {
54         tr.push_back(Node());
55         int x = tr.size() - 1;
56         tr[x].in.l = l;
57         tr[x].in.r = r;
58         return x;
59     }
60
61     void apply(int x, const Tag<T> &v) {
62         tr[x].in += v;
63         tr[x].tag += v;
64     }
65
66     void push(int x) {
67         if (!tr[x].in.hasSon()) return;
68
69         int l = tr[x].in.l, r = tr[x].in.r;
70         int mid = l + ((r - l) >> 1);
71
72         if (!tr[x].ls) tr[x].ls = node(l, mid);
73         if (!tr[x].rs) tr[x].rs = node(mid + 1, r);
74
75         if (tr[x].tag.has()) {
76             apply(tr[x].ls, tr[x].tag);
77             apply(tr[x].rs, tr[x].tag);
78             tr[x].tag = Tag<T>();
79         }
80     }
81
82     void pull(int x) {
83         tr[x].in = tr[tr[x].ls].in + tr[tr[x].rs].in;
84     }
85
86     void print(int x) {
87         if (!x) return;
88         cout << x << ":[" << tr[x].in.l << ", " << tr[x].in.r << "], val:" << tr[x].in.val << ", tag:" << tr[x].tag.v
<< endl;
89         print(tr[x].ls);
90         print(tr[x].rs);
91     }
92
93     template <class A> void build(int x, A &dat) {
94         int l = tr[x].in.l, r = tr[x].in.r;
95         tr[x].tag = Tag<T>();
96
97         if (l == r) {
98             tr[x].in.val = dat[l];
99             return;
100         }
101
102         int mid = l + ((r - l) >> 1);
103         tr[x].ls = node(l, mid);
104         tr[x].rs = node(mid + 1, r);
105
106         build(tr[x].ls, dat);
107         build(tr[x].rs, dat);
108         pull(x);
109     }
110
111     void update(int x, int lq, int rq, const Tag<T> &v) {
112         int l = tr[x].in.l, r = tr[x].in.r;
113         if (rq < l || r < lq) return;
114
115         if (lq <= l && r <= rq) {
116             apply(x, v);
117             return;
118         }
119

```

```

120     push(x);
121     int mid = l + ((r - l) >> 1);
122
123     if (lq <= mid) update(tr[x].ls, lq, rq, v);
124     if (rq > mid) update(tr[x].rs, lq, rq, v);
125
126     pull(x);
127 }
128
129 void modify(int x, int pos, T v) {
130     int l = tr[x].in.l, r = tr[x].in.r;
131
132     if (l == r) {
133         tr[x].in.val = v;
134         tr[x].tag = Tag<T>();
135         return;
136     }
137
138     push(x);
139     int mid = l + ((r - l) >> 1);
140
141     if (pos <= mid)
142         modify(tr[x].ls, pos, v);
143     else
144         modify(tr[x].rs, pos, v);
145
146     pull(x);
147 }
148
149 Info<T> ask(int x, int lq, int rq) {
150     int l = tr[x].in.l, r = tr[x].in.r;
151     if (rq < l || r < lq) return Info<T>();
152
153     if (lq <= l && r <= rq) return tr[x].in;
154
155     push(x);
156     int mid = l + ((r - l) >> 1);
157
158     if (rq <= mid) return ask(tr[x].ls, lq, rq);
159     if (lq > mid) return ask(tr[x].rs, lq, rq);
160     return ask(tr[x].ls, lq, rq) + ask(tr[x].rs, lq, rq);
161 }
162
163 public:
164     SegTree(int _n) : n(_n) {
165         tr.push_back(Node());
166         rt = node(1, n);
167     }
168
169     void print() {
170         print(rt);
171     }
172
173     template <class A> void build(A &dat) {
174         tr.clear();
175         tr.push_back(Node());
176         rt = node(1, n);
177         build(rt, dat);
178     }
179
180     void update(int l, int r, Tag<T> v) {
181         if (l > r) return;
182         update(rt, l, r, v);
183     }
184
185     void modify(int pos, T v) {
186         if (pos < 1 || pos > n) return;
187         modify(rt, pos, v);
188     }
189
190     Info<T> ask(int l, int r) {
191         if (l > r) return Info<T>();
192         return ask(rt, l, r);
193     }
194 };

```

02G - Segment Tree Beats.cpp

```

1  /*
2  用途: Segment Tree Beats, 支持区间 chmin、chmax、加法、和/最小/最大值查询。

```

```

3
4 约定：初始数组为 1-indexed。接口：build(a)、updMin、updMax、update、
5 askSum/askMin/askMax（均为闭区间）。均摊复杂度  $O(\log n)$ 。
6 */
7
8 struct SegTree {
9     static constexpr int NI = numeric_limits<int>::lowest() / 4;
10    static constexpr int PI = numeric_limits<int>::max() / 4;
11
12    struct Node {
13        int sum = 0, mx1 = NI, mx2 = NI, mn1 = PI, mn2 = PI, add = 0;
14        int cmx = 0, cmn = 0;
15    };
16
17    int n = 0;
18    vector<Node> tr;
19
20    SegTree() = default;
21    explicit SegTree(int n) {
22        init(n);
23    }
24
25    void init(int _n) {
26        assert(_n > 0);
27        n = _n;
28        tr.assign(4 * n + 5, Node{});
29    }
30
31    static int ls(int p) {
32        return p << 1;
33    }
34    static int rs(int p) {
35        return p << 1 | 1;
36    }
37
38    void pull(int p) {
39        const Node &a = tr[ls(p)], &b = tr[rs(p)];
40        Node &x = tr[p];
41        x.sum = a.sum + b.sum;
42
43        x.mx1 = max(a.mx1, b.mx1);
44        x.cmx = 0;
45        x.mx2 = NI;
46        for (const Node *son : {&a, &b}) {
47            if (son->mx1 == x.mx1) {
48                x.cmx += son->cmx;
49                x.mx2 = max(x.mx2, son->mx2);
50            } else {
51                x.mx2 = max(x.mx2, son->mx1);
52            }
53        }
54
55        x.mn1 = min(a.mn1, b.mn1);
56        x.cmn = 0;
57        x.mn2 = PI;
58        for (const Node *son : {&a, &b}) {
59            if (son->mn1 == x.mn1) {
60                x.cmn += son->cmn;
61                x.mn2 = min(x.mn2, son->mn2);
62            } else {
63                x.mn2 = min(x.mn2, son->mn1);
64            }
65        }
66        x.add = 0;
67    }
68
69    void setAdd(int p, int l, int r, int val) {
70        Node &x = tr[p];
71        x.sum += val * (r - l + 1);
72        x.mx1 += val;
73        if (x.mx2 != NI) x.mx2 += val;
74        x.mn1 += val;
75        if (x.mn2 != PI) x.mn2 += val;
76        x.add += val;
77    }
78
79    void setMin(int p, int val) {
80        Node &x = tr[p];
81        assert(x.mx2 < val && val < x.mx1);
82        x.sum += (val - x.mx1) * x.cmx;
83        if (x.mn1 == x.mx1) x.mn1 = val;
84        else if (x.mn2 == x.mx1) x.mn2 = val;

```



```

85     x.mx1 = val;
86 }
87
88 void setMax(int p, int val) {
89     Node &x = tr[p];
90     assert(x.mn1 < val && val < x.mn2);
91     x.sum += (val - x.mn1) * x.cmn;
92     if (x.mx1 == x.mn1) x.mx1 = val;
93     else if (x.mx2 == x.mn1) x.mx2 = val;
94     x.mn1 = val;
95 }
96
97 void push(int p, int l, int r) {
98     if (l == r) return;
99     int mid = (l + r) >> 1;
100    if (tr[p].add) {
101        setAdd(ls(p), l, mid, tr[p].add);
102        setAdd(rs(p), mid + 1, r, tr[p].add);
103        tr[p].add = 0;
104    }
105    if (tr[ls(p)].mx1 > tr[p].mx1) setMin(ls(p), tr[p].mx1);
106    if (tr[rs(p)].mx1 > tr[p].mx1) setMin(rs(p), tr[p].mx1);
107    if (tr[ls(p)].mn1 < tr[p].mn1) setMax(ls(p), tr[p].mn1);
108    if (tr[rs(p)].mn1 < tr[p].mn1) setMax(rs(p), tr[p].mn1);
109 }
110
111 template <class A>
112 void build(int p, int l, int r, const A &a) {
113     if (l == r) {
114         int val = a[l];
115         tr[p] = {val, val, NI, val, PI, 0, 1, 1};
116         return;
117     }
118     int mid = (l + r) >> 1;
119     build(ls(p), l, mid, a);
120     build(rs(p), mid + 1, r, a);
121     pull(p);
122 }
123
124 template <class A>
125 void build(const A &a) {
126     build(1, 1, n, a);
127 }
128
129 void updMin(int p, int l, int r, int ql, int qr, int val) {
130     if (qr < l || r < ql || tr[p].mx1 <= val) return;
131     if (ql <= l && r <= qr && tr[p].mx2 < val) {
132         setMin(p, val);
133         return;
134     }
135     push(p, l, r);
136     int mid = (l + r) >> 1;
137     updMin(ls(p), l, mid, ql, qr, val);
138     updMin(rs(p), mid + 1, r, ql, qr, val);
139     pull(p);
140 }
141
142 void updMax(int p, int l, int r, int ql, int qr, int val) {
143     if (qr < l || r < ql || tr[p].mn1 >= val) return;
144     if (ql <= l && r <= qr && tr[p].mn2 > val) {
145         setMax(p, val);
146         return;
147     }
148     push(p, l, r);
149     int mid = (l + r) >> 1;
150     updMax(ls(p), l, mid, ql, qr, val);
151     updMax(rs(p), mid + 1, r, ql, qr, val);
152     pull(p);
153 }
154
155 void update(int p, int l, int r, int ql, int qr, int val) {
156     if (qr < l || r < ql) return;
157     if (ql <= l && r <= qr) {
158         setAdd(p, l, r, val);
159         return;
160     }
161     push(p, l, r);
162     int mid = (l + r) >> 1;
163     update(ls(p), l, mid, ql, qr, val);
164     update(rs(p), mid + 1, r, ql, qr, val);
165     pull(p);
166 }

```

```

167
168 int askSum(int p, int l, int r, int ql, int qr) {
169     if (qr < l || r < ql) return 0;
170     if (ql <= l && r <= qr) return tr[p].sum;
171     push(p, l, r);
172     int mid = (l + r) >> 1;
173     return askSum(ls(p), l, mid, ql, qr) + askSum(rs(p), mid + 1, r, ql, qr);
174 }
175
176 int askMin(int p, int l, int r, int ql, int qr) {
177     if (qr < l || r < ql) return PI;
178     if (ql <= l && r <= qr) return tr[p].mn1;
179     push(p, l, r);
180     int mid = (l + r) >> 1;
181     return min(askMin(ls(p), l, mid, ql, qr), askMin(rs(p), mid + 1, r, ql, qr));
182 }
183
184 int askMax(int p, int l, int r, int ql, int qr) {
185     if (qr < l || r < ql) return NI;
186     if (ql <= l && r <= qr) return tr[p].mx1;
187     push(p, l, r);
188     int mid = (l + r) >> 1;
189     return max(askMax(ls(p), l, mid, ql, qr), askMax(rs(p), mid + 1, r, ql, qr));
190 }
191
192 void updMin(int l, int r, int val) {
193     if (l <= r) updMin(1, 1, n, l, r, val);
194 }
195 void updMax(int l, int r, int val) {
196     if (l <= r) updMax(1, 1, n, l, r, val);
197 }
198 void update(int l, int r, int val) {
199     if (l <= r) update(1, 1, n, l, r, val);
200 }
201 int askSum(int l, int r) {
202     return askSum(1, 1, n, l, r);
203 }
204 int askMin(int l, int r) {
205     return askMin(1, 1, n, l, r);
206 }
207 int askMax(int l, int r) {
208     return askMax(1, 1, n, l, r);
209 }
210 };

```

02H - Li Chao Tree.cpp

```

1  /*
2  用途: 动态 Li Chao Tree, 维护直线  $y = m \cdot x + b$  的点集, 查询指定  $x$  的最小值。
3
4  定义域为整数闭区间  $[L, R]$ 。add 添加整条直线, addSeg 添加只在  $[ql, qr]$  有效的线段。
5  ask(x) 返回最小值; 若尚未添加直线, 返回 INF。比较过程使用 i128,
6  但最终返回 int, 需保证实际答案不溢出 int。
7  */
8
9  using i128 = __int128_t;
10
11 struct LiChaoTree {
12     static constexpr int INF = numeric_limits<int>::max();
13
14     struct Line {
15         int m = 0, b = INF;
16         i128 eval(int x) const {
17             return i128(m) * x + b;
18         }
19     };
20
21     struct Node {
22         Line f;
23         int ch[2]{};
24     };
25
26     int lo, hi;
27     vector<Node> tr;
28     int rt = 0;
29
30     LiChaoTree(int l, int r) : lo(l), hi(r), tr(1) {
31         assert(lo <= hi);
32     }
33

```

```

34 int node() {
35     tr.push_back(Node{});
36     return tr.size() - 1;
37 }
38
39 int add0(int p, int l, int r, Line f) {
40     if (!p) p = node();
41     int mid = l + (r - l) / 2;
42     bool bl = f.eval(l) < tr[p].f.eval(l);
43     bool bm = f.eval(mid) < tr[p].f.eval(mid);
44     if (bm) swap(f, tr[p].f);
45     if (l == r) return p;
46     if (bl != bm) tr[p].ch[0] = add0(tr[p].ch[0], l, mid, f);
47     else tr[p].ch[1] = add0(tr[p].ch[1], mid + 1, r, f);
48     return p;
49 }
50
51 int add1(int p, int l, int r, int ql, int qr, Line f) {
52     if (qr < l || r < ql) return p;
53     if (ql <= l && r <= qr) {
54         return add0(p, l, r, f);
55     }
56     if (!p) p = node();
57     int mid = l + (r - l) / 2;
58     tr[p].ch[0] = add1(tr[p].ch[0], l, mid, ql, qr, f);
59     tr[p].ch[1] = add1(tr[p].ch[1], mid + 1, r, ql, qr, f);
60     return p;
61 }
62
63 i128 ask0(int p, int l, int r, int x) const {
64     if (!p) return i128(INF);
65     i128 ans = tr[p].f.eval(x);
66     if (l == r) return ans;
67     int mid = l + (r - l) / 2;
68     if (x <= mid) return min(ans, ask0(tr[p].ch[0], l, mid, x));
69     return min(ans, ask0(tr[p].ch[1], mid + 1, r, x));
70 }
71
72 void add(int m, int b) {
73     rt = add0(rt, lo, hi, {m, b});
74 }
75
76 void addSeg(int m, int b, int ql, int qr) {
77     ql = max(ql, lo);
78     qr = min(qr, hi);
79     if (ql <= qr) rt = add1(rt, lo, hi, ql, qr, {m, b});
80 }
81
82 int ask(int x) const {
83     assert(lo <= x && x <= hi);
84     i128 ans = ask0(rt, lo, hi, x);
85     assert(ans >= numeric_limits<int>::lowest() && ans <= numeric_limits<int>::max());
86     return ans;
87 }
88 };

```

02I - 仿射懒标记线段树（自动取模）.cpp

```

1  /*
2  用途：模意义下维护区间和，支持区间乘、加、赋值；依赖自动取模类型 Z。
3  接口：build(a)、update(l,r,m,a) 表示  $x \leftarrow m \cdot x + a$ ；updMul/updAdd/updSet/ask 均为闭区间。
4  复杂度：建树  $O(n)$ ，每次操作  $O(\log n)$ 。
5  */
6  template <class Z> struct SegTree {
7      int n;
8      vector<Z> s, m, a;
9
10     explicit SegTree(int _n) : n(_n), s(4 * _n + 5), m(4 * _n + 5, Z(1)), a(4 * _n + 5) {
11         assert(n > 0);
12     }
13
14     template <class V> void build(const V& v) {
15         assert(v.size() > n);
16         build(1, 1, n, v);
17     }
18
19     void update(int l, int r, Z x, Z y) {
20         if (l > r) return;
21         assert(1 <= l && r <= n);
22         upd(1, 1, n, l, r, x, y);

```

```

23     }
24     void updMul(int l, int r, Z x) { update(l, r, x, Z(0)); }
25     void updAdd(int l, int r, Z x) { update(l, r, Z(1), x); }
26     void updSet(int l, int r, Z x) { update(l, r, Z(0), x); }
27     Z ask(int l, int r) {
28         if (l > r) return Z(0);
29         assert(1 <= l && r <= n);
30         return ask(1, 1, n, l, r);
31     }
32
33 private:
34     template <class V> void build(int p, int l, int r, const V& v) {
35         if (l == r) return void(s[p] = Z(v[l]));
36         int q = (l + r) >> 1;
37         build(p << 1, l, q, v), build(p << 1 | 1, q + 1, r, v);
38         pull(p);
39     }
40
41     void apply(int p, int l, int r, Z x, Z y) {
42         s[p] = s[p] * x + y * (r - l + 1);
43         m[p] *= x;
44         a[p] = a[p] * x + y;
45     }
46
47     void pull(int p) { s[p] = s[p << 1] + s[p << 1 | 1]; }
48
49     void push(int p, int l, int r) {
50         if (l == r) return;
51         int q = (l + r) >> 1;
52         apply(p << 1, l, q, m[p], a[p]);
53         apply(p << 1 | 1, q + 1, r, m[p], a[p]);
54         m[p] = Z(1), a[p] = Z(0);
55     }
56
57     void upd(int p, int l, int r, int x, int y, Z u, Z v) {
58         if (x <= l && r <= y) return apply(p, l, r, u, v);
59         push(p, l, r);
60         int q = (l + r) >> 1;
61         if (x <= q) upd(p << 1, l, q, x, y, u, v);
62         if (y > q) upd(p << 1 | 1, q + 1, r, x, y, u, v);
63         pull(p);
64     }
65
66     Z ask(int p, int l, int r, int x, int y) {
67         if (x <= l && r <= y) return s[p];
68         push(p, l, r);
69         int q = (l + r) >> 1;
70         if (y <= q) return ask(p << 1, l, q, x, y);
71         if (x > q) return ask(p << 1 | 1, q + 1, r, x, y);
72         return ask(p << 1, l, q, x, y) + ask(p << 1 | 1, q + 1, r, x, y);
73     }
74 };

```

02J - 可合并信息懒标记线段树 (LazyMergeSegTree).cpp

```

1  /*
2  用途: 通用可合并信息懒标记线段树。
3
4  Info 需要默认构造, 并定义 Info operator+(const Info&) 与 Info::operator+=(const Tag&);
5  Tag 需要默认构造为恒等操作, 并定义 Tag::operator+=(const Tag&) 和 bool has() const。
6  tag += o 表示先作用当前 tag, 再作用 o。数组与区间均为 1-indexed 闭区间。
7  支持 build、区间 update、单点 modify、区间 ask 和整段 ask, 均为 O(log n)。
8  */
9  template <class Info, class Tag> class LazySegTree {
10 private:
11     int n = 0;
12     vector<Info> val;
13     vector<Tag> tag;
14
15     int ls(int x) {
16         return x << 1;
17     }
18     int rs(int x) {
19         return x << 1 | 1;
20     }
21
22     void apply(int x, const Tag &v) {
23         val[x] += v;
24         tag[x] += v;
25     }

```

```

26 void pull(int x) {
27     val[x] = val[ls(x)] + val[rs(x)];
28 }
29
30
31 void push(int x) {
32     if (!tag[x].has()) return;
33     apply(ls(x), tag[x]);
34     apply(rs(x), tag[x]);
35     tag[x] = Tag();
36 }
37
38 template <class A> void build(int x, int l, int r, const A &a) {
39     tag[x] = Tag();
40     if (l == r) {
41         val[x] = a[l];
42         return;
43     }
44     int mid = (l + r) >> 1;
45     build(ls(x), l, mid, a);
46     build(rs(x), mid + 1, r, a);
47     pull(x);
48 }
49
50 void update(int x, int l, int r, int ql, int qr, const Tag &v) {
51     if (ql <= l && r <= qr) {
52         apply(x, v);
53         return;
54     }
55     push(x);
56     int mid = (l + r) >> 1;
57     if (ql <= mid) update(ls(x), l, mid, ql, qr, v);
58     if (qr > mid) update(rs(x), mid + 1, r, ql, qr, v);
59     pull(x);
60 }
61
62 void modify(int x, int l, int r, int p, const Info &v) {
63     if (l == r) {
64         val[x] = v;
65         tag[x] = Tag();
66         return;
67     }
68     push(x);
69     int mid = (l + r) >> 1;
70     if (p <= mid) modify(ls(x), l, mid, p, v);
71     else modify(rs(x), mid + 1, r, p, v);
72     pull(x);
73 }
74
75 Info ask(int x, int l, int r, int ql, int qr) {
76     if (ql <= l && r <= qr) return val[x];
77     push(x);
78     int mid = (l + r) >> 1;
79     if (qr <= mid) return ask(ls(x), l, mid, ql, qr);
80     if (ql > mid) return ask(rs(x), mid + 1, r, ql, qr);
81     return ask(ls(x), l, mid, ql, qr) + ask(rs(x), mid + 1, r, ql, qr);
82 }
83
84 public:
85     explicit LazySegTree(int _n = 0) {
86         if (_n) init(_n);
87     }
88
89     void init(int _n) {
90         assert(_n > 0);
91         n = _n;
92         val.assign(4 * n + 5, Info());
93         tag.assign(4 * n + 5, Tag());
94     }
95
96     template <class A> void build(const A &a) {
97         assert(n > 0);
98         build(1, 1, n, a);
99     }
100
101     void update(int l, int r, const Tag &v) {
102         if (l > r) return;
103         assert(1 <= l && r <= n);
104         update(1, 1, n, l, r, v);
105     }
106
107     void modify(int p, const Info &v) {

```

```

108     assert(1 <= p && p <= n);
109     modify(1, 1, n, p, v);
110 }
111
112 Info ask(int l, int r) {
113     if (l > r) return Info();
114     assert(1 <= l && r <= n);
115     return ask(1, 1, n, l, r);
116 }
117
118 Info ask() const {
119     return val[1];
120 }
121 };

```

03 - 树状数组 (BIT, with Int) .cpp

```

1  /*
2  用途：一维树状数组，默认维护单点加、前缀和与区间和。
3
4  接口（均为 1-indexed）：modify(p,x)、ask(p)、ask(l,r)、askKth(k)。
5  update(l,r,x) 是差分用法：区间加、单点查询；不要与区间和语义混用。
6  askKth(k) 返回最小的 p 使前缀和 >= k；要求各点权非负，非法 k 返回 n + 1。
7  */
8  template <class T> struct BIT {
9      int n = 0;
10     vector<T> bit;
11
12     BIT() = default;
13     explicit BIT(int n) {
14         init(n);
15     }
16
17     void init(int _n) {
18         n = _n;
19         bit.assign(n + 1, T{});
20     }
21
22     template <class A> void build(const A &a) {
23         fill(bit.begin(), bit.end(), T{});
24         for (int i = 1; i <= n; ++i) {
25             bit[i] += a[i];
26             int j = i + (i & -i);
27             if (j <= n) bit[j] += bit[i];
28         }
29     }
30
31     void modify(int x, const T &v) {
32         assert(1 <= x && x <= n);
33         for (; x <= n; x += x & -x) bit[x] += v;
34     }
35
36     void update(int l, int r, const T &v) {
37         if (l > r) return;
38         assert(1 <= l && r <= n);
39         modify(l, v);
40         if (r < n) modify(r + 1, -v);
41     }
42
43     T ask(int x) const {
44         x = min(x, n);
45         T res{};
46         for (; x > 0; x -= x & -x) res += bit[x];
47         return res;
48     }
49
50     T ask(int l, int r) const {
51         if (l > r) return T{};
52         assert(1 <= l && r <= n);
53         return ask(r) - ask(l - 1);
54     }
55
56     int askKth(const T &k) const {
57         if (!(k > T{})) return 1;
58         int pos = 0;
59         T pre{};

```

```

60     int stp = 1;
61     while ((stp <= 1) <= n) stp <= 1;
62     for (; stp; stp >= 1) {
63         int nxt = pos + stp;
64         if (nxt <= n && pre + bit[nxt] < k) {
65             pre += bit[nxt];
66             pos = nxt;
67         }
68     }
69     return pos + 1;
70 }
71 };

```

03A - 区间加区间和树状数组.cpp

```

1  /*
2  用途：双树状数组，支持区间加和区间求和。
3
4  下标和区间均为 1-indexed 闭区间。
5  接口：update(l,r,v)、ask(x)、askSum(l,r)。每次 O(log n)。
6  */
7  template <class T> struct BIT {
8      int n = 0;
9      vector<T> b1, b2;
10
11      BIT() = default;
12      explicit BIT(int n) {
13          init(n);
14      }
15
16      void init(int _n) {
17          n = _n;
18          b1.assign(n + 1, T{});
19          b2.assign(n + 1, T{});
20      }
21
22      void mod0(vector<T> &bit, int x, const T &v) {
23          for (; x <= n; x += x & -x) bit[x] += v;
24      }
25
26      T ask0(const vector<T> &bit, int x) const {
27          T res{};
28          for (; x > 0; x -= x & -x) res += bit[x];
29          return res;
30      }
31
32      void update(int l, int r, const T &v) {
33          if (l > r) return;
34          assert(1 <= l && r <= n);
35          mod0(b1, l, v);
36          mod0(b2, l, v * (l - 1));
37          if (r < n) {
38              mod0(b1, r + 1, -v);
39              mod0(b2, r + 1, -v * r);
40          }
41      }
42
43      T ask(int x) const {
44          x = min(x, n);
45          return ask0(b1, x) * x - ask0(b2, x);
46      }
47
48      T askSum(int l, int r) const {
49          if (l > r) return T{};
50          assert(1 <= l && r <= n);
51          return ask(r) - ask(l - 1);
52      }
53 };

```

03B - 二维树状数组.cpp

```

1  /*
2  用途：稠密二维树状数组，支持单点加与矩形和。
3  约定：坐标均为 1-indexed；ask(x1,y1,x2,y2) 为闭矩形。
4  复杂度：修改/查询 O(log n log m)，空间 O(nm)。坐标很大且操作稀疏时应离线压缩或改动态结构。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T> struct BIT {

```

```

8   int n = 0, m = 0;
9   vector<vector<T>>> t;
10
11   BIT() = default;
12   BIT(int n, int m) { init(n, m); }
13
14   void init(int _n, int _m) {
15       assert(_n >= 0 && _m >= 0);
16       n = _n, m = _m;
17       t.assign(n + 1, vector<T>(m + 1));
18   }
19
20   void modify(int x, int y, const T& v) {
21       assert(1 <= x && x <= n && 1 <= y && y <= m);
22       for (int i = x; i <= n; i += i & -i)
23           for (int j = y; j <= m; j += j & -j) t[i][j] += v;
24   }
25
26   T ask(int x, int y) const {
27       x = min(x, n), y = min(y, m);
28       T r{};
29       for (int i = x; i > 0; i -= i & -i)
30           for (int j = y; j > 0; j -= j & -j) r += t[i][j];
31       return r;
32   }
33
34   T ask(int x1, int y1, int x2, int y2) const {
35       if (x1 > x2 || y1 > y2) return T{};
36       assert(1 <= x1 && x2 <= n && 1 <= y1 && y2 <= m);
37       return ask(x2, y2) - ask(x1 - 1, y2) - ask(x2, y1 - 1) + ask(x1 - 1, y1 - 1);
38   }
39 };

```

03C - 二维树状数组（区间加、矩形和）.cpp

```

1  /*
2  用途：二维区间加、二维矩形和。
3  约定：坐标均为 1-indexed: update(x1,y1,x2,y2,v) 修改闭矩形, ask 为前缀或闭矩形和。
4  复杂度：单次操作  $O(\log n \log m)$ ，空间  $O(nm)$ 。坐标很大且操作稀疏时先离线压缩。
5  */
6  template <class T> struct BIT {
7      int n = 0, m = 0;
8      array<vector<vector<T>>>, 4> t;
9
10     BIT() = default;
11     BIT(int n, int m) { init(n, m); }
12
13     void init(int _n, int _m) {
14         assert(_n >= 0 && _m >= 0);
15         n = _n, m = _m;
16         for (auto& a : t) a.assign(n + 1, vector<T>(m + 1));
17     }
18
19     void add0(int k, int x, int y, const T& v) {
20         for (int i = x; i <= n; i += i & -i)
21             for (int j = y; j <= m; j += j & -j) a(k, i, j) += v;
22     }
23
24     T ask0(int k, int x, int y) const {
25         T r{};
26         for (int i = x; i > 0; i -= i & -i)
27             for (int j = y; j > 0; j -= j & -j) r += t[k][i][j];
28         return r;
29     }
30
31     T& a(int k, int x, int y) {
32         return t[k][x][y];
33     }
34
35     void upd0(int x, int y, const T& v) {
36         add0(0, x, y, v);
37         add0(1, x, y, v * (x - 1));
38         add0(2, x, y, v * (y - 1));
39         add0(3, x, y, v * (x - 1) * (y - 1));
40     }
41
42     void update(int x1, int y1, int x2, int y2, const T& v) {
43         if (x1 > x2 || y1 > y2) return;
44         assert(1 <= x1 && x2 <= n && 1 <= y1 && y2 <= m);
45         upd0(x1, y1, v);

```



```

46     if (x2 < n) upd0(x2 + 1, y1, -v);
47     if (y2 < m) upd0(x1, y2 + 1, -v);
48     if (x2 < n && y2 < m) upd0(x2 + 1, y2 + 1, v);
49 }
50
51 T ask(int x, int y) const {
52     x = min(x, n), y = min(y, m);
53     if (x <= 0 || y <= 0) return T{};
54     T p = ask0(0, x, y);
55     return p * x * y - ask0(1, x, y) * y - ask0(2, x, y) * x + ask0(3, x, y);
56 }
57
58 T ask(int x1, int y1, int x2, int y2) const {
59     if (x1 > x2 || y1 > y2) return T{};
60     assert(1 <= x1 && x2 <= n && 1 <= y1 && y2 <= m);
61     return ask(x2, y2) - ask(x1 - 1, y2) - ask(x2, y1 - 1) + ask(x1 - 1, y1 - 1);
62 }
63 };

```

04A - ST表 (ST, with Int) .cpp

```

1  /*
2  用途：静态区间最值 ST 表，查询最大/最小值及其最左位置。
3
4  约定：数组和查询均为 1-indexed 闭区间；askMax/askMin 自动交换 l、r。
5  仅适用于 max/min/gcd 等幂等操作，区间和请用“04B - 可结合区间查询 (Disjoint Sparse Table)” 。
6  预处理  $O(n \log n)$ ，每次查询  $O(1)$ 。
7  性能：为返回最左位置，表项额外保存 int 下标；极限内存不足时可只将下标改为 i32。
8  */
9  template <class T>
10 struct ST {
11     int n = 0, lg = 0;
12     vector<vector<pair<T, int>>> mx, mn;
13
14     ST() = default;
15     explicit ST(int n) {
16         init(n);
17     }
18
19     void init(int _n) {
20         assert(_n > 0);
21         n = _n;
22         lg = __lg(n);
23         mx.assign(lg + 1, vector<pair<T, int>>(n + 1));
24         mn.assign(lg + 1, vector<pair<T, int>>(n + 1));
25     }
26
27     template <class A>
28     void build(const A &a) {
29         assert(n > 0);
30         for (int i = 1; i <= n; ++i) {
31             mx[0][i] = {a[i], -i}; // 值相同时优先较小下标。
32             mn[0][i] = {a[i], i};
33         }
34         for (int k = 1; k <= lg; ++k) {
35             int len = 1 << k;
36             int h = len >> 1;
37             for (int l = 1; l + len - 1 <= n; ++l) {
38                 mx[k][l] = max(mx[k - 1][l], mx[k - 1][l + h]);
39                 mn[k][l] = min(mn[k - 1][l], mn[k - 1][l + h]);
40             }
41         }
42     }
43
44     pair<T, int> askMax(int l, int r) const {
45         if (l > r) swap(l, r);
46         assert(1 <= l && r <= n);
47         int k = __lg(r - l + 1);
48         auto res = max(mx[k][l], mx[k][r - (1 << k) + 1]);
49         return {res.first, -res.second};
50     }
51
52     pair<T, int> askMin(int l, int r) const {
53         if (l > r) swap(l, r);
54         assert(1 <= l && r <= n);
55         int k = __lg(r - l + 1);
56         return min(mn[k][l], mn[k][r - (1 << k) + 1]);
57     }
58 };

```

04B - 可结合区间查询 (Disjoint Sparse Table) .cpp

```

1  /*
2  用途: Disjoint Sparse Table, 支持任意结合操作的静态区间查询, 例如区间和、gcd、矩阵乘法。
3
4  数组与查询为 0-indexed: ask(l,r) 的 r 为闭端点, 要求 l <= r。
5  Op 必须满足结合律, 不需要幂等, 也不需要单位元。预处理 O(n log n), 查询 O(1)。
6  */
7  template <class T, class Op>
8  struct DST {
9      int n = 0, lg = 0;
10     Op op;
11     vector<T> val;
12     vector<vector<T>> tab;
13
14     DST() = default;
15     DST(const vector<T> &a, Op op = Op()) : op(move(op)) {
16         build(a);
17     }
18
19     void build(const vector<T> &a) {
20         val = a;
21         n = val.size();
22         if (n == 0) {
23             lg = 0;
24             tab.clear();
25             return;
26         }
27         lg = __lg(max<int>(1, n - 1)) + 1;
28         tab.assign(lg, vector<T>(n));
29         for (int lev = 0; lev < lg; ++lev) {
30             int h = 1 << lev;
31             int blk = h << 1;
32             for (int beg = 0; beg < n; beg += blk) {
33                 int mid = min(beg + h, n);
34                 int end = min(beg + blk, n);
35                 if (mid > beg) {
36                     tab[lev][mid - 1] = val[mid - 1];
37                     for (int i = mid - 2; i >= beg; --i) tab[lev][i] = op(val[i], tab[lev][i + 1]);
38                 }
39                 if (mid < end) {
40                     tab[lev][mid] = val[mid];
41                     for (int i = mid + 1; i < end; ++i) tab[lev][i] = op(tab[lev][i - 1], val[i]);
42                 }
43             }
44         }
45     }
46
47     T ask(int l, int r) const {
48         assert(0 <= l && l <= r && r < n);
49         if (l == r) return val[l];
50         int lev = __lg(l ^ r);
51         return op(tab[lev][l], tab[lev][r]);
52     }
53 };

```

04C - 线性RMQ (状压) .cpp

```

1  /*
2  用途: 静态幂等最值 RMQ: 64 位块内单调栈状压, 块间 Sparse Table。
3  约定: 数组与查询均为 0-indexed, ask/askPos 使用闭区间 [l,r]: 相等时返回最左位置。
4  复杂度: 字长模型下 O(n) 预处理、O(1) 查询, 空间 O(n)。Cmp 默认为求最小值。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7
8  using u64 = uint64_t;
9
10 template <class T, class Cmp = less<T>> struct LinearRMQ {
11     static constexpr int B = 64;
12
13     int n = 0, nb = 0;
14     Cmp cmp;
15     vector<T> a;
16     vector<int> pre, suf;
17     vector<u64> stk;
18     vector<vector<int>> st;
19
20     LinearRMQ() = default;
21     explicit LinearRMQ(const vector<T> &a, Cmp cmp = Cmp()) : cmp(move(cmp)) { build(a); }

```

```

22
23 int best(int x, int y) const {
24     if (x < 0) return y;
25     if (y < 0) return x;
26     if (cmp(a[y], a[x])) return y;
27     if (cmp(a[x], a[y])) return x;
28     return min(x, y);
29 }
30
31 void build(const vector<T>& v) {
32     a = v;
33     n = a.size();
34     if (!n) {
35         nb = 0, pre.clear(), suf.clear(), stk.clear(), st.clear();
36         return;
37     }
38     nb = (n + B - 1) / B;
39     pre.resize(n), suf.resize(n), stk.resize(n);
40     vector<int> b(nb);
41     for (int q = 0; q < nb; ++q) {
42         int l = q * B, r = min<int>(n, l + B);
43         u64 s = 0;
44         for (int i = l; i < r; ++i) {
45             pre[i] = i == l ? i : best(pre[i - 1], i);
46             while (s) {
47                 int k = 63 - __builtin_clzll(s);
48                 if (!cmp(a[i], a[l + k])) break;
49                 s ^= u64(1) << k;
50             }
51             s |= u64(1) << (i - l);
52             stk[i] = s;
53         }
54         for (int i = r; i-- > l;) suf[i] = i + 1 == r ? i : best(i, suf[i + 1]);
55         b[q] = pre[r - 1];
56     }
57     int lg = nb == 1 ? 0 : 31 - __builtin_clz(nb);
58     st.assign(lg + 1, vector<int>(nb));
59     st[0] = move(b);
60     for (int k = 1; k <= lg; ++k)
61         for (int i = 0; i + (1LL << k) <= nb; ++i)
62             st[k][i] = best(st[k - 1][i], st[k - 1][i + (1LL << (k - 1))]);
63 }
64
65 int askPos(int l, int r) const {
66     assert(0 <= l && l <= r && r < n);
67     int x = l / B, y = r / B;
68     if (x == y) {
69         u64 s = stk[r] >> (l - x * B);
70         return l + __builtin_ctzll(s);
71     }
72     int z = best(suf[l], pre[r]);
73     int ql = x + 1, qr = y - 1;
74     if (ql <= qr) {
75         int k = 31 - __builtin_clz(qr - ql + 1);
76         z = best(z, best(st[k][ql], st[k][qr - (1LL << k) + 1]));
77     }
78     return z;
79 }
80
81 const T& ask(int l, int r) const { return a[askPos(l, r)]; }
82 };

```

05 - Bitset.cpp

```

1  /*
2  用途: 运行时长度 Bitset, 支持位运算、移位和枚举置位。
3
4  下标为 0..askSz()-1, askNxt/askPre 找不到时返回 askSz()。
5  resize(n, true) 会将“新增的位”置 1, 行为与 vector 的扩容语义一致。
6  */
7
8  using u64 = uint64_t;
9
10 class Bitset {
11 public:
12     static constexpr int W = 64;
13
14 private:
15     vector<u64> a;
16     int nb = 0;

```

```

17     u64 msk = ~u64(0);
18
19     static int wds(int bit) {
20         if (bit < 0) throw invalid_argument("negative bitset size");
21         return (bit + W - 1) / W;
22     }
23
24     void trim() {
25         int rem = nb % W;
26         msk = rem == 0 ? ~u64(0) : (u64(1) << rem) - 1;
27         if (!a.empty()) a.back() &= msk;
28     }
29
30     void check(int pos) const {
31         if (pos < 0 || pos >= nb) throw out_of_range("bit index out of range");
32     }
33
34 public:
35     Bitset() = default;
36     explicit Bitset(int n, bool val = false) {
37         resize(n, val);
38     }
39
40     int askSz() const noexcept {
41         return nb;
42     }
43
44     bool askEmp() const noexcept {
45         return nb == 0;
46     }
47
48     void reset() noexcept {
49         fill(all(a), 0);
50     }
51
52     void set() noexcept {
53         fill(all(a), ~u64(0));
54         if (!a.empty()) a.back() &= msk;
55     }
56
57     void resize(int n, bool val = false) {
58         int nw = wds(n);
59         int ob = nb;
60         if (n == ob) return;
61
62         if (n > ob) {
63             a.resize(nw, val ? ~u64(0) : 0);
64             if (val) {
65                 int fw = ob / W;
66                 int fb = ob % W;
67                 if (fb < nw) {
68                     if (fb) a[fw] |= (~u64(0) << fb);
69                     else a[fw] = ~u64(0);
70                     for (int i = fw + 1; i < nw; ++i) a[i] = ~u64(0);
71                 }
72             }
73         } else {
74             a.resize(nw);
75         }
76         nb = n;
77         trim();
78     }
79
80     void modify(int pos, bool v) {
81         check(pos);
82         u64 b = u64(1) << (pos % W);
83         if (v) a[pos / W] |= b;
84         else a[pos / W] &= ~b;
85     }
86
87     void flip(int pos) {
88         check(pos);
89         a[pos / W] ^= u64(1) << (pos % W);
90     }
91
92     bool ask(int pos) const {
93         check(pos);
94         return (a[pos / W] >> (pos % W)) & 1;
95     }
96
97     bool askAny() const noexcept {
98         for (u64 wd : a) {

```

```

99         if (wd) return true;
100     }
101     return false;
102 }
103
104 bool askNone() const noexcept {
105     return !askAny();
106 }
107
108 bool askAll() const noexcept {
109     if (a.empty()) return true;
110     for (int i = 0; i + 1 < a.size(); ++i) {
111         if (a[i] != ~u64(0)) return false;
112     }
113     return a.back() == msk;
114 }
115
116 int askCnt() const noexcept {
117     int ans = 0;
118     for (u64 wd : a) ans += __builtin_popcountll(wd);
119     return ans;
120 }
121
122 Bitset &operator&=(const Bitset &o) {
123     assert(nb == o.nb);
124     for (int i = 0; i < a.size(); ++i) a[i] &= o.a[i];
125     return *this;
126 }
127
128 Bitset &operator|=(const Bitset &o) {
129     assert(nb == o.nb);
130     for (int i = 0; i < a.size(); ++i) a[i] |= o.a[i];
131     if (!a.empty()) a.back() &= msk;
132     return *this;
133 }
134
135 Bitset &operator^=(const Bitset &o) {
136     assert(nb == o.nb);
137     for (int i = 0; i < a.size(); ++i) a[i] ^= o.a[i];
138     if (!a.empty()) a.back() &= msk;
139     return *this;
140 }
141
142 Bitset operator~() const {
143     Bitset res(*this);
144     for (u64 &wd : res.a) wd = ~wd;
145     if (!res.a.empty()) res.a.back() &= res.msk;
146     return res;
147 }
148
149 friend Bitset operator&(Bitset lhs, const Bitset &rhs) {
150     return lhs &= rhs;
151 }
152
153 friend Bitset operator|(Bitset lhs, const Bitset &rhs) {
154     return lhs |= rhs;
155 }
156
157 friend Bitset operator^(Bitset lhs, const Bitset &rhs) {
158     return lhs ^= rhs;
159 }
160
161 Bitset &operator<=(int sh) {
162     if (sh < 0) throw invalid_argument("negative bit shift");
163     if (sh == 0 || nb == 0) return *this;
164     int sw = sh / W;
165     int sb = sh % W;
166     int wds = a.size();
167     if (sw >= wds) {
168         reset();
169         return *this;
170     }
171     if (sb == 0) {
172         for (int i = wds - 1; i >= sw; --i) a[i] = a[i - sw];
173     } else {
174         for (int i = wds - 1; i >= sw; --i) {
175             u64 hi = a[i - sw] << sb;
176             u64 low = i - sw > 0 ? a[i - sw - 1] >> (W - sb) : 0;
177             a[i] = hi | low;
178         }
179     }
180     for (int i = 0; i < sw; ++i) a[i] = 0;

```

```

181     a.back() &= msk;
182     return *this;
183 }
184
185 Bitset &operator>>=(int sh) {
186     if (sh < 0) throw invalid_argument("negative bit shift");
187     if (sh == 0 || nb == 0) return *this;
188     int sw = sh / W;
189     int sb = sh % W;
190     int wds = a.size();
191     if (sw >= wds) {
192         reset();
193         return *this;
194     }
195     if (sb == 0) {
196         for (int i = 0; i + sw < wds; ++i) a[i] = a[i + sw];
197     } else {
198         for (int i = 0; i + sw < wds; ++i) {
199             u64 low = a[i + sw] >> sb;
200             u64 hi = i + sw + 1 < wds ? a[i + sw + 1] << (W - sb) : 0;
201             a[i] = low | hi;
202         }
203     }
204     for (int i = wds - sw; i < wds; ++i) a[i] = 0;
205     a.back() &= msk;
206     return *this;
207 }
208
209 friend Bitset operator<<(Bitset lhs, int sh) {
210     return lhs <<= sh;
211 }
212
213 friend Bitset operator>>(Bitset lhs, int sh) {
214     return lhs >>= sh;
215 }
216
217 int askNxt(int pos) const noexcept {
218     if (pos < 0) pos = 0;
219     if (pos >= nb) return nb;
220     int id = pos / W;
221     u64 wd = a[id] & (~u64(0) << (pos % W));
222     if (wd) return id * W + __builtin_ctzll(wd);
223     for (++id; id < a.size(); ++id) {
224         if (a[id]) return id * W + __builtin_ctzll(a[id]);
225     }
226     return nb;
227 }
228
229 int askPre(int pos) const noexcept {
230     if (pos < 0 || nb == 0) return nb;
231     if (pos >= nb) pos = nb - 1;
232     int id = pos / W;
233     int off = pos % W;
234     u64 msk = off == 63 ? ~u64(0) : (u64(1) << (off + 1)) - 1;
235     u64 wd = a[id] & msk;
236     if (wd) return id * W + 63 - __builtin_clzll(wd);
237     while (id > 0) {
238         --id;
239         if (a[id]) return id * W + 63 - __builtin_clzll(a[id]);
240     }
241     return nb;
242 }
243
244 template <class F>
245 void each(F f) const {
246     for (int i = 0; i < a.size(); ++i) {
247         u64 wd = a[i];
248         while (wd) {
249             int bit = __builtin_ctzll(wd);
250             f(i * W + bit);
251             wd &= wd - 1;
252         }
253     }
254 }
255
256 string str() const {
257     string res;
258     res.reserve(nb);
259     for (int i = nb - 1; i >= 0; --i) res.push_back(ask(i) ? '1' : '0');
260     return res;
261 }
262

```

```
263     const u64 *askData() const noexcept {  
264         return a.empty() ? nullptr : a.data();  
265     }  
266  
267     u64 *askData() noexcept {  
268         return a.empty() ? nullptr : a.data();  
269     }  
270  
271     static Bitset read(const string &s) {  
272         Bitset res(s.size());  
273         for (int i = 0; i < s.size(); ++i) {  
274             char c = s[s.size() - 1 - i];  
275             if (c == '1') res.modify(i, true);  
276             else if (c != '0') throw invalid_argument("invalid char in bitstring");  
277         }  
278         return res;  
279     }  
280 };
```

数据结构：顺序统计、Trie、堆、平衡树与哈希

22 个代码模板 · 林月 · v1.0.0 · 2026.8.1

06 - 小波矩阵 (WaveletMatrix) .cpp

```

1  /*
2  用途：压缩值域 Wavelet Matrix，支持 0-indexed 半开区间 [l, r) 的访问、k 小、计数和频率查询。
3  askLT/askLE 分别表示 < x / <= x: askCnt(l,r,L,R) 统计 [L,R] 内元素个数。
4  预处理 O(n log sigma)，单次查询 O(log sigma)。
5  性能：层前缀、位置和压缩编号默认使用 int；极限内存不足时可只将这些纯计数与下标改为 i32。
6  */
7
8  using u64 = uint64_t;
9
10 template <class T> struct Wavelet {
11     struct BitVec {
12         int n;
13         vector<u64> bit;
14         vector<int> pre;
15
16         BitVec() : n(0) {}
17         BitVec(int _n) {
18             init(_n);
19         }
20
21         void init(int _n) {
22             n = _n;
23             int blk = (n >> 6) + 1;
24             bit.assign(blk, 0);
25             pre.assign(blk + 1, 0);
26         }
27
28         inline void modify(int i) {
29             bit[i >> 6] |= (1ull << (i & 63));
30         }
31
32         void build() {
33             for (int i = 0; i < bit.size(); ++i) {
34                 pre[i + 1] = pre[i] + __builtin_popcountll(bit[i]);
35             }
36         }
37
38         inline int askl(int pos) const {
39             return pre[pos >> 6] + __builtin_popcountll(bit[pos >> 6] & ((1ull << (pos & 63)) - 1));
40         }
41
42         inline int ask0(int pos) const {
43             return pos - askl(pos);
44         }
45
46         inline int ask(int i) const {
47             return (bit[i >> 6] >> (i & 63)) & 1;
48         }
49     };
50
51     int n, len;
52     vector<BitVec> mat;
53     vector<int> mid;
54     vector<T> val;
55
56     Wavelet() : n(0), len(0) {}
57
58     Wavelet(vector<T> dat) {
59         build(dat);
60     }
61
62     void build(vector<T> dat) {
63         n = dat.size();
64         val = dat;
65         sort(val.begin(), val.end());
66         val.erase(unique(val.begin(), val.end()), val.end());
67         int sig = val.size();
68         if (sig == 0) {
69             len = 0;
70             mat.clear();
71             mid.clear();

```



```

72     return;
73 }
74
75 vector<int> cod(n);
76 for (int i = 0; i < n; ++i) {
77     cod[i] = lower_bound(val.begin(), val.end(), dat[i]) - val.begin();
78 }
79
80 len = 0;
81 while ((1ll << len) < sig) ++len;
82 if (len == 0) len = 1;
83
84 mat.assign(len, BitVec());
85 mid.assign(len, 0);
86
87 for (int lev = 0; lev < len; ++lev) {
88     mat[lev].init(n);
89 }
90
91 vector<int> nxt(n);
92 for (int lev = len - 1; lev >= 0; --lev) {
93     auto &bv = mat[lev];
94     for (int i = 0; i < n; ++i) {
95         if ((cod[i] >> lev) & 1) bv.modify(i);
96     }
97     bv.build();
98
99     int cnt = 0;
100    for (int i = 0; i < n; ++i) {
101        if (!((cod[i] >> lev) & 1)) {
102            nxt[cnt++] = cod[i];
103        }
104    }
105    int idx = cnt;
106    for (int i = 0; i < n; ++i) {
107        if ((cod[i] >> lev) & 1) {
108            nxt[idx++] = cod[i];
109        }
110    }
111    mid[lev] = cnt;
112    cod.swap(nxt);
113 }
114 }
115
116 inline int askSz() const {
117     return n;
118 }
119 inline bool askEmp() const {
120     return n == 0;
121 }
122
123 T ask(int pos) const {
124     assert(pos >= 0 && pos < n);
125     int cod = 0;
126     for (int lev = len - 1; lev >= 0; --lev) {
127         int b = mat[lev].ask(pos);
128         if (b) {
129             cod |= (1 << lev);
130             pos = mid[lev] + mat[lev].ask1(pos);
131         } else {
132             pos = mat[lev].ask0(pos);
133         }
134     }
135     return val[cod];
136 }
137
138 T askKth(int l, int r, int k) const {
139     assert(l >= 0 && l <= r && r <= n && k >= 0 && k < r - l);
140     int cod = 0;
141     for (int lev = len - 1; lev >= 0; --lev) {
142         const BitVec &bv = mat[lev];
143         int zl = bv.ask0(l);
144         int zr = bv.ask0(r);
145         int zs = zr - zl;
146         if (k < zs) {
147             l = zl;
148             r = zr;
149         } else {
150             k -= zs;
151             cod |= (1 << lev);
152             l = mid[lev] + bv.ask1(l);
153             r = mid[lev] + bv.ask1(r);

```

```

154     }
155     }
156     return val[cod];
157 }
158
159 int askLT(int l, int r, T x) const {
160     if (l >= r || val.empty()) return 0;
161     auto it = lower_bound(val.begin(), val.end(), x);
162     if (it == val.end()) return r - l;
163     int xc = it - val.begin();
164     if (xc == 0) return 0;
165     int res = 0;
166     for (int lev = len - 1; lev >= 0; --lev) {
167         const BitVec &bv = mat[lev];
168         int zl = bv.ask0(l), zr = bv.ask0(r);
169         if ((xc >> lev) & 1) {
170             res += zr - zl;
171             l = mid[lev] + bv.ask1(l);
172             r = mid[lev] + bv.ask1(r);
173         } else {
174             l = zl;
175             r = zr;
176         }
177     }
178     return res;
179 }
180
181 int askLE(int l, int r, T x) const {
182     if (l >= r || val.empty()) return 0;
183     auto it = upper_bound(val.begin(), val.end(), x);
184     if (it == val.end()) return r - l;
185     int xc = it - val.begin();
186     if (xc == 0) return 0;
187     int res = 0;
188     for (int lev = len - 1; lev >= 0; --lev) {
189         const BitVec &bv = mat[lev];
190         int zl = bv.ask0(l), zr = bv.ask0(r);
191         if ((xc >> lev) & 1) {
192             res += zr - zl;
193             l = mid[lev] + bv.ask1(l);
194             r = mid[lev] + bv.ask1(r);
195         } else {
196             l = zl;
197             r = zr;
198         }
199     }
200     return res;
201 }
202
203 int askCnt(int l, int r, T mn, T mx) const {
204     if (mn > mx) return 0;
205     return askLE(l, r, mx) - askLT(l, r, mn);
206 }
207
208 int askFreq(int l, int r, T x) const {
209     if (l >= r || val.empty()) return 0;
210     auto it = lower_bound(val.begin(), val.end(), x);
211     if (it == val.end() || *it != x) return 0;
212     int cod = it - val.begin();
213     for (int lev = len - 1; lev >= 0; --lev) {
214         const BitVec &bv = mat[lev];
215         if (!(cod >> lev) & 1) {
216             l = bv.ask0(l);
217             r = bv.ask0(r);
218         } else {
219             l = mid[lev] + bv.ask1(l);
220             r = mid[lev] + bv.ask1(r);
221         }
222     }
223     return r - l;
224 }
225 };

```

06A - 区间第k大 (WaveletMatrix) .cpp

```

1  /*
2  用途：只关注区间第 k 小/第 k 大的轻量 Wavelet Matrix 封装。
3
4  区间和 k 均为 0-indexed 闭区间：askKth(l,r,k) 是第 k 小，askKth<true>(l,r,k) 是第 k 大。
5  构造后静态查询，预处理  $O(n \log \sigma)$ 、查询  $O(\log \sigma)$ 。

```

```

6 性能: 默认使用展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
7  */
8
9  using u64 = uint64_t;
10
11  template <typename T> class Wavelet {
12  private:
13      struct Core {
14          struct BitVec {
15              static constexpr int W = 64;
16              static int ask64(u64 x, int i) {
17                  return __builtin_popcountll(x & ((1ull << i) - 1));
18              }
19  #pragma pack(4)
20              struct Block {
21                  u64 bit;
22                  int sum;
23              };
24  #pragma pack()
25              int n, z;
26              vector<Block> b;
27              BitVec(int _n = 0) : n(_n), b(n / W + 1) {}
28              int operator[](int i) const {
29                  return b[i / W].bit >> i % W & 1;
30              }
31              void modify(int i) {
32                  b[i / W].bit |= u64(1) << i % W;
33              }
34              void build() {
35                  for (int j = 0; j < n / W; ++j) b[j + 1].sum = b[j].sum + __builtin_popcountll(b[j].bit);
36                  z = ask0(n);
37              }
38              int ask0(int i) const {
39                  return i - ask1(i);
40              }
41              int ask1(int i) const {
42                  auto &e = b[i / W];
43                  return e.sum + ask64(e.bit, i % W);
44              }
45          };
46
47          int n = 0, lg = 0;
48          vector<T> a;
49          vector<BitVec> bv;
50          Core(int _n = 0) : n(_n), a(n) {}
51          Core(const vector<T> &a) : n(a.size()), a(a) {
52              build();
53          }
54          T &operator[](int i) {
55              return a[i];
56          }
57          void build() {
58              if (n == 0) {
59                  lg = 0;
60                  bv.clear();
61                  return;
62              }
63              lg = __lg(max<T>(*max_element(begin(a), end(a)), 1)) + 1;
64              bv.assign(lg, n);
65              vector<T> cur = a, nxt(n);
66              for (auto h = lg; h--;) {
67                  for (int i = 0; i < n; ++i)
68                      if (cur[i] >> h & 1) bv[h].modify(i);
69                  bv[h].build();
70                  array<typename vector<T>::iterator, 2> it{begin(nxt), begin(nxt) + bv[h].z};
71                  for (int i = 0; i < n; ++i) *it[bv[h][i]]++ = cur[i];
72                  swap(cur, nxt);
73              }
74          }
75          T askKth(int l, int r, int k) const {
76              T res = 0;
77              for (auto h = lg; h--;) {
78                  auto l0 = bv[h].ask0(l), r0 = bv[h].ask0(r);
79                  if (k < r0 - l0)
80                      l = l0, r = r0;
81                  else {
82                      k -= r0 - l0;
83                      res |= T{1} << h;
84                      l += bv[h].z - l0;
85                      r += bv[h].z - r0;
86                  }
87              }
88          }

```

```

88     return res;
89 }
90 int askCnt(int l, int r, T ub) const {
91     if (ub >= T{1} << lg) return r - l;
92     int res = 0;
93     for (auto h = lg; h--;) {
94         auto l0 = bv[h].ask0(l), r0 = bv[h].ask0(r);
95         if (~ub >> h & 1)
96             l = l0, r = r0;
97         else {
98             res += r0 - l0;
99             l += bv[h].z - l0;
100            r += bv[h].z - r0;
101        }
102    }
103    return res;
104 }
105 int askCnt(int l, int r, T lb, T ub) const {
106     return askCnt(l, r, ub) - askCnt(l, r, lb);
107 }
108 };
109 pair<vector<int>, vector<T>> zip(const vector<T> &a) {
110     int n = a.size();
111     vector<pair<T, int>> p(n);
112     for (int i = 0; i < n; ++i) p[i] = {a[i], i};
113     sort(begin(p), end(p));
114     vector<int> na(n);
115     vector<T> v;
116     for (int k = 0, rnk = -1; k < n; ++k) {
117         if (k == 0 or p[k - 1].first < p[k].first) v.push_back(p[k].first), ++rnk;
118         na[p[k].second] = rnk;
119     }
120     return make_pair(na, v);
121 }
122 void check(int l, int r, int k) const {
123     assert(l >= 0 && l <= r && r < n && k <= r - l && k >= 0);
124 }
125 vector<T> vec;
126 Core wm;
127 int n = 0;
128 public:
129 Wavelet() {}
130 Wavelet(const vector<T> &v) : n(v.size()) {
131     auto [a, _v] = zip(v);
132     wm = Core(a);
133     vec = _v;
134 }
135 template <bool Rev = false> T askKth(int l, int r, int k) const {
136     check(l, r, k);
137     if constexpr (Rev) k = r - l - k;
138     r++;
139     return vec[wm.askKth(l, r, k)];
140 }
141 }
142 };

```

07 - 线性基.cpp

```

1  /*
2  用途：异或线性基，支持可表示性、最大异或、最小非空异或和第 k 小非空子集异或值。
3
4  T 必须是整数类型；内部按 T 的全部二进制位工作，不会遗漏最高位。
5  - insert(x): 插入 x，返回是否线性无关。
6  - ask(x): 是否能由已插入数异或得到 x。
7  - askMax(seed = 0): max(seed xor subsetXor)。
8  - askMin(): 最小非空子集异或值；若存在非空子集异或为 0 则返回 0。
9  - askKth(k): 第 k 小“非空子集”异或结果（1-indexed，重复结果只计一次），不存在返回 -1。
10
11 调用 askKth/askMin 前会自动化为最简基，复杂度  $O(B^2)$ ，B 为位数（最多 64）。
12  */
13
14 using u64 = uint64_t;
15
16 template <class T = u64>
17 struct LB {
18     static_assert(is_integral_v<T>);
19     using U = make_unsigned_t<T>;
20     static constexpr int B = numeric_limits<U>::digits;
21
22     array<U, B> bas{};

```

```

23 vector<U> ord;
24 int cnt = 0;
25 bool dep = false;
26
27 bool insert(T val) {
28     U x = static_cast<U>(val);
29     for (int bit = B - 1; bit >= 0; --bit) {
30         if (!(x >> bit) & U(1)) continue;
31         if (!bas[bit]) {
32             bas[bit] = x;
33             ++cnt;
34             ord.clear();
35             return true;
36         }
37         x ^= bas[bit];
38     }
39     dep = true;
40     return false;
41 }
42
43 bool ask(T val) const {
44     U x = static_cast<U>(val);
45     for (int bit = B - 1; bit >= 0; --bit) {
46         if (!(x >> bit) & U(1)) continue;
47         if (!bas[bit]) return false;
48         x ^= bas[bit];
49     }
50     return true;
51 }
52
53 T askMax(T sd = T{}) const {
54     U ans = static_cast<U>(sd);
55     for (int bit = B - 1; bit >= 0; --bit) {
56         if ((ans ^ bas[bit]) > ans) ans ^= bas[bit];
57     }
58     return static_cast<T>(ans);
59 }
60
61 void rebuild() {
62     for (int hi = B - 1; hi >= 0; --hi) {
63         if (!bas[hi]) continue;
64         for (int low = hi - 1; low >= 0; --low) {
65             if ((bas[hi] >> low) & U(1)) bas[hi] ^= bas[low];
66         }
67     }
68     ord.clear();
69     for (int bit = 0; bit < B; ++bit) {
70         if (bas[bit]) ord.push_back(bas[bit]);
71     }
72 }
73
74 T askMin() {
75     if (dep) return T{};
76     rebuild();
77     return ord.empty() ? T{} : static_cast<T>(ord.front());
78 }
79
80 T askKth(u64 k) {
81     if (k == 0) return T(-1);
82     rebuild();
83     u64 msk;
84     if (dep) {
85         msk = k - 1; // 非空子集也可得到 0。
86     } else {
87         msk = k; // 0 对应空集，非空结果从 mask = 1 开始。
88     }
89     if (cnt < 64 && msk >= (1ull << cnt)) return T(-1);
90     U ans = 0;
91     for (int i = 0; i < cnt; ++i) {
92         if ((msk >> i) & 1ull) ans ^= ord[i];
93     }
94     return static_cast<T>(ans);
95 }
96
97 void merge(const LB &o) {
98     for (int bit = B - 1; bit >= 0; --bit) {
99         if (o.bas[bit]) insert(static_cast<T>(o.bas[bit]));
100     }
101     dep = dep || o.dep;
102 }
103 };

```

08A - 字典树.cpp

```

1  /*
2  用途: 定长连续字符集 Trie, 支持插入、删除、单词/前缀计数与最长前缀词链。
3  用法: Trie<26, 'a'>; 节点转移、编号和计数使用 int。
4  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
5  */
6  template <int A = 26, char F = 'a'> struct Trie {
7
8      struct Node {
9          array<int, A> to{};
10         int ps = 0, ed = 0;
11     };
12
13     vector<Node> tr;
14
15     Trie() {
16         clear();
17     }
18
19     void clear() {
20         tr.assign(1, Node());
21     }
22
23     int id(char c) const {
24         int x = c - F;
25         assert(0 <= x && x < A);
26         return x;
27     }
28
29     void insert(const string& s) {
30         int u = 0;
31         ++tr[u].ps;
32         for (char c : s) {
33             int v = id(c);
34             if (!tr[u].to[v]) {
35                 tr[u].to[v] = tr.size();
36                 tr.emplace_back();
37             }
38             u = tr[u].to[v];
39             ++tr[u].ps;
40         }
41         ++tr[u].ed;
42     }
43
44     bool erase(const string& s) {
45         int u = 0;
46         vector<int> p{0};
47         for (char c : s) {
48             int v = id(c);
49             if (!tr[u].to[v]) return false;
50             u = tr[u].to[v];
51             p.push_back(u);
52         }
53         if (!tr[u].ed) return false;
54         --tr[u].ed;
55         for (int v : p) --tr[v].ps;
56         return true;
57     }
58
59     int ask(const string& s) const {
60         int u = 0;
61         for (char c : s) {
62             int v = id(c);
63             if (!tr[u].to[v]) return 0;
64             u = tr[u].to[v];
65         }
66         return tr[u].ed;
67     }
68
69     int askPre(const string& s) const {
70         int u = 0;
71         for (char c : s) {
72             int v = id(c);
73             if (!tr[u].to[v]) return 0;
74             u = tr[u].to[v];
75         }
76         return tr[u].ps;
77     }
78
79     int askMax() const {

```

```

80     int r = 0;
81     auto dfs = [&](auto&& go, int u, int d) -> void {
82         d += tr[u].ed > 0;
83         r = max(r, d);
84         for (int v : tr[u].to)
85             if (v && tr[v].ps) go(go, v, d);
86     };
87     dfs(dfs, 0, 0);
88     return r;
89 }
90
91 };

```

08B - 二进制字典树 (XOR Trie) .cpp

```

1  /*
2  用途: 二进制 Trie (XOR Trie), 维护可重集合并查询异或最值。
3
4  U 必须为无符号整数类型, B 默认为其完整位数。
5  接口: insert(x)、erase(x)、askMax(x)、askMin(x)、ask(x,lim)。
6  askMax/askMin 在空 Trie 时返回 nullopt; ask 统计所有 y 使 (x xor y) < lim。
7  */
8
9  using u64 = uint64_t;
10
11 template <class U = u64, int B = numeric_limits<U>::digits>
12 struct XorTrie {
13     static_assert(is_unsigned_v<U>);
14
15     struct Node {
16         array<int, 2> nxt{};
17         int cnt = 0;
18     };
19
20     vector<Node> tr = {Node{}};
21
22     int askSz() const {
23         return tr[0].cnt;
24     }
25
26     bool askEmp() const {
27         return askSz() == 0;
28     }
29
30     void insert(U val) {
31         int u = 0;
32         ++tr[u].cnt;
33         for (int bit = B - 1; bit >= 0; --bit) {
34             int b = (val >> bit) & U(1);
35             if (!tr[u].nxt[b]) {
36                 tr[u].nxt[b] = tr.size();
37                 tr.emplace_back();
38             }
39             u = tr[u].nxt[b];
40             ++tr[u].cnt;
41         }
42     }
43
44     bool erase(U val) {
45         int u = 0;
46         vector<int> pth = {0};
47         for (int bit = B - 1; bit >= 0; --bit) {
48             int b = (val >> bit) & U(1);
49             if (!tr[u].nxt[b] || tr[tr[u].nxt[b]].cnt == 0) return false;
50             u = tr[u].nxt[b];
51             pth.push_back(u);
52         }
53         for (int p : pth) --tr[p].cnt;
54         return true;
55     }
56
57     optional<U> askMax(U val) const {
58         if (askEmp()) return nullopt;
59         U ans = 0;
60         int u = 0;
61         for (int bit = B - 1; bit >= 0; --bit) {
62             int b = (val >> bit) & U(1);
63             int pre = b ^ 1;
64             if (tr[u].nxt[pre] && tr[tr[u].nxt[pre]].cnt) {
65                 ans |= U(1) << bit;

```

```

66         u = tr[u].nxt[pre];
67     } else {
68         u = tr[u].nxt[b];
69     }
70 }
71 return ans;
72 }
73
74 optional<U> askMin(U val) const {
75     if (askEmp()) return nullopt;
76     U ans = 0;
77     int u = 0;
78     for (int bit = B - 1; bit >= 0; --bit) {
79         int b = (val >> bit) & U(1);
80         if (tr[u].nxt[b] && tr[tr[u].nxt[b]].cnt) {
81             u = tr[u].nxt[b];
82         } else {
83             ans |= U(1) << bit;
84             u = tr[u].nxt[b ^ 1];
85         }
86     }
87     return ans;
88 }
89
90 int ask(U val, U lim) const {
91     int u = 0;
92     int ans = 0;
93     for (int bit = B - 1; bit >= 0; --bit) {
94         int xb = (val >> bit) & U(1);
95         int b = (lim >> bit) & U(1);
96         if (b) {
97             int sam = tr[u].nxt[xb];
98             if (sam) ans += tr[sam].cnt;
99             u = tr[u].nxt[xb ^ 1];
100         } else {
101             u = tr[u].nxt[xb];
102         }
103         if (!u) break;
104     }
105     return ans;
106 }
107 };

```

08C - PBDS前缀字典树（GNU环境）.cpp

```

1  /*
2  用途: GNU PBDS Patricia trie; 维护唯一字符串, 并直接枚举具有给定前缀的字符串。
3  限制: 仅 GCC/libstdc++ 可用, 节点开销较大; 只需插入/查询时优先使用更轻的 08A。
4
5  insert/erase/find/size 与集合相同; prefix_range(s) 返回 [l,r), 其中均以 s 为前缀。
6  键不可重复; 若需要出现次数, 把次数另存在哈希表中或改用 08A。
7  */
8  #ifndef int
9  #pragma push_macro("int")
10 #undef int
11 #define LYE_RESTORE_INT
12 #endif
13 #ifdef endl
14 #pragma push_macro("endl")
15 #undef endl
16 #define LYE_RESTORE_ENDL
17 #endif
18 #include <ext/pb_ds/assoc_container.hpp>
19 #include <ext/pb_ds/trie_policy.hpp>
20 #ifdef LYE_RESTORE_ENDL
21 #pragma pop_macro("endl")
22 #undef LYE_RESTORE_ENDL
23 #endif
24 #ifdef LYE_RESTORE_INT
25 #pragma pop_macro("int")
26 #undef LYE_RESTORE_INT
27 #endif
28
29 using PTrie = __gnu_pbds::trie<
30     string, __gnu_pbds::null_type,
31     __gnu_pbds::trie_string_access_traits<>,
32     __gnu_pbds::pat_trie_tag,
33     __gnu_pbds::trie_prefix_search_node_update>;

```


09A - 可并堆 (LeftistHeap, 可移植) .cpp

```

1  /*
2  用途: 可移植的左偏可并堆。默认 Cmp = less<T> 时为大根堆; 使用 greater<T> 可得到小根堆。
3
4  初始化 Heap(n) 后, h[i] 代表第 i 个堆, 支持 push/pop/ask/merge/clear。
5  merge(x, y) 是破坏性合并, 合并后 y 变为空堆。所有操作 O(log n)。
6  与 09B 解决同一问题; 非 GCC/libstdc++ 环境使用本模板。
7  */
8  template <class T, class Cmp = less<T>> struct Heap {
9      struct Node {
10         T v;
11         int l, r, d;
12     };
13
14     vector<Node> tr;
15     vector<int> rt, siz;
16     Cmp cmp;
17     int tot;
18
19     Heap() {}
20     Heap(int n) {
21         init(n);
22     }
23
24     void init(int n) {
25         tr.assign(1, {});
26         tr.reserve(n + 1);
27         rt.assign(n + 1, 0);
28         siz.assign(n + 1, 0);
29         tot = 0;
30     }
31
32     int merge0(int x, int y) {
33         if (!x || !y) return x | y;
34         if (cmp(tr[x].v, tr[y].v)) swap(x, y);
35         tr[x].r = merge0(tr[x].r, y);
36         if (tr[tr[x].l].d < tr[tr[x].r].d) swap(tr[x].l, tr[x].r);
37         tr[x].d = tr[tr[x].r].d + 1;
38         return x;
39     }
40
41     void merge(int x, int y) {
42         if (x == y) return;
43         rt[x] = merge0(rt[x], rt[y]);
44         siz[x] += siz[y];
45         rt[y] = siz[y] = 0;
46     }
47
48     void push(int x, const T &v) {
49         tr.push_back({v, 0, 0, 1});
50         rt[x] = merge0(rt[x], ++tot);
51         ++siz[x];
52     }
53
54     T ask(int x) {
55         return tr[rt[x]].v;
56     }
57
58     T pop(int x) {
59         int u = rt[x];
60         T res = tr[u].v;
61         rt[x] = merge0(tr[u].l, tr[u].r);
62         --siz[x];
63         return res;
64     }
65
66     void clear(int x) {
67         rt[x] = siz[x] = 0;
68     }
69
70     bool askEmp(int x) {
71         return siz[x] == 0;
72     }
73
74     int askSz(int x) {
75         return siz[x];
76     }
77
78     struct Ref {
79         Heap *h;

```

```

80     int x;
81
82     Ref &operator+=(Ref y) {
83         h->merge(x, y.x);
84         return *this;
85     }
86
87     Ref &operator<<(const T &v) {
88         h->push(x, v);
89         return *this;
90     }
91
92     T ask() {
93         return h->ask(x);
94     }
95
96     T pop() {
97         return h->pop(x);
98     }
99
100    void clear() {
101        h->clear(x);
102    }
103
104    bool askEmp() {
105        return h->askEmp(x);
106    }
107
108    int askSz() {
109        return h->askSz(x);
110    }
111 };
112
113 Ref operator[](int x) {
114     return {this, x};
115 }
116 };

```

09B - PBDS可并堆 (GNU环境) .cpp

```

1  /*
2  用途: GNU PBDS pairing heap: 支持普通优先队列、修改/删除任意节点及破坏性合并。
3  限制: 仅 GCC/libstdc++ 可用。默认 Cmp = less<T> 为大根堆, greater<T> 为小根堆。
4
5  push 返回的句柄可交给 modify/erase; join 后另一个堆清空。除 empty/size 外, 操作空堆未定义。
6  pairing heap 的 push/join 为 O(1), pop 为均摊 O(log n), modify/erase 的实际复杂度依赖修改方向。
7  */
8  #ifdef int
9  #pragma push_macro("int")
10 #undef int
11 #define LYE_RESTORE_INT
12 #endif
13 #ifdef endl
14 #pragma push_macro("endl")
15 #undef endl
16 #define LYE_RESTORE_ENDL
17 #endif
18 #include <ext/pb_ds/priority_queue.hpp>
19 #ifdef LYE_RESTORE_ENDL
20 #pragma pop_macro("endl")
21 #undef LYE_RESTORE_ENDL
22 #endif
23 #ifdef LYE_RESTORE_INT
24 #pragma pop_macro("int")
25 #undef LYE_RESTORE_INT
26 #endif
27
28 template <class T, class Cmp = less<T>> struct Heap {
29     using H = __gnu_pbds::priority_queue<T, Cmp, __gnu_pbds::pairing_heap_tag>;
30     using It = typename H::point_iterator;

```

```

31     H h;
32
33     It push(const T &x) { return h.push(x); }
34     void pop() { h.pop(); }
35     const T &ask() const { return h.top(); }
36     void modify(It p, const T &x) { h.modify(p, x); }
37     void erase(It p) { h.erase(p); }
38     void merge(Heap &o) { h.join(o.h); }
39     bool askEmp() const { return h.empty(); }
40     int askSz() const { return h.size(); }
41     void clear() { h.clear(); }
42 };

```

10 - 可合并信息FHQTreap (MergeFHQTreap) .cpp

```

1  /*
2  用途: 带自定义聚合信息的键值 FHQ Treap 多重集。
3
4  Info 需支持默认构造与 operator+, MakeInfo(key, count) 生成单键贡献。
5  当前接口聚焦 insert/erase/ask (全局聚合); 若需要 rank/kth/前驱后继, 使用 10A。
6  */
7  template <class T, class F> class FHQTreap {
8  private:
9      struct Node {
10         int l = 0, r = 0;
11         int key = 0, cnt = 0;
12         unsigned pri = 0;
13         T val;
14     };
15
16     vector<Node> tr;
17     int rt = 0;
18     F mk;
19     mt19937 rng;
20
21     int node(int key, int cnt = 1) {
22         tr.push_back(Node());
23         int p = tr.size() - 1;
24         tr[p].key = key;
25         tr[p].cnt = cnt;
26         tr[p].pri = rng();
27         tr[p].val = mk(key, cnt);
28         return p;
29     }
30
31     void pull(int p) {
32         if (!p) return;
33         tr[p].val = tr[tr[p].l].val + mk(tr[p].key, tr[p].cnt) + tr[tr[p].r].val;
34     }
35
36     void split(int p, int key, int &x, int &y) {
37         if (!p) {
38             x = y = 0;
39             return;
40         }
41         if (tr[p].key <= key) {
42             x = p;
43             split(tr[p].r, key, tr[p].r, y);
44             pull(x);
45         } else {
46             y = p;
47             split(tr[p].l, key, x, tr[p].l);
48             pull(y);
49         }
50     }
51
52     int merge(int x, int y) {
53         if (!x || !y) return x | y;
54         if (tr[x].pri < tr[y].pri) {
55             tr[x].r = merge(tr[x].r, y);
56             pull(x);
57             return x;
58         } else {
59             tr[y].l = merge(x, tr[y].l);
60             pull(y);
61             return y;
62         }
63     }
64
65 public:

```

```

66   FHQTreap(F f)
67       : tr(1), mk(f),
68         rng(chrono::steady_clock::now().time_since_epoch().count()) {}
69
70   bool askEmp() const {
71       return rt == 0;
72   }
73
74   void insert(int key, int cnt = 1) {
75       int a, b, c;
76       split(rt, key - 1, a, b);
77       split(b, key, b, c);
78       if (b) {
79           tr[b].cnt += cnt;
80           pull(b);
81       } else {
82           b = node(key, cnt);
83       }
84       rt = merge(merge(a, b), c);
85   }
86
87   void erase(int key, int cnt = 1) {
88       int a, b, c;
89       split(rt, key - 1, a, b);
90       split(b, key, b, c);
91       if (b) {
92           if (tr[b].cnt > cnt) {
93               tr[b].cnt -= cnt;
94               pull(b);
95           } else {
96               b = merge(tr[b].l, tr[b].r);
97           }
98       }
99       rt = merge(merge(a, b), c);
100   }
101
102   T ask() const {
103       return tr[rt].val;
104   }
105 };

```

10A - FHQTreap (有序多重集) .cpp

```

1  /*
2  用途: FHQ Treap 有序多重集, 支持插入、删除、计数、排名、第 k 小、前驱和后继。
3
4  Key 需支持 <; askRank/askKth 为 1-indexed。erase(x,c) 删除至多 c 个。
5  所有操作期望 O(log n), askKth/askPre/askNxt 不存在时返回 nullopt。
6  */
7  template <class Key = int>
8  struct OSet {
9      struct Node {
10         int l = 0, r = 0;
11         int sz = 0, cnt = 0;
12         int pri = 0;
13         Key key{};
14     };
15
16     vector<Node> tr = {Node{}};
17     int rt = 0;
18     mt19937 rng{chrono::steady_clock::now().time_since_epoch().count()};
19
20     int sz(int p) const {
21         return p ? tr[p].sz : 0;
22     }
23
24     void pull(int p) {
25         if (p) tr[p].sz = tr[tr[p].l].sz + tr[p].cnt + tr[tr[p].r].sz;
26     }
27
28     int node(const Key &key, int cnt) {
29         tr.push_back(Node{});
30         int p = tr.size() - 1;
31         tr[p].key = key;
32         tr[p].cnt = tr[p].sz = cnt;
33         tr[p].pri = rng();
34         return p;
35     }
36
37     // a 中键 < key, b 中键 >= key。

```

```

38 void split(int p, const Key &key, int &a, int &b) {
39     if (!p) {
40         a = b = 0;
41     } else if (tr[p].key < key) {
42         a = p;
43         split(tr[p].r, key, tr[p].r, b);
44         pull(a);
45     } else {
46         b = p;
47         split(tr[p].l, key, a, tr[p].l);
48         pull(b);
49     }
50 }
51
52 // a 中键 <= key, b 中键 > key。
53 void splitR(int p, const Key &key, int &a, int &b) {
54     if (!p) {
55         a = b = 0;
56     } else if (!(key < tr[p].key)) {
57         a = p;
58         splitR(tr[p].r, key, tr[p].r, b);
59         pull(a);
60     } else {
61         b = p;
62         splitR(tr[p].l, key, a, tr[p].l);
63         pull(b);
64     }
65 }
66
67 int merge(int a, int b) {
68     if (!a || !b) return a | b;
69     if (tr[a].pri < tr[b].pri) {
70         tr[a].r = merge(tr[a].r, b);
71         pull(a);
72         return a;
73     }
74     tr[b].l = merge(a, tr[b].l);
75     pull(b);
76     return b;
77 }
78
79 int askSz() const {
80     return sz(rt);
81 }
82
83 bool askEmp() const {
84     return rt == 0;
85 }
86
87 void clear() {
88     tr.assign(1, Node{});
89     rt = 0;
90 }
91
92 void insert(const Key &key, int cnt = 1) {
93     if (cnt <= 0) return;
94     int a, b, c;
95     split(rt, key, a, b);
96     splitR(b, key, b, c);
97     if (b) tr[b].cnt += cnt, pull(b);
98     else b = node(key, cnt);
99     rt = merge(merge(a, b), c);
100 }
101
102 bool erase(const Key &key, int cnt = 1) {
103     if (cnt <= 0) return false;
104     int a, b, c;
105     split(rt, key, a, b);
106     splitR(b, key, b, c);
107     bool ok = b;
108     if (b) {
109         if (tr[b].cnt > cnt) tr[b].cnt -= cnt, pull(b);
110         else b = merge(tr[b].l, tr[b].r);
111     }
112     rt = merge(merge(a, b), c);
113     return ok;
114 }
115
116 int askCnt(const Key &key) const {
117     int p = rt;
118     while (p) {
119         if (key < tr[p].key) p = tr[p].l;

```

```

120         else if (tr[p].key < key) p = tr[p].r;
121         else return tr[p].cnt;
122     }
123     return 0;
124 }
125
126 int askRank(const Key &key) const {
127     int p = rt, ans = 1;
128     while (p) {
129         if (!(tr[p].key < key)) {
130             p = tr[p].l;
131         } else {
132             ans += sz(tr[p].l) + tr[p].cnt;
133             p = tr[p].r;
134         }
135     }
136     return ans;
137 }
138
139 optional<Key> askKth(int k) const {
140     if (k <= 0 || k > askSz()) return nullopt;
141     int p = rt;
142     while (p) {
143         int nl = sz(tr[p].l);
144         if (k <= nl) p = tr[p].l;
145         else if (k <= nl + tr[p].cnt) return tr[p].key;
146         else k -= nl + tr[p].cnt, p = tr[p].r;
147     }
148     return nullopt;
149 }
150
151 optional<Key> askPre(const Key &key) const {
152     int p = rt, ans = 0;
153     bool ok = false;
154     while (p) {
155         if (tr[p].key < key) ans = p, ok = true, p = tr[p].r;
156         else p = tr[p].l;
157     }
158     return ok ? optional<Key>(tr[ans].key) : nullopt;
159 }
160
161 optional<Key> askNxt(const Key &key) const {
162     int p = rt, ans = 0;
163     bool ok = false;
164     while (p) {
165         if (key < tr[p].key) ans = p, ok = true, p = tr[p].l;
166         else p = tr[p].r;
167     }
168     return ok ? optional<Key>(tr[ans].key) : nullopt;
169 }
170 };

```

10B - 隐式 FHQ Treap (序列) .cpp

```

1  /*
2  用途: 隐式 FHQ Treap, 维护可修改序列。
3
4  下标均为 0-indexed, 区间为半开 [l, r)。支持插入、删除、区间加、区间翻转、区间和与访问单点。
5  T 通常为 int; 所有操作期望 O(log n)。删除节点不回收, 适合竞赛一次性使用。
6  */
7
8  template <class T = int>
9  struct FHQTreap {
10     struct Node {
11         int l = 0, r = 0;
12         int sz = 0;
13         int pri = 0;
14         T v{}, sum{}, add{};
15         bool rev = false;
16     };
17
18     vector<Node> tr = {Node{}};
19     int rt = 0;
20     mt19937 rng{chrono::steady_clock::now().time_since_epoch().count()};
21
22     int sz(int p) const {
23         return p ? tr[p].sz : 0;
24     }
25     T sum(int p) const {
26         return p ? tr[p].sum : T{};

```

```

27     }
28
29     int node(const T &v) {
30         tr.push_back(Node{});
31         int p = tr.size() - 1;
32         tr[p].v = tr[p].sum = v;
33         tr[p].sz = 1;
34         tr[p].pri = rng();
35         return p;
36     }
37
38     void pull(int p) {
39         if (!p) return;
40         tr[p].sz = sz(tr[p].l) + 1 + sz(tr[p].r);
41         tr[p].sum = sum(tr[p].l) + tr[p].v + sum(tr[p].r);
42     }
43
44     void setAdd(int p, const T &v) {
45         if (!p) return;
46         tr[p].v += v;
47         tr[p].sum += v * tr[p].sz;
48         tr[p].add += v;
49     }
50
51     void rev(int p) {
52         if (!p) return;
53         swap(tr[p].l, tr[p].r);
54         tr[p].rev = !tr[p].rev;
55     }
56
57     void push(int p) {
58         if (!p) return;
59         if (tr[p].add != T{}) {
60             setAdd(tr[p].l, tr[p].add);
61             setAdd(tr[p].r, tr[p].add);
62             tr[p].add = T{};
63         }
64         if (tr[p].rev) {
65             rev(tr[p].l);
66             rev(tr[p].r);
67             tr[p].rev = false;
68         }
69     }
70
71     // a 为前 k 个元素, b 为其余元素。
72     void split(int p, int k, int &a, int &b) {
73         if (!p) {
74             a = b = 0;
75             return;
76         }
77         push(p);
78         if (sz(tr[p].l) >= k) {
79             b = p;
80             split(tr[p].l, k, a, tr[p].l);
81             pull(b);
82         } else {
83             a = p;
84             split(tr[p].r, k - sz(tr[p].l) - 1, tr[p].r, b);
85             pull(a);
86         }
87     }
88
89     int merge(int a, int b) {
90         if (!a || !b) return a | b;
91         if (tr[a].pri < tr[b].pri) {
92             push(a);
93             tr[a].r = merge(tr[a].r, b);
94             pull(a);
95             return a;
96         }
97         push(b);
98         tr[b].l = merge(a, tr[b].l);
99         pull(b);
100        return b;
101    }
102
103    int askSz() const {
104        return sz(rt);
105    }
106
107    void insert(int pos, const T &v) {
108        assert(0 <= pos && pos <= askSz());

```

```

109     int a, b;
110     split(rt, pos, a, b);
111     rt = merge(merge(a, node(v)), b);
112 }
113
114 void erase(int l, int r) {
115     assert(0 <= l && l <= r && r <= askSz());
116     int a, b, c;
117     split(rt, r, a, c);
118     split(a, l, a, b);
119     rt = merge(a, c);
120 }
121
122 void update(int l, int r, const T &v) {
123     assert(0 <= l && l <= r && r <= askSz());
124     int a, b, c;
125     split(rt, r, a, c);
126     split(a, l, a, b);
127     setAdd(b, v);
128     rt = merge(merge(a, b), c);
129 }
130
131 void updRev(int l, int r) {
132     assert(0 <= l && l <= r && r <= askSz());
133     int a, b, c;
134     split(rt, r, a, c);
135     split(a, l, a, b);
136     rev(b);
137     rt = merge(merge(a, b), c);
138 }
139
140 T askSum(int l, int r) {
141     assert(0 <= l && l <= r && r <= askSz());
142     int a, b, c;
143     split(rt, r, a, c);
144     split(a, l, a, b);
145     T ans = sum(b);
146     rt = merge(merge(a, b), c);
147     return ans;
148 }
149
150 optional<T> ask(int pos) {
151     if (pos < 0 || pos >= askSz()) return nullopt;
152     int a, b, c;
153     split(rt, pos + 1, a, c);
154     split(a, pos, a, b);
155     T ans = tr[b].v;
156     rt = merge(merge(a, b), c);
157     return ans;
158 }
159
160 vector<T> askAll() {
161     vector<T> res;
162     res.reserve(askSz());
163     auto dfs = [&](auto &&go, int p) -> void {
164         if (!p) return;
165         push(p);
166         go(go, tr[p].l);
167         res.push_back(tr[p].v);
168         go(go, tr[p].r);
169     };
170     dfs(dfs, rt);
171     return res;
172 }
173 };

```

10BA - 隐式 FHQTreap (自定义聚合与懒标记) .cpp

```

1  /*
2  用途: 带非交换可合并信息、区间翻转和点值作用懒标记的隐式 FHQ Treap。
3
4  下标均为 0-indexed, 区间为半开 [l,r)。支持插入、删除、单点替换、
5  区间作用、区间翻转、剪切插入和区间聚合, 所有操作期望  $O(\log n)$ 。
6
7  Info 需要支持:
8  - static Info unit(): 结合单位元;
9  - Info operator+(const Info&) const: 按序合并, 允许不交换。
10
11 Op 需要支持:
12 - void apply(Info&, const Tag&) const: 将 Tag 作用到单点或整段聚合信息;

```


13 - Tag join(const Tag& old, const Tag& now) const: 返回先 old 后 now 的复合标记。

```

14
15 Tag 必须默认构造。apply 必须保持 Info 的结合运算，且与 reverse 交换；
16 例如全体元素的同一重标号、仿射变换等。节点不回收，适合竞赛一次性使用。
17 */
18 template <class Info, class Tag, class Op>
19 struct SeqTreap {
20     struct Node {
21         int l = 0, r = 0, sz = 0;
22         unsigned pri = 0;
23         Info val{}, fwd{}, bwd{};
24         Tag tag{};
25         bool rev = false, has = false;
26     };
27
28     vector<Node> tr = {Node{}};
29     int rt = 0;
30     Op op;
31     mt19937 rng{static_cast<mt19937::result_type>(
32         chrono::steady_clock::now().time_since_epoch().count())};
33
34     explicit SeqTreap(Op ac = Op()) : op(ac) {}
35
36     void reserve(int n) {
37         tr.reserve(n + 1);
38     }
39
40     int sz(int p) const {
41         return p ? tr[p].sz : 0;
42     }
43     Info fwd(int p) const {
44         return p ? tr[p].fwd : Info::unit();
45     }
46     Info bwd(int p) const {
47         return p ? tr[p].bwd : Info::unit();
48     }
49
50     int node(const Info &v) {
51         tr.push_back(Node{});
52         int p = tr.size() - 1;
53         tr[p].val = tr[p].fwd = tr[p].bwd = v;
54         tr[p].sz = 1;
55         tr[p].pri = rng();
56         return p;
57     }
58
59     void pull(int p) {
60         if (!p) return;
61         tr[p].sz = sz(tr[p].l) + 1 + sz(tr[p].r);
62         tr[p].fwd = (fwd(tr[p].l) + tr[p].val) + fwd(tr[p].r);
63         tr[p].bwd = (bwd(tr[p].r) + tr[p].val) + bwd(tr[p].l);
64     }
65
66     void setTag(int p, const Tag &t) {
67         if (!p) return;
68         op.apply(tr[p].val, t);
69         op.apply(tr[p].fwd, t);
70         op.apply(tr[p].bwd, t);
71         if (tr[p].has) tr[p].tag = op.join(tr[p].tag, t);
72         else tr[p].tag = t, tr[p].has = true;
73     }
74
75     void flip(int p) {
76         if (!p) return;
77         swap(tr[p].l, tr[p].r);
78         swap(tr[p].fwd, tr[p].bwd);
79         tr[p].rev = !tr[p].rev;
80     }
81
82     void push(int p) {
83         if (!p) return;
84         if (tr[p].has) {
85             setTag(tr[p].l, tr[p].tag);
86             setTag(tr[p].r, tr[p].tag);
87             tr[p].has = false;
88         }
89         if (tr[p].rev) {
90             flip(tr[p].l);
91             flip(tr[p].r);
92             tr[p].rev = false;
93         }
94     }

```

```

95 // a 为前 k 个元素, b 为其余元素。
96 void split(int p, int k, int &a, int &b) {
97     if (!p) {
98         a = b = 0;
99         return;
100     }
101     push(p);
102     if (sz(tr[p].l) >= k) {
103         b = p;
104         split(tr[p].l, k, a, tr[p].l);
105         pull(b);
106     } else {
107         a = p;
108         split(tr[p].r, k - sz(tr[p].l) - 1, tr[p].r, b);
109         pull(a);
110     }
111 }
112
113 int merge(int a, int b) {
114     if (!a || !b) return a | b;
115     if (tr[a].pri < tr[b].pri) {
116         push(a);
117         tr[a].r = merge(tr[a].r, b);
118         pull(a);
119         return a;
120     }
121     push(b);
122     tr[b].l = merge(a, tr[b].l);
123     pull(b);
124     return b;
125 }
126
127 int askSz() const {
128     return sz(rt);
129 }
130
131 void insert(int pos, const Info &v) {
132     assert(0 <= pos && pos <= askSz());
133     int a, b;
134     split(rt, pos, a, b);
135     rt = merge(merge(a, node(v)), b);
136 }
137
138 void erase(int l, int r) {
139     assert(0 <= l && l <= r && r <= askSz());
140     int a, b, c;
141     split(rt, r, a, c);
142     split(a, l, a, b);
143     rt = merge(a, c);
144 }
145
146 void modify(int pos, const Info &v) {
147     assert(0 <= pos && pos < askSz());
148     int a, b, c;
149     split(rt, pos + 1, a, c);
150     split(a, pos, a, b);
151     push(b);
152     tr[b].val = tr[b].fwd = tr[b].bwd = v;
153     pull(b);
154     rt = merge(merge(a, b), c);
155 }
156
157 void update(int l, int r, const Tag &t) {
158     assert(0 <= l && l <= r && r <= askSz());
159     int a, b, c;
160     split(rt, r, a, c);
161     split(a, l, a, b);
162     setTag(b, t);
163     rt = merge(merge(a, b), c);
164 }
165
166 void reverse(int l, int r) {
167     assert(0 <= l && l <= r && r <= askSz());
168     int a, b, c;
169     split(rt, r, a, c);
170     split(a, l, a, b);
171     flip(b);
172     rt = merge(merge(a, b), c);
173 }
174
175 // 将当前 [l,r) 剪下, 插到删除后位置 p 之前。
176

```

```

177 void move(int l, int r, int p) {
178     assert(0 <= l && l <= r && r <= askSz());
179     assert(0 <= p && p <= askSz() - (r - l));
180     int a, b, c;
181     split(rt, r, a, c);
182     split(a, l, a, b);
183     rt = merge(a, c);
184     split(rt, p, a, c);
185     rt = merge(merge(a, b), c);
186 }
187
188 Info ask(int l, int r) {
189     assert(0 <= l && l <= r && r <= askSz());
190     int a, b, c;
191     split(rt, r, a, c);
192     split(a, l, a, b);
193     Info res = fwd(b);
194     rt = merge(merge(a, b), c);
195     return res;
196 }
197 };

```

10C - PBDS有序多重集 (GNU环境) .cpp

```

1  /*
2  用途: GNU PBDS 有序多重集: FHQ Treap 的 GNU 环境高性能替代实现。
3  限制: 仅 GCC/libstdc++ 可用, Clang+libc++、MSVC 不可用, 因此与可移植的 10A 同时保留。
4  排名和 kth 均为 1-indexed: 重复键通过唯一编号区分。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  #ifdef int
8  #pragma push_macro("int")
9  #undef int
10 #define LYE_RESTORE_INT
11 #endif
12 #ifdef endl
13 #pragma push_macro("endl")
14 #undef endl
15 #define LYE_RESTORE_ENDL
16 #endif
17 #include <ext/pb_ds/assoc_container.hpp>
18 #include <ext/pb_ds/tree_policy.hpp>
19 #ifdef LYE_RESTORE_ENDL
20 #pragma pop_macro("endl")
21 #undef LYE_RESTORE_ENDL
22 #endif
23 #ifdef LYE_RESTORE_INT
24 #pragma pop_macro("int")
25 #undef LYE_RESTORE_INT
26 #endif
27
28 template <class Key> struct OSet {
29     using P = pair<Key, int>;
30     using Tree = __gnu_pbds::tree<P, __gnu_pbds::null_type, less<P>,
31                                     __gnu_pbds::rb_tree_tag,
32                                     __gnu_pbds::tree_order_statistics_node_update>;
33     Tree t;
34     int id = 0;
35
36     int insert(const Key& x) {
37         int k = ++id;
38         t.insert({x, k});
39         return k;
40     }
41
42     bool erase(const Key& x, int k) { return t.erase({x, k}); }
43
44     bool erase(const Key& x) {
45         auto it = t.lower_bound({x, 0});
46         if (it == t.end() || it->first != x) return false;
47         t.erase(it);
48         return true;
49     }
50
51     int askSz() const { return t.size(); }
52     bool askEmp() const { return t.empty(); }
53     void clear() { t.clear(), id = 0; }
54     int askCnt(const Key& x) const {
55         return t.order_of_key({x, numeric_limits<int>::max()}) - t.order_of_key({x, 0});
56     }

```

```

57     int askRank(const Key& x) const { return t.order_of_key({x, 0}) + 1; }
58
59     optional<Key> askKth(int k) const {
60         if (k < 1 || k > askSz()) return nullopt;
61         return t.find_by_order(k - 1)->first;
62     }
63
64     optional<Key> askPre(const Key& x) const {
65         auto it = t.lower_bound({x, 0});
66         if (it == t.begin()) return nullopt;
67         return (--it)->first;
68     }
69
70     optional<Key> askNxt(const Key& x) const {
71         auto it = t.upper_bound({x, numeric_limits<int>::max()});
72         return it == t.end() ? nullopt : optional<Key>(it->first);
73     }
74
75     optional<Key> askGE(const Key& x) const {
76         auto it = t.lower_bound({x, 0});
77         return it == t.end() ? nullopt : optional<Key>(it->first);
78     }
79 };

```

10D - vector有序多重集（小数据环境）.cpp

```

1  /*
2  用途：用排序 vector 模拟有序多重集；接口与常见平衡树题一致。
3  环境：查询  $O(\log n)$ ，插入/删除  $O(n)$ ； $n$  较小、修改少或只想极短实现时使用。
4  */
5  template <class T> struct OSet {
6      vector<T> a;
7
8      int askSz() const { return a.size(); }
9      bool askEmp() const { return a.empty(); }
10     void clear() { a.clear(); }
11
12     void insert(const T &x) { a.insert(lower_bound(a.begin(), a.end(), x), x); }
13
14     bool erase(const T &x) {
15         auto p = lower_bound(a.begin(), a.end(), x);
16         if (p == a.end() || *p != x) return false;
17         a.erase(p);
18         return true;
19     }
20
21     int askRank(const T &x) const { return lower_bound(a.begin(), a.end(), x) - a.begin() + 1; }
22     int askCnt(const T &x) const { return upper_bound(a.begin(), a.end(), x) - lower_bound(a.begin(), a.end(), x); }
23 }
24
25     optional<T> askKth(int k) const {
26         if (k < 1 || k > askSz()) return nullopt;
27         return a[k - 1];
28     }
29
30     optional<T> askPre(const T &x) const {
31         auto p = lower_bound(a.begin(), a.end(), x);
32         return p == a.begin() ? nullopt : optional<T>(*--p);
33     }
34
35     optional<T> askNxt(const T &x) const {
36         auto p = upper_bound(a.begin(), a.end(), x);
37         return p == a.end() ? nullopt : optional<T>(*p);
38     }
39 };

```

10E - PBDS有序集合与映射（GNU环境）.cpp

```

1  /*
2  用途：GNU PBDS 唯一键有序集合/映射，支持排名与第  $k$  小；10C 另行处理重复键。
3  限制：仅 GCC/libstdc++ 可用。默认红黑树；切换 Tag 可用 splay_tree_tag / ov_tree_tag。
4
5  OST<T> 常用操作（ $k$  为 0-indexed）：
6      insert/erase/find/lower_bound/upper_bound
7      order_of_key(x)          严格小于  $x$  的元素数
8      find_by_order(k)         第  $k$  小迭代器，越界等于 end()
9      a.split(x, b)             $a$  保留  $\leq x$ ， $b$  得到  $> x$ 
10     a.join(b)                 两棵树键域必须严格分离，完成后  $b$  为空

```

```

11 OMT<K,V> 还支持 t[k]、find_by_order(k)->second。
12 禁止用 less_equal 模拟 multiset: 它不满足严格弱序, 会破坏 erase/join 等接口。
13 */
14 #ifdef int
15 #pragma push_macro("int")
16 #undef int
17 #define LYE_RESTORE_INT
18 #endif
19 #ifdef endl
20 #pragma push_macro("endl")
21 #undef endl
22 #define LYE_RESTORE_ENDL
23 #endif
24 #include <ext/pb_ds/assoc_container.hpp>
25 #include <ext/pb_ds/tree_policy.hpp>
26 #ifdef LYE_RESTORE_ENDL
27 #pragma pop_macro("endl")
28 #undef LYE_RESTORE_ENDL
29 #endif
30 #ifdef LYE_RESTORE_INT
31 #pragma pop_macro("int")
32 #undef LYE_RESTORE_INT
33 #endif
34
35 template <class K, class Cmp = less<K>, class Tag = __gnu_pbds::rb_tree_tag>
36 using OST = __gnu_pbds::tree<K, __gnu_pbds::null_type, Cmp, Tag,
37                             __gnu_pbds::tree_order_statistics_node_update>;
38
39 template <class K, class V, class Cmp = less<K>, class Tag = __gnu_pbds::rb_tree_tag>
40 using OMT = __gnu_pbds::tree<K, V, Cmp, Tag,
41                             __gnu_pbds::tree_order_statistics_node_update>;

```

10EA - PBDS平衡树 (GNU环境, 基础) .cpp

```

1  /*
2  用途: GNU PBDS 的最短基础有序平衡树, 维护唯一键并支持排名、第 k 小、前驱后继。
3  限制: 仅 GCC/libstdc++ 可用; 重复键请使用 10C, 需移植实现请使用 10A。
4  接口: add/del、askRank (插入位置, 1-indexed)、askKth (第 k 小, 1-indexed)、
5        askPre/askNxt、askSz、askEmp。
6  复杂度: 各操作 O(log n)。find_by_order 在 PBDS 内部为 0-indexed, 模板已统一成 1-indexed。
7  */
8  #ifdef int
9  #pragma push_macro("int")
10 #undef int
11 #define LYE_RESTORE_INT
12 #endif
13 #ifdef endl
14 #pragma push_macro("endl")
15 #undef endl
16 #define LYE_RESTORE_ENDL
17 #endif
18 #include <ext/pb_ds/assoc_container.hpp>
19 #include <ext/pb_ds/tree_policy.hpp>
20 #ifdef LYE_RESTORE_ENDL
21 #pragma pop_macro("endl")
22 #undef LYE_RESTORE_ENDL
23 #endif
24 #ifdef LYE_RESTORE_INT
25 #pragma pop_macro("int")
26 #undef LYE_RESTORE_INT
27 #endif
28
29 template <class T = int, class C = less<T>, class Tag = __gnu_pbds::rb_tree_tag>
30 struct PBTree {
31     using Tree = __gnu_pbds::tree<T, __gnu_pbds::null_type, C, Tag,
32                                 __gnu_pbds::tree_order_statistics_node_update>;
33     Tree t;
34
35     bool add(const T& x) { return t.insert(x).second; }
36     bool del(const T& x) { return t.erase(x); }
37     bool ask(const T& x) const { return t.find(x) != t.end(); }
38     int askSz() const { return t.size(); }
39     bool askEmp() const { return t.empty(); }
40     void clear() { t.clear(); }
41
42     int askRank(const T& x) const { return t.order_of_key(x) + 1; }
43
44     optional<T> askKth(int k) const {
45         if (k < 1 || k > askSz()) return nullopt;
46         return *t.find_by_order(k - 1);

```

```

47     }
48
49     optional<T> askPre(const T& x) const {
50         auto it = t.lower_bound(x);
51         if (it == t.begin()) return nullopt;
52         return *--it;
53     }
54
55     optional<T> askNxt(const T& x) const {
56         auto it = t.upper_bound(x);
57         return it == t.end() ? nullopt : optional<T>(*it);
58     }
59 };

```

11A - KDTree (二维静态查询) .cpp

```

1  /*
2  用途：二维静态 KD Tree，支持最近点（可排除一个编号）与闭矩形计数。
3  点下标为 build 时的 0-indexed 顺序。D 默认 i128，防止坐标差平方溢出 int。
4  平均查询接近  $O(\sqrt{n})$ ；最坏  $O(n)$ ，构造  $O(n \log n)$ 。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7
8  using i128 = __int128_t;
9
10 template <class T = int, class D = i128> struct KDTree {
11     struct Point {
12         T x{}, y{};
13     };
14     struct Node {
15         Point p;
16         T mnx{}, mxx{}, mny{}, mxy{};
17         int l = -1, r = -1, id = -1, sz = 1;
18     };
19
20     vector<pair<Point, int>> a;
21     vector<Node> tr;
22     int rt = -1;
23
24     KDTree() = default;
25     explicit KDTree(const vector<Point>& p) { build(p); }
26
27     static D sq(T x, T y) {
28         D d = D(x) - D(y);
29         return d * d;
30     }
31     static D dis(const Point& a, const Point& b) { return sq(a.x, b.x) + sq(a.y, b.y); }
32
33     void pull(int u) {
34         Node& x = tr[u];
35         x.mnx = x.mxx = x.p.x, x.mny = x.mxy = x.p.y, x.sz = 1;
36         for (int v : {x.l, x.r}) if (v != -1) {
37             x.mnx = min(x.mnx, tr[v].mnx), x.mxx = max(x.mxx, tr[v].mxx);
38             x.mny = min(x.mny, tr[v].mny), x.mxy = max(x.mxy, tr[v].mxy);
39             x.sz += tr[v].sz;
40         }
41     }
42
43     int make(int l, int r, int d) {
44         if (l >= r) return -1;
45         int m = (l + r) >> 1;
46         auto cmp = [d](const auto& x, const auto& y) {
47             T a = d ? x.first.y : x.first.x;
48             T b = d ? y.first.y : y.first.x;
49             return a != b ? a < b : x.second < y.second;
50         };
51         nth_element(a.begin() + l, a.begin() + m, a.begin() + r, cmp);
52         int u = tr.size();
53         tr.push_back({a[m].first, a[m].first.x, a[m].first.x, a[m].first.y, a[m].first.y,
54             -1, -1, a[m].second, 1});
55         tr[u].l = make(l, m, d ^ 1);
56         tr[u].r = make(m + 1, r, d ^ 1);
57         pull(u);
58         return u;
59     }
60
61     void build(const vector<Point>& p) {
62         a.clear(), tr.clear();
63         a.reserve(p.size()), tr.reserve(p.size());
64         for (int i = 0; i < p.size(); ++i) a.push_back({p[i], i});

```

```

65     rt = make(0, a.size(), 0);
66 }
67
68 D box(int u, const Point& p) const {
69     D r{};
70     if (p.x < tr[u].mnx) r += sq(p.x, tr[u].mnx);
71     if (p.x > tr[u].mxx) r += sq(p.x, tr[u].mxx);
72     if (p.y < tr[u].mny) r += sq(p.y, tr[u].mny);
73     if (p.y > tr[u].mxy) r += sq(p.y, tr[u].mxy);
74     return r;
75 }
76
77 void near(int u, const Point& p, int ban, optional<D>& z) const {
78     if (u == -1) return;
79     if (tr[u].id != ban) {
80         D d = dis(p, tr[u].p);
81         if (!z || d < *z) z = d;
82     }
83     int x = tr[u].l, y = tr[u].r;
84     D dx = x == -1 ? D(-1) : box(x, p);
85     D dy = y == -1 ? D(-1) : box(y, p);
86     if (y != -1 && (x == -1 || dy < dx)) swap(x, y), swap(dx, dy);
87     if (x != -1 && (!z || dx <= *z)) near(x, p, ban, z);
88     if (y != -1 && (!z || dy <= *z)) near(y, p, ban, z);
89 }
90
91 optional<D> askNear(const Point& p, int ban = -1) const {
92     optional<D> r;
93     near(rt, p, ban, r);
94     return r;
95 }
96
97 int rect(int u, T x1, T y1, T x2, T y2) const {
98     if (u == -1 || tr[u].mxx < x1 || x2 < tr[u].mnx || tr[u].mxy < y1 || y2 < tr[u].mny) return 0;
99     if (x1 <= tr[u].mnx && tr[u].mxx <= x2 && y1 <= tr[u].mny && tr[u].mxy <= y2) return tr[u].sz;
100     int r = x1 <= tr[u].p.x && tr[u].p.x <= x2 && y1 <= tr[u].p.y && tr[u].p.y <= y2;
101     return r + rect(tr[u].l, x1, y1, x2, y2) + rect(tr[u].r, x1, y1, x2, y2);
102 }
103
104 int ask(T x1, T y1, T x2, T y2) const {
105     if (x1 > x2 || y1 > y2) return 0;
106     return rect(rt, x1, y1, x2, y2);
107 }
108 };

```

11B - KDTree (二维动态插入与偏序计数) .cpp

```

1  /*
2  用途：二维动态 KD Tree，插入点并查询  $x \leq qx$  且  $y \leq qy$  的点数；重复点分别计数。
3  复杂度：替罪羊式失衡重构，均摊表现良好；单次查询最坏仍可能  $O(n)$ 。
4  注意：节点下标显式 int。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7
8  template <class T = int> struct KDTree {
9      struct Node {
10         T x = 0, y = 0, ax = 0, bx = 0, ay = 0, by = 0;
11         int l = 0, r = 0, sz = 1, d = 0;
12     };
13
14     vector<Node> t{};
15     int rt = 0;
16
17     int sz(int p) const { return p ? t[p].sz : 0; }
18
19     void pull(int p) {
20         auto &u = t[p];
21         u.sz = 1;
22         u.ax = u.bx = u.x;
23         u.ay = u.by = u.y;
24         for (int v : {u.l, u.r}) if (v) {
25             u.sz += t[v].sz;
26             u.ax = min(u.ax, t[v].ax);
27             u.bx = max(u.bx, t[v].bx);
28             u.ay = min(u.ay, t[v].ay);
29             u.by = max(u.by, t[v].by);
30         }
31     }
32
33     int build(vector<int> &a, int l, int r, int d = 0) {

```

```

34     if (l >= r) return 0;
35     int m = (l + r) >> 1;
36     nth_element(a.begin() + l, a.begin() + m, a.begin() + r, [&](int p, int q) {
37         if (!d) return pair(t[p].x, t[p].y) < pair(t[q].x, t[q].y);
38         return pair(t[p].y, t[p].x) < pair(t[q].y, t[q].x);
39     });
40     int p = a[m];
41     t[p].d = d;
42     t[p].l = build(a, l, m, d ^ 1);
43     t[p].r = build(a, m + 1, r, d ^ 1);
44     pull(p);
45     return p;
46 }
47
48 void flat(int p, vector<int> &a) {
49     if (!p) return;
50     flat(t[p].l, a);
51     a.push_back(p);
52     flat(t[p].r, a);
53 }
54
55 void rebuild(int &p) {
56     int d = t[p].d;
57     vector<int> a;
58     a.reserve(t[p].sz);
59     flat(p, a);
60     p = build(a, 0, a.size(), d);
61 }
62
63 bool bad(int p) const { return max(sz(t[p].l), sz(t[p].r)) * 4 > sz(p) * 3; }
64
65 void ins(int &p, int q) {
66     if (!p) {
67         p = q;
68         pull(p);
69         return;
70     }
71     if ((!t[p].d && pair(t[q].x, t[q].y) < pair(t[p].x, t[p].y)) ||
72         (t[p].d && pair(t[q].y, t[q].x) < pair(t[p].y, t[p].x)))
73         ins(t[p].l, q);
74     else
75         ins(t[p].r, q);
76     pull(p);
77     if (bad(p)) rebuild(p);
78 }
79
80 void insert(T x, T y) {
81     Node u;
82     u.x = u.ax = u.bx = x;
83     u.y = u.ay = u.by = y;
84     t.push_back(u);
85     ins(rt, t.size() - 1);
86 }
87
88 int ask0(int p, T x, T y) const {
89     if (!p || x < t[p].ax || y < t[p].ay) return 0;
90     if (t[p].bx <= x && t[p].by <= y) return t[p].sz;
91     return (t[p].x <= x && t[p].y <= y) + ask0(t[p].l, x, y) + ask0(t[p].r, x, y);
92 }
93
94 int ask(T x, T y) const { return ask0(rt, x, y); }
95 };

```

12 - 珂朵莉树 (ODT, 随机区间赋值环境) .cpp

```

1  /*
2  用途: 维护分段常值序列, 支持半开区间 [l,r) 赋值、加法、第 k 小与幂和。
3  前提: 只有当区间赋值足够随机、区间数长期很少时才使用; 不存在可靠通用最坏复杂度。
4  自动取模: askSum<M> 令 M 为 MInt 即可让中间过程自动取模。
5  */
6
7  template <class T, class I = int> struct ODT {
8      struct Node {
9          I l, r;
10         mutable T v;
11         bool operator<(const Node &o) const { return l < o.l; }
12     };
13
14     using It = typename set<Node>::iterator;
15     I n = 0;

```



```

16 set<Node> s;
17
18 ODT() = default;
19 explicit ODT(const vector<T> &a) { build(a); }
20
21 void build(const vector<T> &a) {
22     n = a.size();
23     s.clear();
24     for (I l = 0, r; l < n; l = r) {
25         for (r = l + 1; r < n && a[r] == a[l]; ++r) {}
26         s.insert({l, r, a[l]});
27     }
28 }
29
30 It split(I p) {
31     if (p >= n) return s.end();
32     auto q = prev(s.upper_bound({p, p, T{}}));
33     if (q->l == p) return q;
34     auto [l, r, v] = *q;
35     s.erase(q);
36     s.insert({l, p, v});
37     return s.insert({p, r, v}).first;
38 }
39
40 void update(I l, I r, const T &x) {
41     auto e = split(r), b = split(l);
42     s.erase(b, e);
43     auto p = s.insert({l, r, x}).first;
44     if (p != s.begin()) {
45         auto q = prev(p);
46         if (q->r == p->l && q->v == p->v) {
47             I nl = q->l;
48             s.erase(q);
49             s.erase(p);
50             p = s.insert({nl, r, x}).first;
51         }
52     }
53     auto q = next(p);
54     if (q != s.end() && p->r == q->l && p->v == q->v) {
55         I nl = p->l;
56         I nr = q->r;
57         s.erase(q);
58         s.erase(p);
59         s.insert({nl, nr, x});
60     }
61 }
62
63 void updAdd(I l, I r, const T &x) {
64     auto e = split(r), b = split(l);
65     for (auto p = b; p != e; ++p) p->v += x;
66 }
67
68 T askKth(I l, I r, I k) {
69     auto e = split(r), b = split(l);
70     vector<pair<T, I>> a;
71     for (auto p = b; p != e; ++p) a.push_back({p->v, p->r - p->l});
72     sort(a.begin(), a.end());
73     for (auto [x, c] : a) if ((k -= c) <= 0) return x;
74     throw out_of_range("ODT::askKth");
75 }
76
77 template <class M> M askSum(I l, I r, int e) {
78     auto qp = [](M a, int b) {
79         M z = 1;
80         for (; b >= 1, a *= a) if (b & 1) z *= a;
81         return z;
82     };
83     auto q = split(r), b = split(l);
84     M z = 0;
85     for (auto p = b; p != q; ++p) z += qp(M(p->v), e) * M(p->r - p->l);
86     return z;
87 }
88 };

```

13A - PBDS开放寻址哈希表 (gp_hash_table, GNU环境) .cpp

```

1  /*
2  用途: GNU PBDS gp_hash_table; 开放寻址, 常数小, 适合大量唯一键的映射/集合。
3  限制: 仅 GCC/libstdc++ 可用; 增删可能使全部迭代器失效, 不要长期保存迭代器。
4

```

```

5 HMap<K,V>: h[k], insert, find, erase, size, empty, clear。
6 HSet<K>: insert, find, erase, size, empty, clear; 不允许重复键。
7 默认 resize policy 不开放手动 resize/resize, 不能照搬 unordered_map::reserve。
8 Hash 对整数、字符串及 pair 均可用, 并加入运行时盐以避免构造碰撞。
9 */
10
11 using u64 = uint64_t;
12
13 #ifdef int
14 #pragma push_macro("int")
15 #undef int
16 #define LYE_RESTORE_INT
17 #endif
18 #ifdef endl
19 #pragma push_macro("endl")
20 #undef endl
21 #define LYE_RESTORE_ENDL
22 #endif
23 #include <ext/pb_ds/assoc_container.hpp>
24 #ifdef LYE_RESTORE_ENDL
25 #pragma pop_macro("endl")
26 #undef LYE_RESTORE_ENDL
27 #endif
28 #ifdef LYE_RESTORE_INT
29 #pragma pop_macro("int")
30 #undef LYE_RESTORE_INT
31 #endif
32
33 struct Hash {
34     inline static const u64 r =
35         chrono::steady_clock::now().time_since_epoch().count();
36
37     static u64 mix(u64 x) {
38         x += 0x9e3779b97f4a7c15ULL;
39         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
40         x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
41         return x ^ (x >> 31);
42     }
43
44     template <class T> size_t operator()(const T &x) const {
45         return mix(u64(hash<T>{}(x)) + r);
46     }
47
48     template <class A, class B> size_t operator()(const pair<A, B> &x) const {
49         u64 a = (*this)(x.first), b = (*this)(x.second);
50         return mix(a ^ (b + 0x9e3779b97f4a7c15ULL + (a << 6) + (a >> 2)));
51     }
52 };
53
54 template <class K, class V, class H = Hash, class Eq = equal_to<K>>
55 using HMap = __gnu_pbds::gp_hash_table<K, V, H, Eq>;
56
57 template <class K, class H = Hash, class Eq = equal_to<K>>
58 using HSet = __gnu_pbds::gp_hash_table<K, __gnu_pbds::null_type, H, Eq>;

```

13B - PBDS拉链哈希表 (cc_hash_table, GNU环境) .cpp

```

1 /*
2 用途: GNU PBDS cc_hash_table; 拉链法哈希, 适合希望避免开放寻址聚簇或存较大值的环境。
3 限制: 仅 GCC/libstdc++ 可用; 通常比 13A 多指针和节点分配开销, 优先实测后选型。
4
5 HMap<K,V>: h[k], insert, find, erase, size, empty, clear。
6 HSet<K>: insert, find, erase, size, empty, clear; 不允许重复键。
7 13A/13B 是同一哈希字典问题的不同实现, 通常只粘贴其中一个。
8 */
9
10 using u64 = uint64_t;
11
12 #ifdef int
13 #pragma push_macro("int")
14 #undef int
15 #define LYE_RESTORE_INT
16 #endif
17 #ifdef endl
18 #pragma push_macro("endl")
19 #undef endl
20 #define LYE_RESTORE_ENDL
21 #endif
22 #include <ext/pb_ds/assoc_container.hpp>
23 #ifdef LYE_RESTORE_ENDL

```

```

24 #pragma pop_macro("endl")
25 #undef LYE_RESTORE_ENDL
26 #endif
27 #ifdef LYE_RESTORE_INT
28 #pragma pop_macro("int")
29 #undef LYE_RESTORE_INT
30 #endif
31
32 struct Hash {
33     inline static const u64 r =
34         chrono::steady_clock::now().time_since_epoch().count();
35
36     static u64 mix(u64 x) {
37         x += 0x9e3779b97f4a7c15ULL;
38         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
39         x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
40         return x ^ (x >> 31);
41     }
42
43     template <class T> size_t operator()(const T &x) const {
44         return mix(u64(hash<T>{}(x)) + r);
45     }
46
47     template <class A, class B> size_t operator()(const pair<A, B> &x) const {
48         u64 a = (*this)(x.first), b = (*this)(x.second);
49         return mix(a ^ (b + 0x9e3779b97f4a7c15ULL + (a << 6) + (a >> 2)));
50     }
51 };
52
53 template <class K, class V, class H = Hash, class Eq = equal_to<K>>
54 using HMap = __gnu_pbds::cc_hash_table<K, V, H, Eq>;
55
56 template <class K, class H = Hash, class Eq = equal_to<K>>
57 using HSet = __gnu_pbds::cc_hash_table<K, __gnu_pbds::null_type, H, Eq>;

```

14 - 分块 (区间加、区间和) .cpp

```

1  /*
2  用途：根号分块，维护区间加、区间和与单点赋值。
3  用法：Block<int> b(a)，其中 a 为 1-indexed 初值；update(l,r,x)、modify(p,x)、ask(l,r)。
4  复杂度：每次 O(sqrt(n))，空间 O(n)。适合操作不使用线段树维护或追求短实现的中等规模场景。
5  */
6
7  template <class T = int> struct Block {
8      int n, b, bc;
9      vector<T> a, sm, tg;
10
11      Block() : n(0), b(1), bc(0) {}
12
13      explicit Block(const vector<T>& v) {
14          init(v);
15      }
16
17      int id(int p) const {
18          return (p - 1) / b;
19      }
20
21      int lb(int x) const {
22          return x * b + 1;
23      }
24
25      int rb(int x) const {
26          return min(n, (x + 1) * b);
27      }
28
29      void init(const vector<T>& v) {
30          assert(v.size() >= 2);
31          n = v.size() - 1;
32          b = sqrtl(n) + 1;
33          bc = (n + b - 1) / b;
34          a = v;
35          sm.assign(bc, T{});
36          tg.assign(bc, T{});
37          for (int i = 1; i <= n; ++i) sm[id(i)] += a[i];
38      }
39
40      void push(int x) {
41          if (tg[x] == T{}) return;
42          for (int i = lb(x); i <= rb(x); ++i) a[i] += tg[x];
43          tg[x] = T{};

```

```

44     }
45
46     void pull(int x) {
47         sm[x] = T{};
48         for (int i = lb(x); i <= rb(x); ++i) sm[x] += a[i];
49     }
50
51     void update(int l, int r, const T& v) {
52         if (l > r) return;
53         assert(1 <= l && r <= n);
54         int x = id(l), y = id(r);
55         if (x == y) {
56             push(x);
57             for (int i = l; i <= r; ++i) a[i] += v;
58             pull(x);
59             return;
60         }
61         push(x);
62         for (int i = l; i <= rb(x); ++i) a[i] += v;
63         pull(x);
64         push(y);
65         for (int i = lb(y); i <= r; ++i) a[i] += v;
66         pull(y);
67         for (int z = x + 1; z < y; ++z) {
68             tg[z] += v;
69             sm[z] += v * (rb(z) - lb(z) + 1);
70         }
71     }
72
73     void modify(int p, const T& v) {
74         assert(1 <= p && p <= n);
75         int x = id(p);
76         push(x);
77         sm[x] += v - a[p];
78         a[p] = v;
79     }
80
81     T ask(int l, int r) {
82         if (l > r) return T{};
83         assert(1 <= l && r <= n);
84         int x = id(l), y = id(r);
85         T ans{};
86         if (x == y) {
87             push(x);
88             for (int i = l; i <= r; ++i) ans += a[i];
89             return ans;
90         }
91         push(x);
92         for (int i = l; i <= rb(x); ++i) ans += a[i];
93         push(y);
94         for (int i = lb(y); i <= r; ++i) ans += a[i];
95         for (int z = x + 1; z < y; ++z) ans += sm[z];
96         return ans;
97     }
98 };

```

图论：存图、路径、生成树与连通性

22 个代码模板 · 林月 · v1.0.0 · 2026.8.1

版块说明与知识结论

README.md

图论模板

需求	模板	主要复杂度
无权图 BFS/DFS、二分图判定	00A - 图遍历 (BFS、DFS、二分图判定)	$O(n+m)$
超大图连续内存存边	00B - 链式前向星 (大图环境)	遍历 $O(n+m)$
非负权单源最短路	01A - 单源最短路 (Dijkstra) (无负权)	$O((n+m) \log n)$
稠密非负权单源最短路	01H - 单源最短路 (Dijkstra, 稠密图)	$O(n^2)$ 时间、 $O(n^2)$ 空间
有限权值为 0/1 的单源最短路	01E - 0-1 BFS	$O(n+m)$
含负边权、并标记负环影响范围	01B - 单源最短路 (Bellman-ford, 负权图)	$O(nm)$
含负边权、只需判源点可达负环	01C - 单源最短路 (SPFA, 负权图)	最坏 $O(nm)$
稠密图/小规模全源最短路	01D - 多源最短路 (APSP)	$O(n^3)$
稀疏负权图全源最短路	01G - Johnson全源最短路	$O(nm+n(m+n)\log n)$
DAG 单源最短 / 最长路	01I - DAG路径 (最短、最长)	$O(n+m)$, 允许负边
正权图最短、严格次短路与计数	01J - 最短和次短路 (Dijkstra, 计数)	$O((n+m)\log n)$
差分不等式可行性	01F - 差分约束	平均较快, 最坏 $O(nm)$
最小生成树/森林	02A - 最小生成树 (Kruskal) 或 02B - 最小生成树 (Prim)	$O(m \log m) / O(m \log n)$
稠密无向图最小生成森林	02D - 最小生成树 (Prim, 稠密图)	$O(n^2)$ 时间、 $O(n^2)$ 空间
有向最小树形图	02C - 有向最小生成树 (Chu-Liu-Edmonds)	$O(nm)$
DAG 判环、拓扑序	03A - 拓扑排序 (Kahn)	$O(n+m)$
强连通分量、缩点 DAG	03B - 强连通分量 (Tarjan)	$O(n+m)$
两文字布尔可满足性	03C - 2-SAT	$O(V+E)$
桥、边双、桥树	04A - 边双连通分量 (桥)	$O(n+m)$
割点、点双、圆方树	04B - 点双连通分量 (割点、圆方树)	$O(n+m)$
有向/无向欧拉通路与回路	05A - 欧拉路 (有向、无向)	$O(n+m)$
二分图最大匹配、最小点覆盖	06A - 二分图最大匹配 (Hopcroft-Karp)	$O(m \sqrt{n})$
小图 / 左部较小的二分图匹配	06E - 二分图最大匹配 (Kuhn, 小图)	$O(n_l m)$
一般图最大基数匹配	06B - 一般图最大匹配 (带花树)	$O(n^3)$
完全二分图最优指派 (KM)	06C - 二分图最优匹配 (Hungarian, KM)	$O(L^2R)$
一般图最大权匹配	06D - 一般图最大权匹配 (带权带花树)	$O(n^3)$
无向图全局最小割	07 - 全局最小割 (Stoer-Wagner)	$O(n^3)$
至多 64 点最大团	08 - 最大团 (位集分支限界)	最坏指数级, 带贪心染色上界
已知直线平面嵌入的面与对偶图	09 - 平面嵌入面与对偶图	$O(m \log m)$
稠密有向/无向图最小环	10 - 最小环 (Floyd)	$O(n^3)$
正权有向图最小环长度及计数	10A - 最小环 (长度与计数)	$O(n^3)$

需求	模板	主要复杂度
小图的本质不同简单环计数	11 - 简单环计数（状态压缩）	$O(n2^n)$, $n \leq 20$
检测并恢复任意有向/无向环	12 - 环检测与恢复（有向、无向）	$O(n+m)$
有向图从根出发的直接支配点	13 - 支配树（Lengauer-Tarjan）	常用 $O((n+m) \log n)$
终端点较少的最小 Steiner 树	14 - Steiner 树（终端点较少）	$O(3^k n + 2^k m \log n)$, $k \leq 20$

使用提示

- 最短路默认以 INF 表示不可达；不要直接输出它。带负环时，Bellman-Ford 只有 `affectedByNegativeCycle[u] == false` 的点才有定义良好的最短路。
- 普通题优先 `vector<vector<Edge>>` 的可读性；边数很大、内存或缓存成为瓶颈时使用 00B。两者属于不同环境适配，均保留。
- 非负单源最短路在稀疏图选 01A；邻接矩阵已经存在、 m 接近 n^2 时选 01H。01H 的矩阵在默认 64 位 int 下占用较大内存，先估算后使用。
- Dijkstra 仅允许非负边；0-1 BFS 仅允许 0/1 权。SPFA 有最坏 $O(nm)$ 的构造，优先使用 Dijkstra、Bellman-Ford 或题目专用算法。
- Kruskal 和 Prim 都会返回最小生成森林；通过 `res.ok` 判断原图是否连通。
- 边双/点双使用边 ID 跳过父边，支持重边。点双模板默认不放自环；圆方树中原点是 $1..n$ ，块节点接在其后。
- 欧拉路模板会验证所有边能否被同一条通路使用；返回的点列相邻点正好对应一条原图边。
- 2-SAT 的变量从 0 开始，`literal(x, true)` 是文字 x ，`literal(x, false)` 是 $\neg x$ 。
- 差分约束统一写成 $x[v] - x[u] \leq w$ 。严格不等式只有在整数域才能安全改成 $\leq w - 1$ ；浮点域不能这样处理。
- Floyd 与 Johnson 是同一全源最短路问题的稠密/稀疏环境分支；Johnson 会检测任意负环，返回 `nullopt` 时距离无定义。
- 01I 只适用于 DAG；其拓扑 DP 同时支持最短和最长路。一般图的最长简单路仍是 NP-hard，不能把比较符号直接反过来。
- 01J 的“次短”是严格大于最短长度的第二个不同长度，边权必须严格为正；零权边会破坏“同距离答案在弹堆前已汇总”的计数前提。计数很大时将第二模板参数换为 `MInt`。
- Chu-Liu-Edmonds 求的是从指定根可达所有点的最小树形图，不是把无向 MST 的边改成有向边。
- 02D 与堆 Prim 语义相同但适配稠密邻接矩阵；稀疏图不应为它额外开 $O(n^2)$ 空间。
- Hopcroft-Karp 的左右部各自从 1 编号；Kuhn 是同一最大基数匹配的短代码小图分支。Hungarian/KM 处理的是完整二分图，要求左部大小不超过右部大小，并为每对点设置权值。
- 06B 只最大化边数，06C 只处理二分图权值，06D 才是一般图最大权匹配。06D 默认允许不匹配点并忽略非正权边；完美匹配含负权时先统一平移权值并检查边数。
- Stoer-Wagner 不指定源汇；若需要固定 S-T 割、恢复残量图或所有点对最小割，分别使用网络流目录的 Dinic / Gomory-Hu。
- 最大团使用 64 位候选集，点号为 $0..n-1$ ；若 n 更大，应利用补图、二分图或题目结构，不能把本指数算法硬套。
- 平面对偶模板要求输入已经是连通、无交叉的直线嵌入；它按每点出边极角恢复面。只有抽象邻接表、没有旋转系统或坐标时，无法唯一恢复对偶图。
- 10 的无向分支支持自环和重边，先直接处理 1/2 边环，再用 Floyd 枚举至少三条边的环；有向分支只要求非负边。两者都不是稀疏图算法。
- 10A 只适用于严格正权有向图；它按环上最大编号计数最短路径，因此同权重的平行边视为不同环。零边会使“最短路径必为简单路径”的计数前提失效。
- 11 只统计长度至少 3 的简单环，图必须无自环、重边；无向图不区分正反方向，有向图区分。默认 64 位 int 下 $n=20$ 的 DP 约占 160 MB。
- 12 支持自环、重边，返回的点列不重复首点，阅读时将最后一点连回第一点即可；它用于找任意一个环，不用于枚举或计数全部环。
- 支配树只对根可达点定义；13 返回 `idom[root]=0`，不可达点也为 0，不能把它们当成普通树节点继续做祖先查询。
- 14 是 Dreyfus-Wagner 的终端集合 DP，边权要求非负；终端点数量是指数参数， k 超过约 20 时先重新建模。若只连接两个终端，直接使用最短路更合适。

覆盖边界

最大流、最小割与费用流独立维护在 05 - 网络流。树上的 LCA、树链剖分、点分树等放在 02 - 树上问题，避免与一般图模板重复。

连通、最短路、MST、欧拉、匹配与平面图结论见 [图论知识与结论](#)。

图论知识与结论.md

图论知识与结论

1. 遍历、连通与 DAG

- 无向图度数和为 $2m$ ，奇度点个数为偶数；有向图总入度等于总出度等于 m 。
- SCC 缩点后一定是 DAG。需要让整图强连通时，若缩点数大于 1，最少补边数为 $\max(\text{入度为0的点数}, \text{出度为0的点数})$ 。
- DAG 最短/最长路均按拓扑序 DP；负边不影响 DAG 路径。一般图最长简单路是 NP-hard，不能把最短路模板的比较符号反过来。
- 二分图等价于不存在奇环。无向图判二分必须遍历每个连通块。

2. 最短路与差分约束

- 非负边用 Dijkstra：稀疏图用堆优化，邻接矩阵或稠密图可用 $O(n^2)$ 朴素版；0/1 权用双端队列，含负边用 Bellman-Ford；SPFA 最坏 $O(nm)$ ，只作为数据友好或差分约束实现。
- Dijkstra 保存父边即可恢复一棵最短路径树；零权边导致最短路径不唯一，父边规则会影响所得树但不影响距离。
- Floyd 可判 $d[i][i] < 0$ 的负环；任意两点距离需防止 INF+值 溢出。
- Johnson 先用虚拟源求势能，再把边重标为非负并从每个点跑 Dijkstra；适合稀疏负权图，任意负环都会使全源最短路无定义。
- 最短/严格次短路的双距离 Dijkstra 需要边权严格为正；此时所有能贡献同一距离的前驱都更短，计数可在节点出堆前汇总。含零边时必须先额外处理同距离 DAG 或改用题目专用方法。
- 差分约束 $x[v] - x[u] \leq w$ 建边 $u \rightarrow v(w)$ ，可行当且仅当无负环。求最大下界时可整体取反或改写不等号，不能混用边方向。

3. 生成树、割与平面图

- MST 的割性质：跨任意割的最轻边至少存在于一棵 MST；环性质：环上严格最重边不在任何 MST 中。
- 边权相同会使 MST 不唯一；判断某条边“必选”需在同权批次的缩点图上找桥，不能只看一次 Kruskal 结果。
- Stoer-Wagner 求不指定源汇的全局最小割；固定 S-T 割用最大流，所有点对割值可建 Gomory-Hu 树。
- 稠密非负权图的最小环可按顶点编号枚举环上最大编号 k ：在仅使用 $< k$ 的最短路 $i \rightarrow j$ 两端补 $i-k-j$ 。无向重边和自环另行比较；对应 10，复杂度 $O(n^3)$ 。
- 有向正权图需要同时统计最小环时，固定环上的最大编号 k ，枚举边 $k \rightarrow i$ 并连接只经 $< k$ 顶点的最短路 $i \rightarrow k$ ；每个最短简单环恰被计算一次。10A 把同权平行边按不同边计数，零边不能直接使用该计数式。
- 小 n 统计全部简单环可固定环上最小编号，再做状态压缩路径 DP；无向环会以两个方向各出现一次，最后除以 2。11 的 $n \leq 20$ 是空间而非逻辑限制。
- 只需找到一个环时，DFS 栈中遇到返祖边即可截取该栈段。无向图必须按边编号跳过父边，否则会漏掉重边构成的二元环；对应 12。
- 连通平面图满足 $V - E + F = 2$ 。给定平面嵌入后，原图割与对偶图环/路径可互换；没有嵌入信息时不能直接套“平面图最短路”。09 对带坐标的直线平面嵌入按极角恢复各面与对偶边，网格图只是这一过程的特例。

4. 欧拉、双连通与匹配

- 无向图存在欧拉回路当且仅当非零度点连通且奇度点为 0；欧拉通路允许恰有 2 个奇度点。
- 有向欧拉回路要求非零度部分连通且每点入度等于出度；通路只允许起点出度多 1、终点入度多 1。
- 桥不属于任何环；点双分量适合共享割点，圆方树把原点和点双块分别作为两类节点。
- König 定理：二分图最大匹配数等于最小点覆盖数；最大独立集大小为总点数减最小点覆盖数。小图可用 Kuhn 的 DFS 增广路，大图再换 Hopcroft-Karp。
- 一般图最大基数匹配用无权带花树；二分图最大权完美匹配用 KM；一般图最大权匹配用带权带花树，三者前提不同。

- 团等价于补图中的独立集。最大团在一般图上是 NP-hard; $n \leq 64$ 时可用位集候选集与贪心染色上界剪枝, 但复杂度仍是指数级。

6. 支配树与 Steiner 树

- 在有向图中固定根 rt , 若所有从 rt 到 v 的路径都经过 u , 则称 u 支配 v ; 直接支配点是支配关系的树父亲。只对 rt 可达点定义, 支配树不是原图的 DFS 树。
- Lengauer-Tarjan 先按 DFS 序维护半支配点, 再在逆图上做 `eval` 路径压缩; 实现时 `par/sem/dom` 都以 DFS 序编号, 最后才映射回原点号。递归栈和 rt 不可达点是最常见边界。
- 支配树可用于统计必经点、控制流图分析和路径计数的祖先关系; 若题目问“至少经过一条指定边”, 应转成边支配或拆点, 不能直接读取点支配树。
- 给定 k 个终端的无向非负权 Steiner 树用状态 $f[S][v]$ 表示连接终端集合 S 且在 v 汇合的最小代价。先做子集合并, 再以所有有限 $f[S][v]$ 为源跑一次 Dijkstra。
- 子集合并只枚举一半划分 (如 $sub < (S^{sub})$), 否则常数会接近翻倍; 终端重复、负权和 3^k 内存爆炸都要在调用前排除。模板 14 返回 `nullopt` 表示终端不连通。

5. 2-SAT 与树形图

- 文字蕴含 $a \Rightarrow b$ 同时添加逆否 $!b \Rightarrow !a$; 变量与其否定落在同一 SCC 时无解。
- Chu-Liu-Edmonds 每轮先选最小入边, 再把有向环缩点并调整入边权; 某个非根点没有入边时不存在根树形图。

00A - 图遍历 (BFS、DFS、二分图判定) .cpp

```

1  /*
2  用途: 无权图的 BFS/迭代 DFS; 另附无向图二分图染色。
3  约定: 点编号 1..n。Graph 可按需加有向边或无向边;
4  Bipartite 只应加入无向边。BFS 的 dis 为 -1 表示不可达。
5  复杂度: 每次遍历或染色  $O(n + m)$ 。
6  */
7  struct Graph {
8      struct BFSResult {
9          vector<int> dis;
10         vector<int> fa;
11         vector<int> ord;
12     };
13
14     int n;
15     vector<vector<int>> g;
16
17     explicit Graph(int n) : n(n), g(n + 1) {}
18
19     void add(int u, int v) {
20         g[u].push_back(v);
21     }
22
23     void add2(int u, int v) {
24         add(u, v);
25         add(v, u);
26     }
27
28     BFSResult bfs(int s) const {
29         BFSResult res{vector<int>(n + 1, -1), vector<int>(n + 1, -1), {}};
30         queue<int> q;
31         res.dis[s] = 0;
32         q.push(s);
33         while (!q.empty()) {
34             int u = q.front();
35             q.pop();
36             res.ord.push_back(u);
37             for (int v : g[u]) {
38                 if (res.dis[v] != -1) continue;
39                 res.dis[v] = res.dis[u] + 1;
40                 res.fa[v] = u;
41                 q.push(v);
42             }
43         }
44         return res;
45     }
46
47     // 显式栈版先序 DFS, 避免链状图递归爆栈。
48     vector<int> dfs(int s) const {
49         vector<char> vis(n + 1, false);

```



```

50     vector<int> it(n + 1), ord;
51     vector<int> stk = {s};
52     vis[s] = true;
53     ord.push_back(s);
54     while (!stk.empty()) {
55         int u = stk.back();
56         if (it[u] == g[u].size()) {
57             stk.pop_back();
58             continue;
59         }
60         int v = g[u][it[u]++];
61         if (vis[v]) continue;
62         vis[v] = true;
63         ord.push_back(v);
64         stk.push_back(v);
65     }
66     return ord;
67 }
68 };
69
70 struct Bipartite {
71     int n;
72     vector<vector<int>> g;
73
74     explicit Bipartite(int n) : n(n), g(n + 1) {}
75
76     void add(int u, int v) {
77         g[u].push_back(v);
78         g[v].push_back(u);
79     }
80
81     // color[u] 为 0/1: 返回 nullopt 表示含奇环。
82     optional<vector<int>> solve() const {
83         vector<int> col(n + 1, -1);
84         for (int s = 1; s <= n; ++s) {
85             if (col[s] != -1) continue;
86             queue<int> q;
87             q.push(s);
88             col[s] = 0;
89             while (!q.empty()) {
90                 int u = q.front();
91                 q.pop();
92                 for (int v : g[u]) {
93                     if (col[v] == -1) {
94                         col[v] = col[u] ^ 1;
95                         q.push(v);
96                     } else if (col[v] == col[u]) {
97                         return nullopt;
98                     }
99                 }
100             }
101         }
102         return col;
103     }
104 };

```

00B - 链式前向星（大图环境）.cpp

```

1  /*
2  用途：大图的连续内存邻接表；与 vector<vector<Edge>> 是同一存图问题的内存/缓存特化分支。
3  点编号 0..n-1, head[u] 起沿 edge[i].next 遍历。预先 reserve(m) 可避免扩容。
4  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
5  */
6
7  template <class W = int> struct Graph {
8      struct Edge {
9          int v, nxt;
10         W w;
11     };
12     int n;
13     vector<int> hd;
14     vector<Edge> e;
15
16     explicit Graph(int n, int m = 0) : n(n), hd(n, -1) {
17         assert(n >= 0 && m >= 0);
18         e.reserve(m);
19     }
20

```

```

21     int add(int u, int v, W w = W(1)) {
22         assert(0 <= u && u < n && 0 <= v && v < n);
23         e.push_back({v, hd[u], w});
24         return hd[u] = e.size() - 1;
25     }
26
27     array<int, 2> add2(int u, int v, W w = W(1)) {
28         int a = add(u, v, w);
29         int b = add(v, u, w);
30         return {a, b};
31     }
32 };

```

01A - 单源最短路 (Dijkstra) (无负权) .cpp

```

1  /*
2  用途: 非负边权单源最短路。
3  用法: Dijkstra<T> g(n); 若权值类型不同写 Dijkstra<T>。
4  约定: 点号和父亲数组使用 int, 距离类型由模板 T 决定。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7
8  template <class T = int> struct Dijkstra {
9      static constexpr T INF = numeric_limits<T>::max() / 4;
10
11      struct Edge {
12          int v;
13          T w;
14      };
15
16      int n;
17      vector<vector<Edge>> g;
18
19      explicit Dijkstra(int n) : n(n), g(n + 1) {
20          assert(n >= 0);
21      }
22
23      void add(int u, int v, T w) {
24          assert(w >= 0);
25          g[u].push_back({v, w});
26      }
27
28      void add2(int u, int v, T w) {
29          add(u, v, w);
30          add(v, u, w);
31      }
32
33      vector<T> solve(int s, vector<int>* fa = nullptr) const {
34          vector<T> dis(n + 1, INF);
35          if (fa) fa->assign(n + 1, -1);
36          using S = pair<T, int>;
37          priority_queue<S, vector<S>, greater<S>> q;
38          dis[s] = 0;
39          q.push({0, s});
40          while (!q.empty()) {
41              auto [d, u] = q.top();
42              q.pop();
43              if (d != dis[u]) continue;
44              for (auto [v, w] : g[u]) {
45                  if (dis[v] <= d + w) continue;
46                  dis[v] = d + w;
47                  if (fa) (*fa)[v] = u;
48                  q.push({dis[v], v});
49              }
50          }
51          return dis;
52      }
53 };

```

01B - 单源最短路 (Bellman-ford, 负权图) .cpp

```

1  /*
2  用途: 允许负边权的单源最短路, 并标记从源点可达的负环所能影响的点。
3  约定: 点编号 1..n, 边均为有向边; 所有有限距离应能放进 int。
4  复杂度: O(nm)。负环不影响的点, 其 distance 才有最短路意义。
5  */
6
7  struct Bellman {

```

```

8   static constexpr int INF = numeric_limits<int>::max() / 4;
9
10  struct Edge {
11      int u, v;
12      int w;
13  };
14
15  struct Result {
16      vector<int> dis;
17      vector<char> neg;
18
19      bool askNeg() const {
20          return any_of(neg.begin() + 1,
21                        neg.end(), [](char v) { return v; });
22      }
23  };
24
25  int n;
26  vector<Edge> e;
27  vector<vector<int>> g;
28
29  explicit Bellman(int n) : n(n), g(n + 1) {}
30
31  void add(int u, int v, int w) {
32      e.push_back({u, v, w});
33      g[u].push_back(v);
34  }
35
36  Result solve(int s) const {
37      vector<int> dis(n + 1, INF);
38      dis[s] = 0;
39
40      for (int k = 1; k < n; ++k) {
41          bool chg = false;
42          for (auto [u, v, w] : e) {
43              if (dis[u] == INF) continue;
44              if (dis[v] > dis[u] + w) {
45                  dis[v] = dis[u] + w;
46                  chg = true;
47              }
48          }
49          if (!chg) break;
50      }
51
52      vector<char> neg(n + 1, false);
53      queue<int> q;
54      for (auto [u, v, w] : e) {
55          if (dis[u] != INF && dis[v] > dis[u] + w && !neg[v]) {
56              neg[v] = true;
57              q.push(v);
58          }
59      }
60      while (!q.empty()) {
61          int u = q.front();
62          q.pop();
63          for (int v : g[u]) {
64              if (!neg[v]) {
65                  neg[v] = true;
66                  q.push(v);
67              }
68          }
69      }
70      return {move(dis), move(neg)};
71  }
72 };

```

01C - 单源最短路 (SPFA, 负权图) .cpp

```

1  /*
2  用途: 允许负边权的单源最短路; 若存在源点可达负环则返回 nullopt。
3  约定: 点编号 1..n, 边均为有向边; 最坏复杂度 O(nm), 通常仅在数据温和时使用。
4  所有有限距离应能放进 int。
5  */
6
7  struct SPFA {
8      static constexpr int INF = numeric_limits<int>::max() / 4;
9
10     struct Edge {
11         int v;
12         int w;

```

```

13     };
14
15     int n;
16     vector<vector<Edge>> g;
17
18     explicit SPFA(int n) : n(n), g(n + 1) {}
19
20     void add(int u, int v, int w) {
21         g[u].push_back({v, w});
22     }
23
24     // 返回 nullopt 表示存在从 source 可达的负环。
25     optional<vector<int>> solve(int s) const {
26         vector<int> dis(n + 1, INF);
27         vector<int> m(n + 1);
28         vector<char> inq(n + 1, false);
29         queue<int> q;
30
31         dis[s] = 0;
32         q.push(s);
33         inq[s] = true;
34
35         while (!q.empty()) {
36             int u = q.front();
37             q.pop();
38             inq[u] = false;
39
40             for (auto [v, w] : g[u]) {
41                 if (dis[v] <= dis[u] + w) continue;
42                 dis[v] = dis[u] + w;
43                 m[v] = m[u] + 1;
44                 if (m[v] >= n) return nullopt;
45                 if (!inq[v]) {
46                     inq[v] = true;
47                     q.push(v);
48                 }
49             }
50         }
51         return dis;
52     }
53 };

```

01D - 多源最短路 (APSP) .cpp

```

1  /*
2  用途: Floyd-Warshall 全源最短路。
3  约定: 点编号 1..n; 允许负边权, 但不允许负环。调用 solve 后 dis[i][j] 即答案。
4  复杂度:  $O(n^3)$  时间、 $O(n^2)$  空间, 适合 n 约 500 以内的稠密图。
5  */
6
7  struct Floyd {
8      static constexpr int INF = numeric_limits<int>::max() / 4;
9
10     int n;
11     vector<vector<int>> dis;
12
13     explicit Floyd(int n) : n(n), dis(n + 1, vector<int>(n + 1, INF)) {
14         for (int i = 1; i <= n; ++i) dis[i][i] = 0;
15     }
16
17     void add(int u, int v, int w) {
18         dis[u][v] = min(dis[u][v], w);
19     }
20
21     void add2(int u, int v, int w) {
22         add(u, v, w);
23         add(v, u, w);
24     }
25
26     void solve() {
27         for (int k = 1; k <= n; ++k) {
28             for (int i = 1; i <= n; ++i) {
29                 if (dis[i][k] == INF) continue;
30                 for (int j = 1; j <= n; ++j) {
31                     if (dis[k][j] == INF) continue;
32                     dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
33                 }
34             }
35         }
36     }
37 };

```

```

33     }
34     }
35     }
36 }
37
38 bool askNeg() const {
39     for (int i = 1; i <= n; ++i) {
40         if (dis[i][i] < 0) return true;
41     }
42     return false;
43 }
44 };

```

01E - 0-1 BFS.cpp

```

1  /*
2  用途：边权仅为 0 或 1 时的单源最短路，比 Dijkstra 更轻。
3  约定：点编号 1..n; add 加有向边，add2 加无向边。
4  复杂度：O(n + m)。
5  */
6
7  struct ZeroOneBFS {
8      static constexpr int INF = numeric_limits<int>::max() / 4;
9
10     struct Edge {
11         int v;
12         int w;
13     };
14
15     int n;
16     vector<vector<Edge>> g;
17
18     explicit ZeroOneBFS(int n) : n(n), g(n + 1) {}
19
20     void add(int u, int v, int w) {
21         assert(w == 0 || w == 1);
22         g[u].push_back({v, w});
23     }
24
25     void add2(int u, int v, int w) {
26         add(u, v, w);
27         add(v, u, w);
28     }
29
30     vector<int> solve(int s) const {
31         vector<int> dis(n + 1, INF);
32         deque<int> q;
33         dis[s] = 0;
34         q.push_back(s);
35
36         while (!q.empty()) {
37             int u = q.front();
38             q.pop_front();
39             for (auto [v, w] : g[u]) {
40                 if (dis[v] <= dis[u] + w) continue;
41                 dis[v] = dis[u] + w;
42                 if (w == 0) {
43                     q.push_front(v);
44                 } else {
45                     q.push_back(v);
46                 }
47             }
48         }
49         return dis;
50     }
51 };

```

01F - 差分约束.cpp

```

1  /*
2  用途：差分约束系统。addLE(u,v,w) 表示  $x[v] - x[u] \leq w$ ，对应边  $u \rightarrow v$  权  $w$ 。
3  solve 返回一组可行解；存在矛盾（负环）时返回 nullopt。变量编号 0..n-1。
4  复杂度：SPFA 平均较快，最坏  $O(nm)$ ；对抗数据可改用 Bellman-Ford。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7
8  template <class T = int> struct DiffCons {
9      struct Edge {

```

```

10     int v;
11     T w;
12 };
13
14     int n;
15     vector<vector<Edge>> g;
16     explicit DiffCons(int n) : n(n), g(n) { assert(n >= 0); }
17
18     void addLE(int u, int v, T w) {
19         assert(0 <= u && u < n && 0 <= v && v < n);
20         g[u].push_back({v, w});
21     }
22     void addGE(int u, int v, T w) { addLE(v, u, -w); }
23     void addEQ(int u, int v, T w) {
24         addLE(u, v, w);
25         addGE(u, v, w);
26     }
27
28     optional<vector<T>> solve() const {
29         vector<T> d(n, T{});
30         vector<int> len(n), q;
31         vector<char> in(n, true);
32         q.reserve(n * 2);
33         for (int i = 0; i < n; ++i) q.push_back(i); // 等价于权 0 超级源。
34         for (int h = 0; h < q.size(); ++h) {
35             int u = q[h];
36             in[u] = false;
37             for (auto [v, w] : g[u]) {
38                 if (d[v] <= d[u] + w) continue;
39                 d[v] = d[u] + w;
40                 len[v] = len[u] + 1;
41                 if (len[v] >= n) return nullopt;
42                 if (!in[v]) in[v] = true, q.push_back(v);
43             }
44             if (h > 2 * n && h * 2 > q.size()) {
45                 q.erase(q.begin(), q.begin() + h + 1);
46                 h = -1;
47             }
48         }
49         return d;
50     }
51 };

```

01G - Johnson全源最短路（稀疏负权图）.cpp

```

1  /*
2  用途：稀疏图全源最短路，允许负边；有负环时返回 nullopt。
3  复杂度： $O(nm+n(m+n)\log n)$ ，空间  $O(n^2+m)$ 。点号 1..n。
4  约定：T 为有符号距离类型；边权和任意简单路长度应落在 T 的安全范围内。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7
8  template <class T = int> struct Johnson {
9      static constexpr T INF = numeric_limits<T>::max() / 4;
10     struct E { int u, v; T w; };
11     int n;
12     vector<E> e;
13     vector<vector<pair<int, T>>> g;
14
15     explicit Johnson(int _n) : n(_n), g(_n + 1) {
16         static_assert(is_signed_v<T>);
17         assert(n >= 0);
18     }
19     void add(int u, int v, T w) {
20         assert(1 <= u && u <= n && 1 <= v && v <= n);
21         e.push_back({u, v, w});
22     }
23
24     optional<vector<vector<T>>> solve() {
25         vector<T> h(n + 1);
26         for (int k = 1; k <= n; ++k) {
27             bool f = false;
28             for (auto [u, v, w] : e) if (h[v] > h[u] + w)
29                 h[v] = h[u] + w, f = true;
30             if (!f) break;
31             if (k == n) return nullopt;
32         }
33         g.assign(n + 1, {});
34         for (auto [u, v, w] : e) {
35             T x = w + h[u] - h[v];

```

```

36         assert(x >= 0);
37         g[u].push_back({v, x});
38     }
39
40     vector<vector<T>> a(n + 1, vector<T>(n + 1, INF));
41     for (int s = 1; s <= n; ++s) {
42         vector<T> d(n + 1, INF);
43         priority_queue<pair<T, int>, vector<pair<T, int>>, greater<pair<T, int>>> q;
44         d[s] = 0, q.push({0, s});
45         while (!q.empty()) {
46             auto [du, u] = q.top(); q.pop();
47             if (du != d[u]) continue;
48             for (auto [v, w] : g[u]) if (du <= INF - w && d[v] > du + w)
49                 d[v] = du + w, q.push({d[v], v});
50         }
51         for (int v = 1; v <= n; ++v) if (d[v] != INF) a[s][v] = d[v] - h[s] + h[v];
52     }
53     return a;
54 }
55 };

```

01H - 单源最短路 (Dijkstra, 稠密图) .cpp

```

1  /*
2  用途: 邻接矩阵 Dijkstra. 适合稠密非负权图。
3  用法: Dijkstra<> g(n); add/add2 后 solve(s, &fa)。
4  约定: 点号为 1..n; fa 非空时记录一棵最短路树。
5  复杂度:  $O(n^2)$  时间、 $O(n^2)$  空间。稀疏图优先 01A 的堆优化版。
6  性能: 默认 int 为 long long 会使邻接矩阵翻倍; 仅在实际内存受限且权值可装 i32 时再局部缩窄。
7  */
8
9  template <class T = int> struct Dijkstra {
10     static constexpr T INF = numeric_limits<T>::max() / 4;
11
12     int n;
13     vector<vector<T>> g;
14
15     explicit Dijkstra(int n) : n(n), g(n + 1, vector<T>(n + 1, INF)) {}
16
17     void add(int u, int v, T w) {
18         assert(w >= 0);
19         if (w < g[u][v]) g[u][v] = w;
20     }
21
22     void add2(int u, int v, T w) {
23         add(u, v, w);
24         add(v, u, w);
25     }
26
27     vector<T> solve(int s, vector<int>* fa = nullptr) const {
28         vector<T> dis(n + 1, INF);
29         vector<char> vis(n + 1);
30         if (fa) fa->assign(n + 1, -1);
31         dis[s] = 0;
32         for (int i = 1; i <= n; ++i) {
33             int u = 0;
34             for (int v = 1; v <= n; ++v) {
35                 if (!vis[v] && (u == 0 || dis[v] < dis[u])) u = v;
36             }
37             if (u == 0 || dis[u] == INF) break;
38             vis[u] = true;
39             for (int v = 1; v <= n; ++v) {
40                 if (vis[v] || g[u][v] == INF || dis[v] <= dis[u] + g[u][v]) continue;
41                 dis[v] = dis[u] + g[u][v];
42                 if (fa) (*fa)[v] = u;
43             }
44         }
45         return dis;
46     }
47 };

```

01I - DAG路径 (最短、最长) .cpp

```

1  /*
2  用途: DAG 上从单源出发的最短/最长路, 可含负边。
3  用法: DAG<int> g(n); add(u,v,w) 后调用 askMin(s) 或 askMax(s)。
4  接口: 若存在环, askMin/askMax 返回 nullptr; 否则返回所有点的距离。
5  复杂度:  $O(n+m)$ 。不可达点分别为 INF / LOW。

```

```

6  */
7
8  template <class T = int> struct DAG {
9      static constexpr T INF = numeric_limits<T>::max() / 4;
10     static constexpr T LOW = numeric_limits<T>::lowest() / 4;
11
12     struct Edge {
13         int v;
14         T w;
15     };
16
17     int n;
18     vector<vector<Edge>> g;
19     vector<int> deg;
20
21     explicit DAG(int n) : n(n), g(n + 1), deg(n + 1) {}
22
23     void add(int u, int v, T w) {
24         g[u].push_back({v, w});
25         ++deg[v];
26     }
27
28     vector<int> topo() const {
29         vector<int> d = deg, ord;
30         queue<int> q;
31         for (int u = 1; u <= n; ++u) {
32             if (!d[u]) q.push(u);
33         }
34         while (!q.empty()) {
35             int u = q.front();
36             q.pop();
37             ord.push_back(u);
38             for (auto [v, w] : g[u]) {
39                 if (--d[v]) q.push(v);
40             }
41         }
42         return ord;
43     }
44
45     optional<vector<T>> askMin(int s, vector<int>* fa = nullptr) const {
46         vector<int> ord = topo();
47         if (ord.size() != deg.size() - 1) return nullopt;
48         vector<T> dis(n + 1, INF);
49         if (fa) fa->assign(n + 1, -1);
50         dis[s] = 0;
51         for (int u : ord) {
52             if (dis[u] == INF) continue;
53             for (auto [v, w] : g[u]) {
54                 if (dis[v] <= dis[u] + w) continue;
55                 dis[v] = dis[u] + w;
56                 if (fa) (*fa)[v] = u;
57             }
58         }
59         return dis;
60     }
61
62     optional<vector<T>> askMax(int s, vector<int>* fa = nullptr) const {
63         vector<int> ord = topo();
64         if (ord.size() != deg.size() - 1) return nullopt;
65         vector<T> dis(n + 1, LOW);
66         if (fa) fa->assign(n + 1, -1);
67         dis[s] = 0;
68         for (int u : ord) {
69             if (dis[u] == LOW) continue;
70             for (auto [v, w] : g[u]) {
71                 if (dis[v] >= dis[u] + w) continue;
72                 dis[v] = dis[u] + w;
73                 if (fa) (*fa)[v] = u;
74             }
75         }
76         return dis;
77     }
78 };

```

01J - 最短和次短路 (Dijkstra, 计数) .cpp

```

1  /*
2  用途：正权图的最短路、严格次短路及其条数。
3  约定：点编号 1..n；边权必须严格为正。计数默认用 int，取模时传入 MInt。
4  接口：solve(s) 返回 dis[u][0/1]、cnt[u][0/1]；INF 表示不存在。

```



```

5 复杂度:  $O((n+m) \log n)$ 。计数的是对应长度的不同有向边序列; 正权保证同距离计数已在弹堆前汇总。
6  */
7  template <class T = int, class Z = int> struct TwoDij {
8      static constexpr T INF = numeric_limits<T>::max() / 4;
9
10     struct Edge {
11         int v;
12         T w;
13     };
14     struct Res {
15         vector<array<T, 2>> dis;
16         vector<array<Z, 2>> cnt;
17     };
18
19     int n;
20     vector<vector<Edge>> g;
21
22     explicit TwoDij(int n) : n(n), g(n + 1) {}
23
24     void add(int u, int v, T w) {
25         assert(w > T{} && w < INF);
26         g[u].push_back({v, w});
27     }
28
29     void add2(int u, int v, T w) {
30         add(u, v, w);
31         add(v, u, w);
32     }
33
34     Res solve(int s) const {
35         vector<array<T, 2>> dis(n + 1, {INF, INF});
36         vector<array<Z, 2>> cnt(n + 1);
37         vector<array<char, 2>> vis(n + 1);
38         using S = tuple<T, int, int>;
39         priority_queue<S, vector<S>, greater<S>> q;
40         dis[s][0] = T{};
41         cnt[s][0] = Z(1);
42         q.emplace(T{}, s, 0);
43
44         while (!q.empty()) {
45             auto [d, u, k] = q.top();
46             q.pop();
47             if (d != dis[u][k] || vis[u][k]) continue;
48             vis[u][k] = true;
49             for (auto [v, w] : g[u]) {
50                 T nd = d + w;
51                 if (nd < dis[v][0]) {
52                     dis[v][1] = dis[v][0];
53                     cnt[v][1] = cnt[v][0];
54                     dis[v][0] = nd;
55                     cnt[v][0] = cnt[u][k];
56                     q.emplace(dis[v][0], v, 0);
57                     if (dis[v][1] != INF) q.emplace(dis[v][1], v, 1);
58                 } else if (nd == dis[v][0]) {
59                     cnt[v][0] += cnt[u][k];
60                 } else if (nd < dis[v][1]) {
61                     dis[v][1] = nd;
62                     cnt[v][1] = cnt[u][k];
63                     q.emplace(nd, v, 1);
64                 } else if (nd == dis[v][1]) {
65                     cnt[v][1] += cnt[u][k];
66                 }
67             }
68         }
69         return {move(dis), move(cnt)};
70     }
71 };

```

02A - 最小生成树 (Kruskal) .cpp

```

1  /*
2  用途: 无向带权图的最小生成森林; ok 为真时即最小生成树。
3  约定: 点编号 1..n, 可含重边和负边权。
4  复杂度:  $O(m \log m)$ 。
5  */
6
7  struct MST {
8      struct Edge {
9          int u, v;
10         int w;

```

```

11     };
12
13     struct Result {
14         bool ok;
15         int w;           // 非连通时为最小生成森林的权值。
16         vector<Edge> vis; // 非连通时为最小生成森林的边。
17     };
18
19     struct DSU {
20         vector<int> fa, sz;
21
22         explicit DSU(int n) : fa(n + 1), sz(n + 1, 1) {
23             iota(fa.begin(), fa.end(), 0);
24         }
25
26         int find(int x) {
27             while (x != fa[x]) {
28                 x = fa[x] = fa[fa[x]];
29             }
30             return x;
31         }
32
33         bool merge(int x, int y) {
34             x = find(x);
35             y = find(y);
36             if (x == y) return false;
37             if (sz[x] < sz[y]) swap(x, y);
38             fa[y] = x;
39             sz[x] += sz[y];
40             return true;
41         }
42     };
43
44     int n;
45     vector<Edge> e;
46
47     explicit MST(int n) : n(n) {}
48
49     void add(int u, int v, int w) {
50         e.push_back({u, v, w});
51     }
52
53     Result solve() const {
54         vector<Edge> ord = e;
55         sort(ord.begin(), ord.end(), [](const Edge &a, const Edge &b) { return a.w < b.w; });
56
57         DSU d(n);
58         int sum = 0;
59         vector<Edge> vis;
60         for (const Edge &x : ord) {
61             if (!d.merge(x.u, x.v)) continue;
62             sum += x.w;
63             vis.push_back(x);
64         }
65         return {vis.size() == n - 1, sum, move(vis)};
66     }
67 };

```

02B - 最小生成树 (Prim) .cpp

```

1  /*
2  用途：无向带权图的最小生成森林；ok 为真时即最小生成树。
3  约定：点编号 1..n，可含重边和负边权。
4  复杂度：O(m log n)。稠密图通常可改为 O(n^2) 朴素 Prim。
5  */
6
7  struct MST {
8
9      struct Edge {
10         int v;
11         int w;
12     };
13
14     struct CEdge {
15         int u, v;
16         int w;
17     };
18
19     struct Result {
20         bool ok;

```

```

21         int w; // 非连通时为最小生成森林的权值。
22         vector<CEdge> vis; // 非连通时为最小生成森林的边。
23     };
24
25     int n;
26     vector<vector<Edge>> g;
27
28     explicit MST(int n) : n(n), g(n + 1) {}
29
30     void add(int u, int v, int w) {
31         g[u].push_back({v, w});
32         g[v].push_back({u, w});
33     }
34
35     Result solve() const {
36         using State = tuple<int, int, int>; // weight, vertex, parent
37         priority_queue<State, vector<State>, greater<State>> q;
38         vector<char> use(n + 1, false);
39         vector<CEdge> vis;
40         int sum = 0;
41         int cc = 0;
42
43         for (int s = 1; s <= n; ++s) {
44             if (use[s]) continue;
45             ++cc;
46             q.push({0, s, 0});
47             while (!q.empty()) {
48                 auto [w, u, fa] = q.top();
49                 q.pop();
50                 if (use[u]) continue;
51                 use[u] = true;
52                 sum += w;
53                 if (fa != 0) vis.push_back({fa, u, w});
54
55                 for (auto [v, w] : g[u]) {
56                     if (!use[v]) q.push({w, v, u});
57                 }
58             }
59         }
60         return {cc <= 1, sum, move(vis)};
61     }
62 };

```

02C - 有向最小生成树 (Chu-Liu-Edmonds) .cpp

```

1  /*
2  用途: 有向图以 root 为根的最小树形图 (每个非根点恰有一条入边, 且均从根可达)。
3  约定: 点编号 0..n-1, 允许负权和重边; 返回 nullopt 表示不存在。仅返回最小权值。
4  复杂度: O(nm)。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7
8  template <class T = int> struct DMST {
9      struct Edge {
10         int u, v;
11         T w;
12     };
13
14     int n;
15     vector<Edge> e;
16     explicit DMST(int n) : n(n) { assert(n >= 0); }
17
18     void add(int u, int v, T w) {
19         assert(0 <= u && u < n && 0 <= v && v < n);
20         e.push_back({u, v, w});
21     }
22
23     optional<T> solve(int rt) const {
24         assert(0 <= rt && rt < n);
25         const T inf = numeric_limits<T>::max() / 4;
26         int z = n, r = rt;
27         vector<Edge> a = e;
28         T ans{};
29         while (true) {
30             vector<T> in(z, inf);
31             vector<int> pre(z, -1);
32             for (auto [u, v, w] : a) {
33                 if (u != v && w < in[v]) in[v] = w, pre[v] = u;
34             }
35             in[r] = T{};

```

```

36     for (int i = 0; i < z; ++i) {
37         if (in[i] == inf) return nullopt;
38         ans += in[i];
39     }
40
41     int cnt = 0;
42     vector<int> id(z, -1), vis(z, -1);
43     for (int i = 0; i < z; ++i) {
44         int v = i;
45         while (vis[v] != i && id[v] == -1 && v != r) vis[v] = i, v = pre[v];
46         if (v == r || id[v] != -1) continue;
47         for (int u = pre[v]; u != v; u = pre[u]) id[u] = cnt;
48         id[v] = cnt++;
49     }
50     if (!cnt) break;
51     for (int i = 0; i < z; ++i) if (id[i] == -1) id[i] = cnt++;
52
53     vector<Edge> b;
54     b.reserve(a.size());
55     for (auto [u, v, w] : a) {
56         int x = id[u], y = id[v];
57         if (x != y) b.push_back({x, y, w - in[v]});
58     }
59     r = id[r], z = cnt, a.swap(b);
60 }
61 return ans;
62 }
63 };

```

02D - 最小生成树 (Prim, 稠密图) .cpp

```

1  /*
2  用途: 邻接矩阵 Prim, 求无向带权图的最小生成森林。
3  用法: MST g(n): add(u,v,w) 后 solve(), ok 为真时原图连通。
4  复杂度:  $O(n^2)$  时间、 $O(n^2)$  空间。稀疏图优先 02A 或 02B。
5  性能: 邻接矩阵受 #define int long long 影响显著; 只在实际内存紧张且权值可装 i32 时缩窄。
6  */
7
8  struct MST {
9      static constexpr int INF = numeric_limits<int>::max() / 4;
10
11      struct CEdge {
12          int u, v, w;
13      };
14
15      struct Result {
16          bool ok;
17          int w;
18          vector<CEdge> vis;
19      };
20
21      int n;
22      vector<vector<int>>> g;
23
24      explicit MST(int n) : n(n), g(n + 1, vector<int>(n + 1, INF)) {}
25
26      void add(int u, int v, int w) {
27          if (w < g[u][v]) {
28              g[u][v] = w;
29              g[v][u] = w;
30          }
31      }
32
33      Result solve() const {
34          vector<int> dis(n + 1, INF), fa(n + 1);
35          vector<char> use(n + 1);
36          vector<CEdge> vis;
37          int sum = 0, cc = 0;
38
39          for (int i = 1; i <= n; ++i) {
40              int u = 0;
41              for (int v = 1; v <= n; ++v) {
42                  if (!use[v] && (u == 0 || dis[v] < dis[u])) u = v;
43              }
44              if (dis[u] == INF) {
45                  dis[u] = 0;
46                  fa[u] = 0;
47                  ++cc;
48              }
49              use[u] = true;

```

```

50         sum += dis[u];
51         if (fa[u]) vis.push_back({fa[u], u, dis[u]});
52
53         for (int v = 1; v <= n; ++v) {
54             if (!use[v] && g[u][v] < dis[v]) {
55                 dis[v] = g[u][v];
56                 fa[v] = u;
57             }
58         }
59     }
60     return {cc <= 1, sum, move(vis)};
61 }
62 };

```

03A - 拓扑排序 (Kahn) .cpp

```

1  /*
2  用途: 有向无环图 (DAG) 的拓扑排序与判环。
3  约定: 点编号 1..n, add 添加有向边。
4  复杂度:  $O(n + m)$ 。solve() 返回 nullopt 表示图中存在有向环。
5  */
6  struct Topo {
7      int n;
8      vector<vector<int>> g;
9      vector<int> in;
10
11      explicit Topo(int n) : n(n), g(n + 1), in(n + 1) {}
12
13      void add(int u, int v) {
14          g[u].push_back(v);
15          ++in[v];
16      }
17
18      optional<vector<int>> solve() const {
19          vector<int> deg = in;
20          queue<int> q;
21          for (int u = 1; u <= n; ++u) {
22              if (deg[u] == 0) q.push(u);
23          }
24
25          vector<int> ord;
26          while (!q.empty()) {
27              int u = q.front();
28              q.pop();
29              ord.push_back(u);
30              for (int v : g[u]) {
31                  if (--deg[v] == 0) q.push(v);
32              }
33          }
34          if (ord.size() != n) return nullopt;
35          return ord;
36      }
37 };

```

03B - 强连通分量 (Tarjan) .cpp

```

1  /*
2  用途: 有向图强连通分量、缩点 DAG。
3  约定: 点编号 1..n, add 添加有向边。bel 编号为 0..cc-1,
4  编号顺序不承诺拓扑性质; dag 是去重后的缩点图。
5  复杂度:  $O(n + m)$ 。
6  */
7  struct SCC {
8      struct Result {
9          int cc;
10         vector<int> bel;
11         vector<vector<int>> scc;
12         vector<vector<int>> dag;
13     };
14
15     int n;
16     vector<vector<int>> g;
17
18     explicit SCC(int n) : n(n), g(n + 1) {}
19
20     void add(int u, int v) {
21         g[u].push_back(v);
22     }

```

```

23
24 Result solve() const {
25     vector<int> dfn(n + 1), low(n + 1), stk, bel(n + 1, -1);
26     vector<char> ins(n + 1, false);
27     vector<vector<int>> scc;
28     int tim = 0;
29
30     auto dfs = [&](auto&& go, int u) -> void {
31         dfn[u] = low[u] = ++tim;
32         stk.push_back(u);
33         ins[u] = true;
34
35         for (int v : g[u]) {
36             if (!dfn[v]) {
37                 go(go, v);
38                 low[u] = min(low[u], low[v]);
39             } else if (ins[v]) {
40                 low[u] = min(low[u], dfn[v]);
41             }
42         }
43
44         if (dfn[u] != low[u]) return;
45         int id = scc.size();
46         scc.push_back({});
47         while (true) {
48             int v = stk.back();
49             stk.pop_back();
50             ins[v] = false;
51             bel[v] = id;
52             scc.back().push_back(v);
53             if (v == u) break;
54         }
55     };
56
57     for (int u = 1; u <= n; ++u) {
58         if (!dfn[u]) dfs(dfs, u);
59     }
60
61     vector<vector<int>> dag(scc.size());
62     for (int u = 1; u <= n; ++u) {
63         for (int v : g[u]) {
64             int fr = bel[u], to = bel[v];
65             if (fr != to) dag[fr].push_back(to);
66         }
67     }
68     for (auto& e : dag) {
69         sort(e.begin(), e.end());
70         e.erase(unique(e.begin(), e.end()), e.end());
71     }
72     return {scc.size(), move(bel), move(scc), move(dag)};
73 }
74 };

```

03C - 2-SAT.cpp

```

1  /*
2  用途: 布尔可满足性 (每个子句至多两个文字)。
3  约定: 变量编号 0..m-1; lit(x,true) 表示 x, lit(x,false) 表示 !x。
4  接口: add2(x,vx,y,vy) 添加 (x == vx) OR (y == vy), force(x,vx) 强制取值。
5  复杂度: O(V + E)。solve() 返回 nullopt 表示无解, 否则 ans[x] 是变量 x 的取值。
6  */
7  struct TwoSAT {
8      int m;
9      vector<vector<int>> g;
10
11      explicit TwoSAT(int m) : m(m), g(2 * m) {}
12
13      int lit(int x, bool val) const {
14          assert(0 <= x && x < m);
15          return 2 * x + (val ? 0 : 1);
16      }
17
18      static int neg(int id) {
19          return id ^ 1;
20      }
21
22      void add(int fr, int to) {
23          g[fr].push_back(to);
24      }
25

```

```

26 void addOr(int a, int b) {
27     add(neg(a), b);
28     add(neg(b), a);
29 }
30
31 void add2(int x, bool vx, int y, bool vy) {
32     addOr(lit(x, vx), lit(y, vy));
33 }
34
35 void force(int x, bool val) {
36     int a = lit(x, val);
37     addOr(a, a);
38 }
39
40 optional<vector<char>> solve() const {
41     const int n = 2 * m;
42     vector<int> dfn(n), low(n), stk, bel(n, -1);
43     vector<char> ins(n, false);
44     int tim = 0, cc = 0;
45
46     auto dfs = [&](auto&& go, int u) -> void {
47         dfn[u] = low[u] = ++tim;
48         stk.push_back(u);
49         ins[u] = true;
50         for (int v : g[u]) {
51             if (!dfn[v]) {
52                 go(go, v);
53                 low[u] = min(low[u], low[v]);
54             } else if (ins[v]) {
55                 low[u] = min(low[u], dfn[v]);
56             }
57         }
58         if (dfn[u] != low[u]) return;
59         while (true) {
60             int v = stk.back();
61             stk.pop_back();
62             ins[v] = false;
63             bel[v] = cc;
64             if (v == u) break;
65         }
66         ++cc;
67     };
68
69     for (int u = 0; u < n; ++u) {
70         if (!dfn[u]) dfs(dfs, u);
71     }
72
73     vector<char> ans(m);
74     for (int x = 0; x < m; ++x) {
75         int vx = lit(x, true);
76         int vy = lit(x, false);
77         if (bel[vx] == bel[vy]) return nullopt;
78         // Tarjan 的分量按出边方向从后向前编号, 编号更小者应优先取真。
79         ans[x] = bel[vx] < bel[vy];
80     }
81     return ans;
82 }
83 };

```

04A - 边双连通分量 (桥) .cpp

```

1  /*
2  用途: 无向图找桥、删桥后的边双连通分量, 以及桥树 (非连通图时为桥森林)。
3  约定: 点编号 1..n; 支持重边, 边编号为 add 的返回值 (0..m-1)。
4  复杂度: O(n + m)。
5  */
6  struct EBCC {
7      struct Result {
8          int cc;
9          vector<int> bel;           // 点所属边双连通分量, 编号 0..componentCount-1。
10         vector<char> br;          // 对应 add 的边编号。
11         vector<vector<int>> tr;    // 分量编号上的桥森林。
12     };
13
14     int n;
15     vector<pair<int, int>> e;
16     vector<vector<pair<int, int>>> g;
17
18     explicit EBCC(int n) : n(n), g(n + 1) {}
19

```

```

20     int add(int u, int v) {
21         int id = e.size();
22         e.push_back({u, v});
23         g[u].push_back({v, id});
24         g[v].push_back({u, id});
25         return id;
26     }
27
28     Result solve() const {
29         const int m = e.size();
30         vector<int> dfn(n + 1), low(n + 1);
31         vector<char> br(m, false);
32         int tim = 0;
33
34         auto dfs = [&](auto&& go, int u, int pe) -> void {
35             dfn[u] = low[u] = ++tim;
36             for (auto [v, id] : g[u]) {
37                 if (id == pe) continue;
38                 if (!dfn[v]) {
39                     go(go, v, id);
40                     low[u] = min(low[u], low[v]);
41                     if (low[v] > dfn[u]) br[id] = true;
42                 } else {
43                     low[u] = min(low[u], dfn[v]);
44                 }
45             }
46         };
47
48         for (int u = 1; u <= n; ++u) {
49             if (!dfn[u]) dfs(dfs, u, -1);
50         }
51
52         vector<int> bel(n + 1, -1);
53         int cc = 0;
54         for (int s = 1; s <= n; ++s) {
55             if (bel[s] != -1) continue;
56             stack<int> st;
57             st.push(s);
58             bel[s] = cc;
59             while (!st.empty()) {
60                 int u = st.top();
61                 st.pop();
62                 for (auto [v, id] : g[u]) {
63                     if (br[id] || bel[v] != -1) continue;
64                     bel[v] = cc;
65                     st.push(v);
66                 }
67             }
68             ++cc;
69         }
70
71         vector<vector<int>> tr(cc);
72         for (int id = 0; id < m; ++id) {
73             if (!br[id]) continue;
74             auto [u, v] = e[id];
75             int x = bel[u], y = bel[v];
76             tr[x].push_back(y);
77             tr[y].push_back(x);
78         }
79         return {cc, move(bel), move(br), move(tr)};
80     }
81 };

```

04B - 点双连通分量（割点、圆方树）.cpp

```

1  /*
2  用途：无向图找割点、点双连通分量，并建立圆方树（block-cut tree）。
3  约定：点编号 1..n；支持重边；默认不放自环。原图非连通时 blockCutTree 为森林。
4  结果中原点仍为 1..n，第 i 个点双连通分量对应圆方树节点 n + i + 1。
5  复杂度：O(n + m)。
6  */
7  struct VBCC {
8      struct Edge {
9          int u, v;
10     };
11
12     struct Result {
13         vector<char> cut;
14         vector<vector<int>> cc;
15         vector<vector<int>> tr;

```



```

16     };
17
18     int n;
19     vector<Edge> e;
20     vector<vector<pair<int, int>>> g;
21
22     explicit VBCC(int n) : n(n), g(n + 1) {}
23
24     int add(int u, int v) {
25         int id = e.size();
26         e.push_back({u, v});
27         g[u].push_back({v, id});
28         g[v].push_back({u, id});
29         return id;
30     }
31
32     Result solve() const {
33         vector<int> dfn(n + 1), low(n + 1), stk;
34         vector<char> cut(n + 1, false);
35         vector<vector<int>> cc;
36         int tim = 0;
37
38         auto dfs = [&](auto&& go, int u, int pe) -> void {
39             dfn[u] = low[u] = ++tim;
40             int son = 0;
41             for (auto [v, id] : g[u]) {
42                 if (id == pe) continue;
43                 if (!dfn[v]) {
44                     ++son;
45                     stk.push_back(id);
46                     go(go, v, id);
47                     low[u] = min(low[u], low[v]);
48
49                     if (low[v] >= dfn[u]) {
50                         if (pe != -1 || son > 1) cut[u] = true;
51                         vector<int> cur;
52                         while (true) {
53                             int eid = stk.back();
54                             stk.pop_back();
55                             cur.push_back(e[eid].u);
56                             cur.push_back(e[eid].v);
57                             if (eid == id) break;
58                         }
59                         sort(cur.begin(), cur.end());
60                         cur.erase(unique(cur.begin(), cur.end()), cur.end());
61                         cc.push_back(move(cur));
62                     }
63                 } else if (dfn[v] < dfn[u]) {
64                     stk.push_back(id);
65                     low[u] = min(low[u], dfn[v]);
66                 }
67             }
68         };
69
70         for (int u = 1; u <= n; ++u) {
71             if (dfn[u]) continue;
72             if (g[u].empty()) {
73                 dfn[u] = low[u] = ++tim;
74                 cc.push_back({u});
75             } else {
76                 dfs(dfs, u, -1);
77             }
78         }
79
80         vector<vector<int>> tr(n + cc.size() + 1);
81         for (int id = 0; id < cc.size(); ++id) {
82             int b = n + id + 1;
83             for (int u : cc[id]) {
84                 tr[u].push_back(b);
85                 tr[b].push_back(u);
86             }
87         }
88         return {move(cut), move(cc), move(tr)};
89     }
90 };

```

05A - 欧拉路 (有向、无向) .cpp

```

1  /*
2  用途: Hierholzer 求欧拉通路/回路, 自动检查度数条件与非孤立部分连通性。

```

```

3 约定：点编号 1..n；构造时指定 dir。返回的点列长度为 m + 1；无解返回 nullopt。
4 没有边时返回 {1}。支持重边和自环。
5 复杂度：O(n + m)。
6 */
7 struct EulerTrail {
8     int n;
9     bool dir;
10     int m = 0;
11     vector<vector<pair<int, int>>> g;
12     vector<int> in, out, deg;
13
14     EulerTrail(int n, bool dir)
15         : n(n), dir(dir), g(n + 1), in(n + 1), out(n + 1), deg(n + 1) {}
16
17     int add(int u, int v) {
18         int id = m++;
19         g[u].push_back({v, id});
20         if (dir) {
21             ++out[u];
22             ++in[v];
23         } else {
24             g[v].push_back({u, id});
25             ++deg[u];
26             ++deg[v];
27         }
28         return id;
29     }
30
31     optional<vector<int>> solve() const {
32         if (m == 0) return vector<int>{1};
33         int s = -1;
34
35         if (dir) {
36             int cs = 0, ct = 0;
37             for (int u = 1; u <= n; ++u) {
38                 int d = out[u] - in[u];
39                 if (d == 1) {
40                     ++cs;
41                     s = u;
42                 } else if (d == -1) {
43                     ++ct;
44                 } else if (d != 0) {
45                     return nullopt;
46                 }
47             }
48             if (cs != ct || cs > 1) return nullopt;
49             if (s == -1) {
50                 for (int u = 1; u <= n; ++u) {
51                     if (out[u] > 0) {
52                         s = u;
53                         break;
54                     }
55                 }
56             }
57         } else {
58             int odd = 0;
59             for (int u = 1; u <= n; ++u) {
60                 if (deg[u] & 1) {
61                     ++odd;
62                     s = u;
63                 }
64             }
65             if (odd != 0 && odd != 2) return nullopt;
66             if (s == -1) {
67                 for (int u = 1; u <= n; ++u) {
68                     if (deg[u] > 0) {
69                         s = u;
70                         break;
71                     }
72                 }
73             }
74         }
75
76         vector<int> it(n + 1);
77         vector<char> vis(m, false);
78         vector<int> stk = {s}, pth;
79         while (!stk.empty()) {
80             int u = stk.back();
81             while (it[u] < g[u].size() && vis[g[u][it[u]].second]) {
82                 ++it[u];
83             }
84             if (it[u] == g[u].size()) {

```

```
85         pth.push_back(u);
86         stk.pop_back();
87     } else {
88         auto [v, id] = g[u][it[u]++];
89         vis[id] = true;
90         stk.push_back(v);
91     }
92 }
93 if (pth.size() != m + 1) return nullopt;
94 reverse(pth.begin(), pth.end());
95 return pth;
96 }
97 };
```

图论：匹配、全局最小割与环

14 个代码模板 · 林月 · v1.0.0 · 2026.8.1

06A - 二分图最大匹配 (Hopcroft-Karp) .cpp

```

1  /*
2  用途：二分图最大匹配，并可由最大匹配求最小点覆盖（König 定理）。
3  约定：左部编号 1..nl, 右部编号 1..nr; add(l,r)。
4  复杂度：最大匹配  $O(m \sqrt{n})$ ; askCov 为  $O(n + m)$ 。
5  */
6  struct HK {
7      struct Result {
8          int cnt;
9          vector<int> ml; // matchLeft[u] = 匹配到的右部点, 0 表示未匹配。
10         vector<int> mr; // matchRight[v] = 匹配到的左部点, 0 表示未匹配。
11     };
12
13     struct Cover {
14         vector<char> l;
15         vector<char> r;
16     };
17
18     int nl, nr;
19     vector<vector<int>> g;
20
21     HK(int nl, int nr) : nl(nl), nr(nr), g(nl + 1) {}
22
23     void add(int l, int r) {
24         assert(1 <= l && l <= nl && 1 <= r && r <= nr);
25         g[l].push_back(r);
26     }
27
28     Result solve() const {
29         const int INF = numeric_limits<int>::max();
30         vector<int> ml(nl + 1), mr(nr + 1), dis(nl + 1);
31
32         auto bfs = [&]() -> bool {
33             queue<int> q;
34             for (int u = 1; u <= nl; ++u) {
35                 if (ml[u] == 0) {
36                     dis[u] = 0;
37                     q.push(u);
38                 } else {
39                     dis[u] = INF;
40                 }
41             }
42
43             int lim = INF;
44             while (!q.empty()) {
45                 int u = q.front();
46                 q.pop();
47                 if (dis[u] >= lim) continue;
48                 for (int v : g[u]) {
49                     int x = mr[v];
50                     if (x == 0) {
51                         lim = dis[u] + 1;
52                     } else if (dis[x] == INF) {
53                         dis[x] = dis[u] + 1;
54                         q.push(x);
55                     }
56                 }
57             }
58             return lim != INF;
59         };
60
61         auto dfs = [&](auto&& go, int u) -> bool {
62             for (int v : g[u]) {
63                 int x = mr[v];
64                 if (x == 0 || (dis[x] == dis[u] + 1 && go(go, x))) {
65                     ml[u] = v;
66                     mr[v] = u;
67                     return true;
68                 }
69             }
70             dis[u] = INF;
71             return false;

```

```

72     };
73
74     int cnt = 0;
75     while (bfs()) {
76         for (int u = 1; u <= nl; ++u) {
77             if (ml[u] == 0 && dfs(dfs, u)) ++cnt;
78         }
79     }
80     return {cnt, move(ml), move(mr)};
81 }
82
83 // 传入 solve 的结果; 返回 (左部未访问点) + (右部访问点)。
84 Cover askCov(const Result& mat) const {
85     vector<char> vl(nl + 1, false), vr(nr + 1, false);
86     queue<int> q;
87     for (int u = 1; u <= nl; ++u) {
88         if (mat.ml[u] == 0) {
89             vl[u] = true;
90             q.push(u);
91         }
92     }
93
94     while (!q.empty()) {
95         int u = q.front();
96         q.pop();
97         for (int v : g[u]) {
98             if (mat.ml[u] == v || vr[v]) continue;
99             vr[v] = true;
100             int x = mat.mr[v];
101             if (x != 0 && !vl[x]) {
102                 vl[x] = true;
103                 q.push(x);
104             }
105         }
106     }
107
108     Cover res{vector<char>(nl + 1), vector<char>(nr + 1)};
109     for (int u = 1; u <= nl; ++u) res.l[u] = !vl[u];
110     for (int v = 1; v <= nr; ++v) res.r[v] = vr[v];
111     return res;
112 }
113 };

```

06B - 一般图最大匹配 (带花树) .cpp

```

1  /*
2  用途: Edmonds 带花树求一般无向图最大基数匹配。
3  约定: 点编号 1..n; 支持重边, 默认不放自环。mat[u] 为匹配点, 0 表示未匹配。
4  复杂度: O(n^3), 适合中等规模图。
5  */
6  struct Blossom {
7      struct Result {
8          int cnt;
9          vector<int> mat;
10     };
11
12     int n;
13     vector<vector<int>> g;
14
15     explicit Blossom(int n) : n(n), g(n + 1) {}
16
17     void add(int u, int v) {
18         assert(u != v);
19         g[u].push_back(v);
20         g[v].push_back(u);
21     }
22
23     Result solve() const {
24         vector<int> mat(n + 1);
25         vector<int> fa(n + 1), bas(n + 1);
26         vector<char> use(n + 1), in(n + 1);
27
28         auto lca = [&](int a, int b) -> int {
29             vector<char> vis(n + 1, false);
30             while (true) {
31                 a = bas[a];
32                 vis[a] = true;
33                 if (mat[a] == 0) break;
34                 a = fa[mat[a]];
35             }

```

```

36         while (true) {
37             b = bas[b];
38             if (vis[b]) return b;
39             b = fa[mat[b]];
40         }
41     };
42
43     auto tag = [&](int u, int rt, int son) -> void {
44         while (bas[u] != rt) {
45             in[bas[u]] = true;
46             in[bas[mat[u]]] = true;
47             fa[u] = son;
48             son = mat[u];
49             u = fa[mat[u]];
50         }
51     };
52
53     auto bfs = [&](int s) -> int {
54         fill(use.begin(), use.end(), false);
55         fill(fa.begin(), fa.end(), 0);
56         iota(bas.begin(), bas.end(), 0);
57         queue<int> q;
58         q.push(s);
59         use[s] = true;
60
61         while (!q.empty()) {
62             int u = q.front();
63             q.pop();
64             for (int v : g[u]) {
65                 if (bas[u] == bas[v] || mat[u] == v) continue;
66
67                 if (v == s || (mat[v] != 0 && fa[mat[v]] != 0)) {
68                     int rt = lca(u, v);
69                     fill(in.begin(), in.end(), false);
70                     tag(u, rt, v);
71                     tag(v, rt, u);
72                     for (int x = 1; x <= n; ++x) {
73                         if (!in[bas[x]]) continue;
74                         bas[x] = rt;
75                         if (!use[x]) {
76                             use[x] = true;
77                             q.push(x);
78                         }
79                     }
80                 } else if (fa[v] == 0) {
81                     fa[v] = u;
82                     if (mat[v] == 0) return v;
83                     v = mat[v];
84                     use[v] = true;
85                     q.push(v);
86                 }
87             }
88         }
89         return 0;
90     };
91
92     for (int s = 1; s <= n; ++s) {
93         if (mat[s] != 0) continue;
94         int t = bfs(s);
95         while (t != 0) {
96             int pre = fa[t];
97             int nxt = mat[pre];
98             mat[t] = pre;
99             mat[pre] = t;
100             t = nxt;
101         }
102     }
103
104     int cnt = 0;
105     for (int u = 1; u <= n; ++u) {
106         if (mat[u] > u) ++cnt;
107     }
108     return {cnt, move(mat)};
109 }
110 };

```

06C - 二分图最优匹配 (Hungarian、KM) .cpp

```

1  /*
2  用途：完全二分图的最小/最大权完备匹配 (Hungarian / KM)。

```

```

3 约定：左部大小 <= 右部大小；每个左部点都必须匹配一个不同右部点，需为每一对点设置权值。
4 askMin/askMax 分别返回最小/最大权值；权值及答案应远离 int 边界。
5 复杂度：O(leftSize^2 * rightSize)。
6 */
7
8 struct KM {
9     static constexpr int INF = numeric_limits<int>::max() / 4;
10
11     struct Result {
12         int val;
13         vector<int> ml;    // matchLeft[u] = 匹配到的右部点。
14         vector<int> mr;    // matchRight[v] = 匹配到的左部点，0 表示未使用。
15     };
16
17     int nl, nr;
18     vector<vector<int>> cst;
19
20     KM(int nl, int nr)
21         : nl(nl), nr(nr), cst(nl + 1, vector<int>(nr + 1)) {
22         assert(nl <= nr);
23     }
24
25     void modify(int l, int r, int v) {
26         assert(1 <= l && l <= nl && 1 <= r && r <= nr);
27         cst[l][r] = v;
28     }
29
30     Result askMin() const {
31         vector<int> hl(nl + 1), hr(nr + 1);
32         vector<int> mr(nr + 1), pre(nr + 1);
33
34         for (int x = 1; x <= nl; ++x) {
35             mr[0] = x;
36             int r0 = 0;
37             vector<int> mn(nr + 1, INF);
38             vector<char> use(nr + 1, false);
39
40             do {
41                 use[r0] = true;
42                 int l = mr[r0];
43                 int d = INF;
44                 int r = 0;
45                 for (int y = 1; y <= nr; ++y) {
46                     if (use[y]) continue;
47                     int rc = cst[l][y] - hl[l] - hr[y];
48                     if (rc < mn[y]) {
49                         mn[y] = rc;
50                         pre[y] = r0;
51                     }
52                     if (mn[y] < d) {
53                         d = mn[y];
54                         r = y;
55                     }
56                 }
57                 for (int y = 0; y <= nr; ++y) {
58                     if (use[y]) {
59                         hl[mr[y]] += d;
60                         hr[y] -= d;
61                     } else {
62                         mn[y] -= d;
63                     }
64                 }
65                 r0 = r;
66             } while (mr[r0] != 0);
67
68             do {
69                 int prv = pre[r0];
70                 mr[r0] = mr[prv];
71                 r0 = prv;
72             } while (r0 != 0);
73         }
74
75         vector<int> ml(nl + 1);
76         for (int y = 1; y <= nr; ++y) {
77             if (mr[y] != 0) ml[mr[y]] = y;
78         }
79         return {-hr[0], move(ml), move(mr)};
80     }
81 }

```

```

82     Result askMax() const {
83         KM neg(nl, nr);
84         for (int x = 1; x <= nl; ++x) {
85             for (int y = 1; y <= nr; ++y) {
86                 neg.cst[x][y] = -cst[x][y];
87             }
88         }
89         Result res = neg.askMin();
90         res.val = -res.val;
91         return res;
92     }
93 };

```

06D - 一般图最大权匹配（带权带花树）.cpp

```

1  /*
2  用途：一般无向图最大权匹配（允许点不匹配），Edmonds 带权带花树。
3  约定：原点编号 1..n, n<N, 只考虑正权边；重边取最大权。复杂度  $O(n^3)$ 。
4  若要求完美匹配且有负权，可先给每条边统一加足够大的常数，再检查匹配边数为  $n/2$ 。
5  点、花与矩阵下标使用 int。
6  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
7  */
8
9  namespace Blossom {
10     using W = int;
11     constexpr int N = 405;
12     constexpr int V = N * 2 + 5;
13     constexpr W INF = numeric_limits<W>::max() / 4;
14
15     struct Edge {
16         int u = 0, v = 0;
17         W w = 0;
18     };
19     struct Result {
20         W w = 0;
21         vector<array<int, 2>> es;
22     };
23
24     inline Edge e[V][V];
25     inline W lab[V];
26     inline int n, m, tim, ql, qr;
27     inline int lk[V], sl[V], st[V], fa[V], bel[V][V], typ[V], vis[V], que[V];
28     inline vector<int> cyc[V];
29
30     inline W slack(const Edge& x) { return lab[x.u] + lab[x.v] - 2 * x.w; }
31
32     void upd(int u, int v) {
33         if (!sl[v] || slack(e[u][v]) < slack(e[sl[v]][v])) sl[v] = u;
34     }
35
36     void slack(int v) {
37         sl[v] = 0;
38         for (int u = 1; u <= n; ++u)
39             if (e[u][v].w > 0 && st[u] != v && typ[st[u]] == 0) upd(u, v);
40     }
41
42     void push(int u) {
43         if (u <= n) que[++qr] = u;
44         else for (int v : cyc[u]) push(v);
45     }
46
47     void setSt(int u, int v) {
48         st[u] = v;
49         if (u > n) for (int x : cyc[u]) setSt(x, v);
50     }
51
52     int rot(int u, int v) {
53         int k = find(cyc[u].begin(), cyc[u].end(), v) - cyc[u].begin();
54         if (k & 1) {
55             reverse(cyc[u].begin() + 1, cyc[u].end());
56             return cyc[u].size() - k;
57         }
58         return k;
59     }
60
61     void match(int u, int v) {
62         lk[u] = e[u][v].v;
63         if (u <= n) return;
64         Edge w = e[u][v];
65         int x = bel[u][w.u], y = rot(u, x);

```



```

66     for (int i = 0; i < y; ++i) match(cyc[u][i], cyc[u][i ^ 1]);
67     match(x, v);
68     rotate(cyc[u].begin(), cyc[u].begin() + y, cyc[u].end());
69 }
70
71 void augment(int u, int v) {
72     int w = st[lk[u]];
73     match(u, v);
74     if (!w) return;
75     match(w, st[fa[w]]);
76     augment(st[fa[w]], w);
77 }
78
79 int lca(int u, int v) {
80     for (++tim; u || v; swap(u, v)) {
81         if (!u) continue;
82         if (vis[u] == tim) return u;
83         vis[u] = tim;
84         if ((u = st[lk[u]]) u = st[fa[u]];
85     }
86     return 0;
87 }
88
89 void add(int u, int a, int v) {
90     int x = n + 1;
91     while (x <= m && st[x]) ++x;
92     if (x > m) ++m;
93     assert(m < V);
94     lab[x] = 0, typ[x] = 0, st[x] = x, lk[x] = lk[a];
95     cyc[x].clear();
96     cyc[x].push_back(a);
97     int i, j = 0;
98     for (i = u; i != a; i = st[fa[j]]) {
99         cyc[x].push_back(i);
100         cyc[x].push_back(j = st[lk[i]]);
101         push(j);
102     }
103     reverse(cyc[x].begin() + 1, cyc[x].end());
104     for (i = v; i != a; i = st[fa[j]]) {
105         cyc[x].push_back(i);
106         cyc[x].push_back(j = st[lk[i]]);
107         push(j);
108     }
109     setSt(x, x);
110     for (i = 1; i <= m; ++i) e[x][i] = e[i][x] = {};
111     fill(bel[x], bel[x] + n + 1, 0);
112     for (int z : cyc[x]) {
113         for (int y = 1; y <= m; ++y) {
114             if (!e[x][y].w || slack(e[z][y]) < slack(e[x][y])) e[x][y] = e[z][y], e[y][x] = e[y][z];
115         }
116         for (int y = 1; y <= n; ++y) if (bel[z][y]) bel[x][y] = z;
117     }
118     slack(x);
119 }
120
121 void expand(int u) {
122     for (int x : cyc[u]) setSt(x, x);
123     int a = bel[u][e[u][fa[u]].u], r = rot(u, a);
124     for (int i = 0; i < r; i += 2) {
125         int x = cyc[u][i], y = cyc[u][i + 1];
126         fa[x] = e[y][x].u;
127         typ[x] = 1;
128         typ[y] = sl[x] = 0;
129         slack(y), push(y);
130     }
131     typ[a] = 1, fa[a] = fa[u];
132     for (int i = r + 1; i < cyc[u].size(); ++i) typ[cyc[u][i]] = -1, slack(cyc[u][i]);
133     st[u] = 0;
134 }
135
136 bool onEdge(const Edge& x) {
137     int u = st[x.u], v = st[x.v];
138     if (typ[v] == -1) {
139         fa[v] = x.u, typ[v] = 1;
140         int a = st[lk[v]];
141         sl[v] = sl[a] = typ[a] = 0;
142         push(a);
143     } else if (typ[v] == 0) {
144         int a = lca(u, v);
145         if (!a) {
146             augment(u, v), augment(v, u);
147             return true;

```

```

148     }
149     add(u, a, v);
150 }
151 return false;
152 }
153
154 bool bfs() {
155     fill(typ + 1, typ + m + 1, -1);
156     fill(sl + 1, sl + m + 1, 0);
157     ql = 1, qr = 0;
158     for (int i = 1; i <= m; ++i) if (st[i] == i && !lk[i]) {
159         fa[i] = typ[i] = 0;
160         push(i);
161     }
162     if (ql > qr) return false;
163     while (true) {
164         while (ql <= qr) {
165             int u = que[ql++];
166             if (typ[st[u]] == 1) continue;
167             for (int v = 1; v <= n; ++v) if (e[u][v].w > 0 && st[u] != st[v]) {
168                 if (slack(e[u][v])) upd(u, st[v]);
169                 else if (onEdge(e[u][v])) return true;
170             }
171         }
172
173         W d = INF;
174         for (int i = n + 1; i <= m; ++i) if (st[i] == i && typ[i] == 1) d = min(d, lab[i] / 2);
175         for (int i = 1; i <= m; ++i) if (st[i] == i && sl[i] && typ[i] != 1)
176             d = min(d, slack(e[sl[i]][i]) / (typ[i] == 0 ? 2 : 1));
177         if (d == INF) return false;
178
179         for (int i = 1; i <= n; ++i) if (typ[st[i]] != -1) {
180             lab[i] += (2 * typ[st[i]] - 1) * d;
181             if (lab[i] <= 0) return false;
182         }
183         for (int i = n + 1; i <= m; ++i) if (st[i] == i && typ[i] != -1)
184             lab[i] += (2 - 4 * typ[i]) * d;
185
186         ql = 1, qr = 0;
187         for (int i = 1; i <= m; ++i)
188             if (st[i] == i && sl[i] && st[sl[i]] != i && !slack(e[sl[i]][i]) && onEdge(e[sl[i]][i])) return true;
189         for (int i = n + 1; i <= m; ++i) if (st[i] == i && typ[i] == 1 && !lab[i]) expand(i);
190     }
191 }
192
193 Result solve(int _n, const vector<tuple<int, int, W>>& es) {
194     assert(0 <= _n && _n < N);
195     n = m = _n, tim = 0;
196     int lim = min<int>(V, 2 * n + 5);
197     fill(lk, lk + lim, 0), fill(st, st + lim, 0), fill(vis, vis + lim, 0);
198     for (int i = 0; i < lim; ++i) {
199         fill(e[i], e[i] + lim, Edge{});
200         fill(bel[i], bel[i] + lim, 0);
201         cyc[i].clear();
202     }
203     for (int i = 1; i <= n; ++i) st[i] = i;
204     W mx = 0;
205     for (int i = 1; i <= n; ++i) for (int j = 1; j <= n; ++j) e[i][j] = {i, j, 0};
206     for (auto [u, v, w] : es) {
207         assert(1 <= u && u <= n && 1 <= v && v <= n);
208         if (u == v || w <= 0) continue;
209         if (w > e[u][v].w) e[u][v] = {u, v, w}, e[v][u] = {v, u, w};
210         mx = max(mx, w);
211     }
212     for (int i = 1; i <= n; ++i) {
213         lab[i] = mx;
214         for (int j = 1; j <= n; ++j) if (i == j) bel[i][j] = i;
215     }
216     while (bfs()) {}
217     Result r;
218     for (int i = 1; i <= n; ++i) if (lk[i] && i < lk[i]) {
219         r.w += e[i][lk[i]].w;
220         r.es.push_back({i, lk[i]});
221     }
222     return r;
223 }
224 } // namespace Blossom

```

06E - 二分图最大匹配 (Kuhn, 小图) .cpp

```

1  /*
2  用途: DFS 增广路版二分图最大匹配, 代码短, 适合小图或左部较小的场景。
3  约定: 左部为 1..nl, 右部为 1..nr; add(l,r)。
4  接口: solve() 返回最大匹配; askCov(res) 返回一组最小点覆盖。
5  复杂度:  $O(nl * m)$ 。大图优先 06A 的 Hopcroft-Karp。
6  */
7
8  struct Kuhn {
9      struct Result {
10          int cnt;
11          vector<int> ml, mr;
12      };
13
14      struct Cover {
15          vector<char> l, r;
16      };
17
18      int nl, nr;
19      vector<vector<int>> g;
20
21      Kuhn(int nl, int nr) : nl(nl), nr(nr), g(nl + 1) {}
22
23      void add(int l, int r) {
24          assert(1 <= l && l <= nl && 1 <= r && r <= nr);
25          g[l].push_back(r);
26      }
27
28      Result solve() const {
29          vector<int> ml(nl + 1), mr(nr + 1), vis(nl + 1);
30          int tim = 0, cnt = 0;
31          auto dfs = [&](auto go, int u) -> bool {
32              if (vis[u] == tim) return false;
33              vis[u] = tim;
34              for (int v : g[u]) {
35                  int x = mr[v];
36                  if (x && !go(go, x)) continue;
37                  ml[u] = v;
38                  mr[v] = u;
39                  return true;
40              }
41              return false;
42          };
43          for (int u = 1; u <= nl; ++u) {
44              ++tim;
45              if (dfs(dfs, u)) ++cnt;
46          }
47          return {cnt, move(ml), move(mr)};
48      }
49
50      Cover askCov(const Result& mat) const {
51          vector<char> vl(nl + 1), vr(nr + 1);
52          queue<int> q;
53          for (int u = 1; u <= nl; ++u) {
54              if (mat.ml[u]) continue;
55              vl[u] = true;
56              q.push(u);
57          }
58          while (!q.empty()) {
59              int u = q.front();
60              q.pop();
61              for (int v : g[u]) {
62                  if (mat.ml[u] == v || vr[v]) continue;
63                  vr[v] = true;
64                  int x = mat.mr[v];
65                  if (x && !vl[x]) {
66                      vl[x] = true;
67                      q.push(x);
68                  }
69              }
70          }
71          Cover res{vector<char>(nl + 1), vector<char>(nr + 1)};
72          for (int u = 1; u <= nl; ++u) res.l[u] = !vl[u];
73          for (int v = 1; v <= nr; ++v) res.r[v] = vr[v];
74          return res;
75      }
76  };

```

07 - 全局最小割 (Stoer-Wagner) .cpp

```

1  /*
2  用途：无向非负权图的全局最小割，不指定源汇；同时返回割的一侧原点集合。
3  约定：点编号 0..n-1；重边容量累加。n<2 时割值为 0。
4  复杂度：O(n^3) 时间、O(n^2) 空间，适合稠密中小图。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7
8  template <class Cap = int> struct MinCut {
9      struct Result {
10          Cap val;
11          vector<int> sd;
12      };
13
14      int n;
15      vector<vector<Cap>> w;
16      explicit MinCut(int n) : n(n), w(n, vector<Cap>(n)) {
17          assert(n >= 0);
18      }
19
20      void add(int u, int v, Cap c) {
21          assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
22          if (u == v) return;
23          w[u][v] += c, w[v][u] += c;
24      }
25
26      Result solve() const {
27          if (n < 2) return {Cap{}, n ? vector<int>{0} : vector<int>{}};
28          vector<vector<Cap>> a = w;
29          vector<int> v(n);
30          iota(v.begin(), v.end(), 0);
31          vector<vector<int>> g(n);
32          for (int i = 0; i < n; ++i) g[i] = {i};
33          Cap ans = numeric_limits<Cap>::max();
34          vector<int> sd;
35
36          while (v.size() > 1) {
37              vector<Cap> d(n);
38              vector<char> use(n);
39              int pre = -1, sel = -1;
40              for (int k = 0; k < v.size(); ++k) {
41                  sel = -1;
42                  for (int x : v)
43                      if (!use[x] && (sel == -1 || d[x] > d[sel])) sel = x;
44                  if (k + 1 == v.size()) {
45                      if (d[sel] < ans) ans = d[sel], sd = g[sel];
46                      for (int x : v)
47                          if (x != pre) a[pre][x] += a[sel][x], a[x][pre] = a[pre][x];
48                      g[pre].insert(g[pre].end(), g[sel].begin(), g[sel].end());
49                      v.erase(find(v.begin(), v.end(), sel));
50                      break;
51                  }
52                  use[sel] = true;
53                  for (int x : v)
54                      if (!use[x]) d[x] += a[sel][x];
55                  pre = sel;
56              }
57          }
58          return {ans, move(sd)};
59      }
60 };

```

08 - 最大团 (位集分支限界) .cpp

```

1  /*
2  用途：求 n<=64 无向简单图的最大团及一组方案。
3  复杂度：最坏指数级；贪心染色给上界，适合中小规模稠密图。
4  性能：邻接与候选集使用 u64，避免 #define int long long 扩大状态。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7
8  using u64 = uint64_t;
9
10 struct Clique {
11     int n;
12     vector<u64> g;
13     vector<int> now, ans;
14 };

```

```

15     explicit Clique(int _n) : n(_n), g(_n) { assert(0 <= n && n <= 64); }
16     void add(int u, int v) {
17         assert(0 <= u && u < n && 0 <= v && v < n && u != v);
18         g[u] |= u64{1} << v;
19         g[v] |= u64{1} << u;
20     }
21
22     vector<int> solve() {
23         now.clear(), ans.clear();
24         dfs(n == 64 ? ~u64{} : (u64{1} << n) - 1);
25         return ans;
26     }
27
28 private:
29     void color(u64 p, vector<int>& v, vector<int>& c) {
30         for (int k = 1; p; ++k) {
31             u64 q = p;
32             while (q) {
33                 int x = __builtin_ctzll(q);
34                 u64 b = u64{1} << x;
35                 v.push_back(x), c.push_back(k);
36                 p ^= b, q ^= b, q &= ~g[x];
37             }
38         }
39     }
40
41     void dfs(u64 p) {
42         if (!p) {
43             if (now.size() > ans.size()) ans = now;
44             return;
45         }
46         vector<int> v, c;
47         color(p, v, c);
48         for (int i = v.size(); i--;) {
49             if (now.size() + c[i] <= ans.size()) return;
50             int x = v[i];
51             now.push_back(x), dfs(p & g[x]), now.pop_back();
52             p &= ~(u64{1} << x);
53         }
54     }
55 };

```

09 - 平面嵌入面与对偶图.cpp

```

1  /*
2  用途：已知连通直线平面嵌入（无交叉）时，枚举面并建立带权对偶图。
3  约定：每条无向边给一个权；对偶边连接其左右两面。桥的左右面相同，不加入对偶自环。
4  复杂度： $O(m \log m)$ ，主要为每点按极角排序。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7
8  using i128 = __int128_t;
9
10 template <class T = int, class W = int, class D = i128> struct PlanarDual {
11     struct P { T x, y; };
12     struct E { int u, v; W w; };
13
14     vector<P> p;
15     vector<E> e;
16     vector<array<int, 2>> he;
17     vector<int> rev, nxt, bel;
18     vector<vector<int>> g;
19     vector<D> ar;
20     int out = -1;
21
22     PlanarDual() = default;
23     explicit PlanarDual(vector<P> p) : p(move(p)), g(this->p.size()) {}
24
25     void add(int u, int v, W w = 1) {
26         int id = e.size(), a = he.size();
27         e.push_back({u, v, w});
28         he.push_back({u, v});
29         he.push_back({v, u});
30         rev.push_back(a + 1);
31         rev.push_back(a);
32         g[u].push_back(a);
33         g[v].push_back(a + 1);
34     }
35
36     D cr(int a, int b, int c) const {

```

```

37     return D(p[b].x - p[a].x) * (p[c].y - p[a].y) - D(p[b].y - p[a].y) * (p[c].x - p[a].x);
38 }
39
40 void build() {
41     int m = he.size();
42     vector<int> pos(m);
43     auto hf = [&](T x, T y) { return y < 0 || (y == 0 && x < 0); };
44     for (int u = 0; u < g.size(); ++u) {
45         sort(g[u].begin(), g[u].end(), [&](int a, int b) {
46             auto [ua, x] = he[a];
47             auto [ub, y] = he[b];
48             T ax = p[x].x - p[u].x, ay = p[x].y - p[u].y;
49             T bx = p[y].x - p[u].x, by = p[y].y - p[u].y;
50             if (hf(ax, ay) != hf(bx, by)) return hf(ax, ay) < hf(bx, by);
51             D z = D(ax) * by - D(ay) * bx;
52             if (z) return z > 0;
53             return D(ax) * ax + D(ay) * ay < D(bx) * bx + D(by) * by;
54         });
55         for (int i = 0; i < g[u].size(); ++i) pos[g[u][i]] = i;
56     }
57
58     nxt.assign(m, -1);
59     for (int h = 0; h < m; ++h) {
60         int v = he[h][1], k = pos[rev[h]], d = g[v].size();
61         nxt[h] = g[v][(k + d - 1) % d];
62     }
63
64     bel.assign(m, -1);
65     ar.clear();
66     for (int s = 0; s < m; ++s) if (bel[s] < 0) {
67         int f = ar.size(), h = s;
68         D a = 0;
69         do {
70             bel[h] = f;
71             auto [u, v] = he[h];
72             a += D(p[u].x) * p[v].y - D(p[u].y) * p[v].x;
73             h = nxt[h];
74         } while (h != s);
75         ar.push_back(a);
76     }
77     out = min_element(ar.begin(), ar.end()) - ar.begin();
78 }
79
80 vector<vector<pair<int, W>>> ask() const {
81     vector<vector<pair<int, W>>> d(ar.size());
82     for (int i = 0; i < e.size(); ++i) {
83         int a = bel[i < 1], b = bel[i < 1 | 1];
84         if (a == b) continue;
85         d[a].push_back({b, e[i].w});
86         d[b].push_back({a, e[i].w});
87     }
88     return d;
89 }
90 };

```

10 - 最小环 (Floyd) .cpp

```

1  /*
2  用途: 稠密非负权图的最小简单环。
3  MinCycle: 无向图, 支持自环和重边; DirCycle: 有向图, 允许自环和重边。
4  接口: add/add2 后调用 solve; 返回 INF 表示无环。
5  复杂度:  $O(n^3)$  时间、 $O(n^2)$  空间。路径长度必须小于 INF, 适合  $n$  约 500 以内。
6  */
7  struct MinCycle {
8      static constexpr int INF = numeric_limits<int>::max() / 4;
9
10     int n, ans = INF;
11     vector<vector<int>>> a;
12
13     explicit MinCycle(int n) : n(n), a(n + 1, vector<int>(n + 1, INF)) {
14         for (int i = 1; i <= n; ++i) a[i][i] = 0;
15     }
16
17     void add(int u, int v, int w) {
18         assert(0 <= w && w < INF);
19         if (u == v) {
20             ans = min(ans, w);
21             return;
22         }
23         if (a[u][v] != INF) ans = min(ans, a[u][v] + w);

```

```

24     a[u][v] = a[v][u] = min(a[u][v], w);
25 }
26
27 int solve() const {
28     auto d = a;
29     int res = ans;
30     for (int k = 1; k <= n; ++k) {
31         for (int i = 1; i < k; ++i) {
32             if (a[i][k] == INF) continue;
33             for (int j = 1; j < i; ++j) {
34                 if (d[i][j] == INF || a[k][j] == INF) continue;
35                 res = min(res, d[i][j] + a[i][k] + a[k][j]);
36             }
37         }
38         for (int i = 1; i <= n; ++i) {
39             if (d[i][k] == INF) continue;
40             for (int j = 1; j <= n; ++j) {
41                 if (d[k][j] == INF) continue;
42                 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
43             }
44         }
45     }
46     return res;
47 }
48 };
49
50 struct DirCycle {
51     static constexpr int INF = numeric_limits<int>::max() / 4;
52
53     int n;
54     vector<vector<int>>> a;
55
56     explicit DirCycle(int n) : n(n), a(n + 1, vector<int>(n + 1, INF)) {}
57
58     void add(int u, int v, int w) {
59         assert(0 <= w && w < INF);
60         a[u][v] = min(a[u][v], w);
61     }
62
63     int solve() const {
64         auto d = a;
65         for (int k = 1; k <= n; ++k) {
66             for (int i = 1; i <= n; ++i) {
67                 if (d[i][k] == INF) continue;
68                 for (int j = 1; j <= n; ++j) {
69                     if (d[k][j] == INF) continue;
70                     d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
71                 }
72             }
73         }
74         int ans = INF;
75         for (int i = 1; i <= n; ++i) ans = min(ans, d[i][i]);
76         return ans;
77     }
78 };

```

10A - 最小环 (长度与计数) .cpp

```

1  /*
2  用途: 有向正权图的最小简单环长度与计数。
3  约定: 点编号 1..n; add 可加入重边, 同权重边视为不同边; 自环视为一个环。
4  接口: ask 返回 {len,cnt}; 无环时 len=INF,cnt=0。
5  前提: 边权严格为正; Z 用于计数, 通常传 MInt。
6  复杂度: O(n^3) 时间、O(n^2) 空间。
7  */
8  template <class Z>
9  struct MinCycCnt {
10     static constexpr int INF = numeric_limits<int>::max() / 4;
11
12     struct Res {
13         int len;
14         Z cnt;
15     };
16
17     int n, sl = INF;
18     Z sc{};
19     vector<vector<int>>> a;
20     vector<vector<Z>>> b;
21
22     explicit MinCycCnt(int n) : n(n), a(n + 1, vector<int>(n + 1, INF)), b(n + 1, vector<Z>(n + 1)) {}

```

```

23
24 void add(int u, int v, int w) {
25     assert(1 <= u && u <= n && 1 <= v && v <= n && 0 < w && w < INF);
26     if (u == v) {
27         if (w < sl) sl = w, sc = Z(1);
28         else if (w == sl) sc += Z(1);
29         return;
30     }
31     if (w < a[u][v]) a[u][v] = w, b[u][v] = Z(1);
32     else if (w == a[u][v]) b[u][v] += Z(1);
33 }
34
35 Res ask() const {
36     auto d = a;
37     auto c = b;
38     for (int i = 1; i <= n; ++i) d[i][i] = 0, c[i][i] = Z(1);
39     int ans = sl;
40     Z cnt = sc;
41     auto upd = [&](int len, Z now) {
42         if (len < ans) ans = len, cnt = now;
43         else if (len == ans) cnt += now;
44     };
45
46     for (int k = 1; k <= n; ++k) {
47         for (int i = 1; i < k; ++i) {
48             if (a[k][i] != INF && d[i][k] != INF) upd(a[k][i] + d[i][k], b[k][i] * c[i][k]);
49         }
50         for (int i = 1; i <= n; ++i) {
51             if (i == k || d[i][k] == INF) continue;
52             for (int j = 1; j <= n; ++j) {
53                 if (j == k || d[k][j] == INF) continue;
54                 int nd = d[i][k] + d[k][j];
55                 if (nd < d[i][j]) d[i][j] = nd, c[i][j] = c[i][k] * c[k][j];
56                 else if (nd == d[i][j]) c[i][j] += c[i][k] * c[k][j];
57             }
58         }
59     }
60     return {ans, cnt};
61 }
62 };

```

11 - 简单环计数 (状态压缩) .cpp

```

1  /*
2  用途: 计数无向或有向简单图中长度至少 3 的本质不同简单环。
3  约定: 点编号 0..n-1; CycleCnt<false> 为无向图, CycleCnt<true> 为有向图; 不支持重边和自环。
4  接口: add(u,v), ask() 返回环数。无向环不区分方向, 有向环区分方向。
5  复杂度: O(n*2^n) 时间、O(n*2^n) 空间; 默认 int 使 n=20 约需 160 MB。
6  */
7  template <bool Dir = false>
8  struct CycleCnt {
9      using u64 = unsigned long long;
10
11      int n;
12      vector<u64> g;
13
14      explicit CycleCnt(int n) : n(n), g(n) {
15          assert(1 <= n && n <= 20);
16      }
17
18      void add(int u, int v) {
19          assert(0 <= u && u < n && 0 <= v && v < n && u != v);
20          g[u] |= u64(1) << v;
21          if constexpr (!Dir) g[v] |= u64(1) << u;
22      }
23
24      int ask() const {
25          int m = 1 << n;
26          u64 all = (u64(1) << n) - 1;
27          vector<int> f(m * n);
28          for (int s = 0; s < n; ++s) f[(1 << s) * n + s] = 1;
29          int ans = 0;
30          for (int s = 1; s < m; ++s) {
31              int p = __builtin_ctz(static_cast<unsigned>(s));
32              int pc = __builtin_popcount(static_cast<unsigned>(s));
33              u64 ok = all ^ ((u64(1) << (p + 1)) - 1);
34              for (int v = 0; v < n; ++v) {
35                  int cur = f[s * n + v];
36                  if (!cur) continue;
37                  if (v != p && pc >= 3 && (g[v] >> p & 1)) ans += cur;

```



```

38         u64 can = g[v] & (all ^ u64(s)) & ok;
39         while (can) {
40             int u = __builtin_ctzll(can);
41             f[(s | (1 << u)) * n + u] += cur;
42             can &= can - 1;
43         }
44     }
45 }
46 if constexpr (!Dir) ans /= 2;
47 return ans;
48 }
49 };

```

12 - 环检测与恢复 (有向、无向) .cpp

```

1  /*
2  用途: 在线性时间内检测并恢复任意一个有向或无向环。
3  约定: 点编号 1..n: FindCycle<true> 为有向图, FindCycle<false> 为无向图, 均支持自环和重边。
4  接口: add(u,v) 后 ask() 返回环上的点, 按环顺序且不重复首点; 无环返回空数组。
5  复杂度: O(n+m) 时间、O(n+m) 空间。
6  */
7  template <bool Dir>
8  struct FindCycle {
9      int n, ed = 0;
10     vector<vector<pii>> g;
11     vector<int> col, pos, st;
12
13     explicit FindCycle(int n) : n(n), g(n + 1) {}
14
15     void add(int u, int v) {
16         assert(1 <= u && u <= n && 1 <= v && v <= n);
17         g[u].push_back({v, ed});
18         if constexpr (!Dir) g[v].push_back({u, ed});
19         ++ed;
20     }
21
22     vector<int> ask() {
23         col.assign(n + 1, 0);
24         pos.assign(n + 1, -1);
25         st.clear();
26         auto dfs = [&](auto&& go, int u, int pe) -> bool {
27             col[u] = 1;
28             pos[u] = st.size();
29             st.push_back(u);
30             for (auto [v, id] : g[u]) {
31                 if constexpr (!Dir) {
32                     if (id == pe) continue;
33                 }
34                 if (col[v] == 0) {
35                     if (go(go, v, id)) return true;
36                 } else if (col[v] == 1) {
37                     vector<int> res(st.begin() + pos[v], st.end());
38                     st = move(res);
39                     return true;
40                 }
41             }
42             col[u] = 2;
43             st.pop_back();
44             return false;
45         };
46         for (int i = 1; i <= n; ++i) {
47             if (col[i] == 0 && dfs(dfs, i, -1)) return st;
48         }
49         return {};
50     }
51 };

```

13 - 支配树 (Lengauer-Tarjan) .cpp

```

1  /*
2  用途: 有向图从根 rt 出发的支配树; idom[v] 是 v 的直接支配点。
3  约定: 点编号 1..n; 不可达点和根的 idom 均为 0; 允许重边、自环。
4  复杂度: 实现使用 Lengauer-Tarjan, 常用分析为 O((n+m) log n), 空间 O(n+m)。
5  注意: dfs 与 eval 为递归; 链状图很深时需提高栈或改成迭代 DFS。
6  */
7  struct DomTree {
8      int n, ti;
9      vector<vector<int>> g, rg, buc;

```

```

10     vector<int> dfn, rev, par, sem, dom, anc, lab;
11
12     explicit DomTree(int _n) : n(_n), g(_n + 1) {
13         assert(n >= 0);
14     }
15
16     void add(int u, int v) {
17         assert(1 <= u && u <= n && 1 <= v && v <= n);
18         g[u].pb(v);
19     }
20
21     void dfs(int u) {
22         dfn[u] = ++ti;
23         rev[ti] = u;
24         sem[ti] = lab[ti] = ti;
25         for (int v : g[u]) {
26             if (!dfn[v]) {
27                 dfs(v);
28                 par[dfn[v]] = dfn[u];
29             }
30             rg[dfn[v]].pb(dfn[u]);
31         }
32     }
33
34     int find(int x) {
35         if (anc[x] == x) return x;
36         int y = anc[x];
37         int z = find(y);
38         if (sem[lab[y]] < sem[lab[x]]) lab[x] = lab[y];
39         return anc[x] = z;
40     }
41
42     int eval(int x) {
43         find(x);
44         return lab[x];
45     }
46
47     vector<int> ask(int rt) {
48         assert(1 <= rt && rt <= n);
49         ti = 0;
50         dfn.assign(n + 1, 0);
51         rev.assign(n + 1, 0);
52         par.assign(n + 1, 0);
53         sem.assign(n + 1, 0);
54         dom.assign(n + 1, 0);
55         anc.assign(n + 1, 0);
56         lab.assign(n + 1, 0);
57         rg.assign(n + 1, {});
58         buc.assign(n + 1, {});
59         dfs(rt);
60         for (int i = 1; i <= ti; ++i) anc[i] = lab[i] = i;
61         for (int i = ti; i >= 2; --i) {
62             for (int v : rg[i]) sem[i] = min(sem[i], sem[eval(v)]);
63             buc[sem[i]].pb(i);
64             for (int v : buc[par[i]]) {
65                 int u = eval(v);
66                 dom[v] = sem[u] < sem[v] ? u : par[i];
67             }
68             buc[par[i]].clear();
69             anc[i] = par[i];
70         }
71         for (int i = 2; i <= ti; ++i)
72             if (dom[i] != sem[i]) dom[i] = dom[dom[i]];
73         vector<int> r(n + 1);
74         for (int i = 2; i <= ti; ++i) r[rev[i]] = rev[dom[i]];
75         return r;
76     }
77 };

```

14 - Steiner树 (终端点较少) .cpp

```

1  /*
2  用途：无向非负权图中连接给定终端点的最小 Steiner 树（允许经过非终端点）。
3  约定：点编号 1..n；终端点应互异；k<=20；不连通时返回 nullopt。
4  复杂度：O(3^k n + 2^k m log n)，空间 O(2^k n)。
5  注意：这是“终端点较少”的精确算法；k 大时应改用题目专用结构或近似算法。
6  */
7  template <class T = int> struct Steiner {
8      struct E {
9          int v;

```

```

10     T w;
11 };
12
13 int n;
14 vector<vector<E>> g;
15
16 explicit Steiner(int _n) : n(_n), g(_n + 1) {
17     assert(n >= 0);
18 }
19
20 void add(int u, int v, T w) {
21     assert(1 <= u && u <= n && 1 <= v && v <= n && w >= 0);
22     g[u].pb({v, w});
23     g[v].pb({u, w});
24 }
25
26 optional<T> ask(const vector<int> &t) const {
27     int k = t.size();
28     assert(k <= 20);
29     vector<char> vis(n + 1);
30     for (int i = 0; i < k; ++i) {
31         assert(1 <= t[i] && t[i] <= n && !vis[t[i]]);
32         vis[t[i]] = true;
33     }
34     if (!k) return T{};
35     int s = 1LL << k;
36     T inf = numeric_limits<T>::max() / 4;
37     vector<vector<T>> f(s, vector<T>(n + 1, inf));
38     for (int i = 0; i < k; ++i) f[1LL << i][t[i]] = 0;
39     using P = pair<T, int>;
40     for (int m = 1; m < s; ++m) {
41         for (int z = (m - 1) & m; z; z = (z - 1) & m) {
42             int y = m ^ z;
43             if (z >= y) continue;
44             for (int v = 1; v <= n; ++v) {
45                 if (f[z][v] == inf || f[y][v] == inf) continue;
46                 if (f[z][v] > inf - f[y][v]) continue;
47                 f[m][v] = min(f[m][v], f[z][v] + f[y][v]);
48             }
49         }
50         priority_queue<P, vector<P>, greater<P>> q;
51         for (int v = 1; v <= n; ++v)
52             if (f[m][v] != inf) q.push({f[m][v], v});
53         while (!q.empty()) {
54             auto [d, u] = q.top();
55             q.pop();
56             if (d != f[m][u]) continue;
57             for (auto e : g[u]) {
58                 if (d > inf - e.w) continue;
59                 T nd = d + e.w;
60                 if (nd < f[m][e.v]) {
61                     f[m][e.v] = nd;
62                     q.push({nd, e.v});
63                 }
64             }
65         }
66     }
67     T ans = inf;
68     for (int v = 1; v <= n; ++v) ans = min(ans, f[s - 1][v]);
69     if (ans == inf) return nullopt;
70     return ans;
71 }
72 };

```

网络流

8 个代码模板 · 林月 · v1.0.0 · 2026.8.1

版块说明与知识结论

README.md

网络流模板

默认约定为 $0 \dots n-1$ 编号的图，容量与费用通常直接使用宏展开后的 `int`。所有代码片段需放在已包含 `bits/stdc++.h` 且使用 `std` 命名空间的 C++17 环境中。

需求	模板	主要复杂度
最大流、残量图最小割恢复	01A - 最大流 (Dinic)	一般图 $O(n^2m)$
大图最大流、预流推进	01B - 最大流 (HLPP)	理论 $O(n^2 \sqrt{m})$
极限大图最大流、卡常预流推进	01C - 最大流 (HLPP, 极致优化)	同阶；窄边表、指针当前弧、gap、周期全局重标号
S-T 最小割值与源点侧	02 - 最小割	与 Dinic 一致
最大权闭合图	02A - 最大权闭合图 (最小割建模)	一次 Dinic
无向图任意两点最小割	03 - 最小割树 (Gomory-Hu)	$n-1$ 次最大流
含负费用边的最小费用最大流	04 - 最小费用最大流 (MinCostFlow)	增广次数 $\times O(m \log n)$
上下界可行循环流/最大流	05 - 上下界网络流	常数次 Dinic

使用提示

- Dinic::maxFlow 和 MinCostMaxFlow::minCostMaxFlow 都会保留残量图；想从原始容量重复求解时先调用 reset()。MinimumSTCut、最大权闭合图和最小割树已在内部重置。
- 01A / 01B / 01C 是同一最大流问题的不同实现：建模和中小图默认 Dinic；需要完整易改接口时用 01B；极限点边数再换 01C。01C 保留每个点的连续邻接块，并把边目标、反向下标、标号和当前弧压成 i32；这些是卡常核心，不能按普通风格改回 64 位 int。
- 最大流后，Dinic::minCutReachable(source) 返回最小割的源点侧；edge(id)/edges() 可取得原边容量与流量。
- 最大权闭合图中，addDependency(u, v) 表示“选 u 必须选 v”。依赖边容量必须严格大于任意可行解的总权值差。
- Gomory-Hu 仅适用于无向容量图；build() 后 minCutValue(u, v) 为 $O(\log n)$ 。
- 费用流允许负费用边，但该模板检测到源点可达负费用环时会返回 nullopt；一般题目应保证不存在这种环。若只需要固定流量，传入 limit 并检查返回的 flow 是否达到该值。
- 上下界流的 edgeFlow 与 addEdge 顺序相同，且已包含下界；若 feasible == false，不存在满足所有上下界的流。

覆盖边界

二分图匹配、一般图带花树与 KM 已维护在 04 - 图论；它们和网络流建模可互相转化，但不在此重复实现。

最大流最小割、上下界与费用流的建模速查见 [网络流建模与结论](#)。

网络流建模与结论.md

网络流建模与结论

1. 最大流与最小割

- 最大流值等于最小 S-T 割容量。整数容量下存在整数最大流，因此很多“选/不选、匹配、条数”模型无需额外证明取整。
- 最大流结束后，从源点沿正残量边可达的点是一个最小割源侧；跨向不可达侧的原边构成割边。

- 无向容量边通常建两条独立的有向容量边；“一条边共用总容量”与此不同，必须按题意建模。
- INF 只需严格大于任何可行有限割的总容量，过大可能使加法溢出。

2. 高频建模

模型	建边方式
二分图匹配	源到左部、原匹配边、右部到汇，容量均为 1。
点容量	拆成入点与出点，中间边容量为点限制。
最大权闭合图	正权点由源连，负权点连汇，依赖 $u \Rightarrow v$ 建 $u \rightarrow v(\text{INF})$ 。
最小点权覆盖/最大收益选择	先写“不选/选”的割代价，再决定点放源侧还是汇侧。
多源多汇	加超级源汇；容量上界用所有真实容量之和加一。

最大权闭合答案为正权总和减最小割。恢复选择集合时取最大流后源侧可达的原点。

3. 上下界流

边流量限制 $[l,r]$ 改成容量 $r-l$ ，并令 $\text{bal}[u]-=l$ ， $\text{bal}[v]+=l$ 。余额为正的点由超级源连入，为负的点连向超级汇；所有平衡边满流才存在可行流。

求有源汇可行流时再加 $t \rightarrow s(\text{INF})$ 。得到可行流后移除超级源汇及辅助边，再在残量图上从 s 到 t 增广；原边实际流量要把下界加回。

4. 费用流

- 每次沿最短费用增广路增流。存在负费用边时，第一次势能需由 Bellman-Ford/SPFA 初始化；之后约化费用非负才可用 Dijkstra。
- 求最大费用可把费用取反；固定流量必须检查最终流量是否达到目标。
- 若源点可达负费用环且还能循环增流，目标可能无界或模板前提不成立。
- 容量与费用乘积可能超过 64 位；极端范围用 i128 计算总费用。

5. 算法选择

Dinic 代码短且适合大多数建模题；HLPP 对层数深、边多的图常更稳。01B 保留易读的 64 位下标与完整接口；01C 使用连续邻接块、i32 边下标、指针当前弧、最高标号、gap 和周期全局重标号，只有确实卡常时再换。两版最大流后都应以“容量限制、流量守恒、最大流等于残量图最小割”三项不变量验算。

Gomory-Hu 用 $n-1$ 次最大流把无向图所有点对最小割压到一棵树上，只适用于无向容量图。

01A - 最大流 (Dinic) .cpp

```
1  /*
2  用途: Dinic 最大流、残量图最小割恢复; 作为短代码与通用默认分支。
3  约定: 容量类型用 Cap 模板适配; 点号和反向边下标使用 int。
4  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
5  */
6
7  template <class Cap = int> struct MaxFlow {
8      static constexpr Cap INF = numeric_limits<Cap>::max() / 4;
9
10     struct Edge {
11         int v, r;
12         Cap c, oc;
13     };
14
15     struct EdgeInfo {
16         int u, v;
17         Cap c, f;
18     };
19
20     int n;
21     vector<vector<Edge>> g;
22     vector<array<int, 2>> pos;
23     vector<int> dep, cur;
24
25     explicit MaxFlow(int n) : n(n), g(n) {
26         assert(n >= 0);
27     }
```

```

28
29 int add(int u, int v, Cap c) {
30     assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
31     int x = g[u].size();
32     int y = g[v].size() + (u == v);
33     g[u].push_back({v, y, c, c});
34     g[v].push_back({u, x, 0, 0});
35     pos.push_back({u, x});
36     return pos.size() - 1;
37 }
38
39 void add2(int u, int v, Cap c) {
40     add(u, v, c);
41     add(v, u, c);
42 }
43
44 void reset() {
45     for (auto &a : g)
46         for (Edge &e : a) e.c = e.oc;
47 }
48
49 EdgeInfo askEdge(int id) const {
50     assert(0 <= id && id < pos.size());
51     auto [u, i] = pos[id];
52     const Edge &e = g[u][i];
53     return {u, e.v, e.oc, e.oc - e.c};
54 }
55
56 Cap askFlow(int id) const {
57     return askEdge(id).f;
58 }
59
60 vector<EdgeInfo> askAll() const {
61     vector<EdgeInfo> a;
62     a.reserve(pos.size());
63     for (int i = 0; i < pos.size(); ++i) a.push_back(askEdge(i));
64     return a;
65 }
66
67 void erase(int id) {
68     assert(0 <= id && id < pos.size());
69     auto [u, i] = pos[id];
70     Edge &e = g[u][i];
71     Edge &f = g[e.v][e.r];
72     e.c = f.c = 0;
73 }
74
75 bool bfs(int s, int t) {
76     dep.assign(n, -1);
77     queue<int> q;
78     dep[s] = 0;
79     q.push(s);
80     while (!q.empty()) {
81         int u = q.front();
82         q.pop();
83         for (const Edge &e : g[u]) {
84             if (e.c <= 0 || dep[e.v] != -1) continue;
85             dep[e.v] = dep[u] + 1;
86             q.push(e.v);
87         }
88     }
89     return dep[t] != -1;
90 }
91
92 Cap dfs(int u, int t, Cap f) {
93     if (u == t || !f) return f;
94     for (int &i = cur[u]; i < g[u].size(); ++i) {
95         Edge &e = g[u][i];
96         if (e.c <= 0 || dep[e.v] != dep[u] + 1) continue;
97         Cap x = dfs(e.v, t, min(f, e.c));
98         if (!x) continue;
99         e.c -= x;
100         g[e.v][e.r].c += x;
101         return x;
102     }
103     return 0;
104 }
105
106 Cap flow(int s, int t, Cap lim = INF) {
107     assert(0 <= s && s < n && 0 <= t && t < n && s != t && lim >= 0);
108     Cap r = 0;
109     while (r < lim && bfs(s, t)) {

```

```

110         cur.assign(n, 0);
111         while (r < lim) {
112             Cap x = dfs(s, t, min(lim - r, INF));
113             if (!x) break;
114             r += x;
115         }
116     }
117     return r;
118 }
119
120 vector<char> askCut(int s) const {
121     assert(0 <= s && s < n);
122     vector<char> a(n);
123     queue<int> q;
124     q.push(s);
125     a[s] = 1;
126     while (!q.empty()) {
127         int u = q.front();
128         q.pop();
129         for (const Edge &e : g[u]) {
130             if (e.c > 0 && !a[e.v]) {
131                 a[e.v] = 1;
132                 q.push(e.v);
133             }
134         }
135     }
136     return a;
137 }
138 };

```

01B - 最大流 (HLPP) .cpp

```

1  /*
2  用途: 最高标号预流推进 (HLPP) 最大流, 含反向 BFS 初始标号、当前弧与 gap 优化。
3  适合点边较多、Dinic 分层增广偏慢的图; 小图和建模题通常优先更短的 01A Dinic。
4  约定: 点号/边下标使用 int, 容量由 Cap 适配; 点编号 0..n-1。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7
8  template <class Cap = int> struct MaxFlow {
9      static constexpr Cap INF = numeric_limits<Cap>::max() / 4;
10
11      struct Edge {
12          int v, r;
13          Cap c, oc;
14      };
15      struct EdgeInfo {
16          int u, v;
17          Cap c, f;
18      };
19
20      int n;
21      vector<vector<Edge>> g;
22      vector<array<int, 2>> pos;
23      vector<Cap> ex;
24      vector<int> h, cnt, cur;
25      vector<vector<int>> b;
26      int s = -1, t = -1, hi = 0, lmh = 0;
27
28      explicit MaxFlow(int n) : n(n), g(n) { assert(n >= 0); }
29
30      int add(int u, int v, Cap c) {
31          assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
32          int x = g[u].size();
33          int y = g[v].size() + (u == v);
34          g[u].push_back({v, y, c, c});
35          g[v].push_back({u, x, 0, 0});
36          pos.push_back({u, x});
37          return pos.size() - 1;
38      }
39
40      void add2(int u, int v, Cap c) {
41          add(u, v, c), add(v, u, c);
42      }
43
44      void reset() {
45          for (auto& a : g) for (auto& e : a) e.c = e.oc;
46      }
47
48      EdgeInfo askEdge(int id) const {

```

```

49     assert(0 <= id && id < pos.size());
50     auto [u, i] = pos[id];
51     const auto& e = g[u][i];
52     return {u, e.v, e.oc, e.oc - e.c};
53 }
54
55 Cap askFlow(int id) const {
56     return askEdge(id).f;
57 }
58
59 vector<EdgeInfo> askAll() const {
60     vector<EdgeInfo> a;
61     a.reserve(pos.size());
62     for (int i = 0; i < pos.size(); ++i) a.push_back(askEdge(i));
63     return a;
64 }
65
66 void erase(int id) {
67     assert(0 <= id && id < pos.size());
68     auto [u, i] = pos[id];
69     Edge& e = g[u][i];
70     Edge& f = g[e.v][e.r];
71     e.c = f.c = 0;
72 }
73
74 void active(int u) {
75     if (u == s || u == t || ex[u] <= 0 || h[u] >= lmh) return;
76     b[h[u]].push_back(u);
77     hi = max(hi, h[u]);
78 }
79
80 void label() {
81     lmh = 2 * n + 1;
82     h.assign(n, n + 1); // 不可达汇点的点仍可沿反向边把预流退回源点。
83     queue<int> q;
84     h[t] = 0, q.push(t);
85     while (!q.empty()) {
86         int u = q.front();
87         q.pop();
88         for (const auto& e : g[u]) {
89             const auto& r = g[e.v][e.r];
90             if (r.c <= 0 || h[e.v] != n + 1) continue;
91             h[e.v] = h[u] + 1;
92             q.push(e.v);
93         }
94     }
95     h[s] = n;
96     cnt.assign(lmh + 1, 0);
97     for (int x : h) ++cnt[x];
98     cur.assign(n, 0);
99     b.assign(lmh + 1, {});
100     hi = 0;
101     for (int u = 0; u < n; ++u) active(u);
102 }
103
104 void push(int u, Edge& e) {
105     Cap x = min(ex[u], e.c);
106     if (x <= 0 || h[u] != h[e.v] + 1) return;
107     bool z = ex[e.v] == 0;
108     e.c -= x;
109     g[e.v][e.r].c += x;
110     ex[u] -= x, ex[e.v] += x;
111     if (z) active(e.v);
112 }
113
114 void relabel(int u) {
115     int old = h[u], nh = lmh;
116     for (const auto& e : g[u]) if (e.c > 0) nh = min(nh, min<int>(lmh, h[e.v] + 1));
117     --cnt[old];
118     h[u] = nh;
119     ++cnt[nh];
120     cur[u] = 0;
121
122     if (old < n && cnt[old] == 0) {
123         for (int v = 0; v < n; ++v) {
124             if (v == s || v == t || h[v] <= old || h[v] >= n) continue;
125             --cnt[h[v]];
126             h[v] = n + 1;
127             ++cnt[h[v]];
128             active(v);
129         }
130     }

```



```

131     }
132
133     void push(int u) {
134         while (ex[u] > 0 && h[u] < lmh) {
135             if (cur[u] == g[u].size()) {
136                 relabel(u);
137                 continue;
138             }
139             Edge& e = g[u][cur[u]];
140             if (e.c > 0 && h[u] == h[e.v] + 1) push(u, e);
141             else ++cur[u];
142         }
143         active(u);
144     }
145
146     Cap flow(int _s, int _t) {
147         assert(0 <= _s && _s < n && 0 <= _t && _t < n && _s != _t);
148         s = _s, t = _t;
149         ex.assign(n, Cap{});
150         label();
151         for (auto& e : g[s]) {
152             if (e.v == s || e.c <= 0) continue;
153             Cap x = e.c;
154             e.c = 0;
155             g[e.v][e.r].c += x;
156             ex[s] -= x, ex[e.v] += x;
157             active(e.v);
158         }
159         while (hi >= 0) {
160             if (b[hi].empty()) {
161                 --hi;
162                 continue;
163             }
164             int u = b[hi].back();
165             b[hi].pop_back();
166             if (h[u] != hi || ex[u] <= 0) continue;
167             push(u);
168         }
169         return ex[t];
170     }
171
172     vector<char> askCut(int s) const {
173         vector<char> a(n);
174         queue<int> q;
175         q.push(s), a[s] = true;
176         while (!q.empty()) {
177             int u = q.front();
178             q.pop();
179             for (const auto& e : g[u]) if (e.c > 0 && !a[e.v]) a[e.v] = true, q.push(e.v);
180         }
181         return a;
182     }
183 };

```

01C - 最大流 (HLPP, 极致优化) .cpp

```

1  /*
2  用途: highest-label 预流推进的极低常数版, 使用反向 BFS、gap、指针当前弧与周期重标号。
3  用法: 接口与 01A / 01B 的 MaxFlow 一致; 大图卡常时替换使用。
4  约定: 点编号 0..n-1, Cap 必须是有符号整型; 容量默认使用宏展开后的 int。
5  性能: 点边下标必须保留 i32; 改成宏展开后的 int 会增大边表并降低缓存命中率。
6  */
7
8  using i32 = int32_t;
9
10 template <class Cap = int> struct MaxFlow {
11     static constexpr Cap INF = numeric_limits<Cap>::max() / 4;
12
13     struct Edge {
14         i32 v, r;
15         Cap f, c;
16     };
17     struct EdgeInfo {
18         int u, v;
19         Cap c, f;
20     };
21
22     i32 n, s = -1, t = -1, hi = 0, lm = 0;
23     int wk = 0;
24     vector<vector<Edge>> g;

```

```

25     vector<array<i32, 2>> pos;
26     vector<Cap> ex;
27     vector<i32> h, cnt;
28     vector<Edge*> cur;
29     vector<vector<i32>> b;
30
31     explicit MaxFlow(int n) : n(n), g(n) {
32         assert(0 <= n && n <= numeric_limits<i32>::max());
33     }
34
35     int add(int u, int v, Cap c) {
36         assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
37         i32 x = g[u].size(), y = g[v].size() + (u == v);
38         g[u].push_back({i32(v), y, 0, c});
39         g[v].push_back({i32(u), x, 0, 0});
40         pos.push_back({i32(u), x});
41         return pos.size() - 1;
42     }
43
44     void add2(int u, int v, Cap c) {
45         add(u, v, c), add(v, u, c);
46     }
47
48     void reset() {
49         for (auto& a : g) {
50             for (auto& e : a) e.c += e.f, e.f = 0;
51         }
52     }
53
54     EdgeInfo askEdge(int id) const {
55         int m = pos.size();
56         assert(0 <= id && id < m);
57         auto [u, i] = pos[id];
58         const auto& e = g[u][i];
59         return {u, e.v, e.c + e.f, e.f};
60     }
61
62     Cap askFlow(int id) const {
63         return askEdge(id).f;
64     }
65
66     vector<EdgeInfo> askAll() const {
67         vector<EdgeInfo> a;
68         int m = pos.size();
69         a.reserve(m);
70         for (int i = 0; i < m; ++i) a.push_back(askEdge(i));
71         return a;
72     }
73
74     void erase(int id) {
75         int m = pos.size();
76         assert(0 <= id && id < m);
77         auto [u, i] = pos[id];
78         auto& e = g[u][i];
79         auto& r = g[e.v][e.r];
80         e.f = e.c = r.f = r.c = 0;
81     }
82
83     void active(i32 u) {
84         if (u == s || u == t || ex[u] <= 0 || h[u] >= lm) return;
85         b[h[u]].push_back(u);
86         hi = max(hi, h[u]);
87     }
88
89     void bfs() {
90         lm = 2 * n + 1;
91         h.assign(n, n + 1);
92         cnt.assign(lm + 1, 0);
93         cur.resize(n);
94         b.assign(lm + 1, {});
95         queue<i32> q;
96         h[t] = 0, q.push(t);
97         while (!q.empty()) {
98             i32 u = q.front();
99             q.pop();
100             for (const auto& e : g[u]) {
101                 if (e.v == s || h[e.v] != n + 1 || g[e.v][e.r].c <= 0) continue;
102                 h[e.v] = h[u] + 1;
103                 q.push(e.v);
104             }
105         }
106         h[s] = n;

```

```

107     for (i32 u = 0; u < n; ++u) ++cnt[h[u]], cur[u] = g[u].data();
108     hi = 0, wk = 0;
109     for (i32 u = 0; u < n; ++u) active(u);
110 }
111
112 void gap(i32 k, i32 z) {
113     for (i32 u = 0; u < n; ++u) {
114         if (u == s || u == t || h[u] <= k || h[u] >= n + 1) continue;
115         --cnt[h[u]];
116         h[u] = n + 1;
117         ++cnt[h[u]];
118         cur[u] = g[u].data();
119         if (u != z) active(u);
120     }
121 }
122
123 void label(i32 u) {
124     i32 oh = h[u], nh = lm;
125     for (const auto& e : g[u]) {
126         ++wk;
127         if (e.c > 0) nh = min(nh, i32(h[e.v] + 1));
128     }
129     --cnt[oh];
130     h[u] = nh;
131     ++cnt[nh];
132     cur[u] = g[u].data();
133     if (oh < n && !cnt[oh]) gap(oh, u);
134 }
135
136 void push(i32 u, Edge& e) {
137     Cap f = min(ex[u], e.c);
138     if (f <= 0 || h[u] != h[e.v] + 1) return;
139     bool z = ex[e.v] == 0;
140     auto& r = g[e.v][e.r];
141     e.f += f, e.c -= f;
142     r.f -= f, r.c += f;
143     ex[u] -= f, ex[e.v] += f;
144     if (z) active(e.v);
145 }
146
147 void work(i32 u) {
148     Edge* ed = g[u].data() + g[u].size();
149     while (ex[u] > 0 && h[u] < lm) {
150         if (cur[u] == ed) {
151             label(u);
152             ed = g[u].data() + g[u].size();
153         } else if (cur[u]->c > 0 && h[u] == h[cur[u]->v] + 1) {
154             push(u, *cur[u]);
155         } else {
156             ++cur[u], ++wk;
157         }
158     }
159     active(u);
160 }
161
162 Cap flow(int _s, int _t) {
163     assert(0 <= _s && _s < n && 0 <= _t && _t < n && _s != _t);
164     s = _s, t = _t;
165     ex.assign(n, Cap{});
166     bfs();
167     for (auto& e : g[s]) {
168         if (e.v == s || e.c <= 0) continue;
169         Cap f = e.c;
170         auto& r = g[e.v][e.r];
171         e.f += f, e.c = 0;
172         r.f -= f, r.c += f;
173         ex[s] -= f, ex[e.v] += f;
174         active(e.v);
175     }
176     int m = pos.size();
177     while (hi >= 0) {
178         if (b[hi].empty()) {
179             --hi;
180             continue;
181         }
182         i32 u = b[hi].back();
183         b[hi].pop_back();
184         if (h[u] != hi || ex[u] <= 0) continue;
185         work(u);
186         if (wk > 4 * m + n) bfs();
187     }
188     return ex[t];

```

```

189     }
190
191     vector<char> askCut(int s) const {
192         assert(0 <= s && s < n);
193         vector<char> a(n);
194         queue<i32> q;
195         q.push(s), a[s] = true;
196         while (!q.empty()) {
197             i32 u = q.front();
198             q.pop();
199             for (const auto& e : g[u]) {
200                 if (e.c > 0 && !a[e.v]) a[e.v] = true, q.push(e.v);
201             }
202         }
203         return a;
204     }
205 };

```

02 - 最小割.cpp

```

1  /*
2  用途: S-T 最小割值及源点侧点集恢复。
3  依赖: 01A - 最大流 (Dinic) 的 MaxFlow。
4  约定: 点编号 0..n-1; solve 会从初始容量开始计算, 可重复调用。
5  复杂度: 与 Dinic 一致。
6  */
7
8  template <class Cap = int> struct MinCut {
9      struct Result {
10         Cap val;
11         vector<char> sd;
12     };
13
14     int n;
15     MaxFlow<Cap> mf;
16
17     explicit MinCut(int n) : n(n), mf(n) {}
18
19     int add(int u, int v, Cap c) {
20         return mf.add(u, v, c);
21     }
22
23     Result solve(int s, int t) {
24         mf.reset();
25         Cap val = mf.flow(s, t);
26         return {val, mf.askCut(s)};
27     }
28 };

```

02A - 最大权闭合图 (最小割建模) .cpp

```

1  /*
2  用途: 最大权闭合图。若选择 u 必须选择 v, 则 add(u,v)。
3  依赖: 01A - 最大流 (Dinic) 的 MaxFlow。
4  约定: 点编号 0..n-1; modify 可对同一点多次调用。依赖边容量应大于所有可能答案。
5  复杂度: 一次 Dinic 最大流。
6  */
7
8  template <class Cap = int> struct MaxClosure {
9      static constexpr Cap INF = numeric_limits<Cap>::max() / 4;
10
11     struct Result {
12         Cap val;
13         vector<char> vis;
14     };
15
16     int n;
17     int s, t;
18     MaxFlow<Cap> mf;
19     Cap sum = 0;
20
21     explicit MaxClosure(int n) : n(n), s(n), t(n + 1), mf(n + 2) {}
22
23     void modify(int u, Cap w) {
24         assert(0 <= u && u < n);
25         if (w > 0) {
26             mf.add(s, u, w);
27             sum += w;

```

```

28     } else if (w < 0) {
29         mf.add(u, t, -w);
30     }
31 }
32
33 void add(int u, int v, Cap c = INF) {
34     assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
35     mf.add(u, v, c);
36 }
37
38 Result solve() {
39     mf.reset();
40     Cap cut = mf.flow(s, t);
41     vector<char> sd = mf.askCut(s);
42     vector<char> vis(n, false);
43     for (int u = 0; u < n; ++u) vis[u] = sd[u];
44     return {sum - cut, move(vis)};
45 }
46 };

```

03 - 最小割树.cpp

```

1  /*
2  用途: Gomory-Hu Tree (无向图任意两点最小割)。
3  依赖: 01A - 最大流 (Dinic) 的 MaxFlow。
4  约定: 点编号 0..n-1, 边容量非负; build 后 g 是一棵带权树,
5  任意两点最小割等于树路径上的最小边权。ask(u,v) 单次 O(log n)。
6  复杂度: 建树调用 n-1 次最大流, 额外 O(n log n) 预处理。
7  */
8
9  template <class Cap = int> struct GHTree {
10     static constexpr Cap INF = numeric_limits<Cap>::max() / 4;
11
12     int n;
13     MaxFlow<Cap> mf;
14     vector<vector<pair<int, Cap>>> g;
15     vector<int> dep;
16     vector<vector<int>> up;
17     vector<vector<Cap>> mn;
18     bool ok = false;
19
20     explicit GHTree(int n) : n(n), mf(n), g(n) {
21         assert(n > 0);
22     }
23
24     void add(int u, int v, Cap c) {
25         assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
26         mf.add(u, v, c);
27         mf.add(v, u, c);
28         ok = false;
29     }
30
31     void build() {
32         vector<int> fa(n, 0);
33         vector<Cap> cut(n);
34         fa[0] = -1;
35
36         for (int s = 1; s < n; ++s) {
37             int t = fa[s];
38             mf.reset();
39             Cap val = mf.flow(s, t);
40             vector<char> sd = mf.askCut(s);
41             cut[s] = val;
42
43             for (int u = s + 1; u < n; ++u) {
44                 if (fa[u] == t && sd[u]) fa[u] = s;
45             }
46             if (fa[t] != -1 && sd[fa[t]]) {
47                 fa[s] = fa[t];
48                 fa[t] = s;
49                 cut[s] = cut[t];
50                 cut[t] = val;
51             }
52         }
53
54         g.assign(n, {});
55         for (int u = 1; u < n; ++u) {
56             g[u].push_back({fa[u], cut[u]});
57             g[fa[u]].push_back({u, cut[u]});
58         }

```

```

59     init();
60     ok = true;
61 }
62
63 Cap ask(int u, int v) const {
64     assert(ok && 0 <= u && u < n && 0 <= v && v < n);
65     if (u == v) return 0;
66     Cap ans = INF;
67     if (dep[u] < dep[v]) swap(u, v);
68     int d = dep[u] - dep[v];
69     for (int bit = 0; d > 0; ++bit, d >= 1) {
70         if (d & 1) {
71             ans = min(ans, mn[bit][u]);
72             u = up[bit][u];
73         }
74     }
75     if (u == v) return ans;
76
77     for (int bit = up.size(); bit--;) {
78         if (up[bit][u] == up[bit][v]) continue;
79         ans = min(ans, mn[bit][u]);
80         ans = min(ans, mn[bit][v]);
81         u = up[bit][u];
82         v = up[bit][v];
83     }
84     ans = min(ans, mn[0][u]);
85     ans = min(ans, mn[0][v]);
86     return ans;
87 }
88
89 private:
90 void init() {
91     int log = 1;
92     while ((1 << log) <= n) ++log;
93     dep.assign(n, 0);
94     up.assign(log, vector<int>(n));
95     mn.assign(log, vector<Cap>(n, INF));
96
97     queue<int> q;
98     vector<char> vis(n, false);
99     q.push(0);
100    vis[0] = true;
101    up[0][0] = 0;
102    while (!q.empty()) {
103        int u = q.front();
104        q.pop();
105        for (auto [v, w] : g[u]) {
106            if (vis[v]) continue;
107            vis[v] = true;
108            dep[v] = dep[u] + 1;
109            up[0][v] = u;
110            mn[0][v] = w;
111            q.push(v);
112        }
113    }
114
115    for (int bit = 1; bit < log; ++bit) {
116        for (int u = 0; u < n; ++u) {
117            up[bit][u] = up[bit - 1][up[bit - 1][u]];
118            mn[bit][u] = min(mn[bit - 1][u], mn[bit - 1][up[bit - 1][u]]);
119        }
120    }
121 }
122 };

```

04 - 最小费用最大流 (MinCostFlow) .cpp

```

1  /*
2  用途: 连续最短增广路最小费用最大流, 支持初始负费用边。
3  约定: 点编号 0..n-1, 容量非负; 不存在从源点可达的负费用环。
4  flow 不自动 reset, 若需重复求解请先 reset。返回 nullopt 表示检测到可达负环。
5  复杂度: 每次增广 Dijkstra  $O(m \log n)$ , 总次数至多为增广次数。
6  */
7
8  template <class Cap = int, class C = int> struct MCMF {
9      static constexpr Cap FI = numeric_limits<Cap>::max() / 4;
10     static constexpr C CI = numeric_limits<C>::max() / 4;
11
12     struct Edge {
13         int v, r;

```

```

14     Cap c, oc;
15     C w;
16 };
17
18 struct EdgeInfo {
19     int u, v;
20     Cap c, f;
21     C w;
22 };
23
24 struct Result {
25     Cap f;
26     C c;
27 };
28
29 int n;
30 vector<vector<Edge>> g;
31 vector<pair<int, int>> pos;
32
33 explicit MCMF(int n) : n(n), g(n) {}
34
35 int add(int u, int v, Cap c, C w) {
36     assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
37     int id = g[u].size();
38     int r = g[v].size() + (u == v);
39     g[u].push_back({v, r, c, c, w});
40     g[v].push_back({u, id, 0, 0, -w});
41     pos.push_back({u, id});
42     return pos.size() - 1;
43 }
44
45 void reset() {
46     for (auto& a : g) {
47         for (Edge& e : a) e.c = e.oc;
48     }
49 }
50
51 EdgeInfo askEdge(int id) const {
52     assert(0 <= id && id < pos.size());
53     auto [u, i] = pos[id];
54     const Edge& e = g[u][i];
55     return {u, e.v, e.oc, e.oc - e.c, e.w};
56 }
57
58 vector<EdgeInfo> askAll() const {
59     vector<EdgeInfo> res;
60     res.reserve(pos.size());
61     for (int id = 0; id < pos.size(); ++id) res.push_back(askEdge(id));
62     return res;
63 }
64
65 optional<Result> flow(int s, int t, Cap lim = FI) {
66     static_assert(std::is_signed_v<C>, "Cost must be signed because reverse edges negate it.");
67     assert(0 <= s && s < n && 0 <= t && t < n && s != t && lim >= 0);
68
69     vector<C> h;
70     if (!initPot(s, h)) return nullopt;
71
72     Cap f = 0;
73     C c = 0;
74     while (f < lim) {
75         vector<C> dis(n, CI);
76         vector<pair<int, int>> fa(n, {-1, -1});
77         priority_queue<pair<C, int>, vector<pair<C, int>>, greater<pair<C, int>>> pq;
78         dis[s] = 0;
79         pq.push({0, s});
80
81         while (!pq.empty()) {
82             auto [du, u] = pq.top();
83             pq.pop();
84             if (du != dis[u]) continue;
85             for (int i = 0; i < g[u].size(); ++i) {
86                 const Edge& e = g[u][i];
87                 if (e.c <= 0) continue;
88                 C rc = e.w + h[u] - h[e.v];
89                 if (dis[e.v] > du + rc) {
90                     dis[e.v] = du + rc;
91                     fa[e.v] = {u, i};
92                     pq.push({dis[e.v], e.v});
93                 }
94             }
95         }

```

```

96         if (fa[t].first == -1) break;
97
98         for (int u = 0; u < n; ++u) {
99             if (dis[u] != CI) h[u] += dis[u];
100         }
101
102         Cap aug = lim - f;
103         C pc = 0;
104         for (int u = t; u != s;) {
105             auto [pre, i] = fa[u];
106             aug = min(aug, g[pre][i].c);
107             pc += g[pre][i].w;
108             u = pre;
109         }
110         for (int u = t; u != s;) {
111             auto [pre, i] = fa[u];
112             Edge& e = g[pre][i];
113             e.c -= aug;
114             g[e.v][e.r].c += aug;
115             u = pre;
116         }
117         f += aug;
118         c += static_cast<C>(aug) * pc;
119     }
120     return Result{f, c};
121 }
122
123 private:
124     bool initPot(int s, vector<C>& h) const {
125         vector<C> dis(n, CI);
126         vector<int> pec(n);
127         vector<char> inq(n, false);
128         queue<int> q;
129         dis[s] = 0;
130         q.push(s);
131         inq[s] = true;
132
133         while (!q.empty()) {
134             int u = q.front();
135             q.pop();
136             inq[u] = false;
137             for (const Edge& e : g[u]) {
138                 if (e.c <= 0 || dis[e.v] <= dis[u] + e.w) continue;
139                 dis[e.v] = dis[u] + e.w;
140                 pec[e.v] = pec[u] + 1;
141                 if (pec[e.v] >= n) return false;
142                 if (!inq[e.v]) {
143                     inq[e.v] = true;
144                     q.push(e.v);
145                 }
146             }
147         }
148
149         h.assign(n, 0);
150         for (int u = 0; u < n; ++u) {
151             if (dis[u] != CI) h[u] = dis[u];
152         }
153         return true;
154     }
155 };

```

05 - 上下界网络流.cpp

```

1  /*
2  用途：带下界/上界的可行循环流，以及带源汇的最大流。
3  依赖：01A - 最大流 (Dinic)。
4  约定：点编号 0..n-1, add(u,v,lo,hi) 满足 0 <= lo <= hi。
5  ask 求可行循环流；flow(s,t) 求满足上下界后的最大 S-T 流。
6  复杂度：常数 * Dinic 最大流。
7  */
8
9  template <class Cap = int> struct BoundFlow {
10     static constexpr Cap INF = numeric_limits<Cap>::max() / 4;
11
12     struct Edge {
13         int u, v;
14         Cap lo, hi;
15     };
16
17     struct Result {

```



```

18     bool ok;
19     Cap val;           // 循环流时为 0; S-T 流时为最大流值。
20     vector<Cap> a;      // 与 add 顺序对应, 已包含下界。
21 };
22
23 int n;
24 vector<Edge> e;
25
26 explicit BoundFlow(int n) : n(n) {}
27
28 int add(int u, int v, Cap lo, Cap hi) {
29     assert(0 <= u && u < n && 0 <= v && v < n && 0 <= lo && lo <= hi);
30     e.push_back({u, v, lo, hi});
31     return e.size() - 1;
32 }
33
34 Result ask() const {
35     static_assert(std::is_signed_v<Cap>, "Cap must be signed because balances may be negative.");
36     Network z = build(false, -1, -1);
37     Cap f = z.mf.flow(z.ss, z.tt);
38     if (f != z.nd) return {false, 0, {}};
39     return {true, 0, get(z)};
40 }
41
42 Result flow(int s, int t) const {
43     static_assert(std::is_signed_v<Cap>, "Cap must be signed because balances may be negative.");
44     assert(0 <= s && s < n && 0 <= t && t < n && s != t);
45     Network z = build(true, s, t);
46     Cap f = z.mf.flow(z.ss, z.tt);
47     if (f != z.nd) return {false, 0, {}};
48
49     Cap bas = z.mf.askFlow(z.rid);
50     for (int id : z.aid) z.mf.erase(id);
51     z.mf.erase(z.rid);
52     Cap add = z.mf.flow(s, t);
53     return {true, bas + add, get(z)};
54 }
55
56 private:
57     struct Network {
58         MaxFlow<Cap> mf;
59         int ss, tt;
60         Cap nd = 0;
61         vector<int> eid;
62         vector<int> aid;
63         int rid = -1;
64
65         explicit Network(int n) : mf(n + 2), ss(n), tt(n + 1) {}
66     };
67
68     Network build(bool bk, int s, int t) const {
69         Network z(n);
70         vector<Cap> bal(n, 0);
71         z.eid.reserve(e.size());
72         for (const Edge& x : e) {
73             z.eid.push_back(z.mf.add(x.u, x.v, x.hi - x.lo));
74             bal[x.u] -= x.lo;
75             bal[x.v] += x.lo;
76         }
77         if (bk) {
78             z.rid = z.mf.add(t, s, INF);
79         }
80         for (int u = 0; u < n; ++u) {
81             if (bal[u] > 0) {
82                 z.aid.push_back(z.mf.add(z.ss, u, bal[u]));
83                 z.nd += bal[u];
84             } else if (bal[u] < 0) {
85                 z.aid.push_back(z.mf.add(u, z.tt, -bal[u]));
86             }
87         }
88         return z;
89     }
90
91     vector<Cap> get(const Network& z) const {
92         vector<Cap> res(e.size());
93         for (int id = 0; id < e.size(); ++id) {
94             res[id] = e[id].lo + z.mf.askFlow(z.eid[id]);
95         }
96         return res;
97     }
98 };

```

数学：数论、组合、线性代数与几何

71 个代码模板 · 林月 · v1.0.0 · 2026.8.1

版块说明与知识结论

README.md

数学模板

取模接口

默认先引入 02A - 自动取模 (MInt) , 例如 `using Z = MInt<998244353>;`。MInt 内部按性能需要使用 i32 存储, 开启全局 `#define int long long` 后也不会把模数数组扩大为 64 位。

- 需要域上四则运算的模板均以 `template<class Z>` 接收自动取模类型: 组合数、Lucas、插值、线性递推、模高斯、矩阵树、FWT/子集卷积等直接传入 Z 即可。
- 03 - 组合数学使用 `Comb<Z> c(n)`; 02D 使用 `invAll<Z>(n)`; 03C 额外提供 `Cantor::askRank<Z>(p)`。
- 08B (完整) 和 08C (轻量) 均使用 `Poly<Z, 3>`, 不再依赖全局 `using Z`; 要求 Z 的模数是对应原根的 NTT 友好质数。
- `MInt<0>` 适合单一运行时质数模数; 同一程序不能同时保存多个动态模数对象。
- 02AA 的 Barrett 只用于 $1 \leq \text{mod} < 2^{31}$ 的 u32 模乘热点; 它是确定字宽卡常工具, 不替代 MInt, 也不承诺完整 u32 模数。

以下模板不能只靠 MInt: 它们的模数是运行时输入、可能为合数或需要完整 u64 范围, 因此仍保留显式 `mod` 参数: 64 位快速幂/Miller-Rabin/Pollard-Rho、CRT/扩展 Lucas/BSGS/Pohlig-Hellman、原根、整除分块和十进制大指数模幂。任意模 FFT (08E) 和 CRT-NTT (08F / 08IA) 保留显式模数接口。普通 2 幂 NTT 用 08, 任意整除长度 DFT 用 08P; 极限卷积改用 08I 的 i32/u32 Montgomery radix-4 分支。自动取模多项式选 08B、08C。

本目录以覆盖优先: 低频、专项、不同数据范围与不同模数环境的模板均保留, 不按使用频率删除。所有 .cpp 默认已有常用头文件和 std; 公式、前提与典型模型见 [数学知识与结论.md](#)。

赛时检索入口

遇到的问题	优先查找
大整数判素/分解、质数计数/质数和、约数函数	01F、01FA、01I、01J、01H
模逆、CRT、离散对数、原根、模平方根、合数模组合数	02BA、02BAA、02C、02I、02F、02E / 02EA、02BAAB
固定小值域 x 与大整数 y 的高频 gcd	02J
底数/指数是十进制大数的模幂	02H
组合计数、插值、递推、分拆、快速阶乘	03、03A、03D 至 03M; 大规模斯特林/Bell 整行查 03EA/03FA
普通或带权整除和	02G、02GA
整除容斥、莫比乌斯、Dirichlet 卷积	04A/04B、01D、01DA、01DB、01DC
线性方程、行列式、线性规划	05B 至 05G
普通/按位/子集卷积、多项式与 Power Projection	08、08A 至 08P
正常游戏、Wythoff Nim	09、09A
数值求根、牛顿迭代、积分、三分	11A 至 11D
位操作、异或线性基、Gray 编码	12A 至 12C
Fibonacci、约瑟夫环	13A、13B

编号规约

- X 是核心问题或概念分组; 不同问题使用不同根编号。

- 若 B 以 A 为直接前置，使用 $X \rightarrow XA \rightarrow XAA$ ，例如 $02B \rightarrow 02BA \rightarrow 02BAA$ 。
- 同一问题的不同实现使用同级分支，按数据范围、精度或模数环境选择，绝不因相似而删除。
- 跨类别依赖写在文件首部；同名 Poly、FFT 等备选实现不可同时粘贴。

01 - 素数、积性函数与约数

编号	模板	适用场景
01	素数（线性筛）	批量得到质数、minPrime、phi、mu。
01A	质因数分解（最小质因子）	已筛范围内的大量分解。
01B / 01C	欧拉函数（单点试除 / 整表筛）	少量单点 / 单独整张 phi 表。
01D	莫比乌斯函数（Mertens、杜教筛）	$M(n)=\sum \mu(i)$ 的整除分块。
01DA	莫比乌斯反演（约数和变换）	约数和、倍数和 zeta 与反演。
01DB	加权莫比乌斯函数（ d^2 ）	$\sum \mu(i)i^2$ 的加权杜教筛。
01DC	狄利克雷卷积与逆	积性函数卷积、Dirichlet 逆。
01EA / 01EB	检查是否是素数的幂（试除 / 最小质因子）	少量单点 / 已筛范围大量查询。
01F / 01G	Miller-Rabin / 素性检验（试除）	u64 大数 / 小整数无预处理。
01FA	Pollard-Rho 质因数分解	依赖 01F 的 64 位大整数分解。
01H	约数函数（枚举、个数、和）	已知分解后求 tau、sigma、phi、枚举约数。
01I	质数计数（Lehmer）	无法开到 n 时求 $\pi(n)$ 。
01J	质数和（Min_25 筛）	无法筛到 n 时求所有 $p \leq n$ 的和，支持精确或自动取模。

02 - 模运算、同余与数论方程

编号	模板	适用场景
02	快速幂（ksm）	64 位安全模乘、模幂、费马逆元。
02A	自动取模（MInt）	常用模数类型，组合与多项式的基础。
02AA	Barrett 约简（u32）	$\text{mod} < 2^{31}$ 的运行时模乘热点。
02B $\rightarrow$ 02BA	裴蜀定理 $\rightarrow$ 扩展欧几里得	整数线性组合、一般模逆、线性同余。
02BAA	中国剩余定理	可不互质的广义 CRT。
02BAAB	扩展 Lucas（组合数模合数）	质因子幂可预处理的合数模 $C(n,k)$ 。
02C	BSGS	互素/非互素离散对数。
02D	逆元线性预处理	质数模下 $1..n$ 的逆元表。
02E / 02EA	模平方根（Tonelli-Shanks / Cipolla）	确定性迭代 / 随机化二次扩域。
02F	原根	判断一般模数是否存在原根，寻找最小/全部原根，并验证给定数。
02G	整除分块求和（floor_sum）	$\sum \text{floor}((ai+b)/m)$ 、格点计数。
02GA	加权整除和（万能欧几里得）	同时求 floor 值的一、二次矩；自动取模域中使用。
02H	大指数模幂（十进制）	底数或指数为十进制串，不要求互素。
02I	Pohlig-Hellman	质数模、已知原根且 $p-1$ 平滑的离散对数。
02J	值域 GCD 预处理	$x \leq n$ 固定小值域、y 很大且查询很多的 gcd。

03 - 组合、排列、插值与递推

编号	模板	适用场景
03	组合数学（Comb, with Z）	$n < \text{mod}$ 的排列、组合。
03A	Lucas 定理	大 n,k 与小质数模。
03B / 03C	康托展开（暴力 / 树状数组）	小 n 精确可反排名 / 大 n 模意义排名。

编号	模板	适用场景
03D	卡特兰数	括号序列、二叉树、栈排列、三角剖分。
03E / 03EA	斯特林数（第一、二类，递推 / NTT 快速）	小规模整表 / 固定 n 的大规模整行。
03F / 03FA	错排数与贝尔数（递推 / NTT 快速）	小规模预处理 / 大规模 Bell 整表。
03G / 03H	拉格朗日插值（任意 / 连续横坐标）	$O(n^2)$ 通用 / $O(n)$ 连续点单值查询。
03I	牛顿插值（差分级数）	Newton series、显式有限差分系数。
03J / 03JA	整数分拆（五边形数 / NTT 快速）	小规模递推 / 大规模整表。
03K $\rightarrow$ 03KA	Berlekamp-Massey $\rightarrow$ Kitamasa	恢复最短递推，再求超大下标项。
03L	Burnside 引理（群作用计数）	旋转、翻转、Pólya 类对称计数。
03M	快速阶乘（NTT）	NTT 友好质数模下的 $O(\sqrt{n} \log^2 n)$ 阶乘。

04 - 容斥

编号	模板	适用场景
04A / 04B	容斥（bitmask / DFS 剪枝）	整除集合并计数；小因子数 / lcm 很快超界。

05 - 矩阵、线性方程与优化

编号	模板	适用场景
05	矩阵（Matrix, with Int）	矩阵乘法、快速幂、泛型行列式。
05A	Min-Plus 矩阵	固定边数最短路。
05B / 05C	高斯消元 / 对称 LDLT	一般实系数方程组 / 对称无零枢轴专用。
05D	高斯消元（模质数）	质数模线性方程组。
05E	行列式（模质数）	域上的精确消元。
05F	线性规划（单纯形）	$\max c \cdot x, A \cdot x \leq b, x \geq 0$ 。
05G	高斯消元（GF(2) 位集）	异或方程、布尔线性约束。
05H	矩阵树定理（Kirchhoff）	无向带权图生成树计数。

06 - 代数数据类型与 07 - 平面几何

编号	模板	适用场景
06	分数（Frac, with Int）	有理数四则运算。
06A / 06B	复数 / 三次单位根复数	普通复数代数 / $w^2+w+1=0$ 的二元代数。
07	平面几何（Point&Line, with Int）	点线、线段、多边形、半平面交。
07A	凸包与旋转卡壳	凸包、直径、凸多边形点包含；依赖 07。
07AA	闵可夫斯基和	两个点集凸包的向量和；依赖 07A。
07B	圆与最小覆盖圆	圆交、外接圆、随机增量最小覆盖圆；依赖 07。
07C	三维几何	三维向量、体积、点线/面距离与线面交。
07D	平面最近点对	分治 $O(n \log n)$ ，返回原下标与距离平方；依赖 07。

08 - 变换、多项式与子集卷积

编号	模板	适用场景
08 / 08I	快速数论变换（NTT，常规 / 极致）	普通模卷积 / Montgomery radix-4 卡常分支。
08A / 08B / 08C	三种 Poly（int / Z 全功能 / Z 轻量）	按类型与模数环境三选一。
08D / 08E	FFT / ModFFT	实数或小系数卷积 / 任意模数卷积。
08F / 08IA	多模快速数论变换（CRTNTT，常规 / 极致）	任意模精确重构；08IA 面向 FFT killer 与极限长度。

编号	模板	适用场景
08G	FWT (OR、AND、XOR 卷积)	按位运算卷积。
08H	SOS DP (子集、超集 Zeta 变换)	按位包含关系求和与反演。
08HA	子集卷积 (Subset Convolution)	$c[S]=\sum A \text{ subset } S a[A]b[S\setminus A]$ 。
08J	二维卷积 (2D NTT)	双变量多项式卷积; 依赖 08。
08K	有理函数第 n 项 (Bostan-Mori)	求 $[x^n]f/g$; 依赖 08B。
08L	连续点值平移	已知 $f(0..n)$ 求 $f(m..m+n)$; 依赖 08B。
08M	稀疏多项式除法	稀疏分母的截断幂级数除法。
08N / 08NA	多项式开方与幂 (普通 / 十进制大指数)	非负整数幂 / 指数为十进制串; 依赖 02E、08B。
08O	Power Projection (基础)	求 $[x^N]f(x)g(x)^i$ 的连续 i 值; 依赖 08B。
08OA	Power Projection (极致优化)	同一问题的 998244353 准线性卡常分支; 依赖 08I。
08P	任意因子长度 DFT	长度整除 $p-1$ 的 DFT/IDFT; 依赖 08。

09 - 博弈论

编号	模板	适用场景
09	博弈论 (Nim、SG)	Bash、Nim 取法/变体、SG、树上删边与常见结论函数。
09A	Wythoff Nim	两堆可同减博弈的黄金分割必败态。

10 - 高精度常数

编号	模板	适用场景
10A / 10B	圆周率 (Machin / Chudnovsky)	约 2×10^3 位 / 更高精度; 依赖 01A + 02A, 10B 推荐改用 FFT 的 01B + 02A。

11 - 数值计算

编号	模板	适用场景
11A	牛顿迭代 (方程、平方根、开 k 次方)	可导方程的局部快速收敛。
11B	自适应辛普森积分	平滑函数的定积分。
11C	三分搜索 (实数、整数)	单峰/单谷函数最值点。
11D	数值求根 (二分、割线)	连续函数求根; 稳定二分与快速割线。

12 - 位运算

编号	模板	适用场景
12A	位运算与子集枚举	lowbit、位长、子掩码、超掩码、同 popcount 后继。
12B	异或线性基 (XorBasis64)	最大异或、可表示性、基向量化简。
12C	二进制 GCD 与 Gray 编码	Stein gcd、Gray 互转。

13 - 常用数列与递推小工具

编号	模板	适用场景
13A	Fibonacci (快速倍增)	任意支持环运算的 $O(\log n)$ Fibonacci。
13B	约瑟夫环	固定步长循环删除的幸存者。
14 / 14A	伯努利数与幂和 (递推 / NTT 快速)	小阶数直接预处理幂和 / 大阶数快速求 B_i 。

数学知识与结论.md

数学知识、结论与例题

使用公式前先检查四件事：

1. 模数是否为质数、分母是否可逆；
2. 计数对象是否有标号、是否允许为空、是否允许重复；
3. 是否存在负数、溢出或浮点误差；
4. 结论是否只适用于有限无环、互素、非零主元等前提。

0. 赛时检索入口

看到的结构	先想的结论	对应模板
大 n 、小质数模的组合数	Lucas 分位相乘	03A
多个整除条件的并集	容斥，交集取 lcm	04A / 04B
“互质”计数、约数和反演	$\sum_{d n} \mu(d)$	01D、01DA、01DB、01DC
球盒、集合划分、合法括号	隔板法、Stirling、Catalan	03、03D、03E、03F
排列排名或反排名	康托展开	03B / 03C
大整数判素或分解	Miller-Rabin + Pollard-Rho	01F / 01FA
同余方程、逆元、多个余数	exgcd、CRT	02BA / 02BAA
质数模平方根	Tonelli-Shanks / Cipolla	02E / 02EA
离散对数且 $p-1$ 的质因子较小	Pohlig-Hellman 分解群阶	02I
一个操作数在固定小值域、另一个很大且 gcd 查询很多	分解 x 为三个小/素因子块	02J
底数或指数是十进制大数	逐位十进制快速幂	02H
需要 floor 值及其一、二次矩	加权整除和（万能欧几里得）	02GA
按位和、异或、子掩码	恒等式、FWT / SOS / 线性基	08G、08H、08HA、12A、12B
二维卷积、生成函数第 n 项、连续点值平移、Power Projection	二维 NTT、Bostan-Mori、卷积平移、幂投影	08J、08K、08L、08O / 08OA
正常规则的公平组合游戏	Nim、SG	09 / 09A
递推第 n 项	矩阵、BM、Kitamasa、快速倍增	05、03K、03KA、13A

1. 模运算、逆元与整除分块

1.1 除法、余数与负数

对于 $x \geq 0, m > 0$ ，令 $q = \lfloor x/m \rfloor$ ，则

$$x = qm + r, \quad r = x - qm, \quad 0 \leq r < m.$$

- C++ 整数除法对负数向 0 截断，不能把负数情形直接当作数学上的下取整。若需要标准余数，使用 $((x \bmod m) + m) \bmod m$ 。
- 把 x 加到下一个 m 的倍数所需的非负增量为 $(m - x \bmod m) \bmod m$ ；不能漏掉最后一层模，否则 $x \bmod m = 0$ 时会误得 m 。
- 若只依赖 $\lfloor n/i \rfloor$ ，从左端点 l 开始的一段相同商为

$$r = \left\lfloor \frac{n}{\lfloor n/l \rfloor} \right\rfloor, \quad \left\lfloor \frac{n}{i} \right\rfloor = \left\lfloor \frac{n}{l} \right\rfloor \quad (l \leq i \leq r).$$

不同商只有 $O(\sqrt{n})$ 个。对应 02G，杜教筛也以此为基础。

- 加权整除和令 $y_i = \lfloor (ai + b)/c \rfloor$ ，02GA 同时返回

$$\sum_{i=0}^n y_i, \quad \sum_{i=0}^n i y_i, \quad \sum_{i=0}^n y_i^2.$$

它把欧几里得递归的边界点计数扩展为一、二次矩；适合后续矩阵转移或二次代价。实现要求 2, 6 在系数域中可逆，通常取质数模的 MInt。

- 调和数 $H_n = \sum_{i=1}^n 1/i$ 满足

$$H_n = \ln n + \gamma + \frac{1}{2n} - \frac{1}{12n^2} + O(n^{-4}), \quad \gamma \approx 0.5772156649.$$

因此许多 $\sum n/i$ 型循环是 $O(n \log n)$ ；近似式只用于估计，不能替代精确计数。

1.2 模逆、快速幂与 CRT

- 若 p 为质数且 $a \not\equiv 0 \pmod{p}$ ，费马小定理给出 $a^{p-1} \equiv 1 \pmod{p}$ ，故 $a^{-1} \equiv a^{p-2} \pmod{p}$ 。对应 02、02A。
- 一般模数 m 下， a 有逆元当且仅当 $\gcd(a, m) = 1$ 。扩展欧几里得给出 $ax + my = 1$ ，其中 $x \bmod m$ 是逆元。对应 02BA。
- 线性同余 $ax \equiv b \pmod{m}$ 有解当且仅当 $d = \gcd(a, m)$ 整除 b ；约去 d 后求逆，可得到模 m/d 的一类解。
- 广义 CRT 合并

$$x \equiv r_1 \pmod{m_1}, \quad x \equiv r_2 \pmod{m_2}$$

的充要条件是 $\gcd(m_1, m_2) \mid (r_2 - r_1)$ ；合并后的模数为 $\text{lcm}(m_1, m_2)$ 。对应 02BAA。

- MInt 的加、减、乘不要求质数模；除法与 `inv()` 要求元素可逆。默认费马逆元不可直接用于复合模。
- Barrett 约简以乘高位近似商，适合固定运行时模数的大量乘法。02AA 只承诺 $1 \leq m < 2^{31}$ ；完整 64 位模乘仍使用 02 的 u128 分支，不能为了常数直接替换。
- 正整数 $n \geq 2$ 存在原根，当且仅当 $n = 2, 4, p^k, 2p^k$ ，其中 p 为奇质数、 $k \geq 1$ 。
- 在模 n 存在原根时， g 是原根当且仅当 $\gcd(g, n) = 1$ ，且对 $\varphi(n)$ 的每个不同质因子 q 均有 $g^{\varphi(n)/q} \not\equiv 1 \pmod{n}$ 。
- 若 g 是模 n 的一个原根，则全部原根恰为

$$g^k \bmod n \quad (1 \leq k \leq \varphi(n), \gcd(k, \varphi(n)) = 1),$$

因而共有 $\varphi(\varphi(n))$ 个。02F 的 `exist(n)` 先判断存在性，`roots(n)` 按数值递增返回这一集合；不存在时返回空数组。

- BSGS 直接在群中做 baby-step/giant-step，通用复杂度是 $O(\sqrt{p})$ 。若 p 为质数、 g 是原根且 $p-1 = \prod q_i^{e_i}$ ，Pohlig-Hellman 把指数分别模 $q_i^{e_i}$ 求出再 CRT 合并，复杂度主要为 $\sum_i O(\sqrt{q_i} \log p)$ 。因此它只在群阶平滑时有优势，对大质因子不应强行替代 BSGS。
- 十进制指数 $b = (\cdots d_2 d_1 d_0)_{10}$ 可从左到右维护 $r \leftarrow r^{10} a^{d_i} \bmod m$ 。该恒等式不要求 a, m 互素，适合指数无法读入整数时的 02H；欧拉降幂只是在满足额外条件时的另一种优化。
- 模平方根的 02E 是确定性 Tonelli-Shanks；02EA 的 Cipolla 先随机找 a 使 $a^2 - n$ 为非二次剩余，再在二次扩域中计算 $(a + \sqrt{a^2 - n})^{(p+1)/2}$ 。两者都只用于质数模；无解先由 Legendre 符号判断。Cipolla 的随机循环期望常数结束，但不能在要求严格确定性的场合替代 02E。

2. 素数、最大公因数与积性函数

2.1 筛、分解与欧拉函数

- 上界可开且查询多时，01 线性筛可同时得到最小质因子、 φ 与 μ 。单个 64 位整数用 01F 判素，合数交给 01FA 分解。
- 若 $n = \prod p_i^{e_i}$ ，则

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right) = \prod_i p_i^{e_i-1} (p_i - 1).$$

- 对 $n > 1$ ， $1 \leq a \leq n$ 且 $\gcd(a, n) = 1$ 的数之和为 $n\varphi(n)/2$ 。
- φ 是积性函数： $\gcd(a, b) = 1$ 时 $\varphi(ab) = \varphi(a)\varphi(b)$ ；任意积性函数都可由各个质数幂上的值相乘得到。
- 常用约数和：

$$\sum_{d|n} \varphi(d) = n, \quad \sum_{i=1}^n \gcd(i, n) = \sum_{d|n} d \varphi(n/d) = \sum_{d|n} \frac{n}{d} \varphi(d).$$

已知分解后，约数个数 $\tau(n) = \prod (e_i + 1)$ ，约数和 $\sigma(n) = \prod (1 + p_i + \dots + p_i^{e_i})$ 。对应 01H。

2.2 gcd 的平移与差分

- 欧几里得算法可写成 $\gcd(x, y) = \gcd(x, y - x)$ 。因此

$$\gcd(a_1, a_2, \dots, a_n) = \gcd(a_1, a_2 - a_1, \dots, a_n - a_1).$$

遇到“所有数同时加 x 后的 gcd”时，先把问题化为一个数与若干差分的 gcd。

- 若大量查询满足 $0 \leq x \leq N$ 且另一个操作数可很大，02J 将 x 拆成至多三个“ $\leq \sqrt{N}$ 或素数”的因子块；小块 gcd 查表、素数块只需一次取模判断。预处理约 $O(N)$ ，只适合固定值域的高频查询，普通 gcd 直接用标准库更短且常数更小。
- 令 $d = \gcd(a, m)$ 。在模 m 的所有 x 中，恰有 $\varphi(m/d)$ 个满足

$$\gcd(a + x, m) = d.$$

精确条件是 $x = dt$ 且 $\gcd(a/d + t, m/d) = 1$ ；不能把它误写成简单的 $\gcd(x, m) = d$ 。

- 若 $a \equiv b \pmod{m}$ ，则对任何整数 k 有 $a + km \equiv b \pmod{m}$ 。周期题要先确认外层操作是否也只依赖这一余数。

2.3 两个常用存在性结论

- 鸽巢原理：把 N 个物体放入 M 个盒子，至少有一个盒子不少于 $\lceil N/M \rceil$ 个；若每盒至多 k 个，则总数至多 Mk 。
- Frobenius（麦乐鸡）定理：若互素正整数 $a, b > 1$ ，则不能写成 $xa + yb$ ($x, y \geq 0$) 的最大整数为 $ab - a - b$ ，所有大于它的整数均可表示。若 $\gcd(a, b) > 1$ ，先注意到非该 gcd 倍数的数永远不可表示。
- 强哥德巴赫命题“每个大于 2 的偶数是两个素数之和”至今不是可直接引用的定理。竞赛构造题应在给定范围内筛素数并枚举验证；若题目明确提供相关条件，再按题意使用。

3. 组合计数：先区分球和盒

设有 N 个球、 M 个盒。盒“不同”即有标号，球“不同”即每个球可区分。

球	盒	限制	答案或模型
相同	不同	每盒非空	$\binom{N-1}{M-1}$ ，要求 $N \geq M \geq 1$
相同	不同	允许空	$\binom{N+M-1}{M-1}$
不同	不同	允许空	M^N
不同	不同	每盒至多一个	$A(M, N)$
不同	不同	每盒非空	$M! S(N, M)$
不同	相同	恰好 M 个非空盒	第二类 Stirling 数 $S(N, M)$
不同	相同	至多 M 个非空盒	$\sum_{j=0}^{\min(N, M)} S(N, j)$
相同	相同	至多 M 个非空盒	整数分拆，生成函数 $\prod_{i=1}^M (1 - x^i)^{-1}$
相同	相同	恰好 M 个非空盒	等价于把 $N - M$ 分拆为至多 M 份

前两行是隔板法；只有盒有标号时才能直接用组合数。最后两行不是简单的隔板法，应使用整数分拆 DP 或 03J 的相关思想。

3.1 组合数、Lucas 与常用恒等式

- 当模数 p 为质数且 $0 \leq n < p$ 时，

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \pmod{p}.$$

03 预处理阶乘和逆阶乘后单次 $O(1)$ 。若 $n \geq p$ ，阶乘含因子 p ，不可直接套用。

- Lucas 定理：把 n, k 写成 p 进制，则

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}.$$

它要求 p 为质数，且 03A 需开到 p 的表，只适合小质数模。复合模组合数使用 02BAAB。

- Pascal、提取与二项式定理：

$$\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}, \quad k \binom{n}{k} = n \binom{n-1}{k-1}, \quad (x+y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}.$$

- 常用求和：

$$\sum_{k=0}^n \binom{n}{k} = 2^n, \quad \sum_{k=0}^n (-1)^k \binom{n}{k} = [n=0],$$

$$\sum_{k=0}^n k \binom{n}{k} = n2^{n-1}, \quad \sum_{k=0}^n k^2 \binom{n}{k} = n(n+1)2^{n-2}, \quad \sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}.$$

交错调和恒等式是 $\sum_{k=1}^n (-1)^{k+1} \binom{n}{k} / k = H_n$ ；没有交错号时不能简化为 H_n 。

- 范德蒙德卷积与曲棍球杆：

$$\sum_i \binom{n}{i} \binom{m}{k-i} = \binom{n+m}{k}, \quad \sum_{i=0}^r \binom{s+i}{i} = \binom{s+r+1}{r}.$$

- 二项式反演：

$$f_n = \sum_{i=0}^n \binom{n}{i} g_i \iff g_n = \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} f_i.$$

若出现“选出一部分后再选一部分”，先尝试 $\binom{n}{k} \binom{k}{r} = \binom{n}{r} \binom{n-r}{k-r}$ 。

- 拉格朗日恒等式（实数或复数向量）：

$$\sum_{i < j} (a_i b_j - a_j b_i)^2 = \left(\sum_i a_i^2 \right) \left(\sum_i b_i^2 \right) - \left(\sum_i a_i b_i \right)^2.$$

它常用于把两两叉积平方和化为三组前缀和；右侧前两个求和必须带平方。

3.2 有上下界的非负整数解

非负整数 $x_1, \dots, x_n$ 满足

$$x_1 + \dots + x_n = m, \quad 0 \leq x_i < k$$

的方案数为

$$\sum_{j=0}^{\min(n, \lfloor m/k \rfloor)} (-1)^j \binom{n}{j} \binom{m - jk + n - 1}{n - 1}.$$

思路是先用隔板法，再把违反 $x_i < k$ 的变量减去 k 后容斥。组合数越界视为 0。模意义下还要满足所用组合数模板的模数前提。

3.3 Catalan、Stirling、错排与对称计数

- Catalan 数

$$\text{Cat}_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}, \quad \text{Cat}_n = \text{Cat}_{n-1} \frac{4n-2}{n+1}.$$

它计数合法括号、栈排列、含 n 个结点的有根二叉树、凸 $(n+2)$ 边形三角剖分。格点路径从 $(0,0)$ 到 (n,n) 且不越过对角线为 Cat_n ；除端点外不触对角线的双侧路径共有 2Cat_{n-1} 。对应 03D。

- 第二类 Stirling 数 $S(n, k)$ 是把 n 个不同元素划分成 k 个非空无标号集合：

$$S(n, k) = S(n-1, k-1) + kS(n-1, k).$$

第一类无符号 Stirling 数 $c(n, k)$ 是有 k 个环的排列：

$$c(n, k) = c(n-1, k-1) + (n-1)c(n-1, k).$$

对应 03E。需要固定 n 的整行时，03EA 使用

$$S(n, k) = \sum_{i=0}^k \frac{i^n (-1)^{k-i}}{i! (k-i)!}$$

做一次卷积，复杂度 $O(n \log n)$ ；无符号第一类数是多项式 $\prod_{i=0}^{n-1} (x+i)$ 的系数，03EA 用乘积树和 NTT，复杂度 $O(n \log^2 n)$ 。两者都要求 NTT 友好质数且 $n < p$ ；小规模或只查少量项仍用 03E 的 $O(n^2)$ 递推。

- 错排数 $D(n) = (n-1)(D(n-1) + D(n-2))$ ；Bell 数为所有集合划分总数。对应 03F。Bell 数整表的指数型生成函数为

$$\sum_{n \geq 0} B_n \frac{x^n}{n!} = \exp(\exp(x) - 1)$$

03FA 用形式幂级数 $\exp$ 在 $O(n \log n)$ 内求出整表；小规模使用 03F 的线性/二次递推更省常数。

- Burnside：不同等价类数为所有群元素不动点数之和除以群大小。先列清旋转、翻转等变换如何作用，再用 03L。
- 当模数是 NTT 友好质数且 $n < p$ 时，03M 将 $n!$ 分成长度约 $\sqrt{n}$ 的连续块，把块积看作多项式并多点求值，复杂度为 $O(\sqrt{n} \log^2 n)$ 。小 n 的普通循环常数更小； $n \geq p$ 时阶乘模 p 为 0。

4. 容斥、莫比乌斯反演与 Dirichlet 卷积

4.1 容斥

若事件为 $d_i \mid x$ ，交集对应 lcm：

$$\left| \bigcup_i A_i \right| = \sum_{\emptyset \neq S} (-1)^{|S|+1} \left\lfloor \frac{n}{\text{lcm}(d_i : i \in S)} \right\rfloor.$$

因子不必互素，但重复因子及被更小因子整除的因子应先删去。04A 是 bitmask 枚举；04B 用 DFS 和 $\text{lcm} > n$ 剪枝，最坏复杂度仍为指数级。

4.2 莫比乌斯函数与两个方向的反演

$$\mu(1) = 1, \quad \mu(n) = \begin{cases} 0, & p^2 \mid n \text{ 对某个质数 } p, \\ (-1)^{\omega(n)}, & \text{否则.} \end{cases} \quad \sum_{d \mid n} \mu(d) = [n=1].$$

- 约数和方向：

$$F(n) = \sum_{d \mid n} f(d) \iff f(n) = \sum_{d \mid n} \mu(d) F(n/d).$$

- 倍数和方向：

$$G(n) = \sum_{n \mid d} f(d) \iff f(n) = \sum_{k \geq 1} \mu(k) G(nk).$$

常用互质指示函数：

$$[\gcd(a, b) = 1] = \sum_{d \mid \gcd(a, b)} \mu(d).$$

例如矩形内有序互质点对数为

$$\sum_{d=1}^{\min(n, m)} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor.$$

普通前缀 $M(n) = \sum_{i \leq n} \mu(i)$ 用 01D；权为 $\mu(i)i^2$ 时用独立的 01DB。约数和、倍数和的 zeta 与反演见 01DA。

4.3 Dirichlet 卷积

$$(f * g)(n) = \sum_{d|n} f(d)g(n/d).$$

令 $\varepsilon(1) = 1, \varepsilon(n > 1) = 0$, $\mathbf{1}(n) = 1$, $\text{id}(n) = n$, 则

$$\mathbf{1} * \mu = \varepsilon, \quad \varphi * \mathbf{1} = \text{id}, \quad \mu * \text{id} = \varphi.$$

若要取 Dirichlet 逆, 必须先确认 $f(1)$ 在系数环中可逆。对应 01DC。

5. 排列、插值、线性递推与常用数列

5.1 康托展开

排列 $p_0, \dots, p_{n-1}$ 的 1 起始排名为

$$1 + \sum_i c_i(n-1-i)!,$$

其中 c_i 是右侧比 p_i 小的元素数。

- 03B 直接统计, $O(n^2)$, 可精确保留至 $20!$, 适合小 n 、调试和反排名;
- 03C 用树状数组, $O(n \log n)$, 适合大 n 的模意义排名。

两者是不同范围的实现, 应同时保留。

5.2 插值、递推与分拆

- 03G 是任意横坐标的 $O(n^2)$ 拉格朗日插值; 03H 仅适合横坐标连续为 $0, \dots, n-1$ 的场景。二者都要求分母可逆。
- Newton 前向级数为

$$f(x) = \sum_{k \geq 0} \Delta^k f(0) \binom{x}{k}.$$

03I 适合需要显式有限差分系数的场景。

- 整数分拆 $p(n)$ 可用 Euler 五边形数 $k(3k \pm 1)/2$ 递推, 见 03J。需要整表且 n 较大时, 03JA 使用

$$\prod_{i \geq 1} (1 - x^i)^{-1} = \exp \left(\sum_{k \geq 1} \frac{\sigma(k)}{k} x^k \right)$$

和 NTT 在 $O(n \log n)$ 内求出 $p(0..n)$; 小规模仍优先 03J。

- BM 从数列前缀恢复最短线性递推, 通常至少准备约两倍递推阶的数据; 03KA 再以 $O(k^2 \log n)$ 求第 n 项。
- 已知 k 阶齐次递推 $a_n = \sum_{j=1}^k c_j a_{n-j}$ 时, 令

$$C(x) = 1 - \sum_{j=1}^k c_j x^j, \quad A(x) = \sum_{i=0}^{k-1} a_i x^i.$$

则 $a_N = [x^N](A(x)C(x) \bmod x^k)/C(x)$; 可接 08K 的 Bostan-Mori。它与 03KA 的 Kitamasa 是同一问题的两种入口, 前者在已有生成函数或快速卷积时更自然。

5.3 Fibonacci 与约瑟夫环

取 $F_0 = 0, F_1 = 1$ 。对 $n \geq 1, m \geq 0$, 有

$$F_{n+m} = F_n F_{m+1} + F_{n-1} F_m, \quad F_{n-1} F_{n+1} - F_n^2 = (-1)^n.$$

常用结论:

$$F_n^2 + F_{n+1}^2 = F_{2n+1}, \quad \sum_{i=1}^n F_i^2 = F_n F_{n+1}, \quad \gcd(F_a, F_b) = F_{\gcd(a,b)}.$$

此外,

$$\sum_{i=1}^n F_{2i-1} = F_{2n}, \quad \sum_{i=1}^n F_{2i} = F_{2n+1} - 1.$$

对正整数 a, b , $F_a \mid F_b$ 当且仅当 $a \mid b$ 。若 $p \neq 5$ 为奇质数, 则 $F_{p-(5/p)} \equiv 0 \pmod{p}$, 其中 $(5/p)$ 是 Legendre 符号。任何正整数可唯一表示为若干个不相邻 Fibonacci 数之和 (Zeckendorf), 可从大到小贪心。

13A 的快速倍增一次返回 (F_n, F_{n+1}) , 适用于任意支持所需环运算的类型。约瑟夫环的 0 起始递推为

$$J(1, k) = 0, \quad J(n, k) = (J(n-1, k) + k) \bmod n.$$

用 13B 时先统一一下标是从 0 还是从 1 开始。

6. 按位运算、构造与子集变换

6.1 基础恒等式

$$x + y = (x \oplus y) + 2(x \& y) = (x \mid y) + (x \& y), \quad x + y \equiv x \oplus y \pmod{2}.$$

- $\bigvee_i (X \& a_i) = X \& (\bigvee_i a_i)$ 。它等于 X 的前提是 X 的每一个 1 位都被某个 a_i 覆盖。
- 两个非负数满足 $a + b = X$, $a \oplus b = Y$ 当且仅当

$$X \geq Y, \quad X - Y \text{ 为偶数}, \quad \left(\frac{X - Y}{2} \right) \& Y = 0.$$

此时令 $t = (X - Y)/2$, 可取 $a = t, b = t + Y$ 。第三个按位无进位条件不可省略。

- 三个非负数满足 $a + b + c = X$, $a \oplus b \oplus c = Y$ 时, 充要条件仅为 $X \geq Y$ 且同奇偶; 令 $t = (X - Y)/2$, 取 $(a, b, c) = (t, t, Y)$ 即可。
- 用恰好 k 个 2 的幂表示正整数 n 的充要条件为

$$\text{popcount}(n) \leq k \leq n.$$

从高位把一个 2^i 拆成两个 2^{i-1} , 每次把项数加一, 即可构造。对应 12A。

6.2 FWT、SOS 与异或线性基

- FWT 的卷积下标分别是 $i \mid j, i \& j, i \oplus j$; 普通下标相加卷积仍用 NTT / FFT。XOR 逆变换最后除以长度, 故变换长度必须在当前系数环中可逆。对应 08G。
- SOS 子集 zeta: $F[S] = \sum_{T \subseteq S} f[T]$; 超集 zeta: $F[S] = \sum_{T \supseteq S} f[T]$ 。子集卷积为

$$c[S] = \sum_{T \subseteq S} a[T] b[S \setminus T].$$

三者不能混用, 见 08H、08HA。

- 单个掩码枚举子掩码为 $2^{\text{popcount}(S)}$, 所有掩码合计为 3^{bits} 。异或线性基解决最大异或、可表示性与独立基, 见 12B; GF(2) 方程组则应使用 05G。

7. 矩阵、卷积与线性代数的前提

- 05 的普通矩阵乘法在代数环上工作; 05A 将加法换为 min、乘法换为加法, 适合固定边数最短路。
- 05B 是实数高斯消元, 需按量级选择误差阈值; 05C 的对称 LDLT 不交换行列, 要求每一步主元非零, 不能替代一般消元。
- 05D、05E 的模消元要求系数所在模数为质数 (或更一般地说, 所除主元可逆)。复合模和整数方程不能直接搬用。
- 08 / 08I NTT 都要求变换长度整除 $P-1$ 。08 是易读通用版; 08I 使用 u32 Montgomery 与 radix-4, 要求奇质数 $P < 2^{30}$, 适合卷积成为主要瓶颈时替换。
- 若变换长度 $n \mid P-1$ 但 n 不是 2 的幂, 使用 08P。它按 $n = rm$ 做 Cooley-Tukey 分解, 2 的幂子问题交给 08; 小因子或无法继续分解的大素因子回退朴素 DFT。因此 “长度能整除 $P-1$ ” 是必要条件, 不能只把普通 NTT 的补零长度直接改成任意数。
- 08E 的拆系数浮点卷积代码短, 但误差依赖长度、系数分布、编译器和浮点环境, 不能对全 32 位对抗数据作无条件正确性承诺。

- 08F / 08IA 使用三模 NTT。若每个输入先规范到 $[0, p)$ ，并满足三质数乘积严格大于 $\min(n, m)(p-1)^2$ ，CRT 代表元就是整数卷积真值，随后对 p 取模是精确的。Library Checker 的 `convolution_mod_1000000007` 含专门的 FFT killer，极限数据选 08IA。
- 08A、08B、08C 是同类不同环境的多项式实现，不可同时定义同名类型。
- 二维卷积先对每一行、再对每一列做 NTT；两维补齐长度都必须整除 $p-1$ ，对应 08J。它不能把普通一维卷积的线性下标直接复用，否则第二维会发生进位混叠。
- 对形式幂级数， $f[0] \neq 0$ 才可求逆； $\ln f$ 要求常数项为 1， $\exp f$ 要求常数项为 0。平方根还要求首个非零项次数为偶数、首系数为二次剩余。08N 会检查这些前提。
- 08N 的 $\text{ppow}(f, e, n)$ 适合普通非负整数 e ；指数读成十进制串时改用 08NA。若 $f = x^t c(1+h)$ 且 $t > 0$ ，先判断 $te \geq n$ ，成立则截断结果全为 0；否则用 $c^e \exp(e \ln(1+h))$ 并右移 te 位。字符串指数中 e 分别按模数和模数减一取余，不能直接读入 64 位整数。
- Bostan-Mori 通过 $g(x)$ 与 $g(-x)$ 相乘后只保留偶/奇项，把所求 $[x^n]f/g$ 的下标折半，适合超大 n ；对应 08K。08L 的点值平移要求所有出现的连续横坐标在模意义下可逆，当前接口限制 $m > n$ 且 $m+n < \text{mod}$ ，不要跨过模数直接调用。
- Power Projection 的目标是 $h(i) = [x^N]f(x)g(x)^i$ 。08O 保留了截断到 N 次的直接 NTT 迭代，适合 n, N 至少一侧较小；两侧都很大时使用 08OA。后者固定 998244353、 $n < \text{mod}$ 与 $N+1 \leq 2^{22}$ ，把二元消元和 NTT 排布合并为准线性分支，不能拿去替换任意模数版本。
- 拉格朗日-Bürmann：若 $T(x) = x\Phi(T(x))$ 、 $\Phi(0) \neq 0$ ，则对 $n \geq 1$ 有

$$[x^n]H(T(x)) = \frac{1}{n} [t^{n-1}]H'(t)\Phi(t)^n.$$

特别地 $[x^n]T(x)^k = \frac{k}{n} [t^{n-k}]\Phi(t)^n$ 。移项后的 $T(x) = a + x\Phi(T(x))$ 先令 $U = T - a$ ；模意义下所有出现的分母必须可逆。

- 单位根筛项：若域中有本原 m 次根 ω 且 m 可逆，则

$$\sum_{q \geq 0} a_{qm+r} x^{qm+r} = \frac{1}{m} \sum_{j=0}^{m-1} \omega^{-rj} A(\omega^j x).$$

这可筛出次数 $\equiv r \pmod{m}$ 的项； $m=2$ 即偶、奇项拆分。模质数时通常要求 $m \mid p-1$ ，否则不能凭空构造所需单位根。

- 稀疏分母 $f(x)$ 的幂级数除法可按系数递推，复杂度 $O(nk)$ ，其中 k 是非零项数；分母不稀疏时改用 08B 的 NTT 求逆。对应 08M。
- Kirchhoff 矩阵树定理：无向图 Laplacian 的任意余子式行列式等于生成树数；平行边权累加，自环不贡献。对应 05H。
- 单纯形 05F 只处理连续变量的 $\max c \cdot x, Ax \leq b, x \geq 0$ ；整数规划需要额外建模。

8. 博弈论与数值计算

8.1 Nim 与 SG

以下结论只用于有限、无环、无偏游戏，并采用正常规则（无路可走者负）。

- P 态为后手必胜，N 态为先手必胜；有 P 后继的状态为 N，全部后继为 N 的状态为 P。
- Nim 的所有堆异或和为 0 当且仅当为 P 态。异或和为 $X \neq 0$ 时，选择一堆 a 变成 $a \oplus X < a$ 。
- 反常 Nim 只有所有非空堆均为 1 时例外：此时偶数堆先手胜；否则仍按普通 Nim 判断。
- $\text{sg}(x) = \text{mex}\{\text{sg}(y) : x \rightarrow y\}$ 。独立子游戏的 SG 值异或，异或为 0 即 P 态。拆分操作的后继值是各子局面的 SG 异或。
- Wythoff Nim 的必败态为 $(\lfloor k\phi \rfloor, \lfloor k\phi \rfloor + k)$ ，见 09A；极大整数范围要避免用浮点直接比较边界。
- Bash 博弈每次取 $1..m$ 个时， n 是 $m+1$ 的倍数恰为 P 态；每次取 $[a, b]$ 时必须先明确“剩余不足 a 个”如何判胜，不能机械套同一余数式。
- Moore Nim 每次至多动 k 堆：把各堆逐位相加，所有位计数均为 $k+1$ 的倍数时为 P 态。阶梯 Nim 只异或奇数级台阶上的石子。
- Anti-SG：若所有子局面 SG 均不超过 1，总异或为 0 时先手胜；若至少一个 SG 大于 1，总异或非 0 时先手胜。它与反常 Nim 的例外同源。
- Lasker Nim（可取石子或把一堆拆成两堆）的单堆 SG： $x \bmod 4=0$ 时为 $x-1$ ， $=3$ 时为 $x+1$ ，其余为 x 。
- 树上删边游戏的根状态为 $\text{xor}(\text{sg}(\text{child})+1)$ ；无向图删边可用 Fusion Principle 把奇环缩成“一点一边”、偶环缩成一点，再化为树上模型。

- Fibonacci Nim (首次不能全取, 以后至多取上次两倍) 当且仅当初始堆大小是 Fibonacci 数时为 P 态; 一般制胜取法来自 Zeckendorf 分解的最小项。

8.2 数值方法

- 牛顿迭代 $x \leftarrow x - f(x)/f'(x)$ 在根附近且导数非零时收敛很快; 初值差、重根或导数近零时应回退到二分。对应 11A、11D。
- 自适应辛普森适合平滑函数的定积分; 间断、奇点和尖峰先手动分段, 误差参数是近似控制而非无条件的严格全局界。对应 11B。
- 三分仅用于单峰或单谷目标; 整数三分结束后必须枚举剩余区间。对应 11C。

9. 典型例题的切入方式

题型	第一步与结论	常用模板
求 $x_i \in [0, k)$ 且和为 m 的方案数	隔板法计无上界方案, 再对 $x_i \geq k$ 容斥, 使用第 3.2 节公式	03、04A / 04B
把正整数拆成恰好 k 个 2 的幂	先验 $\text{popcount}(n) \leq k \leq n$, 再逐位拆分	12A
最少加减多少使 x 被 k 整除	减去 $x \bmod k$; 加上 $(k - x \bmod k) \bmod k$	02G / 基础取模
删一个数使等和划分不可能	若原数组可等分, 取 v_2 最小的正元素删掉; 其余数都多含一个 2 因子, 无法再平分	12A、子集和 DP
大数能否写为两个给定互素面额的非负组合	先用 Frobenius 阈值判必然可行; 小值再枚举或 DP	02BA、完全背包
按素数和构造分组	给定范围内筛素数并实际验证分解; 不以强哥德巴赫猜想作为证明	01 / 01F
矩形内互质点对、gcd 的和	把互质条件改写成 μ 的约数和, 再做整除分块	01D、01DA、01DB
合法括号、栈排列、不过对角线格点路	检查是否是 Catalan 模型, 再注意模除法前提	03D
对称旋转、翻转后视为相同的染色方案	逐个群元素数不动点, 再 Burnside 平均	03L
高阶递推的巨大下标项	若递推已知用矩阵或 Kitamasa; 若未知先 BM 恢复	05、03K、03KA

这些结论的作用是缩短建模, 不替代边界检查: 组合数越界取 0、模除法先验可逆、位运算构造先验非负, 以及所有计数对象是否带标号, 都是最常见的失分点。

10. 几何公式与退化情形

- 二维叉积 $\text{cross}(b-a, c-a)$ 的符号判断转向, 绝对值是平行四边形面积; 多边形有向面积的两倍为 $\sum \text{cross}(p[i], p[i+1])$ 。
- 单调链凸包先排序去重。是否保留共线边界点必须在弹栈条件中统一; 旋转卡壳要求凸包方向一致且不重复首点。
- 两个凸多边形的闵可夫斯基和可按极角归并边向量; 点集未成凸包时先做凸包。最近点对分治在按横坐标切分后按纵坐标归并, 条带中每点只需检查常数个后继点。
- 半平面交中每条有向直线保留左侧; 平行同向直线只保留更严格者, 反向冲突可能直接为空。
- 圆的交点先比较圆心距与 r_1+r_2 、 $|r_1-r_2|$; 重合圆有无穷多个交点, 不能用“返回空”与不相交混为一谈。
- 三维 $\text{dot}(b-a, c-a)=0$ 表示垂直关系, $\text{dot}(b-a, \text{cross}(c-a, d-a))$ 是有向六倍四面体体积; 所有距离公式的分母都要先排除退化直线或零法向量。
- 整数坐标的叉积、平方距离可能超过 64 位时改用 i128; 浮点几何则按题目尺度设置 EPS, 不要直接沿用整数的 $==0$ 。

11. 大范围质数和与 Min_25

01J 对所有不同的 $\text{floor}(n/i)$ 建立状态, 先令 $g(x)=\sum_{k=2}^x k$, 再依次删去最小质因子为当前质数的合数, 最终得到 $\sum_{p \leq x} p$ 。状态数只有 $O(\sqrt{n})$ 。精确求和默认用 i128, 因为初始三角和可能远大于最终质数和; 模意义下直接把 T 换成 MInt。

12. 反射原理与幂式比较补遗

- 从 $(0, 0)$ 每步走到 $(x+1, y \pm 1)$, 走 a 步到 (a, b) 的方案数是 $C(a, (a+b)/2)$; 奇偶性不符或组合数下标越界时为 0。

- 禁止碰到水平线 $y=k$ 时，把首次碰线后的路径反射，得到“总方案减去到镜像终点的方案”。同时限制上下两条线时可继续递归反射或用容斥/生成函数，不能只减一次。
- 比较形如 $x^y z$ 的正数时，不直接求幂。先比较其对数 $y^z \log(x)$ ；若这一层仍溢出，再比较 $z \log(y) + \log(\log(x))$ 。底数为 0/1、对数非正或指数为 0 时必须单独分类。

13. 伯努利数与幂和

采用本库约定 $B_{-1} = -1/2$ 。模板 14 用 $O(d^2)$ 递推预处理并直接求幂和；当只需要大阶数的整列伯努利数时，14A 由

$$\frac{x}{e^x - 1} = \left(\sum_{i \geq 0} \frac{x^i}{(i+1)!} \right)^{-1}$$

通过 NTT 求逆，在 $O(n \log n)$ 内得到 $B_0..B_n$ 。模意义下除以 $k+1$ 前必须确认其可逆。

$$\sum_{j=0}^m \binom{m+1}{j} B_j = 0 \quad (m \geq 1), \quad B_m(x) = \sum_{j=0}^m \binom{m}{j} B_j x^{m-j}.$$

因此（本库采用 $B_{-1} = -1/2$ ，所以减去的是 $B_{\{k+1\}}(1)$ ）

$$\sum_{i=1}^n i^k = \frac{B_{k+1}(n+1) - B_{k+1}(1)}{k+1}.$$

模板构造时会多算 $B_{\{d+1\}}$ ； $n=0$ 的前缀和为 0，区间接口是从 1 开始的闭区间。合数模下若 $k+1$ 不可逆，应改用整数公式、CRT 或分块。

01 - 素数（线性筛）.cpp

```

1  /*
2  用途：一次预处理 1..n 的最小质因子、质数表、欧拉函数和莫比乌斯函数。
3  复杂度：预处理  $O(n)$ ，空间  $O(n)$ 。适合  $n$  能整体预处理（通常  $\leq 1e7$ ）的场景。
4  接口：lp[x]、pri、phi[x]、mu[x]、ask(x)、factor(x)。
5  若  $n$  很大而查询很少：改用 01F - Miller-Rabin + 01FA - Pollard-Rho，不要开大筛。
6  */
7  struct Sieve {
8      int n = 0;
9      vector<int> pri, lp, phi, mu;
10
11      Sieve() = default;
12      explicit Sieve(int n) {
13          init(n);
14      }
15
16      void init(int _n) {
17          assert(_n >= 0);
18          n = _n;
19          pri.clear();
20          lp.assign(n + 1, 0);
21          phi.assign(n + 1, 0);
22          mu.assign(n + 1, 0);
23          if (n >= 1) {
24              lp[1] = phi[1] = mu[1] = 1;
25          }
26          for (int x = 2; x <= n; ++x) {
27              if (lp[x] == 0) {
28                  lp[x] = x;
29                  pri.push_back(x);
30                  phi[x] = x - 1;
31                  mu[x] = -1;
32              }
33              for (int p : pri) {
34                  if (p > n / x) break; // 避免 p * x 溢出。
35                  int y = p * x;
36                  lp[y] = p;
37                  if (x % p == 0) {
38                      phi[y] = phi[x] * p;
39                      mu[y] = 0;
40                      break;
41                  }
42                  phi[y] = phi[x] * (p - 1);
43                  mu[y] = -mu[x];
44              }
45          }
46      }

```

```

47
48     bool ask(int x) const {
49         return 2 <= x && x <= n && lp[x] == x;
50     }
51
52     vector<pair<int, int>> factor(int x) const {
53         assert(1 <= x && x <= n);
54         vector<pair<int, int>> res;
55         while (x > 1) {
56             int p = lp[x], e = 0;
57             do {
58                 x /= p;
59                 ++e;
60             } while (x % p == 0);
61             res.push_back({p, e});
62         }
63         return res;
64     }
65 };

```

01A - 质因数分解（最小质因子）.cpp

```

1  /*
2  用途：使用最小质因子表分解小范围整数。
3  前提：先构造 LinearSieve sieve(maxValue)，且 1 <= n <= sieve.limit。
4  复杂度：O(log n)，适合大量、范围已知的查询；大整数请使用 01FA - Pollard-Rho。
5  */
6  inline vector<pair<int, int>> factor(int n, const vector<int>& lp) {
7      assert(1 <= n && n < lp.size());
8      vector<pair<int, int>> res;
9      while (n > 1) {
10         int p = lp[n], e = 0;
11         do {
12             n /= p;
13             ++e;
14         } while (n % p == 0);
15         res.push_back({p, e});
16     }
17     return res;
18 }

```

01B - 欧拉函数（单点试除）.cpp

```

1  /*
2  用途：计算单个 phi(n)。
3  复杂度：O(sqrt n)，无须预处理；需要整张 phi 表时使用 01C，需要 minPrime / mu 等更多信息时使用 01 - 素数（线性筛）。
4  */
5
6  inline int phi(int n) {
7      assert(n >= 1);
8      int res = n;
9      for (int p = 2; p <= n / p; ++p) {
10         if (n % p != 0) continue;
11         while (n % p == 0) n /= p;
12         res = res / p * (p - 1);
13     }
14     if (n > 1) res = res / n * (n - 1);
15     return res;
16 }

```

01C - 欧拉函数（整表筛）.cpp

```

1  /*
2  用途：预处理 phi(1..n)。
3  复杂度：O(n log log n)；若同时还需 minPrime / mu，使用 01 - 素数（线性筛）的 LinearSieve::phi (O(n))。

```



```

4  */
5  inline vector<int> phiAll(int n) {
6      assert(n >= 0);
7      vector<int> phi(n + 1);
8      iota(phi.begin(), phi.end(), 0);
9      if (n >= 1) phi[1] = 1;
10     for (int p = 2; p <= n; ++p) {
11         if (phi[p] != p) continue;
12         for (int x = p; x <= n; x += p) phi[x] = phi[x] / p * (p - 1);
13     }
14     return phi;
15 }

```

01D - 莫比乌斯函数 (Mertens、杜教筛) .cpp

```

1  /*
2  用途: 杜教筛求  $M(n) = \sum_{i \leq n} \mu(i)$ 。
3  依赖: 01 - 素数 (线性筛 / Sieve)。预处理范围 lim 内  $O(1)$ , 超过后按整除分块递归并记忆化。
4  通常取 lim 约  $n^{2/3}$  或按内存设置; 若只需  $\mu[1..N]$ , 直接使用 Sieve::mu。
5  */
6
7  struct Mertens {
8      Sieve sv;
9      vector<int> pre;
10     unordered_map<int, int> mp;
11
12     Mertens() = default;
13     explicit Mertens(int lim) { init(lim); }
14
15     void init(int lim) {
16         assert(lim >= 1);
17         sv.init(lim);
18         pre.assign(lim + 1, 0);
19         for (int i = 1; i <= lim; ++i) pre[i] = pre[i - 1] + sv.mu[i];
20         mp.clear();
21     }
22
23     int ask(int n) {
24         assert(n >= 0 && sv.n > 0);
25         if (n <= sv.n) return pre[n];
26         if (auto it = mp.find(n); it != mp.end()) return it->second;
27         int ans = 1;
28         for (int l = 2, r; l <= n; l = r + 1) {
29             int q = n / l;
30             r = n / q;
31             ans -= (r - l + 1) * ask(q);
32         }
33         return mp[n] = ans;
34     }
35 };

```

01DA - 莫比乌斯反演 (约数和变换) .cpp

```

1  /*
2  莫比乌斯反演的两种常见方向 (下标从 1 开始, mu 取 LinearSieve::mu):
3   $F(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d) F(n/d)$ ;
4   $G(n) = \sum_{n|d} f(d) \iff f(n) = \sum_{k \geq 1} \mu(k) G(nk)$ 。
5  以下函数都是  $O(N \log N)$ , 用于把公式直接落实为数组变换。
6  */
7  template <class T> vector<T> zetaDiv(const vector<T>& f) {
8      int n = f.size() - 1;
9      assert(n >= 0);
10     vector<T> res(n + 1, T(0));
11     for (int d = 1; d <= n; ++d)
12         for (int x = d; x <= n; x += d) res[x] += f[d];
13     return res;
14 }
15
16 template <class T> vector<T> mobDiv(const vector<T>& F, const vector<int>& mu) {
17     int n = F.size() - 1;
18     assert(mu.size() > n);
19     vector<T> res(n + 1, T(0));
20     for (int d = 1; d <= n; ++d) {
21         if (mu[d] == 0) continue;
22         for (int x = d; x <= n; x += d) res[x] += T(mu[d]) * F[x / d];
23     }
24     return res;
25 }

```

```

26
27 template <class T> vector<T> zetaMul(const vector<T>& f) {
28     int n = f.size() - 1;
29     assert(n >= 0);
30     vector<T> res(n + 1, T(0));
31     for (int d = 1; d <= n; ++d)
32         for (int x = d; x <= n; x += d) res[d] += f[x];
33     return res;
34 }
35
36 template <class T> vector<T> mobMul(const vector<T>& G, const vector<int>& mu) {
37     int n = G.size() - 1;
38     assert(mu.size() > n);
39     vector<T> res(n + 1, T(0));
40     for (int k = 1; k <= n; ++k) {
41         if (mu[k] == 0) continue;
42         for (int d = 1; d * k <= n; ++d) res[d] += T(mu[k]) * G[d * k];
43     }
44     return res;
45 }

```

01DB - 加权莫比乌斯函数 (d^2) .cpp

```

1  /*
2  用途：杜教筛求  $S(n)=\sum_{d \leq n} \mu(d) \cdot d^2$ （模质数）。
3  这是 01D 的加权版本，不能用普通  $M(n)$  替代；常用于带  $d^2$  权值的整除分块式。
4  依赖：02A - 自动取模 (MInt)。Mint 的模数应为大于 3 的质数。
5  */
6
7  template <class Z> struct Mobius2 {
8      int n = 0;
9      vector<int> lp, pri, mu;
10     vector<Z> pre;
11     unordered_map<int, Z> mp;
12
13     Mobius2() = default;
14     explicit Mobius2(int lim) { init(lim); }
15
16     void init(int lim) {
17         assert(lim >= 1);
18         n = lim;
19         lp.assign(n + 1, 0);
20         mu.assign(n + 1, 0);
21         pri.clear();
22         pre.assign(n + 1, Z(0));
23         lp[1] = 1;
24         mu[1] = 1;
25         for (int x = 2; x <= n; ++x) {
26             if (lp[x] == 0) lp[x] = x, mu[x] = -1, pri.push_back(x);
27             for (int p : pri) {
28                 if (p > n / x) break;
29                 int y = p * x;
30                 lp[y] = p;
31                 if (x % p == 0) {
32                     mu[y] = 0;
33                     break;
34                 }
35                 mu[y] = -mu[x];
36             }
37         }
38         for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] + Z(mu[i]) * Z(i) * i;
39         mp.clear();
40     }
41
42     static Z sum2(int n) {
43         Z x = Z(n);
44         return x * (x + 1) * (x * 2 + 1) / Z(6);
45     }
46
47     Z ask(int x) {

```

```

48     assert(x >= 0 && n > 0);
49     if (x <= n) return pre[x];
50     if (auto it = mp.find(x); it != mp.end()) return it->second;
51     Z ans = 1;
52     for (int l = 2, r; l <= x; l = r + 1) {
53         int q = x / l;
54         r = x / q;
55         ans -= (sum2(r) - sum2(l - 1)) * ask(q);
56     }
57     return mp[x] = ans;
58 }
59 };

```

01DC - 狄利克雷卷积与逆.cpp

```

1  /*
2  用途：在下标 1..N 上计算 Dirichlet convolution:
3  (f*g)(n)=sum_{d|n} f(d)g(n/d), 并求 f 的 Dirichlet 逆。
4  前提：数组下标 0 被忽略；求逆要求 f(1) 可逆。复杂度 O(N log N)。
5  关系：与 01DA 的约数 zeta / Möbius 变换配套使用。
6  */
7  namespace Dirichlet {
8  template <class T> vector<T> mul(const vector<T>& l, const vector<T>& r) {
9      int n = min(l.size(), r.size()) - 1;
10     assert(n >= 0);
11     vector<T> ans(n + 1, T(0));
12     for (int d = 1; d <= n; ++d) {
13         for (int k = d; k <= n; k += d) {
14             ans[k] += l[d] * r[k / d];
15         }
16     }
17     return ans;
18 }
19
20 template <class Z> vector<Z> inverse(const vector<Z>& f) {
21     int n = f.size() - 1;
22     assert(n >= 1 && f[1] != Z(0));
23     vector<Z> ans(n + 1, Z(0));
24     ans[1] = f[1].inv();
25     vector<Z> sum(n + 1, Z(0));
26     for (int d = 2; d <= n; ++d) sum[d] += f[d] * ans[1];
27     for (int val = 2; val <= n; ++val) {
28         ans[val] = -sum[val] * ans[1];
29         for (int d = 2; val <= n / d; ++d) {
30             sum[val * d] += f[d] * ans[val];
31         }
32     }
33     return ans;
34 }
35 } // namespace Dirichlet

```

01EA - 检查是否是素数的幂（试除）.cpp

```

1  /*
2  用途：判断 n 是否可写成 p^k (p 为质数, k >= 1)。
3  复杂度：O(sqrt n)，无预处理；适合单次或少量查询。已筛范围内的大量查询使用 01EB。
4  */
5
6  inline bool ask(int n) {
7      if (n <= 1) return false;
8      for (int p = 2; p <= n / p; ++p) {
9          if (n % p != 0) continue;
10         do n /= p; while (n % p == 0);
11         return n == 1;
12     }
13     return true; // 没有真因子, n 本身为质数。
14 }

```

01EB - 检查是否是素数的幂（最小质因子）.cpp

```

1  /*
2  用途：判断 n 是否可写成  $p^k$  ( $p$  为质数,  $k \geq 1$ )。
3  前提：使用 01 - 素数（线性筛）得到 minPrime, 且 n 在筛范围内。
4  复杂度： $O(\log n)$ ；大量、范围已知的查询优于 01EA。
5  */
6  inline bool ask(int n, const vector<int>& lp) {
7      if (n <= 1 || n >= lp.size()) return false;
8      int p = lp[n];
9      if (p <= 1) return false;
10     do n /= p; while (n % p == 0);
11     return n == 1;
12 }

```

01F - Miller-Rabin 素性检验.cpp

```

1  /*
2  用途：64 位整数确定性素性检验。
3  复杂度： $O(\log n)$  次模乘；下列底数对 u64 范围确定正确。
4  环境：GCC/Clang 用 int 为  $O(1)$  模乘；其他编译器自动退化到  $O(\log \text{mod})$  加法模乘。
5  */
6
7  using u64 = uint64_t;
8  using u128 = __uint128_t;
9
10 namespace Miller {
11
12 inline u64 mul(u64 a, u64 b, u64 mod) {
13     assert(mod > 0);
14     a %= mod;
15     b %= mod;
16 #ifdef __SIZEOF_INT128__
17     return u128(a) * b % mod;
18 #else
19     u64 res = 0;
20     while (b > 0) {
21         if (b & 1) res = res >= mod - a ? res - (mod - a) : res + a;
22         a = a >= mod - a ? a - (mod - a) : a + a;
23         b >>= 1;
24     }
25     return res;
26 #endif
27 }
28
29 inline u64 power(u64 a, u64 e, u64 mod) {
30     u64 res = 1 % mod;
31     while (e > 0) {
32         if (e & 1) res = mul(res, a, mod);
33         a = mul(a, a, mod);
34         e >>= 1;
35     }
36     return res;
37 }
38
39 inline bool ask(u64 n) {
40     if (n < 2) return false;
41     for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
42         if (n % p == 0) return n == p;
43     }
44     u64 d = n - 1;
45     int s = 0;
46     while ((d & 1) == 0) d >>= 1, ++s;
47     for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}) {
48         if (a % n == 0) continue;
49         u64 x = power(a % n, d, n);
50         if (x == 1 || x == n - 1) continue;
51         bool ok = true;

```

```

52         for (int r = 1; r < s; ++r) {
53             x = mul(x, x, n);
54             if (x == n - 1) {
55                 ok = false;
56                 break;
57             }
58         }
59         if (ok) return false;
60     }
61     return true;
62 }
63 } // namespace Miller

```

01FA - Pollard-Rho 质因数分解.cpp

```

1  /*
2  用途: 分解 u64 范围的大整数。
3  依赖: 01F - Miller-Rabin 素性检验。
4  复杂度: 期望约  $O(n^{1/4})$ ; 适合 n 很大、查询不多, 不能替代小范围线性筛。
5  */
6
7  using u64 = uint64_t;
8
9  namespace Pollard {
10
11     inline u64 add(u64 a, u64 b, u64 mod) {
12         return a >= mod - b ? a - (mod - b) : a + b;
13     }
14
15     inline u64 next(u64 x, u64 c, u64 mod) {
16         return add(Miller::mul(x, x, mod), c, mod);
17     }
18
19     inline u64 rand(u64 l, u64 r) {
20         static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
21         return uniform_int_distribution<u64>(l, r)(rng);
22     }
23
24     inline u64 find(u64 n) {
25         if (n % 2 == 0) return 2;
26         if (n % 3 == 0) return 3;
27         while (true) {
28             u64 c = rand(1, n - 1);
29             u64 x = rand(0, n - 1), y = x, d = 1;
30             while (d == 1) {
31                 x = next(x, c, n);
32                 y = next(next(y, c, n), c, n);
33                 u64 dif = x >= y ? x - y : y - x;
34                 d = gcd(dif, n);
35             }
36             if (d != n) return d;
37         }
38     }
39
40     inline void dfs(u64 n, vector<u64>& fac) {
41         if (n == 1) return;
42         if (Miller::ask(n)) {
43             fac.push_back(n);
44             return;
45         }
46         u64 d = find(n);
47         dfs(d, fac);
48         dfs(n / d, fac);
49     }
50
51     inline vector<u64> factor(u64 n) {
52         assert(n >= 1);
53         vector<u64> fac;
54         dfs(n, fac);
55         sort(fac.begin(), fac.end());
56         return fac;
57     }
58 } // namespace Pollard

```

01G - 素性检验 (试除) .cpp

```

1  /*
2  用途：无预处理地判断单个小整数是否为质数。
3  复杂度：O(sqrt n)。与 01F 是同一问题的不同环境：小数、少量查询用本模板；int 大数用 Miller-Rabin。
4  */
5
6  inline bool ask(int x) {
7      if (x < 2) return false;
8      for (int d = 2; d <= x / d; ++d) {
9          if (x % d == 0) return false;
10     }
11     return true;
12 }

```

01H - 约数函数 (枚举、个数、和) .cpp

```

1  /*
2  用途：已知  $n = \prod p_i^{e_i}$  的质因数分解后，枚举全部约数并计算约数个数、约数和、欧拉函数。
3  依赖：分解可由 01A (小范围) 或 01FA (64 位) 得到。约数和以 i128 返回，避免中间溢出。
4  复杂度：枚举  $O(\tau(n))$ ，其余  $O(\text{质因子种数})$ 。
5  */
6
7  using i128 = __int128_t;
8
9  namespace Divisor {
10     using Factor = vector<pair<int, int>>;
11
12     inline void check(const Factor& fac) {
13         for (auto [pri, e] : fac) assert(pri > 1 && e >= 1);
14     }
15
16     inline vector<int> getDiv(const Factor& fac) {
17         check(fac);
18         vector<int> d{1};
19         for (auto [pri, e] : fac) {
20             int n = d.size();
21             int pw = 1;
22             for (int i = 1; i <= e; ++i) {
23                 assert(pw <= numeric_limits<int>::max() / pri);
24                 pw *= pri;
25                 for (int i = 0; i < n; ++i) {
26                     assert(d[i] <= numeric_limits<int>::max() / pw);
27                     d.push_back(d[i] * pw);
28                 }
29             }
30         }
31         sort(d.begin(), d.end());
32         return d;
33     }
34
35     inline int askCnt(const Factor& fac) {
36         check(fac);
37         int ans = 1;
38         for (auto [pri, e] : fac) {
39             assert(ans <= numeric_limits<int>::max() / (e + 1));
40             ans *= e + 1;
41         }
42         return ans;
43     }
44
45     inline i128 askSum(const Factor& fac) {
46         check(fac);
47         i128 ans = 1;
48         for (auto [pri, e] : fac) {
49             i128 pw = 1, sum = 1;
50             for (int i = 1; i <= e; ++i) {
51                 pw *= pri;
52                 sum += pw;
53             }
54             ans *= sum;
55         }
56         return ans;
57     }
58 }

```

```

59 inline int phi(const Factor& fac) {
60     check(fac);
61     i128 ans = 1;
62     for (auto [pri, e] : fac) {
63         i128 pk = 1;
64         for (int i = 1; i < e; ++i) pk *= pri;
65         ans *= pk * (pri - 1);
66     }
67     assert(ans <= numeric_limits<int>::max());
68     return ans;
69 }
70 } // namespace Divisor

```

011 - 质数计数 (Lehmer) .cpp

```

1  /*
2  用途: 计算  $\pi(n)$ , 即不超过  $n$  的质数个数。
3  复杂度: Lehmer prime counting, 单次通常远快于开到  $n$  的筛; 默认预处理到  $5e6$ , 适合  $n$  约  $1e13$  及以内。
4  前提: 若  $n$  更大,  $\text{init}(\text{limit})$  的  $\text{limit}$  至少应覆盖  $\sqrt{n}$ ; 大量小范围查询直接使用 01 - 线性筛。
5  */
6
7  using i128 = __int128_t;
8
9  using ld = long double;
10
11 struct Lehmer {
12     int lim = 0;
13     vector<int> pri, pc;
14     unordered_map<int, int> mp;
15
16     explicit Lehmer(int mx = 5'000'000) {
17         init(mx);
18     }
19
20     void init(int mx) {
21         assert(mx >= 2);
22         lim = mx;
23         vector<bool> vis(mx + 1, false);
24         pc.assign(mx + 1, 0);
25         pri.clear();
26         for (int x = 2; x <= mx; ++x) {
27             if (!vis[x]) pri.push_back(x);
28             for (int p : pri) {
29                 if (p > mx / x) break;
30                 vis[p * x] = true;
31                 if (x % p == 0) break;
32             }
33         }
34         for (int x = 1; x <= mx; ++x) pc[x] = pc[x - 1] + (!vis[x] && x >= 2);
35         mp.clear();
36     }
37
38     static int isqrt(int val) {
39         assert(val >= 0);
40         int rt = sqrtl(ld(val));
41         while (i128(rt + 1) * (rt + 1) <= val) ++rt;
42         while (i128(rt) * rt > val) --rt;
43         return rt;
44     }
45
46     static int icbrt(int val) {
47         assert(val >= 0);
48         int rt = cbrtl(ld(val));
49         while (i128(rt + 1) * (rt + 1) * (rt + 1) <= val) ++rt;
50         while (i128(rt) * rt * rt > val) --rt;
51         return rt;
52     }
53
54     int phi(int val, int pi) const {
55         if (pi == 0) return val;
56         if (pi == 1) return val - val / 2;
57         if (pi < pri.size() &&
58             pri[pi - 1] * pri[pi - 1] > val) {
59             assert(val <= lim);
60             return pc[val] - pi + 1;
61         }
62         return phi(val, pi - 1) - phi(val / pri[pi - 1], pi - 1);
63     }
64
65     int ask(int val) {

```

```

66     assert(val >= 0);
67     if (val <= lim) return pc[val];
68     if (auto it = mp.find(val); it != mp.end()) return it->second;
69
70     int a = ask(isqrt(isqrt(val)));
71     int b = ask(isqrt(val));
72     int c = ask(icbrt(val));
73     assert(b <= pri.size());
74     int ans = phi(val, a) + (b + a - 2) * (b - a + 1) / 2;
75     for (int i = a; i < b; ++i) {
76         int q = val / pri[i];
77         ans -= ask(q);
78         if (i < c) {
79             int hi = ask(isqrt(q));
80             for (int j = i; j < hi; ++j) ans -= ask(q / pri[j]) - j;
81         }
82     }
83     return mp[val] = ans;
84 }
85 };

```

01J - 质数和 (Min_25筛) .cpp

```

1  /*
2  用途：求不超过 n 的所有质数之和；这是 Min_25 筛的基础质数幂和筛法。
3  复杂度：约  $O(n^{3/4}/\log n)$  时间、 $O(\sqrt{n})$  空间，适合 n 远大于可直接筛的范围。
4  T 默认 i128 做精确计算，也可传 MInt 自动取模；三角和先整除 2，兼容偶数模数。
5  性能：默认使用展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7
8  using i128 = __int128_t;
9
10 using ld = long double;
11
12 template <class T = i128> struct Min25 {
13     int n = 0, s = 0;
14     vector<int> w;
15     vector<int> id1, id2, pr;
16     vector<T> g, pre;
17
18     int id(int x) const { return x <= s ? id1[x] : id2[n / x]; }
19
20     static T tri(int x) {
21         int a = x, b = x + 1;
22         if (a & 1) b >>= 1;
23         else a >>= 1;
24         return T(a) * T(b);
25     }
26
27     T askSum(int _n) {
28         assert(_n >= 0);
29         n = _n;
30         if (n < 2) return T{};
31         s = sqrtl(static_cast<ld>(n));
32         while ((s + 1) <= n / (s + 1)) ++s;
33         while (s > n / s) --s;
34
35         vector<char> vis(s + 1);
36         pr.clear();
37         pre.assign(1, T{});
38         for (int i = 2; i <= s; ++i) {
39             if (!vis[i]) {
40                 pr.push_back(i);
41                 pre.push_back(pre.back() + T(i));
42             }
43             for (int p : pr) {
44                 if (i * p > s) break;
45                 vis[i * p] = true;
46                 if (i % p == 0) break;
47             }
48         }
49
50         w.clear(), g.clear();
51         id1.assign(s + 1, 0), id2.assign(s + 1, 0);
52         for (int l = 1, r; l <= n; l = r + 1) {
53             int v = n / l;
54             r = n / v;
55             int k = w.size();
56             w.push_back(v);
57             g.push_back(tri(v) - T(1));

```



```

58         if (v <= s) id1[v] = k;
59         else id2[n / v] = k;
60     }
61
62     for (int i = 0; i < pr.size(); ++i) {
63         int p = pr[i];
64         if (p > n / p) break;
65         int pp = p * p;
66         for (int j = 0; j < w.size() && w[j] >= pp; ++j)
67             g[j] -= T(p) * (g[id(w[j] / p)] - pre[i]);
68     }
69     return g[id(n)];
70 }
71 };

```

02 - 快速幂 (ksm) .cpp

```

1  /*
2  用途: 64 位安全模乘、模幂与费马逆元。
3  前提: inv 仅在 mod 为质数且 a 非 0 (mod mod) 时使用。
4  环境: GCC/Clang 的 int 为 O(log e); 无 int 时模乘退化为 O(log mod)。
5  */
6
7  using u64 = uint64_t;
8  using u128 = __uint128_t;
9
10 namespace ModArith64 {
11
12 inline u64 mul(u64 a, u64 b, u64 mod) {
13     assert(mod > 0);
14     a %= mod;
15     b %= mod;
16 #ifdef __SIZEOF_INT128__
17     return u128(a) * b % mod;
18 #else
19     u64 res = 0;
20     while (b > 0) {
21         if (b & 1) res = res >= mod - a ? res - (mod - a) : res + a;
22         a = a >= mod - a ? a - (mod - a) : a + a;
23         b >>= 1;
24     }
25     return res;
26 #endif
27 }
28
29 inline u64 power(u64 a, u64 e, u64 mod) {
30     assert(mod > 0);
31     u64 res = 1 % mod;
32     while (e > 0) {
33         if (e & 1) res = mul(res, a, mod);
34         a = mul(a, a, mod);
35         e >>= 1;
36     }
37     return res;
38 }
39
40 inline u64 inv(u64 a, u64 mod) {
41     assert(mod > 1 && a % mod != 0);
42     return power(a % mod, mod - 2, mod);
43 }
44 } // namespace ModArith64

```

02A - 自动取模 (MInt) .cpp

```

1  /*
2  用途: 自动取模整数。直接写 MInt<P>, P=0 时先 MInt<0>::setMod(p)。
3  接口: pow(n)、inv()、四则运算、输入输出; 默认 Z 为模 998244353。
4  约定: inv()/除法要求模数为质数; 动态模数需满足 1 < p < 2^31。
5  性能: 内部值固定为 i32, 故在 #define int long long 下也不会把大数组翻倍。
6  */
7
8  using i32 = int32_t;
9
10 constexpr i32 mod = 998244353;
11
12 template <class T> T mypow(T a, int n) {
13     assert(n >= 0);
14     T r = T(1);

```

```

15     while (n) {
16         if (n & 1) r *= a;
17         a *= a;
18         n >>= 1;
19     }
20     return r;
21 }
22
23 template <i32 P> struct MInt {
24     static_assert(P >= 0);
25
26     i32 x = 0;
27     inline static i32 md = P ? P : mod;
28
29     MInt() = default;
30     MInt(int v) : x(norm(v)) {}
31
32     static i32 askMod() {
33         return P ? P : md;
34     }
35
36     static void setMod(i32 v) {
37         static_assert(P == 0, "Only MInt<0> supports a dynamic modulus.");
38         assert(v > 1);
39         md = v;
40     }
41
42     static i32 norm(int v) {
43         v %= askMod();
44         if (v < 0) v += askMod();
45         return static_cast<i32>(v);
46     }
47
48     i32 val() const {
49         return x;
50     }
51
52     explicit operator i32() const {
53         return x;
54     }
55
56     bool ask0() const {
57         return x == 0;
58     }
59
60     MInt pow(int n) const {
61         return mypow(*this, n);
62     }
63
64     MInt inv() const {
65         assert(x);
66         return pow(askMod() - 2);
67     }
68
69     MInt operator-() const {
70         return MInt(x ? askMod() - x : 0);
71     }
72
73     MInt& operator+=(MInt a) {
74         i32 p = askMod();
75         if (x >= p - a.x) x -= p - a.x;
76         else x += a.x;
77         return *this;
78     }
79
80     MInt& operator-=(MInt a) {
81         x -= a.x;
82         if (x < 0) x += askMod();
83         return *this;
84     }
85
86     MInt& operator*=(MInt a) {
87         x = static_cast<i32>(static_cast<int>(x) * a.x % askMod());
88         return *this;
89     }
90
91     MInt& operator/=(MInt a) {
92         return *this *= a.inv();
93     }
94
95     MInt& operator++() {
96         return *this += 1;

```

```

97     }
98
99     MInt operator++(i32) {
100         MInt a = *this;
101         ++*this;
102         return a;
103     }
104
105     MInt& operator--() {
106         return *this -= 1;
107     }
108
109     MInt operator--(i32) {
110         MInt a = *this;
111         --*this;
112         return a;
113     }
114
115     friend MInt operator+(MInt a, MInt b) {
116         return a += b;
117     }
118
119     friend MInt operator-(MInt a, MInt b) {
120         return a -= b;
121     }
122
123     friend MInt operator*(MInt a, MInt b) {
124         return a *= b;
125     }
126
127     friend MInt operator/(MInt a, MInt b) {
128         return a /= b;
129     }
130
131     friend bool operator==(MInt a, MInt b) {
132         return a.x == b.x;
133     }
134
135     friend bool operator!=(MInt a, MInt b) {
136         return a.x != b.x;
137     }
138
139     friend istream& operator>>(istream& is, MInt& a) {
140         int x;
141         is >> x;
142         a = MInt(x);
143         return is;
144     }
145
146     friend ostream& operator<<(ostream& os, const MInt& a) {
147         return os << a.x;
148     }
149 };
150
151 template <i32 V, i32 P> const MInt<P> IV = MInt<P>(V).inv();
152
153 using Z = MInt<mod>;

```

02AA - Barrett约简 (u32) .cpp

```

1  /*
2  用途: 固定 u32 模数下的 Barrett 模乘, 避免每次 64 位除法。
3  用法: Barrett bt(p); bt.mul(a,b), 其中  $1 \leq p < 2^{31}$ ,  $0 \leq a, b < p$ 。
4  复杂度:  $O(1)$ 。适合运行时 u32 模数且乘法极多的卡常环境。
5  性能: u32/u64/u128 是本模板为确定字宽和乘高位而保留的必要例外。
6  */
7
8  using u32 = uint32_t;
9  using u64 = uint64_t;
10 using u128 = __uint128_t;
11
12 struct Barrett {
13     u32 md;
14     u64 im;
15

```

```

16     explicit Barrett(u32 md) : md(md), im(u64(-1) / md + 1) {
17         assert(0 < md && md < (u32(1) << 31));
18     }
19
20     u32 mul(u32 a, u32 b) const {
21         u64 z = u64(a) * b;
22         u64 q = u128(z) * im >> 64;
23         u32 r = z - q * md;
24         if (r >= md) r += md;
25         return r;
26     }
27 };

```

02B - 裴蜀定理.cpp

```

1  /*
2  裴蜀定理: 整数  $a_1 \dots a_k$  的所有线性组合恰为  $\gcd(a_1, \dots, a_k)$  的倍数。
3  用途: 判断方程  $\sum a_i * x_i = \text{target}$  是否有整数解。
4  */
5
6  inline int gcdAll(const vector<int>& a) {
7      int res = 0;
8      for (int val : a) res = gcd(res, val);
9      return res >= 0 ? res : -res;
10 }
11
12 inline bool ask(const vector<int>& coe, int t) {
13     int d = gcdAll(coe);
14     return d == 0 ? t == 0 : t % d == 0;
15 }

```

02BA - 扩展欧几里得 (exgcd) .cpp

```

1  /*
2  用途: 求  $ax + by = \gcd(a, b)$  的一组系数, 以及任意模数下的逆元。
3   $\text{inv}(a, m)$  有值当且仅当  $\gcd(a, m) = 1$ :  $m$  不要求为质数。
4  */
5
6  struct ExgcdRes {
7      int gcd, x, y;
8  };
9
10 inline ExgcdRes exgcd(int a, int b) {
11     if (b == 0) return {a >= 0 ? a : -a, a >= 0 ? 1 : -1, 0};
12     auto sub = exgcd(b, a % b);
13     return {sub.gcd, sub.y, sub.x - (a / b) * sub.y};
14 }
15
16 inline optional<int> inv(int a, int mod) {
17     assert(mod > 0);
18     auto res = exgcd(a, mod);
19     if (res.gcd != 1) return nullopt;
20     int inv = res.x % mod;
21     if (inv < 0) inv += mod;
22     return inv;
23 }

```

02BAA - 中国剩余定理.cpp

```

1  /*
2  用途: 广义中国剩余定理, 合并  $x \equiv \text{residue} \pmod{\text{modulus}}$ ; 模数不要求互质。
3  依赖: 02 - 快速幂 (ModArith64) 与 02BA - 扩展欧几里得 (inv)。
4  返回 nullopt: 方程无解, 或合并后的 lcm 超出 int。
5  */
6
7  struct CRTResult {
8      int r; // 始终规范到  $[0, \text{modulus})$ 。
9      int mod;
10 };
11
12 inline int norm(int val, int mod) {
13     val %= mod;
14     return val < 0 ? val + mod : val;
15 }
16
17 inline optional<CRTResult> merge(CRTResult x, CRTResult y) {

```

```

18     assert(x.mod > 0 && y.mod > 0);
19     x.r = norm(x.r, x.mod);
20     y.r = norm(y.r, y.mod);
21
22     int g = gcd(x.mod, y.mod);
23     int dif = y.r - x.r;
24     if (dif % g != 0) return nullopt;
25
26     int b = y.mod / g;
27     if (x.mod > numeric_limits<int>::max() / b) return nullopt;
28     int lcm = x.mod * b;
29
30     int k = 0;
31     if (b != 1) {
32         auto iv = inv(x.mod / g, b);
33         assert(iv.has_value());
34         int rhs = norm(dif / g, b);
35         k = ModArith64::mul(rhs, *iv, b);
36     }
37
38     int add = ModArith64::mul(x.mod, k, lcm);
39     int r = x.r >= lcm - add
40         ? x.r - (lcm - add)
41         : x.r + add;
42     return CRTResult{r, lcm};
43 }
44
45 inline optional<CRTResult> crt(const vector<CRTResult>& eq) {
46     if (eq.empty()) return CRTResult{0, 1};
47     CRTResult res = eq[0];
48     if (res.mod <= 0) return nullopt;
49     res.r = norm(res.r, res.mod);
50     for (int i = 1; i < eq.size(); ++i) {
51         if (eq[i].mod <= 0) return nullopt;
52         auto z = merge(res, eq[i]);
53         if (!z) return nullopt;
54         res = *z;
55     }
56     return res;
57 }

```

02BAAB - 扩展 Lucas (组合数模合数) .cpp

```

1  /*
2  用途: 计算  $C(n, k) \bmod m$ ,  $m$  可以是合数。
3  依赖: 02 - ModArith64, 02BA - inv, 02BAA - 广义 CRT。
4  前提: 每个质数幂  $p^q$  会预处理  $O(p^q)$  数组, 适合  $m$  的各质因子幂可开 (典型  $m \leq 2e6$ ): 质数模优先用 03 / 03A。
5  */
6
7  namespace ExLucas {
8      struct Comb {
9          int pri, pk;
10         int pe = 0;
11         vector<int> fac;
12
13         Comb(int p, int pk, int pe)
14             : pri(p), pk(pk), pe(pe) {
15             assert(pri >= 2 && pk >= pri && pk <= numeric_limits<int>::max());
16             fac.assign(pk + 1, 1);
17             for (int val = 1; val <= pk; ++val) {
18                 fac[val] = fac[val - 1];
19                 if (val % pri != 0) {
20                     fac[val] = ModArith64::mul(fac[val], val, pk);
21                 }
22             }
23         }
24
25         int cntP(int val) const {
26             int ans = 0;
27             while (val > 0) val /= pri, ans += val;
28             return ans;
29         }
30
31         int facP(int val) const {
32             if (val == 0) return 1;
33             int ans = ModArith64::power(fac[pk], val / pk, pk);
34             ans = ModArith64::mul(ans, fac[val % pk], pk);
35             return ModArith64::mul(ans, facP(val / pri), pk);
36         }
37     }

```

```

38     int C(int n, int k) const {
39         if (k < 0 || k > n) return 0;
40         int e = cntP(n) - cntP(k) - cntP(n - k);
41         if (e >= pe) return 0;
42
43         int a = facP(n);
44         int bl = facP(k);
45         int br = facP(n - k);
46         auto il = inv(bl, pk);
47         auto ir = inv(br, pk);
48         assert(il && ir);
49         int ans = ModArith64::mul(a, *il, pk);
50         ans = ModArith64::mul(ans, *ir, pk);
51         ans = ModArith64::mul(ans, ModArith64::power(pri, e, pk), pk);
52         return ans;
53     }
54 };
55
56 inline vector<tuple<int, int, int>> factor(int mod) {
57     assert(mod >= 1);
58     vector<tuple<int, int, int>> fac;
59     for (int d = 2; d <= mod / d; ++d) {
60         if (mod % d != 0) continue;
61         int pk = 1;
62         int e = 0;
63         do {
64             mod /= d;
65             pk *= d;
66             ++e;
67         } while (mod % d == 0);
68         fac.emplace_back(d, pk, e);
69     }
70     if (mod > 1) fac.emplace_back(mod, mod, 1);
71     return fac;
72 }
73
74 inline int C(int n, int k, int mod) {
75     assert(n >= 0 && mod > 0);
76     if (k < 0 || k > n || mod == 1) return 0;
77     vector<CRTResult> eq;
78     for (auto [pri, pk, e] : factor(mod)) {
79         Comb cal(pri, pk, e);
80         eq.push_back({cal.C(n, k), pk});
81     }
82     auto res = crt(eq);
83     assert(res && res->mod == mod);
84     return res->r;
85 }
86 } // namespace ExLucas

```

02C - BSGS.cpp

```

1  /*
2  用途: 求最小非负 x, 使  $a^x \equiv b \pmod m$ 。
3  依赖: 02 - 快速幂 (ModArith64) 和 02BA - 扩展欧几里得 (inv)。
4  接口: bsgs 要求  $\gcd(a, m)=1$ ; exBSGS 允许不互素。均返回 optional<int>。
5  复杂度:  $O(\sqrt{m})$  时间和空间; 模数需在 signed int 范围内。
6  */
7  using ld = long double;
8
9  namespace BSGS {
10 inline int norm(int val, int mod) {
11     val %= mod;
12     return val < 0 ? val + mod : val;
13 }
14
15 inline optional<int> bsgs(int a, int b, int mod) {
16     assert(mod > 0);
17     if (mod == 1) return 0;
18     a = norm(a, mod);
19     b = norm(b, mod);
20     if (b == 1) return 0;
21     if (gcd(a, mod) != 1) return nullopt;
22
23     int blk = sqrtl(ld(mod));
24     while (blk < mod / blk || (blk == mod / blk && blk * blk < mod)) ++blk;
25
26     unordered_map<int, int> bb;
27     bb.reserve(blk * 2 + 1);
28     int cur = 1;

```

```

29     for (int j = 0; j < blk; ++j) {
30         if (!bb.count(cur)) bb.emplace(cur, j);
31         cur = ModArith64::mul(cur, a, mod);
32     }
33
34     auto ia = inv(a, mod);
35     assert(ia.has_value());
36     int gs = ModArith64::power(*ia, blk, mod);
37     cur = b;
38     for (int i = 0; i <= blk; ++i) {
39         auto it = bb.find(cur);
40         if (it != bb.end()) return i * blk + it->second;
41         cur = ModArith64::mul(cur, gs, mod);
42     }
43     return nullopt;
44 }
45
46 inline optional<int> exBSGS(int a, int b, int mod) {
47     assert(mod > 0);
48     if (mod == 1) return 0;
49     a = norm(a, mod);
50     b = norm(b, mod);
51     if (b == 1) return 0;
52
53     int ban = 0, scl = 1;
54     while (true) {
55         int g = gcd(a, mod);
56         if (g == 1) break;
57         if (b == scl) return ban;
58         if (b % g != 0) return nullopt;
59         b /= g;
60         mod /= g;
61         ++ban;
62         if (mod == 1) return ban;
63         scl = ModArith64::mul(scl, a / g, mod);
64     }
65
66     auto is = inv(scl, mod);
67     assert(is.has_value());
68     auto rem = bsgs(a, ModArith64::mul(b, *is, mod), mod);
69     return rem ? optional<int>(*rem + ban) : nullopt;
70 }
71 } // namespace BSGS

```

02D - 逆元线性预处理.cpp

```

1  /*
2  用途: 预处理 1..n 在质数模下的逆元。
3  用法: auto iv = invAll<Z>(n), 其中 Z 可为任意 MInt<P>。
4  前提: Z::askMod() 为质数且 n < mod; 复杂度 O(n)。
5  */
6  template <class Z> vector<Z> invAll(int n) {
7      assert(n >= 0 && n < Z::askMod());
8      vector<Z> iv(n + 1);
9      if (n) iv[1] = Z(1);
10     int p = Z::askMod();
11     for (int i = 2; i <= n; ++i) iv[i] = Z(p - p / i) * iv[p % i];
12     return iv;
13 }

```

02E - 模平方根 (Tonelli-Shanks) .cpp

```

1  /*
2  用途: 求  $x^2 = a \pmod p$  的一组解; p 必须为质数。
3  依赖: 02A - 自动取模 (MInt)。无解返回 nullopt。
4  */
5
6  template <class Z> optional<Z> sqrt(Z a) {
7      int p = Z::askMod();
8      assert(p >= 2);
9      if (a == Z(0) || p == 2) return a;
10     if (a.pow((p - 1) / 2) != Z(1)) return nullopt;
11     if (p % 4 == 3) return a.pow((p + 1) / 4);
12
13     int s = 0;
14     int q = p - 1;
15     while (!(q & 1)) q >>= 1, ++s;
16     Z z = 2;

```

```

17 while (z.pow((p - 1) / 2) == Z(1)) ++z;
18 Z x = a.pow((q + 1) / 2), b = a.pow(q), c = z.pow(q);
19 int m = s;
20 while (b != Z(1)) {
21     int i = 1;
22     Z d = b * b;
23     while (i < m && d != Z(1)) d *= d, ++i;
24     assert(i < m);
25     Z t = c.pow(1LL << (m - i - 1));
26     x *= t;
27     c = t * t;
28     b *= c;
29     m = i;
30 }
31 return x;
32 }

```

02EA - 模平方根 (Cipolla) .cpp

```

1  /*
2  用途: Cipolla 算法求  $x^2 = n \pmod{p}$  的一个解; 无解返回 nullopt。
3  依赖: 02A - 自动取模 (MInt)。p 必须为质数。
4  区别: 02E 是确定性的 Tonelli-Shanks; 本模板随机选非二次剩余, 适合希望保留 Cipolla 写法时使用。
5  */
6
7  template <class Z> optional<Z> cipolla(Z n) {
8      int p = Z::askMod();
9      assert(p >= 2);
10     if (p == 2 || n == Z(0)) return n;
11     if (n.pow((p - 1) / 2) != Z(1)) return nullopt;
12
13     static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
14     Z a, w;
15     do {
16         a = Z(rng() % p);
17         w = a * a - n;
18     } while (w.pow((p - 1) / 2) != Z(p - 1));
19
20     auto mul = [&](array<Z, 2> x, array<Z, 2> y) {
21         return array<Z, 2>{x[0] * y[0] + x[1] * y[1] * w,
22                             x[0] * y[1] + x[1] * y[0]};
23     };
24     array<Z, 2> r{Z(1), Z(0)}, b{a, Z(1)};
25     for (int k = (p + 1) / 2; k; k >>= 1) {
26         if (k & 1) r = mul(r, b);
27         b = mul(b, b);
28     }
29     return r[0];
30 }

```

02F - 原根.cpp

```

1  /*
2  用途: 判断模数是否有原根, 寻找最小/全部原根, 并判断给定数是否为原根。
3  前提: 依赖 02 - 快速幂 (ModArith64)。原根仅存在于 2、4、 $p^k$ 、 $2p^k$  (p 为奇质数)。
4  接口: exist(n) 判断是否存在; solve(n) 返回最小原根, 不存在返回 -1; roots(n) 返回递增的全部原根; ask(g, n) 判断 g。
5  复杂度: 试除分解  $O(\sqrt{n})$ , 单次验证  $O(w(\phi(n)) \log n)$ ; 枚举全部原根  $O(\phi(n) \log n)$ 。
6  */
7
8  namespace PrimRoot {
9      inline vector<int> factor(int val) {
10         assert(val >= 1);
11         vector<int> fac;
12         for (int d = 2; d <= val / d; ++d) {
13             if (val % d != 0) continue;
14             fac.push_back(d);
15             do val /= d; while (val % d == 0);
16         }
17         if (val > 1) fac.push_back(val);
18         return fac;
19     }
20
21     inline bool exist(int mod) {
22         assert(mod >= 2);
23         if (mod == 2 || mod == 4) return true;
24         int x = mod, c = 0;
25         while (x % 2 == 0) x /= 2, ++c;
26         if (c > 1 || x == 1) return false;

```



```

27     int p = 0;
28     for (int d = 3; d <= x / d; d += 2) {
29         if (x % d == 0) {
30             p = d;
31             break;
32         }
33     }
34     if (!p) return true;
35     while (x % p == 0) x /= p;
36     return x == 1;
37 }
38
39 inline int phi(int mod) {
40     int res = mod;
41     for (int p : factor(mod)) res = res / p * (p - 1);
42     return res;
43 }
44
45 inline bool ask(int g, int mod, int ph, const vector<int>& fac) {
46     g %= mod;
47     if (g <= 0) g += mod;
48     if (gcd(g, mod) != 1) return false;
49     for (int p : fac) {
50         if (ModArith64::power(g, ph / p, mod) == 1) return false;
51     }
52     return true;
53 }
54
55 inline bool ask(int g, int mod) {
56     assert(mod >= 2);
57     if (!exist(mod)) return false;
58     int ph = phi(mod);
59     return ask(g, mod, ph, factor(ph));
60 }
61
62 inline int find(int mod, int ph, const vector<int>& fac) {
63     for (int g = 1; g < mod; ++g) {
64         if (ask(g, mod, ph, fac)) return g;
65     }
66     return -1;
67 }
68
69 inline int solve(int mod) {
70     assert(mod >= 2);
71     if (!exist(mod)) return -1;
72     int ph = phi(mod);
73     return find(mod, ph, factor(ph));
74 }
75
76 inline vector<int> roots(int mod) {
77     assert(mod >= 2);
78     if (!exist(mod)) return {};
79     int ph = phi(mod);
80     vector<int> fac = factor(ph);
81     int g = find(mod, ph, fac), cnt = phi(ph);
82     vector<int> res;
83     res.reserve(cnt);
84     for (int k = 1, x = g; k <= ph; ++k) {
85         if (gcd(k, ph) == 1) res.push_back(x);
86         x = ModArith64::mul(x, g, mod);
87     }
88     sort(res.begin(), res.end());
89     return res;
90 }
91 } // namespace PrimRoot

```

02G - 整除分块求和 (floor_sum) .cpp

```

1  /*
2  用途: 求  $\sum_{0 \leq i < n} \text{floor}((a*i+b)/m)$ 。常用于整除分块、格点计数与欧几里得递归。
3  前提:  $n, m, a, b$  均非负且  $m > 0$ 。返回 i128，调用方按答案范围输出。
4  复杂度:  $O(\log(\max(a, m)))$ 。
5  */
6
7  using i128 = __int128_t;
8
9  namespace FloorSum {
10     inline i128 flrSum(int n, int m, int a, int b) {
11         assert(n >= 0 && m > 0 && a >= 0 && b >= 0);
12         i128 cnt = n, mod = m, coe = a, off = b, ans = 0;

```

```

13     while (true) {
14         if (coe >= mod) {
15             ans += (cnt - 1) * cnt * (coe / mod) / 2;
16             coe %= mod;
17         }
18         if (off >= mod) {
19             ans += cnt * (off / mod);
20             off %= mod;
21         }
22         i128 mx = coe * cnt + off;
23         if (mx < mod) break;
24         cnt = mx / mod;
25         off = mx % mod;
26         swap(mod, coe);
27     }
28     return ans;
29 }
30 } // namespace FloorSum

```

02GA - 加权整除和（万能欧几里得）.cpp

```

1  /*
2  用途：同时求  $y_i = \text{floor}((a \cdot i + b) / c)$  ( $0 \leq i \leq n$ ) 的  $\sum y_i$ 、 $\sum i \cdot y_i$ 、 $\sum y_i^2$ 。
3  依赖：02A - 自动取模 (MInt)。接口：Euclid<Z>::ask(a,b,c,n)。
4  前提： $a, b, n$  非负、 $c$  正，且  $a \cdot n + b$  可放入  $\text{int}$ ； $Z$  所在域中 2、6 可逆（通常用质数模 MInt）。
5  复杂度： $O(\log(\max(a, c)))$ 。这是 floor_sum 的带权扩展，可用于二次矩、矩阵转移前的系数统计。
6  */
7  template <class Z> struct Euclid {
8      struct Ans {
9          Z s, t, u;
10     };
11
12     static Z sum1(int n) {
13         return Z(n) * Z(n + 1) / Z(2);
14     }
15
16     static Z sum2(int n) {
17         return Z(n) * Z(n + 1) * Z(n * 2 + 1) / Z(6);
18     }
19
20     static Ans ask(int a, int b, int c, int n) {
21         assert(a >= 0 && b >= 0 && c > 0 && n >= 0);
22         assert(Z(2) != Z(0) && Z(6) != Z(0));
23         if (a >= c || b >= c) {
24             int q = a / c, p = b / c;
25             Ans x = ask(a % c, b % c, c, n);
26             Z m = n + 1, s = sum1(n), t = sum2(n);
27             Z qz = q, pz = p;
28             return {
29                 qz * s + pz * m + x.s,
30                 qz * t + pz * s + x.t,
31                 qz * qz * t + pz * pz * m + Z(2) * qz * pz * s + Z(2) * qz * x.t + Z(2) * pz * x.s + x.u,
32             };
33         }
34         if (!a) return {};
35         int m = (a * n + b) / c;
36         if (!m) return {};
37         Ans x = ask(c, c - b - 1, a, m - 1);
38         Z mz = m, nz = n;
39         return {
40             nz * mz - x.s,
41             mz * sum1(n) - (x.u + x.s) / Z(2),
42             nz * mz * mz - Z(2) * x.t - x.s,
43         };
44     }
45 };

```

02H - 大指数模幂（十进制）.cpp

```

1  /*
2  用途：计算  $a^b \bmod m$ ，其中  $b$  或  $a$  可用非负十进制字符串给出。
3  依赖：02 - 快速幂 (ModArith64)。
4  接口：askMod(s,m) 求十进制串模  $m$ ；solve(a,b,m) 或 solve(aStr,b,m)。
5  复杂度： $O(|a| + |b| \log 10)$ 。不需要  $\gcd(a, m) = 1$ ，也不依赖欧拉降幂。
6  */
7
8  namespace BigPow {
9      inline int askMod(const string& s, int md) {

```

```

10     assert(md > 0 && !s.empty());
11     int ans = 0;
12     for (char c : s) {
13         assert('0' <= c && c <= '9');
14         ans = (ModArith64::mul(ans, 10, md) + c - '0') % md;
15     }
16     return ans;
17 }
18
19 inline int solve(int a, const string& b, int md) {
20     assert(md > 0 && !b.empty());
21     a %= md;
22     if (a < 0) a += md;
23     int ans = 1 % md;
24     for (char c : b) {
25         assert('0' <= c && c <= '9');
26         ans = ModArith64::mul(ans, ModArith64::power(ans, 9, md), md);
27         if (c != '0') ans = ModArith64::mul(ans, ModArith64::power(a, c - '0', md), md);
28     }
29     return ans;
30 }
31
32 inline int solve(const string& a, const string& b, int md) {
33     return solve(askMod(a, md), b, md);
34 }
35 } // namespace BigPow

```

02I - Pohlig-Hellman (离散对数, 光滑群阶) .cpp

```

1  /*
2  用途: Pohlig-Hellman 离散对数。p-1 平滑时, 比通用 BSGS 更快。
3  依赖: 02 - 快速幂 (ModArith64)、02BA - 扩展欧几里得 (inv)。
4  前提: p 为质数, g 是模 p 的原根; fac 是 p-1 的质因子分解 (质数, 指数)。
5  接口: solve(g,h,p,fac) 返回最小非负 x 使  $g^x = h \pmod{p}$ ; solve(g,h,p) 自动试除分解 p-1。
6  复杂度:  $O(\sum \sqrt{q} \log p)$ , q 为 p-1 的不同质因子; 适合 q 较小的光滑群阶。
7  */
8
9  namespace Pohlig {
10 inline vector<pii> factor(int x) {
11     vector<pii> res;
12     for (int p = 2; p <= x / p; ++p) {
13         if (x % p) continue;
14         int e = 0;
15         do x /= p, ++e; while (x % p == 0);
16         res.push_back({p, e});
17     }
18     if (x > 1) res.push_back({x, 1});
19     return res;
20 }
21
22 inline optional<int> askLog(int a, int b, int md, int ord) {
23     int blk = sqrt(ord);
24     while (blk < ord / blk || (blk == ord / blk && blk * blk < ord)) ++blk;
25     unordered_map<int, int> mp;
26     mp.reserve(blk * 2 + 1);
27     int cur = 1;
28     for (int j = 0; j < blk; ++j) {
29         if (!mp.count(cur)) mp.emplace(cur, j);
30         cur = ModArith64::mul(cur, a, md);
31     }
32     int stp = ModArith64::power(a, (ord - blk % ord) % ord, md);
33     cur = b;
34     for (int i = 0; i <= blk; ++i) {
35         auto it = mp.find(cur);
36         if (it != mp.end()) {
37             int x = i * blk + it->second;
38             if (x < ord) return x;
39         }
40         cur = ModArith64::mul(cur, stp, md);
41     }
42     return nullopt;
43 }
44
45 inline optional<int> solve(int g, int h, int md, const vector<pii>& fac) {
46     assert(md >= 2);
47     g %= md;
48     h %= md;
49     if (g < 0) g += md;
50     if (h < 0) h += md;
51     if (!g || !h) return nullopt;

```

```

52     if (md == 2) return h == 1 ? optional<int>(0) : nullopt;
53
54     int ord = md - 1, all = 1;
55     for (auto [q, e] : fac) {
56         assert(q > 1 && e > 0);
57         for (int i = 0; i < e; ++i) all *= q;
58     }
59     if (all != ord) return nullopt;
60
61     int ans = 0, mul = 1;
62     for (auto [q, e] : fac) {
63         int cur = 0, pw = 1, qe = 1;
64         int bas = ModArith64::power(g, ord / q, md);
65         for (int i = 0; i < e; ++i) {
66             int ig = ModArith64::power(g, (ord - cur % ord) % ord, md);
67             int val = ModArith64::mul(h, ig, md);
68             val = ModArith64::power(val, ord / (pw * q), md);
69             auto dig = askLog(bas, val, md, q);
70             if (!dig) return nullopt;
71             cur += *dig * pw;
72             pw *= q;
73             qe *= q;
74         }
75
76         int dif = (cur - ans) % qe;
77         if (dif < 0) dif += qe;
78         auto iv = inv(mul % qe, qe);
79         assert(iv.has_value());
80         int k = ModArith64::mul(dif, *iv, qe);
81         int nxt = mul * qe;
82         ans = (ans + ModArith64::mul(mul, k, nxt)) % nxt;
83         mul = nxt;
84     }
85     return ans;
86 }
87
88 inline optional<int> solve(int g, int h, int md) {
89     return solve(g, h, md, factor(md - 1));
90 }
91 } // namespace Pohlig

```

02J - 值域GCD预处理.cpp

```

1  /*
2  用途：预处理后快速求 gcd(x,y)，其中 0 <= x <= n, y 可为 64 位整数。
3  复杂度：预处理 O(n + sqrt(n)^2)，单次 O(1)。默认 n=1e6。
4  限制：适合 x 在固定小值域内、y 很大且查询很多的场景；普通单次 gcd 不要使用。
5  性能：因 fac/gd 是大表，纯下标和表项必须保留 i32；改为宏展开后的 int 会显著增加内存。
6  */
7
8  using i32 = int32_t;
9
10 struct FastGCD {
11     i32 n, b;
12     vector<i32> p;
13     vector<bool> ip;
14     vector<array<i32, 3>> fac;
15     vector<vector<i32>> gd;
16
17     explicit FastGCD(i32 m = 1'000'000) {
18         init(m);
19     }
20
21     void init(i32 m) {
22         assert(m >= 1);
23         n = m;
24         b = 1;
25         while (1LL * (b + 1) * (b + 1) <= n) ++b;
26         gd.assign(b + 1, vector<i32>(b + 1));
27         for (i32 i = 0; i <= b; ++i) gd[i][0] = gd[0][i] = i;
28         for (i32 i = 1; i <= b; ++i)
29             for (i32 j = i; j <= b; ++j) gd[i][j] = gd[j][i] = gd[i][j % i];
30
31         p.clear();
32         ip.assign(n + 1, true);
33         ip[0] = false;
34         if (n >= 1) ip[1] = false;
35         fac.assign(n + 1, {});
36         fac[1] = {1, 1, 1};
37         for (i32 i = 2; i <= n; ++i) {

```

```

38         if (ip[i]) {
39             p.push_back(i);
40             fac[i] = {1, 1, i};
41         }
42         for (i32 q : p) {
43             if (i > n / q) break;
44             i32 v = i * q;
45             ip[v] = false;
46             fac[v] = fac[i];
47             *min_element(fac[v].begin(), fac[v].end()) *= q;
48             if (i % q == 0) break;
49         }
50     }
51 }
52
53 i32 ask0(i32 x, int y) const {
54     i32 r = y % x;
55     return ip[x] ? (r ? 1 : x) : gd[r][x];
56 }
57
58 int ask(i32 x, int y) const {
59     if (!x) return y < 0 ? -y : y;
60     assert(x <= n);
61     if (y < 0) y = -y;
62     i32 a = ask0(fac[x][0], y);
63     y /= a;
64     i32 b = ask0(fac[x][1], y);
65     y /= b;
66     i32 c = ask0(fac[x][2], y);
67     return 1LL * a * b * c;
68 }
69 };

```

03 - 组合数学 (Comb, with Z) .cpp

```

1  /*
2  用途: 阶乘、逆阶乘与  $C(n, k)$ 、 $A(n, k)$ 。
3  用法: Comb<Z> c(n); Z 可为任意 MInt<P>。
4  前提: 模数为质数, 且查询  $n < \text{mod}$ ; 首次扩展  $O(n)$ , 之后  $O(1)$ 。
5  */
6  template <class Z> struct Comb {
7      int n = 0;
8      vector<Z> f{Z(1)}, g{Z(1)};
9
10     Comb() = default;
11     explicit Comb(int m) {
12         init(m);
13     }
14
15     void init(int m) {
16         assert(m >= 0 && m < Z::askMod());
17         if (m <= n) return;
18         f.resize(m + 1);
19         g.resize(m + 1);
20         for (int i = n + 1; i <= m; ++i) f[i] = f[i - 1] * i;
21         g[m] = f[m].inv();
22         for (int i = m; i > n; --i) g[i - 1] = g[i] * i;
23         n = m;
24     }
25
26     Z fac(int x) {
27         assert(x >= 0);
28         if (x > n) init(x);
29         return f[x];
30     }
31
32     Z ifac(int x) {
33         assert(x >= 0);
34         if (x > n) init(x);
35         return g[x];

```

```

36     }
37
38     Z C(int x, int k) {
39         if (x < 0 || k < 0 || k > x) return Z(0);
40         return fac(x) * ifac(k) * ifac(x - k);
41     }
42
43     Z A(int x, int k) {
44         if (x < 0 || k < 0 || k > x) return Z(0);
45         return fac(x) * ifac(x - k);
46     }
47 };

```

03A - Lucas 定理.cpp

```

1  /*
2  用途: 小质数 p 下求 C(n,k), n、k 可很大。
3  用法: LucasPrime<Z> c(p); 要求 Z::askMod() == p。
4  */
5
6  template <class Z> struct LucasPrime {
7      int p = 0;
8      vector<Z> f, g;
9
10     LucasPrime() = default;
11
12     explicit LucasPrime(int p) {
13         init(p);
14     }
15
16     void init(int x) {
17         assert(x >= 2 && Z::askMod() == x);
18         p = x;
19         f.assign(p, Z(1));
20         g.assign(p, Z(1));
21         for (int i = 1; i < p; ++i) f[i] = f[i - 1] * i;
22         g[p - 1] = f[p - 1].inv();
23         for (int i = p - 1; i; --i) g[i - 1] = g[i] * i;
24     }
25
26     Z smallC(int n, int k) const {
27         if (k < 0 || k > n) return Z(0);
28         return f[n] * g[k] * g[n - k];
29     }
30
31     Z C(int n, int k) const {
32         assert(p);
33         if (k < 0 || k > n) return Z(0);
34         Z r = 1;
35         while (n || k) {
36             int x = n % p, y = k % p;
37             if (y > x) return Z(0);
38             r *= smallC(x, y);
39             n /= p;
40             k /= p;
41         }
42         return r;
43     }
44 };

```

03B - 康托展开 (暴力, 小 n) .cpp

```

1  /*
2  用途: 求 1..n 排列的精确康托展开排名 (从 1 开始), 并支持反排名。
3  复杂度: O(n^2), int 精确范围 n <= 20。大 n 且只需模意义排名时使用 03C。
4  */
5  struct Cantor {
6      static void check(const vector<int>& p) {
7          int n = p.size();
8          vector<char> vis(n + 1, false);
9          for (int val : p) {
10             assert(1 <= val && val <= n && !vis[val]);
11             vis[val] = true;
12         }
13     }
14
15     static int askRank(const vector<int>& p) {
16         int n = p.size();

```

```

17     assert(n <= 20);
18     check(p);
19     vector<int> fac(n + 1, 1);
20     for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * i;
21     int rk = 1;
22     for (int i = 0; i < n; ++i) {
23         int cnt = 0;
24         for (int j = i + 1; j < n; ++j) cnt += p[j] < p[i];
25         rk += cnt * fac[n - 1 - i];
26     }
27     return rk;
28 }
29
30 static vector<int> askPerm(int n, int rk) {
31     assert(0 <= n && n <= 20);
32     vector<int> fac(n + 1, 1);
33     for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * i;
34     assert(1 <= rk && rk <= fac[n]);
35     --rk;
36     vector<int> vis(n), p;
37     iota(vis.begin(), vis.end(), 1);
38     for (int rem = n; rem >= 1; --rem) {
39         int id = rk / fac[rem - 1];
40         rk %= fac[rem - 1];
41         p.push_back(vis[id]);
42         vis.erase(vis.begin() + id);
43     }
44     return p;
45 }
46 };

```

03C - 康托展开 (树状数组, 大 n) .cpp

```

1  /*
2  用途: 求 1..n 排列的康托展开排名模 mod。
3  复杂度: O(n log n)。结果只保留模 mod; n <= 20 且需要精确排名或反排名时使用 03B。
4  */
5
6  using i128 = __int128_t;
7
8  struct Cantor {
9      static int mul(int a, int b, int mod) {
10         assert(mod > 0);
11         a %= mod;
12         b %= mod;
13 #ifdef __SIZEOF_INT128__
14         return i128(a) * b % mod;
15 #else
16         int res = 0;
17         while (b > 0) {
18             if (b & 1) res = res >= mod - a ? res - (mod - a) : res + a;
19             a = a >= mod - a ? a - (mod - a) : a + a;
20             b >>= 1;
21         }
22         return res;
23 #endif
24     }
25
26     static void check(const vector<int>& p) {
27         int n = p.size();
28         vector<char> vis(n + 1, false);
29         for (int val : p) {
30             assert(1 <= val && val <= n && !vis[val]);
31             vis[val] = true;
32         }
33     }
34
35     static int askMod(const vector<int>& p, int mod) {
36         assert(mod > 0);
37         int n = p.size();
38         check(p);
39         struct BIT {
40             int n;
41             vector<int> t;
42             explicit BIT(int n) : n(n), t(n + 1, 0) {}
43             void modify(int x, int v) { for (; x <= n; x += x & -x) t[x] += v; }
44             int ask(int x) const { int r = 0; for (; x > 0; x -= x & -x) r += t[x]; return r; }
45         } bit(n);
46
47         vector<int> fac(n + 1, 1 % mod);

```

```

48     for (int i = 1; i <= n; ++i) fac[i] = mul(fac[i - 1], i, mod);
49     for (int x = 1; x <= n; ++x) bit.modify(x, 1);
50
51     int rk = 1 % mod;
52     for (int i = 0; i < n; ++i) {
53         int cnt = bit.ask(p[i] - 1);
54         int get = mul(cnt, fac[n - 1 - i], mod);
55         rk = rk >= mod - get ? rk - (mod - get) : rk + get;
56         bit.modify(p[i], -1);
57     }
58     return rk;
59 }
60
61 // 质数模时优先使用本重载, Z 自动完成加乘取模。
62 template <class Z> static Z askRank(const vector<int>& p) {
63     int n = p.size();
64     check(p);
65     struct BIT {
66         int n;
67         vector<int> t;
68         explicit BIT(int n) : n(n), t(n + 1) {}
69         void modify(int x, int v) {
70             for (; x <= n; x += x & -x) t[x] += v;
71         }
72         int ask(int x) const {
73             int r = 0;
74             for (; x; x -= x & -x) r += t[x];
75             return r;
76         }
77     } b(n);
78     vector<Z> f(n + 1, Z(1));
79     for (int i = 1; i <= n; ++i) f[i] = f[i - 1] * i;
80     for (int x = 1; x <= n; ++x) b.modify(x, 1);
81     Z r = 1;
82     for (int i = 0; i < n; ++i) {
83         r += Z(b.ask(p[i] - 1)) * f[n - 1 - i];
84         b.modify(p[i], -1);
85     }
86     return r;
87 }
88 };

```

03D - 卡特兰数.cpp

```

1  /*
2  用途: 预处理 Catalan(0..n)。
3  前提: Z 的模数为质数, n < mod。
4  */
5  template <class Z> vector<Z> catalan(int n) {
6      assert(n >= 0 && n < Z::askMod());
7      vector<Z> a(n + 1);
8      a[0] = 1;
9      for (int i = 1; i <= n; ++i) a[i] = a[i - 1] * Z(4 * i - 2) / Z(i + 1);
10     return a;
11 }

```

03E - 斯特林数 (第一、二类) .cpp

```

1  /*
2  用途: 预处理无符号第一类、第二类 Stirling 数。
3  复杂度: O(n^2)。
4  */
5  template <class Z> struct Stirling {
6      int n = 0;
7      vector<vector<Z>> a, b;
8
9      Stirling() = default;
10
11     explicit Stirling(int m) {
12         init(m);
13     }
14
15     void init(int m) {
16         assert(m >= 0);
17         n = m;
18         a.assign(n + 1, vector<Z>(n + 1));
19         b.assign(n + 1, vector<Z>(n + 1));
20         a[0][0] = b[0][0] = 1;

```



```

21     for (int i = 1; i <= n; ++i) {
22         for (int j = 1; j <= i; ++j) {
23             a[i][j] = a[i - 1][j - 1] + Z(i - 1) * a[i - 1][j];
24             b[i][j] = b[i - 1][j - 1] + Z(j) * b[i - 1][j];
25         }
26     }
27 }
28
29 Z stir1(int x, int k) const {
30     assert(0 <= x && x <= n && 0 <= k && k <= x);
31     return (x - k) & 1 ? -a[x][k] : a[x][k];
32 }
33 };

```

03EA - 斯特林数 (NTT快速求法) .cpp

```

1  /*
2  用途: 用 NTT 快速求固定 n 的第一类、第二类 Stirling 数整行。
3  依赖: 02A - 自动取模 (MInt)、08B 或 08C - 多项式 (Poly, Z)。
4  接口: StirFast<Z>::ask1(n) 返回无符号第一类; ask1(n, true) 返回有符号第一类;
5         StirFast<Z>::ask2(n) 返回第二类, 返回值下标 k 为 0..n。
6  复杂度: 第一类  $O(n \log^2 n)$ , 第二类  $O(n \log n)$ ; 均比 03E 的  $O(n^2)$  适合大 n。
7  限制: Z 的模数须为 NTT 友好质数, 且  $n < \text{mod}$ ; 小 n 使用 03E 常数更小。
8  */
9  template <class Z, int G = 3> struct StirFast {
10     using P = Poly<Z, G>;
11
12     static vector<Z> mul(vector<Z> a, vector<Z> b) {
13         if (a.empty() || b.empty()) return {};
14         int n = a.size(), m = b.size();
15         if (min(n, m) <= 32) {
16             vector<Z> c(n + m - 1);
17             for (int i = 0; i < n; ++i)
18                 for (int j = 0; j < m; ++j) c[i + j] += a[i] * b[j];
19             return c;
20         }
21         return P::mul(move(a), move(b));
22     }
23
24     static vector<Z> prod(int l, int r) {
25         if (l >= r) return {Z(1)};
26         if (r - l == 1) return {Z(l), Z(1)};
27         int m = (l + r) >> 1;
28         return mul(prod(l, m), prod(m, r));
29     }
30
31     static vector<Z> ask1(int n, bool sg = false) {
32         assert(n >= 0);
33         vector<Z> a = prod(0, n);
34         a.resize(n + 1);
35         if (sg)
36             for (int k = 0; k <= n; ++k)
37                 if ((n - k) & 1) a[k] = -a[k];
38         return a;
39     }
40
41     static Z ask1(int n, int k, bool sg = false) {
42         assert(0 <= k && k <= n);
43         return ask1(n, sg)[k];
44     }
45
46     static vector<Z> ask2(int n) {
47         assert(0 <= n && n < Z::askMod());
48         vector<Z> f(n + 1), g(n + 1), fac(n + 1), iv(n + 1);
49         fac[0] = 1;
50         for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * Z(i);
51         iv[n] = fac[n].inv();
52         for (int i = n; i; --i) iv[i - 1] = iv[i] * Z(i);
53         for (int i = 0; i <= n; ++i) {
54             f[i] = Z(i).pow(n) * iv[i];
55             g[i] = (i & 1) ? -iv[i] : iv[i];
56         }
57         vector<Z> a = mul(move(f), move(g));
58         a.resize(n + 1);
59         return a;
60     }
61 };

```

03F - 错排数与贝尔数.cpp

```

1  /*
2  用途：预处理错排数  $D(0..n)$  与 Bell 数  $B(0..n)$ 。
3  仅要求  $Z$  支持加减乘。
4  */
5  template <class Z> vector<Z> derange(int n) {
6      assert(n >= 0);
7      vector<Z> a(n + 1);
8      a[0] = 1;
9      for (int i = 2; i <= n; ++i) a[i] = Z(i - 1) * (a[i - 1] + a[i - 2]);
10     return a;
11 }
12
13 template <class Z> vector<Z> bell(int n) {
14     assert(n >= 0);
15     vector<vector<Z>> a(n + 1);
16     vector<Z> b(n + 1);
17     a[0] = {Z(1)};
18     b[0] = 1;
19     for (int i = 1; i <= n; ++i) {
20         a[i].assign(i + 1, Z(0));
21         a[i][0] = a[i - 1][i - 1];
22         for (int j = 1; j <= i; ++j) a[i][j] = a[i][j - 1] + a[i - 1][j - 1];
23         b[i] = a[i][0];
24     }
25     return b;
26 }

```

03FA - 贝尔数（NTT快速求法）.cpp

```

1  /*
2  用途：用指数型生成函数快速求  $Bell(0..n)$ 。
3  依赖：02A - 自动取模（MInt）、08B 或 08C - 多项式（Poly, Z）。
4  公式： $\sum Bell(n)x^n/n! = \exp(\exp(x)-1)$ 。
5  复杂度： $O(n \log n)$ （由 Poly::exp 完成），比 03F 的  $O(n^2)$  适合大  $n$ 。
6  限制： $Z$  的模数须为 NTT 友好质数，且  $n < \text{mod}$ ；小  $n$  使用 03F 常数更小。
7  */
8  template <class Z, int G = 3> vector<Z> bellFast(int n) {
9      assert(0 <= n && n < Z::askMod());
10     vector<Z> iv(n + 1), f(n + 1), fac(n + 1);
11     fac[0] = 1;
12     for (int i = 1; i <= n; ++i) {
13         fac[i] = fac[i - 1] * Z(i);
14     }
15     iv[n] = fac[n].inv();
16     for (int i = n; i; --i) iv[i - 1] = iv[i] * Z(i);
17     f[0] = 0;
18     for (int i = 1; i <= n; ++i) f[i] = iv[i];
19     vector<Z> a = Poly<Z, G>::exp(f, n + 1);
20     for (int i = 0; i <= n; ++i) a[i] *= fac[i];
21     return a;
22 }

```

03G - 拉格朗日插值（任意横坐标）.cpp

```

1  /*
2  用途：给定互异横坐标点  $(x[i], y[i])$ ，在  $p$  处求值。
3  前提： $Z$  为质数模，横坐标两两不同。
4  */
5  template <class Z> Z lagr(const vector<Z>& x, const vector<Z>& y, Z p) {
6      int n = x.size();
7      assert(n && y.size() == n);
8      for (int i = 0; i < n; ++i) {
9          if (p == x[i]) return y[i];
10         for (int j = 0; j < i; ++j) assert(x[i] != x[j]);
11     }

```

```

12     Z r = 0;
13     for (int i = 0; i < n; ++i) {
14         Z a = 1, b = 1;
15         for (int j = 0; j < n; ++j) {
16             if (i == j) continue;
17             a *= p - x[j];
18             b *= x[i] - x[j];
19         }
20         r += y[i] * a / b;
21     }
22     return r;
23 }

```

03H - 拉格朗日插值（连续横坐标）.cpp

```

1  /*
2  用途：已知  $f(0..n-1)$ ，在任意整数  $x$  处求次数小于  $n$  的多项式值。
3  前提： $Z$  为质数模， $n < \text{mod}$ ；复杂度  $O(n)$ 。
4  */
5
6  template <class Z> Z lagr(const vector<Z>& a, int x) {
7      int n = a.size();
8      assert(n && n < Z::askMod());
9      if (0 <= x && x < n) return a[x];
10     Z p = x;
11     vector<Z> l(n + 1, Z(1)), r(n + 1, Z(1)), f(n, Z(1)), g(n, Z(1));
12     for (int i = 0; i < n; ++i) l[i + 1] = l[i] * (p - Z(i));
13     for (int i = n - 1; i >= 0; --i) r[i] = r[i + 1] * (p - Z(i));
14     for (int i = 1; i < n; ++i) f[i] = f[i - 1] * i;
15     g[n - 1] = f[n - 1].inv();
16     for (int i = n - 1; i; --i) g[i - 1] = g[i] * i;
17     Z ans = 0;
18     for (int i = 0; i < n; ++i) {
19         Z t = a[i] * l[i] * r[i + 1] * g[i] * g[n - 1 - i];
20         if ((n - 1 - i) & 1) t = -t;
21         ans += t;
22     }
23     return ans;
24 }

```

03I - 牛顿插值（差分级数）.cpp

```

1  /*
2  用途：整数横坐标  $0..n-1$  上的 Newton 前向差分级数。
3  构造  $O(n^2)$ ，单次 ask  $O(n)$ ； $Z$  为质数模且  $n < \text{mod}$ 。
4  */
5  template <class Z> struct Newton {
6      vector<Z> d;
7
8      Newton() = default;
9
10     explicit Newton(vector<Z> a) {
11         init(move(a));
12     }
13
14     void init(vector<Z> a) {
15         assert(a.size() < Z::askMod());
16         d.clear();
17         while (!a.empty()) {
18             d.push_back(a[0]);
19             for (int i = 0; i + 1 < a.size(); ++i) a[i] = a[i + 1] - a[i];
20             a.pop_back();
21         }
22     }
23
24     Z ask(Z x) const {
25         Z r = 0, c = 1;
26         for (int i = 0; i < d.size(); ++i) {
27             r += d[i] * c;
28             c *= (x - Z(i)) / Z(i + 1);
29         }
30         return r;
31     }
32 };

```

03J - 整数分拆（五边形数）.cpp

```

1  /*
2  用途: Euler 五边形数定理预处理整数分拆 p(0..n)。
3  Z 仅需支持加减; 复杂度 O(n sqrt n)。
4  */
5
6  template <class Z> vector<Z> part(int n) {
7      assert(n >= 0);
8      vector<Z> a(n + 1);
9      a[0] = 1;
10     for (int i = 1; i <= n; ++i) {
11         Z s = 0;
12         for (int k = 1;; ++k) {
13             int x = k * (3 * k - 1) / 2;
14             if (x > i) break;
15             Z t = k & 1 ? Z(1) : Z(-1);
16             s += t * a[i - x];
17             int y = k * (3 * k + 1) / 2;
18             if (y <= i) s += t * a[i - y];
19         }
20         a[i] = s;
21     }
22     return a;
23 }

```

03JA - 整数分拆（NTT快速求法）.cpp

```

1  /*
2  用途: 用形式幂级数快速求整数分拆 p(0..n), 即不计顺序的正整数分解。
3  依赖: 02A - 自动取模 (MInt)、08B 或 08C - 多项式 (Poly, Z)。
4  公式:  $\prod_{i=1}^{\infty} (1-x^i)^{-1} = \exp(\sum_{k=1}^{\infty} \sigma(k) x^k / k)$ 。
5  复杂度: O(n log n) (sigma 用约数筛, exp 由 Poly 完成); 小 n 使用 03J 常数更小。
6  限制: Z 的模数须为 NTT 友好质数, 且 n < mod。
7  */
8  template <class Z, int G = 3> vector<Z> partFast(int n) {
9      assert(0 <= n && n < Z::askMod());
10     vector<Z> f(n + 1);
11     for (int d = 1; d <= n; ++d)
12         for (int k = d; k <= n; k += d) f[k] += Z(d);
13     for (int k = 1; k <= n; ++k) f[k] /= Z(k);
14     return Poly<Z, G>::exp(f, n + 1);
15 }

```

03K - Berlekamp-Massey（最短线性递推）.cpp

```

1  /*
2  用途: 求最短递推 a[t] = sum c[i] a[t-1-i]。
3  前提: Z 为域; 返回的系数可直接交给 03KA。
4  */
5  template <class Z> vector<Z> BM(const vector<Z>& a) {
6      vector<Z> c{Z(1)}, b{Z(1)};
7      int l = 0, m = 1;
8      Z d = 1;
9      for (int i = 0; i < a.size(); ++i) {
10         Z x = a[i];
11         for (int j = 1; j <= l; ++j) x += c[j] * a[i - j];
12         if (x == Z(0)) {
13             ++m;
14             continue;
15         }
16         vector<Z> t = c;
17         Z q = x / d;
18         if (c.size() < b.size() + m) c.resize(b.size() + m);
19         for (int j = 0; j < b.size(); ++j) c[j + m] -= q * b[j];
20         if (l * 2 <= i) {

```

```

21         l = i + 1 - l;
22         b = move(t);
23         d = x;
24         m = 1;
25     } else {
26         ++m;
27     }
28 }
29 vector<Z> r(l);
30 for (int i = 0; i < l; ++i) r[i] = -c[i + 1];
31 return r;
32 }

```

03KA - 线性递推第 n 项 (Kitamasa) .cpp

```

1  /*
2  用途：已知递推系数  $c$  和前  $k$  项  $a$ ，在  $O(k^2 \log n)$  求第  $n$  项。
3  */
4
5  namespace LinearRec {
6  template <class Z> vector<Z> combine(const vector<Z>& a, const vector<Z>& b, const vector<Z>& c) {
7      int k = c.size();
8      vector<Z> t(k * 2 - 1);
9      for (int i = 0; i < k; ++i)
10         for (int j = 0; j < k; ++j) t[i + j] += a[i] * b[j];
11     for (int i = k * 2 - 2; i >= k; --i)
12         for (int j = 0; j < k; ++j) t[i - 1 - j] += t[i] * c[j];
13     t.resize(k);
14     return t;
15 }
16
17 template <class Z> Z ask(const vector<Z>& a, const vector<Z>& c, int n) {
18     int k = c.size();
19     assert(k && a.size() == k && n >= 0);
20     if (n < k) return a[n];
21     vector<Z> r(k), x(k);
22     r[0] = 1;
23     if (k == 1) x[0] = c[0];
24     else x[1] = 1;
25     while (n) {
26         if (n & 1) r = combine(r, x, c);
27         x = combine(x, x, c);
28         n >>= 1;
29     }
30     Z ans = 0;
31     for (int i = 0; i < k; ++i) ans += r[i] * a[i];
32     return ans;
33 }
34 } // namespace LinearRec

```

03L - Burnside 引理 (群作用计数) .cpp

```

1  /*
2  用途：群作用下的轨道数； $f(i)$  返回第  $i$  个变换的不动点数。
3  前提：群大小在  $Z$  中可逆。
4  */
5  namespace Burnside {
6  template <class Z, class F> Z burn(int n, F f) {
7      assert(n > 0);
8      Z s = 0;
9      for (int i = 0; i < n; ++i) s += f(i);
10     return s / Z(n);
11 }
12 } // namespace Burnside

```

03M - 快速阶乘 (NTT) .cpp

```

1  /*
2  用途：在 NTT 友好质数模下求  $n!$ 。依赖：02A - 自动取模 (MInt)、08B - 多项式 (Poly, Z)。
3  接口：FastFac<Z>::ask(n)。n>=mod 时直接返回 0；n<mod 时使用分块多点求值。
4  复杂度： $O(\sqrt{n} \log^2 n)$  时间、 $O(\sqrt{n})$  空间；小  $n$  直接循环通常更快。
5  */
6  template <class Z, int G = 3> struct FastFac {
7      static vector<Z> prod(int n) {
8          vector<vector<Z>> a;
9          a.reserve(n);

```

```

10     for (int i = 1; i <= n; ++i) a.push_back({Z(i), Z(1)});
11     while (a.size() > 1) {
12         vector<vector<Z>> b;
13         b.reserve((a.size() + 1) >> 1);
14         for (int i = 0; i < a.size(); i += 2) {
15             if (i + 1 == a.size()) b.push_back(move(a[i]));
16             else b.push_back(Poly<Z, G>::mul(a[i], a[i + 1]));
17         }
18         a = move(b);
19     }
20     return a.empty() ? vector<Z>{Z(1)} : move(a[0]);
21 }
22
23 static Z ask(int n) {
24     assert(n >= 0);
25     if (!n) return Z(1);
26     if (n >= Z::askMod()) return Z(0);
27     int b = sqrtl(n) + 1;
28     int q = n / b, r = n % b;
29     vector<Z> f = prod(b), x(q);
30     for (int i = 0; i < q; ++i) x[i] = Z(i * b);
31     Z ans = 1;
32     for (Z v : Poly<Z, G>::eval(f, x)) ans *= v;
33     for (int i = 1; i <= r; ++i) ans *= Z(q * b + i);
34     return ans;
35 }
36 };

```

04A - 容斥 (bitmask, 小 m) .cpp

```

1  /*
2  用途: 计算  $[1, n]$  中至少被一个 divisor 整除的数的个数。
3  复杂度:  $O(2^m * m)$ ,  $m$  为去重、去冗余后的 divisor 数, 要求  $m \leq 22$ 。
4  与 04B 是同一问题的两种实现;  $m$  较大且 lcm 常很快超过  $n$  时可改用 04B。
5  */
6
7  using i128 = __int128_t;
8
9  namespace IE {
10 inline vector<int> norm(vector<int> d) {
11     for (int val : d) assert(val > 0);
12     sort(d.begin(), d.end());
13     d.erase(unique(d.begin(), d.end()), d.end());
14     vector<int> res;
15     for (int val : d) {
16         bool bad = false;
17         for (int kp : res) {
18             if (val % kp == 0) {
19                 bad = true;
20                 break;
21             }
22         }
23         if (!bad) res.push_back(val);
24     }
25     return res;
26 }
27
28 inline bool lcm(int a, int b, int lim, int& res) {
29     int x = a / gcd(a, b);
30     if (x > lim / b) return false;
31     res = x * b;
32     return true;
33 }
34
35 inline int solve(int n, vector<int> d) {
36     assert(n >= 0);
37     d = norm(move(d));
38     int m = d.size();
39     assert(m <= 22);
40     i128 ans = 0;
41     for (int msk = 1; msk < (1ULL << m); ++msk) {
42         int cur = 1;
43         bool ok = true;
44         for (int i = 0; i < m; ++i) {
45             if ((msk >> i & 1) && !lcm(cur, d[i], n, cur)) {

```

```

46         ok = false;
47         break;
48     }
49 }
50 if (!ok) continue;
51 if (__builtin_popcountll(msk) & 1) ans += n / cur;
52 else ans -= n / cur;
53 }
54 assert(ans >= numeric_limits<int>::min() && ans <= numeric_limits<int>::max());
55 return ans;
56 }
57 } // namespace IE

```

04B - 容斥 (DFS 剪枝) .cpp

```

1  /*
2  用途: 计算 [1,n] 中至少被一个 divisor 整除的数的个数。
3  复杂度: 最坏  $O(2^m)$ , 但会在  $lcm > n$  时剪枝; 适合 divisor 较多且 lcm 很快超界的情形。
4  与 04A 是同一问题的两种实现;  $m \leq 22$  且希望实现路径固定时使用 04A。
5  */
6
7  using i128 = __int128_t;
8
9  namespace IE {
10 inline vector<int> norm(vector<int> d) {
11     for (int val : d) assert(val > 0);
12     sort(d.begin(), d.end());
13     d.erase(unique(d.begin(), d.end()), d.end());
14     vector<int> res;
15     for (int val : d) {
16         bool bad = false;
17         for (int kp : res) {
18             if (val % kp == 0) {
19                 bad = true;
20                 break;
21             }
22         }
23         if (!bad) res.push_back(val);
24     }
25     return res;
26 }
27
28 inline bool lcm(int a, int b, int lim, int& res) {
29     int x = a / gcd(a, b);
30     if (x > lim / b) return false;
31     res = x * b;
32     return true;
33 }
34
35 inline int solve(int n, vector<int> d) {
36     assert(n >= 0);
37     d = norm(move(d));
38     i128 ans = 0;
39     function<void(int, int, int)> dfs = [&](int st, int cur, int sgn) {
40         for (int i = st; i < d.size(); ++i) {
41             int nxt;
42             if (!lcm(cur, d[i], n, nxt)) continue;
43             ans += sgn * i128(n / nxt);
44             dfs(i + 1, nxt, -sgn);
45         }
46     };
47     dfs(0, 1, 1);
48     assert(ans >= numeric_limits<int>::min() && ans <= numeric_limits<int>::max());
49     return ans;
50 }
51 } // namespace IE

```

05 - 矩阵 (Matrix, with Int) .cpp

```

1
2 template <class T> struct Matrix {
3     int n, m;
4     vector<vector<T>>> a;
5
6     Matrix(int n = 0, int m = 0, T val = T()) : n(n), m(m), a(n, vector<T>(m, val)) {}
7
8     static Matrix unit(int n) {
9         Matrix I(n, n);

```

```

10     for (int i = 0; i < n; i++) I.a[i][i] = T(1);
11     return I;
12 }
13
14 vector<T> &operator[](int i) {
15     return a[i];
16 }
17 const vector<T> &operator[](int i) const {
18     return a[i];
19 }
20
21 Matrix operator+(const Matrix &o) const {
22     assert(n == o.n && m == o.m);
23     Matrix res(n, m);
24     for (int i = 0; i < n; i++)
25         for (int j = 0; j < m; j++) res[i][j] = a[i][j] + o[i][j];
26     return res;
27 }
28
29 Matrix operator-(const Matrix &o) const {
30     assert(n == o.n && m == o.m);
31     Matrix res(n, m);
32     for (int i = 0; i < n; i++)
33         for (int j = 0; j < m; j++) res[i][j] = a[i][j] - o[i][j];
34     return res;
35 }
36
37 Matrix operator*(const Matrix &o) const {
38     assert(m == o.n);
39     Matrix res(n, o.m, T(0));
40     for (int i = 0; i < n; i++)
41         for (int k = 0; k < m; k++)
42             for (int j = 0; j < o.m; j++) res[i][j] = res[i][j] + a[i][k] * o[k][j];
43     return res;
44 }
45
46 Matrix operator*(const T &k) const {
47     Matrix res(n, m);
48     for (int i = 0; i < n; i++)
49         for (int j = 0; j < m; j++) res[i][j] = a[i][j] * k;
50     return res;
51 }
52
53 Matrix trans() const {
54     Matrix res(m, n);
55     for (int i = 0; i < n; i++)
56         for (int j = 0; j < m; j++) res[j][i] = a[i][j];
57     return res;
58 }
59
60 Matrix ksm(int exp) const {
61     assert(n == m);
62     assert(exp >= 0);
63     Matrix bas = *this;
64     Matrix res = unit(n);
65     while (exp > 0) {
66         if (exp & 1) res = res * bas;
67         bas = bas * bas;
68         exp >>= 1;
69     }
70     return res;
71 }
72 };
73
74 template <class T> T det(int n, Matrix<T> &mat) {
75     T det = T(1);
76     int sgn = 1;
77
78     for (int i = 0; i < n; i++) {
79         int piv = i;
80         while (piv < n && mat[piv][i] == T()) piv++;
81         if (piv == n) return T();
82
83         if (piv != i) {
84             swap(mat[i], mat[piv]);
85             sgn = -sgn;
86         }
87
88         det *= mat[i][i];
89         for (int j = i + 1; j < n; j++) {
90             if (!(mat[j][i] == T())) {
91                 T k = mat[j][i] / mat[i][i];

```



```

92         for (int k = i; k < n; k++) {
93             mat[j][k] -= k * mat[i][k];
94         }
95     }
96 }
97 }
98
99 if (sgn == -1) {
100     det = T() - det;
101 }
102 return det;
103 }

```

05A - Min-Plus矩阵 (MinPlusMatrix, with Int) .cpp

```

1 // 性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
2
3 template <int N, class T = int> struct Matrix {
4     static constexpr T INF = numeric_limits<T>::max() / 4;
5     array<array<T, N>, N> a;
6
7     Matrix(T v = INF) {
8         for (int i = 0; i < N; ++i) a[i].fill(v);
9     }
10
11     static Matrix unit() {
12         Matrix res;
13         for (int i = 0; i < N; ++i) res[i][i] = 0;
14         return res;
15     }
16
17     array<T, N> &operator[](int i) {
18         return a[i];
19     }
20     const array<T, N> &operator[](int i) const {
21         return a[i];
22     }
23
24     Matrix operator+(const Matrix &o) const {
25         Matrix res;
26         for (int i = 0; i < N; ++i)
27             for (int j = 0; j < N; ++j) res[i][j] = min(a[i][j], o[i][j]);
28         return res;
29     }
30
31     Matrix operator*(const Matrix &o) const {
32         Matrix res;
33         for (int i = 0; i < N; ++i) {
34             for (int k = 0; k < N; ++k) {
35                 if (a[i][k] >= INF) continue;
36                 for (int j = 0; j < N; ++j) {
37                     if (o[k][j] >= INF) continue;
38                     res[i][j] = min(res[i][j], a[i][k] + o[k][j]);
39                 }
40             }
41         }
42         return res;
43     }
44
45     Matrix &operator+=(const Matrix &o) {
46         return *this = *this + o;
47     }
48     Matrix &operator*=(const Matrix &o) {
49         return *this = *this * o;
50     }
51
52     Matrix ksm(int b) const {
53         assert(b >= 0);
54         Matrix a = *this, res = unit();
55         while (b) {
56             if (b & 1) res *= a;
57             a *= a;
58             b >>= 1;
59         }
60         return res;
61     }
62 };

```

05B - 高斯消元（一般线性方程组）.cpp

```

1  /*
2  用途：求一般实系数  $m$  元线性方程组，区分唯一解、无穷多解、无解。
3  复杂度： $O(m^2 * rows)$ 。浮点题请按数据范围调整 eps；精确模意义方程不能直接套用。
4  与 05C 都用于线性方程组，但 05C 只适合对称、无零主元的特殊矩阵。
5  */
6  using ld = long double;
7
8  enum class GaussStat { Unique, Infinite, Inconsistent };
9
10 struct GaussRes {
11     GaussStat st;
12     vector<ld> sol; // Infinite 时给出一个自由元均为 0 的特解。
13     vector<int> pc;
14 };
15
16 inline GaussRes gauss(vector<vector<ld>> a, ld eps = 1e-12L) {
17     int n = a.size();
18     if (n == 0) return {GaussStat::Unique, {}, {}};
19     int m = a[0].size() - 1;
20     assert(m >= 0);
21     for (const auto& row : a) assert(row.size() == m + 1);
22
23     vector<int> pc;
24     int pr = 0;
25     for (int c = 0; c < m && pr < n; ++c) {
26         int vis = pr;
27         for (int row = pr + 1; row < n; ++row) {
28             if (fabsl(a[row][c]) > fabsl(a[vis][c])) vis = row;
29         }
30         if (fabsl(a[vis][c]) <= eps) continue;
31         swap(a[vis], a[pr]);
32         ld iv = 1 / a[pr][c];
33         for (int j = c; j <= m; ++j) a[pr][j] *= iv;
34         for (int row = 0; row < n; ++row) {
35             if (row == pr || fabsl(a[row][c]) <= eps) continue;
36             ld coe = a[row][c];
37             for (int j = c; j <= m; ++j) a[row][j] -= coe * a[pr][j];
38         }
39         pc.push_back(c);
40         ++pr;
41     }
42
43     for (int row = pr; row < n; ++row) {
44         if (fabsl(a[row][m]) > eps) return {GaussStat::Inconsistent, {}, pc};
45     }
46     vector<ld> sol(m, 0);
47     for (int row = 0; row < pr; ++row) sol[pc[row]] = a[row][m];
48     return {pr == m ? GaussStat::Unique : GaussStat::Infinite, sol, pc};
49 }

```

05C - 对称 LDLT 分解（线性方程组）.cpp

```

1  /*
2  用途：以  $LDL^T$  分解求对称线性方程组。
3  复杂度： $O(n^3)$ 。不做主元交换，要求各顺序主子式不出现零枢轴；否则使用 05B。
4  与 05B 是同一问题的不同实现：本模板常数和内存更小，但适用条件更强。
5  */
6  using ld = long double;
7
8  inline optional<vector<ld>> solve(
9     const vector<vector<ld>>& mat, const vector<ld>& rhs, ld eps = 1e-12L) {
10     int n = mat.size();
11     if (rhs.size() != n) return nullopt;
12     for (const auto& row : mat) if (row.size() != n) return nullopt;
13
14     vector<vector<ld>> lo(n, vector<ld>(n, 0));
15     vector<ld> dia(n);
16     for (int i = 0; i < n; ++i) {
17         for (int j = 0; j < i; ++j) {
18             ld val = mat[i][j];
19             for (int k = 0; k < j; ++k) val -= lo[i][k] * dia[k] * lo[j][k];
20             if (fabsl(dia[j]) <= eps) return nullopt;
21             lo[i][j] = val / dia[j];
22         }
23         ld val = mat[i][i];
24         for (int k = 0; k < i; ++k) val -= lo[i][k] * lo[i][k] * dia[k];
25         if (fabsl(val) <= eps) return nullopt;

```

```

26     dia[i] = val;
27     lo[i][i] = 1;
28 }
29
30 vector<ld> y(n), z(n), ans(n);
31 for (int i = 0; i < n; ++i) {
32     y[i] = rhs[i];
33     for (int j = 0; j < i; ++j) y[i] -= lo[i][j] * y[j];
34     z[i] = y[i] / dia[i];
35 }
36 for (int i = n - 1; i >= 0; --i) {
37     ans[i] = z[i];
38     for (int j = i + 1; j < n; ++j) ans[i] -= lo[j][i] * ans[j];
39 }
40 return ans;
41 }

```

05D - 高斯消元 (模质数) .cpp

```

1  /*
2  用途: 质数模域上的线性方程组, 区分唯一解、无穷多解、无解。
3  依赖: 02A - 自动取模 (MInt)。
4  */
5  enum class GaussStat { Unique, Infinite, Inconsistent };
6
7  template <class Z> struct GaussRes {
8      GaussStat st;
9      vector<Z> x;
10     vector<int> p;
11 };
12
13 template <class Z> GaussRes<Z> gauss(vector<vector<Z>> a) {
14     int n = a.size();
15     if (!n) return {GaussStat::Unique, {}, {}};
16     int m = a[0].size() - 1;
17     assert(m >= 0);
18     for (const auto& v : a) assert(v.size() == m + 1);
19
20     vector<int> p;
21     int r = 0;
22     for (int c = 0; c < m && r < n; ++c) {
23         int x = r;
24         while (x < n && a[x][c] == Z(0)) ++x;
25         if (x == n) continue;
26         swap(a[x], a[r]);
27         Z iv = a[r][c].inv();
28         for (int j = c; j <= m; ++j) a[r][j] *= iv;
29         for (int i = 0; i < n; ++i) {
30             if (i == r || a[i][c] == Z(0)) continue;
31             Z z = a[i][c];
32             for (int j = c; j <= m; ++j) a[i][j] -= z * a[r][j];
33         }
34         p.push_back(c);
35         ++r;
36     }
37     for (int i = r; i < n; ++i)
38         if (a[i][m] != Z(0)) return {GaussStat::Inconsistent, {}, p};
39     vector<Z> x(m);
40     for (int i = 0; i < r; ++i) x[p[i]] = a[i][m];
41     return {r == m ? GaussStat::Unique : GaussStat::Infinite, x, p};
42 }

```

05E - 行列式 (模质数) .cpp

```

1  /*
2  用途: 域上的方阵行列式; MInt 时模数须为质数。
3  */
4  template <class Z> Z det(vector<vector<Z>> a) {
5      int n = a.size();
6      for (const auto& v : a) assert(v.size() == n);
7      Z r = 1;
8      for (int c = 0; c < n; ++c) {
9          int p = c;
10         while (p < n && a[p][c] == Z(0)) ++p;
11         if (p == n) return Z(0);
12         if (p != c) {
13             swap(a[p], a[c]);
14             r = -r;

```

```

15     }
16     Z x = a[c][c];
17     r *= x;
18     Z iv = x.inv();
19     for (int i = c + 1; i < n; ++i) {
20         if (a[i][c] == Z(0)) continue;
21         Z q = a[i][c] * iv;
22         for (int j = c; j < n; ++j) a[i][j] -= q * a[c][j];
23     }
24 }
25 return r;
26 }

```

05F - 线性规划 (单纯形) .cpp

```

1  /*
2  用途: 求  $\max c \cdot x$ , 满足  $A \cdot x \leq b, x \geq 0$  的浮点线性规划。
3  返回最优、无界或无解; 使用两阶段单纯形法。浮点数据请按量级调整 EPS。
4  复杂度: 单纯形最坏指数, 竞赛中的中小规模通常可用; 不能处理整数规划。
5  */
6  using ld = long double;
7
8  enum class LPStatus { Optimal, Infeasible, Unbounded };
9
10 struct LPres {
11     LPStatus st;
12     ld ans = 0;
13     vector<ld> sol;
14 };
15
16 struct Simplex {
17     static constexpr ld EPS = 1e-12L;
18     int n, m;
19     vector<int> bas, nb;
20     vector<vector<ld>> a;
21
22     Simplex(const vector<vector<ld>>& A, const vector<ld>& b,
23             const vector<ld>& c)
24         : n(b.size()), m(c.size()), bas(n),
25           nb(m + 1), a(n + 2, vector<ld>(m + 2, 0)) {
26         assert(A.size() == n);
27         for (int i = 0; i < n; ++i) {
28             assert(A[i].size() == m);
29             for (int j = 0; j < m; ++j) a[i][j] = A[i][j];
30         }
31         for (int i = 0; i < n; ++i) {
32             bas[i] = m + i;
33             a[i][m] = -1;
34             a[i][m + 1] = b[i];
35         }
36         for (int j = 0; j < m; ++j) {
37             nb[j] = j;
38             a[n][j] = -c[j];
39         }
40         nb[m] = -1;
41         a[n + 1][m] = 1;
42     }
43
44     void pivot(int row, int c) {
45         ld inv = 1 / a[row][c];
46         for (int i = 0; i < n + 2; ++i) {
47             if (i == row) continue;
48             for (int j = 0; j < m + 2; ++j) {
49                 if (j == c) continue;
50                 a[i][j] -= a[row][j] * a[i][c] * inv;
51             }
52         }
53         for (int j = 0; j < m + 2; ++j) if (j != c) a[row][j] *= inv;
54         for (int i = 0; i < n + 2; ++i) if (i != row) a[i][c] *= -inv;
55         a[row][c] = inv;
56         swap(bas[row], nb[c]);
57     }
58
59     bool simplex(int ph) {
60         int obj = ph == 1 ? n + 1 : n;
61         while (true) {
62             int in = -1;
63             for (int c = 0; c <= m; ++c) {
64                 if (ph == 2 && nb[c] == -1) continue;
65                 if (in == -1 || a[obj][c] < a[obj][in] - EPS ||

```

```

66         (fabsl(a[obj][c] - a[obj][in]) <= EPS &&
67         nb[c] < nb[in])) {
68             in = c;
69         }
70     }
71     if (a[obj][in] >= -EPS) return true;
72
73     int out = -1;
74     for (int row = 0; row < n; ++row) {
75         if (a[row][in] <= EPS) continue;
76         if (out == -1 ||
77             a[row][m + 1] / a[row][in] <
78             a[out][m + 1] / a[out][in] - EPS ||
79             (fabsl(a[row][m + 1] / a[row][in] -
80                 a[out][m + 1] / a[out][in]) <= EPS &&
81                 bas[row] < bas[out])) {
82             out = row;
83         }
84     }
85     if (out == -1) return false;
86     pivot(out, in);
87 }
88
89
90 LPRes solve() {
91     if (n == 0) {
92         for (int j = 0; j < m; ++j)
93             if (a[n][j] < -EPS) return {LPStatus::Unbounded, 0, {}};
94         return {LPStatus::Optimal, 0, vector<ld>(m, 0)};
95     }
96
97     int row = 0;
98     for (int i = 1; i < n; ++i)
99         if (a[i][m + 1] < a[row][m + 1]) row = i;
100     if (a[row][m + 1] < -EPS) {
101         pivot(row, m);
102         if (!simplex(1) || a[n + 1][m + 1] < -EPS) {
103             return {LPStatus::Infeasible, 0, {}};
104         }
105         if (fabsl(a[n + 1][m + 1]) > EPS) {
106             return {LPStatus::Infeasible, 0, {}};
107         }
108         for (int i = 0; i < n; ++i) {
109             if (bas[i] != -1) continue;
110             int c = 0;
111             for (int j = 1; j <= m; ++j) {
112                 if (a[i][j] < a[i][c] - EPS ||
113                     (fabsl(a[i][j] - a[i][c]) <= EPS && nb[j] < nb[c])) {
114                     c = j;
115                 }
116             }
117             pivot(i, c);
118         }
119     }
120     if (!simplex(2)) return {LPStatus::Unbounded, 0, {}};
121     vector<ld> sol(m, 0);
122     for (int i = 0; i < n; ++i) {
123         if (bas[i] < m) sol[bas[i]] = a[i][m + 1];
124     }
125     return {LPStatus::Optimal, a[n][m + 1], sol};
126 }
127 };

```

05G - 高斯消元 (GF(2) 位集) .cpp

```

1  /*
2  用途: 在 GF(2) 上求解线性方程组。每行 bitset 的 [0,variables) 为系数, variables 位为右端常数。
3  前提: variables <= MAX_VARIABLES; 加减与乘法均为 xor/and。
4  复杂度: O(rows * variables * MAX_VARIABLES / word_bits), 适合布尔方程、异或约束。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  enum class GaussStat { Unique, Infinite, Inconsistent };
8
9  template <int M> struct GaussRes {
10     GaussStat st;
11     vector<int> sol; // Infinite 时自由元取 0 的一组特解。
12     vector<int> pc;
13 };
14
15 template <int M>

```

```

16 GaussRes<M> gauss(vector<bitset<M + 1>> a,
17                                     int m) {
18     assert(0 <= m && m <= M);
19     int n = a.size();
20     vector<int> pc;
21     int pr = 0;
22     for (int c = 0; c < m && pr < n; ++c) {
23         int vis = pr;
24         while (vis < n && !a[vis][c]) ++vis;
25         if (vis == n) continue;
26         swap(a[vis], a[pr]);
27         for (int row = 0; row < n; ++row) {
28             if (row != pr && a[row][c]) a[row] ^= a[pr];
29         }
30         pc.push_back(c);
31         ++pr;
32     }
33     for (int row = pr; row < n; ++row) {
34         bool has = false;
35         for (int c = 0; c < m; ++c) has = has || a[row][c];
36         if (!has && a[row][m]) return {GaussStat::Inconsistent, {}, pc};
37     }
38     vector<int> sol(m, 0);
39     for (int row = 0; row < pr; ++row) sol[pc[row]] = a[row][m];
40     return {pr == m ? GaussStat::Unique : GaussStat::Infinite, sol, pc};
41 }

```

05H - 矩阵树定理 (Kirchhoff) .cpp

```

1  /*
2  用途: 无向带权图生成树数。
3  依赖: 05E - 行列式 (模质数); Z 通常为 MInt<P>。
4  */
5  template <class Z> Z kirch(int n, const vector<tuple<int, int, Z>>& e) {
6      assert(n >= 0);
7      if (n <= 1) return Z(1);
8      vector<vector<Z>> a(n, vector<Z>(n));
9      for (auto [u, v, w] : e) {
10         assert(0 <= u && u < n && 0 <= v && v < n);
11         if (u == v) continue;
12         a[u][u] += w;
13         a[v][v] += w;
14         a[u][v] -= w;
15         a[v][u] -= w;
16     }
17     vector<vector<Z>> b(n - 1, vector<Z>(n - 1));
18     for (int i = 0; i + 1 < n; ++i)
19         for (int j = 0; j + 1 < n; ++j) b[i][j] = a[i][j];
20     return det(move(b));
21 }

```

06 - 分数 (Frac, with Int) .cpp

```

1  template <class T> struct Frac {
2      T x, y;
3      Frac() : x(0), y(1) {}
4      Frac(T a, T b) : x(a), y(b) {
5          assert(y != 0);
6          T g = gcd(x, y);
7          assert(g != 0);
8          x /= g, y /= g;
9          if (y < 0) x = -x, y = -y;
10     }
11
12     constexpr double val() const {
13         return 1.0 * x / y;
14     }
15
16     friend ostream &operator<<(ostream &o, const Frac &b) {
17         T g = gcd(b.x, b.y);
18         if (b.y == g) {
19             return o << b.x / g;
20         } else {
21             return o << b.x / g << "/" << b.y / g;
22         }
23     }
24     Frac &operator+=(const Frac o) {
25         *this = Frac(x * o.y + y * o.x, y * o.y);

```

```

26     return *this;
27 }
28 Frac &operator--=(const Frac o) {
29     *this = Frac(x * o.y - y * o.x, y * o.y);
30     return *this;
31 }
32 Frac &operator*=(const Frac o) {
33     *this = Frac(x * o.x, y * o.y);
34     return *this;
35 }
36 Frac &operator/=(const Frac o) {
37     assert(o.x != 0);
38     *this = Frac(x * o.y, y * o.x);
39     return *this;
40 }
41
42 constexpr Frac operator-() const {
43     return Frac(-x, y);
44 }
45 friend Frac operator+(Frac a, const Frac &b) {
46     return a += b;
47 }
48 friend Frac operator-(Frac a, const Frac &b) {
49     return a -= b;
50 }
51 friend Frac operator*(Frac a, const Frac &b) {
52     return a *= b;
53 }
54 friend Frac operator/(Frac a, const Frac &b) {
55     return a /= b;
56 }
57 friend bool operator<(const Frac &a, const Frac &b) {
58     return a.x * b.y < a.y * b.x;
59 }
60 friend bool operator>(const Frac &a, const Frac &b) {
61     return a.x * b.y > a.y * b.x;
62 }
63 friend bool operator==(const Frac &a, const Frac &b) {
64     return a.x * b.y == a.y * b.x;
65 }
66 friend bool operator!=(const Frac &a, const Frac &b) {
67     return a.x * b.y != a.y * b.x;
68 }
69 };

```

06A - 复数 (Complex, with Int) .cpp

```

1  template <typename T> class Complex {
2      public:
3          T a, b;
4
5          constexpr Complex(T r = 0, T i = 0) : a(r), b(i) {}
6
7          constexpr Complex operator+(const Complex &rhs) const {
8              return {a + rhs.a, b + rhs.b};
9          }
10
11         constexpr Complex operator-(const Complex &rhs) const {
12             return {a - rhs.a, b - rhs.b};
13         }
14
15         constexpr Complex operator*(const Complex &rhs) const {
16             return {a * rhs.a - b * rhs.b, a * rhs.b + b * rhs.a};
17         }
18
19         Complex operator/(const Complex &rhs) const {
20             T den = rhs.a * rhs.a + rhs.b * rhs.b;
21             assert(den != 0);
22             return {(a * rhs.a + b * rhs.b) / den, (b * rhs.a - a * rhs.b) / den};
23         }
24
25         Complex inv() const {
26             T den = a * a + b * b;
27             assert(den != 0);
28             return Complex(a / den, -b / den);
29         }
30
31         Complex &operator+=(const Complex &rhs) {
32             *this = (*this) + rhs;
33             return *this;

```

```

34     }
35
36     Complex &operator--(const Complex &rhs) {
37         *this = (*this) - rhs;
38         return *this;
39     }
40
41     Complex &operator*=(const Complex &rhs) {
42         *this = (*this) * rhs;
43         return *this;
44     }
45
46     Complex &operator/=(const Complex &rhs) {
47         *this = (*this) / rhs;
48         return *this;
49     }
50
51     T norm() const {
52         return a * a + b * b;
53     }
54
55     bool operator==(const Complex &rhs) const {
56         return a == rhs.a && b == rhs.b;
57     }
58
59     bool operator!=(const Complex &rhs) const {
60         return !(*this == rhs);
61     }
62
63     bool operator<(const Complex &rhs) const {
64         return norm() < rhs.norm();
65     }
66
67     bool operator>(const Complex &rhs) const {
68         return norm() > rhs.norm();
69     }
70
71     friend ostream &operator<<(ostream &os, const Complex &c) {
72         os << "(" << c.a;
73         if (c.b >= 0) os << "+";
74         os << c.b << "i";
75         return os;
76     }
77 };
78
79 template <class T> Complex<T> conj(const Complex<T> &z) {
80     return Complex<T>(z.a, -z.b);
81 }

```

06B - 三次单位根复数 (Complex3) .cpp

```

1 // x^3 = 1, x != 1
2 template <class T> struct Complex3 {
3     T a, b;
4     Complex3(T a = T(), T b = T()) : a(a), b(b) {}
5
6     Complex3 operator+(const Complex3 &rhs) const {
7         return {a + rhs.a, b + rhs.b};
8     }
9
10    Complex3 operator-(const Complex3 &rhs) const {
11        return {a - rhs.a, b - rhs.b};
12    }
13
14    Complex3 operator*(const Complex3 &rhs) const {
15        T ac = a * rhs.a;
16        T bd = b * rhs.b;
17        T ad = a * rhs.b;
18        T bc = b * rhs.a;
19
20        return {ac - bd, ad + bc - bd};
21    }
22
23    Complex3 operator/(const Complex3 &rhs) const {
24        return (*this) * rhs.inv();
25    }
26
27    Complex3 inv() const {
28        T z = a * a - a * b + b * b;
29        assert(z != 0);

```



```

30     return Complex3<T>((a - b) / z, -b / z);
31 }
32
33 Complex3 &operator+=(const Complex3 &rhs) {
34     *this = (*this) + rhs;
35     return *this;
36 }
37
38 Complex3 &operator-=(const Complex3 &rhs) {
39     *this = (*this) - rhs;
40     return *this;
41 }
42
43 Complex3 &operator*=(const Complex3 &rhs) {
44     *this = (*this) * rhs;
45     return *this;
46 }
47
48 Complex3 &operator/=(const Complex3 &rhs) {
49     *this = (*this) / rhs;
50     return *this;
51 }
52 bool ask0() const {
53     return a == 0 && b == 0;
54 }
55 };

```

07 - 平面几何 (Point&Line, with Int) .cpp

```

1  using ld = long double;
2
3  template <class T> struct Point {
4      T x;
5      T y;
6      Point(const T &_x = 0, const T &_y = 0) : x(_x), y(_y) {}
7
8      template <class U> operator Point<U>() {
9          return Point<U>(U(x), U(y));
10     }
11     Point &operator+=(const Point &p) & {
12         x += p.x;
13         y += p.y;
14         return *this;
15     }
16     Point &operator-=(const Point &p) & {
17         x -= p.x;
18         y -= p.y;
19         return *this;
20     }
21     Point &operator*=(const T &v) & {
22         x *= v;
23         y *= v;
24         return *this;
25     }
26     Point &operator/=(const T &v) & {
27         x /= v;
28         y /= v;
29         return *this;
30     }
31     Point operator-() const {
32         return Point(-x, -y);
33     }
34     friend Point operator+(Point a, const Point &b) {
35         return a += b;
36     }
37     friend Point operator-(Point a, const Point &b) {
38         return a -= b;
39     }
40     friend Point operator*(Point a, const T &b) {
41         return a *= b;
42     }
43     friend Point operator/(Point a, const T &b) {
44         return a /= b;
45     }
46     friend Point operator*(const T &a, Point b) {
47         return b *= a;
48     }
49     friend bool operator==(const Point &a, const Point &b) {
50         return a.x == b.x && a.y == b.y;
51     }

```

```

52     friend bool operator!=(const Point &a, const Point &b) {
53         return !(a == b);
54     }
55     friend istream &operator>>(istream &is, Point &p) {
56         return is >> p.x >> p.y;
57     }
58     friend ostream &operator<<(ostream &os, const Point &p) {
59         return os << "(" << p.x << ", " << p.y << ")";
60     }
61 };
62
63 template <class T> struct Line {
64     Point<T> a;
65     Point<T> b;
66     Line(const Point<T> &a = Point<T>(), const Point<T> &b = Point<T>()) : a(_a), b(_b) {}
67 };
68
69 template <class T> T dot(const Point<T> &a, const Point<T> &b) {
70     return a.x * b.x + a.y * b.y;
71 }
72
73 template <class T> T cross(const Point<T> &a, const Point<T> &b) {
74     return a.x * b.y - a.y * b.x;
75 }
76
77 template <class T> T square(const Point<T> &p) {
78     return dot(p, p);
79 }
80
81 template <class T> double length(const Point<T> &p) {
82     return sqrt(square(p));
83 }
84
85 template <class T> double length(const Line<T> &l) {
86     return length(l.a - l.b);
87 }
88
89 template <class T> Point<T> norm(const Point<T> &p) {
90     return p / length(p);
91 }
92
93 template <class T> bool isPara(const Line<T> &l1, const Line<T> &l2) {
94     return cross(l1.b - l1.a, l2.b - l2.a) == 0;
95 }
96
97 template <class T> double dis(const Point<T> &a, const Point<T> &b) {
98     return length(a - b);
99 }
100
101 template <class T> double disPL(const Point<T> &p, const Line<T> &l) {
102     return abs(cross(l.a - l.b, l.a - p)) / length(l);
103 }
104
105 template <class T> double disPS(const Point<T> &p, const Line<T> &l) {
106     if (dot(p - l.a, l.b - l.a) < 0) {
107         return dis(p, l.a);
108     }
109     if (dot(p - l.b, l.a - l.b) < 0) {
110         return dis(p, l.b);
111     }
112     return disPL(p, l);
113 }
114
115 template <class T> Point<T> rotate(const Point<T> &a) {
116     return Point(-a.y, a.x);
117 }
118
119 template <class T> int sgn(const Point<T> &a) {
120     return a.y > 0 || (a.y == 0 && a.x > 0) ? 1 : -1;
121 }
122
123 template <class T> bool side(const Point<T> &p, const Line<T> &l) {
124     return cross(l.b - l.a, p - l.a) > 0;
125 }
126
127 template <class T> Point<T> inter(const Line<T> &l1, const Line<T> &l2) {
128     return l1.a + (l1.b - l1.a) * (cross(l2.b - l2.a, l1.a - l2.a) / cross(l2.b - l2.a, l1.a - l1.b));
129 }
130
131 template <class T> bool onSeg(const Point<T> &p, const Line<T> &l) {
132     return cross(p - l.a, l.b - l.a) == 0 && min(l.a.x, l.b.x) <= p.x && p.x <= max(l.a.x, l.b.x) && min(l.a.y,
l.b.y) <= p.y && p.y <= max(l.a.y, l.b.y);

```

```

133 }
134
135 template <class T> bool inPoly(const Point<T> &a, const vector<Point<T>> &p) {
136     int n = p.size();
137     for (int i = 0; i < n; i++) {
138         if (onSeg(a, Line(p[i], p[(i + 1) % n]))) {
139             return true;
140         }
141     }
142
143     int t = 0;
144     for (int i = 0; i < n; i++) {
145         auto u = p[i];
146         auto v = p[(i + 1) % n];
147         if (u.x < a.x && v.x >= a.x && side(a, Line(v, u))) {
148             t ^= 1;
149         }
150         if (u.x >= a.x && v.x < a.x && side(a, Line(u, v))) {
151             t ^= 1;
152         }
153     }
154
155     return t == 1;
156 }
157
158 template <class T> tuple<int, Point<T>, Point<T>> segInt(const Line<T> &l1, const Line<T> &l2) {
159     if (max(l1.a.x, l1.b.x) < min(l2.a.x, l2.b.x)) {
160         return {0, Point<T>(), Point<T>()};
161     }
162     if (min(l1.a.x, l1.b.x) > max(l2.a.x, l2.b.x)) {
163         return {0, Point<T>(), Point<T>()};
164     }
165     if (max(l1.a.y, l1.b.y) < min(l2.a.y, l2.b.y)) {
166         return {0, Point<T>(), Point<T>()};
167     }
168     if (min(l1.a.y, l1.b.y) > max(l2.a.y, l2.b.y)) {
169         return {0, Point<T>(), Point<T>()};
170     }
171     if (cross(l1.b - l1.a, l2.b - l2.a) == 0) {
172         if (cross(l1.b - l1.a, l2.a - l1.a) != 0) {
173             return {0, Point<T>(), Point<T>()};
174         } else {
175             auto xx1 = max(l1.a.x, l1.b.x);
176             auto nx1 = min(l1.a.x, l1.b.x);
177             auto yy1 = max(l1.a.y, l1.b.y);
178             auto ny1 = min(l1.a.y, l1.b.y);
179             auto xx2 = max(l2.a.x, l2.b.x);
180             auto nx2 = min(l2.a.x, l2.b.x);
181             auto yy2 = max(l2.a.y, l2.b.y);
182             auto ny2 = min(l2.a.y, l2.b.y);
183             Point<T> p1(max(nx1, nx2), max(ny1, ny2));
184             Point<T> p2(min(xx1, xx2), min(yy1, yy2));
185             if (!onSeg(p1, l1)) {
186                 swap(p1.y, p2.y);
187             }
188             if (p1 == p2) {
189                 return {3, p1, p2};
190             } else {
191                 return {2, p1, p2};
192             }
193         }
194     }
195     auto cp1 = cross(l2.a - l1.a, l2.b - l1.a);
196     auto cp2 = cross(l2.a - l1.b, l2.b - l1.b);
197     auto cp3 = cross(l1.a - l2.a, l1.b - l2.a);
198     auto cp4 = cross(l1.a - l2.b, l1.b - l2.b);
199
200     if ((cp1 > 0 && cp2 > 0) || (cp1 < 0 && cp2 < 0) || (cp3 > 0 && cp4 > 0) || (cp3 < 0 && cp4 < 0)) {
201         return {0, Point<T>(), Point<T>()};
202     }
203
204     Point p = inter(l1, l2);
205     if (cp1 != 0 && cp2 != 0 && cp3 != 0 && cp4 != 0) {
206         return {1, p, p};
207     } else {
208         return {3, p, p};
209     }
210 }
211
212 template <class T> double disSS(const Line<T> &l1, const Line<T> &l2) {
213     if (get<0>(segInt(l1, l2)) != 0) {
214         return 0.0;

```

```

215     }
216     return min({disPS(l1.a, l2), disPS(l1.b, l2), disPS(l2.a, l1), disPS(l2.b, l1)});
217 }
218
219 template <class T> bool segIn(const Line<T> &l, const vector<Point<T>> &p) {
220     int n = p.size();
221     if (!inPoly(l.a, p)) {
222         return false;
223     }
224     if (!inPoly(l.b, p)) {
225         return false;
226     }
227     for (int i = 0; i < n; i++) {
228         auto u = p[i];
229         auto v = p[(i + 1) % n];
230         auto w = p[(i + 2) % n];
231         auto [t, p1, p2] = segInt(l, Line(u, v));
232
233         if (t == 1) {
234             return false;
235         }
236         if (t == 0) {
237             continue;
238         }
239         if (t == 2) {
240             if (onSeg(v, l) && v != l.a && v != l.b) {
241                 if (cross(v - u, w - v) > 0) {
242                     return false;
243                 }
244             }
245         } else {
246             if (p1 != u && p1 != v) {
247                 if (side(l.a, Line(v, u)) || side(l.b, Line(v, u))) {
248                     return false;
249                 }
250             } else if (p1 == v) {
251                 if (l.a == v) {
252                     if (side(u, l)) {
253                         if (side(w, l) && side(w, Line(u, v))) {
254                             return false;
255                         }
256                     } else {
257                         if (side(w, l) || side(w, Line(u, v))) {
258                             return false;
259                         }
260                     }
261                 } else if (l.b == v) {
262                     if (side(u, Line(l.b, l.a))) {
263                         if (side(w, Line(l.b, l.a)) && side(w, Line(u, v))) {
264                             return false;
265                         }
266                     } else {
267                         if (side(w, Line(l.b, l.a)) || side(w, Line(u, v))) {
268                             return false;
269                         }
270                     }
271                 } else {
272                     if (side(u, l)) {
273                         if (side(w, Line(l.b, l.a)) || side(w, Line(u, v))) {
274                             return false;
275                         }
276                     } else {
277                         if (side(w, l) || side(w, Line(u, v))) {
278                             return false;
279                         }
280                     }
281                 }
282             }
283         }
284     }
285     return true;
286 }
287
288 template <class T> vector<Point<T>> hp(vector<Line<T>> a) {
289     sort(a.begin(), a.end(), [&](auto l1, auto l2) {
290         auto d1 = l1.b - l1.a;
291         auto d2 = l2.b - l2.a;
292
293         if (sgn(d1) != sgn(d2)) {
294             return sgn(d1) == 1;
295         }
296     });

```

```

297     return cross(d1, d2) > 0;
298 });
299
300 deque<Line<T>> ls;
301 deque<Point<T>> ps;
302 for (auto l : a) {
303     if (ls.empty()) {
304         ls.push_back(l);
305         continue;
306     }
307
308     while (!ps.empty() && !side(ps.back(), l)) {
309         ps.pop_back();
310         ls.pop_back();
311     }
312
313     while (!ps.empty() && !side(ps[0], l)) {
314         ps.pop_front();
315         ls.pop_front();
316     }
317
318     if (cross(l.b - l.a, ls.back().b - ls.back().a) == 0) {
319         if (dot(l.b - l.a, ls.back().b - ls.back().a) > 0) {
320             if (!side(ls.back().a, l)) {
321                 assert(ls.size() == 1);
322                 ls[0] = l;
323             }
324             continue;
325         }
326         return {};
327     }
328
329     ps.push_back(inter(ls.back(), l));
330     ls.push_back(l);
331 }
332
333 while (!ps.empty() && !side(ps.back(), ls[0])) {
334     ps.pop_back();
335     ls.pop_back();
336 }
337 if (ls.size() <= 2) {
338     return {};
339 }
340 ps.push_back(inter(ls[0], ls.back()));
341
342 return vector(ps.begin(), ps.end());
343 }
344
345 using Real = ld;
346 using P = Point<Real>;
347
348 constexpr Real eps = 0;

```

07A - 凸包与旋转卡壳（依赖07）.cpp

```

1  /*
2  依赖: 07 - 平面几何 (Point&Line, with Int)。
3  用途: 二维凸包、凸多边形直径、 $O(\log n)$  点包含判定。凸包不重复首点并按逆时针排列。
4  整数坐标时 T 应能容纳叉积和距离平方; 坐标接近  $1e9$  时通常用 int。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T>
8  vector<Point<T>> convex(vector<Point<T>> a, bool kp = false) {
9      sort(a.begin(), a.end(), [](const auto& p, const auto& q) {
10         return p.x != q.x ? p.x < q.x : p.y < q.y;
11     });
12     a.erase(unique(a.begin(), a.end()), a.end());
13     int n = a.size();
14     if (n <= 2) return a;
15     bool col = true;
16     for (int i = 2; i < n; ++i) col &= cross(a[1] - a[0], a[i] - a[0]) == 0;
17     if (kp && col) return a;
18
19     auto ch = [&](auto fst, auto lst) {
20         vector<Point<T>> h;
21         for (; fst != lst; ++fst) {
22             const auto& p = *fst;
23             while (h.size() >= 2) {
24                 T c = cross(h.back() - h[h.size() - 2], p - h.back());
25                 if (kp ? c >= 0 : c > 0) break;

```

```

26         h.pop_back();
27     }
28     h.push_back(p);
29 }
30 return h;
31 };
32 auto lo = ch(a.begin(), a.end());
33 auto up = ch(a.rbegin(), a.rend());
34 lo.pop_back(), up.pop_back();
35 lo.insert(lo.end(), up.begin(), up.end());
36 return lo;
37 }
38
39 template <class T> T area2(const vector<Point<T>>& p) {
40     T r{};
41     for (int i = 0, n = p.size(); i < n; ++i) r += cross(p[i], p[(i + 1) % n]);
42     return r;
43 }
44
45 // 输入为逆时针凸多边形: 返回 {最大距离平方, 端点下标}。
46 template <class T>
47 tuple<T, int, int> diam2(const vector<Point<T>>& p) {
48     int n = p.size();
49     if (!n) return {T{}, -1, -1};
50     if (n == 1) return {T{}, 0, 0};
51     auto ab = [](T x) { return x < 0 ? -x : x; };
52     T ans{};
53     int x = 0, y = 1, j = 1;
54     auto upd = [&](int u, int v) {
55         T d = square(p[u] - p[v]);
56         if (d > ans) ans = d, x = u, y = v;
57     };
58     for (int i = 0; i < n; ++i) {
59         int k = (i + 1) % n;
60         while (ab(cross(p[k] - p[i], p[(j + 1) % n] - p[i])) >
61             ab(cross(p[k] - p[i], p[j] - p[i]))) j = (j + 1) % n;
62         upd(i, j), upd(k, j);
63     }
64     return {ans, x, y};
65 }
66
67 // -1: 外部, 0: 边界, 1: 内部; 要求 p 为逆时针凸多边形且无重复首点。
68 template <class T> int inConv(const vector<Point<T>>& p, const Point<T>& q) {
69     int n = p.size();
70     auto on = [&](const Point<T>& a, const Point<T>& b) {
71         return cross(q - a, b - a) == 0 && dot(q - a, q - b) <= 0;
72     };
73     if (!n) return -1;
74     if (n == 1) return q == p[0] ? 0 : -1;
75     if (n == 2) return on(p[0], p[1]) ? 0 : -1;
76     T a = cross(p[1] - p[0], q - p[0]);
77     T b = cross(p[n - 1] - p[0], q - p[0]);
78     if (a < 0 || b > 0) return -1;
79     if (!a) return on(p[0], p[1]) ? 0 : -1;
80     if (!b) return on(p[0], p[n - 1]) ? 0 : -1;
81     int l = 1, r = n - 1;
82     while (r - l > 1) {
83         int m = (l + r) >> 1;
84         if (cross(p[m] - p[0], q - p[0]) >= 0) l = m;
85         else r = m;
86     }
87     T c = cross(p[r] - p[l], q - p[l]);
88     return c < 0 ? -1 : c == 0 ? 0 : 1;
89 }

```

07AA - 闵可夫斯基和 (依赖07A) .cpp

```

1  /*
2  依赖: 07 - 平面几何、07A - 凸包。
3  用途: 求两个点集凸包的闵可夫斯基和; 结果为逆时针凸包, 不重复首点、去共线中间点。
4  复杂度: 含两次凸包为  $O(n \log n + m \log m)$ , 已给凸包时主体为  $O(n+m)$ 。
5  性能: 默认使用展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T>
8  vector<Point<T>> minkSum(vector<Point<T>> a, vector<Point<T>> b) {
9      a = convex(move(a)), b = convex(move(b));
10     if (a.empty() || b.empty()) return {};
11     if (a.size() == 1) {
12         for (auto& p : b) p += a[0];
13         return b;

```

```

14     }
15     if (b.size() == 1) {
16         for (auto& p : a) p += b[0];
17         return a;
18     }
19     auto z = [](vector<Point<T>>& p) {
20         auto it = min_element(p.begin(), p.end(), [](const auto& x, const auto& y) {
21             return x.y != y.y ? x.y < y.y : x.x < y.x;
22         });
23         rotate(p.begin(), it, p.end());
24     };
25     z(a), z(b);
26     vector<Point<T>> x(a.size()), y(b.size());
27     for (int i = 0; i < a.size(); ++i) x[i] = a[(i + 1) % a.size()] - a[i];
28     for (int i = 0; i < b.size(); ++i) y[i] = b[(i + 1) % b.size()] - b[i];
29
30     int i = 0, j = 0;
31     Point<T> p = a[0] + b[0];
32     vector<Point<T>> c{p};
33     while (i < x.size() || j < y.size()) {
34         Point<T> d;
35         if (j == y.size() || (i < x.size() && cross(x[i], y[j]) > 0)) d = x[i++];
36         else if (i == x.size() || cross(x[i], y[j]) < 0) d = y[j++];
37         else d = x[i++] + y[j++];
38         p += d, c.push_back(p);
39     }
40     c.pop_back();
41     return convex(move(c));
42 }

```

07B - 圆与最小覆盖圆 (依赖07) .cpp

```

1  /*
2  依赖: 07 中的 Real、P、Point/Line、dot/cross/square/rotate。
3  用途: 圆与直线/圆的交点、三点外接圆、随机增量最小覆盖圆。
4  浮点比较统一使用 EP; 重合圆有无穷交点时 interCC 返回空数组, 需按题意另判。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  constexpr Real EP = 1e-12L;
8  struct Circle {
9      P o;
10     Real r = -1;
11 };
12
13 inline bool inCir(const Circle& c, const P& p) {
14     return c.r >= 0 && square(p - c.o) <= c.r * c.r + EP;
15 }
16
17 inline Circle diaCir(const P& a, const P& b) {
18     P o = (a + b) / Real(2);
19     return {o, sqrtl(square(a - b)) / 2};
20 }
21
22 inline Circle cir3(const P& a, const P& b, const P& c) {
23     P x = b - a, y = c - a;
24     Real d = 2 * cross(x, y);
25     if (fabs(d) <= EP) {
26         Circle r = diaCir(a, b);
27         for (auto [u, v] : {pair<P, P>{a, c}, pair<P, P>{b, c}}) {
28             Circle q = diaCir(u, v);
29             if (q.r > r.r) r = q;
30         }
31         return r;
32     }
33     P o = a + P((y.y * square(x) - x.y * square(y)) / d,
34               (x.x * square(y) - y.x * square(x)) / d);
35     return {o, sqrtl(square(o - a))};
36 }
37
38 inline vector<P> interCL(const Circle& c, const Line<Real>& l) {
39     P d = l.b - l.a;
40     Real z = square(d);
41     assert(z > EP);
42     P h = l.a + d * (dot(c.o - l.a, d) / z);
43     Real q = c.r * c.r - square(h - c.o);
44     if (q < -EP) return {};
45     if (q <= EP) return {h};
46     P v = d * sqrtl(q / z);
47     return {h - v, h + v};
48 }

```

```

49
50 inline vector<P> interCC(const Circle& a, const Circle& b) {
51     P d = b.o - a.o;
52     Real z = sqrtl(square(d));
53     if (z <= EP) return {};
54     if (z > a.r + b.r + EP || z < fabs(a.r - b.r) - EP) return {};
55     Real x = (z * z + a.r * a.r - b.r * b.r) / (2 * z);
56     Real h2 = a.r * a.r - x * x;
57     P v = d / z;
58     P p = a.o + v * x;
59     if (h2 <= EP) return {p};
60     P q = rotate(v) * sqrtl(max<Real>(0, h2));
61     return {p - q, p + q};
62 }
63
64 inline Circle minCir(vector<P> p, int sd = 712367821) {
65     if (p.empty()) return {{0, 0}, 0};
66     shuffle(p.begin(), p.end(), mt19937_64(sd));
67     Circle c{p[0], 0};
68     for (int i = 1; i < p.size(); ++i) if (!inCir(c, p[i])) {
69         c = {p[i], 0};
70         for (int j = 0; j < i; ++j) if (!inCir(c, p[j])) {
71             c = diaCir(p[i], p[j]);
72             for (int k = 0; k < j; ++k)
73                 if (!inCir(c, p[k])) c = cir3(p[i], p[j], p[k]);
74         }
75     }
76     return c;
77 }

```

07C - 三维几何.cpp

```

1  /*
2  用途：三维点、向量、直线与平面的基础运算。整数 T 可精确做点积/叉积/体积判断；距离返回 long double。
3  */
4  using ld = long double;
5
6  template <class T> struct Point3 {
7      T x{}, y{}, z{};
8      Point3() = default;
9      Point3(T x, T y, T z) : x(x), y(y), z(z) {}
10     Point3& operator+=(const Point3& p) { x += p.x, y += p.y, z += p.z; return *this; }
11     Point3& operator-=(const Point3& p) { x -= p.x, y -= p.y, z -= p.z; return *this; }
12     Point3& operator*=(const T& k) { x *= k, y *= k, z *= k; return *this; }
13     friend Point3 operator+(Point3 a, const Point3& b) { return a + b; }
14     friend Point3 operator-(Point3 a, const Point3& b) { return a - b; }
15     friend Point3 operator*(Point3 a, const T& k) { return a * k; }
16     friend bool operator==(const Point3& a, const Point3& b) { return a.x == b.x && a.y == b.y && a.z == b.z; }
17 };
18
19 template <class T> T dot(const Point3<T>& a, const Point3<T>& b) {
20     return a.x * b.x + a.y * b.y + a.z * b.z;
21 }
22
23 template <class T> Point3<T> cross(const Point3<T>& a, const Point3<T>& b) {
24     return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
25 }
26
27 template <class T> T square(const Point3<T>& a) { return dot(a, a); }
28
29 template <class T> T volume6(const Point3<T>& a, const Point3<T>& b, const Point3<T>& c, const Point3<T>& d) {
30     return dot(b - a, cross(c - a, d - a));
31 }
32
33 template <class T> ld disPL(const Point3<T>& p, const Point3<T>& a, const Point3<T>& b) {
34     return sqrtl(static_cast<ld>(square(cross(p - a, b - a)))) /
35         static_cast<ld>(square(b - a));
36 }
37
38 template <class T> ld disPP(const Point3<T>& p, const Point3<T>& a, const Point3<T>& n) {
39     return fabsl(static_cast<ld>(dot(p - a, n))) /
40         sqrtl(static_cast<ld>(square(n)));
41 }
42
43 // 直线 p+t*d 与平面 (x-a)·n=0 的交点；平行时返回 nullopt。
44 template <class T>
45 optional<Point3<ld>> interLP(const Point3<T>& p, const Point3<T>& d,
46                             const Point3<T>& a, const Point3<T>& n) {
47     T z = dot(d, n);
48     if (z == 0) return nullopt;
49     ld t = static_cast<ld>(dot(a - p, n)) / static_cast<ld>(z);
50     return Point3<ld>(p.x + t * d.x, p.y + t * d.y, p.z + t * d.z);
51 }

```


07D - 平面最近点对 (依赖07) .cpp

```

1  /*
2  依赖: 07 - 平面几何中的 Point。
3  用途: 求平面点集最近点对; 返回 {距离平方, 原下标1, 原下标2}, 不足两点返回 {INF, -1, -1}。
4  复杂度: 分治  $O(n \log n)$ 。距离统一转 long double, 避免整数坐标作差/平方溢出。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  using ld = long double;
8
9  template <class T>
10 tuple<ld, int, int> close(vector<Point<T>> p) {
11     struct N { Point<T> p; int id; };
12     vector<N> a;
13     for (int i = 0; i < p.size(); ++i) a.push_back({p[i], i});
14     sort(a.begin(), a.end(), [](const N& x, const N& y) {
15         return x.p.x != y.p.x ? x.p.x < y.p.x : x.p.y < y.p.y;
16     });
17     ld ans = numeric_limits<ld>::infinity();
18     int ax = -1, ay = -1;
19     auto upd = [&](const N& x, const N& y) {
20         ld dx = ld(x.p.x) - ld(y.p.x);
21         ld dy = ld(x.p.y) - ld(y.p.y);
22         ld d = dx * dx + dy * dy;
23         if (d < ans) ans = d, ax = x.id, ay = y.id;
24     };
25     auto cy = [](const N& x, const N& y) {
26         return x.p.y != y.p.y ? x.p.y < y.p.y : x.p.x < y.p.x;
27     };
28     auto dfs = [&](auto&& go, int l, int r) -> void {
29         if (r - l <= 3) {
30             for (int i = l; i < r; ++i) for (int j = i + 1; j < r; ++j) upd(a[i], a[j]);
31             sort(a.begin() + l, a.begin() + r, cy);
32             return;
33         }
34         int m = (l + r) >> 1;
35         ld mx = ld(a[m].p.x);
36         go(go, l, m), go(go, m, r);
37         inplace_merge(a.begin() + l, a.begin() + m, a.begin() + r, cy);
38         vector<N> q;
39         for (int i = l; i < r; ++i) {
40             ld dx = ld(a[i].p.x) - mx;
41             if (dx * dx >= ans) continue;
42             for (int j = q.size(); j--;) {
43                 ld dy = ld(a[i].p.y) - ld(q[j].p.y);
44                 if (dy * dy >= ans) break;
45                 upd(a[i], q[j]);
46             }
47             q.push_back(a[i]);
48         }
49     };
50     dfs(dfs, 0, a.size());
51     return {ans, ax, ay};
52 }

```

数学：多项式、博弈、数值计算与位运算

36 个代码模板 · 林月 · v1.0.0 · 2026.8.1

08 - 快速数论变换 (NTT) .cpp

```

1  /*
2  用途：高性能整数 NTT 卷积。
3  用法：auto c = NTT<998244353, 3>::mul(a, b)。
4  约定：系数和下标缓存使用 int。
5  自动取模版本使用 08B / 08C 的 Poly<Z, 3>。
6  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
7  */
8
9  template <int P, int G> struct NTT {
10
11      static int pow(int a, int n) {
12          int r = 1;
13          while (n) {
14              if (n & 1) r = r * a % P;
15              a = a * a % P;
16              n >>= 1;
17          }
18          return r;
19      }
20
21      static void ntt(vector<int>& a, bool iv) {
22          int n = a.size();
23          assert(n && !(n & (n - 1)));
24          static vector<int> rv;
25          static vector<int> rt{0, 1};
26          if (rv.size() != n) {
27              int k = __builtin_ctz(static_cast<unsigned>(n));
28              rv.assign(n, 0);
29              if (n > 1)
30                  for (int i = 0; i < n; ++i) rv[i] = (rv[i >> 1] >> 1) | ((i & 1) << (k - 1));
31          }
32          if (rt.size() < n) {
33              int k = __builtin_ctz(static_cast<unsigned>(rt.size()));
34              rt.resize(n);
35              while ((1LL << k) < n) {
36                  int z = pow(G, (P - 1) >> (k + 1));
37                  for (int i = 1LL << (k - 1); i < (1LL << k); ++i) {
38                      rt[i << 1] = rt[i];
39                      rt[i << 1 | 1] = rt[i] * z % P;
40                  }
41                  ++k;
42              }
43          }
44          for (int i = 0; i < n; ++i)
45              if (i < rv[i]) swap(a[i], a[rv[i]]);
46          for (int len = 1; len < n; len <= 1) {
47              for (int i = 0; i < n; i += len << 1) {
48                  for (int j = 0; j < len; ++j) {
49                      int u = a[i + j];
50                      int v = a[i + j + len] * rt[len + j] % P;
51                      a[i + j] = (u + v) % P;
52                      a[i + j + len] = (u - v + P) % P;
53                  }
54              }
55          }
56          if (iv) {
57              reverse(a.begin() + 1, a.end());
58              int x = pow(n, P - 2);
59              for (int& v : a) v = v * x % P;
60          }
61      }
62
63      static vector<int> mul(vector<int> a, vector<int> b) {
64          if (a.empty() || b.empty()) return {};
65          int m = a.size() + b.size() - 1;

```

```

66     int n = 1;
67     while (n < m) n <= 1;
68     a.resize(n);
69     b.resize(n);
70     ntt(a, false);
71     ntt(b, false);
72     for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % P;
73     ntt(a, true);
74     a.resize(m);
75     return a;
76 }
77 };

```

08A - 多项式 (Poly, int) (NTT) .cpp

```

1  /*
2  用途：高性能整数系数多项式（乘法、求逆、ln/exp、求值、插值）。
3  依赖：08 - 快速数论变换（NTT）。
4  用法：using P = Poly<998244353, 3>; 系数容器为 vector<int>。
5  约定：这里刻意不用 MInt，系数/下标缓存使用 int。
6  自动取模版本使用 08B / 08C 的 Poly<Z, 3>。
7  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
8  */
9
10 template <int P, int G = 3> struct Poly {
11
12     static int pw(int a, int n) {
13         return NTT<P, G>::pow(a, n);
14     }
15
16     static void ntt(vector<int>& a, bool iv) {
17         NTT<P, G>::ntt(a, iv);
18     }
19
20     static vector<int> add(vector<int> a, vector<int> b) {
21         int n = max(a.size(), b.size());
22         a.resize(n);
23         b.resize(n);
24         for (int i = 0; i < n; ++i) {
25             a[i] += b[i];
26             if (a[i] >= P) a[i] -= P;
27         }
28         return a;
29     }
30
31     static vector<int> sub(vector<int> a, vector<int> b) {
32         int n = max(a.size(), b.size());
33         a.resize(n);
34         b.resize(n);
35         for (int i = 0; i < n; ++i) {
36             a[i] -= b[i];
37             if (a[i] < 0) a[i] += P;
38         }
39         return a;
40     }
41
42     static vector<int> mul(vector<int> a, vector<int> b) {
43         return NTT<P, G>::mul(move(a), move(b));
44     }
45
46     static vector<int> inv(const vector<int>& a, int n) {
47         assert(n >= 0);
48         if (!n) return {};
49         assert(!a.empty() && a[0]);
50         if (n == 1) return {pw(a[0], P - 2)};
51         vector<int> b = inv(a, (n + 1) >> 1);
52         int m = 1;
53         while (m < n << 1) m <= 1;
54         vector<int> c(a.begin(), a.begin() + min<int>(a.size(), n));
55         c.resize(m);
56         b.resize(m);
57         ntt(c, false);
58         ntt(b, false);
59         for (int i = 0; i < m; ++i) {
60             b[i] = (2 * b[i] - c[i] * b[i] % P * b[i] % P + P) % P;
61         }
62         ntt(b, true);
63         b.resize(n);
64         return b;
65     }

```

```

66
67 static vector<int> div(const vector<int>& a, const vector<int>& b) {
68     assert(!b.empty() && b.back());
69     int n = a.size(), m = b.size();
70     if (n < m) return {0};
71     vector<int> x(a.rbegin(), a.rend()), y(b.rbegin(), b.rend());
72     y = inv(y, n - m + 1);
73     vector<int> q = mul(x, y);
74     q.resize(n - m + 1);
75     reverse(q.begin(), q.end());
76     return q;
77 }
78
79 static vector<int> mod(const vector<int>& a, const vector<int>& b) {
80     vector<int> r = sub(a, mul(div(a, b), b));
81     r.resize(min<int>(r.size(), b.size() - 1));
82     return r;
83 }
84
85 static vector<int> der(const vector<int>& a) {
86     int n = a.size();
87     if (n <= 1) return {};
88     vector<int> b(n - 1);
89     for (int i = 1; i < n; ++i) b[i - 1] = a[i] * i % P;
90     return b;
91 }
92
93 static vector<int> integ(const vector<int>& a) {
94     int n = a.size();
95     vector<int> b(n + 1), iv(n + 1);
96     if (n) iv[1] = 1;
97     for (int i = 2; i <= n; ++i) iv[i] = P - (P / i) * iv[P % i] % P;
98     for (int i = 0; i < n; ++i) b[i + 1] = a[i] * iv[i + 1] % P;
99     return b;
100 }
101
102 static vector<int> ln(const vector<int>& a, int n) {
103     assert(n >= 0);
104     if (!n) return {};
105     assert(!a.empty() && a[0]);
106     vector<int> b = integ(mul(der(a), inv(a, n)));
107     b.resize(n);
108     return b;
109 }
110
111 static vector<int> exp(const vector<int>& a, int n) {
112     assert(n >= 0);
113     if (!n) return {};
114     assert(a.empty() || !a[0]);
115     vector<int> b{1};
116     for (int k = 2; k < n << 1; k <= 1) {
117         int m = min(k, n);
118         vector<int> c(a.begin(), a.begin() + min<int>(a.size(), m));
119         c.resize(m);
120         c = sub(c, ln(b, m));
121         ++c[0];
122         if (c[0] == P) c[0] = 0;
123         b = mul(b, c);
124         b.resize(m);
125     }
126     b.resize(n);
127     return b;
128 }
129
130 static void build(vector<vector<int>>& t, const vector<int>& x, int u, int l, int r) {
131     if (r - l == 1) {
132         t[u] = {x[l] ? P - x[l] : 0, 1};
133         return;
134     }
135     int m = (l + r) >> 1;
136     build(t, x, u << 1, l, m);
137     build(t, x, u << 1 | 1, m, r);
138     t[u] = mul(t[u << 1], t[u << 1 | 1]);
139 }
140
141 static void eval0(const vector<int>& f, const vector<vector<int>>& t, vector<int>& y, int u, int l, int r) {
142     if (r - l == 1) {
143         if (f.empty()) y[l] = 0;
144         else if (f.size() == 1) y[l] = f[0];
145         else y[l] = mod(f, t[u])[0];
146         return;
147     }

```

```

148     vector<int> g = mod(f, t[u]);
149     int m = (l + r) >> 1;
150     eval0(g, t, y, u << 1, l, m);
151     eval0(g, t, y, u << 1 | 1, m, r);
152 }
153
154 static vector<int> eval(const vector<int>& f, const vector<int>& x) {
155     int n = x.size();
156     if (!n) return {};
157     vector<vector<int>> t(n << 2 | 1);
158     build(t, x, 1, 0, n);
159     vector<int> y(n);
160     eval0(f, t, y, 1, 0, n);
161     return y;
162 }
163
164 static vector<int> interp(const vector<int>& x, const vector<int>& y) {
165     int n = x.size();
166     assert(y.size() == n);
167     if (!n) return {};
168     vector<vector<int>> t(n << 2 | 1);
169     build(t, x, 1, 0, n);
170     vector<int> d = eval(der(t[1]), x);
171     vector<int> w(n);
172     for (int i = 0; i < n; ++i) w[i] = y[i] * pw(d[i], P - 2) % P;
173     function<vector<int>(int, int, int)> dfs = [&](int u, int l, int r) -> vector<int> {
174         if (r - l == 1) return {w[l]};
175         int m = (l + r) >> 1;
176         return add(mul(dfs(u << 1, l, m), t[u << 1 | 1]), mul(dfs(u << 1 | 1, m, r), t[u << 1]));
177     };
178     return dfs(1, 0, n);
179 }
180
181 };
182
183 using Poly998 = Poly<998244353, 3>;

```

08B - 多项式 (Poly, Z) (NTT) .cpp

```

1  /*
2  用途: 完整自动取模 NTT 多项式: 乘法、逆、ln/exp、多点求值、插值。
3  用法: using P = Poly<Z, 3>; Z 为 MInt<P>, 模数须是 NTT 友好质数。
4  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
5  */
6  template <class Z, int G = 3> struct Poly {
7
8      static void ntt(vector<Z>& a, bool iv) {
9          int n = a.size();
10         assert(n && !(n & (n - 1)));
11         vector<int> rv(n);
12         int k = __builtin_ctz(static_cast<unsigned>(n));
13         if (n > 1)
14             for (int i = 0; i < n; ++i) rv[i] = (rv[i >> 1] >> 1) | ((i & 1) << (k - 1));
15         for (int i = 0; i < n; ++i)
16             if (i < rv[i]) swap(a[i], a[rv[i]]);
17         for (int len = 2; len <= n; len <= 1) {
18             Z wn = mypow(Z(G), (Z::askMod() - 1) / len);
19             if (iv) wn = wn.inv();
20             for (int i = 0; i < n; i += len) {
21                 Z w = 1;
22                 for (int j = 0; j < len / 2; ++j) {
23                     Z x = a[i + j], y = a[i + j + len / 2] * w;
24                     a[i + j] = x + y;
25                     a[i + j + len / 2] = x - y;
26                     w *= wn;
27                 }
28             }
29         }
30         if (iv) {
31             Z x = Z(n).inv();
32             for (Z& v : a) v *= x;
33         }
34     }
35
36     static vector<Z> add(vector<Z> a, vector<Z> b) {
37         int n = max(a.size(), b.size());
38         a.resize(n);
39         b.resize(n);
40         for (int i = 0; i < n; ++i) a[i] += b[i];
41         return a;

```

```

42     }
43
44     static vector<Z> sub(vector<Z> a, vector<Z> b) {
45         int n = max(a.size(), b.size());
46         a.resize(n);
47         b.resize(n);
48         for (int i = 0; i < n; ++i) a[i] -= b[i];
49         return a;
50     }
51
52     static vector<Z> mul(vector<Z> a, vector<Z> b) {
53         if (a.empty() || b.empty()) return {};
54         int m = a.size() + b.size() - 1, n = 1;
55         while (n < m) n <= 1;
56         a.resize(n);
57         b.resize(n);
58         ntt(a, false);
59         ntt(b, false);
60         for (int i = 0; i < n; ++i) a[i] *= b[i];
61         ntt(a, true);
62         a.resize(m);
63         return a;
64     }
65
66     static vector<Z> inv(const vector<Z>& a, int n) {
67         assert(n >= 0);
68         if (!n) return {};
69         assert(!a.empty() && a[0] != Z(0));
70         if (n == 1) return {a[0].inv()};
71         vector<Z> b = inv(a, (n + 1) >> 1);
72         int m = 1;
73         while (m < n << 1) m <= 1;
74         vector<Z> c(a.begin(), a.begin() + min<int>(a.size(), n));
75         c.resize(m);
76         b.resize(m);
77         ntt(c, false);
78         ntt(b, false);
79         for (int i = 0; i < m; ++i) b[i] *= Z(2) - c[i] * b[i];
80         ntt(b, true);
81         b.resize(n);
82         return b;
83     }
84
85     static vector<Z> div(const vector<Z>& a, const vector<Z>& b) {
86         assert(!b.empty() && b.back() != Z(0));
87         int n = a.size(), m = b.size();
88         if (n < m) return {Z(0)};
89         vector<Z> x(a.rbegin(), a.rend()), y(b.rbegin(), b.rend());
90         y = inv(y, n - m + 1);
91         vector<Z> q = mul(x, y);
92         q.resize(n - m + 1);
93         reverse(q.begin(), q.end());
94         return q;
95     }
96
97     static vector<Z> mod(const vector<Z>& a, const vector<Z>& b) {
98         vector<Z> r = sub(a, mul(div(a, b), b));
99         r.resize(min<int>(r.size(), b.size() - 1));
100        return r;
101    }
102
103    static vector<Z> der(const vector<Z>& a) {
104        int n = a.size();
105        if (n <= 1) return {};
106        vector<Z> b(n - 1);
107        for (int i = 1; i < n; ++i) b[i - 1] = a[i] * Z(i);
108        return b;
109    }
110
111    static vector<Z> integ(const vector<Z>& a) {
112        int n = a.size();
113        vector<Z> b(n + 1);
114        for (int i = 0; i < n; ++i) b[i + 1] = a[i] / Z(i + 1);
115        return b;
116    }
117
118    static vector<Z> ln(const vector<Z>& a, int n) {
119        assert(n >= 0);
120        if (!n) return {};
121        assert(!a.empty() && a[0] != Z(0));
122        vector<Z> b = integ(mul(der(a), inv(a, n)));
123        b.resize(n);

```

```

124     return b;
125 }
126
127 static vector<Z> exp(const vector<Z>& a, int n) {
128     assert(n >= 0);
129     if (!n) return {};
130     assert(a.empty() || a[0] == Z(0));
131     vector<Z> b{Z(1)};
132     for (int k = 2; k < n << 1; k <= 1) {
133         int m = min(k, n);
134         vector<Z> c(a.begin(), a.begin() + min<int>(a.size(), m));
135         c.resize(m);
136         c = sub(c, ln(b, m));
137         c[0] += Z(1);
138         b = mul(b, c);
139         b.resize(m);
140     }
141     b.resize(n);
142     return b;
143 }
144
145 static void build(vector<vector<Z>>& t, const vector<Z>& x, int u, int l, int r) {
146     if (r - l == 1) {
147         t[u] = {-x[l], Z(1)};
148         return;
149     }
150     int m = (l + r) >> 1;
151     build(t, x, u << 1, l, m);
152     build(t, x, u << 1 | 1, m, r);
153     t[u] = mul(t[u << 1], t[u << 1 | 1]);
154 }
155
156 static void eval0(const vector<Z>& f, const vector<vector<Z>>& t, vector<Z>& y, int u, int l, int r) {
157     if (r - l == 1) {
158         if (f.empty()) y[l] = Z(0);
159         else if (f.size() == 1) y[l] = f[0];
160         else y[l] = mod(f, t[u])[0];
161         return;
162     }
163     vector<Z> g = mod(f, t[u]);
164     int m = (l + r) >> 1;
165     eval0(g, t, y, u << 1, l, m);
166     eval0(g, t, y, u << 1 | 1, m, r);
167 }
168
169 static vector<Z> eval(const vector<Z>& f, const vector<Z>& x) {
170     int n = x.size();
171     if (!n) return {};
172     vector<vector<Z>> t(n << 2 | 1);
173     build(t, x, 1, 0, n);
174     vector<Z> y(n);
175     eval0(f, t, y, 1, 0, n);
176     return y;
177 }
178
179 static vector<Z> interp(const vector<Z>& x, const vector<Z>& y) {
180     int n = x.size();
181     assert(y.size() == n);
182     if (!n) return {};
183     vector<vector<Z>> t(n << 2 | 1);
184     build(t, x, 1, 0, n);
185     vector<Z> d = eval(der(t[1]), x), w(n);
186     for (int i = 0; i < n; ++i) w[i] = y[i] / d[i];
187     function<vector<Z>(int, int, int)> dfs = [&](int u, int l, int r) -> vector<Z> {
188         if (r - l == 1) return {w[l]};
189         int m = (l + r) >> 1;
190         return add(mul(dfs(u << 1, l, m), t[u << 1 | 1]), mul(dfs(u << 1 | 1, m, r), t[u << 1]));
191     };
192     return dfs(1, 0, n);
193 }
194
195 };
196
197 template <class Z> using NTT = Poly<Z, 3>;

```

08C - 轻量多项式 (Poly, Z) (NTT, ln, exp) .cpp

```

1  /*
2  用途: 轻量 NTT 多项式 (乘法、求逆、ln、exp)。
3  用法: using P = Poly<Z, 3>; Z 为任意 MInt<P>, 当前模数须为 NTT 友好质数。

```

性能：默认使用展开后的 `int`；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 `i32`。

```

4  */
5  template <class Z, int G = 3> struct Poly {
6      static vector<Z> cut(vector<Z> a, int n) {
7          a.resize(n);
8          return a;
9      }
10 }
11
12 static vector<Z> add(vector<Z> a, const vector<Z> &b) {
13     if (a.size() < b.size()) a.resize(b.size());
14     for (int i = 0; i < b.size(); ++i) a[i] += b[i];
15     return a;
16 }
17
18 static vector<Z> sub(vector<Z> a, const vector<Z> &b) {
19     if (a.size() < b.size()) a.resize(b.size());
20     for (int i = 0; i < b.size(); ++i) a[i] -= b[i];
21     return a;
22 }
23
24 static void ntt(vector<Z> &a, bool inv) {
25     int n = a.size();
26     if (n <= 1) return;
27     vector<int> rev(n);
28     int lg = __builtin_ctzll(n);
29     for (int i = 0; i < n; ++i) {
30         rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (lg - 1));
31         if (i < rev[i]) swap(a[i], a[rev[i]]);
32     }
33     for (int len = 2; len <= n; len <= 1) {
34         Z wn = mypow(Z(G), (Z::askMod() - 1) / len);
35         if (inv) wn = wn.inv();
36         for (int i = 0; i < n; i += len) {
37             Z w = 1;
38             for (int j = 0; j < len / 2; ++j) {
39                 Z x = a[i + j], y = a[i + j + len / 2] * w;
40                 a[i + j] = x + y;
41                 a[i + j + len / 2] = x - y;
42                 w *= wn;
43             }
44         }
45     }
46     if (inv) {
47         Z iv = Z(n).inv();
48         for (auto &x : a) x *= iv;
49     }
50 }
51
52 static vector<Z> mul(vector<Z> a, vector<Z> b) {
53     if (a.empty() || b.empty()) return {};
54     int nd = a.size() + b.size() - 1;
55     int n = 1;
56     while (n < nd) n <= 1;
57     a.resize(n), b.resize(n);
58     ntt(a, false), ntt(b, false);
59     for (int i = 0; i < n; ++i) a[i] *= b[i];
60     ntt(a, true);
61     a.resize(nd);
62     return a;
63 }
64
65 static vector<Z> der(const vector<Z> &a) {
66     if (a.size() <= 1) return {};
67     vector<Z> b(a.size() - 1);
68     for (int i = 1; i < a.size(); ++i) b[i - 1] = a[i] * Z(i);
69     return b;
70 }
71
72 static vector<Z> integ(const vector<Z> &a) {
73     vector<Z> b(a.size() + 1);
74     for (int i = 0; i < a.size(); ++i) b[i + 1] = a[i] / Z(i + 1);
75     return b;
76 }
77
78 static vector<Z> inv(const vector<Z> &a, int m) {
79     assert(m >= 0);
80     if (m == 0) return {};
81     assert(!a.empty() && a[0] != Z(0));
82     vector<Z> b{a[0].inv()};
83     for (int k = 2; k < 2 * m; k <= 1) {
84         vector<Z> f = cut(a, min(k, m));
85         vector<Z> t = mul(mul(b, b), f);

```



```

86         b.resize(min(k, m));
87         for (int i = 0; i < b.size(); ++i) b[i] += b[i] - t[i];
88     }
89     b.resize(m);
90     return b;
91 }
92
93 static vector<Z> ln(const vector<Z> &a, int m) {
94     assert(m >= 0);
95     if (m == 0) return {};
96     assert(!a.empty() && a[0] == Z(1));
97     vector<Z> b = integ(mul(der(a), inv(a, m)));
98     b.resize(m);
99     return b;
100 }
101
102 static vector<Z> exp(const vector<Z> &a, int m) {
103     assert(m >= 0);
104     if (m == 0) return {};
105     assert(a.empty() || a[0] == Z(0));
106     vector<Z> b{1};
107     for (int k = 2; k < 2 * m; k <= 1) {
108         int n = min(k, m);
109         vector<Z> t = sub(cut(a, n), ln(b, n));
110         t[0] += Z(1);
111         b = cut(mul(b, t), n);
112     }
113     b.resize(m);
114     return b;
115 }
116 };

```

08D - 快速傅里叶变换 (FFT, with Complex) .cpp

```

1  struct FFT {
2      using cd = Complex<double>;
3      static constexpr double PI = 3.141592653589793238462643383279502884;
4
5      void fft(vector<cd> &a, bool inv) {
6          int n = a.size();
7          static vector<int> rev;
8          static vector<cd> rts{{0, 0}, {1, 0}};
9
10         if (rev.size() != n) {
11             int k = __builtin_ctz(n);
12             rev.assign(n, 0);
13             for (int i = 0; i < n; i++) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (k - 1));
14         }
15
16         if (rts.size() < n) {
17             int k = __builtin_ctz(rts.size());
18             rts.resize(n);
19             while ((1 << k) < n) {
20                 double da = 2 * PI / (1 << (k + 1));
21                 for (int i = 1 << (k - 1); i < (1 << k); i++) {
22                     rts[i << 1] = rts[i];
23                     double ag = da * (2 * i + 1 - (1 << k));
24                     rts[i << 1 | 1] = cd(cos(ag), sin(ag));
25                 }
26                 k++;
27             }
28         }
29
30         for (int i = 0; i < n; i++)
31             if (i < rev[i]) swap(a[i], a[rev[i]]);
32
33         for (int len = 1; len < n; len <= 1) {
34             for (int i = 0; i < n; i += len << 1) {
35                 for (int j = 0; j < len; j++) {
36                     cd u = a[i + j];
37                     cd v = a[i + j + len] * rts[len + j];
38                     a[i + j] = u + v;
39                     a[i + j + len] = u - v;
40                 }
41             }
42         }
43
44         if (inv) {
45             reverse(a.begin() + 1, a.end());
46         }

```

```

47     for (cd &x : a) x /= n;
48     }
49 }
50
51 vector<int> mul(const vector<int> &a, const vector<int> &b) {
52     if (a.empty() || b.empty()) return {};
53     int nd = a.size() + b.size() - 1;
54     vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
55     int n = 1;
56     while (n < nd) n <= 1;
57     fa.resize(n);
58     fb.resize(n);
59     fft(fa, false);
60     fft(fb, false);
61     for (int i = 0; i < n; i++) fa[i] *= fb[i];
62     fft(fa, true);
63
64     vector<int> res(nd);
65     for (int i = 0; i < nd; i++) res[i] = llround(fa[i].a);
66     return res;
67 }
68 };

```

08E - 快速傅里叶变换 (ModFFT, with Complex) .cpp

```

1  /*
2  用途: 任意 32 位模数卷积 (系数按 15 位拆分)。
3  依赖: 06A - 复数 (Complex)。
4  用法: 原始整数写 mul(vector<int>, vector<int>, p);
5       自动取模写 mul(vector<Z>, vector<Z>), Z 为 MInt<P>。
6  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
7  */
8
9  struct FFT {
10     using C = Complex<double>;
11
12     static constexpr double pi = 3.141592653589793238462643383279502884;
13
14     static void fft(vector<C>& a, bool iv) {
15         int n = a.size();
16         assert(n && !(n & (n - 1)));
17         static vector<int> rv;
18         static vector<C> rt{{0, 0}, {1, 0}};
19         if (rv.size() != n) {
20             int k = __builtin_ctz(static_cast<unsigned>(n));
21             rv.assign(n, 0);
22             if (n > 1)
23                 for (int i = 0; i < n; ++i) rv[i] = (rv[i >> 1] >> 1) | ((i & 1) << (k - 1));
24         }
25         if (rt.size() < n) {
26             int k = __builtin_ctz(static_cast<unsigned>(rt.size()));
27             rt.resize(n);
28             while ((1LL << k) < n) {
29                 double ag = 2 * pi / (1LL << (k + 1));
30                 for (int i = 1LL << (k - 1); i < (1LL << k); ++i) {
31                     rt[i << 1] = rt[i];
32                     double x = ag * (2 * i + 1 - (1LL << k));
33                     rt[i << 1 | 1] = C(cos(x), sin(x));
34                 }
35                 ++k;
36             }
37         }
38         for (int i = 0; i < n; ++i)
39             if (i < rv[i]) swap(a[i], a[rv[i]]);
40         for (int len = 1; len < n; len <= 1) {
41             for (int i = 0; i < n; i += len << 1) {
42                 for (int j = 0; j < len; ++j) {
43                     C x = a[i + j], y = a[i + j + len] * rt[len + j];
44                     a[i + j] = x + y;
45                     a[i + j + len] = x - y;
46                 }
47             }
48         }
49         if (iv) {
50             reverse(a.begin() + 1, a.end());
51             for (C& x : a) x /= n;
52         }
53     }
54
55     static vector<int> mul(const vector<int>& a, const vector<int>& b, int p) {

```

```

56     if (a.empty() || b.empty()) return {};
57     assert(p > 0);
58     int m = a.size() + b.size() - 1, n = 1;
59     while (n < m) n <= 1;
60     vector<C> x(n), y(n);
61     for (int i = 0; i < a.size(); ++i) x[i] = C(a[i] & 32767, a[i] >> 15);
62     for (int i = 0; i < b.size(); ++i) y[i] = C(b[i] & 32767, b[i] >> 15);
63     fft(x, false);
64     fft(y, false);
65     vector<C> u(n), v(n);
66     for (int i = 0; i < n; ++i) {
67         int j = (n - i) & (n - 1);
68         C a1 = (x[i] + conj(x[j])) * C(0.5, 0);
69         C a2 = (x[i] - conj(x[j])) * C(0, -0.5);
70         C b1 = (y[i] + conj(y[j])) * C(0.5, 0);
71         C b2 = (y[i] - conj(y[j])) * C(0, -0.5);
72         u[i] = a1 * b1 + a2 * b2 * C(0, 1);
73         v[i] = a1 * b2 + a2 * b1;
74     }
75     fft(u, true);
76     fft(v, true);
77     vector<int> r(m);
78     for (int i = 0; i < m; ++i) {
79         int x0 = llround(u[i].a) % p;
80         int y0 = llround(v[i].a) % p;
81         int z0 = llround(u[i].b) % p;
82         r[i] = (x0 + (y0 << 15) + (z0 << 30)) % p;
83     }
84     return r;
85 }
86
87 template <class Z> static vector<Z> mul(const vector<Z>& x, const vector<Z>& y) {
88     vector<int> a(x.size()), b(y.size());
89     for (int i = 0; i < x.size(); ++i) a[i] = static_cast<int>(x[i]);
90     for (int i = 0; i < y.size(); ++i) b[i] = static_cast<int>(y[i]);
91     vector<int> v = mul(a, b, Z::askMod());
92     vector<Z> r(v.size());
93     for (int i = 0; i < v.size(); ++i) r[i] = Z(v[i]);
94     return r;
95 }
96 };

```

08F - 多模快速数论变换 (CRTNTT, with NTT) .cpp

```

1  /*
2  用途: 用三模 NTT + CRT 做任意 32 位模数卷积。
3  依赖: 08 - 快速数论变换 (NTT)。
4  用法: 原始整数写 mul(vector<int>, vector<int>, p);
5         自动取模写 mul(vector<Z>, vector<Z>), Z 为 MInt<P>。
6  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
7  */
8
9  using i128 = __int128_t;
10
11 struct CRTNTT {
12     static constexpr int p1 = 998244353;
13     static constexpr int p2 = 1004535809;
14     static constexpr int p3 = 104857601;
15
16     NTT<p1, 3> a;
17     NTT<p2, 3> b;
18     NTT<p3, 3> c;
19
20     static int mul(int a, int b, int p) {
21         assert(p > 0);
22         a %= p;
23         b %= p;
24 #ifdef __SIZEOF_INT128__
25         return i128(a) * b % p;
26 #else
27         int r = 0;
28         while (b) {
29             if (b & 1) r = r >= p - a ? r - (p - a) : r + a;
30             a = a >= p - a ? a - (p - a) : a + a;
31             b >>= 1;
32         }
33         return r;
34 #endif
35     }
36 };

```

```

37 static int crt(int a, int b, int c, int p) {
38     assert(p > 0);
39     static constexpr int m1 = p1, m2 = p2, m3 = p3;
40     static constexpr int i1 = 669690699;
41     static constexpr int i2 = 99747662;
42     int x = a;
43     int t = (b - x) * i1 % m2;
44     if (t < 0) t += m2;
45     x += t * m1;
46     t = (c - x % m3) * i2 % m3;
47     if (t < 0) t += m3;
48     int y = mul(t, mul(m1 % p, m2 % p, p), p);
49     int z = x % p;
50     return z >= p - y ? z - (p - y) : z + y;
51 }
52
53 vector<int> mul(const vector<int>& x, const vector<int>& y, int p) {
54     if (x.empty() || y.empty()) return {};
55     auto u = a.mul(x, y);
56     auto v = b.mul(x, y);
57     auto w = c.mul(x, y);
58     vector<int> r(u.size());
59     for (int i = 0; i < u.size(); ++i) r[i] = crt(u[i], v[i], w[i], p);
60     return r;
61 }
62
63 template <class Z> vector<Z> mul(const vector<Z>& x, const vector<Z>& y) {
64     vector<int> a(x.size()), b(y.size());
65     for (int i = 0; i < x.size(); ++i) a[i] = static_cast<int>(x[i]);
66     for (int i = 0; i < y.size(); ++i) b[i] = static_cast<int>(y[i]);
67     vector<int> v = mul(a, b, Z::askMod());
68     vector<Z> r(v.size());
69     for (int i = 0; i < v.size(); ++i) r[i] = Z(v[i]);
70     return r;
71 }
72 };

```

08G - FWT (OR、AND、XOR 卷积) .cpp

```

1  /*
2  用途: 长度  $2^k$  的 OR / AND / XOR 卷积。
3  前提: XOR 逆变换要求 2 在 Z 中可逆。
4  */
5  enum class FWType { Or, And, Xor };
6
7  template <class Z> void fwt(vector<Z>& a, FWType tp, bool iv) {
8      int n = a.size();
9      assert(n && !(n & (n - 1)));
10     for (int l = 1; l < n; l <= l * 2) {
11         for (int i = 0; i < n; i += l * 2) {
12             for (int j = 0; j < l; ++j) {
13                 Z& x = a[i + j];
14                 Z& y = a[i + j + l];
15                 if (tp == FWType::Or) {
16                     if (iv) y -= x;
17                     else y += x;
18                 } else if (tp == FWType::And) {
19                     if (iv) x -= y;
20                     else x += y;
21                 } else {
22                     Z u = x, v = y;
23                     x = u + v;
24                     y = u - v;
25                 }
26             }
27         }
28     }
29     if (tp == FWType::Xor && iv) {
30         Z x = Z(n).inv();

```

```

31     for (Z& v : a) v *= x;
32 }
33 }
34
35 template <class Z> vector<Z> conv(vector<Z> a, vector<Z> b, FWType tp) {
36     assert(a.size() == b.size() && !a.empty());
37     fwt(a, tp, false);
38     fwt(b, tp, false);
39     for (int i = 0; i < a.size(); ++i) a[i] *= b[i];
40     fwt(a, tp, true);
41     return a;
42 }

```

08H - SOS DP (子集、超集 Zeta 变换) .cpp

```

1  /*
2  用途: 对所有 mask 计算子集和或超集和, 以及对应的 Möbius 逆变换。
3  前提: 数组长度为 2^k, 元素类型 T 支持 += 和 -=。
4  复杂度: O(k 2^k)。常用于按位包含关系计数、容斥和子集 DP。
5  */
6  namespace SOSDP {
7      inline int bits(int siz) {
8          assert(siz > 0 && (siz & (siz - 1)) == 0);
9          return __builtin_ctz((unsigned)siz);
10     }
11
12     template <class T> void zetaSub(vector<T>& a) {
13         int k = bits(a.size());
14         for (int i = 0; i < k; ++i)
15             for (int msk = 0; msk < a.size(); ++msk)
16                 if (msk >> i & 1) a[msk] += a[msk ^ (1 << i)];
17     }
18
19     template <class T> void mobSub(vector<T>& a) {
20         int k = bits(a.size());
21         for (int i = 0; i < k; ++i)
22             for (int msk = 0; msk < a.size(); ++msk)
23                 if (msk >> i & 1) a[msk] -= a[msk ^ (1 << i)];
24     }
25
26     template <class T> void zetaSup(vector<T>& a) {
27         int k = bits(a.size());
28         for (int i = 0; i < k; ++i)
29             for (int msk = 0; msk < a.size(); ++msk)
30                 if ((msk >> i & 1) == 0) a[msk] += a[msk | (1 << i)];
31     }
32
33     template <class T> void mobSup(vector<T>& a) {
34         int k = bits(a.size());
35         for (int i = 0; i < k; ++i)
36             for (int msk = 0; msk < a.size(); ++msk)
37                 if ((msk >> i & 1) == 0) a[msk] -= a[msk | (1 << i)];
38     }
39 } // namespace SOSDP

```

08HA - 子集卷积 (Subset Convolution) .cpp

```

1  /*
2  用途:  $c[S] = \sum_{A \text{ subset } S} a[A] b[S-A]$ 。
3  前提: 长度为  $2^k$ , Z 支持加减乘; 复杂度  $O(k^2 2^k)$ 。
4  */
5  namespace SubConv {
6      template <class Z> void zetaSub(vector<Z>& a) {
7          int n = a.size(), k = __builtin_ctz((unsigned)n);
8          assert(n && !(n & (n - 1)));
9          for (int i = 0; i < k; ++i)
10             for (int s = 0; s < n; ++s)
11                 if (s >> i & 1) a[s] += a[s ^ (1 << i)];
12     }
13
14     template <class Z> void mobSub(vector<Z>& a) {
15         int n = a.size(), k = __builtin_ctz((unsigned)n);
16         assert(n && !(n & (n - 1)));
17         for (int i = 0; i < k; ++i)
18             for (int s = 0; s < n; ++s)
19                 if (s >> i & 1) a[s] -= a[s ^ (1 << i)];
20     }
21 }

```

```

22 template <class Z> vector<Z> mul(const vector<Z>& a, const vector<Z>& b) {
23     int n = a.size(), k = __builtin_ctz((unsigned)n);
24     assert(n && !(n & (n - 1)) && b.size() == n);
25     vector<vector<Z>> x(k + 1, vector<Z>(n)), y(k + 1, vector<Z>(n)), z(k + 1, vector<Z>(n));
26     for (int s = 0; s < n; ++s) {
27         int d = __builtin_popcount((unsigned)s);
28         x[d][s] = a[s];
29         y[d][s] = b[s];
30     }
31     for (int i = 0; i <= k; ++i) {
32         zetaSub(x[i]);
33         zetaSub(y[i]);
34     }
35     for (int i = 0; i <= k; ++i) {
36         for (int j = 0; j <= i; ++j)
37             for (int s = 0; s < n; ++s) z[i][s] += x[j][s] * y[i - j][s];
38         mobSub(z[i]);
39     }
40     vector<Z> r(n);
41     for (int s = 0; s < n; ++s) r[s] = z[__builtin_popcount((unsigned)s)][s];
42     return r;
43 }
44 } // namespace SubConv

```

081 - 快速数论变换 (NTT, 极致优化) .cpp

```

1  /*
2  用途: 32 位 Montgomery + radix-4 NTT, 接口与 08 的 NTT<P,G> 一致。
3  用法: auto c = NTT<998244353, 3>::mul(a, b); 卡常时替换 08。
4  限制: P 为小于 2^30 的奇质数, G 为原根, 变换长度整除 P-1。
5  性能: 模数值与变换缓存必须保留 u32, 循环下标保留 i32; 改用宏展开后的 int 会显著变慢并扩大内存。
6  */
7
8  using i32 = int32_t;
9  using u32 = uint32_t;
10 using u64 = uint64_t;
11
12 template <u32 P, u32 G> struct NTT {
13     struct Z {
14         u32 x = 0;
15
16         static constexpr u32 askR() {
17             u32 r = P;
18             for (i32 i = 0; i < 5; ++i) r *= 2 - P * r;
19             return -r;
20         }
21
22         static constexpr u32 mr = askR();
23         static constexpr u32 m2 = -u64(P) % P;
24
25         static constexpr u32 red(u64 x) {
26             return (x + u64(u32(x) * mr) * P) >> 32;
27         }
28
29         Z() = default;
30         Z(int v) {
31             v %= P;
32             if (v < 0) v += P;
33             x = red(u64(v) * m2);
34         }
35
36         static Z raw(u32 x) {
37             Z a;
38             a.x = x;
39             return a;
40         }
41
42         u32 val() const {
43             u32 v = red(x);
44             return v >= P ? v - P : v;
45         }
46
47         Z pow(int n) const {
48             Z a = *this, r = 1;
49             while (n) {
50                 if (n & 1) r *= a;
51                 a *= a;
52                 n >>= 1;
53             }
54             return r;

```

```

55     }
56
57     Z inv() const {
58         return pow(P - 2);
59     }
60
61     Z& operator+=(Z a) {
62         x += a.x;
63         if (x >= 2 * P) x -= 2 * P;
64         return *this;
65     }
66
67     Z& operator-=(Z a) {
68         x += 2 * P - a.x;
69         if (x >= 2 * P) x -= 2 * P;
70         return *this;
71     }
72
73     Z& operator*=(Z a) {
74         x = red(u64(x) * a.x);
75         return *this;
76     }
77
78     friend Z operator+(Z a, Z b) { return a += b; }
79     friend Z operator-(Z a, Z b) { return a -= b; }
80     friend Z operator*(Z a, Z b) { return a *= b; }
81 };
82
83 static_assert(P & 1);
84 static_assert(P < (1u << 30));
85 static_assert(u32(-NTT::Z::mr * P) == 1);
86
87 static inline bool ok = false;
88 static inline array<Z, 30> rt, ir, q2, iq2, q3, iq3;
89
90 static void init() {
91     if (ok) return;
92     ok = true;
93     i32 k = __builtin_ctz(P - 1);
94     rt[k] = Z(G).pow((P - 1) >> k);
95     ir[k] = rt[k].inv();
96     for (i32 i = k; i--;) {
97         rt[i] = rt[i + 1] * rt[i + 1];
98         ir[i] = ir[i + 1] * ir[i + 1];
99     }
100     Z a = 1, b = 1;
101     for (i32 i = 0; i <= k - 2; ++i) {
102         q2[i] = rt[i + 2] * a;
103         iq2[i] = ir[i + 2] * b;
104         a *= ir[i + 2], b *= rt[i + 2];
105     }
106     a = b = 1;
107     for (i32 i = 0; i <= k - 3; ++i) {
108         q3[i] = rt[i + 3] * a;
109         iq3[i] = ir[i + 3] * b;
110         a *= ir[i + 3], b *= rt[i + 3];
111     }
112 }
113
114 static void ntt(vector<Z>& a, bool iv) {
115     i32 n = a.size();
116     assert(n && !(n & (n - 1)));
117     i32 h = __builtin_ctz(u32(n));
118     assert(h <= __builtin_ctz(P - 1));
119     init();
120
121     if (!iv) {
122         i32 len = 0;
123         while (len < h) {
124             if (h - len == 1) {
125                 i32 p = 1 << (h - len - 1);
126                 Z w = 1;
127                 for (i32 s = 0; s < (1 << len); ++s) {
128                     i32 o = s << (h - len);
129                     for (i32 i = 0; i < p; ++i) {
130                         Z x = a[o + i], y = a[o + i + p] * w;
131                         a[o + i] = x + y;
132                         a[o + i + p] = x - y;
133                     }
134                     w *= q2[__builtin_ctz(~u32(s))];
135                 }
136                 ++len;

```

```

137         } else {
138             i32 p = 1 << (h - len - 2);
139             Z w = 1, im = rt[2];
140             for (i32 s = 0; s < (1 << len); ++s) {
141                 Z w2 = w * w, w3 = w2 * w;
142                 i32 o = s << (h - len);
143                 for (i32 i = 0; i < p; ++i) {
144                     Z a0 = a[o + i];
145                     Z a1 = a[o + i + p] * w;
146                     Z a2 = a[o + i + 2 * p] * w2;
147                     Z a3 = a[o + i + 3 * p] * w3;
148                     Z x = (a1 - a3) * im;
149                     a[o + i] = a0 + a1 + a2 + a3;
150                     a[o + i + p] = a0 - a1 + a2 - a3;
151                     a[o + i + 2 * p] = a0 - a2 + x;
152                     a[o + i + 3 * p] = a0 - a2 - x;
153                 }
154                 w *= q3[__builtin_ctz(~u32(s))];
155             }
156             len += 2;
157         }
158     }
159 } else {
160     i32 len = h;
161     while (len) {
162         if (len == 1) {
163             i32 p = 1 << (h - len);
164             Z w = 1;
165             for (i32 s = 0; s < (1 << (len - 1)); ++s) {
166                 i32 o = s << (h - len + 1);
167                 for (i32 i = 0; i < p; ++i) {
168                     Z x = a[o + i], y = a[o + i + p];
169                     a[o + i] = x + y;
170                     a[o + i + p] = (x - y) * w;
171                 }
172                 w *= iq2[__builtin_ctz(~u32(s))];
173             }
174             --len;
175         } else {
176             i32 p = 1 << (h - len);
177             Z w = 1, im = ir[2];
178             for (i32 s = 0; s < (1 << (len - 2)); ++s) {
179                 Z w2 = w * w, w3 = w2 * w;
180                 i32 o = s << (h - len + 2);
181                 for (i32 i = 0; i < p; ++i) {
182                     Z a0 = a[o + i];
183                     Z a1 = a[o + i + p];
184                     Z a2 = a[o + i + 2 * p];
185                     Z a3 = a[o + i + 3 * p];
186                     Z x = (a2 - a3) * im;
187                     a[o + i] = a0 + a1 + a2 + a3;
188                     a[o + i + p] = (a0 - a1 + x) * w;
189                     a[o + i + 2 * p] = (a0 + a1 - a2 - a3) * w2;
190                     a[o + i + 3 * p] = (a0 - a1 - x) * w3;
191                 }
192                 w *= iq3[__builtin_ctz(~u32(s))];
193             }
194             len -= 2;
195         }
196     }
197     Z z = Z(n).inv();
198     for (auto& x : a) x *= z;
199 }
200
201 static u32 norm(int x) {
202     x %= P;
203     if (x < 0) x += P;
204     return x;
205 }
206
207 static vector<u32> mul32(const vector<int>& a, const vector<int>& b) {
208     if (a.empty() || b.empty()) return {};
209     i32 an = a.size(), bn = b.size();
210     i32 m = an + bn - 1;
211     if (min(an, bn) <= 32) {
212         vector<Z> x(an), y(bn), z(m);
213         for (i32 i = 0; i < an; ++i) x[i] = Z(a[i]);
214         for (i32 i = 0; i < bn; ++i) y[i] = Z(b[i]);
215         for (i32 i = 0; i < an; ++i)
216             for (i32 j = 0; j < bn; ++j) z[i + j] += x[i] * y[j];
217         vector<u32> r(m);
218     }

```



```

219         for (i32 i = 0; i < m; ++i) r[i] = z[i].val();
220         return r;
221     }
222
223     i32 n = 1;
224     while (n < m) n <= 1;
225     if (m - 2 <= n / 2) {
226         vector<int> x(a.begin(), a.end() - 1), y(b.begin(), b.end() - 1);
227         u32 al = norm(a.back()), bl = norm(b.back());
228         auto r = mul32(x, y);
229         r.resize(m);
230         r[m - 1] = u64(al) * bl % P;
231         for (i32 i = 0; i < an - 1; ++i)
232             r[i + bn - 1] = (r[i + bn - 1] + u64(norm(x[i])) * bl) % P;
233         for (i32 i = 0; i < bn - 1; ++i)
234             r[i + an - 1] = (r[i + an - 1] + u64(norm(y[i])) * al) % P;
235         return r;
236     }
237
238     vector<Z> x(n), y(n);
239     for (i32 i = 0; i < an; ++i) x[i] = Z(a[i]);
240     for (i32 i = 0; i < bn; ++i) y[i] = Z(b[i]);
241     bool eq = a == b;
242     ntt(x, false);
243     if (eq) y = x;
244     else ntt(y, false);
245     for (i32 i = 0; i < n; ++i) x[i] *= y[i];
246     ntt(x, true);
247     vector<u32> r(m);
248     for (i32 i = 0; i < m; ++i) r[i] = x[i].val();
249     return r;
250 }
251
252 static vector<int> mul(const vector<int>& a, const vector<int>& b) {
253     auto x = mul32(a, b);
254     return vector<int>(x.begin(), x.end());
255 }
256 };

```

08IA - 任意模卷积 (CRTNTT, 极致优化) .cpp

```

1  /*
2  用途：三次极低常数 NTT + CRT 的精确任意模卷积，可过针对浮点拆分的 FFT killer。
3  依赖：08I - 快速数论变换 (NTT, 极致优化)。
4  用法：CRTNTT f; auto c = f.mul(a, b, p)。输入会先规范到 [0,p)。
5  限制：p <= 2^32-1，变换长度不超过 2^21；三模乘积必须大于单项卷积的绝对上界。
6  性能：三份余数缓存必须保留 u32；改用宏展开后的 int 会额外占用大量内存并降低缓存命中率。
7  */
8
9  using u32 = uint32_t;
10 using u64 = uint64_t;
11
12 struct CRTNTT {
13     static constexpr u32 p1 = 998244353;
14     static constexpr u32 p2 = 1004535809;
15     static constexpr u32 p3 = 104857601;
16     static constexpr u32 i1 = 669690699;
17     static constexpr u32 i2 = 99747662;
18
19     static int norm(int x, int p) {
20         x %= p;
21         if (x < 0) x += p;
22         return x;
23     }
24
25     vector<int> mul(const vector<int>& x, const vector<int>& y, int p) const {
26         if (x.empty() || y.empty()) return {};
27         assert(0 < p && p <= 4294967295LL);
28         int nx = x.size(), ny = y.size();
29         assert(nx + ny - 1 <= (1LL << 21));
30         vector<int> a(nx), b(ny);
31         for (int i = 0; i < nx; ++i) a[i] = norm(x[i], p);
32         for (int i = 0; i < ny; ++i) b[i] = norm(y[i], p);
33
34         auto a1 = NTT<p1, 3>::mul32(a, b);
35         auto a2 = NTT<p2, 3>::mul32(a, b);
36         auto a3 = NTT<p3, 3>::mul32(a, b);
37         vector<int> r(a1.size());

```

```

38     int n = r.size();
39     u64 m1 = p1 % p;
40     u64 m2 = m1 * (p2 % p) % p;
41     for (int i = 0; i < n; ++i) {
42         u64 x1 = a1[i];
43         u64 x2 = (u64(a2[i]) + p2 - x1 % p2) * i1 % p2;
44         u64 x3 = (u64(a3[i]) + p3 - (x1 + x2 * p1) % p3) * i2 % p3;
45         r[i] = (x1 % p + x2 % p * m1 + x3 % p * m2) % p;
46     }
47     return r;
48 }
49 };

```

08J - 二维卷积 (2D NTT, 依赖08) .cpp

```

1  /*
2  用途：二维多项式卷积。返回  $c[i][j] = \sum a[x][y] b[i-x][j-y] \pmod P$ 。
3  依赖：08 - 快速数论变换 (NTT)。
4  用法：auto c = conv2<998244353, 3>(a,b), a/b 均为非空矩形矩阵。
5  复杂度： $O(NM(\log N + \log M))$ , N/M 为两维补齐后的 2 的幂。
6  前提：P 是原根 G 对应的 NTT 友好质数, 且 N/M 分别整除 P-1。
7  */
8
9  template <int P = 998244353, int G = 3>
10 vector<vector<int>> conv2(const vector<vector<int>>& a, const vector<vector<int>>& b) {
11     assert(!a.empty() && !b.empty() && !a[0].empty() && !b[0].empty());
12     int n1 = a.size(), m1 = a[0].size(), n2 = b.size(), m2 = b[0].size();
13     for (const auto& row : a) assert(row.size() == a[0].size());
14     for (const auto& row : b) assert(row.size() == b[0].size());
15     int n = n1 + n2 - 1, m = m1 + m2 - 1;
16     int nn = 1, mm = 1;
17     while (nn < n) nn <= 1;
18     while (mm < m) mm <= 1;
19
20     vector<vector<int>> x(nn, vector<int>(mm)), y(nn, vector<int>(mm));
21     for (int i = 0; i < n1; ++i) {
22         for (int j = 0; j < m1; ++j) {
23             x[i][j] = a[i][j] % P;
24             if (x[i][j] < 0) x[i][j] += P;
25         }
26     }
27     for (int i = 0; i < n2; ++i) {
28         for (int j = 0; j < m2; ++j) {
29             y[i][j] = b[i][j] % P;
30             if (y[i][j] < 0) y[i][j] += P;
31         }
32     }
33
34     auto ntt2 = [&](vector<vector<int>>& mat, bool iv) {
35         for (int i = 0; i < nn; ++i) NTT<P, G>::ntt(mat[i], iv);
36         vector<int> col(nn);
37         for (int j = 0; j < mm; ++j) {
38             for (int i = 0; i < nn; ++i) col[i] = mat[i][j];
39             NTT<P, G>::ntt(col, iv);
40             for (int i = 0; i < nn; ++i) mat[i][j] = col[i];
41         }
42     };
43     ntt2(x, false);
44     ntt2(y, false);
45     for (int i = 0; i < nn; ++i) {
46         for (int j = 0; j < mm; ++j) x[i][j] = x[i][j] * y[i][j] % P;
47     }
48     ntt2(x, true);
49     x.resize(n);
50     for (auto& v : x) v.resize(m);
51     return x;
52 }

```

08K - 有理函数第n项 (Bostan-Mori, 依赖08B) .cpp

```

1  /*
2  用途：求  $[x^n] f(x)/g(x)$ 。常用于由生成函数给出的超大下标递推。
3  依赖：02A - 自动取模 (MInt)、08B - 多项式 (Poly, Z)。
4  接口：divAt<Z,3>(f,g,n)。要求  $g[0] \neq 0$ , 返回  $f/g$  的第 n 项。
5  复杂度： $O(d \log d \log n)$ , d 为 g 的次数量级; 模数须为 NTT 友好质数。
6  */
7
8  template <class Z, int G = 3> Z divAt(vector<Z> f, vector<Z> g, int n) {

```

```

9   assert(n >= 0 && !g.empty() && g[0] != Z(0));
10  while (n) {
11      vector<Z> h = g;
12      for (int i = 1; i < h.size(); i += 2) h[i] = -h[i];
13      vector<Z> a = Poly<Z, G>::mul(f, h);
14      vector<Z> b = Poly<Z, G>::mul(g, h);
15      vector<Z> nf, ng;
16      for (int i = n & 1; i < a.size(); i += 2) nf.push_back(a[i]);
17      for (int i = 0; i < b.size(); i += 2) ng.push_back(b[i]);
18      f = move(nf);
19      g = move(ng);
20      n >>= 1;
21  }
22  return f[0] / g[0];
23 }

```

08L - 连续点值平移 (依赖08B) .cpp

```

1  /*
2  用途: 已知  $f(0), \dots, f(n)$ , 求  $f(m), \dots, f(m+n)$ 。
3  依赖: 02A - 自动取模 (MInt)、08B - 多项式 (Poly, Z)。
4  接口: shift<Z,3>(f,m),  $f$  的下标即横坐标。
5  前提:  $m == 0$ , 或  $n < m$  且  $m+n < \text{mod}$ ; 横坐标分母必须在模意义下可逆。
6  复杂度:  $O(n \log n)$ , 模数须为 NTT 友好质数。
7  */
8
9  template <class Z, int G = 3> vector<Z> shift(vector<Z> f, int m) {
10      int n = f.size() - 1;
11      assert(n >= 0);
12      if (!m) return f;
13      assert(n < m && m + n < Z::askMod());
14      vector<Z> fac(n + 1, Z(1)), ifac(n + 1, Z(1));
15      for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * Z(i);
16      ifac[n] = fac[n].inv();
17      for (int i = n; i > 0; --i) ifac[i - 1] = ifac[i] * Z(i);
18
19      vector<Z> a(n + 1), b(n * 2 + 1);
20      for (int i = 0; i <= n; ++i) {
21          a[i] = f[i] * ifac[i] * ifac[n - i];
22          if ((n - i) & 1) a[i] = -a[i];
23      }
24      for (int i = 0; i <= n * 2; ++i) b[i] = Z(m - n + i).inv();
25      a = Poly<Z, G>::mul(a, b);
26
27      Z cur = 1;
28      for (int i = m - n; i <= m; ++i) cur *= Z(i);
29      for (int i = 0; i <= n; ++i) {
30          f[i] = a[n + i] * cur;
31          cur *= Z(m + i + 1);
32          cur /= Z(m - n + i);
33      }
34      return f;
35 }

```

08M - 稀疏多项式除法.cpp

```

1  /*
2  用途: 计算  $g(x)/f(x)$  的前  $n$  项, 其中  $f$  稀疏且  $f[0] \neq 0$ 。
3  依赖: 02A - 自动取模 (MInt)。
4  接口: spDiv(g,f), 返回与  $g$  等长的截断商;  $g/f$  均按低次到高次存储。
5  复杂度:  $O(nk)$ ,  $k$  为  $f$  的非零项数。 $f$  很稀疏时比 NTT 求逆更短、更快。
6  */
7
8  template <class Z> vector<Z> spDiv(vector<Z> g, const vector<Z>& f) {
9      assert(!f.empty() && f[0] != Z(0));
10     vector<pair<int, Z>> nz;
11     for (int i = 1; i < f.size(); ++i) {

```

```

12         if (f[i] != Z(0)) nz.push_back({i, f[i]});
13     }
14     Z iv = f[0].inv();
15     for (int i = 0; i < g.size(); ++i) {
16         for (auto [j, x] : nz) {
17             if (j > i) break;
18             g[i] -= x * g[i - j];
19         }
20         g[i] *= iv;
21     }
22     return g;
23 }

```

08N - 多项式开方与幂 (依赖08B) .cpp

```

1  /*
2  用途：形式幂级数平方根与非负整数幂。
3  依赖：02A - 自动取模 (MInt)、02E - 模平方根、08B - 多项式 (Poly, Z)。
4  接口：psqrt<Z,3>(f,n) 返回截断到 n 项的平方根；ppow<Z,3>(f,e,n) 返回  $f^e$  的前 n 项。
5  前提：psqrt 的首个非零项次数为偶数且首项为二次剩余；模数须为 NTT 友好奇质数。
6  复杂度： $O(n \log n)$ 。psqrt 无解返回 nullopt。
7  */
8
9  template <class Z, int G = 3> optional<vector<Z>> psqrt(vector<Z> f, int n) {
10     assert(n >= 0);
11     f.resize(n);
12     int p = 0;
13     while (p < n && f[p] == Z(0)) ++p;
14     if (p == n) return vector<Z>(n);
15     if (p & 1) return nullopt;
16
17     int sh = p >> 1, m = n - p;
18     vector<Z> a(f.begin() + p, f.end());
19     auto rt = sqrt(a[0]);
20     if (!rt) return nullopt;
21     vector<Z> b{*rt};
22     Z iv = Z(2).inv();
23     for (int k = 2; k < m << 1; k <= 1) {
24         int len = min(k, m);
25         vector<Z> c(a.begin(), a.begin() + min<int>(a.size(), len));
26         c.resize(len);
27         c = Poly<Z, G>::mul(c, Poly<Z, G>::inv(b, len));
28         c.resize(len);
29         b.resize(len);
30         for (int i = 0; i < len; ++i) b[i] = (b[i] + c[i]) * iv;
31     }
32     vector<Z> ans(n);
33     for (int i = 0; i < b.size() && i + sh < n; ++i) ans[i + sh] = b[i];
34     return ans;
35 }
36
37 template <class Z, int G = 3> vector<Z> ppow(vector<Z> f, int e, int n) {
38     assert(e >= 0 && n >= 0);
39     if (!n) return {};
40     if (!e) {
41         vector<Z> ans(n);
42         ans[0] = Z(1);
43         return ans;
44     }
45     int p = 0;
46     while (p < f.size() && f[p] == Z(0)) ++p;
47     if (p == f.size() || (p && e > (n - 1) / p)) return vector<Z>(n);
48
49     int sh = p * e, m = n - sh;
50     Z c = f[p];
51     vector<Z> a(f.begin() + p, f.end());
52     a.resize(m);
53     for (Z& x : a) x /= c;
54     a = Poly<Z, G>::ln(a, m);
55     for (Z& x : a) x *= Z(e);
56     a = Poly<Z, G>::exp(a, m);
57     c = c.pow(e);
58     vector<Z> ans(n);
59     for (int i = 0; i < m; ++i) ans[i + sh] = a[i] * c;
60     return ans;
61 }

```

08NA - 多项式大指数幂（依赖08B）.cpp

```

1  /*
2  用途：求  $f(x)^e$  的前  $n$  项， $e$  为非负十进制整数串。
3  依赖：02A - 自动取模 (MInt)、08B - 多项式 (Poly, Z)。接口：ppow<Z,3>(f, e, n)。
4  限制：模数为 NTT 友好质数且  $n < \text{mod}$ ； $e$  只能含十进制数字。
5  区别：08N 的 ppow 接收普通整数指数；本分支保留超大指数环境。
6  */
7
8  template <class Z, int G = 3> vector<Z> ppow(vector<Z> f, string e, int n) {
9      assert(n >= 0);
10     if (!n) return {};
11     assert(n < Z::askMod());
12     assert(!e.empty());
13     for (char c : e) assert('0' <= c && c <= '9');
14     auto q = e.find_first_not_of('0');
15     e = q == string::npos ? "0" : e.substr(q);
16     if (e == "0") {
17         vector<Z> r(n);
18         r[0] = Z(1);
19         return r;
20     }
21
22     int p = 0;
23     f.resize(n);
24     while (p < n && f[p] == Z(0)) ++p;
25     if (p == n) return vector<Z>(n);
26
27     int lim = p ? (n - 1) / p : 0, v = 0;
28     bool big = false;
29     if (p) {
30         for (char c : e) {
31             int d = c - '0';
32             if (v > lim / 10 || (v == lim / 10 && d > lim % 10)) {
33                 big = true;
34                 break;
35             }
36             v = v * 10 + d;
37         }
38     }
39     if (big) return vector<Z>(n);
40
41     int md = Z::askMod(), a = 0, b = 0;
42     for (char c : e) {
43         a = (a * 10 + c - '0') % md;
44         b = (b * 10 + c - '0') % (md - 1);
45     }
46     int sh = p * v, m = n - sh;
47     Z c = f[p];
48     vector<Z> x(f.begin() + p, f.end());
49     x.resize(m);
50     for (Z& z : x) z /= c;
51     x = Poly<Z, G>::ln(x, m);
52     for (Z& z : x) z *= Z(a);
53     x = Poly<Z, G>::exp(x, m);
54     c = c.pow(b);
55
56     vector<Z> r(n);
57     for (int i = 0; i < m; ++i) r[i + sh] = x[i] * c;
58     return r;
59 }

```

08O - Power Projection（基础）.cpp

```

1  /*
2  用途：求  $h(i)=[x^N] f(x)g(x)^i$  ( $0 \leq i \leq n$ )。依赖：02A - 自动取模 (MInt)、08B - 多项式 (Poly, Z)。
3  接口：PowerProj<Z>::ask(f,g,N,n)。f/g 的  $N$  次以上系数自动忽略。
4  复杂度： $O(n \cdot N \cdot \log N)$ ，适合  $n$  或  $N$  中至少一个较小的场景；两者都很大时需专项分治实现。
5  */
6  template <class Z, int G = 3> struct PowerProj {
7      static vector<Z> ask(vector<Z> f, vector<Z> g, int N, int n) {
8          assert(N >= 0 && n >= 0);
9          f.resize(N + 1);
10         g.resize(N + 1);
11         vector<Z> cur{Z(1)}, ans(n + 1);

```

```

12     for (int i = 0; i <= n; ++i) {
13         for (int j = 0; j <= N; ++j) {
14             int k = N - j;
15             if (k < cur.size()) ans[i] += f[j] * cur[k];
16         }
17         if (i == n) break;
18         cur = Poly<Z, G>::mul(cur, g);
19         cur.resize(N + 1);
20     }
21     return ans;
22 }
23 };

```

080A - Power Projection (极致优化, 依赖08I) .cpp

```

1  /*
2  用途: 求  $h(i)=[x^N]f(x)g(x)^i$  ( $0 \leq i \leq n$ ) 的极致优化版。
3  依赖: 08I - 快速数论变换 (NTT, 极致优化)。接口: PPFast::ask(f, g, N, n)。
4  限制: 固定模数 998244353;  $n < \text{mod}$ ,  $N+1 \leq 2^{22}$ , 输入会截断到 N 次。
5  区别: 080 为短小通用基线; 本模板将二元消元与 NTT 排布融合, 适合 N、n 都大时。
6  性能: 大数组存 Montgomery 32 位 Z, 循环下标用 i32; 改回宏展开后的 int 会明显增大内存。
7  */
8
9  struct PPFast {
10     using T = NTT<998244353, 3>;
11     using Z = T::Z;
12
13     static vector<Z> mul(vector<Z> a, vector<Z> b) {
14         if (a.empty() || b.empty()) return {};
15         i32 m = a.size() + b.size() - 1;
16         if (min(a.size(), b.size()) <= 32) {
17             vector<Z> c(m);
18             for (i32 i = 0; i < a.size(); ++i)
19                 for (i32 j = 0; j < b.size(); ++j) c[i + j] += a[i] * b[j];
20             return c;
21         }
22         i32 n = 1;
23         while (n < m) n <<= 1;
24         a.resize(n), b.resize(n);
25         T::ntt(a, false);
26         T::ntt(b, false);
27         for (i32 i = 0; i < n; ++i) a[i] *= b[i];
28         T::ntt(a, true);
29         a.resize(m);
30         return a;
31     }
32
33     static vector<Z> pp0(vector<Z> wt, vector<Z> h, i32 m) {
34         i32 len = 1;
35         while (len < h.size()) len <<= 1;
36         assert(len <= (1 << 22));
37         h.resize(len), wt.resize(len);
38         for (Z& x : h) x = Z(0) - x;
39         reverse(wt.begin(), wt.end());
40         vector<Z> p = wt, q = h;
41         p.resize(4 * len), q.resize(4 * len);
42
43         vector<Z> w(len);
44         if (len > 1) {
45             T::init();
46             vector<i32> rv(len);
47             i32 lg = __builtin_ctz(len);
48             for (i32 i = 0; i < len; ++i) rv[i] = (rv[i >> 1] >> 1) | ((i & 1) << (lg - 1));
49             Z dw = T::rt[23].inv().pow((1 << 23) / (2 * len)), z = 1;
50             for (i32 i : rv) w[i] = z, z *= dw;
51         } else w[0] = 1;
52
53         for (i32 k = 1, cur = len; cur > 1; cur >>= 1, k <<= 1) {
54             vector<Z> fy(k), hx(2 * cur), f(2 * cur), g(2 * cur), x(cur), y(cur);
55             auto dy = [&](vector<Z>& a, i32 l, i32 r) {
56                 Z z = w[k >> 1].inv();
57                 for (i32 i = l; i < r; ++i) {
58                     for (i32 j = 0; j < k; ++j) fy[j] = a[2 * cur * j + i];
59                     T::ntt(fy, true);
60                     Z v = z;
61                     for (i32 j = 1; j < k; ++j, v *= z) fy[j] *= v;
62                     T::ntt(fy, false);
63                     for (i32 j = 0; j < k; ++j) a[2 * cur * (k + j) + i] = fy[j];
64                 }
65             };

```

```

66     auto fx = [&](vector<Z>& a, i32 l, i32 r) {
67         for (i32 j = l; j < r; ++j) {
68             for (i32 i = 0; i < 2 * cur; ++i) hx[i] = a[2 * cur * j + i];
69             T::ntt(hx, false);
70             for (i32 i = 0; i < 2 * cur; ++i) a[2 * cur * j + i] = hx[i];
71         }
72     };
73
74     if (cur <= k) {
75         dy(p, 0, cur), dy(q, 1, cur);
76         fx(p, 0, 2 * k), fx(q, 0, 2 * k);
77     } else {
78         fx(p, 0, k), fx(q, 0, k);
79         dy(p, 0, 2 * cur), dy(q, 0, 2 * cur);
80     }
81     for (i32 i = 0; i < 2 * cur * k; ++i) q[i] += Z(1);
82     for (i32 i = 2 * cur * k; i < 4 * cur * k; ++i) q[i] -= Z(1);
83
84     for (i32 j = 0; j < 2 * k; ++j) {
85         for (i32 i = 0; i < 2 * cur; ++i) f[i] = p[2 * cur * j + i], g[i] = q[2 * cur * j + i];
86         for (i32 i = 0; i < cur; ++i) {
87             x[i] = w[i] * (f[2 * i] * g[2 * i + 1] - f[2 * i + 1] * g[2 * i]);
88             y[i] = g[2 * i] * g[2 * i + 1];
89         }
90         T::ntt(x, true), T::ntt(y, true);
91         fill(x.begin() + cur / 2, x.end(), Z(0));
92         fill(y.begin() + cur / 2, y.end(), Z(0));
93         for (i32 i = 0; i < cur; ++i) p[cur * j + i] = x[i], q[cur * j + i] = y[i];
94     }
95     fill(p.begin() + 2 * cur * k, p.end(), Z(0));
96     fill(q.begin() + 2 * cur * k, q.end(), Z(0));
97     for (i32 j = 0; j < 4 * k; ++j) q[cur * j] = Z(0);
98 }
99
100 for (i32 i = 0; i < len; ++i) p[i] = p[2 * i];
101 p.resize(len);
102 Z z = Z(len).inv();
103 for (Z& x : p) x *= z;
104 T::ntt(p, true);
105 reverse(p.begin(), p.end());
106 p.resize(m + 1);
107 return p;
108 }
109
110 static vector<int> ask(vector<int> f, vector<int> g, int N, int n) {
111     assert(N >= 0 && n >= 0 && n < 998244353);
112     f.resize(N + 1), g.resize(N + 1);
113     vector<Z> a(N + 1), b(N + 1), fc(n + 1), ifc(n + 1);
114     for (i32 i = 0; i <= N; ++i) a[i] = Z(f[i]), b[i] = Z(g[i]);
115     fc[0] = 1;
116     for (i32 i = 1; i <= n; ++i) fc[i] = fc[i - 1] * Z(i);
117     ifc[n] = fc[n].inv();
118     for (i32 i = n; i; --i) ifc[i - 1] = ifc[i] * Z(i);
119
120     auto pp = [&](auto&& self, vector<Z> wt, vector<Z> h, i32 m) -> vector<Z> {
121         if (wt.size() < h.size()) wt.resize(h.size());
122         else if (h.size() < wt.size()) h.resize(wt.size());
123         if (h.empty()) return vector<Z>(m + 1);
124         if (h[0].val()) {
125             Z c = h[0];
126             h[0] = 0;
127             vector<Z> x = self(self, wt, h, m);
128             for (i32 i = 0; i <= m; ++i) x[i] *= ifc[i];
129             vector<Z> y(m + 1);
130             for (i32 i = 0; i <= m; ++i) y[i] = ifc[i] * c.pow(i);
131             x = mul(x, y);
132             x.resize(m + 1);
133             for (i32 i = 0; i <= m; ++i) x[i] *= fc[i];
134             return x;
135         }
136         if (h.size() == 1) {
137             vector<Z> x(m + 1);
138             x[0] = wt[0];
139             return x;

```

```

140     }
141     return pp0(wt, h, m);
142 };
143
144     vector<Z> wt(N + 1);
145     for (i32 i = 0; i <= N; ++i) wt[i] = a[N - i];
146     vector<Z> z = pp(pp, wt, b, n);
147     vector<int> r(n + 1);
148     for (i32 i = 0; i <= n; ++i) r[i] = z[i].val();
149     return r;
150 }
151 };

```

08P - 任意因子长度DFT（依赖08）.cpp

```

1  /*
2  用途：长度 n 整除 P-1 的 DFT/IDFT，不局限于 2 的幂。
3  依赖：08 - 快速数论变换（NTT）。用法：DFT<998244353, 3>::dft(a, false/true)。
4  复杂度：可分解为小因子时  $O(n \log n)$ ；大素因子会退化为  $O(r^2)$  的朴素 DFT。
5  限制：P 必须为 NTT 友好质数，输入系数先规范到  $[0, P)$ 。
6  */
7
8  template <int P, int G> struct DFT {
9      static int pow(int a, int n) {
10         return NTT<P, G>::pow(a, n);
11     }
12
13     static void naive(vector<int>& a, bool iv) {
14         int n = a.size();
15         if (n <= 1) return;
16         int w = pow(G, (P - 1) / n);
17         if (iv) w = pow(w, P - 2);
18         vector<int> b(n);
19         for (int i = 0; i < n; ++i) {
20             int q = pow(w, i), z = 1;
21             for (int j = 0; j < n; ++j) {
22                 b[i] = (b[i] + a[j] * z) % P;
23                 z = z * q % P;
24             }
25         }
26         if (iv) {
27             int z = pow(n, P - 2);
28             for (int& x : b) x = x * z % P;
29         }
30         a.swap(b);
31     }
32
33     static int fac(int n) {
34         for (int i = 3; i * i <= n; i += 2)
35             if (n % i == 0) return i;
36         return n;
37     }
38
39     static void dft(vector<int>& a, bool iv = false) {
40         int n = a.size();
41         if (!n) return;
42         assert((P - 1) % n == 0);
43         if (n <= 32) return naive(a, iv);
44         if (!(n & (n - 1))) return NTT<P, G>::ntt(a, iv);
45
46         int r = fac(n);
47         if (r == n) return naive(a, iv);
48         int m = n / r;
49         vector<int> b(n), t(max(r, m));
50         for (int x = 0; x < m; ++x) {
51             t.resize(r);
52             for (int y = 0; y < r; ++y) t[y] = a[x + m * y];
53             dft(t, iv);
54             for (int s = 0; s < r; ++s) b[s * m + x] = t[s];
55         }
56
57         int w = pow(G, (P - 1) / n);
58         if (iv) w = pow(w, P - 2);

```



```

59     for (int s = 0; s < r; ++s) {
60         int q = pow(w, s), z = 1;
61         t.resize(m);
62         for (int x = 0; x < m; ++x) {
63             t[x] = b[s * m + x] * z % P;
64             z = z * q % P;
65         }
66         dft(t, iv);
67         for (int x = 0; x < m; ++x) a[s + r * x] = t[x];
68     }
69 }
70 };

```

09 - 博弈论 (Nim、SG) .cpp

```

1  /*
2  适用范围：有限、无环、无偏（双方可走的边相同）、正常游戏规则（不能走者负）。
3  Nim：异或非 0 为先手必胜；misere 单独处理“最后一步输”的反常 Nim。
4  SG：多个独立子游戏的 Grundy 值异或；buildSG 要求所有转移均指向更小编号状态。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7
8  namespace Game {
9
10 inline bool nim(const vector<int>& pil) {
11     int xs = 0;
12     for (int pil : pil) {
13         assert(pil >= 0);
14         xs ^= pil;
15     }
16     return xs != 0;
17 }
18
19 // 返回 {堆下标, 操作后剩余石子数}；必败态返回 nullopt。
20 inline optional<pair<int, int>> nimMove(const vector<int>& a) {
21     int x = 0;
22     for (int v : a) assert(v >= 0), x ^= v;
23     if (!x) return nullopt;
24     for (int i = 0; i < a.size(); ++i) {
25         int v = a[i] ^ x;
26         if (v < a[i]) return pair<int, int>{i, v};
27     }
28     return nullopt;
29 }
30
31 inline bool bash(int n, int m) {
32     assert(n >= 0 && m > 0);
33     return n % (m + 1) != 0;
34 }
35
36 // 每次可同时修改至多 k 堆；P 态当且仅当每一二进制位的 1 数量均为 k+1 的倍数。
37 inline bool moore(const vector<int>& a, int k) {
38     assert(k >= 1);
39     for (int b = 0; b < 64; ++b) {
40         int c = 0;
41         for (auto x : a) c += x >> b & 1;
42         if (c % (k + 1)) return true;
43     }
44     return false;
45 }
46
47 // a[0] 表示第 1 级台阶；只异或奇数级台阶。
48 inline bool stair(const vector<int>& a) {
49     int x = 0;
50     for (int i = 0; i < a.size(); i += 2) assert(a[i] >= 0), x ^= a[i];
51     return x != 0;
52 }
53
54 inline bool misere(const vector<int>& pil) {
55     bool one = true;
56     int cnt = 0;
57     int xs = 0;
58     for (int pil : pil) {
59         assert(pil >= 0);
60         one &= pil <= 1;
61         cnt += pil > 0;
62         xs ^= pil;
63     }
64     return one ? (cnt % 2 == 0) : (xs != 0);
65 }

```

```

66
67 inline int mex(const vector<int>& a) {
68     vector<char> vis(a.size() + 1);
69     for (int x : a) if (0 <= x && x <= a.size()) vis[x] = true;
70     for (int x = 0;; ++x) if (!vis[x]) return x;
71 }
72
73 template <class F> vector<int> buildSG(int n, F to) {
74     assert(n >= 0);
75     vector<int> sg(n);
76     for (int s = 0; s < n; ++s) {
77         vector<int> a;
78         for (int v : to(s)) {
79             assert(0 <= v && v < s);
80             a.push_back(sg[v]);
81         }
82         sg[s] = mex(a);
83     }
84     return sg;
85 }
86
87 inline vector<int> subSG(int n, vector<int> mv) {
88     assert(n >= 0);
89     for (int x : mv) assert(x > 0);
90     sort(mv.begin(), mv.end());
91     mv.erase(unique(mv.begin(), mv.end()), mv.end());
92     return buildSG(n + 1, [&](int s) {
93         vector<int> a;
94         for (int x : mv) {
95             if (x > s) break;
96             a.push_back(s - x);
97         }
98         return a;
99     });
100 }
101
102 inline bool sumSG(const vector<int>& a) {
103     int x = 0;
104     for (int v : a) {
105         assert(v >= 0);
106         x ^= v;
107     }
108     return x != 0;
109 }
110
111 inline bool antiSG(const vector<int>& a) {
112     int x = 0, big = 0;
113     for (int v : a) assert(v >= 0), x ^= v, big += v > 1;
114     return big ? x != 0 : x == 0;
115 }
116
117 inline int lasker(int x) {
118     assert(x >= 0);
119     if (!x) return 0;
120     return x % 4 == 0 ? x - 1 : x % 4 == 3 ? x + 1 : x;
121 }
122
123 inline int treeSG(const vector<vector<int>>& g, int rt = 1) {
124     auto dfs = [&](auto&& go, int u, int fa) -> int {
125         int x = 0;
126         for (int v : g[u]) if (v != fa) x ^= go(go, v, u) + 1;
127         return x;
128     };
129     return dfs(dfs, rt, -1);
130 }
131
132 inline bool fib(int n) {
133     if (n <= 1) return false;
134     int a = 1, b = 2;
135     while (b < n) {
136         if (numeric_limits<int>::max() - b < a) return true;
137         auto c = a + b;
138         a = b, b = c;
139     }
140     return b != n;
141 }
142 } // namespace Game

```

09A - Wythoff Nim.cpp

```

1  /*
2  用途: 判断 Wythoff Nim (两堆, 每步可取一堆任意个或两堆各取相同正数) 的必败态。
3  结论: 设  $a \leq b$ 、 $k = b - a$ , 则  $(a, b)$  为必败态当且仅当  $a = \text{floor}(k * \phi)$ 。
4  浮点实现适合  $a, b$  约  $1e15$  及以内; 更大范围需额外做精确无理数比较。
5  */
6  using ld = long double;
7
8  namespace WythoffNim {
9  inline int flrPhi(int k) {
10     static const ld phi = (1.0L + sqrtl(5.0L)) / 2.0L;
11     return floorl(ld(k) * phi);
12 }
13
14 inline pair<int, int> lose(int k) {
15     int fst = flrPhi(k);
16     return {fst, fst + k};
17 }
18
19 inline bool ask(int fst, int b) {
20     if (fst > b) swap(fst, b);
21     return fst == flrPhi(b - fst);
22 }
23 } // namespace WythoffNim

```

10A - 圆周率计算 (Machin, with FixedPoint) .cpp

```

1  /*
2  用途: 用 Machin 公式计算圆周率。
3  依赖: 高精度 01A (或 01B) BigInt + 02A FixedPoint。适合约  $2 * 10^3$  位及以下; 更高位数使用 10B。
4  */
5
6  FixedPoint machin(int n) {
7      const int ex = 40;
8      const int pre = n + ex;
9      FixedPoint::setPre(pre);
10
11      auto arc = [&](int iv) -> FixedPoint {
12          FixedPoint trm = FixedPoint(1) / FixedPoint(iv);
13          FixedPoint sum = trm;
14          int sgn = -1;
15          FixedPoint iv2 = FixedPoint(1) / FixedPoint(iv * iv);
16          FixedPoint eps(BigInt(1), pre + 15, 1);
17          for (int k = 3;; k += 2) {
18              trm = trm * iv2;
19              FixedPoint cur = trm / FixedPoint(k);
20              if (cur.abs() < eps) break;
21              if (sgn > 0)
22                  sum = sum + cur;
23              else
24                  sum = sum - cur;
25              sgn = -sgn;
26          }
27          sum.round(FixedPoint::pre);
28          sum.norm();
29          return sum;
30      };
31
32      FixedPoint a1 = arc(5);
33      FixedPoint a2 = arc(239);
34      FixedPoint pi = a1 * FixedPoint(16) - a2 * FixedPoint(4);
35      pi.round(n);
36      pi.norm();
37      return pi;
38 }

```

10B - 圆周率计算 (Chudnovsky, with FixedPoint) .cpp

```

1  /*
2  用途: 用 Chudnovsky 公式计算圆周率。
3  依赖: 高精度 01B BigInt + 02A FixedPoint。适合约  $2 * 10^5$  位及以下; 低精度时 10A 常数更小。
4  */
5  FixedPoint chud(int n) {
6      const int ex = 20;
7      const int pre = n + ex;
8      FixedPoint::setPre(pre);

```

```

9
10 auto fac = [](int x) -> BigInt {
11     BigInt res(1);
12     for (int i = 2; i <= x; ++i) res *= BigInt(i);
13     return res;
14 };
15 auto tf = [](const BigInt& x) -> FixedPoint {
16     return FixedPoint(x, 0, 1);
17 };
18 auto sf = [&](const FixedPoint& x) -> FixedPoint {
19     FixedPoint y(to_string(sqrt(stod(x.str()))));
20     for (int i = 0; i < 80; ++i) {
21         y = (y + x / y) / FixedPoint(2);
22     }
23     y.round(pre);
24     y.norm();
25     return y;
26 };
27
28 FixedPoint sum(0);
29 string es = "0." + string(pre + 5, '0') + "1";
30 FixedPoint eps(es);
31
32 for (int k = 0; ++k) {
33     BigInt a = fac(6 * k) * (BigInt(545140134) * k + BigInt(13591409));
34     BigInt d = fac(3 * k) * BigInt::pow(fac(k), 3) * BigInt::pow(BigInt(640320), 3 * k);
35     FixedPoint trm = tf(a) / tf(d);
36     if (k % 2 == 1) trm = FixedPoint(0) - trm;
37     sum += trm;
38     if (trm.abs() < eps) break;
39 }
40
41 FixedPoint pi = FixedPoint(426880) * sf(FixedPoint(10005)) / sum;
42 pi.round(n);
43 pi.norm();
44 return pi;
45 }

```

11A - 牛顿迭代 (方程、平方根、开 k 次方) .cpp

```

1  /*
2  用途: Newton-Raphson 数值求根, 以及平方根、非负 k 次根。
3  前提: 函数在根附近可导且导数不接近 0; 失败返回 nullopt。浮点题应按量级设置 eps。
4  复杂度: 收敛区间内通常二次收敛; 远离根或多重根时不保证收敛。
5  */
6  using ld = long double;
7
8  namespace Newton {
9  template <class F, class DF>
10 optional<ld> solve(F f, DF df, ld x,
11                  ld eps = 1e-12L, int lim = 100) {
12     assert(eps > 0 && lim > 0);
13     ld cur = x;
14     for (int it = 0; it < lim; ++it) {
15         ld val = f(cur);
16         ld slp = df(cur);
17         if (!isfinite(val) || !isfinite(slp) || fabs(slp) <= eps) return nullopt;
18         ld nxt = cur - val / slp;
19         if (!isfinite(nxt)) return nullopt;
20         if (fabs(nxt - cur) <= eps * max(ld(1), fabs(nxt))) return nxt;
21         cur = nxt;
22     }
23     return nullopt;
24 }
25
26 inline ld sqrt(ld val, ld eps = 1e-15L) {
27     assert(val >= 0);
28     if (val == 0) return 0;
29     ld cur = max(ld(1), val);
30     for (int it = 0; it < 200; ++it) {
31         ld nxt = (cur + val / cur) / 2;
32         if (fabs(nxt - cur) <= eps * max(ld(1), fabs(nxt))) return nxt;
33         cur = nxt;
34     }
35     return cur;
36 }
37

```

```

38 inline ld root(ld val, int k, ld eps = 1e-15L) {
39     assert(val >= 0 && k >= 1);
40     if (val == 0 || k == 1) return val;
41     ld cur = max(ld(1), powl(val, 1.0L / k));
42     for (int it = 0; it < 200; ++it) {
43         ld nxt = (ld(k - 1) * cur + val / powl(cur, k - 1)) / k;
44         if (fabsl(nxt - cur) <= eps * max(ld(1), fabsl(nxt))) return nxt;
45         cur = nxt;
46     }
47     return cur;
48 }
49 } // namespace Newton

```

11B - 自适应辛普森积分.cpp

```

1  /*
2  用途: 数值计算定积分  $\int [l, r] f(x) dx$ 。
3  前提: 函数在区间上足够光滑; 奇点、极高频振荡、间断点应先分段处理。
4  复杂度: 自适应递归,  $\text{eps}$  为绝对误差目标,  $\text{maxDepth}$  防止病态数据无限递归。
5  */
6  using ld = long double;
7
8  namespace Simpson {
9  template <class F>
10 ld solve(F f, ld l, ld r, ld eps = 1e-10L, int dep = 30) {
11     assert(eps > 0 && dep >= 0);
12     if (l > r) return -solve(f, r, l, eps, dep);
13     auto sp = [](ld a, ld b, ld x, ld y, ld z) {
14         return (b - a) * (x + 4 * y + z) / 6;
15     };
16
17     ld mid = (l + r) / 2;
18     ld fl = f(l), fm = f(mid), fr = f(r);
19     ld s = sp(l, r, fl, fm, fr);
20     std::function<ld(ld, ld, ld, ld, ld, ld, ld, int)> dfs;
21     dfs = [&](ld a, ld b, ld x, ld y, ld z, ld v,
22             ld e, int d) -> ld {
23         ld m = (a + b) / 2;
24         ld lm = (a + m) / 2, rm = (m + b) / 2;
25         ld xl = f(lm), xr = f(rm);
26         ld sl = sp(a, m, x, xl, y);
27         ld sr = sp(m, b, y, xr, z);
28         ld err = sl + sr - v;
29         if (d == 0 || fabsl(err) <= 15 * e) {
30             return sl + sr + err / 15;
31         }
32         return dfs(a, m, x, xl, y, sl, e / 2, d - 1) +
33             dfs(m, b, y, xr, z, sr, e / 2, d - 1);
34     };
35     return dfs(l, r, fl, fm, fr, s, eps, dep);
36 }
37 } // namespace Simpson

```

11C - 三分搜索 (实数、整数).cpp

```

1  /*
2  用途: 在单峰/单谷函数上找最小值点; 实数版返回近似  $\text{argmin}$ , 整数版返回精确  $\text{argmin}$ 。
3  前提: 目标函数在给定闭区间上单峰 (或单谷); 不满足时不能套用。
4  复杂度: 实数版  $O(\text{iterations})$ , 整数版  $O(\log(r-l))$  次后常数枚举。
5  */
6  using ld = long double;
7
8  namespace Ternary {
9  template <class F>
10 ld argmin(ld l, ld r, F f, int lim = 200) {
11     assert(l <= r && lim > 0);
12     for (int it = 0; it < lim; ++it) {
13         ld fst = (2 * l + r) / 3;
14         ld b = (l + 2 * r) / 3;
15         if (f(fst) <= f(b)) r = b;
16         else l = fst;
17     }
18     return (l + r) / 2;
19 }
20
21 template <class F>
22 int argmin(int l, int r, F f) {
23     assert(l <= r);

```

```

24     while (r - l > 3) {
25         int fst = l + (r - l) / 3;
26         int b = r - (r - l) / 3;
27         if (f(fst) <= f(b)) r = b - 1;
28         else l = fst + 1;
29     }
30     int ans = l;
31     for (int val = l + 1; val <= r; ++val) {
32         if (f(val) < f(ans)) ans = val;
33     }
34     return ans;
35 }
36 } // namespace Ternary

```

11D - 数值求根（二分、割线）.cpp

```

1  /*
2  用途：连续函数的二分求根和割线法求根。
3  前提：二分要求端点异号或已有根；割线法需要足够好的两个初值。失败返回 nullopt。
4  复杂度：二分  $O(\log((r-l)/\epsilon))$ ；割线法通常超线性收敛但无全局保证。
5  */
6  using ld = long double;
7
8  namespace Root {
9  template <class F>
10 optional<ld> bisect(F f, ld l, ld r, ld eps = 1e-12L,
11                    int lim = 200) {
12     assert(l <= r && eps > 0 && lim > 0);
13     ld fl = f(l), fr = f(r);
14     if (fabsl(fl) <= eps) return l;
15     if (fabsl(fr) <= eps) return r;
16     if ((fl < 0) == (fr < 0)) return nullopt;
17     for (int it = 0; it < lim; ++it) {
18         ld mid = (l + r) / 2;
19         ld fm = f(mid);
20         if (fabsl(fm) <= eps || r - l <= eps) return mid;
21         if ((fl < 0) == (fm < 0)) l = mid, fl = fm;
22         else r = mid, fr = fm;
23     }
24     return (l + r) / 2;
25 }
26
27 template <class F>
28 optional<ld> secant(F f, ld fst, ld b, ld eps = 1e-12L,
29                   int lim = 100) {
30     assert(eps > 0 && lim > 0);
31     ld fa = f(fst), fb = f(b);
32     for (int it = 0; it < lim; ++it) {
33         ld d = fb - fa;
34         if (fabsl(d) <= eps) return nullopt;
35         ld nxt = b - fb * (b - fst) / d;
36         if (!isfinite(nxt)) return nullopt;
37         if (fabsl(nxt - b) <= eps * max(ld(1), fabsl(nxt))) return nxt;
38         fst = b;
39         fa = fb;
40         b = nxt;
41         fb = f(b);
42     }
43     return nullopt;
44 }
45 } // namespace Root

```

12A - 位运算与子集枚举.cpp

```

1  /*
2  用途：常用位操作、子掩码与超掩码枚举。
3  前提：mask 使用 u64；超集枚举要求 bitCount < 64，实际通常 <= 20。
4  复杂度：子掩码枚举  $O(2^{\text{popcount}(\text{mask})})$ ，所有 mask 的子掩码总复杂度  $O(3^n)$ 。
5  */
6
7  using u64 = uint64_t;
8
9  namespace BitToolkit {
10 inline u64 lowbit(u64 val) {
11     return val & (0ULL - val);
12 }
13
14 inline int bitLen(u64 val) {

```

```

15     return val == 0 ? 0 : 64 - __builtin_clzll(val);
16 }
17
18 inline int ctz(u64 val) {
19     return val == 0 ? 64 : __builtin_ctzll(val);
20 }
21
22 template <class F> void submask(u64 msk, F vis) {
23     u64 s = msk;
24     while (true) {
25         vis(s);
26         if (s == 0) break;
27         s = (s - 1) & msk;
28     }
29 }
30
31 template <class F> void supset(u64 msk, int k, F vis) {
32     assert(0 <= k && k < 64 && (msk >> k) == 0);
33     u64 all = (1ULL << k) - 1;
34     submask(all ^ msk, [&](u64 c) { vis(msk | c); });
35 }
36
37 inline u64 nxtComb(u64 val) {
38     assert(val != 0);
39     u64 lb = lowbit(val);
40     u64 nxt = val + lb;
41     assert(nxt > val);
42     return nxt | ((val ^ nxt) / lb >> 2);
43 }
44 } // namespace BitToolkit

```

12B - 异或线性基 (XorBasis64) .cpp

```

1  /*
2  用途: 64 位异或线性基: 插入、可表示性、最大异或、化简后的基向量。
3  复杂度: 每次插入/查询 O(64), 化简 O(64^2)。可合并、可持久化等扩展见数据结构目录的线性基。
4  */
5
6  using u64 = uint64_t;
7
8  struct LB {
9      array<u64, 64> a{};
10     int rk = 0;
11
12     bool insert(u64 val) {
13         for (int bit = 63; bit >= 0; --bit) {
14             if (((val >> bit) & 1ULL) == 0) continue;
15             if (a[bit] == 0) {
16                 a[bit] = val;
17                 ++rk;
18                 return true;
19             }
20             val ^= a[bit];
21         }
22         return false;
23     }
24
25     bool ask(u64 val) const {
26         for (int bit = 63; bit >= 0; --bit) {
27             if (((val >> bit) & 1ULL) == 0) continue;
28             if (a[bit] == 0) return false;
29             val ^= a[bit];
30         }
31         return true;
32     }
33
34     u64 askMax(u64 sd = 0) const {
35         u64 ans = sd;
36         for (int bit = 63; bit >= 0; --bit) {
37             if ((ans ^ a[bit]) > ans) ans ^= a[bit];
38         }
39         return ans;
40     }
41
42     void reduce() {
43         for (int hi = 63; hi >= 0; --hi) {
44             if (a[hi] == 0) continue;
45             for (int low = hi - 1; low >= 0; --low) {
46                 if (a[low] != 0 && ((a[hi] >> low) & 1ULL)) a[hi] ^= a[low];
47             }

```

```

48     }
49 }
50
51 vector<u64> askVec() const {
52     LB z = *this;
53     z.reduce();
54     vector<u64> res;
55     for (int bit = 0; bit < 64; ++bit)
56         if (z.a[bit] != 0) res.push_back(z.a[bit]);
57     return res;
58 }
59 };

```

12C - 二进制 GCD 与 Gray 编码.cpp

```

1  /*
2  用途: Stein (二进制) gcd, 以及 Gray 编码互转。
3  复杂度: binaryGcd 为  $O(\log \max(a,b))$ ; Gray 编码/解码  $O(1)$  /  $O(\log \text{word bits})$ 。
4  */
5
6  using u64 = uint64_t;
7
8  namespace BinArith {
9      inline u64 gcd(u64 l, u64 r) {
10         if (l == 0) return r;
11         if (r == 0) return l;
12         int s = __builtin_ctzll(l | r);
13         l >>= __builtin_ctzll(l);
14         do {
15             r >>= __builtin_ctzll(r);
16             if (l > r) swap(l, r);
17             r -= l;
18         } while (r != 0);
19         return l << s;
20     }
21
22     inline u64 gray(u64 val) {
23         return val ^ (val >> 1);
24     }
25
26     inline u64 igray(u64 g) {
27         for (int sh = 1; sh < 64; sh <= 1) g ^= g >> sh;
28         return g;
29     }
30 } // namespace BinArith

```

13A - Fibonacci (快速倍增) .cpp

```

1  /*
2  用途:  $O(\log n)$  求 Fibonacci; T 可为整数、MInt 或高精整数。
3  */
4
5  template <class T> pair<T, T> fib2(int n) {
6      assert(n >= 0);
7      if (!n) return {T(0), T(1)};
8      auto [x, y] = fib2<T>(n >> 1);
9      T a = x * (T(2) * y - x);
10     T b = x * x + y * y;
11     return n & 1 ? pair<T, T>{b, a + b} : pair<T, T>{a, b};
12 }
13
14 template <class T> T fib(int n) {
15     return fib2<T>(n).first;
16 }

```

13B - 约瑟夫环.cpp

```

1  /*
2  用途: n 人 (0..n-1) 从 0 开始, 每次数 k 个并删除, 求最后幸存者。
3  复杂度:  $O(n)$ , 适合 n 可线性枚举; 返回 0-indexed 或 1-indexed 答案。

```



```

4  */
5
6  inline int joseph(int n, int stp) {
7      assert(n >= 1 && stp >= 1);
8      int ans = 0;
9      for (int siz = 2; siz <= n; ++siz) ans = (ans + stp) % siz;
10     return ans;
11 }
12
13 inline int joseph1(int n, int stp) {
14     return joseph(n, stp) + 1;
15 }

```

14 - 伯努利数与幂和.cpp

```

1  /*
2  用途：在自动取模类型 Z 上预处理伯努利数，并  $O(k)$  求  $1^k + \dots + n^k$ 。
3  约定： $B_1 = -1/2$ ；Z 需支持构造、加减乘除和 pow/inv；模数应满足除法前提。
4  接口：Bernoulli<Z> b(d)，b.ask(k,n) 求前  $n$  项，b.ask(k,l,r) 求闭区间和。
5  复杂度：预处理  $O(d^2)$ ，单次幂和  $O(k)$ ，空间  $O(d)$ 。
6  */
7  template <class Z> struct Bernoulli {
8      int d;
9      vector<Z> b, fac, iv;
10
11     explicit Bernoulli(int _d) : d(_d), b(_d + 2), fac(_d + 3), iv(_d + 3) {
12         assert(d >= 0);
13         fac[0] = 1;
14         for (int i = 1; i <= d + 2; ++i) fac[i] = fac[i - 1] * Z(i);
15         iv[d + 2] = fac[d + 2].inv();
16         for (int i = d + 2; i; --i) iv[i - 1] = iv[i] * Z(i);
17         b[0] = 1;
18         for (int i = 1; i <= d + 1; ++i) {
19             Z s = 0;
20             for (int j = 0; j < i; ++j) s += C(i + 1, j) * b[j];
21             b[i] = -s / Z(i + 1);
22         }
23     }
24
25     Z C(int n, int k) const {
26         if (k < 0 || k > n) return 0;
27         return fac[n] * iv[k] * iv[n - k];
28     }
29
30     //  $1^k + 2^k + \dots + n^k$ ,  $n=0$  时返回 0。
31     Z ask(int k, int n) const {
32         assert(0 <= k && k <= d && n >= 0);
33         vector<Z> pw(k + 2);
34         pw[0] = 1;
35         Z x = Z(n + 1);
36         for (int i = 1; i <= k + 1; ++i) pw[i] = pw[i - 1] * x;
37         Z s = 0, q = 0;
38         for (int j = 0; j <= k + 1; ++j) {
39             Z c = C(k + 1, j) * b[j];
40             s += c * pw[k + 1 - j];
41             q += c;
42         }
43         return (s - q) / Z(k + 1);
44     }
45
46     //  $l^k + (l+1)^k + \dots + r^k$ , 要求  $1 \leq l \leq r$ 。
47     Z ask(int k, int l, int r) const {
48         assert(1 <= l && l <= r);
49         return ask(k, r) - ask(k, l - 1);
50     }
51 };

```

14A - 伯努利数 (NTT快速求法).cpp

```

1  /*
2  用途：用形式幂级数快速求 Bernoulli 数  $B_0 \dots B_n$ ，约定  $B_1 = -1/2$ 。
3  依赖：02A - 自动取模 (MInt)、08B 或 08C - 多项式 (Poly, Z)。
4  公式： $x / (\exp(x) - 1) = \sum B_k x^k / k!$ 。
5  复杂度： $O(n \log n)$ ；普通 14 的  $O(n^2)$  版本适合小阶数并直接支持幂和查询。
6  限制：Z 的模数须为 NTT 友好质数，且  $n + 1 < \text{mod}$ 。
7  */

```

```
8 template <class Z, int G = 3> vector<Z> bernFast(int n) {
9     assert(0 <= n && n + 1 < Z::askMod());
10    vector<Z> fac(n + 2), iv(n + 2), h(n + 1);
11    fac[0] = 1;
12    for (int i = 1; i <= n + 1; ++i) fac[i] = fac[i - 1] * Z(i);
13    iv[n + 1] = fac[n + 1].inv();
14    for (int i = n + 1; i; --i) iv[i - 1] = iv[i] * Z(i);
15    for (int i = 0; i <= n; ++i) h[i] = iv[i + 1];
16    vector<Z> a = Poly<Z, G>::inv(h, n + 1);
17    for (int i = 0; i <= n; ++i) a[i] *= fac[i];
18    return a;
19 }
```

字符串

14 个代码模板 · 林月 · v1.0.0 · 2026.8.1

版块说明与知识结论

README.md

字符串模板

需求	模板	复杂度与选型
单模式匹配、border、周期	01A - KMP	预处理与匹配均为 $O(n + m)$ ；空模式串匹配所有位置。
匹配每个后缀的 LCP、扩展 KMP	01B - Z函数	$Z[0]$ 约定为串长；适合前缀匹配信息。
模式串状态 DP、避免或统计匹配	01AA - KMP自动机	$\$O($
静态最长回文、区间回文判定	02A - Manacher	$O(n)$ ；模板开关分别控制奇、偶半径。
在线不同回文、回文出现次数	02B - 回文自动机	均摊 $O(n)$ ；askCnt 需在全部 append 后调用。
多模式串出现次数	03 - AC自动机	先 add 所有非空模式串，再 build，askCnt(text)[end] 取终点答案。
不同子串、子串出现次数、在线追加、两串 LCS	04A - 后缀自动机	均摊 $O(n)$ ；用 append 顺序建串后，askCnt 才有出现次数语义。
后缀字典序、相邻 LCP、任意序列后缀排序	04B - 后缀数组	倍增 $O(n \log n)$ ；任意两后缀 LCP 用 rank 区间上的 lc 最小值。
超长串后缀排序、相邻 LCP	04C - 后缀数组 (SA-IS)	已压缩整数序列为 $O(n)$ ；通用序列压缩阶段为 $O(n \log n)$ 。
静态子串相等、LCP	05 - 字符串哈希	$O(n)$ 预处理、 $O(1)$ 相等、 $O(\log n)$ LCP；有碰撞风险。
循环串最小/最大字典序起点	06 - 最小表示法	Booth, $O(n)$ 。
Lyndon 因子分解	07 - Lyndon 分解	Duval, $O(n)$ ，返回各因子的 $[l, r)$ 。
大量子序列判定	08A / 08B - 子序列自动机	任意可排序类型的位置表 / 小字符集的稠密 $O(m)$ 查询。

编号与边界

- 01 是“单模式匹配”概念组，01A (KMP) 与 01B (Z) 是同一任务的两种线性实现；01AA 是以 01A 的前缀函数为前置的 DFA 分支。
- 02 是“回文”概念组，02A 优先解决静态半径和查询，02B 优先解决在线不同回文和出现次数。
- 04 是“后缀结构”概念组。04A 与 04B 都能处理子串问题，但没有互相完全替代：SAM 擅长在线构造和状态 DP，SA 擅长后缀字典序与 LCP/RMQ；04C 是 04B 的 SA-IS 高性能实现。
- 08A 与 08B 解决同一子序列查询：08A 空间线性且支持任意可排序序列；08B 仅适合连续小字符集，但每个查询严格线性。
- 普通字符串集合 Trie 位于 [数据结构](#)，不在此重复维护；AC 是在 Trie 上补失败指针后的多模式匹配结构。

使用提示

- KMP、Z、Manacher、Booth、Duval 均通过模板参数或容器类型支持可比较序列，并不限于小写字符串；AC、SAM、回文自动机默认连续小写字符集，可用模板参数 A、F 调整。
- StringHash 的 T 可取 u64（自然溢出）或自动取模类型 Z。对抗碰撞或结果作为判定依据时，应并行维护两组独立哈希，或在命中后再做原串比较。
- AC 的 build 会把缺失转移补成失败转移；build 后不再 add。重复模式串会返回同一终点，出现次数需按题意处理重复权重。

- SAM 的 askUni 返回不同非空子串数；状态出现次数只对应通过 append 依次加入的一条字符串。若以 extend 构建字符串集合或 Trie，需自行定义终点权值。
- 后缀数组中 $lc[r]$ 是 $sa[r]$ 与 $sa[r+1]$ 的 LCP；比较后缀 i, j 时，设 $x=\text{rank}[i], y=\text{rank}[j]$ ，答案为 $\min(lc[x..y-1])$ （先交换使 $x < y$ ）。静态 RMQ 可直接复用数据结构目录的 ST 表。
- SA-IS 的 $\text{build}(\text{string})$ 按字节处理， $\text{askLCP}(s, sa)$ 返回与 04B 相同定义的相邻 LCP。若输入是整数序列，先离散到 $[0, up]$ 后调用 $\text{build}(a, up)$ ，才能保持线性复杂度；直接使用通用 build 会自动排序压缩。
- 最长公共子序列 LCS 属于动态规划，常规恢复方案与 Bitset 卡常版见 10 - 动态规划/06A、06B。

border、周期、回文、后缀结构和子序列的性质见 [字符串知识与结论](#)。

字符串知识与结论.md

字符串知识与结论

1. 子串、子序列、前缀与 border

子串要求位置连续，子序列只要求下标递增。长度为 n 的串有 $n(n+1)/2$ 个非空位置子串、 2^n 个位置子序列，但不同内容数量可能更少。

border 是同时为真前缀与真后缀的串。KMP 的前缀函数沿 $pi[x-1]$ 反复跳可枚举所有 border；若 $n\%(n-pi[n-1])=0$ ，最小周期为 $n-pi[n-1]$ ，否则整串本身是周期块。

2. 匹配与哈希

- KMP 适合单模式匹配、border 与模式自动机；Z 函数给每个后缀和整串前缀的 LCP。
- AC 自动机是 Trie 加失败指针；统计出现次数时先在文本上累计终点，再按 fail 树拓扑逆序上传。
- 字符串哈希存在碰撞。对抗输入用双模/自然溢出双基，关键判等命中后可回原串核验。
- 字符类型若可能为负，作为数组下标前转 unsigned char。

3. 回文

- Manacher 的奇半径与偶半径定义不同，查询区间前先确认半径是否包含中心。
- 回文树每个节点表示一个不同回文子串；两棵根分别代表长度 0 与 -1，失败指针用于寻找最长可扩展回文后缀。
- 回文出现次数需在整串加入后按长度递减沿 fail 指针汇总。

4. 后缀结构

- 后缀数组给出后缀字典序， $\text{LCP}(sa[i], sa[j])$ 是排名区间上相邻 LCP 的最小值，可接 RMQ。
- 后缀自动机状态代表一组 endpos 相同的子串，状态贡献不同子串数为 $\text{len}[v]-\text{len}[\text{link}[v]]$ 。
- 两串最长公共子串可让一串建 SAM、另一串沿转移匹配；最长公共子序列则是 DP，不是 SAM 问题。
- SA-IS 的线性复杂度要求整数序列已压到连续小值域；通用排序压缩会额外产生 $O(n \log n)$ 。

5. 序列查询与分解

- 子序列自动机的位置表版空间 $O(n)$ 、每字符二分；稠密转移版空间 $O(nA)$ 、查询 $O(m)$ ，字符集大时不能使用后者。
- Booth 求循环串最小表示；Duval 把串唯一分解为非增的 Lyndon 串序列。Lyndon 技巧常用于最小后缀、runs 与字典序构造。
- LCS 常规 DP 可恢复方案；只求字节串长度时，位并行公式把一行转移压成若干 u64 运算，见动态规划 06B。

01A - KMP.cpp

```

1  /*
2  用途：单模式串匹配、border 与前缀函数。
3  约定：前缀函数和匹配位置使用 int；空模式串匹配所有 n+1 个位置。
4  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
5  */
6  struct KMP {
7
8      string p;
9      vector<int> pi;
10
11     explicit KMP(const string& s) : p(s) {

```

```

12     build();
13 }
14
15 void build() {
16     int n = p.size();
17     pi.assign(n, 0);
18     for (int i = 1, j = 0; i < n; ++i) {
19         while (j && p[i] != p[j]) j = pi[j - 1];
20         if (p[i] == p[j]) ++j;
21         pi[i] = j;
22     }
23 }
24
25 vector<int> ask(const string& s) const {
26     vector<int> a;
27     int n = s.size(), m = p.size();
28     if (!m) {
29         a.resize(n + 1);
30         iota(a.begin(), a.end(), 0);
31         return a;
32     }
33     for (int i = 0, j = 0; i < n; ++i) {
34         while (j && s[i] != p[j]) j = pi[j - 1];
35         if (s[i] == p[j]) ++j;
36         if (j == m) {
37             a.push_back(i - m + 1);
38             j = pi[j - 1];
39         }
40     }
41     return a;
42 }
43 };

```

01AA - KMP自动机.cpp

```

1  /*
2  用途: KMP 的 DFA 转移, 适合数字 DP、避免/统计模式串出现次数。
3  用法: 状态为已匹配前缀长度 0..|p|; askNxt 后用 ask 判断是否匹配完整模式串。
4  约定: 字符集为连续 [F,F+A), 允许空模式串。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  template <int A = 26, char F = 'a'> struct KMP {
8
9      string p;
10     vector<int> pi;
11     vector<array<int, A>> to;
12
13     KMP() = default;
14
15     explicit KMP(const string& s) {
16         build(s);
17     }
18
19     int id(char c) const {
20         int x = c - F;
21         assert(0 <= x && x < A);
22         return x;
23     }
24
25     void build(const string& s) {
26         p = s;
27         int n = p.size();
28         for (char c : p) id(c);
29         pi.assign(n, 0);
30         for (int i = 1, j = 0; i < n; ++i) {
31             while (j && p[i] != p[j]) j = pi[j - 1];
32             if (p[i] == p[j]) ++j;
33             pi[i] = j;
34         }
35         to.assign(n + 1, array<int, A>());
36         for (int i = 0; i <= n; ++i) {
37             for (int x = 0; x < A; ++x) {
38                 int j = i;
39                 char c = static_cast<char>(F + x);
40                 while (j && (j == n || p[j] != c)) j = pi[j - 1];
41                 if (j < n && p[j] == c) ++j;
42                 to[i][x] = j;
43             }
44         }
45     }

```

```

46
47     int askSz() const {
48         return p.size();
49     }
50
51     int askNxt(int p, int x) const {
52         assert(0 <= p && p <= askSz() && 0 <= x && x < A);
53         return to[p][x];
54     }
55
56     int askNxt(int p, char c) const {
57         return askNxt(p, id(c));
58     }
59
60     bool ask(int p) const {
61         return p == askSz();
62     }
63 };

```

01B - Z函数 (扩展KMP) .cpp

```

1  /*
2  用途: Z 函数、两串每个后缀与模式串的最长公共前缀。
3  约定: z[0]=|s|; askLCP(p, s)[i] 是 p 与 s[i..] 的 LCP 长度。
4  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
5  */
6  struct ZFunction {
7
8      template <class S> static vector<int> build(const S& s) {
9          int n = s.size();
10         vector<int> z(n);
11         if (!n) return z;
12         z[0] = n;
13         for (int l = 0, r = 0, i = 1; i < n; ++i) {
14             if (i < r) z[i] = min<int>(r - i, z[i - l]);
15             while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
16             if (i + z[i] > r) l = i, r = i + z[i];
17         }
18         return z;
19     }
20
21     template <class P, class S> static vector<int> askLCP(const P& p, const S& s) {
22         int n = s.size(), m = p.size();
23         vector<int> a(n);
24         if (!m) return a;
25         auto z = build(p);
26         for (int l = 0, r = 0, i = 0; i < n; ++i) {
27             if (i < r) a[i] = min<int>(r - i, z[i - l]);
28             while (a[i] < m && i + a[i] < n && p[a[i]] == s[i + a[i]]) ++a[i];
29             if (i + a[i] > r) l = i, r = i + a[i];
30         }
31         return a;
32     }
33
34     template <class P, class S> static vector<int> ask(const P& p, const S& s) {
35         int n = s.size(), m = p.size();
36         if (!m) {
37             vector<int> a(n + 1);
38             iota(a.begin(), a.end(), 0);
39             return a;
40         }
41         auto z = askLCP(p, s);
42         vector<int> a;
43         for (int i = 0; i < n; ++i)
44             if (z[i] == m) a.push_back(i);
45         return a;
46     }
47 };

```

02A - Manacher.cpp

```

1  /*
2  用途: 回文半径、区间回文判定与最长回文。
3  模板开关: Manacher<true, false> 只求偶回文; Manacher<false, true> 只求奇回文。
4  约定: 半径数组使用 int。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  template <bool E = true, bool O = true> struct Manacher {

```

```

8
9   int n = 0;
10  vector<int> o, e;
11
12  Manacher() = default;
13
14  template <class A> Manacher(int n, A a) {
15      init(n, a);
16  }
17
18  template <class It> Manacher(It l, It r) {
19      init(r - l, [&](int i) { return *(l + i); });
20  }
21
22  template <class S> explicit Manacher(const S& s) : Manacher(s.begin(), s.end()) {}
23
24  template <class A> void init(int m, A a) {
25      assert(m >= 0);
26      n = m;
27      if constexpr (O) {
28          o.assign(n, 0);
29          for (int l = 0, r = -1, i = 0; i < n; ++i) {
30              int k = i <= r ? min(o[l + r - i], r - i) + 1 : 1;
31              while (k <= i && k < n - i && a(i - k) == a(i + k)) ++k;
32              o[i] = k - 1;
33              if (i + k - 1 > r) l = i - k + 1, r = i + k - 1;
34          }
35      } else {
36          o.clear();
37      }
38      if constexpr (E) {
39          e.assign(n, 0);
40          for (int l = 0, r = -1, i = 0; i < n; ++i) {
41              int k = i <= r ? min(e[l + r - i + 1], r - i + 1) + 1 : 1;
42              while (k <= i && k < n - i + 1 && a(i - k) == a(i + k - 1)) ++k;
43              e[i] = k - 1;
44              if (i + k - 2 > r) l = i - k + 1, r = i + k - 2;
45          }
46      } else {
47          e.clear();
48      }
49  }
50
51  template <bool Odd> int askRad(int x) const {
52      assert(0 <= x && x < n);
53      if constexpr (Odd) {
54          static_assert(O, "odd radii are disabled");
55          return o[x];
56      } else {
57          static_assert(E, "even radii are disabled");
58          return e[x];
59      }
60  }
61
62  template <bool Odd> bool askPal(int l, int r) const {
63      assert(0 <= l && l <= r && r < n);
64      if constexpr (Odd) {
65          static_assert(O, "odd radii are disabled");
66          assert(!((l ^ r) & 1));
67          return o[(l + r) >> 1] >= (r - l) / 2;
68      } else {
69          static_assert(E, "even radii are disabled");
70          assert((l ^ r) & 1);
71          return e[(l + r + 1) >> 1] > (r - l) / 2;
72      }
73  }
74
75  bool ask(int l, int r) const {
76      assert(0 <= l && l <= r && r < n);
77      if ((l ^ r) & 1) {
78          if constexpr (E) return askPal<false>(l, r);
79          assert(false);
80          return false;
81      }
82      if constexpr (O) return askPal<true>(l, r);
83      assert(false);
84      return false;
85  }
86
87  array<int, 2> askMax() const {
88      array<int, 2> a{0, 0};
89      if constexpr (O) {

```

```

90         for (int i = 0; i < n; ++i) {
91             array<int, 2> b{i - o[i], i + o[i] + 1};
92             if (b[1] - b[0] > a[1] - a[0]) a = b;
93         }
94     }
95     if constexpr (E) {
96         for (int i = 0; i < n; ++i) {
97             array<int, 2> b{i - e[i], i + e[i]};
98             if (b[1] - b[0] > a[1] - a[0]) a = b;
99         }
100     }
101     return a;
102 }
103
104 int askLen() const {
105     auto a = askMax();
106     return a[1] - a[0];
107 }
108 };

```

02B - 回文自动机 (PalindromicTree) .cpp

```

1  /*
2  用途：在线维护所有本质不同回文串、每个回文串的出现次数。
3  用法：append 后 askUni() 是本质不同回文数；askCnt()[state] 是该状态回文的总出现次数。
4  约定：字符集为连续 [F,F+A)，节点和出现次数使用 int。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  template <int A = 26, char F = 'a'> struct PAM {
8
9      struct Node {
10         int len = 0, lk = 0, cnt = 0;
11         array<int, A> to{};
12     };
13
14     vector<Node> t;
15     vector<int> s;
16     int lst = 1, mx = 0;
17
18     PAM() {
19         init();
20     }
21
22     void init() {
23         t.assign(2, Node());
24         t[0].len = -1;
25         s.assign(1, -1);
26         lst = 1;
27         mx = 0;
28     }
29
30     int id(char c) const {
31         int x = c - F;
32         assert(0 <= x && x < A);
33         return x;
34     }
35
36     int get(int p, int x) const {
37         int n = s.size() - 1;
38         while (s[n - 1 - t[p].len] != x) p = t[p].lk;
39         return p;
40     }
41
42     int append(int x) {
43         assert(0 <= x && x < A);
44         s.push_back(x);
45         int p = get(lst, x);
46         if (t[p].to[x]) {
47             lst = t[p].to[x];
48             ++t[lst].cnt;
49             mx = max(mx, t[lst].len);
50             return lst;
51         }
52         int q = t.size();
53         t.emplace_back();
54         t[q].len = t[p].len + 2;
55         t[p].to[x] = q;
56         if (t[q].len == 1) {
57             t[q].lk = 1;
58         } else {

```



```

59         p = get(t[p].lk, x);
60         t[q].lk = t[p].to[x];
61     }
62     lst = q;
63     ++t[lst].cnt;
64     mx = max(mx, t[lst].len);
65     return lst;
66 }
67
68 int append(char c) {
69     return append(id(c));
70 }
71
72 void append(const string& a) {
73     for (char c : a) append(c);
74 }
75
76 int askSz() const {
77     return t.size();
78 }
79
80 int askUni() const {
81     return askSz() - 2;
82 }
83
84 int askMax() const {
85     return mx;
86 }
87
88 vector<int> askOrd() const {
89     vector<int> c(s.size()), a(askSz() - 1);
90     for (int i = 1; i < askSz(); ++i) ++c[t[i].len];
91     for (int i = 1; i < c.size(); ++i) c[i] += c[i - 1];
92     for (int i = 1; i < askSz(); ++i) a[--c[t[i].len]] = i;
93     return a;
94 }
95
96 vector<int> askCnt() const {
97     vector<int> a(askSz());
98     for (int i = 1; i < askSz(); ++i) a[i] = t[i].cnt;
99     auto o = askOrd();
100     for (int i = o.size(); --i > 0;) {
101         int p = o[i];
102         a[t[p].lk] += a[p];
103     }
104     return a;
105 }
106 };

```

03 - AC自动机.cpp

```

1  /*
2  用途：多模式串匹配自动机。
3  用法：add 返回模式串终点；build 后 askCnt(text)[end] 是该模式串出现次数。
4  约定：节点编号、转移表和 BFS 队列使用 int；字符集可通过 A、F 适配。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  template <int A = 26, char F = 'a'> struct ACAM {
8
9      struct Node {
10         int len = 0, lk = 0;
11         array<int, A> to{};
12     };
13
14     vector<Node> t;
15     vector<int> ord;
16     bool ok = false;
17
18     ACAM() {
19         init();
20     }
21
22     void init() {
23         t.assign(2, Node());
24         t[0].to.fill(1);
25         t[0].len = -1;
26         ord.clear();
27         ok = false;
28     }
29

```

```

30 int node() {
31     t.emplace_back();
32     return t.size() - 1;
33 }
34
35 int id(char c) const {
36     int x = c - F;
37     assert(0 <= x && x < A);
38     return x;
39 }
40
41 int add(const string& s) {
42     assert(!ok);
43     assert(!s.empty());
44     int p = 1;
45     for (char c : s) {
46         int x = id(c);
47         if (!t[p].to[x]) {
48             t[p].to[x] = node();
49             t[t[p].to[x]].len = t[p].len + 1;
50         }
51         p = t[p].to[x];
52     }
53     return p;
54 }
55
56 void build() {
57     if (ok) return;
58     queue<int> q;
59     q.push(1);
60     ord.clear();
61     ord.reserve(t.size() - 1);
62     ord.push_back(1);
63     while (!q.empty()) {
64         int x = q.front();
65         q.pop();
66         for (int i = 0; i < A; ++i) {
67             if (!t[x].to[i]) {
68                 t[x].to[i] = t[t[x].lk].to[i];
69             } else {
70                 t[t[x].to[i]].lk = t[t[x].lk].to[i];
71                 q.push(t[x].to[i]);
72                 ord.push_back(t[x].to[i]);
73             }
74         }
75     }
76     ok = true;
77 }
78
79 int askNxt(int p, int x) const {
80     assert(0 <= p && p < askSz() && 0 <= x && x < A);
81     return t[p].to[x];
82 }
83
84 int askNxt(int p, char c) const {
85     return askNxt(p, id(c));
86 }
87
88 int askLink(int p) const {
89     return t[p].lk;
90 }
91
92 int askLen(int p) const {
93     return t[p].len;
94 }
95
96 int askSz() const {
97     return t.size();
98 }
99
100 const vector<int>& askOrd() const {
101     assert(ok);
102     return ord;
103 }
104
105 vector<int> askCnt(const string& s) const {
106     assert(ok);
107     vector<int> a(askSz());

```

```

108     int p = 1;
109     for (char c : s) {
110         p = askNxt(p, c);
111         ++a[p];
112     }
113     for (int i = ord.size(); --i > 0;) {
114         int u = ord[i];
115         a[t[u].lk] += a[u];
116     }
117     return a;
118 }
119 };

```

04A - 后缀自动机.cpp

```

1  /*
2  用途：后缀自动机，append 后可求不同子串数及各状态出现次数。
3  约定：状态编号、转移和出现次数使用 int。
4  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
5  */
6  template <int A = 26, char F = 'a'> struct SAM {
7
8      struct Node {
9          int len = 0, lk = 0, cnt = 0;
10         array<int, A> to{};
11     };
12
13     vector<Node> t;
14     int lst = 1;
15
16     SAM() {
17         init();
18     }
19
20     void init() {
21         t.assign(2, Node());
22         t[0].to.fill(1);
23         t[0].len = -1;
24         lst = 1;
25     }
26
27     int node() {
28         t.emplace_back();
29         return t.size() - 1;
30     }
31
32     int extend(int p, int c) {
33         assert(0 <= p && p < askSz() && 0 <= c && c < A);
34         if (t[p].to[c]) {
35             int q = t[p].to[c];
36             if (t[q].len == t[p].len + 1) return q;
37             int r = node();
38             t[r].len = t[p].len + 1;
39             t[r].lk = t[q].lk;
40             t[r].to = t[q].to;
41             t[q].lk = r;
42             while (t[p].to[c] == q) {
43                 t[p].to[c] = r;
44                 p = t[p].lk;
45             }
46             return r;
47         }
48         int q = node();
49         t[q].len = t[p].len + 1;
50         while (!t[p].to[c]) {
51             t[p].to[c] = q;
52             p = t[p].lk;
53         }
54         t[q].lk = extend(p, c);
55         return q;
56     }
57
58     int id(char c) const {
59         int x = c - F;
60         assert(0 <= x && x < A);
61         return x;
62     }
63
64     int extend(int p, char c) {
65         return extend(p, id(c));
66     }

```

```

66     }
67
68     int append(char c) {
69         lst = extend(lst, c);
70         ++t[lst].cnt;
71         return lst;
72     }
73
74     void append(const string& s) {
75         for (char c : s) append(c);
76     }
77
78     int askNxt(int p, int x) const {
79         return t[p].to[x];
80     }
81
82     int askNxt(int p, char c) const {
83         return askNxt(p, id(c));
84     }
85
86     int askLink(int p) const {
87         return t[p].lk;
88     }
89
90     int askLen(int p) const {
91         return t[p].len;
92     }
93
94     int askSz() const {
95         return t.size();
96     }
97
98     bool ask(const string& s) const {
99         int p = 1;
100         for (char c : s) {
101             p = askNxt(p, c);
102             if (!p) return false;
103         }
104         return true;
105     }
106
107     int askLCS(const string& s) const {
108         int p = 1, w = 0, r = 0;
109         for (char c : s) {
110             int x = id(c);
111             while (p != 1 && !t[p].to[x]) {
112                 p = t[p].lk;
113                 w = t[p].len;
114             }
115             if (t[p].to[x]) {
116                 p = t[p].to[x];
117                 ++w;
118             } else {
119                 p = 1;
120                 w = 0;
121             }
122             r = max(r, w);
123         }
124         return r;
125     }
126
127     vector<int> askOrd() const {
128         vector<int> c(askSz()), a(askSz() - 1);
129         for (int i = 1; i < askSz(); ++i) ++c[t[i].len];
130         for (int i = 1; i < askSz(); ++i) c[i] += c[i - 1];
131         for (int i = 1; i < askSz(); ++i) a[--c[t[i].len]] = i;
132         return a;
133     }
134
135     vector<int> askCnt() const {
136         vector<int> a(askSz());
137         for (int i = 1; i < askSz(); ++i) a[i] = t[i].cnt;
138         auto o = askOrd();
139         for (int i = o.size(); --i > 0;) {

```

```

140         int p = o[i];
141         a[t[p].lk] += a[p];
142     }
143     return a;
144 }
145
146 int askUni() const {
147     int r = 0;
148     for (int i = 2; i < askSz(); ++i) r += t[i].len - t[t[i].lk].len;
149     return r;
150 }
151 };

```

04B - 后缀数组 (SuffixArray) .cpp

```

1  /*
2  用途: 倍增后缀数组与 height (lc) 数组, 支持 string 或可比较整数字列。
3  约定: rank[i] 为后缀 i 的名次; lc[r] 为 sa[r]、sa[r+1] 的 LCP。
4  性能: 默认使用展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
5  */
6  struct SA {
7
8      int n = 0;
9      vector<int> sa, rk, lc;
10
11      SA() = default;
12
13      template <class S> explicit SA(const S& s) {
14          build(s);
15      }
16
17      template <class S> void build(const S& s) {
18          n = s.size();
19          sa.resize(n);
20          rk.resize(n);
21          lc.assign(n ? n - 1 : 0, 0);
22          if (!n) return;
23          iota(sa.begin(), sa.end(), 0);
24          sort(sa.begin(), sa.end(), [&](int a, int b) { return s[a] < s[b]; });
25          rk[sa[0]] = 0;
26          for (int i = 1; i < n; ++i) rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
27
28          vector<int> a, b, c(n);
29          a.reserve(n);
30          for (int w = 1; rk[sa[n - 1]] < n - 1; w <= 1) {
31              a.clear();
32              for (int i = max<int>(0, n - w); i < n; ++i) a.push_back(i);
33              for (int i : sa)
34                  if (i >= w) a.push_back(i - w);
35              fill(c.begin(), c.end(), 0);
36              for (int i = 0; i < n; ++i) ++c[rk[i]];
37              for (int i = 1; i < n; ++i) c[i] += c[i - 1];
38              for (int i = n - 1; i >= 0; --i) sa[--c[rk[sa[i]]]] = a[i];
39              b = rk;
40              rk[sa[0]] = 0;
41              auto get = [&](int p) { return p < n ? b[p] : -1; };
42              for (int i = 1; i < n; ++i) {
43                  int x = sa[i - 1], y = sa[i];
44                  rk[y] = rk[x] + (b[x] != b[y] || get(x + w) != get(y + w));
45              }
46          }
47
48          for (int i = 0, j = 0; i < n; ++i) {
49              if (!rk[i]) {
50                  j = 0;
51                  continue;
52              }
53              int p = sa[rk[i] - 1];
54              while (i + j < n && p + j < n && s[i + j] == s[p + j]) ++j;
55              lc[rk[i] - 1] = j;
56              if (j) --j;
57          }
58      }
59 };

```

04C - 后缀数组 (SA-IS, 线性) .cpp

```

1  /*

```

```

2 用途: SA-IS 后缀数组与 Kasai LCP: 字符集已压缩到  $[0, up]$  时为  $O(n)$ 。
3 用法: build(string) 直接处理字节串; 整数序列先压缩后调用 build(a, up)。
4 约定: 返回 sa, 所有下标为 int; 通用 build 会排序压缩, 复杂度为  $O(n \log n)$ 。
5 性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6 */
7 struct SA {
8
9     static vector<int> sais(const vector<int>& s, int up) {
10         int n = s.size();
11         assert(up >= 0);
12         for (int x : s) assert(0 <= x && x <= up);
13         if (!n) return {};
14         if (n == 1) return {0};
15         if (n == 2) return s[0] < s[1] ? vector<int>{0, 1} : vector<int>{1, 0};
16
17         vector<uint8_t> tp(n);
18         for (int i = n - 2; i >= 0; --i) {
19             tp[i] = s[i] == s[i + 1] ? tp[i + 1] : s[i] < s[i + 1];
20         }
21         vector<int> sl(up + 2), ss(up + 2), sa(n);
22         for (int i = 0; i < n; ++i) {
23             if (tp[i])
24                 ++sl[s[i] + 1];
25             else
26                 ++ss[s[i]];
27         }
28         for (int i = 0; i <= up; ++i) {
29             ss[i] += sl[i];
30             sl[i + 1] += ss[i];
31         }
32
33         auto ind = [&](const vector<int>& lms) {
34             fill(sa.begin(), sa.end(), -1);
35             vector<int> b = ss;
36             for (int x : lms)
37                 if (x < n) sa[b[s[x]]++] = x;
38             b = sl;
39             sa[b[s[n - 1]]++] = n - 1;
40             for (int i = 0; i < n; ++i) {
41                 int x = sa[i];
42                 if (x > 0 && !tp[x - 1]) sa[b[s[x - 1]]++] = x - 1;
43             }
44             b = sl;
45             for (int i = n - 1; i >= 0; --i) {
46                 int x = sa[i];
47                 if (x > 0 && tp[x - 1]) sa[--b[s[x - 1] + 1]] = x - 1;
48             }
49         };
50
51         vector<int> id(n + 1, -1), lms;
52         int m = 0;
53         for (int i = 1; i < n; ++i)
54             if (!tp[i - 1] && tp[i]) id[i] = m++, lms.push_back(i);
55         ind(lms);
56         if (!m) return sa;
57
58         vector<int> ord;
59         ord.reserve(m);
60         for (int x : sa)
61             if (id[x] != -1) ord.push_back(x);
62         vector<int> rec(m);
63         int rup = 0;
64         rec[id[ord[0]]] = 0;
65         for (int i = 1; i < m; ++i) {
66             int x = ord[i - 1], y = ord[i];
67             int rx = id[x] + 1 < m ? lms[id[x] + 1] : n;
68             int ry = id[y] + 1 < m ? lms[id[y] + 1] : n;
69             bool sam = rx - x == ry - y;
70             for (; sam && x < rx; ++x, ++y)
71                 if (s[x] != s[y]) break;
72             if (x == n || s[x] != s[y]) sam = false;
73             if (!sam) ++rup;
74             rec[id[ord[i]]] = rup;
75         }
76         auto rsa = sais(rec, rup);
77         for (int i = 0; i < m; ++i) ord[i] = lms[rsa[i]];
78         ind(ord);
79         return sa;
80     }
81
82     static vector<int> build(const string& s) {
83         vector<int> a(s.size());

```

```

84     for (int i = 0; i < s.size(); ++i) a[i] = static_cast<unsigned char>(s[i]);
85     return sais(a, 255);
86 }
87
88 static vector<int> build(const vector<int>& s, int up) {
89     return sais(s, up);
90 }
91
92 template <class S> static vector<int> build(const S& s) {
93     int n = s.size();
94     vector<int> a(n), p(n);
95     iota(p.begin(), p.end(), 0);
96     sort(p.begin(), p.end(), [&](int x, int y) { return s[x] < s[y]; });
97     int up = 0;
98     for (int i = 0; i < n; ++i) {
99         if (i && s[p[i - 1]] != s[p[i]]) ++up;
100         a[p[i]] = up;
101     }
102     return sais(a, up);
103 }
104
105 template <class S> static vector<int> askLCP(const S& s, const vector<int>& sa) {
106     int n = s.size();
107     assert(sa.size() == n);
108     vector<int> rk(n), a(n ? n - 1 : 0);
109     for (int i = 0; i < n; ++i) {
110         assert(0 <= sa[i] && sa[i] < n);
111         rk[sa[i]] = i;
112     }
113     for (int i = 0, h = 0; i < n; ++i) {
114         if (!rk[i]) continue;
115         int j = sa[rk[i] - 1];
116         while (i + h < n && j + h < n && s[i + h] == s[j + h]) ++h;
117         a[rk[i] - 1] = h;
118         if (h) --h;
119     }
120     return a;
121 }
122 };

```

05 - 字符串哈希 (StringHash, with T) .cpp

```

1  /*
2  用途: 静态子串哈希、子串比较与 LCP。
3  用法: StringHash<u64> h(s, base) 使用自然溢出; StringHash<Z> 使用自动取模类型。
4  约定: 区间为 [l,r), T 必须支持 +、-、* 和从整数构造; 哈希存在碰撞概率。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7
8  using u64 = uint64_t;
9
10 template <class T> struct StringHash {
11
12     int n = 0;
13     T b{};
14     vector<T> h, pw;
15
16     StringHash() = default;
17
18     template <class S> StringHash(const S& s, T x) {
19         build(s, x);
20     }
21
22     template <class S, class F> StringHash(const S& s, T x, F f) {
23         build(s, x, f);
24     }
25
26     template <class S> void build(const S& s, T x) {
27         build(s, x, [](const auto& c) {
28             return static_cast<u64>(static_cast<unsigned char>(c)) + 1;
29         });
30     }
31
32     template <class S, class F> void build(const S& s, T x, F f) {
33         n = s.size();
34         b = x;
35         h.assign(n + 1, T(0));
36         pw.assign(n + 1, T(0));
37         pw[0] = T(1);
38         for (int i = 0; i < n; ++i) {

```

```

39         pw[i + 1] = pw[i] * b;
40         h[i + 1] = h[i] * b + T(f(s[i]));
41     }
42 }
43
44 T ask(int l, int r) const {
45     assert(0 <= l && l <= r && r <= n);
46     return h[r] - h[l] * pw[r - l];
47 }
48
49 bool askEq(int l, int r, int x, int y) const {
50     assert(r - l == y - x);
51     return ask(l, r) == ask(x, y);
52 }
53
54 int askLCP(int l, int r) const {
55     assert(0 <= l && l <= n && 0 <= r && r <= n);
56     int x = 0, y = min(n - l, n - r) + 1;
57     while (x + 1 < y) {
58         int m = x + (y - x) / 2;
59         if (askEq(l, l + m, r, r + m))
60             x = m;
61         else
62             y = m;
63     }
64     return x;
65 }
66 };

```

06 - 最小表示法 (Booth) .cpp

```

1  /*
2  用途：循环串的最小或最大字典序表示起点。
3  用法：booth(s) 求最小表示法；booth<false>(s) 求最大表示法。
4  约定：返回 0-indexed 起点；空串返回 0；元素类型只需可比较。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  template <bool Min = true, class S> int booth(const S& s) {
8      int n = s.size();
9      if (!n) return 0;
10     int i = 0, j = 1, k = 0;
11     while (i < n && j < n && k < n) {
12         auto x = s[(i + k) % n], y = s[(j + k) % n];
13         if (x == y) {
14             ++k;
15             continue;
16         }
17         if ((x > y) == Min)
18             i += k + 1;
19         else
20             j += k + 1;
21         if (i == j) ++j;
22         k = 0;
23     }
24     return min(i, j);
25 }

```

07 - Lyndon 分解 (Duval) .cpp

```

1  /*
2  用途：Lyndon 分解，返回每个 Lyndon 因子的半开区间。
3  约定：返回的因子字典序单调不减；元素类型只需可比较。
4  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
5  */
6  template <class S> vector<array<int, 2>> duval(const S& s) {
7      int n = s.size();
8      vector<array<int, 2>> a;
9      for (int i = 0; i < n; i) {
10         int j = i + 1, k = i;
11         while (j < n && s[k] <= s[j]) {
12             if (s[k] < s[j])
13                 k = i;

```



```

14         else
15             ++k;
16         ++j;
17     }
18     int w = j - k;
19     while (i <= k) {
20         a.push_back({i, i + w});
21         i += w;
22     }
23 }
24 return a;
25 }

```

08A - 子序列自动机 (位置表) .cpp

```

1  /*
2  用途: 判定查询序列是否为固定主序列的子序列; 支持任意可排序元素类型。
3  预处理  $O(n \log \sigma)$ , 单次查询  $O(m(\log \sigma + \log n))$ , 空间  $O(n)$ 。
4  askPos 返回匹配最后一个元素的 0-indexed 位置; 空序列返回 -1。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T, class Cmp = less<T>> struct SubSeq {
8      Cmp cmp;
9      vector<T> val;
10     vector<vector<int>> pos;
11
12     SubSeq() = default;
13     template <class Seq> explicit SubSeq(const Seq& a) { build(a); }
14
15     template <class Seq> void build(const Seq& a) {
16         val.assign(a.begin(), a.end());
17         sort(val.begin(), val.end(), cmp);
18         auto eq = [&](const T& x, const T& y) { return !cmp(x, y) && !cmp(y, x); };
19         val.erase(unique(val.begin(), val.end(), eq), val.end());
20         pos.assign(val.size(), {});
21         int i = 0;
22         for (const auto& x : a) {
23             int k = lower_bound(val.begin(), val.end(), x, cmp) - val.begin();
24             pos[k].push_back(i++);
25         }
26     }
27
28     template <class Seq> optional<int> askPos(const Seq& a) const {
29         int at = -1;
30         for (const auto& x : a) {
31             auto p = lower_bound(val.begin(), val.end(), x, cmp);
32             if (p == val.end() || cmp(x, *p) || cmp(*p, x)) return nullopt;
33             int k = p - val.begin();
34             auto it = upper_bound(pos[k].begin(), pos[k].end(), at);
35             if (it == pos[k].end()) return nullopt;
36             at = *it;
37         }
38         return at;
39     }
40
41     template <class Seq> bool ask(const Seq& a) const { return askPos(a).has_value(); }
42 };

```

08B - 子序列自动机 (稠密转移) .cpp

```

1  /*
2  用途: 小且连续字符集上的子序列判定; 以  $O(nA)$  空间换  $O(m)$  查询。
3  默认处理 'a'..'z'。状态和转移显式使用 int。
4  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
5  */
6  template <int A = 26, char F = 'a'> struct SubSeq {
7      int n = 0;
8      vector<array<int, A>> nx;
9
10     SubSeq() = default;
11     explicit SubSeq(const string& s) { build(s); }
12
13     void build(const string& s) {
14         n = s.size();
15         nx.assign(n + 1, {});
16         nx[n].fill(0);
17         for (int i = n; i-- > 0;) {
18             nx[i] = nx[i + 1];

```

```
19         int c = static_cast<unsigned char>(s[i]) - static_cast<unsigned char>(F);
20         assert(0 <= c && c < A);
21         nx[i][c] = i;
22     }
23 }
24
25 optional<int> askPos(const string& s) const {
26     int p = 0, at = -1;
27     for (unsigned char x : s) {
28         int c = x - static_cast<unsigned char>(F);
29         if (c < 0 || c >= A || p > n) return nullopt;
30         int q = nx[p][c];
31         if (q == n) return nullopt;
32         at = q, p = q + 1;
33     }
34     return at;
35 }
36
37 bool ask(const string& s) const { return askPos(s).has_value(); }
38 };
```

高精度

8 个代码模板 · 林月 · v1.0.0 · 2026.8.1

版块说明与知识结论

README.md

高精度

编号	模板	常规 / 极致版的取舍
01A	整数 (BigInt, 常规)	10^9 进制; 加减 $O(n)$ 、朴素乘法 $O(nm)$ 、稳健逐位除法。适合一般高精度与代码量敏感的题。
01B	整数 (BigInt, 极致优化)	10^9 进制, 乘法在约 1500 个内部块后自动切 FFT; 两组实系数打包为一次正 FFT, 再做一次逆 FFT。适合万位以上乘法、阶乘和常数计算。该卡常分支按需使用 i32 / u32 压缩下标和十进制块。
02A	定点数 (FixedPoint, 常规)	小数位数可变; setPre(p)、round(p)、四则与比较。除法会保留设定精度, 适合 Machin / Chudnovsky 等模板。
02B	定点数 (FastFixedPoint, 极致优化)	FastFixedPoint<P> 将小数位数固定为编译期常量; 四则不做动态对齐与去零。解析时四舍五入, 乘除向 0 截断。
03A	浮点数 (Decimal, 常规)	setPre(p) 后运算自动控制精度; 额外提供 sqrt、pow、trunc、% 等接口。
03B	浮点数 (FastDecimal, 极致优化)	FastDecimal<P> 是 FastFixedPoint<P> 的别名; 保留高频四则、比较、pow 和 I/O。需要动态精度或 sqrt 时选 03A。
04A	2 至 62 进制转换 (常规)	自包含逐位短除, 代码短, 不依赖 BigInt; 复杂度为输入位数乘输出位数。
04B	2 至 62 进制转换 (极致优化)	依赖 01A/01B, 按不超过 $1e9$ 的块读入与除出; 适合超长字符串。

选择与粘贴顺序

需求	粘贴顺序
普通大整数、少量高精度运算	01A
大数乘法、阶乘、圆周率等卡常场景	01B
动态精度小数	01A 或 01B → 02A; 需要自动精度小数功能再加 03A
固定 P 位、频繁四则	01B → 02B; 若想使用 Decimal 命名再加 03B
普通 2..62 进制转换	04A
超长 2..62 进制转换	01A 或 01B → 04B

10A - 圆周率 (Machin) 需要 BigInt + FixedPoint, 通常选 01A + 02A; 高位数的 10B - 圆周率 (Chudnovsky) 建议选 01B + 02A。

约定

- BigInt 除法向 0 截断, 余数与被除数同号。
- 01A、01B 均提供 mul10 / div10, 小数模板借此进行十进制位移, 避免缓存全部 10^k 导致的二次内存。
- 02B / 03B 的 P 是结果保留的小数位数; 若连乘、连除误差敏感, 手动多开若干保护位, 再用 str(need) 输出。
- 04A / 04B 都使用 0-9A-Za-z 的数码顺序并支持负号; 它们是同一问题的普通/卡常环境分支, 只选一个。

01A - 整数 (BigInt, 常规) .cpp

```
1  /*
2  用途: 有符号大整数, 常规赛时版。
3  特点:  $10^9$  进制、 $O(n^2)$  乘法、规范化长除; 适合千至数千位且代码体积可控的场景。
```

```

4 替代：万位级乘法或更大输入使用 01B 的 FFT 版，二者均定义 BigInt，不能同时粘贴。
5 性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6 */
7
8 using u64 = uint64_t;
9 using i128 = __int128_t;
10
11 struct BigInt {
12
13     static constexpr int B = 1000000000u;
14     static constexpr int W = 9;
15
16     vector<int> d;
17     int s = 0;
18
19     BigInt() = default;
20     BigInt(int x) {
21         *this = x;
22     }
23     BigInt(const string& a) {
24         read(a);
25     }
26
27     static u64 mag(int x) {
28         return x < 0 ? u64(-(x + 1)) + 1 : u64(x);
29     }
30
31     BigInt& operator=(int x) {
32         d.clear();
33         if (!x) {
34             s = 0;
35             return *this;
36         }
37         s = x < 0 ? -1 : 1;
38         for (u64 y = mag(x); y; y /= B) d.push_back(y % B);
39         return *this;
40     }
41
42     BigInt& operator=(const string& a) {
43         read(a);
44         return *this;
45     }
46
47     bool ask0() const {
48         return !s;
49     }
50
51     void norm() {
52         while (!d.empty() && !d.back()) d.pop_back();
53         if (d.empty()) s = 0;
54     }
55
56     void read(const string& a) {
57         d.clear();
58         int n = a.size(), p = 0;
59         while (p < n && isspace(static_cast<unsigned char>(a[p]))) ++p;
60         s = 1;
61         if (p < n && (a[p] == '+' || a[p] == '-')) {
62             if (a[p] == '-') s = -1;
63             ++p;
64         }
65         while (p < n && a[p] == '0') ++p;
66         if (p == n) {
67             s = 0;
68             return;
69         }
70         for (int r = n; r > p; r -= W) {
71             int l = max<int>(p, r - W);
72             int x = 0;
73             for (int i = l; i < r; ++i) {
74                 assert('0' <= a[i] && a[i] <= '9');
75                 x = x * 10 + a[i] - '0';
76             }
77             d.push_back(x);
78         }
79         norm();
80     }
81
82     string str() const {
83         if (ask0()) return "0";
84         string a = s < 0 ? "-" : "";
85         a += std::to_string(d.back());

```

```

86     string b;
87     for (int i = d.size() - 1; i--;) {
88         b = to_string(d[i]);
89         a += string(W - b.size(), '0') + b;
90     }
91     return a;
92 }
93
94 static int cmpAbs(const BigInt& a, const BigInt& b) {
95     if (a.d.size() != b.d.size()) return a.d.size() < b.d.size() ? -1 : 1;
96     for (int i = a.d.size(); i--;)
97         if (a.d[i] != b.d[i]) return a.d[i] < b.d[i] ? -1 : 1;
98     return 0;
99 }
100
101 static int cmp(const BigInt& a, const BigInt& b) {
102     if (a.s != b.s) return a.s < b.s ? -1 : 1;
103     if (!a.s) return 0;
104     int x = cmpAbs(a, b);
105     return a.s > 0 ? x : -x;
106 }
107
108 static BigInt addAbs(const BigInt& a, const BigInt& b) {
109     BigInt c;
110     int n = max<int>(a.d.size(), b.d.size());
111     c.d.assign(n, 0);
112     u64 w = 0;
113     for (int i = 0; i < n; ++i) {
114         w += i < a.d.size() ? a.d[i] : 0;
115         w += i < b.d.size() ? b.d[i] : 0;
116         c.d[i] = w % B;
117         w /= B;
118     }
119     if (w) c.d.push_back(w);
120     c.s = 1;
121     return c;
122 }
123
124 static BigInt subAbs(const BigInt& a, const BigInt& b) {
125     BigInt c;
126     c.d.assign(a.d.size(), 0);
127     int w = 0;
128     for (int i = 0; i < a.d.size(); ++i) {
129         int x = a.d[i] - (i < b.d.size() ? b.d[i] : 0) - w;
130         if (x < 0) x += B, w = 1;
131         else w = 0;
132         c.d[i] = x;
133     }
134     c.s = 1;
135     c.norm();
136     return c;
137 }
138
139 friend BigInt operator+(const BigInt& a, const BigInt& b) {
140     if (!a.s) return b;
141     if (!b.s) return a;
142     if (a.s == b.s) {
143         BigInt c = addAbs(a, b);
144         c.s = a.s;
145         return c;
146     }
147     int x = cmpAbs(a, b);
148     if (!x) return BigInt(0);
149     BigInt c = x > 0 ? subAbs(a, b) : subAbs(b, a);
150     c.s = x > 0 ? a.s : b.s;
151     return c;
152 }
153
154 friend BigInt operator-(const BigInt& a, const BigInt& b) {
155     BigInt c = b;
156     c.s = -c.s;
157     return a + c;
158 }
159
160 static BigInt mul0(const BigInt& a, const BigInt& b) {
161     if (!a.s || !b.s) return BigInt(0);
162     BigInt c;
163     int n = a.d.size(), m = b.d.size();
164     c.d.assign(n + m, 0);
165     for (int i = 0; i < n; ++i) {
166         u64 w = 0;
167         for (int j = 0; j < m || w; ++j) {

```

```

168         u64 x = c.d[i + j] + w;
169         if (j < m) x += u64(a.d[i]) * b.d[j];
170         c.d[i + j] = x % B;
171         w = x / B;
172     }
173 }
174 c.s = a.s * b.s;
175 c.norm();
176 return c;
177 }
178
179 static BigInt mulInt(const BigInt& a, int b) {
180     if (!a.s || !b) return BigInt(0);
181     BigInt c;
182     u64 y = mag(b);
183     c.d.assign(a.d.size(), 0);
184     i128 w = 0;
185     for (int i = 0; i < a.d.size() || w; ++i) {
186         if (i == c.d.size()) c.d.push_back(0);
187         w += i < a.d.size() ? static_cast<i128>(a.d[i]) * y : 0;
188         c.d[i] = w % B;
189         w /= B;
190     }
191     c.s = a.s * (b < 0 ? -1 : 1);
192     c.norm();
193     return c;
194 }
195
196 friend BigInt operator*(const BigInt& a, const BigInt& b) {
197     return mul0(a, b);
198 }
199
200 static pair<BigInt, int> divmod(const BigInt& a, int b) {
201     if (!b) throw runtime_error("division by zero");
202     BigInt c;
203     c.d.assign(a.d.size(), 0);
204     u64 y = mag(b), w = 0;
205     for (int i = a.d.size(); i--;) {
206         i128 x = static_cast<i128>(w) * B + a.d[i];
207         c.d[i] = x / y;
208         w = static_cast<u64>(x % y);
209     }
210     c.s = a.s * (b < 0 ? -1 : 1);
211     c.norm();
212     int r = w;
213     if (a.s < 0) r = -r;
214     return {c, r};
215 }
216
217 static pair<BigInt, BigInt> divmod(const BigInt& a, const BigInt& b) {
218     if (!b.s) throw runtime_error("division by zero");
219     if (!a.s) return {BigInt(0), BigInt(0)};
220     BigInt x = a, y = b;
221     x.s = y.s = 1;
222     if (cmpAbs(x, y) < 0) return {BigInt(0), a};
223     if (y.d.size() == 1) {
224         auto [q, r] = divmod(a, b.s * y.d[0]);
225         return {q, BigInt(r)};
226     }
227     u64 f = u64(B) / (u64(y.d.back()) + 1);
228     x = mulInt(x, f);
229     y = mulInt(y, f);
230     BigInt q;
231     q.d.assign(x.d.size(), 0);
232     BigInt r;
233     for (int i = x.d.size(); i--;) {
234         if (r.d.empty()) r.d.push_back(x.d[i]);
235         else r.d.insert(r.d.begin(), x.d[i]);
236         r.s = 1;
237         r.norm();
238         u64 u = r.d.size() <= y.d.size() ? 0 : r.d[y.d.size()];
239         u64 v = r.d.size() < y.d.size() ? 0 : r.d[y.d.size() - 1];
240         u64 z = min<u64>(B - 1, (u64(B) * u + v) / y.d.back());
241         r -= mulInt(y, z);
242         while (r.s < 0) {
243             r += y;
244             --z;
245         }
246         q.d[i] = z;
247     }
248     q.s = a.s * b.s;
249     q.norm();

```

```

250     r = divmod(r, f).first;
251     r.s = a.s;
252     r.norm();
253     return {q, r};
254 }
255
256 friend BigInt operator/(const BigInt& a, const BigInt& b) {
257     return divmod(a, b).first;
258 }
259
260 friend BigInt operator%(const BigInt& a, const BigInt& b) {
261     return divmod(a, b).second;
262 }
263
264 static BigInt divBig(const BigInt& a, const BigInt& b) {
265     return divmod(a, b).first;
266 }
267
268 static BigInt mod(const BigInt& a, const BigInt& b) {
269     return divmod(a, b).second;
270 }
271
272 BigInt operator/(int x) const {
273     return divmod(*this, x).first;
274 }
275
276 BigInt operator%(int x) const {
277     return BigInt(divmod(*this, x).second);
278 }
279
280 static BigInt pow(BigInt a, u64 n) {
281     BigInt r(1);
282     while (n) {
283         if (n & 1) r *= a;
284         a *= a;
285         n >>= 1;
286     }
287     return r;
288 }
289
290 static BigInt gcd(BigInt a, BigInt b) {
291     if (a.s < 0) a.s = 1;
292     if (b.s < 0) b.s = 1;
293     while (b.s) {
294         BigInt r = a % b;
295         a = b;
296         b = r;
297     }
298     return a;
299 }
300
301 static BigInt fac(int n) {
302     BigInt r(1);
303     for (int i = 2; i <= n; ++i) r = mulInt(r, i);
304     return r;
305 }
306
307 static BigInt mul1(const BigInt& a, int x) {
308     return mulInt(a, x);
309 }
310
311 static BigInt mul10(BigInt a, int k) {
312     if (k < 0) throw runtime_error("negative decimal shift");
313     if (!a.s || !k) return a;
314     int q = k / W, r = k % W;
315     int p = 1;
316     for (int i = 0; i < r; ++i) p *= 10;
317     if (r) a = mulInt(a, p);
318     if (q) a.d.insert(a.d.begin(), q, 0);
319     return a;
320 }
321
322 static BigInt div10(BigInt a, int k) {
323     if (k < 0) throw runtime_error("negative decimal shift");
324     if (!a.s || !k) return a;
325     int q = k / W, r = k % W;
326     int p = 1;
327     for (int i = 0; i < r; ++i) p *= 10;
328     if (r) a = divmod(a, p).first;
329     if (q >= a.d.size()) return BigInt(0);
330     if (q) a.d.erase(a.d.begin(), a.d.begin() + q);
331     a.norm();

```

```

332     return a;
333 }
334
335 static void useFFT(bool) {}
336
337 BigInt& operator+=(const BigInt& a) {
338     return *this = *this + a;
339 }
340
341 BigInt& operator-=(const BigInt& a) {
342     return *this = *this - a;
343 }
344
345 BigInt& operator*=(const BigInt& a) {
346     return *this = *this * a;
347 }
348
349 BigInt& operator/=(const BigInt& a) {
350     return *this = *this / a;
351 }
352
353 friend bool operator==(const BigInt& a, const BigInt& b) {
354     return cmp(a, b) == 0;
355 }
356
357 friend bool operator!=(const BigInt& a, const BigInt& b) {
358     return cmp(a, b) != 0;
359 }
360
361 friend bool operator<(const BigInt& a, const BigInt& b) {
362     return cmp(a, b) < 0;
363 }
364
365 friend bool operator<=(const BigInt& a, const BigInt& b) {
366     return cmp(a, b) <= 0;
367 }
368
369 friend bool operator>(const BigInt& a, const BigInt& b) {
370     return cmp(a, b) > 0;
371 }
372
373 friend bool operator>=(const BigInt& a, const BigInt& b) {
374     return cmp(a, b) >= 0;
375 }
376
377 friend ostream& operator<<(ostream& os, const BigInt& a) {
378     return os << a.str();
379 }
380
381 friend istream& operator>>(istream& is, BigInt& a) {
382     string s;
383     is >> s;
384     a.read(s);
385     return is;
386 }
387 };

```

01B - 整数 (BigInt, 极致优化) .cpp

```

1  /*
2  用途: 有符号大整数, FFT 自动切换的极致优化版。
3  特点:  $10^9$  进制、长除、超大乘法自动使用 FFT; 与 01A 同名 BigInt, 二者不能同时粘贴。
4  性能: 十进制块和 FFT 拆分数组保留 i32/u32, 并把两组实系数打包进一次正变换;
5      改成宏展开后的 int 或拆成两次正变换都会显著增加内存与常数。
6  */
7
8  using i32 = int32_t;
9  using u32 = uint32_t;
10 using u64 = uint64_t;
11 using i128 = __int128_t;
12
13 struct BigInt {
14 public:
15     static constexpr u32 B = 1000000000u;
16     static constexpr i32 D = 9;
17
18 private:
19     static constexpr double PI = 3.141592653589793238462643383279502884;
20
21     template <typename T>

```



```

22 class Complex {
23     public:
24         T a, b;
25
26         constexpr Complex(T r = 0, T i = 0) : a(r), b(i) {}
27
28         constexpr Complex operator+(const Complex &rhs) const {
29             return {a + rhs.a, b + rhs.b};
30         }
31
32         constexpr Complex operator-(const Complex &rhs) const {
33             return {a - rhs.a, b - rhs.b};
34         }
35
36         constexpr Complex operator*(const Complex &rhs) const {
37             return {
38                 a * rhs.a - b * rhs.b,
39                 a * rhs.b + b * rhs.a};
40         }
41
42         Complex &operator/=(T v) {
43             a /= v;
44             b /= v;
45             return *this;
46         }
47 };
48 using cd = Complex<double>;
49
50 static void fft(vector<cd> &a, bool inv) {
51     i32 n = a.size();
52     static vector<i32> rev;
53     static vector<cd> rts{{0, 0}, {1, 0}};
54
55     i32 rn = rev.size();
56     if (rn != n) {
57         i32 k = __builtin_ctz(u32(n));
58         rev.assign(n, 0);
59         for (i32 i = 0; i < n; i++)
60             rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (k - 1));
61     }
62
63     i32 sn = rts.size();
64     if (sn < n) {
65         i32 k = __builtin_ctz(u32(sn));
66         rts.resize(n);
67         while ((i32(1) << k) < n) {
68             double da = 2 * PI / (i32(1) << (k + 1));
69             for (i32 i = i32(1) << (k - 1); i < (i32(1) << k); i++) {
70                 rts[i << 1] = rts[i];
71                 double ag = da * (2 * i + 1 - (i32(1) << k));
72                 rts[i << 1 | 1] = cd(cos(ag), sin(ag));
73             }
74             k++;
75         }
76     }
77
78     for (i32 i = 0; i < n; i++)
79         if (i < rev[i])
80             swap(a[i], a[rev[i]]);
81
82     for (i32 len = 1; len < n; len <= 1) {
83         for (i32 i = 0; i < n; i += len << 1) {
84             for (i32 j = 0; j < len; j++) {
85                 cd u = a[i + j];
86                 cd v = a[i + j + len] * rts[len + j];
87                 a[i + j] = u + v;
88                 a[i + j + len] = u - v;
89             }
90         }
91     }
92
93     if (inv) {
94         reverse(a.begin() + 1, a.end());
95         for (cd &x : a)
96             x /= n;
97     }
98 }
99
100 static BigInt mulFFT(const BigInt &a, const BigInt &b) {
101     if (a.ask0() || b.ask0()) return BigInt(0);
102     constexpr i32 FB = 1000;
103     constexpr i32 FD = 3;

```

```

104     vector<i32> fa, fb;
105     i32 na = a.d.size(), nb = b.d.size();
106     fa.reserve(na * 3);
107     fb.reserve(nb * 3);
108     for (i32 i = 0; i < na; ++i) {
109         u32 x = a.d[i];
110         for (i32 k = 0; k < D / FD; ++k) {
111             fa.push_back(x % FB);
112             x /= FB;
113         }
114     }
115     for (i32 i = 0; i < nb; ++i) {
116         u32 x = b.d[i];
117         for (i32 k = 0; k < D / FD; ++k) {
118             fb.push_back(x % FB);
119             x /= FB;
120         }
121     }
122     while (!fa.empty() && !fa.back()) fa.pop_back();
123     while (!fb.empty() && !fb.back()) fb.pop_back();
124
125     i32 n = 1;
126     i32 fn = fa.size(), gn = fb.size();
127     while (n < fn + gn) n <= 1;
128     vector<cd> x(n), y(n);
129     for (i32 i = 0; i < fn; ++i) x[i].a = fa[i];
130     for (i32 i = 0; i < gn; ++i) x[i].b = fb[i];
131     fft(x, false);
132     for (i32 i = 0; i < n; ++i) {
133         i32 j = (n - i) & (n - 1);
134         cd p{(x[i].a + x[j].a) * 0.5, (x[i].b - x[j].b) * 0.5};
135         cd q{(x[i].b + x[j].b) * 0.5, (x[j].a - x[i].a) * 0.5};
136         y[i] = p * q;
137     }
138     fft(y, true);
139
140     vector<i32> c;
141     c.reserve(n + 4);
142     u64 cy = 0;
143     for (i32 i = 0; i < n; ++i) {
144         int v = y[i].a + 0.5;
145         if (v < 0) v = 0;
146         u64 w = cy + static_cast<u64>(v);
147         c.push_back(static_cast<i32>(w % FB));
148         cy = w / FB;
149     }
150     while (cy) {
151         c.push_back(static_cast<i32>(cy % FB));
152         cy /= FB;
153     }
154     while (!c.empty() && !c.back()) c.pop_back();
155
156     BigInt res;
157     i32 cn = c.size();
158     res.d.reserve((cn + 2) / 3);
159     for (i32 i = 0; i < cn; i += 3) {
160         u64 w = 0, p = 1;
161         for (i32 j = 0; j < 3 && i + j < cn; ++j) {
162             w += static_cast<u64>(c[i + j]) * p;
163             p *= FB;
164         }
165         res.d.push_back(static_cast<u32>(w));
166     }
167     res.sgn = a.sgn * b.sgn;
168     res.norm();
169     return res;
170 }
171
172 vector<u32> d;
173 short sgn;
174
175 public:
176     BigInt() : sgn(0) {}
177     BigInt(int v) { *this = v; }
178     BigInt(const string &s) { read(s); }
179     static u64 mag(int v) {
180         return v < 0 ? u64(-(v + 1)) + 1 : u64(v);
181     }
182     BigInt &operator=(int v) {
183         d.clear();
184         if (v == 0) {
185             sgn = 0;

```

```

186         return *this;
187     }
188     sgn = v < 0 ? -1 : 1;
189     u64 av = mag(v);
190     while (av) {
191         d.push_back(u32(av % B));
192         av /= B;
193     }
194     return *this;
195 }
196
197 BigInt &operator=(const string &s) {
198     read(s);
199     return *this;
200 }
201
202 bool ask0() const { return sgn == 0; }
203
204 void norm() {
205     while (!d.empty() && d.back() == 0)
206         d.pop_back();
207     if (d.empty())
208         sgn = 0;
209 }
210
211 string str() const {
212     if (ask0()) return "0";
213     string s = sgn < 0 ? "-" : "";
214     s += std::to_string(d.back());
215     char buf[32];
216     for (i32 i = d.size() - 1; i--;) {
217         snprintf(buf, sizeof(buf), "%09u", d[i]);
218         s += buf;
219     }
220     return s;
221 }
222
223 void read(const string &s) {
224     d.clear();
225     i32 n = s.size(), p = 0, e = n;
226     while (p < n && isspace(static_cast<unsigned char>(s[p]))) ++p;
227     sgn = 1;
228     if (p < n && (s[p] == '+' || s[p] == '-')) {
229         if (s[p] == '-') sgn = -1;
230         ++p;
231     }
232     while (e > p && isspace(static_cast<unsigned char>(s[e - 1]))) --e;
233     while (p < e && s[p] == '0') ++p;
234     if (p == e) {
235         sgn = 0;
236         return;
237     }
238     for (i32 i = e; i > p; i -= D) {
239         u32 x = 0;
240         i32 l = max<i32>(p, i - D);
241         for (i32 j = l; j < i; ++j) {
242             assert('0' <= s[j] && s[j] <= '9');
243             x = x * 10 + (s[j] - '0');
244         }
245         d.push_back(x);
246     }
247
248     norm();
249 }
250
251 public:
252 static i32 cmpAbs(const BigInt &a, const BigInt &b) {
253     if (a.d.size() != b.d.size())
254         return a.d.size() < b.d.size() ? -1 : 1;
255     for (i32 i = a.d.size(); i--;)
256         if (a.d[i] != b.d[i])
257             return a.d[i] < b.d[i] ? -1 : 1;
258     return 0;
259 }
260 static i32 cmp(const BigInt &a, const BigInt &b) {
261     if (a.sgn != b.sgn)
262         return a.sgn < b.sgn ? -1 : 1;
263     if (a.sgn == 0)
264         return 0;
265     i32 c = cmpAbs(a, b);
266     return a.sgn > 0 ? c : -c;
267 }

```

```

268
269 private:
270     static BigInt addAbs(const BigInt &a, const BigInt &b) {
271         BigInt res;
272         u64 cy = 0;
273         i32 na = a.d.size(), nb = b.d.size();
274         i32 n = max(na, nb);
275         res.d.assign(n, 0);
276         for (i32 i = 0; i < n; ++i) {
277             u64 av = i < na ? a.d[i] : 0;
278             u64 bv = i < nb ? b.d[i] : 0;
279             u64 sum = av + bv + cy;
280             res.d[i] = u32(sum % B);
281             cy = sum / B;
282         }
283         if (cy)
284             res.d.push_back(u32(cy));
285         res.sgn = res.d.empty() ? 0 : 1;
286         return res;
287     }
288
289     static BigInt subAbs(const BigInt &a, const BigInt &b) {
290         BigInt res;
291         res.d.assign(a.d.size(), 0);
292         u64 cy = 0;
293         i32 na = a.d.size(), nb = b.d.size();
294         for (i32 i = 0; i < na; ++i) {
295             u64 av = a.d[i];
296             u64 bv = i < nb ? b.d[i] : 0;
297             int cur = static_cast<int>(av) - static_cast<int>(bv) - static_cast<int>(cy);
298             if (cur < 0) {
299                 cur += B;
300                 cy = 1;
301             } else
302                 cy = 0;
303             res.d[i] = u32(cur);
304         }
305         res.norm();
306         res.sgn = res.d.empty() ? 0 : 1;
307         return res;
308     }
309
310 public:
311     BigInt operator+(const BigInt &v) const {
312         if (sgn == 0)
313             return v;
314         if (v.sgn == 0)
315             return *this;
316         BigInt res;
317         if (sgn == v.sgn) {
318             res = addAbs(*this, v);
319             res.sgn = sgn;
320         } else {
321             i32 cmp = cmpAbs(*this, v);
322             if (cmp >= 0) {
323                 res = subAbs(*this, v);
324                 res.sgn = sgn;
325             } else {
326                 res = subAbs(v, *this);
327                 res.sgn = v.sgn;
328             }
329         }
330         if (res.ask0())
331             res.sgn = 0;
332         return res;
333     }
334
335     BigInt operator-(const BigInt &v) const {
336         if (v.ask0())
337             return *this;
338         BigInt tmp = v;
339         tmp.sgn = -tmp.sgn;
340         return *this + tmp;
341     }
342
343     static BigInt mul0(const BigInt &a, const BigInt &b) {
344         if (a.ask0() || b.ask0())
345             return BigInt(0);
346         BigInt res;
347         i32 n = a.d.size(), m = b.d.size();
348         res.d.assign(n + m + 1, 0);
349         for (i32 i = 0; i < n; ++i) {

```

```

350         u64 cy = 0;
351         for (i32 j = 0; j < m || cy; ++j) {
352             u64 cur = res.d[i + j] + cy + u64(a.d[i]) * (j < m ? b.d[j] : 0);
353             res.d[i + j] = u32(cur % B);
354             cy = cur / B;
355         }
356     }
357     res.sgn = a.sgn * b.sgn;
358     res.norm();
359     return res;
360 }
361
362 static BigInt mulInt(const BigInt &a, int m) {
363     if (a.ask0() || m == 0)
364         return BigInt(0);
365     BigInt res;
366     short sg2 = m < 0 ? -1 : 1;
367     u64 mm = mag(m);
368     res.d.assign(a.d.size(), 0);
369     u64 cy = 0;
370     i32 n = a.d.size();
371     for (i32 i = 0; i < n || cy; ++i) {
372         i128 cur = cy + static_cast<i128>(i < n ? a.d[i] : 0) * mm;
373         if (i == n)
374             res.d.push_back(0);
375         res.d[i] = static_cast<u32>(cur % B);
376         cy = static_cast<u64>(cur / B);
377     }
378     res.sgn = a.sgn * sg2;
379     res.norm();
380     return res;
381 }
382
383 public:
384     static constexpr i32 lim = 1500;
385     BigInt operator*(const BigInt &v) const {
386         if (ask0() || v.ask0())
387             return BigInt(0);
388         i32 n = d.size(), m = v.d.size();
389         if (min(n, m) == 0)
390             return BigInt(0);
391         if (n + m < lim)
392             return mul0(*this, v);
393         else
394             return mulFFT(*this, v);
395     }
396
397     static pair<BigInt, int> divmod(const BigInt &a, int b) {
398         if (b == 0)
399             throw runtime_error("div by zero");
400         BigInt q;
401         q.d.assign(a.d.size(), 0);
402         u64 cy = 0, ub = mag(b);
403         for (i32 i = a.d.size(); i--;) {
404             i128 cur = static_cast<i128>(cy) * B + a.d[i];
405             q.d[i] = static_cast<u32>(cur / ub);
406             cy = static_cast<u64>(cur % ub);
407         }
408         q.sgn = (q.d.empty() ? 0 : (a.sgn * (b < 0 ? -1 : 1)));
409         q.norm();
410         int rem = cy;
411         if (a.sgn < 0)
412             rem = -rem;
413         return {q, rem};
414     }
415
416     static pair<BigInt, BigInt> divmod(const BigInt &a, const BigInt &b) {
417         if (b.ask0())
418             throw runtime_error("div by zero");
419         if (a.ask0())
420             return {BigInt(0), BigInt(0)};
421         i32 sgn = a.sgn * b.sgn;
422         BigInt aa = a;
423         aa.sgn = 1;
424         BigInt bb = b;
425         bb.sgn = 1;
426         if (cmpAbs(aa, bb) < 0)
427             return {BigInt(0), a};
428
429         if (bb.d.size() == 1) {
430             auto [q, r] = divmod(a, static_cast<int>(b.sgn) * bb.d[0]);
431             return {q, BigInt(r)};

```

```

432     }
433     u64 f = u64(B) / (u64(bb.d.back()) + 1);
434     aa = mulInt(aa, f);
435     bb = mulInt(bb, f);
436     BigInt q;
437     q.d.assign(aa.d.size(), 0);
438     BigInt r;
439     for (i32 i = aa.d.size(); i--;) {
440         if (r.d.empty()) r.d.push_back(aa.d[i]);
441         else r.d.insert(r.d.begin(), aa.d[i]);
442         r.sgn = 1;
443         r.norm();
444         u64 x = r.d.size() <= bb.d.size() ? 0 : r.d[bb.d.size()];
445         u64 y = r.d.size() < bb.d.size() ? 0 : r.d[bb.d.size() - 1];
446         u64 z = min<u64>(B - 1, (B * x + y) / bb.d.back());
447         r -= mulInt(bb, z);
448         while (r.sgn < 0) {
449             r += bb;
450             --z;
451         }
452         q.d[i] = static_cast<u32>(z);
453     }
454     q.sgn = sgn;
455     q.norm();
456     r = divmod(r, f).first;
457     r.sgn = a.sgn;
458     r.norm();
459     return {q, r};
460 }
461
462 static BigInt mod(const BigInt &a, const BigInt &b) {
463     return divmod(a, b).second;
464 }
465
466 static BigInt divBig(const BigInt &a, const BigInt &b) {
467     return divmod(a, b).first;
468 }
469
470 BigInt operator/(const BigInt &v) const { return divBig(*this, v); }
471 BigInt operator%(const BigInt &v) const { return mod(*this, v); }
472 BigInt operator/(int v) const { return divmod(*this, v).first; }
473 BigInt operator%(int v) const { return BigInt(divmod(*this, v).second); }
474
475 static BigInt pow(BigInt a, u64 e) {
476     BigInt res(1);
477     while (e) {
478         if (e & 1)
479             res = res * a;
480         a = a * a;
481         e >>= 1;
482     }
483     return res;
484 }
485
486 static void useFFT(bool) {}
487
488 static BigInt gcd(BigInt a, BigInt b) {
489     if (a.sgn < 0) a.sgn = 1;
490     if (b.sgn < 0) b.sgn = 1;
491     while (!b.ask0()) {
492         BigInt r = a % b;
493         a = b;
494         b = r;
495     }
496     return a;
497 }
498
499 static BigInt fac(int n) {
500     BigInt res(1);
501     for (int i = 2; i <= n; i++) {
502         res = mulInt(res, i);
503     }
504     return res;
505 }
506
507 static BigInt mul1(const BigInt &a, u32 sm) { return mulInt(a, sm); }
508
509 static BigInt mul10(BigInt a, i32 k) {
510     if (k < 0) throw runtime_error("negative decimal shift");
511     if (a.ask0() || !k) return a;
512     i32 q = k / D, r = k % D;
513     u32 p = 1;

```

```

514     for (i32 i = 0; i < r; ++i) p *= 10;
515     if (r) a = mulInt(a, p);
516     if (q) a.d.insert(a.d.begin(), q, 0);
517     return a;
518 }
519
520 static BigInt div10(BigInt a, i32 k) {
521     if (k < 0) throw runtime_error("negative decimal shift");
522     if (a.ask0() || !k) return a;
523     i32 q = k / D, r = k % D;
524     u32 p = 1;
525     for (i32 i = 0; i < r; ++i) p *= 10;
526     if (r) a = divmod(a, p).first;
527     i32 n = a.d.size();
528     if (q >= n) return BigInt(0);
529     if (q) a.d.erase(a.d.begin(), a.d.begin() + q);
530     a.norm();
531     return a;
532 }
533
534 friend ostream &operator<<(ostream &os, const BigInt &x) {
535     os << x.str();
536     return os;
537 }
538 friend istream &operator>>(istream &is, BigInt &x) {
539     string s;
540     is >> s;
541     x.read(s);
542     return is;
543 }
544
545 BigInt &operator+=(const BigInt &v) {
546     *this = *this + v;
547     return *this;
548 }
549 BigInt &operator-=(const BigInt &v) {
550     *this = *this - v;
551     return *this;
552 }
553 BigInt &operator*=(const BigInt &v) {
554     *this = *this * v;
555     return *this;
556 }
557 BigInt &operator/=(const BigInt &v) {
558     *this = *this / v;
559     return *this;
560 }
561
562 friend bool operator==(const BigInt &a, const BigInt &b) { return cmp(a, b) == 0; }
563 friend bool operator!=(const BigInt &a, const BigInt &b) { return cmp(a, b) != 0; }
564 friend bool operator<(const BigInt &a, const BigInt &b) { return cmp(a, b) < 0; }
565 friend bool operator<=(const BigInt &a, const BigInt &b) { return cmp(a, b) <= 0; }
566 friend bool operator>(const BigInt &a, const BigInt &b) { return cmp(a, b) > 0; }
567 friend bool operator>=(const BigInt &a, const BigInt &b) { return cmp(a, b) >= 0; }
568 };

```

02A - 定点数 (FixedPoint, 常规) .cpp

```

1  /*
2  用途: 动态小数位数的高精度定点数; 适合精度会变化、需要 round 的常规场景。
3  依赖: 先粘贴 01A 或 01B 中恰好一个 BigInt。01A 足够常用, 01B 适合超大乘法。
4  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
5  */
6
7  struct FixedPoint {
8      BigInt man;
9      int sgn;
10     int scl;
11     static int pre;
12     static constexpr int LIM = (1LL << 31) - 1;
13
14     public:
15     FixedPoint() : man(0), sgn(0), scl(0) {}
16     FixedPoint(int v) { read(std::to_string(v)); }
17     FixedPoint(const string &s) { read(s); }
18     FixedPoint(const BigInt &m, int sc, int sgn = 1) : man(m), sgn(m.ask0() ? 0 : sgn), scl(sc) {}
19
20     static void setPre(int d) { pre = d < 0 ? 0 : d; }
21
22     static BigInt pow10(int k) {

```

```

23     if (k < 0)
24         throw runtime_error("negative pow10");
25     if (k > LIM)
26         throw overflow_error("decimal scale too large");
27     return BigInt::mul10(BigInt(1), k);
28 }
29
30 static BigInt shift(BigInt a, int k) {
31     if (k < 0 || k > LIM)
32         throw overflow_error("decimal scale out of range");
33     return BigInt::mul10(std::move(a), k);
34 }
35
36 void norm() {
37     if (man.ask0()) {
38         sgn = 0;
39         scl = 0;
40         return;
41     }
42     while (scl > 0) {
43         auto qr = BigInt::divmod(man, 10);
44         if (qr.second != 0)
45             break;
46         man = qr.first;
47         --scl;
48     }
49     if (man.ask0())
50         sgn = 0;
51 }
52
53 void round(int kp) {
54     if (kp < 0)
55         throw runtime_error("negative round");
56     if (man.ask0()) {
57         sgn = 0;
58         scl = 0;
59         return;
60     }
61     if (scl <= kp) {
62         norm();
63         return;
64     }
65
66     int drp = scl - kp;
67     BigInt p10 = pow10(drp);
68     auto qr = BigInt::divmod(man, p10);
69     BigInt q = qr.first;
70     BigInt r = qr.second;
71     BigInt r2 = BigInt::mulInt(r, 2);
72     if (BigInt::cmp(r2, p10) >= 0)
73         q = q + BigInt(1);
74     man = q;
75     scl = kp;
76     norm();
77 }
78
79 FixedPoint abs() const {
80     if (sgn == 0)
81         return FixedPoint();
82     FixedPoint r = *this;
83     r.sgn = 1;
84     return r;
85 }
86
87 void read(const string &s) {
88     int n = s.size();
89     int i = 0;
90     while (i < n && isspace((unsigned char)s[i]))
91         ++i;
92     int sg = 1;
93     if (i < n && (s[i] == '+' || s[i] == '-')) {
94         if (s[i] == '-')
95             sg = -1;
96         ++i;
97     }
98     string d;
99     int fd = 0;
100     bool dot = false;
101     for (; i < n; ++i) {
102         char c = s[i];
103         if (c == '.') {
104             if (dot)

```



```

105         break;
106         dot = true;
107         continue;
108     }
109     if (c >= '0' && c <= '9') {
110         d.push_back(c);
111         if (dot)
112             ++fd;
113     } else
114         break;
115 }
116 int p = 0;
117 while (p < d.size() && d[p] == '0')
118     ++p;
119 if (p == d.size()) {
120     sgn = 0;
121     scl = 0;
122     man = BigInt(0);
123     return;
124 }
125 string cor = d.substr(p);
126 man = BigInt(cor);
127 sgn = sg;
128 scl = fd;
129 norm();
130 }
131
132 string str(int p = -1) const {
133     if (sgn == 0) {
134         if (p <= 0)
135             return "0";
136         string s = "0.";
137         s += string(p, '0');
138         return s;
139     }
140     string ms = man.str();
141     if (scl == 0) {
142         string out = (sgn < 0 ? "-" : "") + ms;
143         if (p > 0)
144             out += "." + string(p, '0');
145         return out;
146     }
147     if (ms.size() <= scl)
148         ms = string(scl + 1 - ms.size(), '0') + ms;
149     string ip = ms.substr(0, ms.size() - scl);
150     string fra = ms.substr(ms.size() - scl);
151     if (p >= 0) {
152         if (fra.size() < p)
153             fra += string(p - fra.size(), '0');
154         else if (fra.size() > p)
155             fra = fra.substr(0, p);
156     } else {
157         int t = fra.size();
158         while (t > 0 && fra[t - 1] == '0')
159             --t;
160         fra.resize(t);
161     }
162     string out = (sgn < 0 ? "-" : "") + (ip.empty() ? string("0") : ip);
163     if (!fra.empty())
164         out += "." + fra;
165     else if (p > 0)
166         out += "." + string(p, '0');
167     return out;
168 }
169
170 friend FixedPoint operator+(const FixedPoint &a, const FixedPoint &b) {
171     if (a.sgn == 0)
172         return b;
173     if (b.sgn == 0)
174         return a;
175     FixedPoint _a = a, _b = b;
176     if (_a.scl < _b.scl) {
177         int dif = _b.scl - _a.scl;
178         _a.man = shift(std::move(_a.man), dif);
179         _a.scl = _b.scl;
180     } else if (_b.scl < _a.scl) {
181         int dif = _a.scl - _b.scl;
182         _b.man = shift(std::move(_b.man), dif);
183         _b.scl = _a.scl;
184     }
185     FixedPoint res;
186     if (_a.sgn == _b.sgn) {

```

```

187         res.man = _a.man + _b.man;
188         res.sgn = _a.sgn;
189         res.scl = _a.scl;
190     } else {
191         int c = BigInt::cmp(_a.man, _b.man);
192         if (c == 0)
193             return FixedPoint();
194         else if (c > 0) {
195             res.man = _a.man - _b.man;
196             res.sgn = _a.sgn;
197             res.scl = _a.scl;
198         } else {
199             res.man = _b.man - _a.man;
200             res.sgn = _b.sgn;
201             res.scl = _a.scl;
202         }
203     }
204     res.round(max<int>(0, pre + 5));
205     res.norm();
206     return res;
207 }
208
209 friend FixedPoint operator-(const FixedPoint &a, const FixedPoint &b) {
210     FixedPoint _b = b;
211     _b.sgn = -_b.sgn;
212     return a + _b;
213 }
214
215 friend FixedPoint operator*(const FixedPoint &a, const FixedPoint &b) {
216     if (a.sgn == 0 || b.sgn == 0)
217         return FixedPoint();
218     FixedPoint res;
219     res.man = a.man * b.man;
220     res.sgn = a.sgn * b.sgn;
221     if (a.scl > numeric_limits<int>::max() - b.scl)
222         throw overflow_error("scale overflow");
223     res.scl = a.scl + b.scl;
224     res.round(max<int>(0, pre + 5));
225     res.norm();
226     return res;
227 }
228
229 friend FixedPoint operator/(const FixedPoint &a, const FixedPoint &b) {
230     if (b.sgn == 0)
231         throw runtime_error("division by zero");
232     if (a.sgn == 0)
233         return FixedPoint();
234     int p = pre, wk = p + 15;
235     int e = wk + b.scl - a.scl;
236     BigInt num = a.man, den = b.man;
237     if (e >= 0) num = shift(std::move(num), e);
238     else den = shift(std::move(den), -e);
239     auto qr = BigInt::divmod(num, den);
240     BigInt q = qr.first;
241     FixedPoint res(q, wk, a.sgn * b.sgn);
242     res.round(p);
243     res.norm();
244     return res;
245 }
246
247 FixedPoint &operator+=(const FixedPoint &o) {
248     *this = *this + o;
249     return *this;
250 }
251 FixedPoint &operator-=(const FixedPoint &o) {
252     *this = *this - o;
253     return *this;
254 }
255 FixedPoint &operator*=(const FixedPoint &o) {
256     *this = *this * o;
257     return *this;
258 }
259 FixedPoint &operator/=(const FixedPoint &o) {
260     *this = *this / o;
261     return *this;
262 }
263
264 friend bool operator==(const FixedPoint &a, const FixedPoint &b) {
265     if (a.sgn != b.sgn)
266         return false;
267     if (a.sgn == 0)
268         return true;

```

```

269     FixedPoint _a = a, _b = b;
270     if (_a.scl < _b.scl) {
271         _a.man = shift(std::move(_a.man), _b.scl - _a.scl);
272         _a.scl = _b.scl;
273     } else if (_b.scl < _a.scl) {
274         _b.man = shift(std::move(_b.man), _a.scl - _b.scl);
275         _b.scl = _a.scl;
276     }
277     return _a.man == _b.man;
278 }
279
280 friend bool operator!=(const FixedPoint &a, const FixedPoint &b) { return !(a == b); }
281
282 friend bool operator<(const FixedPoint &a, const FixedPoint &b) {
283     if (a.sgn != b.sgn)
284         return a.sgn < b.sgn;
285     if (a.sgn == 0)
286         return b.sgn > 0;
287     FixedPoint _a = a, _b = b;
288     if (_a.scl < _b.scl) {
289         _a.man = shift(std::move(_a.man), _b.scl - _a.scl);
290         _a.scl = _b.scl;
291     } else if (_b.scl < _a.scl) {
292         _b.man = shift(std::move(_b.man), _a.scl - _b.scl);
293         _b.scl = _a.scl;
294     }
295     if (_a.sgn > 0)
296         return _a.man < _b.man;
297     else
298         return _a.man > _b.man;
299 }
300
301 friend bool operator<=(const FixedPoint &a, const FixedPoint &b) { return !(b < a); }
302 friend bool operator>(const FixedPoint &a, const FixedPoint &b) { return b < a; }
303 friend bool operator>=(const FixedPoint &a, const FixedPoint &b) { return !(a < b); }
304
305 friend ostream &operator<<(ostream &os, const FixedPoint &x) {
306     os << x.str();
307     return os;
308 }
309 };
310
311 int FixedPoint::pre = 50;

```

02B - 定点数 (FastFixedPoint, 极致优化) .cpp

```

1  /*
2  用途: 编译期固定 P 位小数的高精度定点数; 频繁四则运算时避免动态对齐、去零和幂缓存。
3  依赖: 先粘贴 01B (推荐) 或 01A 中恰好一个 BigInt。P 是小数位, 构造和乘除均向 0 截断。
4  示例: using F = FastFixedPoint<1000>; F a("1.25"), b("3");
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7
8  template <int P>
9  struct FixedPoint {
10     static_assert(P >= 0);
11
12     BigInt x;
13
14     FixedPoint() : x(0) {}
15     FixedPoint(int a) : x(BigInt::mul10(BigInt(a), P)) {}
16     explicit FixedPoint(const string& s) { read(s); }
17
18     static FixedPoint raw(BigInt x) {
19         FixedPoint a;
20         a.x = std::move(x);
21         return a;
22     }
23
24     bool ask0() const { return x.ask0(); }
25
26     FixedPoint abs() const {
27         return x < BigInt(0) ? raw(BigInt(0) - x) : *this;
28     }
29
30     void read(const string& s) {
31         int n = s.size(), i = 0, f = 0;
32         while (i < n && isspace(static_cast<unsigned char>(s[i]))) ++i;
33         int sg = 1;
34         if (i < n && (s[i] == '+' || s[i] == '-')) {

```

```

35         if (s[i] == '-') sg = -1;
36         ++i;
37     }
38     string a;
39     bool dot = false;
40     for (; i < n; ++i) {
41         char c = s[i];
42         if (c == '.') {
43             if (dot) break;
44             dot = true;
45         } else if ('0' <= c && c <= '9') {
46             a += c;
47             if (dot) ++f;
48         } else {
49             break;
50         }
51     }
52     int p = 0;
53     while (p < a.size() && a[p] == '0') ++p;
54     if (p == a.size()) {
55         x = BigInt(0);
56         return;
57     }
58     BigInt z(a.substr(p));
59     if (f < P) z = BigInt::mul10(std::move(z), P - f);
60     if (f > P) {
61         BigInt q = BigInt::mul10(BigInt(1), f - P);
62         auto [u, v] = BigInt::divmod(z, q);
63         if (BigInt::cmp(BigInt::mulInt(v, 2), q) >= 0) u += BigInt(1);
64         z = std::move(u);
65     }
66     x = sg < 0 ? BigInt(0) - z : z;
67 }
68
69 string str(int o = -1) const {
70     if (x.ask0()) {
71         if (o <= 0) return "0";
72         return "0." + string(o, '0');
73     }
74     string a = x.str();
75     bool neg = a[0] == '-';
76     if (neg) a.erase(a.begin());
77     if constexpr (P == 0) {
78         if (o > 0) a += "." + string(o, '0');
79         return (neg ? "-" : "") + a;
80     }
81     if (a.size() <= P)
82         a = string(P + 1 - a.size(), '0') + a;
83     string ip = a.substr(0, a.size() - P);
84     string fp = a.substr(a.size() - P);
85     if (o >= 0) {
86         if (o < P) fp.resize(o);
87         else fp += string(o - P, '0');
88     } else {
89         while (!fp.empty() && fp.back() == '0') fp.pop_back();
90     }
91     string r = (neg ? "-" : "") + ip;
92     if (!fp.empty()) r += "." + fp;
93     else if (o > 0) r += "." + string(o, '0');
94     return r;
95 }
96
97 friend FixedPoint operator+(const FixedPoint& a, const FixedPoint& b) {
98     return raw(a.x + b.x);
99 }
100
101 friend FixedPoint operator-(const FixedPoint& a, const FixedPoint& b) {
102     return raw(a.x - b.x);
103 }
104
105 friend FixedPoint operator*(const FixedPoint& a, const FixedPoint& b) {
106     return raw(BigInt::div10(a.x * b.x, P));
107 }
108
109 friend FixedPoint operator/(const FixedPoint& a, const FixedPoint& b) {
110     if (b.x.ask0()) throw runtime_error("division by zero");
111     return raw(BigInt::mul10(a.x, P) / b.x);
112 }
113
114 friend FixedPoint operator%(const FixedPoint& a, const FixedPoint& b) {
115     if (b.x.ask0()) throw runtime_error("division by zero");
116     return raw(a.x % b.x);

```

```

117     }
118
119     FixedPoint& operator+=(const FixedPoint& a) { return *this = *this + a; }
120     FixedPoint& operator-=(const FixedPoint& a) { return *this = *this - a; }
121     FixedPoint& operator*=(const FixedPoint& a) { return *this = *this * a; }
122     FixedPoint& operator/=(const FixedPoint& a) { return *this = *this / a; }
123     FixedPoint& operator%=(const FixedPoint& a) { return *this = *this % a; }
124
125     static FixedPoint pow(FixedPoint a, int n) {
126         FixedPoint r(1);
127         while (n) {
128             if (n & 1) r *= a;
129             a *= a;
130             n >>= 1;
131         }
132         return r;
133     }
134
135     friend bool operator==(const FixedPoint& a, const FixedPoint& b) { return a.x == b.x; }
136     friend bool operator!=(const FixedPoint& a, const FixedPoint& b) { return a.x != b.x; }
137     friend bool operator<(const FixedPoint& a, const FixedPoint& b) { return a.x < b.x; }
138     friend bool operator<=(const FixedPoint& a, const FixedPoint& b) { return a.x <= b.x; }
139     friend bool operator>(const FixedPoint& a, const FixedPoint& b) { return a.x > b.x; }
140     friend bool operator>=(const FixedPoint& a, const FixedPoint& b) { return a.x >= b.x; }
141
142     friend ostream& operator<<(ostream& os, const FixedPoint& a) {
143         return os << a.str();
144     }
145
146     friend istream& operator>>(istream& is, FixedPoint& a) {
147         string s;
148         is >> s;
149         a.read(s);
150         return is;
151     }
152 };

```

03A - 浮点数 (Decimal, 常规) .cpp

```

1  /*
2  用途: 自动按 setPre 截断的高精度十进制数, 支持 sqrt、pow、trunc 等常规接口。
3  依赖: 先粘贴 01A 或 01B 中恰好一个 BigInt: 高精度乘法建议选择 01B。
4  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
5  */
6
7  class Decimal {
8  public:
9      static void setPre(int d) {
10         if (d < 0)
11             d = 0;
12         pre = d;
13     }
14     static void useFFT(bool on) { BigInt::useFFT(on); }
15
16     Decimal() : sgn(0), scl(0) {}
17     Decimal(int v) { read(std::to_string(v)); }
18     explicit Decimal(const string &s) { read(s); }
19
20     friend std::ostream &operator<<(std::ostream &os, const Decimal &x) {
21         if (x.sgn == 0) {
22             os << '0';
23             return os;
24         }
25         string m = x.man.str();
26         if (x.scl == 0) {
27             if (x.sgn < 0)
28                 os << '-';
29             os << m;
30             return os;
31         }
32         if (m.size() <= x.scl) {
33             string pad(x.scl + 1 - m.size(), '0');
34             m = pad + m;
35         }
36         string ip = m.substr(0, m.size() - x.scl);
37         string fp = m.substr(m.size() - x.scl);
38         int trm = fp.size();
39         while (trm > 0 && fp[trm - 1] == '0')
40             --trm;
41         fp.resize(trm);

```

```

42     if (x.sgn < 0)
43         os << '-';
44     if (ip.empty())
45         os << '0';
46     else
47         os << ip;
48     if (!fp.empty()) {
49         os << '.';
50         os << fp;
51     }
52     return os;
53 }
54
55 friend std::istream &operator>>(std::istream &is, Decimal &x) {
56     string s;
57     is >> s;
58     x.read(s);
59     return is;
60 }
61
62 string str() const { return strD(); }
63
64 friend int cmp(const Decimal &a, const Decimal &b) {
65     if (a.sgn != b.sgn)
66         return a.sgn < b.sgn ? -1 : 1;
67     if (a.sgn == 0)
68         return 0;
69     if (a.scl == b.scl) {
70         int c = BigInt::cmp(a.man, b.man);
71         return a.sgn > 0 ? c : -c;
72     }
73     Decimal _a = a, _b = b;
74     if (a.scl < b.scl) {
75         int dif = b.scl - a.scl;
76         _a.man = shift(std::move(_a.man), dif);
77         _a.scl = b.scl;
78     } else {
79         int dif = a.scl - b.scl;
80         _b.man = shift(std::move(_b.man), dif);
81         _b.scl = a.scl;
82     }
83     int c = BigInt::cmp(_a.man, _b.man);
84     return a.sgn > 0 ? c : -c;
85 }
86 friend bool operator==(const Decimal &a, const Decimal &b) { return cmp(a, b) == 0; }
87 friend bool operator!=(const Decimal &a, const Decimal &b) { return cmp(a, b) != 0; }
88 friend bool operator<(const Decimal &a, const Decimal &b) { return cmp(a, b) < 0; }
89 friend bool operator<=(const Decimal &a, const Decimal &b) { return cmp(a, b) <= 0; }
90 friend bool operator>(const Decimal &a, const Decimal &b) { return cmp(a, b) > 0; }
91 friend bool operator>=(const Decimal &a, const Decimal &b) { return cmp(a, b) >= 0; }
92
93 friend Decimal operator+(const Decimal &a, const Decimal &b) {
94     if (a.sgn == 0)
95         return b;
96     if (b.sgn == 0)
97         return a;
98     Decimal _a = a, _b = b;
99     if (_a.scl < _b.scl) {
100         int dif = _b.scl - _a.scl;
101         _a.man = shift(std::move(_a.man), dif);
102         _a.scl = _b.scl;
103     } else if (_b.scl < _a.scl) {
104         int dif = _a.scl - _b.scl;
105         _b.man = shift(std::move(_b.man), dif);
106         _b.scl = _a.scl;
107     }
108     Decimal res;
109     if (_a.sgn == _b.sgn) {
110         res.man = _a.man + _b.man;
111         res.sgn = res.man.ask0() ? 0 : _a.sgn;
112         res.scl = _a.scl;
113     } else {
114         int c = BigInt::cmp(_a.man, _b.man);
115         if (c == 0) {
116             res = Decimal(); // zero
117         } else if (c > 0) {
118             res.man = _a.man - _b.man;
119             res.sgn = _a.sgn;
120             res.scl = _a.scl;
121         } else {
122             res.man = _b.man - _a.man;
123             res.sgn = _b.sgn;

```

```

124         res.scl = _a.scl;
125     }
126 }
127     res.norm();
128     return res;
129 }
130
131 friend Decimal operator-(const Decimal &a, const Decimal &b) {
132     Decimal _b = b;
133     _b.sgn = -_b.sgn;
134     return a + _b;
135 }
136
137 friend Decimal operator*(const Decimal &a, const Decimal &b) {
138     if (a.sgn == 0 || b.sgn == 0)
139         return Decimal();
140     Decimal res;
141     res.man = a.man * b.man;
142     res.sgn = a.sgn * b.sgn;
143     res.scl = a.scl + b.scl;
144     res.round(pre);
145     return res;
146 }
147
148 friend Decimal operator/(const Decimal &a, const Decimal &b) {
149     if (b.sgn == 0)
150         throw std::runtime_error("Decimal division by zero");
151     if (a.sgn == 0)
152         return Decimal();
153     int prc = pre, wk = prc + 10;
154     int e = wk + b.scl - a.scl;
155     BigInt num = a.man, den = b.man;
156     if (e >= 0) num = shift(std::move(num), e);
157     else den = shift(std::move(den), -e);
158     auto qr = BigInt::divmod(num, den);
159     Decimal res;
160     res.man = qr.first;
161     res.sgn = a.sgn * b.sgn;
162     res.scl = wk;
163     res.round(prc);
164     return res;
165 }
166
167 friend Decimal operator%(const Decimal &a, const Decimal &b) {
168     if (b.sgn == 0)
169         throw std::runtime_error("Decimal modulo by zero");
170     if (a.sgn == 0)
171         return Decimal();
172     Decimal div = a / b;
173     Decimal q = div.trunc();
174     Decimal prd = q * b;
175     Decimal rem = a - prd;
176     return rem;
177 }
178
179 Decimal& operator+=(const Decimal& a) { return *this = *this + a; }
180 Decimal& operator-=(const Decimal& a) { return *this = *this - a; }
181 Decimal& operator*=(const Decimal& a) { return *this = *this * a; }
182 Decimal& operator/=(const Decimal& a) { return *this = *this / a; }
183 Decimal& operator%=(const Decimal& a) { return *this = *this % a; }
184
185 Decimal abs() const {
186     Decimal a = *this;
187     if (a.sgn < 0) a.sgn = 1;
188     return a;
189 }
190
191 static Decimal pow(Decimal bas, int exp) {
192     if (exp == 0)
193         return Decimal(1);
194     Decimal res(1);
195     while (exp) {
196         if (exp & 1)
197             res = res * bas;
198         bas = bas * bas;
199         exp >>= 1;
200     }
201     return res;
202 }
203
204 Decimal sqrt() const {
205     if (sgn < 0)

```

```

206         throw std::runtime_error("sqrt of negative");
207     if (sgn == 0)
208         return Decimal(0);
209     int prc = pre;
210     int e = man.str().size() - scl;
211     int t = e >= 0 ? e / 2 : -((-e + 1) / 2);
212     if (std::abs(t) > LIM)
213         throw std::overflow_error("sqrt exponent too large");
214     string s;
215     if (t >= 0) s = "1" + string(t, '0');
216     else s = "0." + string(-t - 1, '0') + "1";
217     Decimal x(s);
218     for (int it = 0; it < 60; ++it) {
219         Decimal nx = (x + (*this) / x) * Decimal("0.5");
220         Decimal dif = nx - x;
221         dif.norm();
222         if (dif.ask0() || dif.small(-prc - 2)) {
223             x = nx;
224             break;
225         }
226         x = nx;
227     }
228     x.round(prc);
229     return x;
230 }
231
232 BigInt fac() const {
233     if (scl != 0)
234         throw std::runtime_error("factorial only for integers");
235     if (sgn < 0)
236         throw std::runtime_error("factorial only for non-negative ints");
237     int n = 0;
238     string s = man.str();
239     if (s.size() > 20)
240         throw std::runtime_error("factorial too large to compute by this helper");
241     n = std::stoull(s);
242     return BigInt::fac(n);
243 }
244
245 BigInt gcd(const Decimal &o) const {
246     if (scl != 0 || o.scl != 0)
247         throw std::runtime_error("gcd defined for integers only in this template");
248     BigInt a = man, b = o.man;
249     return BigInt::gcd(a, b);
250 }
251
252 Decimal trunc() const {
253     if (sgn == 0)
254         return Decimal();
255     if (scl == 0)
256         return *this;
257     auto qr = BigInt::divmod(man, pow10(scl));
258     Decimal r;
259     r.man = qr.first;
260     r.sgn = sgn;
261     r.scl = 0;
262     return r;
263 }
264
265 bool ask0() const {
266     return sgn == 0;
267 }
268
269 bool small(int e) const {
270     if (e >= 0)
271         return false;
272     int k = -e;
273     if (scl < k)
274         return false;
275     int l = scl - k;
276     if (l == 0)
277         return man.ask0();
278     BigInt p10 = pow10(l);
279     int c = BigInt::cmp(man, p10);
280     return c < 0;
281 }
282
283 string strD() const {
284     std::ostringstream oss;
285     oss << *this;
286     string s = oss.str();
287     return s;

```



```

288     }
289
290 private:
291     int sgn;
292     BigInt man;
293     int scl;
294
295     static int pre;
296     static constexpr int LIM = (1LL << 31) - 1;
297
298     void norm() {
299         if (man.ask0()) {
300             sgn = 0;
301             scl = 0;
302             return;
303         }
304         while (scl > 0) {
305             auto qr = BigInt::divmod(man, 10);
306             if (qr.second != 0)
307                 break;
308             man = qr.first;
309             --scl;
310         }
311     }
312
313     void round(int kp) {
314         if (man.ask0()) {
315             sgn = 0;
316             scl = 0;
317             return;
318         }
319         if (scl <= kp) {
320             norm();
321             return;
322         }
323         int drp = scl - kp;
324         BigInt p10 = pow10(drp);
325         auto qr = BigInt::divmod(man, p10);
326         BigInt q = qr.first;
327         BigInt r = qr.second;
328         BigInt rr = BigInt::mulInt(r, 2);
329         if (BigInt::cmp(rr, p10) >= 0)
330             q = q + BigInt(1);
331         man = q;
332         scl = kp;
333         norm();
334     }
335
336     void read(const string &s) {
337         string t;
338         int n = s.size(), pos = 0;
339         while (pos < n && isspace(s[pos]))
340             ++pos;
341         int sg = 1;
342         if (pos < n && (s[pos] == '+' || s[pos] == '-')) {
343             if (s[pos] == '-')
344                 sg = -1;
345             ++pos;
346         }
347         string d;
348         int fd = 0;
349         bool dot = false;
350         while (pos < n) {
351             char c = s[pos++];
352             if (c == '.') {
353                 if (dot)
354                     break;
355                 dot = true;
356                 continue;
357             }
358             if (c >= '0' && c <= '9') {
359                 d.push_back(c);
360                 if (dot)
361                     ++fd;
362             } else
363                 break;
364         }
365         int p = 0;
366         while (p < d.size() && d[p] == '0')
367             ++p;
368         if (p == d.size()) {
369             sgn = 0;

```

```

370         scl = 0;
371         man = BigInt(0);
372         return;
373     }
374     string cor = d.substr(p);
375     man = BigInt(cor);
376     sgn = sg;
377     scl = fd;
378     norm();
379 }
380
381 static BigInt shift(BigInt a, int k) {
382     if (k < 0 || k > LIM)
383         throw std::overflow_error("decimal scale out of range");
384     return BigInt::mul10(std::move(a), k);
385 }
386
387 static BigInt pow10(int k) {
388     if (k < 0)
389         throw std::runtime_error("negative pow10");
390     return shift(BigInt(1), k);
391 }
392 static int pow10i(int k) {
393     if (k < 0)
394         throw std::runtime_error("negative pow10");
395     static const int mx = 1'000'000'000'000'000'000'000LL;
396     int r = 1;
397     for (int i = 0; i < k; ++i) {
398         if (r > mx / 10)
399             throw std::overflow_error("pow10Int overflow");
400         r *= 10;
401     }
402     return r;
403 }
404 };
405
406 int Decimal::pre = 50;

```

03B - 浮点数 (FastDecimal, 极致优化) .cpp

```

1  /*
2  用途: 03A Decimal 的卡常替代; 精度改为编译期常量, 避免每次运算读取/维护动态 scale。
3  依赖: 先粘贴 01B, 再粘贴 02B。FastDecimal<P> 与 FastFixedPoint<P> 语义相同, P 为小数位数。
4  取舍: 保留常用四则、比较、pow、输入输出; 需要 sqrt、动态 setPre、round 时使用 03A。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  template <int P>
8  using Decimal = FixedPoint<P>;

```

04A - 2至62进制转换 (常规) .cpp

```

1  /*
2  用途: 非负/负整数字符串在 2..62 进制间精确转换, 不依赖 BigInt。
3  复杂度: 逐次短除, O(输入位数*输出位数); 普通长度优先使用。
4  字符顺序: 0-9A-Za-z。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  struct BaseConv {
8      static int val(char c) {
9          if (isdigit((unsigned char)c)) return c - '0';
10         if (isupper((unsigned char)c)) return c - 'A' + 10;
11         return c - 'a' + 36;
12     }
13     static char chr(int x) {
14         if (x < 10) return char('0' + x);
15         if (x < 36) return char('A' + x - 10);
16         return char('a' + x - 36);
17     }
18
19     static string cvt(string s, int a, int b) {
20         assert(2 <= a && a <= 62 && 2 <= b && b <= 62);
21         bool neg = !s.empty() && s[0] == '-';
22         if (neg) s.erase(s.begin());
23         vector<int> d;
24         for (char c : s) {
25             int x = val(c);
26             if (x >= a) throw invalid_argument("BaseConv digit");
27             if (!d.empty() || x) d.push_back(x);

```

```

28     }
29     if (d.empty()) return "0";
30     string z;
31     while (!d.empty()) {
32         int r = 0;
33         for (auto &x : d) {
34             int y = r * a + x;
35             x = y / b;
36             r = y % b;
37         }
38         z += chr(r);
39         auto p = find_if(d.begin(), d.end(), [](int x) { return x != 0; });
40         d.erase(d.begin(), p);
41     }
42     if (neg) z += '-';
43     reverse(z.begin(), z.end());
44     return z;
45 }
46 };

```

04B - 2至62进制转换 (极致优化) .cpp

```

1  /*
2  用途: 超长 2..62 进制整数转换; 依赖已选择的 01A 或 01B `BigInt`。
3  优化: 按不超过 1e9 的大块读入/除出, 显著减少高精度乘除次数。
4  字符顺序: 0-9A-Za-z。
5  */
6  struct BaseConv {
7      static int val(char c) {
8          if (isdigit((unsigned char)c)) return c - '0';
9          if (isupper((unsigned char)c)) return c - 'A' + 10;
10         return c - 'a' + 36;
11     }
12     static char chr(int x) {
13         if (x < 10) return char('0' + x);
14         if (x < 36) return char('A' + x - 10);
15         return char('a' + x - 36);
16     }
17     static pii block(int b) {
18         int p = 1, k = 0;
19         while (p <= 1000000000 / b) p *= b, ++k;
20         return {p, k};
21     }
22
23     static string cvt(string s, int a, int b) {
24         assert(2 <= a && a <= 62 && 2 <= b && b <= 62);
25         bool neg = !s.empty() && s[0] == '-';
26         if (neg) s.erase(s.begin());
27         int p = 0;
28         while (p < s.size() && s[p] == '0') ++p;
29         s.erase(0, p);
30         if (s.empty()) return "0";
31
32         auto [pa, ka] = block(a);
33         BigInt x = 0;
34         int f = s.size() % ka;
35         if (!f) f = ka;
36         for (int l = 0; l < s.size(); ) {
37             int z = l ? ka : f, q = 1, v = 0;
38             for (int i = 0; i < z; ++i) {
39                 int d = val(s[l + i]);
40                 if (d >= a) throw invalid_argument("FastBaseConv digit");
41                 v = v * a + d;
42                 q *= a;
43             }
44             x = x * BigInt(q) + BigInt(v);
45             l += z;
46         }
47
48         auto [pb, kb] = block(b);
49         string z;
50         while (!x.ask0()) {
51             auto [q, r] = BigInt::divmod(x, pb);

```

```
52         x = move(q);
53         for (int i = 0; i < kb; ++i) {
54             z += chr(r % b);
55             r /= b;
56         }
57     }
58     while (z.size() > 1 && z.back() == '0') z.pop_back();
59     if (neg) z += '-';
60     reverse(z.begin(), z.end());
61     return z;
62 }
63 };
```

杂类

19 个代码模板 · 林月 · v1.0.0 · 2026.8.1

版块说明与知识结论

README.md

杂类模板

这里存放 XCPC / NOI 中会用到、但不适合归入基础算法、树、数据结构、图论、网络流、数学、字符串或高精度的代码。

编号	模板	适用场景
01A	普通莫队	静态区间询问，增删均容易。
01B	带修改莫队	区间询问中穿插单点修改。
01C	回滚莫队	加入容易、删除困难，能保存并回滚状态。
01D	Hilbert 序莫队	静态区间询问的空间填充曲线排序，免块长调参。
02	笛卡尔树	序列同时满足中序顺序与堆序；RMQ、区间结构、单调栈建树。
03	CDQ 分治（三维偏序）	三维偏序计数；重复点已合并，可据事件符号改成动态二维数点。
04	整体二分（带修改区间第 k 小）	离线单点赋值与动态区间顺序统计。
05	WQS 二分	带“恰选 k 个”限制且答案关于数量具有凸/凹性。
06	舞蹈链（DLX）	精确覆盖、数独和稀疏 0/1 约束。
07	模拟退火	允许启发式和随机化、没有可靠精确算法的优化题。
08	公历日期换算	日期、天数、星期互转。
09	Python 短代码	大整数、精确分数、枚举、容器和位集等 Python 优势写法。
10	德州扑克五张牌比较	标准五张牌牌型与完整同级比较键。
11A	物品装箱 First Fit	放入最靠前可容纳箱，线段树实现。
11B	物品装箱 Best Fit	放入剩余容量最小的可容纳箱，multiset 实现。
12	奇阶幻方	Siamese 法构造 $1 \dots n^2$ 标准幻方。
13	阿达马矩阵	Sylvester 法构造 2^k 阶正交 $+1/-1$ 矩阵。
14A	选数枚举 DFS	通用类型、可回调并保留下标。
14B	选数枚举 Gosper	$n \leq 63$ 的定大小位集枚举。
15	麻将胡牌判定（标准、七对子、国士）	34 种牌编码下的十四张手牌判定；不含番数和地方规则。

另见 [杂类知识与结论](#)，包含莫队与离线算法的适用条件、笛卡尔树性质、WQS/决策单调性、精确覆盖、随机化及语言差异；赛时检查清单见 [错题本](#)。

选择说明

- 01A / 01B / 01C / 01D 是同类区间离线算法的不同实现，因此共用编号 01 的分支。
- 原 01 - 随机数 与 基础算法/07 - 随机数 功能重复且没有特殊环境适配，已只保留基础算法中的均匀分布版本。
- 笛卡尔树虽然由单调栈构造，但产物和主要性质不同于普通“下一个更小元素”，故保留为独立模板。
- CDQ、整体二分和 WQS 都只在各自单调性成立时使用；不满足条件时不能把模板当作通用分治。
- 11A / 11B 是同一在线装箱问题的不同策略实现；二者都不等于 NP-hard 装箱问题的全局最优解。14A / 14B 是同一选数枚举在通用规模与 64 位掩码环境下的分支。
- 麻将模板只判三种常见和牌结构：标准型、七对子、国士无双；副露、番数、听牌和地方规则不在代码语义内，题面若有特殊规则需另写状态。

#define int long long 的影响

莫队询问、笛卡尔树节点、CDQ、整体二分和 DLX 默认都用公共头中的 `int` 保存下标与计数，以保持代码简洁统一。只有规模较大并实际超时或超空间时，才把纯下标和状态局部改为 `i32`；日期年份、WQS 代价与答案继续使用 `int`。

杂类知识与结论.md

杂类知识与结论

1. 莫队

- 普通莫队要求答案能在端点移动时维护。若 `add`、`del` 都是 $O(1)$ ，典型复杂度为 $O((n+q)\sqrt{n})$ 。
- 询问数与序列长度差异很大时，块长可取 $n/\sqrt{q}$ ；固定取 $\sqrt{n}$ 是更稳妥的默认值。
- 奇偶块反向排序右端点，可以减少相邻询问间的移动距离。
- Hilbert 序让相邻二维端点在空间上尽量接近，通常常数稳定且无需调块长，但排序键计算更复杂。
- 带修改莫队多一个时间维。三维块长通常取 $n^{2/3}$ ；修改回调必须在修改点处于当前区间时同步删除旧值、加入新值。
- 回滚莫队适合“加入容易、删除困难”的维护量，例如区间最大出现次数。跨块询问只扩展右端点，临时加入左端点后回滚；同块询问直接暴力。
- 莫队依赖离线重排；若询问必须在线回答，不能使用。

2. 笛卡尔树

- 中序遍历恰为原数组顺序，堆序由比较器决定，因此每棵子树对应原数组中的一个连续区间。
- 小根笛卡尔树中，两点的 LCA 是它们下标区间内的最小值位置；大根树对应区间最大值。
- 元素互异时树唯一。存在相等元素时必须固定“左侧优先”或“右侧优先”，否则父子关系和区间边界不稳定。
- 单调数组会退化成链；需要在树上递归时注意栈深，可改为迭代遍历。

3. CDQ 分治与事件顺序

- CDQ 的核心是按一维分治、在归并时统计另一维，再用 BIT 等结构处理最后一维。
- 三维偏序中的相等坐标必须统一处理。统计 $\leq$ 时，归并阶段相同第二维应先加入左半事件，再查询右半事件。
- 把修改视为带权点、查询视为前缀询问后，时间维 CDQ 可以处理动态二维数点：修改权值为 $+1/-1$ ，矩形查询拆成四个前缀。
- 归并结束必须撤销本层加入的数据结构；否则贡献会泄漏到兄弟递归区间。
- CDQ 只允许统计“左半对右半”的贡献。若关系不具有时间或第一维方向，需要先重新建模。

4. 整体二分

- 整体二分同时二分许多询问的答案。每层按中值把修改和询问稳定地分到左右两侧，原有时间顺序不能被打乱。
- 对区间第 k 小：先把值不超过中值的修改加入 BIT；若区间计数不少于 k ，询问进入左侧，否则令 k 减去该计数后进入右侧。
- 每层结束要回滚 BIT。典型复杂度为 $O((n+q)\log V \log n)$ 。
- 它要求答案域有单调判定；若一次修改会对判定产生无法线性撤销的影响，需换用别的数据结构。

5. WQS 二分与决策单调性

- WQS 用惩罚 c 把“恰选 k 个”变为无数量限制的 DP。最小化时计算 $\text{base} + c \cdot \text{cnt}$ ，最大化时常计算 $\text{base} - c \cdot \text{cnt}$ 。
- 随惩罚增大，最优方案的 cnt 必须单调不减。若同一带权值存在多种方案，必须固定为优先较大的 cnt ，否则二分边界会抖动。
- 最小化恢复式为 $\text{weighted} - c \cdot k$ ，最大化恢复式为 $\text{weighted} + c \cdot k$ 。
- 只有答案关于数量具有离散凸性/凹性且目标数量可达时，恢复出的才是“恰选 k 个”的真实答案。
- 分治 DP 优化只需事先证明最优决策点单调；不能因为转移长得相似就套模板。
- 区间 DP 的 Knuth 优化常见判据是四边形不等式与区间包含单调性，并得到 $\text{opt}[l][r-1] \leq \text{opt}[l][r] \leq \text{opt}[l+1][r]$ 。

6. 精确覆盖与 DLX

- 精确覆盖要求每个约束列恰好被选中一次；“至少一次”或“至多一次”不能直接使用同一建模。
- DLX 每层选择当前候选行最少的列，通常能显著减少搜索树，但最坏复杂度仍为指数级。
- 数独中通常把“格子取值、行内数字、列内数字、宫内数字”建成四组精确覆盖列。

7. 随机化与启发式

- 模拟退火没有正确性保证。温度、降温率、邻域尺度和重复次数都是题目参数，模板默认值只能作为起点。
- 目标值尺度很大时，接受概率 $\exp(-\text{delta} / \text{temperature})$ 中的温度也应同比例放大。
- 对拍或提交前应固定一个种子复现错误；正式随机哈希再使用时间种子。
- 单个 64 位随机哈希的碰撞概率很低但不为零；需要严格确定性时使用可证明算法或多模哈希。

8. 日期性质

- 公历每 400 年有 146097 天，恰好是 7 的倍数，因此星期分布以 400 年为周期。
- 闰年规则：能被 4 整除，但整百年还必须能被 400 整除。
- 日期差统一转为某个纪元起的整数天数后再相减，通常比逐月模拟更安全。

9. C++ 与 Python 容易混淆的语义

- C++ 整数除法向 0 截断，余数与被除数同号；Python `//` 向负无穷取整，`%` 与除数同号。
- C++ 的 `1 << k` 左值仍可能是 32 位；涉及高位应写 `1LL << k` 或 `u64(1) << k`。
- 默认 `int` 的最小值直接取绝对值会溢出；先转无符号幅值或使用 `i128`。
- Python 整数任意精度，但大整数乘除仍有复杂度，不能把它当成常数时间。
- Python `round` 使用 `ties-to-even`；竞赛中的“四舍五入”规则应显式实现。
- PyPy/CPython 的深递归既受递归上限也受系统栈影响；树链结构优先写迭代遍历。

10. 使用边界

- 能归入数学、图论、树、数据结构等目录的结论仍以对应目录为准，本文件不重复维护。
- Python 适合大整数、短暴力、组合枚举、字符串解析与精确分数；循环达到千万级、需要紧凑内存或依赖编译器优化时通常优先 C++。

11. 构造与参考例题补遗

- 五张德州扑克比较不能只比较“牌型编号”；同牌型还需按四条/三条/对子点数与踢脚牌逐级比较。A2345 顺子的最高牌按 5 处理。模板 10 直接生成字典序比较键。
- First Fit 与 Best Fit 是两种**在线装箱策略**，不是最少箱数的精确算法。前者用区间最大值找到最早可放箱，后者用 `multiset.lower_bound` 找剩余容量最小的可放箱；删除多重集元素时必须按迭代器删除一个。
- 奇阶幻方把 1 放在首行中点，随后尝试右上；目标格已有数时改为向下。公共和为 $n(n^2+1)/2$ 。阿达马矩阵按 $[[H, H], [H, -H]]$ 倍增，任意两行点积为 0。
- $N \times M$ 分块数独的参考构造中，宫大小为 $2^N \times 2^M$ 时，零下标格 (i, j) 可填 $((i \bmod 2^N) \text{ xor } \text{floor}(j/2^M)) * 2^M + (\text{floor}(i/2^N) \text{ xor } (j \bmod 2^M)) + 1$ 。
- 选数只需遍历组合时，通用环境用 DFS； $n \leq 63$ 时用 Gosper 枚举固定 popcount 的掩码。二者都应暴露回调，避免把“打印和”写死在模板里。

12. 牌面状态判定

德州扑克和麻将都属于“先编码状态，再比较/搜索”的模拟题。扑克先生成牌型键并按字典序比较；麻将先统计 34 种牌的重数，标准型固定将牌后递归拆面子，七对子检查偶数重数，国士检查十三种幺九字牌各至少一张。模板 09 - 杂类/15 明确采用四张牌可计两对的七对子规则；副露和地方番种不在接口内。

错题本.md

错题本

检查

1. 输入范围、是否多测、模数是否质数、图是否有重边/自环、答案是否需要取模。
2. 把所有全局数组、邻接表、堆、哈希表和计数器按测试用例清空；尤其检查 `static`、对象复用和时间戳数组。
3. 搜索 `INF`、`1<<`、`/`、`%`、`size()`、`assert`、`cerr`、`freopen`，逐处确认没有边界或环境问题。
4. 检查最后一个合法下标、空容器、`n=0/1`、只有一条边、全相等、全负数、模数为 2 等极端输入。
5. 本地编译一遍 `-std=c++17 -O2 -pipe -Wall -Wextra`；提交前去掉调试输出和本地文件重定向。

编译与本地调试

调试命令：

```
1 g++ -std=c++17 -O2 -pipe -Wall -Wextra -Wshadow -Wconversion main.cpp -o main
2 g++ -std=c++17 -O0 -g -D_GLIBCXX_DEBUG -fsanitize=address,undefined main.cpp -o main_dbg
```

`-Wconversion` 可能对竞赛中有意的窄化产生大量提示，正式卡常前再决定是否保留；它不是替代逻辑检查的证明工具。递归 DFS、点分治、线段树和支配树在链状数据上可能栈溢出，调试时先用 `ulimit -s unlimited` (Linux) 或改成显式栈。

不要把有副作用的表达式放进 `assert`，因为 `NDEBUG` 会把它删掉。

输入输出与多测

- `ios::sync_with_stdio(false)` 后不要混用 `scanf/printf` 与 `cin/cout`，除非明确处理缓冲顺序。
- 手写快读要处理负号、EOF 和读到非数字；`-0` 应归一化为 0。
- 多测时优先把状态放进 `solve` 的局部对象；必须复用全局数组时使用 `fill`、`clear` 或时间戳。
- 邻接表重置不能只清 `head`：链式前向星的边指针、反图、度数和拓扑队列都要清。
- `freopen`、`assert`、`cerr`、随机种子固定代码都要用条件编译包住；提交前搜索一次这些关键字。

整数、除法与位运算

1 溢出

- 乘法先提升：`i128(a) * b`；不能等乘完再转换。
- `abs(LLONG_MIN)`、`-numeric_limits<int>::lowest()` 和 `a-b` 都可能溢出；先判断范围。
- 最小化用的 `INF` 不要取最大值，留出至少两次加法余量；最大化同理使用 `-INF`。
- `1LL << k` 只在 `0<=k<63` 时安全；`1 << k` 的类型和位宽取决于字面量，`1<<62` 是典型翻车点。
- 位运算前明确是否需要自然溢出。需要 32 位环时局部使用 `u32`，不要让全局 `#define int long long` 破坏语义。

2 向下/向上取整

C++ 的负数 / 向零截断。区间分块、类欧和二分答案中若允许负数，使用公共头的 `floor(a,b)`、`ceil(a,b)`，不要直接写 `(a+b-1)/b`。

3 模运算

- 取模只适合加、乘、计数；最值比较、排序、概率大小不能先取模。
- 负数归一化写成 `(x%p+p)%p` 或使用 `MInt`；不要假定一次 `x+=p` 就足够，除非已证明 `x>-p`。
- `inv`、除法、阶乘逆元依赖模数为质数或元素互质；合数模要改用扩展欧几里得、CRT 或专用模板。
- 多测动态模数要在构造组合数、NTT 根和逆元表之前调用 `setMod`，不能中途切换。

STL、PBDS 与哈希

- `vector::erase` 会使后继迭代器失效；循环删除写 `it = v.erase(it)`。
- `priority_queue` 没有遍历接口；需要删除任意元素时改 `multiset`、懒删除堆或可并堆。

- `lower_bound` 的区间必须已按同一比较器排序; `greater<>` 排序后不能用默认比较器查找。
- `map::operator[]` 会插入默认值; 只读查询用 `find`, 避免无意改变容器和复杂度。
- PBDS 的 `tree` 中 `less_equal` 不是严格弱序, 不能直接作为标准 `tree` 比较器; 有序多重集用 “键值 + 唯一 id” 或本库 `03C/03E` 分支。
- `gp_hash_table` 在攻击数据或大量清空时可能退化; 预留容量、随机化哈希, 或切换到 `cc_hash_table / unordered_map`。
- 迭代器、引用和指针在 `vector` 扩容、`unordered_map` rehash、PBDS 修改后都可能失效。

常用的安全删除写法:

```
1 for (auto it = s.begin(); it != s.end(); ) {
2     if (bad(*it)) it = s.erase(it);
3     else ++it;
4 }
```

图论与网络流

- Dijkstra 只接受非负边; 出现负边时不要 “先跑一遍再修正”, 改 Bellman-Ford、Johnson 或题目专用方法。
- 无向图的重边不能用 `if (v==fa) continue` 跳过父边, 应按边 id 跳过; 否则桥、点双和边双会错。
- 缩点后所有边都要重新去重或允许重边, 且拓扑 DP 的入度必须基于缩点图, 不是原图。
- Floyd 的 `d[i][k]+d[k][j]` 先判断两项可达; 负环检测看 `d[i][i]<0`, 不要把负边误判成负环。
- 欧拉路要同时检查度数条件和所有非零度点连通; 只跑 Hierholzer 会把多个连通块拼成假答案。
- Stoer-Wagner 求无向全局割, 不指定源汇; 固定源汇、割边恢复和点对最小割要用最大流或 Gomory-Hu。
- 最大流边容量、反向边和总流量可能超过 32 位; 反向边必须与正向边成对, 当前弧指针每次增广后才能继续。
- 费用流中最短路势能必须在每次残量图改变后保持可行; 负费用边不能直接套无势能 Dijkstra。
- 二分图匹配的左右部编号、`pairU/pairV` 初值和 BFS 分层终点必须一致; Kuhn 找增广路前要清 `vis`。
- 支配树只讨论从指定根可达的有向子图; 不可达点不是 “被 0 支配”, 而是没有定义, 模板统一返回 0。
- Steiner 树的边权必须非负才能用 Dijkstra 扩展; 终端点重复时先去重或按模板约定拒绝。

树与数据结构

- LCA 的深度、祖先表和 Euler 序必须统一根; 换根后不能继续用旧的 `dep` 解释距离。
- Euler + RMQ 的 $O(1)$ LCA 查询的是 Euler 首次出现区间的最小深度, 不是 DFS 序区间的最小编号。
- 树链剖分路径拆分要保证 `top[x]` 深度不大于 `top[y]` 再交换; 最后一段是否包含端点取决于题意。
- 点分治/点分树中 “删除重心” 只影响递归分治结构, 不应修改原图; 距离桶和子树桶要区分来源, 避免同一节点被重复统计。
- 懒标记下推顺序通常是先赋值再加法; 合并标记时明确新标记作用在旧值之前还是之后。
- 线段树节点空间通常至少 $4*n$; 动态开点需估算 “操作次数 $\times$ 每次新节点数”, 不能只按 n 开数组。
- 主席树两棵根相减时, 左右子树和计数的符号必须保持一致; 第 k 小的 k 是 1-indexed 还是 0-indexed 要在接口注释写明。
- DSU on Tree 的 `add/del` 必须互为逆操作; `keep=false` 的轻儿子清理不能漏掉当前重儿子。
- 可撤销并查集禁止路径压缩, 只能按秩/大小合并并记录修改栈。

数学、多项式与几何

- 组合数先确认 $n < \text{mod}$; 大 n 小质数模使用 Lucas, 合数模不能直接套 Lucas。
- 原根、离散对数、NTT 根都依赖模数结构; 先分解 $p-1$ 或检查单位群阶, 不要凭记忆写常数根。
- CRT 合并时先处理非互质模数和负余数; 溢出风险用 `i128`, 不要把乘法塞回 `int`。
- 多项式长度先补到需要的卷积长度, 再做 NTT; 逆变换必须乘 n^{-1} , 且每次蝶形的模减法要归一化。
- 任意模卷积的 CRT 结果需要确认真实答案小于模数乘积; 否则增加模数或改用分块/FFT killer 分支。
- 形式幂级数的 `inv/log/exp` 通常要求常数项条件 (如 `a[0]!=0`、`a[0]=1`); 截断长度和导数积分的下标要同步。
- 高斯消元选主元时区分 “精确为 0” 和 “绝对值小于 `eps`”; 整数模消元不能使用浮点 `eps`。
- 几何叉积的符号、共线端点和 `long double` 误差必须统一; 排序极角不能直接比较 `atan2` 后忽略边界。
- 凸包是否保留共线点会改变旋转卡壳和点包含答案, 模板的 `<=` / `<` 不能随意替换。

字符串与 DP

- KMP/Z/Manacher 的下标约定要在空串、长度 1 和偶回文上跑一遍；复制字符串时不要混入哨兵字符。
- 双哈希比较仍可能碰撞；可接受概率风险时随机基，要求确定性时使用后缀数组/自动机或直接比较短串。
- AC 自动机统计出现次数时，fail 树上的计数要在匹配结束后统一汇总；在线输出和离线汇总不能混用。
- SAM 的 len/link/next 复制顺序必须先复制转移再改 len；不同字符串插入时要重置 last 或使用广义版本。
- 区间 DP 先枚举长度再枚举左端点；初始化空区间和不可达状态，否则第一层就会污染答案。
- 01 背包倒序、完全背包正序；“恰好装满”不能把所有状态初始化成 0。
- 单调队列优化前确认队首弹出条件、窗口边界和转移中的减法符号；一个 $\leq$ 改成 $<$ 可能改变方案计数。
- CHT 的斜率方向、查询方向和求最小/最大三件事要同时核对；不满足单调性就换 Li Chao。
- WQS 二分必须记录最优数量并验证随惩罚单调；只二分值而不回收数量会得到相邻数量的错误答案。
- Slope Trick 只维护凸函数；若转移产生非凸拐点，先证明可分解或不要套模板。

模拟、牌面与构造

- 牌面比较先做“牌型 + 同级关键字”键，再整体字典序比较；不要只比较最大牌。
- 顺子通常有 A2345 特判；同花顺必须同时满足同花和顺子，四条/葫芦的踢脚牌顺序也要固定。
- 麻将标准型搜索先选将牌，再从最小非零牌尝试刻子和顺子；字牌不能组成顺子，四张相同牌在七对子中的规则要写清。
- 棋盘、坐标、旋转和镜像题先固定一个方向，所有变换都通过同一个函数；不要在输出阶段临时交换行列。
- 构造题先验证总和、奇偶、度数和边界，再输出；“看起来对称”不代表满足最后一列/最后一个元素。
- 日期模拟统一使用半开区间或闭区间，闰年条件写成 $(y\%400==0) || (y\%4==0 \& \& y\%100!=0)$ ，不要只判断 $\%4$ 。

考点

题面	模板	前提
“所有从根到点的路径都必须经过”	支配树	有向图、固定根、只看根可达部分
“连接很少的指定点，允许经过中间点”	Steiner 树状态压缩	终端数 k 是否足够小、边权是否非负
“函数是若干绝对值/折线的和，反复取前缀最小”	Slope Trick	转移后仍是凸分段线性函数
“没有指定源汇，问整张无向图最小割”	Stoer-Wagner	不是固定 $s-t$ 割，也不要求恢复流量
“模数很大/合数，组合数 n 远大于模数”	扩展 Lucas / CRT / 质因数分解	模数分解、 n 与 k 的范围
“ n 很大但只出现 $\text{floor}(n/i)$ 、质数前缀和”	整除分块 / 杜教筛 / Min_25	商值段数和所需积性函数
“集合大小 $n \leq 20$ ，每次合并两个子集”	子集 DP / 子集卷积	2^n 内存是否可接受，是否需要 3^n 合并
“多项式长度不是二的幂或模数不适合 NTT”	任意因子长度 DFT / CRT-NTT / FFT	真实系数上界与模数乘积是否足够
“区间加、区间和，操作在线且没有复杂信息”	懒标记线段树或 BIT 差分	是单点/区间更新，还是要维护最值、历史值
“答案只依赖最后一次出现位置”	离线 BIT / 扫描线	查询能否按右端点排序，更新是否有后效性
“输出任意方案，多个答案都对”	构造 + 校验器思维	输出顺序、重复元素和特殊解是否允许

卡常、缓存与内存池

- 热循环里的数据尽量连续：vector<vector<E>> 适合可读性，边数和访问量极大时改用 CSR/链式前向星；预先 reserve，避免扩容复制。
- 二重循环优先让内层访问连续内存；矩阵按实际存储顺序遍历。不要为了“看起来对称”交换循环，先看缓存访问模式。
- 大数组放静态区或堆上，别把 $O(n)$ 数组放进递归栈；节点数按最坏操作次数估算，不能只按初始 n 开池。
- 频繁创建小对象时使用简单内存池；释放时只回退指针，不在热循环里反复 new/delete：

```

1  template <class T, int N> struct Pool {
2      array<T, N> a;
3      int n = 0;
4      T *get() { assert(n < N); return &a[n++]; }
5      void clear() { n = 0; }
6  };

```

- vector<bool> 是位代理容器，随机访问和取地址语义都特殊；需要高频修改时优先 vector<char> 或 bitset。
- __builtin_ctz*、__builtin_clz*、popcount* 的参数类型和零值边界要确认；对 0 调用 ctz/clz 是未定义行为。

- 需要 32 位自然溢出时局部使用 u32；不要把整个程序改成 32 位而让答案、距离和容量一起溢出。#define int long long 造成的内存/缓存损失只有在测出瓶颈后再处理。
- 输出大量方案时先写入 string 或缓冲区再一次输出；不要在循环里频繁 flush，也不要误用 endl（公共宏已将其替换为换行字符串）。
- 浮点累计误差明显时使用 Kahan 补偿；它增加常数，只在误差确实成为瓶颈时使用：

```
1 ld s = 0, c = 0;
2 for (ld x : a) {
3     ld y = x - c, t = s + y;
4     c = (t - s) - y;
5     s = t;
6 }
```

- -O2、-pipe 通常可用；-march=native 可能在评测机不支持，除非平台明确保证，不要把它写进提交依赖。

随机化与对拍

最小可用对拍流程：随机生成器、暴力解、正式解、规范化输出、限时循环。生成器要覆盖小范围穷举边界和结构化坏例，而不只是均匀随机。

```
1 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
2 int rd(int l, int r) { return uniform_int_distribution<int>(l, r)(rng); }
```

对拍优先加入：全相等、严格递增/递减、星形/链形/完全图、重复边、孤立点、答案为 0、恰好越界、负数和最大模数。发现差异后固定随机种子，并把最小反例保存成回归测试。

随机哈希不要使用固定易猜的基；哈希值参与下标时先转无符号类型，避免负数取模。模拟退火、随机增量几何和哈希冲突都要在题目允许的误差概率内使用。

短片段

安全乘法与归一化

```
1 using i128 = __int128_t;
2 int mul(int a, int b, int p) { return i128(a) * b % p; }
3 int norm(int x, int p) { x %= p; if (x < 0) x += p; return x; }
```

不溢出的二分中点

```
1 int m = l + (r - l) / 2;
```

坐标压缩后找下标

```
1 sort(v);
2 v.erase(unique(v.begin(), v.end()), v.end());
3 int id = lower_bound(v.begin(), v.end(), x) - v.begin();
```

下一段相同值跳转

```
1 for (int l = 1; l <= n; ) {
2     int r = l;
3     while (r < n && a[r + 1] == a[l]) ++r;
4     // 处理 [l,r]
5     l = r + 1;
6 }
```

时间戳清空

```
1 ++tag;
2 if (vis[x] != tag) vis[x] = tag, val[x] = 0;
```

调试断言不改变状态

```
1 auto ok = check(x);
2 assert(ok);
```

易错点

表现	最常见原因	修复动作
小数据正确、大数据 RE	递归栈、节点池或 $4*n$ 不够	估算最坏节点数；链图改迭代或加栈
随机数据偶发 WA	未定义行为、哈希碰撞、浮点比较	ASan/UBSan，对拍固定种子，几何使用 eps
答案差一个模数	负数未归一化或减法只减了一次	统一 <code>norm / MInt</code> ，检查每个分支的范围
TLE 但复杂度正确	<code>#define int long long</code> 放大了下标数组、缓存未命中	只在确认瓶颈后把下标改 <code>i32</code> ，并保留注释说明
RE 只发生在极端图	孤立点、重边、自环、非连通未处理	先写连通性/度数/边界判定，再调用核心算法
本地 AC、评测 CE	依赖 <code>bits/stdc++.h</code> 、PBDS 或本地头文件	确认题目编译器；提交版走 <code>ONLINE_JUDGE</code> 分支
调试版结果和提交版不同	<code>assert</code> 内含赋值/自增，或调试宏有副作用	把表达式拆成普通语句，再断言纯布尔值

09 - Python短代码.md

Python 竞赛短代码

以下片段面向 Python 3 标准库，按需复制，不依赖第三方包。

快速输入输出

```
1 import sys
2 input = sys.stdin.buffer.readline
3
4 n = int(input())
5 a = list(map(int, input().split()))
6 sys.stdout.write(" ".join(map(str, a)))
```

一次读完所有整数，适合输入结构简单的数据：

```
1 import sys
2 it = iter(map(int, sys.stdin.buffer.read().split()))
3 n = next(it)
4 a = [next(it) for _ in range(n)]
```

大整数与常用整数函数

```
1 import sys
2 from math import gcd, lcm, isqrt, comb, perm
3
4 if hasattr(sys, "set_int_max_str_digits"):
5     sys.set_int_max_str_digits(0) # Python 3.11+ 允许超长十进制整数
6
7 x = int(input())
8 y = pow(x, 10) # 大整数幂
9 z = pow(x, -1, 998244353) # 模逆，要求互质
10 r = isqrt(x) # floor(sqrt(x))，完全精确
11 c = comb(1000, 500)
```

日期与时间

```
1 from datetime import date, time, datetime
2
3 d = date.fromisoformat("2026-08-01")
4 t = time.fromisoformat("14:30:45")
5 dt = datetime.combine(d, t)
6 s = dt.strftime("%Y-%m-%d %H:%M:%S")
7 back = datetime.strptime(s, "%Y-%m-%d %H:%M:%S")
```

`date` 只保存年月日，`time` 只保存时分秒；需要两者时使用 `datetime`。字符串格式化和解析的格式串必须完全对应。

进制转换

```
1 x = int("101101", 2)
```

```

2 s2, s8, s16 = bin(x)[2:], oct(x)[2:], hex(x)[2:]
3
4 def base(x, b):
5     d = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
6     if x == 0:
7         return "0"
8     sg = "-" if x < 0 else ""
9     x = abs(x)
10    s = ""
11    while x:
12        x, r = divmod(x, b)
13        s += d[r]
14    return sg + s[::-1]

```

精确分数与高精度小数

```

1 from fractions import Fraction
2
3 a = Fraction(1, 3) + Fraction(2, 5)
4 print(a.numerator, a.denominator)
5 b = Fraction("2/5")
6 c = Fraction("0.125")
7 near = Fraction("3.14159").limit_denominator(1000)

```

```

1 from decimal import Decimal, getcontext
2
3 getcontext().prec = 100
4 x = Decimal(2).sqrt()
5 print(x)

```

Decimal 必须从字符串或整数构造；从 float 构造会把二进制浮点误差带入。

math 浮点工具

```

1 import math
2
3 u = math.ceil(3.2)
4 v = math.floor(3.8)
5 w = math.hypot(3, 4)
6 s = math.fsum([0.1, 0.1, 0.1])
7 rad = math.radians(180)
8 deg = math.degrees(math.pi)

```

math.fsum 比普通 sum 更适合浮点累计；金额或要求精确十进制结果时仍使用 Decimal 或 Fraction。

枚举与组合暴力

```

1 from itertools import permutations, combinations, product
2
3 for p in permutations(range(n)):
4     pass
5 for s in combinations(range(n), k):
6     pass
7 for a in product(range(3), repeat=n): # 3^n 状态
8     pass

```

常用容器

```

1 from collections import Counter, defaultdict, deque
2 from bisect import bisect_left, bisect_right, insort
3 from heapq import heappush, heappop
4
5 cnt = Counter(a)
6 g = defaultdict(list)
7 q = deque([s])
8 p = bisect_left(a, x)

```

最小堆直接存值；最大堆存相反数：

```

1 h = []
2 heappush(h, -x)
3 x = -heappop(h)

```

用 Python 整数充当动态位集

子集和可达性：

```
1 f = 1
2 for x in a:
3     f |= f << x
4 ok = (f >> s) & 1
```

枚举置位位置：

```
1 x = f
2 while x:
3     b = x & -x
4     i = b.bit_length() - 1
5     x ^= b
```

记忆化与递归

```
1 import sys
2 from functools import cache
3 sys.setrecursionlimit(1_000_000)
4
5 @cache
6 def dfs(x, y):
7     return 0
```

链状树和百万层状态不要仅依赖 `setrecursionlimit`，应改写为显式栈。

分组与排序键

```
1 from itertools import groupby
2
3 a.sort(key=lambda x: (x[0], -x[1]))
4 for k, g in groupby(a, key=lambda x: x[0]):
5     g = list(g)
```

C++ 式向 0 整除

Python 的 `//` 对负数向下取整；需要复现 C++ 行为时：

```
1 def div0(a, b):
2     q = abs(a) // abs(b)
3     return -q if (a < 0) ^ (b < 0) else q
4
5 q = div0(a, b)
6 r = a - q * b
```

01A - 普通莫队.cpp

```
1 /*
2 用途：离线处理静态序列区间询问，约  $O((n + q) \sqrt{n})$ 。
3 约定：下标从 1 开始，询问为闭区间；add/del 接收下标，get 返回当前答案。
4 性能：默认使用展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
5 */
6 struct Mo {
7     struct Q {
8         int l, r, id;
9     };
10
11     int n;
12     vector<Q> q;
13
14     explicit Mo(int n) : n(n) {}
15
16     void ask(int l, int r) {
17         q.push_back({l, r, q.size()});
18     }
19
20     template <class A, class D, class G>
21     auto solve(A add, D del, G get) {
22         using R = decay_t<decltype(get())>;
23         int m = q.size();
24         vector<R> ans(m);
25         if (!m) return ans;
```

```

26     int z = max<int>(1, sqrt(m));
27     int b = max<int>(1, n / z);
28     sort(q.begin(), q.end(), [b](const Q& x, const Q& y) {
29         int a = (x.l - 1) / b, c = (y.l - 1) / b;
30         if (a != c) return a < c;
31         return a & 1 ? x.r > y.r : x.r < y.r;
32     });
33
34     int l = 1, r = 0;
35     for (auto [x, y, id] : q) {
36         while (l > x) add(--l);
37         while (r < y) add(++r);
38         while (l < x) del(l++);
39         while (r > y) del(r--);
40         ans[id] = get();
41     }
42     return ans;
43 }
44 };

```

01B - 带修改莫队.cpp

```

1  /*
2  用途：离线处理区间询问与单点修改，n、q 同阶时约  $O(n^{5/3})$ 。
3  约定：下标从 1 开始。每读到一次修改先调用 modify，再加入后续询问。
4  apply/undo 接收修改编号和当前 [l,r]：若修改点在区间内，应先 del，再改值，最后 add。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  struct Mo {
8      struct Q {
9          int l, r, t, id;
10     };
11
12     int n, c = 0;
13     vector<Q> q;
14
15     explicit Mo(int n) : n(n) {}
16
17     void modify() { ++c; }
18
19     void ask(int l, int r) {
20         q.push_back({l, r, c, q.size()});
21     }
22
23     template <class A, class D, class P, class U, class G>
24     auto solve(A add, D del, P ap, U ud, G get) {
25         using R = decay_t<decltype(get())>;
26         int m = q.size();
27         vector<R> ans(m);
28         if (!m) return ans;
29         int b = max<int>(1, pow(max(n, m), 2.0 / 3.0));
30         sort(q.begin(), q.end(), [b](const Q& x, const Q& y) {
31             int a = (x.l - 1) / b, c = (y.l - 1) / b;
32             if (a != c) return a < c;
33             int d = (x.r - 1) / b, e = (y.r - 1) / b;
34             if (d != e) return a & 1 ? d > e : d < e;
35             return (a ^ d) & 1 ? x.t > y.t : x.t < y.t;
36         });
37
38         int l = 1, r = 0, t = 0;
39         for (auto [x, y, z, id] : q) {
40             while (t < z) ap(++t, l, r);
41             while (t > z) ud(t--, l, r);
42             while (l > x) add(--l);
43             while (r < y) add(++r);
44             while (l < x) del(l++);
45             while (r > y) del(r--);
46             ans[id] = get();
47         }
48         return ans;
49     }
50 };

```

01C - 回滚莫队.cpp

```

1  /*
2  用途：add 容易而 del 困难时的静态区间询问，约  $O((n + q) \sqrt{n})$ 。
3  回调：clear 清空状态；save 返回快照；rollback(s) 回到快照；brute(l,r) 独立暴力同块询问。

```

```

4 约定：下标从 1 开始，add 只加入下标，get 返回当前答案。
5 性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  struct Mo {
8      struct Q {
9          int l, r, id;
10     };
11
12     int n;
13     vector<Q> q;
14
15     explicit Mo(int n) : n(n) {}
16
17     void ask(int l, int r) {
18         q.push_back({l, r, q.size()});
19     }
20
21     template <class A, class C, class S, class R, class G, class B>
22     auto solve(A add, C clr, S sav, R ud, G get, B bf) {
23         using V = decay_t<decltype(get())>;
24         int m = q.size();
25         vector<V> ans(m);
26         if (!m) return ans;
27         int b = max<int>(1, sqrt(n));
28         sort(q.begin(), q.end(), [b](const Q& x, const Q& y) {
29             int a = (x.l - 1) / b, c = (y.l - 1) / b;
30             if (a != c) return a < c;
31             return x.r < y.r;
32         });
33
34         for (int p = 0; p < m; p++) {
35             int k = (q[p].l - 1) / b, z = p;
36             while (z < m && (q[z].l - 1) / b == k) ++z;
37             int e = min<int>(n + 1, (k + 1) * b + 1);
38             clr();
39             int r = e - 1;
40             for (int i = p; i < z; ++i) {
41                 auto [l, qr, id] = q[i];
42                 if (qr < e) {
43                     ans[id] = bf(l, qr);
44                     continue;
45                 }
46                 while (r < qr) add(++r);
47                 auto s = sav();
48                 for (int j = e - 1; j >= l; --j) add(j);
49                 ans[id] = get();
50                 ud(s);
51             }
52             p = z;
53         }
54         return ans;
55     }
56 };

```

01D - Hilbert序莫队.cpp

```

1  /*
2  用途：用 Hilbert 序替代分块排序的静态莫队，省去块长调参，常数通常更稳定。
3  约定：下标从 1 开始；add/del 接收下标，get 返回当前答案。
4  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
5   */
6  struct Mo {
7      struct Q {
8          int l, r, id;
9          int o;
10     };
11
12     int n, p = 1;
13     vector<Q> q;
14
15     explicit Mo(int n) : n(n) {
16         int x = max<int>(1, n) - 1;
17         p = 0;
18         do ++p; while (x >= 1);
19     }
20
21     static int ord(int x, int y, int p, int r = 0) {
22         if (!p) return 0;
23         int h = 1LL << (p - 1);
24         int s = x < h ? (y < h ? 0 : 3) : (y < h ? 1 : 2);

```



```

25     s = (s + r) & 3;
26     static constexpr int d[] = {3, 0, 0, 1};
27     int nr = (r + d[s]) & 3;
28     int z = 1LL << (2 * p - 2);
29     int w = ord(x & (h - 1), y & (h - 1), p - 1, nr);
30     return s * z + (s == 1 || s == 2 ? w : z - w - 1);
31 }
32
33 void ask(int l, int r) {
34     q.push_back({l, r, q.size(), ord(l - 1, r - 1, p)});
35 }
36
37 template <class A, class D, class G>
38 auto solve(A add, D del, G get) {
39     using R = decay_t<decltype(get())>;
40     vector<R> ans(q.size());
41     sort(q.begin(), q.end(), [](const Q& a, const Q& b) { return a.o < b.o; });
42     int l = 1, r = 0;
43     for (auto [x, y, id, o] : q) {
44         while (l > x) add(--l);
45         while (r < y) add(++r);
46         while (l < x) del(l++);
47         while (r > y) del(r--);
48         ans[id] = get();
49     }
50     return ans;
51 }
52 };

```

02 - 笛卡尔树.cpp

```

1  /*
2  用途:  $O(n)$  构造笛卡尔树; 中序遍历为原下标顺序, 同时满足堆性质。
3  模板:  $C=\text{less}<T>$  为小根树,  $C=\text{greater}<T>$  为大根树;  $E=\text{true}$  时相等元素中右侧在上层。
4  约定: 输入及  $rt/lc/rc/fa$  均为  $0\text{-indexed}$ , 空节点为  $-1$ 。
5  性能: 默认使用宏展开后的  $\text{int}$ ; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为  $\text{i32}$ 。
6  */
7  template <class T, class C = less<T>, bool E = false>
8  struct CartTree {
9      int rt = -1;
10     vector<int> lc, rc, fa;
11     C c;
12
13     CartTree() = default;
14     explicit CartTree(const vector<T>& a, C c = {}) : c(c) { build(a); }
15
16     void build(const vector<T>& a) {
17         int n = a.size();
18         rt = -1;
19         lc.assign(n, -1);
20         rc.assign(n, -1);
21         fa.assign(n, -1);
22         vector<int> s;
23         s.reserve(n);
24         for (int i = 0; i < n; ++i) {
25             int x = -1;
26             while (!s.empty()) {
27                 int j = s.back();
28                 bool f = c(a[i], a[j]);
29                 if constexpr (E) f = f || !c(a[j], a[i]);
30                 if (!f) break;
31                 x = j;
32                 s.pop_back();
33             }
34             if (s.empty()) rt = i;
35             else fa[i] = s.back(), rc[s.back()] = i;
36             if (x != -1) fa[x] = i, lc[i] = x;
37             s.push_back(i);
38         }
39     }
40 };

```

03 - CDQ分治 (三维偏序) .cpp

```

1  /*
2  用途: 统计三维偏序。返回  $\text{ans}[k]$ : 恰有  $k$  个其他点三维坐标均不大于它的点数。
3  复杂度:  $O(n \log n)$ , 重复点自动合并; 坐标类型由  $T$  适配。
4  性能: 默认使用宏展开后的  $\text{int}$ ; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为  $\text{i32}$ 。

```

```

5  */
6  template <class T>
7  struct CDQ3 {
8      struct P {
9          T y;
10         int z, c, v;
11     };
12
13     vector<P> a, t;
14     vector<int> bit;
15
16     void modify(int x, int v) {
17         for (++x; x < bit.size(); x += x & -x) bit[x] += v;
18     }
19
20     int ask(int x) const {
21         int r = 0;
22         for (++x; x; x -= x & -x) r += bit[x];
23         return r;
24     }
25
26     void cdq(int l, int r) {
27         if (r - l <= 1) return;
28         int m = (l + r) / 2;
29         cdq(l, m);
30         cdq(m, r);
31         int i = l, j = m, k = l;
32         while (i < m && j < r) {
33             if (a[i].y <= a[j].y) {
34                 modify(a[i].z, a[i].c);
35                 t[k++] = a[i++];
36             } else {
37                 a[j].v += ask(a[j].z);
38                 t[k++] = a[j++];
39             }
40         }
41         int p = i;
42         while (j < r) {
43             a[j].v += ask(a[j].z);
44             t[k++] = a[j++];
45         }
46         while (i < m) t[k++] = a[i++];
47         for (int x = l; x < p; ++x) modify(a[x].z, -a[x].c);
48         copy(t.begin() + l, t.begin() + r, a.begin() + l);
49     }
50
51     vector<int> solve(vector<array<T, 3>> p) {
52         int n = p.size();
53         vector<int> ans(n);
54         if (!n) return ans;
55         sort(p.begin(), p.end());
56         vector<T> z;
57         z.reserve(n);
58         for (auto& x : p) z.push_back(x[2]);
59         sort(z.begin(), z.end());
60         z.erase(unique(z.begin(), z.end()), z.end());
61
62         a.clear();
63         for (int i = 0; i < n; i) {
64             int j = i + 1;
65             while (j < n && p[j] == p[i]) ++j;
66             int k = lower_bound(z.begin(), z.end(), p[i][2]) - z.begin();
67             a.push_back({p[i][1], k, j - i, 0});
68             i = j;
69         }
70         t.resize(a.size());
71         bit.assign(z.size() + 1, 0);
72         cdq(0, a.size());
73         for (auto& x : a) ans[x.v + x.c - 1] += x.c;
74         return ans;
75     }
76 };

```

04 - 整体二分（带修改区间第k小）.cpp

```

1  /*
2  用途：离线处理单点赋值与区间第 k 小，复杂度  $O((n+q) \log V \log n)$ 。
3  约定：初始数组 a 为 1-indexed (a[0] 不使用)；modify/ask 的位置同样从 1 开始。
4  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
5  */

```

```

6  template <class T> struct KthModify {
7      struct O {
8          int t, p, l, r, k, id;
9          T x;
10     };
11     struct E {
12         int t, p, d, v, l, r, k, id;
13     };
14
15     vector<T> a, val, ans;
16     vector<O> o;
17     vector<int> bit;
18     int n, qc = 0;
19
20     explicit KthModify(vector<T> a) : a(std::move(a)) {
21         assert(this->a.size() >= 2);
22         n = this->a.size() - 1;
23     }
24
25     void modify(int p, T x) {
26         assert(1 <= p && p <= n);
27         o.push_back({0, p, 0, 0, 0, -1, x});
28     }
29
30     int ask(int l, int r, int k) {
31         assert(1 <= l && l <= r && r <= n && 1 <= k && k <= r - l + 1);
32         o.push_back({1, 0, l, r, k, qc, T{}});
33         return qc++;
34     }
35
36     void bitAdd(int x, int v) {
37         for (; x <= n; x += x & -x) bit[x] += v;
38     }
39
40     int bitAsk(int x) const {
41         int r = 0;
42         for (; x; x -= x & -x) r += bit[x];
43         return r;
44     }
45
46     void dfs(int l, int r, vector<E> e) {
47         bool h = false;
48         for (auto &x : e) h |= x.t;
49         if (!h) return;
50         if (l == r) {
51             for (auto &x : e)
52                 if (x.t) ans[x.id] = val[l];
53             return;
54         }
55         int m = (l + r) / 2;
56         vector<E> x, y;
57         x.reserve(e.size());
58         y.reserve(e.size());
59         for (auto z : e) {
60             if (!z.t) {
61                 if (z.v <= m)
62                     bitAdd(z.p, z.d), x.push_back(z);
63                 else
64                     y.push_back(z);
65             } else {
66                 int c = bitAsk(z.r) - bitAsk(z.l - 1);
67                 if (c >= z.k)
68                     x.push_back(z);
69                 else
70                     z.k -= c, y.push_back(z);
71             }
72         }
73         for (auto &z : e)
74             if (!z.t && z.v <= m) bitAdd(z.p, -z.d);
75         dfs(l, m, std::move(x));
76         dfs(m + 1, r, std::move(y));
77     }
78
79     vector<T> solve() {
80         val.assign(a.begin() + 1, a.end());
81         for (auto &x : o)
82             if (!x.t) val.push_back(x.x);
83         sort(val.begin(), val.end());
84         val.erase(unique(val.begin(), val.end()), val.end());
85         auto id = [&](const T &x) -> int { return lower_bound(val.begin(), val.end(), x) - val.begin(); };
86
87         vector<E> e;

```

```

88     e.reserve(n + 2 * o.size());
89     vector<T> cur = a;
90     for (int i = 1; i <= n; ++i) e.push_back({0, i, 1, id(cur[i]), 0, 0, 0, -1});
91     for (auto &x : o) {
92         if (!x.t) {
93             e.push_back({0, x.p, -1, id(cur[x.p]), 0, 0, 0, -1});
94             e.push_back({0, x.p, 1, id(x.x), 0, 0, 0, -1});
95             cur[x.p] = x.x;
96         } else {
97             e.push_back({1, 0, 0, 0, x.l, x.r, x.k, x.id});
98         }
99     }
100    ans.assign(qc, T{});
101    bit.assign(n + 1, 0);
102    if (qc) dfs(0, val.size() - 1, std::move(e));
103    return ans;
104 }
105 };

```

05 - WQS二分 (带权二分) .cpp

```

1  /*
2  用途: 把“恰选 k 个”的凸/凹离散优化转成带惩罚 DP。
3  wqsMin: f(c) 最小化 base + c*cnt; wqsMax: f(c) 最大化 base - c*cnt。
4  要求: 惩罚 c 增大时 cnt 单调不减, 且同时 f 必须优先返回更大的 cnt。
5  */
6  template <class T, class F>
7  T wqsMin(T l, T r, int k, F f) {
8      while (l < r) {
9          T m = l + (r - l + 1) / 2;
10         auto p = f(m);
11         if (p.second >= k) l = m;
12         else r = m - 1;
13     }
14     return f(l).first - l * static_cast<T>(k);
15 }
16
17 template <class T, class F>
18 T wqsMax(T l, T r, int k, F f) {
19     while (l < r) {
20         T m = l + (r - l + 1) / 2;
21         auto p = f(m);
22         if (p.second >= k) l = m;
23         else r = m - 1;
24     }
25     return f(l).first + l * static_cast<T>(k);
26 }

```

06 - 舞蹈链 (DLX, 精确覆盖) .cpp

```

1  /*
2  用途: 求精确覆盖的一组可行行, 适合数独、棋盘覆盖等; 搜索复杂度取决于实例。
3  列编号为 1..m, addRow(id, cols) 加入一行; 同一行的列不得重复。
4  性能: 默认使用展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
5  */
6  struct DLX {
7      int m;
8      vector<int> l, r, u, d, row, col, s, cur, ans;
9
10     explicit DLX(int m, int z = 0) : m(m) {
11         l.resize(m + 1);
12         r.resize(m + 1);
13         u.resize(m + 1);
14         d.resize(m + 1);
15         row.assign(m + 1, 0);
16         col.resize(m + 1);
17         s.assign(m + 1, 0);
18         for (int i = 0; i <= m; ++i) {
19             l[i] = i - 1;
20             r[i] = i + 1;
21             u[i] = d[i] = col[i] = i;
22         }
23         l[0] = m;
24         r[m] = 0;
25         if (z) {
26             int n = m + 1 + z;
27             l.reserve(n), r.reserve(n), u.reserve(n), d.reserve(n);
28             row.reserve(n), col.reserve(n);

```

```

29     }
30 }
31
32 void addRow(int id, const vector<int>& a) {
33     int h = -1;
34     for (int c : a) {
35         assert(1 <= c && c <= m);
36         int x = l.size();
37         l.push_back(x), r.push_back(x), u.push_back(0), d.push_back(0);
38         row.push_back(id), col.push_back(c);
39         u[x] = u[c], d[x] = c;
40         d[u[c]] = x, u[c] = x;
41         ++s[c];
42         if (h == -1) h = x;
43         else {
44             l[x] = l[h], r[x] = h;
45             r[l[h]] = x, l[h] = x;
46         }
47     }
48 }
49
50 void cover(int c) {
51     r[l[c]] = r[c], l[r[c]] = l[c];
52     for (int i = d[c]; i != c; i = d[i]) {
53         for (int j = r[i]; j != i; j = r[j]) {
54             d[u[j]] = d[j], u[d[j]] = u[j];
55             --s[col[j]];
56         }
57     }
58 }
59
60 void uncover(int c) {
61     for (int i = u[c]; i != c; i = u[i]) {
62         for (int j = l[i]; j != i; j = l[j]) {
63             ++s[col[j]];
64             d[u[j]] = j, u[d[j]] = j;
65         }
66     }
67     r[l[c]] = c, l[r[c]] = c;
68 }
69
70 bool dfs() {
71     if (!r[0]) {
72         ans = cur;
73         return true;
74     }
75     int c = r[0];
76     for (int j = r[c]; j; j = r[j]) if (s[j] < s[c]) c = j;
77     if (!s[c]) return false;
78     cover(c);
79     for (int i = d[c]; i != c; i = d[i]) {
80         cur.push_back(row[i]);
81         for (int j = r[i]; j != i; j = r[j]) cover(col[j]);
82         bool f = dfs();
83         for (int j = l[i]; j != i; j = l[j]) uncover(col[j]);
84         cur.pop_back();
85         if (f) {
86             uncover(c);
87             return true;
88         }
89     }
90     uncover(c);
91     return false;
92 }
93
94 bool solve() {
95     cur.clear();
96     ans.clear();
97     return dfs();
98 }
99 };

```

07 - 模拟退火.cpp

```

1  /*
2  用途：连续或离散状态的启发式优化；不保证正确，只用于允许随机化搜索的题。
3  go(s,t,rng) 返回邻居，f(s) 返回可转为 double 的目标值；MAX=true 时改为最大化。
4  随机数生成器可直接传入基础算法目录中的 Random::rng。
5  */
6  template <bool MAX = false, class S, class F, class G, class R>

```

```

7 S anneal(S s, F f, G go, R& rng, double t = 1e4, double e = 1e-8,
8         double a = 0.995, int rep = 1) {
9     assert(t > e && 0 < a && a < 1 && rep > 0);
10    uniform_real_distribution<double> u(0, 1);
11    S st = s, ans = s;
12    double bv = static_cast<double>(f(s));
13    for (int z = 0; z < rep; ++z) {
14        s = st;
15        double v = static_cast<double>(f(s));
16        for (double c = t; c > e; c *= a) {
17            S ns = go(s, c, rng);
18            double nv = static_cast<double>(f(ns));
19            double d = nv - v;
20            if constexpr (MAX) d = -d;
21            if (d <= 0 || exp(-d / c) > u(rng)) s = std::move(ns), v = nv;
22            bool ok = MAX ? v > bv : v < bv;
23            if (ok) ans = s, bv = v;
24        }
25    }
26    return ans;
27 }

```

08 - 公历日期换算.cpp

```

1  /*
2  用途：公历日期与相对 1970-01-01 的天数互转，支持负年份；所有操作 O(1)。
3  weekday 返回 0..6，分别表示周日到周六。
4  */
5  struct Date {
6      int y, m, d;
7  };
8
9  bool leap(int y) {
10     return y % 4 == 0 && (y % 100 != 0 || y % 400 == 0);
11 }
12
13 int days(int y, int m) {
14     static constexpr int a[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
15     return a[m] + (m == 2 && leap(y));
16 }
17
18 int days(Date a) {
19     int y = a.y - (a.m <= 2);
20     int e = (y >= 0 ? y : y - 399) / 400;
21     int u = y - e * 400;
22     int m = a.m + (a.m > 2 ? -3 : 9);
23     int d = (153 * m + 2) / 5 + a.d - 1;
24     return e * 146097 + u * 365 + u / 4 - u / 100 + d - 719468;
25 }
26
27 Date date(int z) {
28     z += 719468;
29     int e = (z >= 0 ? z : z - 146096) / 146097;
30     int d = z - e * 146097;
31     int u = (d - d / 1460 + d / 36524 - d / 146096) / 365;
32     int y = u + e * 400;
33     d -= 365 * u + u / 4 - u / 100;
34     int m = (5 * d + 2) / 153;
35     int x = d - (153 * m + 2) / 5 + 1;
36     m += m < 10 ? 3 : -9;
37     return {y + (m <= 2), m, x};
38 }
39
40 int weekday(Date a) {
41     int x = (days(a) + 4) % 7;
42     return x < 0 ? x + 7 : x;
43 }

```

10 - 德州扑克五张牌比较.cpp

```

1  /*
2  用途：比较两手五张德州扑克；返回负数/0/正数。
3  输入牌用两字符表示，如 AS、TD；第二字符花色只需相同可比较。
4  牌型从高到低：同花顺、四条、葫芦、同花、顺子、三条、两对、一对、高牌。
5  */
6  struct Poker5 {
7      static int rk(char c) {
8          string s = "--23456789TJQKA";

```

```

9      return s.find(c);
10     }
11
12     static vector<int> val(const array<string, 5> &a) {
13         array<int, 15> c{};
14         vector<int> r;
15         bool f = true;
16         for (auto &s : a) {
17             r.push_back(rk(s[0]));
18             ++c[r.back()];
19             f &= s[1] == a[0][1];
20         }
21         sort(r.begin(), r.end(), greater<>());
22         vector<int> q = r;
23         q.erase(unique(q.begin(), q.end()), q.end());
24         int h = 0;
25         if (q.size() == 5 && q[0] - q[4] == 4) h = q[0];
26         if (q == vector<int>({14, 5, 4, 3, 2})) h = 5;
27
28         vector<pii> g;
29         for (int x = 2; x <= 14; ++x) if (c[x]) g.push_back({c[x], x});
30         sort(g.begin(), g.end(), greater<>());
31         if (f && h) return {8, h};
32         if (g[0][0] == 4) return {7, g[0][1], g[1][1]};
33         if (g[0][0] == 3 && g[1][0] == 2) return {6, g[0][1], g[1][1]};
34         if (f) return {5, r[0], r[1], r[2], r[3], r[4]};
35         if (h) return {4, h};
36         if (g[0][0] == 3) return {3, g[0][1], g[1][1], g[2][1]};
37         if (g[0][0] == 2 && g[1][0] == 2) return {2, g[0][1], g[1][1], g[2][1]};
38         if (g[0][0] == 2) return {1, g[0][1], g[1][1], g[2][1], g[3][1]};
39         return {0, r[0], r[1], r[2], r[3], r[4]};
40     }
41
42     static int cmp(const array<string, 5> &a, const array<string, 5> &b) {
43         auto x = val(a), y = val(b);
44         return (x > y) - (x < y);
45     }
46 };

```

11A - 物品装箱 (First Fit线段树) .cpp

```

1  /*
2  用途: 按输入顺序, 把物品放入最靠前且剩余容量足够的箱子 (First Fit)。
3  说明: 这是指定在线策略, 不保证最少箱数:  $O(n \log n)$ 。
4  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
5  */
6  template <class T> int binPack(const vector<T> &a, T c) {
7      int n = a.size();
8      vector<T> t(n * 4, c);
9      auto sub = [&](auto go, int p, int l, int r, T x) -> int {
10         if (l + 1 == r) {
11             t[p] -= x;
12             return l;
13         }
14         int m = (l + r) >> 1, q;
15         if (t[p << 1] >= x) q = go(go, p << 1, l, m, x);
16         else q = go(go, p << 1 | 1, m, r, x);
17         t[p] = max(t[p << 1], t[p << 1 | 1]);
18         return q;
19     };
20     int z = 0;
21     for (T x : a) {
22         if (x > c) return -1;
23         z = max(z, sub(sub, 1, 0, n, x) + 1);
24     }
25     return z;
26 }

```

11B - 物品装箱 (Best Fit多重集) .cpp

```

1  /*
2  用途: 按输入顺序, 把物品放入“能容纳它且剩余容量最小”的箱子 (Best Fit)。
3  说明: 这是指定在线策略, 不保证最少箱数:  $O(n \log n)$ 。
4  */
5  template <class T> int binPack(const vector<T> &a, T c) {
6      multiset<T> s;

```

```

7   for (T x : a) {
8       if (x > c) return -1;
9       auto p = s.lower_bound(x);
10      if (p == s.end()) s.insert(c - x);
11      else {
12          T y = *p;
13          s.erase(p);
14          s.insert(y - x);
15      }
16  }
17  return s.size();
18 }

```

12 - 奇阶幻方.cpp

```

1  /*
2  用途: Siamese 法构造 n 为正奇数的标准幻方, 使用 1..n^2。
3  性质: 每行、每列、两条主对角线之和均为 n(n^2+1)/2; 0(n^2)。
4  */
5  vector<vector<int>> magic(int n) {
6      assert(n > 0 && (n & 1));
7      vector a(n, vector<int>(n));
8      int x = 0, y = n / 2;
9      for (int z = 1; z <= n * n; ++z) {
10         a[x][y] = z;
11         int nx = (x + n - 1) % n, ny = (y + 1) % n;
12         if (a[nx][ny]) x = (x + 1) % n;
13         else x = nx, y = ny;
14     }
15     return a;
16 }

```

13 - 阿达马矩阵.cpp

```

1  /*
2  用途: Sylvester 构造 2^k 阶阿达马矩阵; 元素为 +1/-1, 任意两行正交。
3  复杂度: O(4^k), 输出矩阵本身即需要该规模。
4  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
5  */
6  vector<vector<int>> hadam(int k) {
7      int n = 1LL << k;
8      vector a(n, vector<int>(n, 1));
9      for (int s = 1; s < n; s <= 1)
10         for (int i = 0; i < s; ++i)
11             for (int j = 0; j < s; ++j) {
12                 a[i][j + s] = a[i][j];
13                 a[i + s][j] = a[i][j];
14                 a[i + s][j + s] = -a[i][j];
15             }
16     return a;
17 }

```

14A - 选数枚举 (DFS) .cpp

```

1  /* 用途: 枚举 n 个数中恰选 k 个; 回调参数为当前元素下标与元素和。
2  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
3  */
4  template <class T, class F> void choose(const vector<T> &a, int k, F f) {
5      vector<int> q;
6      auto dfs = [&](auto go, int p, T s) -> void {
7          if (q.size() == k) {
8              f(q, s);
9              return;
10         }
11         int d = k - q.size();
12         for (int i = p; i + d <= a.size(); ++i) {
13             q.push_back(i);
14             go(go, i + 1, s + a[i]);
15             q.pop_back();
16         }
17     };
18     dfs(dfs, 0, T{});
19 }

```


14B - 选数枚举 (Gosper位运算) .cpp

```

1  /* 用途: n<=63 时枚举所有恰含 k 个 1 的掩码; 回调参数为 u64 掩码。
2  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
3  */
4
5  using u64 = uint64_t;
6
7  template <class F> void choose(int n, int k, F f) {
8      if (k < 0 || k > n || n > 63) return;
9      if (!k) {
10         f(u64(0));
11         return;
12     }
13     u64 x = (u64(1) << k) - 1, u = u64(1) << n;
14     while (x < u) {
15         f(x);
16         u64 a = x & -x, b = x + a;
17         x = b | ((x ^ b) >> 2) / a;
18     }
19 }

```

15 - 麻将胡牌判定 (标准、七对子、国士) .cpp

```

1  /*
2  用途: 判断 34 种牌编码下的十四张麻将手牌。
3  编码: 万 0..8、筒 9..17、索 18..26、字牌 27..33; 同一张牌最多 4 张。
4  包含: 标准型 (四组面子+一对将)、七对子、国士无双; 不处理副露、番数和地方规则。
5  复杂度: 标准型回溯状态很小; 最坏由牌面种类数决定, 实际比赛中可直接使用。
6  注意: 七对子按常见规则允许四张相同牌计作两对; 若题面禁止, 改 ask7 的条件。
7  */
8  struct Mahjong {
9      using C = array<int, 34>;
10
11      static C cnt(const vector<int> &a) {
12          C c{};
13          for (int x : a) {
14              assert(0 <= x && x < 34);
15              ++c[x];
16              assert(c[x] <= 4);
17          }
18          return c;
19      }
20
21      static bool dfs(C &c) {
22          int x = 0;
23          while (x < 34 && !c[x]) ++x;
24          if (x == 34) return true;
25          if (c[x] >= 3) {
26              c[x] -= 3;
27              if (dfs(c)) return true;
28              c[x] += 3;
29          }
30          if (x < 27 && x % 9 <= 6 && c[x + 1] && c[x + 2]) {
31              --c[x], --c[x + 1], --c[x + 2];
32              if (dfs(c)) return true;
33              ++c[x], ++c[x + 1], ++c[x + 2];
34          }
35          return false;
36      }
37
38      static bool askStd(C c) {
39          int s = 0;
40          for (int x : c) s += x;
41          if (s != 14) return false;
42          for (int x = 0; x < 34; ++x) if (c[x] >= 2) {
43              c[x] -= 2;
44              if (dfs(c)) return true;
45              c[x] += 2;
46          }
47          return false;
48      }
49
50      static bool ask7(const C &c) {
51          int s = 0, p = 0;
52          for (int x : c) {
53              s += x;
54              if (x & 1) return false;
55              p += x / 2;

```

```
56     }
57     return s == 14 && p == 7;
58 }
59
60 static bool ask13(const C &c) {
61     static constexpr int q[] = {0, 8, 9, 17, 18, 26,
62                                 27, 28, 29, 30, 31, 32, 33};
63     array<char, 34> ok{};
64     for (int x : q) ok[x] = true;
65     int s = 0, d = 0;
66     for (int x = 0; x < 34; ++x) {
67         if (c[x] && !ok[x]) return false;
68         s += c[x];
69         if (ok[x]) {
70             if (!c[x]) return false;
71             d += c[x] - 1;
72         }
73     }
74     return s == 14 && d == 1;
75 }
76
77 static bool ask(const vector<int> &a) {
78     if (a.size() != 14) return false;
79     C c = cnt(a);
80     return askStd(c) || ask7(c) || ask13(c);
81 }
82 };
```

动态规划：背包与状态设计

13 个代码模板 · 林月 · v1.0.0 · 2026.8.1

版块说明与知识结论

README.md

动态规划模板

编号	模板	适用场景
01A / 01B	01 / 完全背包	每件至多一次 / 可取任意次。
01C / 01D	多重背包（二进制 / 单调队列）	数量较小优先短代码；数量很大用 $O(nm)$ 单调队列。
01E / 01F	分组 / 二维费用背包	每组至多一件 / 同时受二维容量限制。
01G	01 背包最优方案数	恰好装满，同时维护最优值与自动取模方案数。
01H	混合背包	同一输入混合 01、完全与有限数量物品。
01I	有依赖的背包	选择子节点必须选择父节点的树形背包。
01J	01 背包具体方案	$O(nm)$ 保存阶段状态并优先恢复小编号。
02	数位 DP（DFA）	把前导零、余数、上一位等压入有限状态。
03	状压 DP（TSP）	小 n 的集合状态与 Hamilton 回路。
04	轮廓线 DP	窄网格多米诺骨牌铺法，计数类型支持自动取模。
05A / 05B / 05C	分治 DP / Knuth / 单调斜率 CHT	决策单调 / 四边形不等式 / 线性转移且斜率单调。
06A / 06B	LCS（常规 / Bitset）	恢复方案 / 只求字节串长度并卡常。
07	Slope Trick	维护凸分段线性函数，支持绝对值、前后缀最小化和滑动窗口平移。

选型与边界

- 01C 与 01D 是同一多重背包问题在不同数量范围下的实现，均保留；01G 的计数是恰好容量，避免“容量上限状态”的重复语义。01H 汇总混合输入，01I/01J 则补足依赖与方案恢复，不能由一维值 DP 直接替代。
- 数位 DP 的 go 会看到前导零。需要“不足位数不参与性质”时，把 started 编入状态；数字 0 的最终表示也要单独确认。
- 状压 DP 的状态数按 2^n 增长； $n=22$ 已需要约 22×2^{22} 个距离元素，先估算内存。
- 分治优化只要求决策单调；Knuth 还要求更强的四边形不等式与区间单调。不能仅凭转移式长得相似就套用。
- 05C 与数据结构的 Li Chao 同为直线取最值：斜率递减、查询点递增时 05C 的 askMono 是均摊 $O(1)$ ；斜率或查询无序、需要线段直线时改用 Li Chao 02H。WQS 二分见杂类 05，SOS DP 见数学 08H，换根 DP 见树上问题 08。
- 07 只维护凸函数。addAMX(a) 与 addXMA(a) 分别加入 $(a-x)_+$ 和 $(x-a)_+$ ；shift(a,b) 是滑动窗口最小化，不是简单把两个堆都加同一个数。调用前先检查转移是否真的保持凸性。

更完整的状态设计、初始化和优化前提见 [动态规划知识与结论](#)。

动态规划知识与结论.md

动态规划知识与结论

1. 先写清状态语义

每个状态至少写清四件事：处理到哪里、已用资源、是否要求恰好、保存最大值/最小值/方案数中的哪一种。容量不超过 j 可全初始化为 0；恰好使用 j 只能令 $f[0]=0$ ，其余为不可达。

滚动数组的循环方向来自“本轮能否再次使用当前物品”：01 背包倒序，完全背包正序，分组背包必须从上一组的旧数组转移。多重背包可二进制拆分成 01 背包，也可按容量模物品体积分组后做滑动窗口最大值。

2. 方案数与自动取模

若状态同时维护最优值 v 和方案数 c ：新值更优时覆盖二者，相等时只累加 c 。计数类型用 `MInt` 即可自动取模；价值仍应使用有序数值类型，不能拿模数类型比较大小。

求具体方案时保存决策或保留一维 DP 的各阶段快照。要求字典序最小时，固定物品扫描方向，并在价值相同的转移中明确“取/不取”的优先级。

混合背包只是在每件物品处分别选择倒序、正序或有限数量转移；有依赖背包则不能拍平成普通物品，必须先在依赖树上算子树 DP，再像分组背包一样合并儿子。恢复具体方案会把一维空间提升到 $O(nm)$ ，只求值时不要承担这部分内存。

3. 常见状态族

- 数位 DP：位置、自动机状态、是否贴上界；前导零通常也应是一个状态位。
- 状压 DP：集合 S 与末端 u ；先检查 $2^{n \times n}$ 的内存，再考虑滚动或分层。
- 区间 DP：按区间长度递增；枚举断点前先确认空区间和单点的初值。
- 树形 DP：父子合并本质常是分组背包；子树大小可限制无意义的容量枚举。
- 轮廓线 DP：状态表示下一条轮廓上已被占用的位置；每处理完一行，只允许合法轮廓流入下一行。

4. 优化的可用前提

设转移为 $g[j] = \min_k (f[k] + w(k, j))$ ：

- 决策点 $opt[j]$ 单调时，可分治计算一层；证明通常来自 Monge / 四边形不等式。
- 区间合并满足 Knuth 条件时， $opt[l][r-1] \leq opt[l][r] \leq opt[l+1][r]$ ，可从 $O(n^3)$ 降到 $O(n^2)$ 。
- 转移可写成直线在横坐标处取最值时，用单调凸包或 Li Chao Tree。若插入斜率递减、查询点递增，Q5C 的 CHT 用 `askMono` 均摊 $O(1)$ ；查询无序仍可用其 `ask` 的 $O(\log n)$ 二分。斜率无序、定义域动态或直线只覆盖一个区间时改用 Li Chao Tree。
- “恰选 k 个”可把数量乘拉格朗日乘子并二分该乘子，即 WQS；必须证明最优数量随乘子单调。

5. 高频排错

- 不可达值参与加法会溢出；先判断再转移。
- 最小化的 INF 、最大化的 $-INF$ 应留出运算余量。
- 原地更新前确认转移是否会错误使用本轮刚更新的状态。
- 计数题先确认空方案、数字 0、重复物品和相同值物品是否按下标区分。
- 模意义只适用于加乘计数；最值比较、概率期望中的除法前提不能被取模接口掩盖。

01A - 01背包.cpp

```

1  /*
2  用途：01 背包；每件物品至多取一次，求容量不超过 m 的最大价值。
3  约定：容量下标使用 int；T 是价值类型。返回 f[j] 表示容量上限为 j 的答案。
4  复杂度： $O(nm)$  时间、 $O(m)$  空间。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T>
8  vector<T> pack01(int m, const vector<pair<int, T>>& a) {
9      assert(m >= 0);
10     vector<T> f(m + 1);
11     for (auto [w, v] : a) {
12         assert(w > 0);
13         for (int j = m; j >= w; --j) f[j] = max(f[j], f[j - w] + v);
14     }
15     return f;
16 }
17
18 // 恰好装满。不可达状态为 neg，调用者应保证 neg + v 不溢出。
19 template <class T>
20 vector<T> packEq(int m, const vector<pair<int, T>>& a,
```

```

21         T neg = numeric_limits<T>::lowest() / 4) {
22     assert(m >= 0);
23     vector<T> f(m + 1, neg);
24     f[0] = T{};
25     for (auto [w, v] : a) {
26         assert(w > 0);
27         for (int j = m; j >= w; --j) {
28             if (f[j - w] != neg) f[j] = max(f[j], f[j - w] + v);
29         }
30     }
31     return f;
32 }

```

01B - 完全背包.cpp

```

1  /*
2  用途：完全背包；每种物品可取任意多次。
3  约定：容量下标使用 int；返回 f[j] 表示容量不超过 j 的最大价值。
4  复杂度：O(nm) 时间、O(m) 空间。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T>
8  vector<T> packAll(int m, const vector<pair<int, T>>& a) {
9      assert(m >= 0);
10     vector<T> f(m + 1);
11     for (auto [w, v] : a) {
12         assert(w > 0);
13         for (int j = w; j <= m; ++j) f[j] = max(f[j], f[j - w] + v);
14     }
15     return f;
16 }

```

01C - 多重背包（二进制分组）.cpp

```

1  /*
2  用途：多重背包的二进制分组实现；代码短、适合物品数量不太大。
3  复杂度：O(m * sum(log(c+1)))。
4  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
5  */
6  template <class T> struct Item {
7      int w, c;
8      T v;
9  };
10
11 template <class T>
12 vector<T> packMul(int m, const vector<Item<T>>& a) {
13     assert(m >= 0);
14     vector<T> f(m + 1);
15     for (auto [w, c, v] : a) {
16         assert(w > 0 && c >= 0);
17         for (int k = 1; c > 0; k <= 1) {
18             int x = min<int>(k, c);
19             c -= x;
20             int sw = w * x;
21             if (sw > m) continue;
22             T sv = v * T(x);
23             for (int j = m; j >= sw; --j) f[j] = max(f[j], f[j - sw] + sv);
24         }
25     }
26     return f;
27 }

```

01D - 多重背包（单调队列）.cpp

```

1  /*
2  用途：多重背包的单调队列实现；物品数量 c 很大时不再多一个 log c。
3  复杂度：O(nm) 时间、O(m) 空间。
4  与 01C 同名、同接口，二者择一使用。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T> struct Item {
8      int w, c;
9      T v;
10 };
11
12 template <class T>

```

```

13 vector<T> packMul(int m, const vector<Item<T>>& a) {
14     assert(m >= 0);
15     vector<T> f(m + 1);
16     for (auto [w, c, v] : a) {
17         assert(w > 0 && c >= 0);
18         vector<T> g = f;
19         for (int r = 0; r < w && r <= m; ++r) {
20             deque<int> q;
21             int z = (m - r) / w;
22             for (int k = 0; k <= z; ++k) {
23                 while (!q.empty() && q.front() < k - c) q.pop_front();
24                 T x = g[r + k * w] - T(k) * v;
25                 while (!q.empty() && g[r + q.back() * w] - T(q.back()) * v <= x) q.pop_back();
26                 q.push_back(k);
27                 int p = q.front();
28                 f[r + k * w] = g[r + p * w] + T(k - p) * v;
29             }
30         }
31     }
32     return f;
33 }

```

01E - 分组背包.cpp

```

1  /*
2  用途：分组背包；每组至多选择一件物品。
3  groups[g] 中每个元素为 {体积， 价值}。
4  复杂度：O(m * 物品总数)。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T>
8  vector<T> packGrp(int m, const vector<vector<pair<int, T>>& a) {
9      assert(m >= 0);
10     vector<T> f(m + 1);
11     for (const auto& g : a) {
12         vector<T> h = f;
13         for (auto [w, v] : g) {
14             assert(w > 0);
15             for (int j = w; j <= m; ++j) h[j] = max(h[j], f[j - w] + v);
16         }
17         f.swap(h);
18     }
19     return f;
20 }

```

01F - 二维费用背包.cpp

```

1  /*
2  用途：每件物品至多取一次，同时受二维容量约束。
3  item = {第一维费用， 第二维费用， 价值}。
4  复杂度：O(nab) 时间、O(ab) 空间。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T> struct Item2D {
8      int x, y;
9      T v;
10 };
11
12 template <class T>
13 vector<vector<T>> pack2D(int a, int b, const vector<Item2D<T>>& c) {
14     assert(a >= 0 && b >= 0);
15     vector f(a + 1, vector<T>(b + 1));
16     for (auto [x, y, v] : c) {
17         assert(x > 0 || y > 0);
18         for (int i = a; i >= x; --i)
19             for (int j = b; j >= y; --j) f[i][j] = max(f[i][j], f[i - x][j - y] + v);
20     }
21     return f;
22 }

```

01G - 01背包最优方案数（自动取模）.cpp

```

1  /*
2  用途：01 背包恰好装满时，同时求最大价值和达到最大值的方案数。
3  Z 可取 int（精确计数）或 MInt（自动取模）。每件物品按下标区分。
4  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。

```

```

5  */
6  template <class T, class Z> struct PackCount {
7      T val;
8      Z cnt;
9  };
10
11 template <class T, class Z>
12 vector<PackCount<T, Z>> packCnt(int m, const vector<pair<int, T>>& a,
13                                 T neg = numeric_limits<T>::lowest() / 4) {
14     vector<PackCount<T, Z>> f(m + 1, {neg, Z{}});
15     f[0] = {T{}, Z{1}};
16     for (auto [w, v] : a) {
17         assert(w > 0);
18         for (int j = m; j >= w; --j) {
19             if (f[j - w].val == neg) continue;
20             T x = f[j - w].val + v;
21             if (x > f[j].val) f[j] = {x, f[j - w].cnt};
22             else if (x == f[j].val) f[j].cnt += f[j - w].cnt;
23         }
24     }
25     return f;
26 }

```

01H - 混合背包.cpp

```

1  /*
2  用途：同一组物品混合 01、完全、多重三种数量限制，求容量不超过 m 的最大价值。
3  约定：Item.c=-1 表示至多一次，0 表示无限次，正数表示至多 c 次；w>0。
4  复杂度：二进制拆分多重物品， $O(m * \text{拆分后的物品数})$ 。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T> struct MixedItem {
8      int w, c;
9      T v;
10 };
11
12 template <class T> vector<T> packMix(int m, const vector<MixedItem<T>>& a) {
13     assert(m >= 0);
14     vector<T> f(m + 1, T{});
15     auto one = [&](int w, T v) {
16         for (int j = m; j >= w; --j) f[j] = max(f[j], f[j - w] + v);
17     };
18     for (auto [w, c, v] : a) {
19         assert(w > 0 && c >= -1);
20         if (c < 0) one(w, v);
21         else if (!c) for (int j = w; j <= m; ++j) f[j] = max(f[j], f[j - w] + v);
22         else {
23             for (int k = 1; c;) {
24                 int x = min(k, c);
25                 one(w * x, v * x);
26                 c -= x;
27                 k = k <= c ? k << 1 : c;
28             }
29         }
30     }
31     return f;
32 }

```

01I - 有依赖的背包（树形）.cpp

```

1  /*
2  用途：有依赖背包：选择节点前必须选择父节点，多个 parent=0 的根可独立选择。
3  输入：w/v/parent 均为 1-indexed 等长数组，容量 m；返回每个“恰好容量”的最优值。
4  复杂度： $O(nm^2)$ ，适合树形依赖与中小容量；不可达状态为 NEG。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T>
8  vector<T> packDep(int m, const vector<int>& w, const vector<T>& v,
9                    const vector<int>& fa) {
10     static_assert(is_signed_v<T>);
11     assert(m >= 0 && w.size() == v.size() && v.size() == fa.size());
12     int n = w.size() - 1;
13     vector<vector<int>> g(n + 1);
14     for (int i = 1; i <= n; ++i) {
15         assert(0 <= fa[i] && fa[i] <= n && fa[i] != i && w[i] >= 0);
16         g[fa[i]].push_back(i);
17     }
18     const T NEG = numeric_limits<T>::lowest() / 4;

```

```

19     int vis = 0;
20     auto dfs = [&](auto&& go, int u) -> vector<T> {
21         vis += u != 0;
22         vector<T> f(m + 1, NEG);
23         int s = u ? w[u] : 0;
24         if (s <= m) f[s] = u ? v[u] : T{};
25         for (int x : g[u]) {
26             auto d = go(go, x);
27             vector<T> h = f;
28             for (int i = 0; i <= m; ++i) if (f[i] != NEG)
29                 for (int j = 0; i + j <= m; ++j) if (d[j] != NEG)
30                     h[i + j] = max(h[i + j], f[i] + d[j]);
31             f.swap(h);
32         }
33         return f;
34     };
35     auto f = dfs(dfs, 0);
36     assert(vis == n); // 否则 parent 中存在不连向虚根环。
37     return f;
38 }

```

01J - 01背包具体方案.cpp

```

1  /*
2  用途: 01 背包求最大价值并恢复一组方案: 能保持最优时优先选择编号更小的物品。
3  输入: w/v 为 0-indexed 等长数组, 容量 m: 返回 {最大价值, 选中下标}。
4  复杂度: O(nm) 时间与空间。只求最优值时使用 01A 的一维版本。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T>
8  pair<T, vector<int>> packSol(int m, const vector<int>& w, const vector<T>& v) {
9      assert(m >= 0 && w.size() == v.size());
10     int n = w.size();
11     vector<vector<T>> f(n + 1, vector<T>(m + 1));
12     for (int i = n; i-- > 0;) {
13         assert(w[i] > 0);
14         for (int j = 0; j <= m; ++j) {
15             f[i][j] = f[i + 1][j];
16             if (w[i] <= j) f[i][j] = max(f[i][j], f[i + 1][j - w[i]] + v[i]);
17         }
18     }
19     vector<int> a;
20     int c = m;
21     for (int i = 0; i < n; ++i) if (w[i] <= c && f[i][c] == f[i + 1][c - w[i]] + v[i])
22         a.push_back(i), c -= w[i];
23     return {f[0][m], a};
24 }

```

02 - 数位DP (DFA) .cpp

```

1  /*
2  用途: 统计 [0, n] 中十进制表示被有限状态自动机接受的整数个数。
3  go(state, digit) 返回下一状态: 若需要区分前导零, 把 started 一并编码进状态。
4  good(state) 判断读完固定宽度 (含前导零) 后的状态是否接受。
5  T 可取 int、高精整数或 MInt, 后者会自动取模。
6  复杂度: O(位数 * 状态数 * 10)。
7  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
8  */
9  template <class T, class Go, class G>
10 T digitDP(int n, int sc, int st, Go go, G ok) {
11     assert(sc > 0 && 0 <= st && st < sc);
12     vector<int> d;
13     do {
14         d.push_back(n % 10);
15         n /= 10;
16     } while (n);
17     reverse(d.begin(), d.end());
18
19     int m = d.size();
20     vector f(m + 1, vector<T>(sc));
21     vector vis(m + 1, vector<char>(sc));
22     auto dfs = [&](auto&& rec, int p, int s, bool lim) -> T {
23         if (p == m) return T(ok(s));
24         if (!lim && vis[p][s]) return f[p][s];
25         int up = lim ? d[p] : 9;
26         T r{};
27         for (int x = 0; x <= up; ++x) {
28             int t = go(s, x);

```



```

29         assert(0 <= t && t < sc);
30         r += rec(rec, p + 1, t, lim && x == up);
31     }
32     if (!lim) vis[p][s] = 1, f[p][s] = r;
33     return r;
34 };
35 return dfs(dfs, 0, st, true);
36 }
37
38 template <class T, class Go, class G>
39 T digitDP(int l, int r, int sc, int st, Go go, G ok) {
40     assert(l <= r);
41     T x = digitDP<T>(r, sc, st, go, ok);
42     return l ? x - digitDP<T>(l - 1, sc, st, go, ok) : x;
43 }

```

03 - 状压DP (TSP) .cpp

```

1  /*
2  用途: Held-Karp 状压 DP, 求从 s 出发、恰好访问每个点一次并回到 s 的最短 Hamilton 回路。
3  w[u][v] 为边权; 不可达写 INF。n 通常不超过 20。
4  复杂度:  $O(2^n n^2)$  时间、 $O(2^n n)$  空间。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T>
8  optional<T> tsp(const vector<vector<T>>& w, int s = 0,
9                 T inf = numeric_limits<T>::max() / 4) {
10     int n = w.size();
11     assert(n > 0 && n <= 22 && 0 <= s && s < n);
12     for (const auto& a : w) assert(a.size() == n);
13     int z = 1LL << n;
14     vector f(z, vector<T>(n, inf));
15     f[1LL << s][s] = T{};
16     for (int m = 0; m < z; ++m) {
17         if (!(m >> s & 1)) continue;
18         for (int u = 0; u < n; ++u) {
19             if (!(m >> u & 1) || f[m][u] == inf) continue;
20             for (int v = 0; v < n; ++v) {
21                 if (m >> v & 1 || w[u][v] == inf) continue;
22                 int t = m | (1LL << v);
23                 f[t][v] = min(f[t][v], f[m][u] + w[u][v]);
24             }
25         }
26     }
27     T r = inf;
28     for (int u = 0; u < n; ++u)
29         if (u != s && f[z - 1][u] != inf && w[u][s] != inf) r = min(r, f[z - 1][u] + w[u][s]);
30     if (n == 1) r = T{};
31     return r == inf ? nullopt : optional<T>(r);
32 }

```

04 - 轮廓线DP (多米诺骨牌) .cpp

```

1  /*
2  用途: 用 1x2 多米诺骨牌铺满 n*m 网格的方案数。
3  T 可取 int、高精整数或 MInt (自动取模)。宽度取 min(n,m), 通常不超过 15。
4  复杂度: 逐格转移  $O(n * m * 2^m)$ 。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T>
8  T domino(int n, int m) {
9     assert(n >= 0 && m >= 0);
10     if (n < m) swap(n, m);
11     if (!m) return T(1);
12     assert(m < 31);
13     int z = 1LL << m;
14     vector<T> f(z), g(z);
15     f[0] = T(1);
16     for (int r = 0; r < n; ++r) {
17         for (int c = 0; c < m; ++c) {
18             fill(g.begin(), g.end(), T{});

```

```
19         for (int s = 0; s < z; ++s) {
20             if (s & 1) g[s >> 1] += f[s];
21             else {
22                 if (r + 1 < n) g[(s >> 1) | (1LL << (m - 1))] += f[s];
23                 if (c + 1 < m && !(s & 2)) g[(s >> 1) | 1] += f[s];
24             }
25         }
26         f.swap(g);
27     }
28 }
29 return f[0];
30 }
```

动态规划：优化与最长公共子序列

6 个代码模板 · 林月 · v1.0.0 · 2026.8.1

05A - 分治DP优化.cpp

```

1  /*
2  用途: 计算  $g[j] = \min_{0 \leq k < j} (f[k] + \text{cost}(k+1, j))$ 。
3  前提: 最优决策点随  $j$  单调; 单调性不成立时结果错误。
4  调用: divide(l, n, 0, n-1, f, g, p, cst, inf)。
5  复杂度: 每一层 DP  $O(n \log n)$  次转移计算, 常见进一步可证为  $O(n \log n)$ 。
6  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
7  */
8  template <class T, class C>
9  void divide(int l, int r, int ql, int qr,
10             const vector<T>& f, vector<T>& g, vector<int>& p,
11             C cst, T inf) {
12     if (l > r) return;
13     int m = (l + r) >> 1;
14     pair<T, int> z{inf, -1};
15     int up = min(qr, m - 1);
16     for (int k = ql; k <= up; ++k) {
17         if (f[k] == inf) continue;
18         z = min(z, pair<T, int>{f[k] + cst(k + 1, m), k});
19     }
20     g[m] = z.first;
21     p[m] = z.second;
22     if (l == r) return;
23     int q = z.second == -1 ? ql : z.second;
24     divide(l, m - 1, ql, q, f, g, p, cst, inf);
25     divide(m + 1, r, q, qr, f, g, p, cst, inf);
26 }

```

05B - Knuth优化.cpp

```

1  /*
2  用途: 区间 DP:  $f[l][r] = \min(f[l][k] + f[k+1][r]) + \text{cost}(l, r)$ 。
3  前提: cost 满足四边形不等式且区间包含单调, 因而  $\text{opt}[l][r-1] \leq \text{opt}[l][r] \leq \text{opt}[l+1][r]$ 。
4  下标:  $0..n-1$ 。复杂度  $O(n^2)$  时间、 $O(n^2)$  空间。
5  性能: 默认使用宏展开后的 int; 若大规模数据因此超时或超空间, 可将纯下标、状态或计数局部改为 i32。
6  */
7  template <class T, class C>
8  pair<vector<vector<T>>, vector<vector<int>>> knuthDP(int n, C cst) {
9     assert(n >= 0);
10     vector f(n, vector<T>(n));
11     vector p(n, vector<int>(n));
12     for (int i = 0; i < n; ++i) p[i][i] = i;
13     for (int d = 1; d < n; ++d) {
14         for (int l = 0, r = d; r < n; ++l, ++r) {
15             int a = p[l][r - 1];
16             int b = min<int>(r - 1, p[l + 1][r]);
17             f[l][r] = numeric_limits<T>::max() / 4;
18             for (int k = a; k <= b; ++k) {
19                 T x = f[l][k] + f[k + 1][r] + cst(l, r);
20                 if (x < f[l][r]) f[l][r] = x, p[l][r] = k;
21             }
22         }
23     }
24     return {move(f), move(p)};
25 }

```

05C - 斜率优化 (CHT, 单调斜率) .cpp

```

1  /*
2  用途: 维护  $y = m \cdot x + b$  的下包络, 适合 DP 转移  $f[j] = \min_i (m_i \cdot x_j + b_i)$ 。
3  前提: add 的斜率严格递减; ask 可任意顺序, askMono 的  $x$  必须非递减。
4  接口: add(m, b), ask(x) 返回最小值, askMono(x) 为均摊  $O(1)$  查询。
5  复杂度: 插入均摊  $O(1)$ , ask  $O(\log n)$ , askMono 均摊  $O(1)$ 。比较使用 i128; 答案须落在 int。
6  */
7  using i128 = __int128_t;
8
9  struct CHT {
10     static constexpr int INF = numeric_limits<int>::max();

```

```

11
12 struct Line {
13     int m, b;
14
15     i128 val(int x) const {
16         return i128(m) * x + b;
17     }
18 };
19
20 vector<Line> q;
21 int p = 0, lx = numeric_limits<int>::lowest();
22
23 static bool bad(const Line& a, const Line& b, const Line& c) {
24     return (i128(b.b) - a.b) * (b.m - c.m) >= (i128(c.b) - b.b) * (a.m - b.m);
25 }
26
27 void add(int m, int b) {
28     Line f{m, b};
29     if (!q.empty() && q.back().m == m) {
30         if (q.back().b <= b) return;
31         q.pop_back();
32     }
33     assert(q.empty() || q.back().m > m);
34     while (q.size() >= 2 && bad(q[q.size() - 2], q.back(), f)) q.pop_back();
35     q.push_back(f);
36     int z = q.size();
37     if (p >= z) p = z - 1;
38 }
39
40 int ask(int x) const {
41     assert(!q.empty());
42     int l = 0, r = q.size() - 1;
43     while (l < r) {
44         int m = (l + r) >> 1;
45         if (q[m].val(x) <= q[m + 1].val(x)) r = m;
46         else l = m + 1;
47     }
48     i128 ans = q[l].val(x);
49     assert(numeric_limits<int>::lowest() <= ans && ans <= numeric_limits<int>::max());
50     return ans;
51 }
52
53 int askMono(int x) {
54     assert(!q.empty() && lx <= x);
55     lx = x;
56     int z = q.size();
57     if (p >= z) p = z - 1;
58     while (p + 1 < z && q[p + 1].val(x) <= q[p].val(x)) ++p;
59     i128 ans = q[p].val(x);
60     assert(numeric_limits<int>::lowest() <= ans && ans <= numeric_limits<int>::max());
61     return ans;
62 }
63 };

```

06A - 最长公共子序列（常规）.cpp

```

1  /*
2  用途：任意可比较序列的 LCS；返回一组最长公共子序列。
3  复杂度：O(nm) 时间、O(nm) 空间。只求字符串长度且 m 很大时使用 06B。
4  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
5  */
6  template <class Seq>
7  vector<typename Seq::value_type> lcs(const Seq& a, const Seq& b) {
8      int n = a.size(), m = b.size();
9      vector f(n + 1, vector<int>(m + 1));
10     for (int i = 1; i <= n; ++i) {
11         for (int j = 1; j <= m; ++j) {
12             if (a[i - 1] == b[j - 1]) f[i][j] = f[i - 1][j - 1] + 1;
13             else f[i][j] = max(f[i - 1][j], f[i][j - 1]);
14         }
15     }
16     vector<typename Seq::value_type> r;
17     for (int i = n, j = m; i && j;) {
18         if (a[i - 1] == b[j - 1]) r.push_back(a[--i]), --j;
19         else if (f[i - 1][j] >= f[i][j - 1]) --i;
20         else --j;
21     }
22     reverse(r.begin(), r.end());
23     return r;
24 }

```

06B - 最长公共子序列 (Bitset) .cpp

```

1  /*
2  用途：只求两个字符串的 LCS 长度；利用 64 位并行优化。
3  复杂度： $O(|a||b|/64 + |b|)$ ，空间  $O(256|b|/64)$ 。
4  大位集必须使用 u64；若被 #define int long long 替换会显著增大内存和降低速度。
5  性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
6  */
7
8  using u64 = uint64_t;
9
10 int lcs(const string& a, const string& b) {
11     int n = b.size(), z = (n + 63) >> 6;
12     vector<vector<u64>> p(256, vector<u64>(z));
13     for (int i = 0; i < n; ++i) p[(unsigned char)b[i]][i >> 6] |= u64(1) << (i & 63);
14     vector<u64> s(z), x(z), y(z), d(z);
15     for (unsigned char c : a) {
16         u64 q = 1;
17         for (int i = 0; i < z; ++i) {
18             x[i] = p[c][i] | s[i];
19             u64 nq = s[i] >> 63;
20             y[i] = (s[i] << 1) | q;
21             q = nq;
22         }
23         u64 br = 0;
24         for (int i = 0; i < z; ++i) {
25             u64 t = y[i] + br;
26             u64 ov = t < y[i];
27             d[i] = x[i] - t;
28             br = ov || x[i] < t;
29             s[i] = x[i] & ~d[i];
30         }
31     }
32     int r = 0;
33     for (u64 x : s) r += __builtin_popcountll(x);
34     return r;
35 }

```

07 - Slope Trick.cpp

```

1  /*
2  用途：维护整数定义域上的凸分段线性函数。
3  接口：addAMX 加  $\max(a-x, 0)$ ，addXMA 加  $\max(x-a, 0)$ ，addAbs 加绝对值；
4  preMin/sufMin 做前缀/后缀最小化，shift 做平移或滑动窗口最小化，ask 求函数值。
5  约定：函数初始为 0；preMin 后  $f(x)=\min_{y \leq x} \text{old } f(y)$ ，sufMin 对称；
6  shift(a,b) 要求  $a \leq b$ ，表示  $f_{\text{new}}(x)=\min_{x-b \leq y \leq x-a} \text{old } f(y)$ 。
7  复杂度：每次修改  $O(\log n)$ ，最小值和最优区间  $O(1)$ ；ask 会复制堆， $O(n \log n)$ 。
8  */
9  template <class T = int> struct SlopeTrick {
10     using i128 = __int128_t;
11     using Q = priority_queue<T>;
12     using R = priority_queue<T, vector<T>, greater<T>>;
13
14     Q l;
15     R r;
16     T dl = 0, dr = 0, mn = 0;
17
18     T topL() const {
19         assert(!l.empty());
20         return l.top() + dl;
21     }
22
23     T topR() const {
24         assert(!r.empty());
25         return r.top() + dr;
26     }
27
28     void pushL(T x) {
29         l.push(x - dl);
30     }
31
32     void pushR(T x) {
33         r.push(x - dr);
34     }
35
36     void addAll(T x) {
37         mn += x;
38     }
39 }

```

```

40 // f(x) += max(x-a, 0)
41 void addXMA(T a) {
42     if (!l.empty() && topL() > a) {
43         T x = topL();
44         mn += x - a;
45         l.pop();
46         pushL(a);
47         pushR(x);
48     } else {
49         pushR(a);
50     }
51 }
52
53 // f(x) += max(a-x, 0)
54 void addAMX(T a) {
55     if (!r.empty() && topR() < a) {
56         T x = topR();
57         mn += a - x;
58         r.pop();
59         pushR(a);
60         pushL(x);
61     } else {
62         pushL(a);
63     }
64 }
65
66 void addAbs(T a) {
67     addAMX(a);
68     addXMA(a);
69 }
70
71 void clearL() {
72     l = Q{};
73 }
74
75 void clearR() {
76     r = R{};
77 }
78
79 void preMin() {
80     clearR();
81 }
82
83 void sufMin() {
84     clearL();
85 }
86
87 void shift(T a) {
88     dl += a;
89     dr += a;
90 }
91
92 void shift(T a, T b) {
93     assert(a <= b);
94     dl += a;
95     dr += b;
96 }
97
98 T askMin() const {
99     return mn;
100 }
101
102 pair<T, T> askArg() const {
103     T x = l.empty() ? numeric_limits<T>::lowest() / 4 : topL();
104     T y = r.empty() ? numeric_limits<T>::max() / 4 : topR();
105     return {x, y};
106 }
107
108 T ask(T x) const {
109     i128 z = mn;
110     auto a = l;
111     auto b = r;
112     while (!a.empty()) {
113         T y = a.top() + dl;

```

```
114         if (y > x) z += i128(y) - x;  
115         a.pop();  
116     }  
117     while (!b.empty()) {  
118         T y = b.top() + dr;  
119         if (x > y) z += i128(x) - y;  
120         b.pop();  
121     }  
122     assert(numeric_limits<T>::lowest() <= z && z <= numeric_limits<T>::max());  
123     return static_cast<T>(z);  
124 }  
125 };
```